

Bangladesh University of Engineering and Technology

Department of Computer Science & Engineering

Digital Logic Design Question Bank With Answers

CSE 205 · Digital Logic Design

Year Coverage: 2011-12 to 2024-25

Topics: Boolean Algebra, Logic Gates, Minimization, Combinational Logic,
Flip-Flops, Sequential Circuits, Registers & Counters, Memory & PLDs

Authors & Contributors

Md Ratul Hasan (2405009) — Question Curation & Organisation
— $\text{\LaTeX}$ Typesetting

NotebookLM — Research & Extraction

Claude AI — $\text{\LaTeX}$ Typesetting

Gemini — Answer & Diagram Generation

Table of Contents

Part I — Boolean Algebra & Logic Fundamentals

- ▷ **S1.** Boolean Algebra, Theorems & Logic Gates
 - Boolean Algebra & Simplification
 - De Morgan’s Theorem
 - Logic Gates, Universal Gates & Definitions
 - Circuit Analysis
 - Two-Level & Multi-Level Logic Implementation
 - NOR Gate Implementation
 - XOR Properties
 - Parity Checker and Generator
- ▷ **S2.** Canonical Forms & Boolean Functions
 - Canonical vs Standard Forms
 - Minterms, Maxterms
- ▷ **S3.** Minimization Techniques
 - Prime Implicants
 - Karnaugh Map
 - Tabulation Method (Quine-McCluskey)

Part II — Combinational Logic

- ▷ **S4.** Arithmetic & Data Handling Circuits, Decoders, Encoders, MUX/DEMUX
 - Number System & Arithmetic Operations
 - Code Converter
 - Binary Adder & Subtractor
 - BCD Adder-Subtractor
 - Carry Look-Ahead Adder (CLA)
 - 2’s Complement Circuit
 - Overflow Detection
 - Binary Multiplier
 - Incrementer
 - Magnitude Comparator
 - Encoders
 - Decoders
 - Decoder as Demultiplexer
 - BCD to 7-Segment Decoder & BCD Error Detector
 - Multiplexers & Demultiplexers
 - Custom Combinational Circuit Design

Part III — Sequential Logic Design

- ▷ **S5.** Flip-Flops, Latches & Race-Around Problems
 - SR Latch & SR/JK Undefined Condition
 - JK Flip-Flop Characteristics
 - Master-Slave D Flip-Flop
 - Sequential Circuit Timing Analysis
 - Latch vs Flip-Flop
 - Flip-Flop Conversion
- ▷ **S6.** Sequential Circuits, State Diagrams & FSM (Mealy/Moore)

- State Diagram Derivation from Circuit
- Circuit Design from State Diagram
- Sequence Detector
- Mealy & Moore Model Comparison & Block Diagrams
- State Minimization (Implication Table)
- Incompletely Specified Machine Minimization
- Redundant States
- Output Assignment in Flow Table
- Fundamental Mode Design
- Excitation Function & Transition Table
- Hazards in Asynchronous Circuits
- Serial Parity Generation
- Race Around Condition
- Pulse Mode Logic
- Synchronous Sequential Circuit Design

Part IV — Registers, Counters & Advanced Topics

▷ S7. Registers & Counters

- Ripple (Asynchronous) Counter
- Synchronous Counter Design
- Synchronous Gray Code Counter
- Switch-Tail Ring Counter
- Johnson Counter vs Ring Counter
- Counter Direction Analysis
- Clock Generator / Divider
- Universal Shift Register
- Serial Adder & Serial Subtractor

▷ S8. Memory & Programmable Logic (RAM, ROM, PLA)

- RAM / ROM Design & Expansion
- PLA Design

BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Boolean Algebra, Theorems & Logic Gates

Postulates, Duality, Truth Tables, Gate Implementations

S1 Boolean Algebra, Theorems & Logic Gates

Boolean Algebra & Simplification

Q1. Using laws and theorems of Boolean algebra, prove and simplify the following expression:

$$xy + x'z + yz = xy + x'z$$

Apply relevant laws and theorems to show that the term yz is redundant and obtain the simplified form. **(7 marks)** [CSE 205 | L-2/T-1 | 2024-25]

Answer

Proof of the Consensus Theorem

We are asked to algebraically prove the Consensus Theorem, which states that the term yz is redundant in the expression $xy + x'z + yz$.

Step-by-step Derivation:

We start with the Left Hand Side (LHS) of the equation and apply Boolean algebra postulates and theorems to simplify it to the Right Hand Side (RHS).

LHS = $xy + x'z + yz$	
= $xy + x'z + yz(1)$	(Identity Law: $A \cdot 1 = A$)
= $xy + x'z + yz(x + x')$	(Complementarity: $x + x' = 1$)
= $xy + x'z + xyz + x'yz$	(Distributive Law: $A(B + C) = AB + AC$)
= $xy + xyz + x'z + x'yz$	(Commutative Law: Rearranging terms)
= $xy(1 + z) + x'z(1 + y)$	(Distributive Law: Factoring out xy and $x'z$)
= $xy(1) + x'z(1)$	(Null/Annihilation Law: $1 + A = 1$)
= $xy + x'z$	(Identity Law: $A \cdot 1 = A$)
= RHS	

Conclusion:

The algebraic manipulation clearly demonstrates that the term yz (known as the consensus term) does not add any new logical conditions to the function, as its conditions are already fully covered by the terms xy and $x'z$.

$$xy + x'z + yz = xy + x'z$$

Q2. Analyze whether the dual and the complement of the following function are the same:

$$F = ABC + A(B + C) + A'B'C'$$

Justify your answer. **(8 marks)**

[CSE 205 | L-2/T-1 | 2023-24]

Answer

Analysis of the Function

Given the Boolean function:

$$F = ABC + A(B + C) + A'B'C'$$

To determine if the dual and the complement of the function are the same, we will first simplify the original function F , then derive its complement (F') and its dual (F^D), and finally compare the two results.

Step 1: Simplification of the Original Function (F)

We can simplify F using standard Boolean algebra theorems:

$F = ABC + A(B + C) + A'B'C'$	
$F = ABC + AB + AC + A'B'C'$	(Distributive Law)
$F = AB(C + 1) + AC + A'B'C'$	(Factoring out AB)
$F = AB(1) + AC + A'B'C'$	(Null Law: $C + 1 = 1$)
$F = AB + AC + A'B'C'$	(Identity Law)

Alternatively, we can express the simplified F by factoring out A :

$$F = A(B + C) + A'B'C'$$

Step 2: Derivation of the Complement (F')

The complement of a function is obtained by applying De Morgan's Laws, which states that we invert all variables and swap AND

$(\cdot)$ with OR $(+)$, and vice versa. Using the simplified form $F = A(B + C) + A'B'C'$:

$$\begin{aligned} F' &= (A(B + C) + A'B'C')' \\ F' &= (A(B + C))' \cdot (A'B'C')' && \text{(De Morgan's Law)} \\ F' &= (A' + (B + C)') \cdot (A + B + C) && \text{(De Morgan's Law)} \\ F' &= (A' + B'C') \cdot (A + B + C) && \text{(De Morgan's Law)} \end{aligned}$$

Now, we expand the expression using the Distributive Law:

$$\begin{aligned} F' &= A'A + A'B + A'C + AB'C' + BB'C' + CB'C' \\ F' &= 0 + A'B + A'C + AB'C' + 0 + 0 && \text{(Complement Law: } XX' = 0\text{)} \\ F' &= A'B + A'C + AB'C' \end{aligned}$$

Step 3: Derivation of the Dual (F^D)

The dual of a function is obtained by swapping AND $(\cdot)$ with OR $(+)$, and OR $(+)$ with AND $(\cdot)$, as well as swapping 0s and 1s. **Crucially, the literals themselves are not complemented.** Let us find the dual of the original unsimplified function:

$$F = (A \cdot B \cdot C) + (A \cdot (B + C)) + (A' \cdot B' \cdot C')$$

Applying the duality principle:

$$F^D = (A + B + C) \cdot (A + (B \cdot C)) \cdot (A' + B' + C')$$

Now, we simplify F^D :

$$F^D = [(A + B + C)(A + BC)] \cdot (A' + B' + C')$$

Using the Absorption Law $Y(Y + Z) = Y$, let $Y = A + BC$ and $Z = B + C$. Notice that $A + BC$ is a subset of $A + B + C$ because $BC = 1 \implies B = 1$ and $C = 1 \implies B + C = 1$. Thus, $(A + BC)(A + B + C) = A + BC$.

$$\begin{aligned} F^D &= (A + BC) \cdot (A' + B' + C') \\ F^D &= A(A' + B' + C') + BC(A' + B' + C') && \text{(Distributive Law)} \\ F^D &= AA' + AB' + AC' + A'BC + BB'C + BCC' && \text{(Distributive Law)} \\ F^D &= 0 + AB' + AC' + A'BC + 0 + 0 && \text{(Complement Law: } XX' = 0\text{)} \\ F^D &= AB' + AC' + A'BC \end{aligned}$$

Step 4: Comparison and Verification

Comparing the minimized expressions for F' and F^D :

- **Complement:** $F' = A'B + A'C + AB'C'$
- **Dual:** $F^D = AB' + AC' + A'BC$

The expressions are algebraically different. To conclusively prove this, we can evaluate both functions using a truth table.

A	B	C	$F = AB + AC + A'B'C'$	$F' = A'B + A'C + AB'C'$	$F^D = AB' + AC' + A'BC$
0	0	0	1	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	0	1	1
1	0	1	1	0	1
1	1	0	1	0	1
1	1	1	1	0	0

Conclusion

As observed in both the derived algebraic expressions and the truth table, the outputs for F' and F^D do not match for multiple input combinations (e.g., for inputs 001, 010, 101, and 110). Therefore, the dual and the complement of the given function are **not the same**.

Q3. Using Boolean algebra, show that $F = xyz' + wx'y' + (x' + z + w)(x' + z + w') + xyz + wx'y$ can be simplified to $F = xy + wx' + x' + z$.
(11 marks) [CSE 205 | L-2/T-1 | 2023–24]

Answer

Simplification of the Boolean Function

Given the original Boolean function:

$$F = xyz' + wx'y' + (x' + z + w)(x' + z + w') + xyz + wx'y$$

We will simplify this step-by-step using standard Boolean algebra theorems and laws to show that it reduces to $F = xy + wx' + x' + z$.

Step 1: Rearrange and Group Terms

First, we use the **Commutative Law** to rearrange the terms, bringing similar products together to make factoring easier:

$$F = (xyz' + xyz) + (wx'y' + wx'y) + (x' + z + w)(x' + z + w')$$

Step 2: Simplify the Sum of Products (SOP) Terms

Apply the **Distributive Law** to factor out the common literals from the grouped SOP terms:

$$F = xy(z' + z) + wx'(y' + y) + (x' + z + w)(x' + z + w')$$

Apply the **Complement Law** ($A + A' = 1$) to the terms inside the parentheses:

$$F = xy(1) + wx'(1) + (x' + z + w)(x' + z + w')$$

Apply the **Identity Law** ($A \cdot 1 = A$):

$$F = xy + wx' + (x' + z + w)(x' + z + w')$$

Step 3: Simplify the Product of Sums (POS) Term

Now we focus on the third component: $(x' + z + w)(x' + z + w')$. We can treat the sub-expression $(x' + z)$ as a single variable. Let $A = x' + z$. Using the **Distributive Law** $(A + B)(A + C) = A + BC$ in reverse, we get:

$$(x' + z + w)(x' + z + w') = (x' + z) + (w \cdot w')$$

Apply the **Complement Law** ($w \cdot w' = 0$):

$$(x' + z + w)(x' + z + w') = (x' + z) + 0$$

Apply the **Identity Law** ($A + 0 = A$):

$$(x' + z + w)(x' + z + w') = x' + z$$

Step 4: Combine the Simplified Parts

Substitute the simplified POS term back into the main expression for F :

$$F = xy + wx' + (x' + z)$$

$$F = xy + wx' + x' + z$$

Conclusion

Through the step-by-step application of Boolean algebra laws, we have successfully demonstrated that:

$$\mathbf{F = xy + wx' + x' + z}$$

Instructor's Note for Full Minimization:

While we have successfully derived the requested target expression, it is important to recognize that this expression is not yet fully minimized for optimal circuit design.

- Using the **Absorption Law**: $wx' + x' = x'(w + 1) = x'(1) = x'$.
- The function reduces to: $F = xy + x' + z$.
- Using the **Simplification (Redundancy) Law** ($A' + AB = A' + B$): $x' + xy = x' + y$.
- The ultimately minimized form of this function is $F = x' + y + z$.

Q4. Simplify the following Boolean expressions to a minimum number of literals: $(A + B)'(A' + B)'$. **(10 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Answer

Simplification of the Boolean Expression

Given the Boolean expression:

$$F = (A + B)'(A' + B)'$$

We will simplify this expression step-by-step to a minimum number of literals using fundamental Boolean algebra laws.

Step-by-Step Derivation:

$$\begin{aligned}
 F &= (A + B)' \cdot (A' + B)'\quad && \text{(De Morgan's Law: } (X + Y)' = X' \cdot Y') \\
 F &= (A' \cdot B') \cdot (A' + B)'\quad && \text{(De Morgan's Law: } (X + Y)' = X' \cdot Y') \\
 F &= (A' \cdot B') \cdot ((A')' \cdot (B')')\quad && \text{(Involution Law: } (X')' = X) \\
 F &= (A' \cdot B') \cdot (A \cdot B)\quad && \text{(Associative Law)} \\
 F &= A' \cdot B' \cdot A \cdot B\quad && \text{(Commutative Law)} \\
 F &= (A \cdot A') \cdot (B \cdot B')\quad && \text{(Complement Law: } X \cdot X' = 0) \\
 F &= (0) \cdot (0)\quad && \text{(Null Law: } 0 \cdot X = 0) \\
 F &= 0
 \end{aligned}$$

Intuitive Explanation:

We can also analyze the function logically. The expression consists of the product (AND) of two terms:

- (a) $(A + B)'$: This is the NOR operation, which outputs 1 only when both $A = 0$ and $B = 0$.
- (b) $(A' + B)'$: By De Morgan's law, this is equivalent to $A \cdot B$ (the AND operation), which outputs 1 only when both $A = 1$ and $B = 1$.

Because it is impossible for the variables A and B to simultaneously be 0 (to satisfy the NOR term) and 1 (to satisfy the AND term), the intersection of these two conditions is an empty set. Thus, the product of these two terms will always yield 0 regardless of the inputs.

Conclusion:

The Boolean expression logically evaluates to false for all input combinations. It simplifies entirely to the constant logic level **0**. Therefore, the minimum number of literals is zero.

Q5. Simplify the following expression to POS and SOP: $ACD' + C'D + AB' + ABCD$. (5 marks) [CSE 205 | L-2/T-1 | 2018–19]

Answer

Simplification of the Boolean Expression

Given the Boolean function:

$$F = ACD' + C'D + AB' + ABCD$$

We will simplify this expression into its minimum Sum of Products (SOP) and Product of Sums (POS) forms algebraically, and verify our results using Karnaugh Maps.

Step 1: Algebraic Simplification to Sum of Products (SOP)

We simplify the given expression step-by-step using fundamental Boolean algebra laws:

$$\begin{aligned}
 F &= C'D + AB' + ACD' + ABCD\quad && \text{(Commutative Law)} \\
 F &= C'D + AB' + AC(D' + BD)\quad && \text{(Distributive Law: factoring } AC) \\
 F &= C'D + AB' + AC(D' + B)\quad && \text{(Simplification Law: } X' + XY = X' + Y) \\
 F &= C'D + AB' + ACD' + ABC\quad && \text{(Distributive Law)} \\
 F &= C'D + A(B' + BC) + ACD'\quad && \text{(Factoring } A \text{ from 2nd and 4th terms)} \\
 F &= C'D + A(B' + C) + ACD'\quad && \text{(Simplification Law: } X' + XY = X' + Y) \\
 F &= C'D + AB' + AC + ACD'\quad && \text{(Distributive Law)} \\
 F &= C'D + AB' + AC(1 + D')\quad && \text{(Factoring } AC \text{ from 3rd and 4th terms)} \\
 F &= C'D + AB' + AC(1)\quad && \text{(Null Law: } 1 + X = 1) \\
 F &= C'D + AB' + AC\quad && \text{(Identity Law)}
 \end{aligned}$$

Thus, the simplified **SOP** expression is:

$$F_{\text{SOP}} = C'D + AB' + AC$$

Step 2: Algebraic Simplification to Product of Sums (POS)

We can derive the POS form directly from the simplified SOP expression by repeatedly applying the Distributive Law $(X + YZ = (X + Y)(X + Z))$:

$$\begin{aligned}
 F &= C'D + A(B' + C)\quad && \text{(Factoring } A \text{ from } AB' + AC) \\
 F &= (C'D + A) \cdot (C'D + B' + C)\quad && \text{(Distributive Law)}
 \end{aligned}$$

Now, we simplify the two resulting terms separately.

First term:

$$C'D + A = (A + C')(A + D) \quad (\text{Distributive Law})$$

Second term:

$$\begin{aligned} C'D + B' + C &= (C + C'D) + B' && (\text{Commutative Law}) \\ &= (C + C')(C + D) + B' && (\text{Distributive Law}) \\ &= (1)(C + D) + B' && (\text{Complement Law}) \\ &= B' + C + D \end{aligned}$$

Substituting these simplified factors back into the equation for F :

$$\mathbf{F_{POS}} = (\mathbf{A + C'})(\mathbf{A + D})(\mathbf{B' + C + D})$$

Step 3: Karnaugh Map Verification

To rigorously verify our algebraic minimizations, we plot the unsimplified function on a 4-variable K-map. We can derive SOP by grouping the 1s and POS by grouping the 0s.

		SOP Grouping (1s)						POS Grouping (0s)			
		CD						CD			
AB		00	01	11	10	AB		00	01	11	10
00		0	1	0	0	00		0	1	0	0
01		0	1	0	0	01		0	1	0	0
11		0	1	1	1	11		0	1	1	1
10		1	1	1	1	10		1	1	1	1

- **SOP Extraction:** By grouping the 1s, we extract the minterms.

- **Blue Column (01):** Constant $C = 0, D = 1 \implies C'D$
- **Red Row (10):** Constant $A = 1, B = 0 \implies AB'$
- **Green 2x2 Square:** Constant $A = 1, C = 1 \implies AC$

Summing these groups confirms $\mathbf{F_{SOP}} = \mathbf{C'D + AB' + AC}$.

- **POS Extraction:** By grouping the 0s, we extract the maxterms. Recall that for maxterms, logic is inverted (0 $\rightarrow$ uncomplemented, 1 $\rightarrow$ complemented).

- **Magenta 2x2 Square:** Constant $A = 0, C = 1 \implies (A + C')$
- **Orange Edges (Top Corners):** Constant $A = 0, D = 0 \implies (A + D)$
- **Cyan 1x2 Rectangle:** Constant $B = 1, C = 0, D = 0 \implies (B' + C + D)$

Multiplying these groups confirms $\mathbf{F_{POS}} = (\mathbf{A + C'})(\mathbf{A + D})(\mathbf{B' + C + D})$.

Q6. Simplify the following expression to POS and SOP: $BCD' + ABC' + ACD$. (8 marks)

[CSE 205 | L-2/T-1 | 2017–18]

Answer

Simplification of the Boolean Expression

Given the Boolean function:

$$F = BCD' + ABC' + ACD$$

We will simplify this expression into its minimum Sum of Products (SOP) and Product of Sums (POS) forms algebraically, and then verify our results using Karnaugh Maps.

Step 1: Algebraic Simplification to Sum of Products (SOP)

We can simplify the given expression step-by-step using fundamental Boolean algebra laws, specifically relying on the **Consensus Theorem** ($XY + X'Z + YZ = XY + X'Z$).

Observe the terms BCD' and ACD . The variable D is uncomplemented in the latter and complemented in the former. The consensus of these two terms is $(BC)(AC) = ABC$. By the Consensus Theorem, we can add this redundant term without changing

the function's value:

$$\begin{aligned}
 F &= BCD' + ABC' + ACD \\
 F &= BCD' + ABC' + ACD + ABC && \text{(Adding consensus of } BCD' \text{ and } ACD) \\
 F &= BCD' + ACD + ABC' + ABC && \text{(Rearranging terms)} \\
 F &= BCD' + ACD + AB(C' + C) && \text{(Factoring } AB) \\
 F &= BCD' + ACD + AB(1) && \text{(Complement Law: } C' + C = 1) \\
 F &= BCD' + ACD + AB && \text{(Identity Law)}
 \end{aligned}$$

Rearranging to a standard order, the simplified **SOP** expression is:

$$\mathbf{F_{SOP} = AB + ACD + BCD'}$$

Step 2: Algebraic Simplification to Product of Sums (POS)

We derive the POS form directly from our minimal SOP expression using the Distributive Law ($X + YZ = (X + Y)(X + Z)$) and the Consensus Theorem.

$$\begin{aligned}
 F &= AB + ACD + BCD' \\
 F &= AB + C(AD + BD') && \text{(Factoring } C \text{ from the last two terms)} \\
 F &= (AB + C)(AB + AD + BD') && \text{(Distributive Law)}
 \end{aligned}$$

Now, we simplify both factors:

(a) **First factor:** By the Distributive Law, $(AB + C) = (A + C)(B + C)$.

(b) **Second factor:** Notice that AB is the consensus of AD and BD' . Therefore, $AD + BD' + AB = AD + BD'$. We can use this property to eliminate AB :

$$AB + AD + BD' = AD + BD'$$

Furthermore, we can factor $AD + BD'$ by recognizing it is the expanded form of $(A + D')(B + D)$:

$$(A + D')(B + D) = AB + AD + BD' + DD' = AB + AD + BD' = AD + BD'$$

Substituting these simplified factors back into our expression for F :

$$F = (A + C)(B + C) \cdot (A + D')(B + D)$$

Thus, the simplified **POS** expression is:

$$\mathbf{F_{POS} = (A + C)(B + C)(B + D)(A + D')}$$

Step 3: Karnaugh Map Verification

To rigorously verify our algebraic minimizations, we plot the function on a 4-variable K-map. We derive SOP by grouping the 1s and POS by grouping the 0s.

SOP Grouping (1s)

		CD			
AB		00	01	11	10
00		0	0	0	0
01		0	0	0	1
11		1	1	1	1
10		0	0	1	0

POS Grouping (0s)

		CD			
AB		00	01	11	10
00		0	0	0	0
01		0	0	0	1
11		1	1	1	1
10		0	0	1	0

• SOP Extraction (1s):

- **Blue Row:** Constant $A = 1, B = 1 \Rightarrow AB$
- **Red Column Pair:** Constant $A = 1, C = 1, D = 1 \Rightarrow ACD$
- **Green Column Pair:** Constant $B = 1, C = 1, D = 0 \Rightarrow BCD'$

This confirms $\mathbf{F_{SOP} = AB + ACD + BCD'}$.

• POS Extraction (0s): (Note: 0 $\rightarrow$ uncomplemented, 1 $\rightarrow$ complemented)

- **Magenta 2x2:** Constant $A = 0, C = 0 \Rightarrow (A + C)$
- **Brown 2x2:** Constant $A = 0, D = 1 \Rightarrow (A + D')$
- **Orange Split 2x2:** Constant $B = 0, C = 0 \Rightarrow (B + C)$
- **Cyan Four Corners:** Constant $B = 0, D = 0 \Rightarrow (B + D)$

This confirms $\mathbf{F_{POS} = (A + C)(B + C)(B + D)(A + D')}$.

Q7. Prove the following Boolean algebra theorem:

$$(x + y)(x' + z)(y + z) = (x + y)(x' + z)$$

(10 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

Proof of the Boolean Algebra Theorem (Consensus Theorem in POS Form)

The given theorem is the Product of Sums (POS) form of the **Consensus Theorem**. We can prove this theorem using two rigorous methods: Algebraic Manipulation and Truth Table Verification.

Method 1: Algebraic Proof

We start with the Left-Hand Side (LHS) of the equation and apply standard Boolean algebra postulates and theorems to reduce it to the Right-Hand Side (RHS).

$$\begin{aligned} \text{LHS} &= (x + y)(x' + z)(y + z) \\ &= (x + y)(x' + z)(y + z + 0) && \text{(Identity Law: } A + 0 = A\text{)} \\ &= (x + y)(x' + z)(y + z + xx') && \text{(Complement Law: } xx' = 0\text{)} \\ &= (x + y)(x' + z)(x + y + z)(x' + y + z) && \text{(Distributive Law: } A + BC = (A + B)(A + C)\text{)} \end{aligned}$$

Now, we rearrange the terms using the Commutative Law to group related sums together:

$$\text{LHS} = [(x + y)(x + y + z)] \cdot [(x' + z)(x' + z + y)] \quad \text{(Commutative Law)}$$

Next, we apply the **Absorption Law**, which states that $A(A + B) = A$.

- For the first group, let $A = (x + y)$ and $B = z$. Thus, $(x + y)(x + y + z) = (x + y)$.
- For the second group, let $A = (x' + z)$ and $B = y$. Thus, $(x' + z)(x' + z + y) = (x' + z)$.

Substituting these absorbed expressions back into our main equation gives:

$$\begin{aligned} \text{LHS} &= (x + y)(x' + z) \\ &= \text{RHS} \end{aligned}$$

Thus, the theorem is algebraically proven.

Method 2: Verification via Truth Table

To ensure complete verification of the circuit behavior, we can exhaustively test all possible combinations of the variables x, y , and z .

x	y	z	x'	Term 1: $(x + y)$	Term 2: $(x' + z)$	Term 3: $(y + z)$	LHS: $T1 \cdot T2 \cdot T3$	RHS: $T1 \cdot T2$
0	0	0	1	0	1	0	0	0
0	0	1	1	0	1	1	0	0
0	1	0	1	1	1	1	1	1
0	1	1	1	1	1	1	1	1
1	0	0	0	1	0	0	0	0
1	0	1	0	1	1	1	1	1
1	1	0	0	1	0	1	0	0
1	1	1	0	1	1	1	1	1

By comparing the last two columns of the truth table, we observe that the output values for the LHS $(x + y)(x' + z)(y + z)$ and the RHS $(x + y)(x' + z)$ are identical for every possible input combination of x, y , and z .

Conclusion: Both the rigorous algebraic derivation and the exhaustive truth table prove that the expression $(x + y)(x' + z)(y + z)$ accurately simplifies to $(x + y)(x' + z)$.

Q8. How many distinct Boolean functions are there of n Boolean variables? (5 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

Total Number of Distinct Boolean Functions

For n Boolean variables, the total number of distinct Boolean functions is given by the formula:

$$2^{2^n}$$

Step-by-Step Derivation:

To understand how this formula is derived mathematically, we can analyze the structure of a standard logic truth table.

- (a) **Determine the number of input combinations:** A Boolean function of n variables (e.g., $x_1, x_2, \dots, x_n$) operates on inputs that can each take on exactly two possible values in binary logic: 0 or 1. Therefore, the total number of unique input combinations (which corresponds to the number of rows in the truth table) is:

$$\text{Number of Rows} = 2 \times 2 \times \dots \times 2 \text{ (} n \text{ times)} = 2^n$$

- (b) **Determine the number of output assignments:** A specific Boolean function is uniquely defined by the output value it maps to each of these 2^n input combinations. For every single row in the truth table, the function's output can be either a 0 or a 1. This gives exactly 2 independent choices per row.

- (c) **Calculate the total possible functions:** Since there are 2 choices for the output of the first row, 2 choices for the second row, and so on, spanning across all 2^n rows, the fundamental counting principle dictates that we multiply the number of choices together:

$$\text{Total Functions} = \underbrace{2 \times 2 \times 2 \times \dots \times 2}_{2^n \text{ times}} = 2^{(2^n)}$$

Illustrative Examples:

To verify this intuitively, we can evaluate the formula for small values of n .

- **Case for $n = 1$ (One variable, e.g., A):**

- Number of input rows: $2^1 = 2$ rows (Inputs $A = 0$ and $A = 1$).
- Number of possible functions: $2^{2^1} = 2^2 = 4$ distinct functions.
- These 4 functions are: $F = 0$ (Constant 0), $F = 1$ (Constant 1), $F = A$ (Buffer/Identity), and $F = A'$ (NOT/Inverter).

- **Case for $n = 2$ (Two variables, e.g., A, B):**

- Number of input rows: $2^2 = 4$ rows (Inputs 00, 01, 10, 11).
- Number of possible functions: $2^{2^2} = 2^4 = 16$ distinct functions.
- These 16 functions represent all possible 2-input logic mappings, including standard logic gates such as AND, OR, NAND, NOR, XOR, XNOR, Implications, Constants, and single-variable passes/inversions.

- **Case for $n = 3$ (Three variables):**

- Number of input rows: $2^3 = 8$ rows.
- Number of possible functions: $2^{2^3} = 2^8 = 256$ distinct Boolean functions.

Summary Table of Combinatorial Growth:

The number of functions exhibits double exponential growth, as shown below:

Number of Variables (n)	Truth Table Rows (2^n)	Distinct Functions (2^{2^n})
1	2	4
2	4	16
3	8	256
4	16	65,536
5	32	4,294,967,296

Q9. Simplify the Boolean function using algebraic manipulation:

$$f = A'BC + AB'C + ABC' + ABC$$

(10 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

Algebraic Simplification of the Boolean Function

The given Boolean function is:

$$f = A'BC + AB'C + ABC' + ABC$$

This function represents the Sum of Products (SOP) form of the logical Majority Function (or the Carry-Out of a Full Adder). We can simplify this expression step-by-step using standard Boolean algebraic laws.

Step-by-Step Derivation:

$$f = A'BC + AB'C + ABC' + ABC$$

According to the **Idempotent Law** ($X + X = X$), we can duplicate the term ABC multiple times without changing the value of the function. We will add two extra copies of ABC to group with the other three terms:

$$f = A'BC + AB'C + ABC' + (ABC + ABC + ABC)$$

Now, we use the **Commutative Law** to rearrange the terms, pairing one ABC with each of the other terms:

$$f = (A'BC + ABC) + (AB'C + ABC) + (ABC' + ABC)$$

Next, we apply the **Distributive Law** ($XY + XZ = X(Y + Z)$) to factor out the common variables in each grouped pair:

$$f = BC(A' + A) + AC(B' + B) + AB(C' + C)$$

According to the **Complement Law**, the OR sum of a variable and its complement is always 1 ($X + X' = 1$):

$$f = BC(1) + AC(1) + AB(1)$$

Finally, applying the **Identity Law** ($X \cdot 1 = X$), we remove the 1s to obtain the simplified minimal Sum of Products expression:

$$f = BC + AC + AB$$

Rearranging the terms alphabetically for standard presentation, we get the final simplified expression:

$$f = AB + BC + AC$$

Verification using Karnaugh Map (Visualization)

To visually verify our algebraic simplification, we can map the original standard SOP function onto a 3-variable Karnaugh Map. The original function $f = A'BC + AB'C + ABC' + ABC$ corresponds to minterms m_3, m_5, m_6 , and m_7 .

		BC			
		00	01	11	10
A	0	0	0	1	0
	1	0	1	1	1

Group 1 (Vertical): BC
 Group 2 (Horizontal Left): AC
 Group 3 (Horizontal Right): AB

From the K-map, we form three overlapping groups of two adjacent 1s:

- **Red Group** covers m_3 and m_7 , where B and C remain constant (11). This gives the term BC .
- **Blue Group** covers m_5 and m_7 , where A and C remain constant (1 and 1). This gives the term AC .
- **Green Group** covers m_6 and m_7 , where A and B remain constant (11). This gives the term AB .

Combining these groupings yields $f = AB + BC + AC$, which perfectly matches our algebraic derivation.

Q10. Simplify the following Boolean function using algebraic manipulation:

$$(A + BC)' + AB'$$

(7 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

Algebraic Simplification of the Boolean Function

The given Boolean expression is:

$$f = (A + BC)' + AB'$$

We can simplify this function step-by-step using standard Boolean algebraic theorems and postulates.

Step-by-Step Derivation:

$$f = (A + BC)' + AB'$$

Step 1: Apply **De Morgan's Law** $(X + Y)' = X'Y'$ to the first term, treating A as X and BC as Y .

$$f = A'(BC)' + AB'$$

Step 2: Apply **De Morgan's Law** $(XY)' = X' + Y'$ to the term $(BC)'$.

$$f = A'(B' + C') + AB'$$

Step 3: Apply the **Distributive Law** $X(Y + Z) = XY + XZ$ to expand the first part of the expression.

$$f = A'B' + A'C' + AB'$$

Step 4: Rearrange the terms using the **Commutative Law** $(X + Y = Y + X)$ to place terms containing B' adjacent to each other.

$$f = A'B' + AB' + A'C'$$

Step 5: Apply the **Distributive Law** in reverse to factor out the common literal B' from the first two terms.

$$f = B'(A' + A) + A'C'$$

Step 6: Apply the **Complement Law**, which states that the logical OR of a variable and its complement is always true ($X' + X = 1$).

$$f = B'(1) + A'C'$$

Step 7: Finally, apply the **Identity Law** $(X \cdot 1 = X)$ to eliminate the 1.

$$f = B' + A'C'$$

The final simplified Boolean expression is:

$$\mathbf{f = B' + A'C'}$$

Verification using Truth Table

To ensure the accuracy of our algebraic simplification, we can construct a truth table comparing the original function to our simplified result across all possible input combinations for A , B , and C .

A	B	C	A'	B'	C'	Term 1: $(A + BC)'$	Term 2: AB'	Original: $(A + BC)' + AB'$	Simplified: $B' + A'C'$
0	0	0	1	1	1	$(0 + 0)' = 1$	$0 \cdot 1 = 0$	$1 + 0 = 1$	$1 + (1 \cdot 1) = 1$
0	0	1	1	1	0	$(0 + 0)' = 1$	$0 \cdot 1 = 0$	$1 + 0 = 1$	$1 + (1 \cdot 0) = 1$
0	1	0	1	0	1	$(0 + 0)' = 1$	$0 \cdot 0 = 0$	$1 + 0 = 1$	$0 + (1 \cdot 1) = 1$
0	1	1	1	0	0	$(0 + 1)' = 0$	$0 \cdot 0 = 0$	$0 + 0 = 0$	$0 + (1 \cdot 0) = 0$
1	0	0	0	1	1	$(1 + 0)' = 0$	$1 \cdot 1 = 1$	$0 + 1 = 1$	$1 + (0 \cdot 1) = 1$
1	0	1	0	1	0	$(1 + 0)' = 0$	$1 \cdot 1 = 1$	$0 + 1 = 1$	$1 + (0 \cdot 0) = 1$
1	1	0	0	0	1	$(1 + 0)' = 0$	$1 \cdot 0 = 0$	$0 + 0 = 0$	$0 + (0 \cdot 1) = 0$
1	1	1	0	0	0	$(1 + 1)' = 0$	$1 \cdot 0 = 0$	$0 + 0 = 0$	$0 + (0 \cdot 0) = 0$

Conclusion: By observing the last two columns of the truth table, we see that the output vectors for the original function $(A + BC)' + AB'$ and the simplified expression $B' + A'C'$ match perfectly for all $2^3 = 8$ possible input combinations. This conclusively proves that our algebraic simplification is correct.

Q11. State and prove the consensus theorem using Boolean algebra. (8 marks)

[CSE 205 | L-2/T-1 | 2014–15]

Answer

Statement of the Consensus Theorem

The Consensus Theorem is a powerful rule in Boolean algebra used to minimize logic expressions by eliminating redundant terms. It states that if a Boolean expression contains two terms where one variable appears uncomplemented in one term and complemented in the other, and a third term contains the remaining variables from the first two terms, this third term is redundant and can be omitted. The redundant third term is known as the **consensus term**.

The theorem exists in two dual forms:

- **Sum of Products (SOP) Form:**

$$XY + X'Z + YZ = XY + X'Z$$

- **Product of Sums (POS) Form:**

$$(X + Y)(X' + Z)(Y + Z) = (X + Y)(X' + Z)$$

Proof of the Sum of Products (SOP) Form

We will prove the SOP form using standard Boolean algebraic postulates and theorems, starting with the Left-Hand Side (LHS) and simplifying it to match the Right-Hand Side (RHS).

$$\text{LHS} = XY + X'Z + YZ$$

Step 1: Multiply the consensus term (YZ) by 1 using the **Identity Law** ($A \cdot 1 = A$).

$$= XY + X'Z + YZ \cdot 1$$

Step 2: Substitute 1 with $(X + X')$ using the **Complement Law** ($A + A' = 1$).

$$= XY + X'Z + YZ(X + X')$$

Step 3: Expand the expression using the **Distributive Law** ($A(B + C) = AB + AC$).

$$= XY + X'Z + XYZ + X'YZ$$

Step 4: Rearrange the terms using the **Commutative Law** to group related terms together.

$$= XY + XYZ + X'Z + X'YZ$$

Step 5: Factor out common variables using the **Distributive Law** in reverse.

$$= XY(1 + Z) + X'Z(1 + Y)$$

Step 6: Apply the **Annulment Law** (also known as the Null Law), which states that $1 + A = 1$.

$$= XY(1) + X'Z(1)$$

Step 7: Finally, apply the **Identity Law** ($A \cdot 1 = A$) to yield the RHS.

$$\begin{aligned} &= XY + X'Z \\ &= \text{RHS} \end{aligned}$$

Thus, the SOP form of the Consensus Theorem is algebraically proven.

Proof of the Product of Sums (POS) Form

Similarly, we can prove the POS form by direct algebraic manipulation.

$$\text{LHS} = (X + Y)(X' + Z)(Y + Z)$$

Step 1: Add 0 to the consensus term $(Y + Z)$ using the **Identity Law** ($A + 0 = A$).

$$= (X + Y)(X' + Z)(Y + Z + 0)$$

Step 2: Substitute 0 with XX' using the **Complement Law** ($A \cdot A' = 0$).

$$= (X + Y)(X' + Z)(Y + Z + XX')$$

Step 3: Expand using the **Distributive Law** ($A + BC = (A + B)(A + C)$).

$$= (X + Y)(X' + Z)(X + Y + Z)(X' + Y + Z)$$

Step 4: Rearrange terms using the **Commutative Law** to pair terms efficiently.

$$= [(X + Y)(X + Y + Z)] \cdot [(X' + Z)(X' + Z + Y)]$$

Step 5: Apply the **Absorption Law** $A(A + B) = A$. In the first bracketed group, let $A = X + Y$ and $B = Z$. In the second group, let $A = X' + Z$ and $B = Y$.

$$\begin{aligned} &= (X + Y) \cdot (X' + Z) \\ &= \text{RHS} \end{aligned}$$

Thus, the POS form of the Consensus Theorem is also algebraically proven.

Q12. Simplify the following function using Boolean algebra:

$$F(A, B, C) = A'BC + AB'C + ABC$$

(7 marks)

[CSE 205 | L-2/T-1 | 2014–15]

Answer

Algebraic Simplification of the Boolean Function

The given Boolean function is:

$$F(A, B, C) = A'BC + AB'C + ABC$$

We can simplify this expression step-by-step using standard Boolean algebraic postulates and theorems. There are multiple valid algebraic paths to reach the minimal form; we will use the highly effective Redundancy Law (also known as the Simplification Theorem).

Step-by-Step Derivation:

$$F = A'BC + AB'C + ABC$$

Step 1: Apply the **Distributive Law** ($XY + XZ = X(Y + Z)$) to factor out the common variables AC from the second and third terms.

$$F = A'BC + AC(B' + B)$$

Step 2: Apply the **Complement Law**, which states that the logical OR of a variable and its complement is always 1 ($X + X' = 1$). Here, $B' + B = 1$.

$$F = A'BC + AC(1)$$

Step 3: Apply the **Identity Law** ($X \cdot 1 = X$) to eliminate the 1.

$$F = A'BC + AC$$

Step 4: Apply the **Distributive Law** again to factor out the common variable C from both remaining terms.

$$F = C(A'B + A)$$

Step 5: Apply the **Commutative Law** to rearrange the terms inside the parenthesis for clarity.

$$F = C(A + A'B)$$

Step 6: Apply the **Redundancy Law** (or Rule of Simplification), which states that $X + X'Y = X + Y$. Let $X = A$ and $Y = B$.

$$F = C(A + B)$$

Step 7: Finally, expand the expression using the **Distributive Law** to present the answer in standard Sum of Products (SOP) form.

$$F = AC + BC$$

The final simplified Boolean expression is:

$$\mathbf{F = AC + BC \quad \text{or} \quad \mathbf{F = C(A + B)}}$$

Verification using Karnaugh Map (Visualization)

To rigorously verify our algebraic simplification, we can map the original function onto a 3-variable Karnaugh Map. The original function $F = A'BC + AB'C + ABC$ corresponds to the minterms m_3 (011), m_5 (101), and m_7 (111).

		BC			
A		00	01	11	10
	0	0	0	1	0
	1	0	1	1	0

Group 1 (Vertical): BC
Group 2 (Horizontal): AC

By analyzing the K-map, we can form two optimal groups of adjacent 1s:

- **Red Group** covers minterms m_3 and m_7 . In this vertical group, A changes from 0 to 1 (and is eliminated), while B and C remain constant at 11. This yields the term BC .
- **Blue Group** covers minterms m_5 and m_7 . In this horizontal group, B changes from 0 to 1 (and is eliminated), while A and C remain constant at 1 and 1. This yields the term AC .

Combining the terms derived from the logical groupings yields $F = AC + BC$. This visual minimization perfectly corroborates our step-by-step algebraic simplification.

Q13. Simplify the following Boolean expressions to a minimum number of literals using Boolean algebra:

(a) $AB + A(CD + CD') + A'$

(b) $(A + B)'(A' + B)'$

(10 marks)

[CSE 205 | L-2/T-1 | 2013–14]

Answer

Boolean Algebra Simplification

We are tasked with simplifying two Boolean expressions to their minimum number of literals. We will proceed step-by-step, clearly stating the Boolean theorems and postulates used at each stage.

Part 1: Simplify $AB + A(CD + CD') + A'$

Let $F_1 = AB + A(CD + CD') + A'$.

$$F_1 = AB + A(CD + CD') + A'$$

Step 1: Factor out the common variable C from the terms inside the parentheses using the **Distributive Law** ($XY + XZ = X(Y + Z)$).

$$= AB + AC(D + D') + A'$$

Step 2: Apply the **Complement Law**, which states that the logical OR of a variable and its complement is always 1 ($X + X' = 1$). Here, $D + D' = 1$.

$$= AB + AC(1) + A'$$

Step 3: Apply the **Identity Law** ($X \cdot 1 = X$) to eliminate the 1.

$$= AB + AC + A'$$

Step 4: Rearrange the terms using the **Commutative Law** ($X + Y = Y + X$) to position A' next to AB .

$$= A' + AB + AC$$

Step 5: Apply the **Distributive Law** in the form $X + YZ = (X + Y)(X + Z)$ to the first two terms: $A' + AB = (A' + A)(A' + B)$.

$$= (A' + A)(A' + B) + AC$$

Step 6: Apply the **Complement Law** again ($A' + A = 1$).

$$= 1 \cdot (A' + B) + AC$$

Step 7: Apply the **Identity Law** ($1 \cdot X = X$).

$$= A' + B + AC$$

Step 8: Rearrange terms again using the **Commutative Law** to group A' and AC .

$$= B + A' + AC$$

Step 9: Apply the **Distributive Law** to $A' + AC$, expanding it to $(A' + A)(A' + C)$.

$$= B + (A' + A)(A' + C)$$

Step 10: Use the **Complement Law** ($A' + A = 1$) and the **Identity Law** to simplify the term.

$$\begin{aligned} &= B + 1 \cdot (A' + C) \\ &= B + A' + C \end{aligned}$$

Rearranging the variables alphabetically, the final simplified expression is:

$$\mathbf{F_1 = A' + B + C}$$

Part 2: Simplify $(A + B)'(A' + B)'$

Let $F_2 = (A + B)'(A' + B)'$.

$$F_2 = (A + B)'(A' + B)'$$

Step 1: Apply **De Morgan's Law** $[(X + Y)' = X'Y']$ to the first term, $(A + B)'$.

$$= (A'B') \cdot (A' + B')'$$

Step 2: Apply **De Morgan's Law** again to the second term, $(A' + B')'$.

$$= (A'B') \cdot ((A')'(B')')$$

Step 3: Apply the **Involution Law** (or Double Negation Law), which states $(X')' = X$, to simplify $(A')'$ to A and $(B')'$ to B .

$$= (A'B') \cdot (AB)$$

Step 4: Remove the parentheses using the **Associative Law** and rearrange the variables using the **Commutative Law** to group identical variables together.

$$= (A \cdot A') \cdot (B \cdot B')$$

Step 5: Apply the **Complement Law** (AND form), which states that the logical AND of a variable and its complement is always 0 ($X \cdot X' = 0$).

$$= 0 \cdot 0$$

Step 6: Apply the **Idempotent Law** (or Null Law) to conclude the simplification.

$$= 0$$

The final simplified expression is:

$$\mathbf{F_2 = 0}$$

Q14. The Boolean expression for an X-OR gate is $AB' + A'B$. With this as a starting point, use DeMorgan's theorems and other rules or laws to find a simplified expression for an X-NOR gate. **(8 marks)** [CSE 205 | L-2/T-1 | 2011–12]

Answer

Derivation of the X-NOR Expression

The Exclusive-NOR (X-NOR) gate operates as the logical complement (inversion) of the Exclusive-OR (X-OR) gate. Therefore, we can derive the Boolean expression for the X-NOR gate by taking the complement of the given X-OR expression and simplifying it using Boolean algebra.

Given the X-OR expression:

$$\text{X-OR} = AB' + A'B$$

The X-NOR expression is:

$$\text{X-NOR} = (AB' + A'B)'$$

Step-by-Step Algebraic Simplification:

$$F_{\text{XNOR}} = (AB' + A'B)'$$

Step 1: Apply **De Morgan's Law** $[(X + Y)' = X' \cdot Y']$ to the entire expression, treating the term AB' as X and the term $A'B$ as Y .

$$= (AB')' \cdot (A'B)'$$

Step 2: Apply **De Morgan's Law** $[(XY)' = X' + Y']$ individually to both grouped product terms.

$$= (A' + (B')') \cdot ((A')' + B')$$

Step 3: Apply the **Involution Law** (Double Negation Law), which states that $(X')' = X$, to eliminate the double primes on variables A and B .

$$= (A' + B) \cdot (A + B')$$

Step 4: Apply the **Distributive Law** to expand the product of sums (multiplying out the terms: First, Outer, Inner, Last).

$$= A'A + A'B' + BA + BB'$$

Step 5: Apply the **Commutative Law** to rearrange the term BA to AB for standard presentation.

$$= A'A + A'B' + AB + BB'$$

Step 6: Apply the **Complement Law** (AND form), which states that a variable ANDed with its logical complement is always 0 ($X \cdot X' = 0$). Therefore, $A'A = 0$ and $BB' = 0$.

$$= 0 + A'B' + AB + 0$$

Step 7: Apply the **Identity Law** ($X + 0 = X$) to remove the zeros from the sum.

$$= A'B' + AB$$

Step 8: Apply the **Commutative Law** one final time to arrange the terms into the universally recognized standard format.

$$= AB + A'B'$$

The simplified Boolean expression for an X-NOR gate is:

$$\mathbf{F_{XNOR} = AB + A'B'}$$

Verification via Truth Table

To rigorously verify that this derived expression correctly models the X-NOR behavior (often called the Equivalence gate, outputting 1 when inputs match), we can evaluate its truth table:

A	B	A'	B'	Term 1: AB	Term 2: A'B'	Output: AB + A'B'
0	0	1	1	0	1	1
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	1	0	0	1	0	1

Conclusion: The truth table explicitly confirms that the output evaluates to 1 only when both inputs are identical ($A = B = 0$ or $A = B = 1$). This fully validates that our algebraically derived expression $\mathbf{AB + A'B'}$ acts correctly as an X-NOR function.

Q15. Give an example of the non-associativity of the 3-input NOR operator. (8 marks)

[CSE 205 | L-2/T-1 | 2011–12]

Answer

Non-Associativity of the 3-Input NOR Operator

For a binary operator to be associative, the order in which grouping is applied must not affect the final result. That is, an operator $\downarrow$ (representing NOR) is associative if and only if for all possible Boolean values A , B , and C :

$$(A \downarrow B) \downarrow C = A \downarrow (B \downarrow C)$$

We will demonstrate that the NOR operator is **non-associative** by using two methods: algebraic expansion and a definitive truth-table counterexample.

1. Algebraic Demonstration

The NOR operation between two variables is defined as the complement of their logical sum:

$$X \downarrow Y = (X + Y)'$$

Let us expand both sides of the associative equation using De Morgan's laws:

• Left-Hand Side (LHS):

$$\begin{aligned} \text{LHS} &= (A \downarrow B) \downarrow C \\ &= ((A + B)') \downarrow C \\ &= ((A + B)' + C)' \end{aligned}$$

Applying De Morgan's Law $(X + Y)' = X'Y'$ and the Involution Law $(X'') = X$:

$$\begin{aligned} &= ((A + B)')' \cdot C' \\ &= (A + B)C' \\ &= AC' + BC' \end{aligned}$$

• Right-Hand Side (RHS):

$$\begin{aligned} \text{RHS} &= A \downarrow (B \downarrow C) \\ &= A \downarrow ((B + C)') \\ &= (A + (B + C)')' \end{aligned}$$

Applying De Morgan's Law and the Involution Law:

$$\begin{aligned}
 &= A' \cdot ((B + C)')' \\
 &= A'(B + C) \\
 &= A'B + A'C
 \end{aligned}$$

Since the expanded expressions $\mathbf{AC' + BC'}$ (LHS) and $\mathbf{A'B + A'C}$ (RHS) are distinct and not logically equivalent, the operator is non-associative.

2. Concrete Truth Table Counterexample

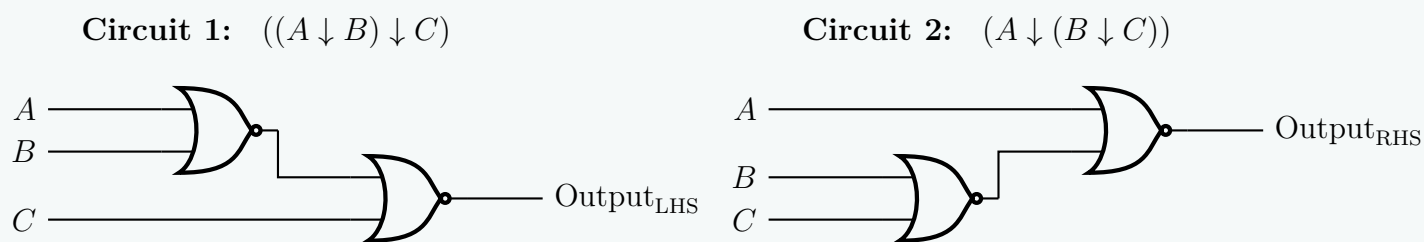
To explicitly prove non-associativity, we evaluate all $2^3 = 8$ combinations of inputs. A single mismatch between the LHS and RHS columns is sufficient to prove non-associativity.

A	B	C	$A \downarrow B$	LHS : $(A \downarrow B) \downarrow C$	$B \downarrow C$	RHS : $A \downarrow (B \downarrow C)$	Status
0	0	0	1	$(1 + 0)' = \mathbf{0}$	1	$(0 + 1)' = \mathbf{0}$	Match
0	0	1	1	$(1 + 1)' = \mathbf{0}$	0	$(0 + 0)' = \mathbf{1}$	Mismatch
0	1	0	0	$(0 + 0)' = \mathbf{1}$	0	$(0 + 0)' = \mathbf{1}$	Match
0	1	1	0	$(0 + 1)' = \mathbf{0}$	0	$(0 + 0)' = \mathbf{1}$	Mismatch
1	0	0	0	$(0 + 0)' = \mathbf{1}$	1	$(1 + 1)' = \mathbf{0}$	Mismatch
1	0	1	0	$(0 + 1)' = \mathbf{0}$	0	$(1 + 0)' = \mathbf{0}$	Match
1	1	0	0	$(0 + 0)' = \mathbf{1}$	0	$(1 + 0)' = \mathbf{0}$	Mismatch
1	1	1	0	$(0 + 1)' = \mathbf{0}$	0	$(1 + 0)' = \mathbf{0}$	Match

As highlighted in the table, rows such as $(A = 0, B = 0, C = 1)$ clearly show a conflict where LHS = 0 while RHS = 1.

3. Structural Visualization via Logic Circuits

The structural asymmetry causing this functional difference can be easily understood by visualizing how the inputs cascade through the hardware connections in each grouping configuration.



Conclusion: Because the grouping changes the propagation structure and produces asymmetric Boolean sub-expressions, **the 3-input NOR operator is inherently non-associative.**

Q16. Prove that $x(x + y) = x$. (5 marks)

[CSE 205 | L-2/T-1 | 2011–12]

Answer

Proof of the Absorption Law

The given Boolean equation, $x(x + y) = x$, is formally known as the **Absorption Law**. It demonstrates how a term can "absorb" another term under specific logical conditions. We can prove this theorem rigorously using two standard methods: algebraic manipulation and perfect induction (truth table verification).

Method 1: Algebraic Proof

We begin with the Left-Hand Side (LHS) of the equation and apply fundamental Boolean postulates and theorems to reduce it to the Right-Hand Side (RHS).

$$\text{LHS} = x(x + y)$$

Step 1: Expand the expression using the **Distributive Law** ($A(B + C) = AB + AC$).

$$= x \cdot x + x \cdot y$$

Step 2: Simplify the term $x \cdot x$ using the **Idempotent Law** ($A \cdot A = A$).

$$= x + xy$$

Step 3: Express the single variable x as $x \cdot 1$ using the **Identity Law** ($A \cdot 1 = A$). This makes factoring easier.

$$= x \cdot 1 + x \cdot y$$

Step 4: Factor out the common variable x by applying the **Distributive Law** in reverse.

$$= x(1 + y)$$

Step 5: Apply the **Annulment Law** (also known as the Dominance Law), which states that the logical OR of 1 and any Boolean variable is always 1 ($1 + A = 1$).

$$= x(1)$$

Step 6: Finally, apply the **Identity Law** ($A \cdot 1 = A$) to eliminate the 1.

$$= x$$
$$= \text{RHS}$$

Thus, the algebraic equivalence is successfully proven.

Method 2: Proof by Perfect Induction (Truth Table)

Another definitive way to prove a Boolean theorem is by evaluating the expression for all possible input combinations of its variables. Since there are two variables (x and y), there are $2^2 = 4$ possible combinations.

Input x	Input y	Intermediate Term: $x + y$	LHS Expression: $x(x + y)$
0	0	$0 + 0 = 0$	$0 \cdot 0 = \mathbf{0}$
0	1	$0 + 1 = 1$	$0 \cdot 1 = \mathbf{0}$
1	0	$1 + 0 = 1$	$1 \cdot 1 = \mathbf{1}$
1	1	$1 + 1 = 1$	$1 \cdot 1 = \mathbf{1}$

Conclusion: By comparing the first column (**Input x**) with the last column (**LHS Expression: $x(x + y)$**), we observe that the logical values are identical for every possible combination of inputs. Because the output of $x(x + y)$ perfectly mirrors the input x , the identity $x(x + y) = x$ is unconditionally true.

De Morgan's Theorem

Q1. Demonstrate the validity of the following identities by means of truth tables:

(a) De Morgan's theorem for three variables: $(x + y + z)' = x'y'z'$

(b) The distributive law: $x(y + z) = xy + xz$

(10 marks)

[CSE 205 | L-2/T-1 | 2019–20]

Answer

Proof of Boolean Identities Using Truth Tables

To demonstrate the validity of a Boolean identity using a truth table, we evaluate both the Left-Hand Side (LHS) and the Right-Hand Side (RHS) expressions for all possible combinations of the input variables. Since both identities contain three binary variables (x , y , and z), there are $2^3 = 8$ distinct input combinations. If the column corresponding to the LHS is identical to the column corresponding to the RHS for every row, the identity is verified.

Part (a): De Morgan's Theorem for Three Variables: $(x + y + z)' = x'y'z'$

De Morgan's theorem states that the complement of a logical sum (OR) is equal to the logical product (AND) of the complements. The table below presents the step-by-step evaluation of both sides of the identity.

x	y	z	$x + y + z$	LHS: $(x + y + z)'$	x'	y'	z'	RHS: $x'y'z'$
0	0	0	0	1	1	1	1	1
0	0	1	1	0	1	1	0	0
0	1	0	1	0	1	0	1	0
0	1	1	1	0	1	0	0	0
1	0	0	1	0	0	1	1	0
1	0	1	1	0	0	1	0	0
1	1	0	1	0	0	0	1	0
1	1	1	1	0	0	0	0	0

Conclusion for Part (a): Comparing the fifth column, **LHS: $(x + y + z)'$** , and the ninth column, **RHS: $x'y'z'$** , we observe that they contain identical values for all eight rows. **Thus, the identity $(x + y + z)' = x'y'z'$ is proven to be valid.**

Part (b): The Distributive Law: $x(y + z) = xy + xz$
The distributive law states that the logical AND operation distributes over the logical OR operation. The table below displays the logical operations to verify this distribution.

x	y	z	$y + z$	LHS: $x(y + z)$	xy	xz	RHS: $xy + xz$
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

Conclusion for Part (b): Comparing the fifth column, **LHS:** $x(y + z)$, and the eighth column, **RHS:** $xy + xz$, we observe that they possess exactly the same truth values for every row configuration. **Thus, the identity $x(y + z) = xy + xz$ is proven to be valid.**

Q2. Given De Morgan’s theorem for two variables: $(A + B)' = A'B'$, prove De Morgan’s theorem for three variables by successive substitution. **(10 marks)** [CSE 205 | L-2/T-1 | 2013–14]

Answer

Proof by Successive Substitution
De Morgan’s theorem for three variables (in its NOR form) states that the complement of the logical sum of three variables is equal to the logical product of their individual complements:

$$(A + B + C)' = A'B'C'$$

We are given the foundation of De Morgan’s theorem for two variables:

$$(X + Y)' = X'Y'$$

We can prove the three-variable theorem mathematically by temporarily grouping variables and applying the two-variable theorem in succession.

Step-by-Step Derivation:
We start by defining the Left-Hand Side (LHS) of the three-variable theorem:

$$\text{LHS} = (A + B + C)'$$

Step 1: Apply the **Associative Law** of Boolean algebra for the OR operation ($A + B + C = (A + B) + C$) to group the first two variables together. Let us define a temporary substitute variable $X = A + B$. Substituting X into the expression yields:

$$= (X + C)'$$

Step 2: The expression is now in the form of the complement of two variables (X and C). Apply the given two-variable **De Morgan’s theorem** $(X + Y)' = X'Y'$.

$$= X' \cdot C'$$

Step 3: Substitute the original sub-expression $(A + B)$ back in place of the temporary variable X .

$$= (A + B)' \cdot C'$$

Step 4: Apply the two-variable **De Morgan’s theorem** a second time, specifically to the first term $(A + B)'$.

$$= (A' \cdot B') \cdot C'$$

Step 5: Finally, apply the **Associative Law** for the logical AND operation to drop the unnecessary parentheses, yielding the Right-Hand Side (RHS).

$$\begin{aligned} &= A' \cdot B' \cdot C' \\ &= A'B'C' \end{aligned}$$

Conclusion: By grouping variables and applying the two-variable De Morgan’s theorem successively, we have rigorously proven that $(A + B + C)' = A'B'C'$.

Extension: The Dual Form (NAND Form)

For absolute completeness in an exam setting, the dual of De Morgan’s theorem for three variables states that $(ABC)' = A' + B' + C'$. This can be proven using the exact same successive substitution logic, provided the two-variable dual $(XY)' = X' + Y'$ is established.

$$\begin{aligned}(ABC)' &= ((AB)C)' && \text{(Associative Law)} \\ &= (AB)' + C' && \text{(Two-variable De Morgan's on } (AB) \text{ and } C) \\ &= (A' + B') + C' && \text{(Two-variable De Morgan's on } AB) \\ &= A' + B' + C' && \text{(Associative Law)}\end{aligned}$$

Both forms rely purely on the principle of extending a two-variable theorem to n -variables through strategic substitution.

Q3. State and prove De Morgan’s theorem using a truth table. (10 marks)

[CSE 205 | L-2/T-1 | 2012–13]

Answer

Statement of De Morgan’s Theorems

De Morgan’s Theorems are two fundamental principles in Boolean algebra that relate the logical AND and OR operations through logical complementation (inversion). They are stated as follows:

- Theorem 1 (NAND to Negative-OR):** The complement of a logical product (AND) of variables is equal to the logical sum (OR) of their individual complements.
$$(A \cdot B)' = A' + B'$$
- Theorem 2 (NOR to Negative-AND):** The complement of a logical sum (OR) of variables is equal to the logical product (AND) of their individual complements.
$$(A + B)' = A' \cdot B'$$

These theorems can be extended to any number of variables (e.g., $(A + B + C + \dots)' = A'B'C' \dots$).

Proof by Perfect Induction (Truth Table)

To prove a Boolean theorem using a truth table, we must evaluate both the Left-Hand Side (LHS) and the Right-Hand Side (RHS) of the equation for all possible combinations of the input variables. If the column representing the LHS matches the column representing the RHS for every row, the theorem is proven valid.

Proof of Theorem 1: $(A \cdot B)' = A' + B'$

A	B	A'	B'	A · B	LHS: (A · B)'	RHS: A' + B'
0	0	1	1	0	1	1
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	1	0	0

Observation: The values in the column for $(A \cdot B)'$ are identical to the values in the column for $A' + B'$ for all input combinations. Thus, Theorem 1 is proven.

Proof of Theorem 2: $(A + B)' = A' \cdot B'$

A	B	A'	B'	A + B	LHS: (A + B)'	RHS: A' · B'
0	0	1	1	0	1	1
0	1	1	0	1	0	0
1	0	0	1	1	0	0
1	1	0	0	1	0	0

Observation: The values in the column for $(A + B)'$ are identical to the values in the column for $A' \cdot B'$ for all input combinations. Thus, Theorem 2 is proven.

Conclusion

Through the exhaustive evaluation of all binary states (perfect induction), both of De Morgan’s theorems are structurally verified and proven universally valid.

Logic Gates, Universal Gates & Definitions

23

Q1. Why are NAND and NOR gates called universal gates? Explain how hardware cost is related to Boolean simplification. If a function is already minimal, how can you verify it? (10 marks) [CSE 205 | L-2/T-1 | 2024–25]

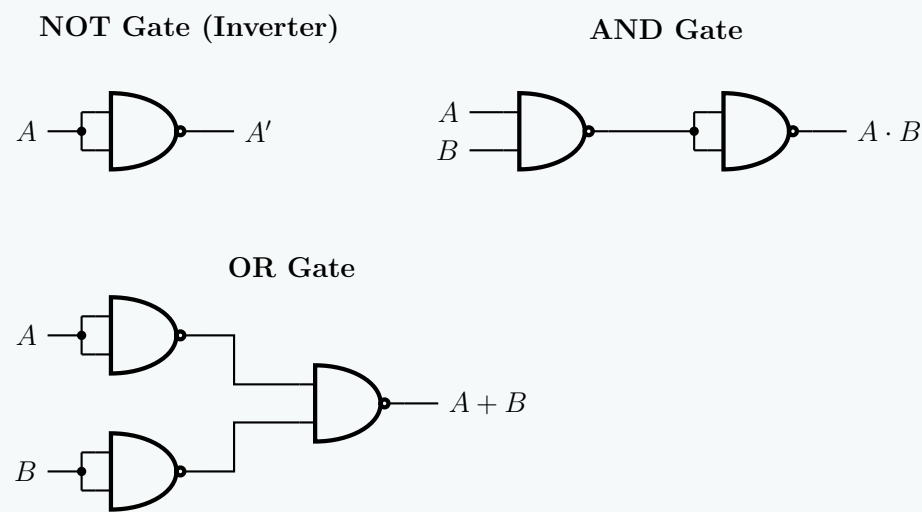
Answer

1. Universality of NAND and NOR Gates

NAND and NOR gates are designated as **universal gates** because either one can be used exclusively to implement *any* Boolean function, without requiring any other type of logic gate.

Any Boolean expression can be written in standard forms such as Sum-of-Products (SOP) or Product-of-Sums (POS), which fundamentally only require NOT, AND, and OR operations. Because the NOT, AND, and OR gates can be constructed entirely out of NAND gates (or entirely out of NOR gates), it follows that any digital circuit can be implemented using only NAND or only NOR gates.

To demonstrate this universality, the realization of the basic gates using only **NAND gates** is shown below:



2. Relationship Between Hardware Cost and Boolean Simplification

In digital logic design, the mathematical structure of a Boolean expression directly correlates to the physical structure of its corresponding hardware circuit. Simplifying a Boolean expression reduces the hardware cost in several critical ways:

- **Number of Terms:** In an SOP expression, each product term represents an AND gate. By minimizing the number of terms using Boolean algebra or K-maps, we directly reduce the *total number of logic gates* required on the silicon chip or Printed Circuit Board (PCB).
- **Number of Literals:** A literal is a primed or unprimed variable in a term (e.g., $A'B$ has two literals). The number of literals in a term dictates the *number of inputs* required for the corresponding gate. Minimizing literals reduces gate complexity, allowing the use of cheaper, standard 2-input or 3-input gates rather than custom multi-input gates.
- **Physical Footprint and Power:** Fewer gates and fewer inputs translate to fewer transistors. This reduces the physical area of the Integrated Circuit (IC), lowers static and dynamic power consumption, and decreases heat dissipation.
- **Propagation Delay:** Simplification often reduces the number of logical levels a signal must traverse, which decreases the overall propagation delay and improves circuit operational speed.

3. Verifying the Minimality of a Boolean Function

If a Boolean function is believed to be minimal, its minimality can be systematically verified using one of the following methods:

(a) Karnaugh Map (K-map) Inspection (For up to 4-5 variables):

- **Check for Prime Implicants:** Ensure that every grouping of 1s (for SOP) or 0s (for POS) is as large as possible (sizes of 1, 2, 4, 8, etc.). If any group can be expanded by combining it with adjacent groups, the function is not yet minimal.
- **Check for Essential Prime Implicants:** Ensure all selected groups are necessary. If all the 1s in a specific group are already covered by other necessary groups, that group is redundant. A strictly minimal expression contains only essential prime implicants and the minimum number of non-essential prime implicants required to cover any remaining 1s.

(b) Quine-McCluskey (Tabulation) Method (For any number of variables):

- This is a rigorous algorithmic approach. You construct a prime implicant chart. If the given expression exactly matches the optimal selection of prime implicants derived from solving the chart (ensuring all minterms are covered with the lowest literal/term count), the expression is proven minimal. Since it is algorithmic, it provides an absolute mathematical proof of minimality, suitable for computer-aided verification.

Q2. How does a three-state gate work? What can be the advantage of using a three-state gate? (6 marks) [CSE 205 | L-2/T-1 | 2024–25]

Answer

Operation of a Three-State Gate

A standard digital logic gate outputs one of two binary states: logic Low (0) or logic High (1). A **three-state gate** (or tri-state

gate), however, exhibits a third stable state called the **High-Impedance state**, denoted by **Z**.

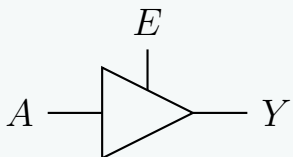
A three-state gate features two primary inputs: a standard **Data Input** (A) and a **Control/Enable Input** (E). The operation depends entirely on the condition of the enable signal:

(a) **Enabled State** ($E = 1$ for active-high): The gate is fully functional and acts as a standard logic buffer. The output follows the input directly ($Y = A$). The gate actively drives the output line to either a low or high voltage.

(b) **Disabled/High-Impedance State** ($E = 0$ for active-high): The output is placed into the high-impedance state (Z). In this state, the internal output transistors are turned off simultaneously, effectively creating an **electrical open circuit**. The gate is disconnected from its output pin, meaning it neither sinks nor sources current, allowing external circuits to dictate the voltage level of the shared line.

Logic Symbol and Truth Table

Below are the schematic representation and the corresponding functional truth table for an active-high three-state buffer:



Enable (E)	Input (A)	Output (Y)	Operating State
0	X (Don't Care)	Z	High-Impedance (Disconnected)
1	0	0	Logic Low (Active Drive)
1	1	1	Logic High (Active Drive)

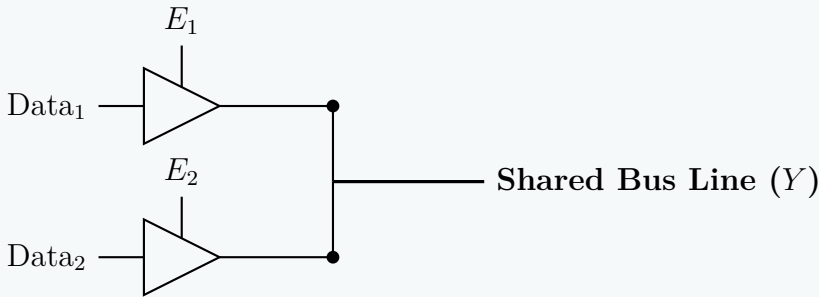
Advantages of Using Three-State Gates

Three-state logic elements provide several critical structural and systemic benefits in modern digital architecture:

- Implementation of Shared Bus Architectures:** The foremost advantage of tri-state gates is their ability to connect multiple gate outputs to a single, common wire or communication bus. Under normal circumstances, connecting two standard gate outputs directly together is forbidden because if one outputs 1 (high voltage) and the other outputs 0 (low voltage), a damaging short-circuit occurs (known as bus contention). By ensuring that the control logic enables **only one tri-state gate at a time** while keeping all others disabled (Z), multiple components can safely share the same channel.
- Bidirectional Data Transmission:** Tri-state gates allow a single physical pin or line to function both as an input and an output. This bidirectional capability is foundational for microprocessor data buses and memory interfaces (RAM/ROM modules), where data must be written to and read from the same location over the same wiring infrastructure.
- Reduction in Routing Overhead and Area Optimization:** Without tri-state logic, multiplexing data from N different registers would require large, multi-stage combinational multiplexing logic networks consisting of dozens of AND-OR gates. Tri-state gates allow multiplexing to occur right at the source directly onto the wire, eliminating excessive gating logic and drastically lowering routing complexity on modern silicon chips.

Visualizing a Tri-State Shared Bus System

The circuit diagram below demonstrates how two independent modules can safely drive a single shared bus line using mutual exclusion on their enable signals ($E_1 \cdot E_2 = 0$):



Q3. What do you mean by Universal gate? Show that both NAND gate and NOR gate are universal gates. (14 marks) [CSE 205 | L-2/T-1 | 2022–23, 2013–14]

↔ Similar: 2019–20

Answer

1. Definition of a Universal Gate

A **Universal Gate** is a logic gate that can be used to implement any Boolean function without requiring the use of any other type of logic gate. In digital logic design, any Boolean expression can be realized using a combination of the three fundamental logic gates: **NOT**, **AND**, and **OR**. Therefore, if we can demonstrate that a specific gate can successfully implement the NOT, AND, and OR operations purely by connecting multiple instances of itself, we prove that it is a universal gate. The two standard universal gates are the **NAND** gate and the **NOR** gate.

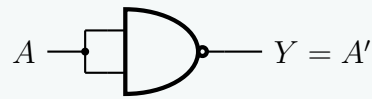
2. Proof: NAND as a Universal Gate

The NAND gate produces the complement of the logical product of its inputs: $Y = (A \cdot B)'$. We can construct the basic gates as follows:

a) Implementing a NOT Gate using a NAND Gate To invert a signal, we apply the same input signal A to all inputs of the NAND gate. According to the *Idempotent Law* ($A \cdot A = A$), the operation simplifies to the logical complement.

$$Y = (A \cdot A)'$$

$$Y = A'$$

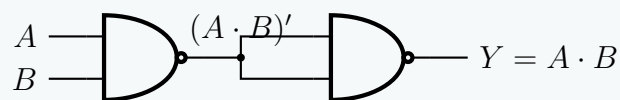


b) Implementing an AND Gate using NAND Gates An AND operation is simply a NAND operation followed by a NOT operation. Since we already know how to create a NOT gate using a NAND gate, we cascade two NAND gates.

$$Y = ((A \cdot B)')$$

Using the *Involution Law* ($X'' = X$):

$$Y = A \cdot B$$



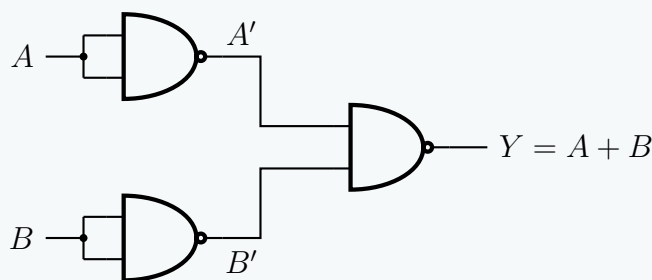
c) Implementing an OR Gate using NAND Gates To perform an OR operation, we first invert both inputs using NAND-based NOT gates, and then feed those inverted signals into a third NAND gate. We apply *De Morgan's Theorem* to prove the equivalence.

$$Y = (A' \cdot B)'$$

Applying De Morgan's Law $(X \cdot Y)' = X' + Y'$:

$$Y = (A')' + (B)'$$

$$Y = A + B$$



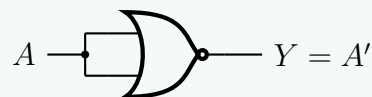
3. Proof: NOR as a Universal Gate

The NOR gate produces the complement of the logical sum of its inputs: $Y = (A + B)'$. The proofs follow a symmetric logic to the NAND gate implementations.

a) Implementing a NOT Gate using a NOR Gate Applying the same input to all terminals of a NOR gate acts as an inverter. According to the *Idempotent Law* ($A + A = A$).

$$Y = (A + A)'$$

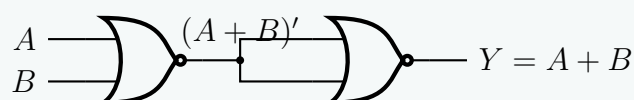
$$Y = A'$$



b) Implementing an OR Gate using NOR Gates An OR operation is a NOR operation followed by an inversion (NOT gate created from a NOR gate).

$$Y = ((A + B)')$$

$$Y = A + B$$



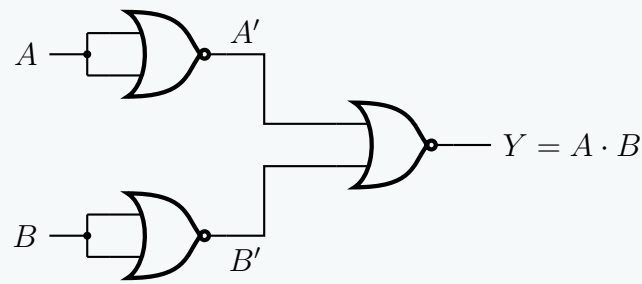
c) Implementing an AND Gate using NOR Gates To perform an AND operation, we invert both inputs using NOR gates, and then supply them to a third NOR gate. We prove this mathematically using *De Morgan's Theorem*.

$$Y = (A' + B)'$$

Applying De Morgan's Law $(X + Y)' = X' \cdot Y'$:

$$Y = (A')' \cdot (B)'$$

$$Y = A \cdot B$$



Conclusion: Because we have successfully mapped the NOT, AND, and OR functional operators entirely to NAND gates and NOR gates, both are formally validated as universal logic gates.

Q4. Show the circuit diagram of the expression $AB + CDE$ using only NAND gates. (10 marks) [CSE 205 | L-2/T-1 | 2021–22]

Answer

1. Mathematical Derivation

To implement a given Sum-of-Products (SOP) expression using only NAND gates, we must transform the expression into a form that exclusively uses the NAND operation $((X \cdot Y)')$. We achieve this by applying double negation and De Morgan's laws.

Let the output function be Y :

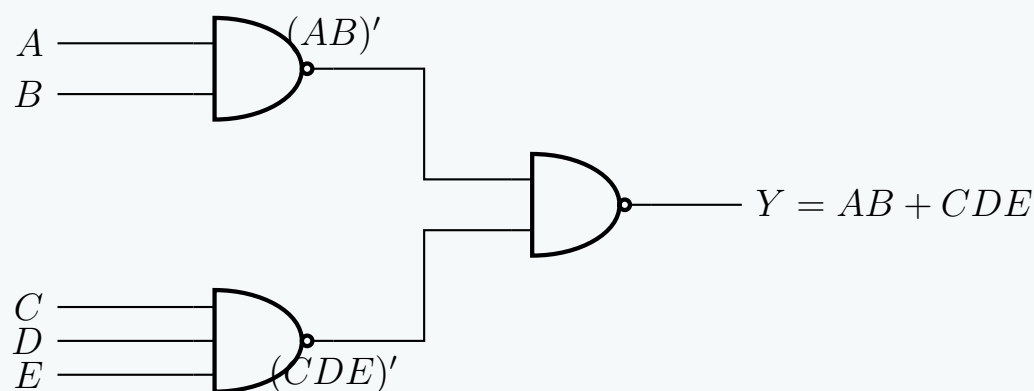
$$\begin{aligned}
 Y &= AB + CDE && \text{(Original SOP expression)} \\
 Y &= ((AB + CDE)')' && \text{(Applying the Involution Law: } X = (X')') \\
 Y &= ((AB)' \cdot (CDE)')' && \text{(Applying De Morgan's Law: } (X + Y)' = X' \cdot Y')
 \end{aligned}$$

Circuit Analysis: The resulting equation $Y = ((AB)' \cdot (CDE)')'$ clearly defines a two-level NAND-NAND implementation:

- **Level 1:** Two NAND gates generate the terms $(AB)'$ and $(CDE)'$. We will use a 2-input NAND gate for AB and a 3-input NAND gate for CDE .
- **Level 2:** A final 2-input NAND gate takes the outputs from Level 1 and produces the target expression.

2. Logic Circuit Diagram

The circuit diagram below demonstrates the realization of the Boolean expression using standard 2-input and 3-input NAND gates.



Note: If the design constraints strictly required only 2-input NAND gates, the 3-input NAND gate $(CDE)'$ would be expanded. We would first compute $(CD)'$, invert it using a NAND gate acting as a NOT gate to get CD , and then feed CD and E into another 2-input NAND gate to produce $(CDE)'$. However, 3-input NAND gates are standard components (e.g., the 7410 IC), so utilizing one directly provides the most optimal and generalized solution.

Q5. A majority circuit is a combinational circuit whose output is equal to 1 if the input variables have more 1's than 0's, and 0 otherwise. Design a 3-input majority circuit by finding the circuit's truth table, simplified Boolean equation, and a logic diagram. Use only NAND gate(s) for your design. (18 marks) [CSE 205 | L-2/T-1 | 2019–20]

Answer

Design of a 3-Input Majority Circuit

A majority circuit output is equal to 1 if the majority of its inputs are 1. For a 3-input combinational circuit with inputs A , B , and C , a majority occurs when two or more inputs are high (≥ 2).

1. Truth Table

Based on the design criteria, the output F is high for the minterms where the binary count of 1s is greater than or equal to 2. This corresponds to the minterms m_3, m_5, m_6 , and m_7 .

Inputs			Output	
A	B	C	F	Minterm
0	0	0	0	m_0
0	0	1	0	m_1
0	1	0	0	m_2
0	1	1	1	$m_3 = \overline{A}BC$
1	0	0	0	m_4
1	0	1	1	$m_5 = A\overline{B}C$
1	1	0	1	$m_6 = AB\overline{C}$
1	1	1	1	$m_7 = ABC$

2. Boolean Simplification (Karnaugh Map)

To find the simplified Sum-of-Products (SOP) expression, we map the 1s into a 3-variable Karnaugh Map:

		BC			
		00	01	11	10
A	0	0	0	1	0
	1	0	1	1	1

From the loops grouped above, we obtain three essential prime implicants:

- Vertical loop grouping m_3 and m_7 : BC
- Horizontal loop grouping m_5 and m_7 : AC
- Bottom-right loop grouping m_6 and m_7 : AB

Thus, the minimal SOP expression is:

$$F = AB + BC + AC$$

3. Conversion to NAND-Only Logic

To implement the circuit using exclusively NAND gates, we apply the Double Negation Law followed by De Morgan’s Theorem to transform the SOP expression into a Product-of-Inversions format:

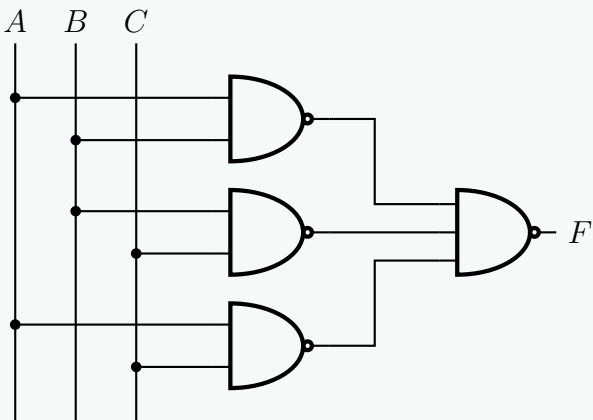
$$\begin{aligned} F &= AB + BC + AC \\ &= \overline{\overline{AB + BC + AC}} && \text{(Double Negation Law)} \\ &= \overline{\overline{AB} \cdot \overline{BC} \cdot \overline{AC}} && \text{(De Morgan's Law)} \end{aligned}$$

This final expression indicates that the circuit can be implemented using:

- (a) Three 2-input NAND gates to generate $\overline{AB}$, $\overline{BC}$, and $\overline{AC}$.
- (b) One 3-input NAND gate to combine the intermediate outputs.

4. Logic Diagram

The professional schematic below illustrates the NAND-only gate realization of the 3-input majority circuit.



Q6. Show that NAND gate is a universal logic gate. Also show that NAND operation does not follow the associative law. **(5+5 marks)**
 [CSE 205 | L-2/T-1 | 2015–16]

Answer

Part 1: Universality of the NAND Gate

A logic gate is considered **universal** if it can be used to implement the three fundamental Boolean operations: **NOT**, **AND**, and **OR**. By demonstrating that a NAND gate can synthesize these three basic gates, we prove its universality.

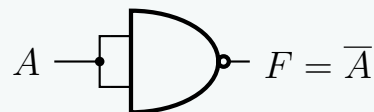
1. Implementing a NOT Gate

A NOT gate (inverter) takes a single input A and produces its complement $\bar{A}$. To achieve this using a 2-input NAND gate, we tie both inputs together.

Boolean Derivation:

$$\begin{aligned} F &= \text{NAND}(A, A) \\ F &= \overline{A \cdot A} && \text{(Definition of NAND)} \\ F &= \bar{A} && \text{(Idempotent Law: } A \cdot A = A) \end{aligned}$$

Logic Diagram:



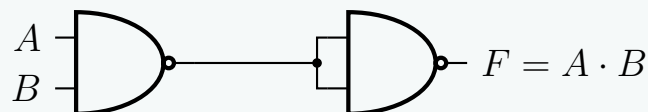
2. Implementing an AND Gate

An AND gate performs the Boolean product $A \cdot B$. Since a NAND gate produces $\overline{A \cdot B}$, we simply need to invert its output using a second NAND gate configured as a NOT gate.

Boolean Derivation:

$$\begin{aligned} F &= \text{NOT}(\text{NAND}(A, B)) \\ F &= \overline{\overline{A \cdot B}} && \text{(Definition of NAND)} \\ F &= A \cdot B && \text{(Double Negation Law)} \end{aligned}$$

Logic Diagram:



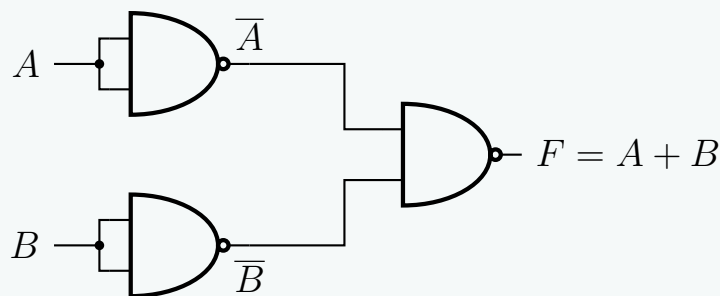
3. Implementing an OR Gate

An OR gate performs the Boolean sum $A + B$. We can implement this by first inverting both inputs using NAND gates, and then passing those inverted signals into a third NAND gate.

Boolean Derivation:

$$\begin{aligned} F &= \text{NAND}(\text{NOT}(A), \text{NOT}(B)) \\ F &= \text{NAND}(\bar{A}, \bar{B}) && \text{(From NOT implementation)} \\ F &= \overline{\bar{A} \cdot \bar{B}} && \text{(Definition of NAND)} \\ F &= \overline{\bar{A}} + \overline{\bar{B}} && \text{(De Morgan's Law)} \\ F &= A + B && \text{(Double Negation Law)} \end{aligned}$$

Logic Diagram:



Because the NAND gate can implement NOT, AND, and OR functions, it is formally proven to be a **universal gate**.

Part 2: Non-Associativity of the NAND Operation

The associative law states that for a given binary operator $\uparrow$, the grouping of operands does not affect the result:

$$(A \uparrow B) \uparrow C = A \uparrow (B \uparrow C)$$

Let the operator $\uparrow$ denote the NAND operation, where $A \uparrow B = \overline{A \cdot B}$. We will evaluate the Left-Hand Side (LHS) and Right-Hand Side (RHS) of the associative equation independently.

Evaluating the Left-Hand Side (LHS):

$$\begin{aligned}
 \text{LHS} &= (A \uparrow B) \uparrow C \\
 &= (\overline{A \cdot B}) \uparrow C && \text{(Definition of NAND)} \\
 &= \overline{(\overline{A \cdot B}) \cdot C} && \text{(Definition of NAND)} \\
 &= \overline{(\overline{A \cdot B})} + \overline{C} && \text{(De Morgan's Law)} \\
 &= (A \cdot B) + \overline{C} && \text{(Double Negation Law)}
 \end{aligned}$$

Evaluating the Right-Hand Side (RHS):

$$\begin{aligned}
 \text{RHS} &= A \uparrow (B \uparrow C) \\
 &= A \uparrow (\overline{B \cdot C}) && \text{(Definition of NAND)} \\
 &= \overline{A \cdot (\overline{B \cdot C})} && \text{(Definition of NAND)} \\
 &= \overline{A} + \overline{(\overline{B \cdot C})} && \text{(De Morgan's Law)} \\
 &= \overline{A} + (B \cdot C) && \text{(Double Negation Law)}
 \end{aligned}$$

Comparing the simplified expressions, we clearly see that:

$$(A \cdot B) + \overline{C} \neq \overline{A} + (B \cdot C)$$

Counterexample via Truth Values: To definitively prove the inequality, we can assign an arbitrary set of values, such as $A = 0$, $B = 0$, and $C = 1$.

- **LHS:** $((0 \cdot 0) + \overline{1}) = (0 + 0) = 0$
- **RHS:** $(\overline{0} + (0 \cdot 1)) = (1 + 0) = 1$

Since $0 \neq 1$, the LHS and RHS do not yield the same result for the same input combination. Therefore, **the NAND operation does not follow the associative law.**

Q7. Define: Odd Function, Even Function, Don't-care conditions. (5 marks)

[CSE 205 | L-2/T-1 | 2013–14]

Answer

Definitions of Logic Function Properties

1. Odd Function

In digital logic, an **odd function** is a Boolean function that evaluates to logic 1 if and only if an **odd number** of its input variables are equal to logic 1.

- **Operation:** The multiple-input exclusive-OR (XOR) operation inherently implements an odd function.
- **Example (3-Variable):** $F_{\text{odd}}(A, B, C) = A \oplus B \oplus C = \sum m(1, 2, 4, 7)$

2. Even Function

An **even function** is a Boolean function that evaluates to logic 1 if and only if an **even number** of its input variables are equal to logic 1. In binary logic, zero is considered an even number, so the function also outputs 1 when all inputs are 0.

- **Operation:** For a 3-variable system, the complement of the XOR function (which is the exclusive-NOR, or XNOR, evaluated sequentially) yields an even function.
- **Example (3-Variable):** $F_{\text{even}}(A, B, C) = (A \oplus B \oplus C)' = \sum m(0, 3, 5, 6)$

Truth Table Comparison (3 Variables):

Inputs			Count	Odd Function	Even Function	
A	B	C	of 1s	$F = A \oplus B \oplus C$	$F = (A \oplus B \oplus C)'$	Minterm
0	0	0	0	0	1	m_0
0	0	1	1	1	0	m_1
0	1	0	1	1	0	m_2
0	1	1	2	0	1	m_3
1	0	0	1	1	0	m_4
1	0	1	2	0	1	m_5
1	1	0	2	0	1	m_6
1	1	1	3	1	0	m_7

3. Don't-Care Conditions

In certain digital systems, specific input combinations may either never physically occur (e.g., states 1010 to 1111 in a BCD, Binary-Coded Decimal, system) or the system's output for those specific input combinations does not affect the overall circuit behavior. The unassigned output states for these combinations are defined as **don't-care conditions**.

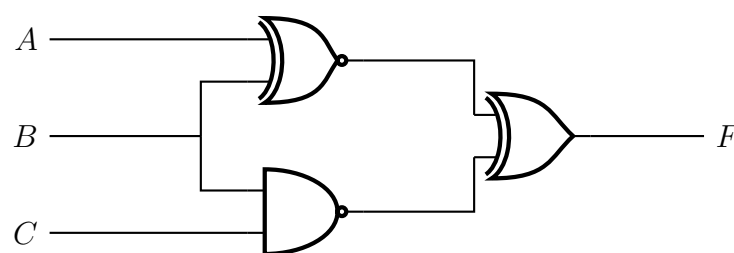
- **Notation:** They are typically denoted by **X**, **d**, or Φ in truth tables and Boolean expressions. For example: $F(A, B, C) = \sum m(0, 1) + \sum d(2, 3)$.
- **Application in Minimization:** During logic simplification (e.g., using Karnaugh Maps or the Quine-McCluskey algorithm), a designer can arbitrarily treat a don't-care condition as either a 1 or a 0. The strategic choice is to assign them the value that leads to the largest possible groupings (implicants), thereby yielding the most minimized and cost-effective logic circuit.

Example of K-Map Optimization using Don't-Cares: Consider $F(A, B, C) = \sum m(0, 1) + \sum d(2, 3)$. Without don't-cares, m_0 and m_1 group to form $\overline{A}B$. By treating the don't-cares (d_2, d_3) as 1s, we can form a quad, simplifying the expression significantly to $F = \overline{A}$.

		BC			
		00	01	11	10
A	0	1	1	-	-
	1	0	0	0	0

Circuit Analysis

Q1. Analyze the following circuit and determine the output F as sum of minterms and product of maxterms.



(9 marks)

[CSE 205 | L-2/T-1 | 2011–12]

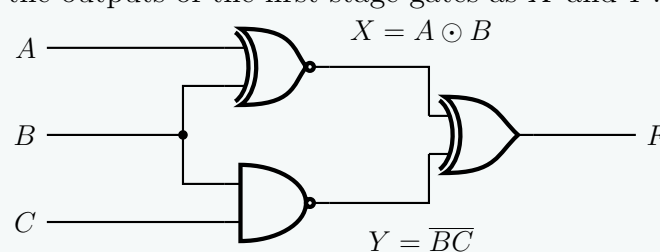
Answer

Circuit Analysis: Sum of Minterms and Product of Maxterms

To determine the output F in its canonical forms, we will first extract the Boolean expression from the logic circuit, simplify it algebraically, and verify the results using a Truth Table.

1. Boolean Expression Extraction

Let us define the intermediate signals at the outputs of the first-stage gates as X and Y .



From the circuit diagram, we can write the equations for the intermediate nodes and the final output:

- Top Gate (XNOR): $X = A \odot B$
- Bottom Gate (NAND): $Y = \overline{BC}$
- Output Gate (XOR): $F = X \oplus Y$

2. Algebraic Derivation

Substitute X and Y into the XOR expression. By definition, $X \oplus Y = X\overline{Y} + \overline{X}Y$.

First, find the complements $\bar{X}$ and $\bar{Y}$:

$$\bar{X} = \overline{A \odot B} = A \oplus B = A\bar{B} + \bar{A}B$$

$$\bar{Y} = \overline{\overline{BC}} = BC$$

(Double Negation Law)

Now, substitute X , $\bar{X}$, Y , and $\bar{Y}$ into the expression for F :

$$F = (A \odot B)(BC) + (A \oplus B)(\overline{BC})$$

$$F = (AB + \bar{A}\bar{B})(BC) + (A\bar{B} + \bar{A}B)(\bar{B} + \bar{C})$$

(De Morgan's Law on $\overline{BC}$)

Expand both terms using the Distributive Law:

$$\text{Term 1: } (AB)(BC) + (\bar{A}\bar{B})(BC)$$

$$= ABC + \bar{A}(\bar{B}B)C$$

(Idempotent Law: $BB = B$)

$$= ABC + 0$$

(Complement Law: $\bar{B}B = 0$)

$$= ABC$$

$$\text{Term 2: } A\bar{B}\bar{B} + A\bar{B}\bar{C} + \bar{A}B\bar{B} + \bar{A}B\bar{C}$$

$$= A\bar{B} + A\bar{B}\bar{C} + 0 + \bar{A}B\bar{C}$$

(Idempotent $\bar{B}\bar{B} = \bar{B}$, Complement $B\bar{B} = 0$)

$$= A\bar{B}(1 + \bar{C}) + \bar{A}B\bar{C}$$

(Factoring $A\bar{B}$)

$$= A\bar{B}(1) + \bar{A}B\bar{C}$$

(Identity Law: $1 + X = 1$)

$$= A\bar{B} + \bar{A}B\bar{C}$$

Combine the simplified terms:

$$F = ABC + A\bar{B} + \bar{A}B\bar{C}$$

To express F as a sum of minterms, we expand the incomplete term $(A\bar{B})$ by multiplying it by $(C + \bar{C})$:

$$F = ABC + A\bar{B}(C + \bar{C}) + \bar{A}B\bar{C}$$

$$F = ABC + A\bar{B}C + A\bar{B}\bar{C} + \bar{A}B\bar{C}$$

Translating the boolean products to their binary equivalents (where uncomplemented = 1, complemented = 0):

- $ABC \rightarrow 111 = m_7$
- $A\bar{B}C \rightarrow 101 = m_5$
- $A\bar{B}\bar{C} \rightarrow 100 = m_4$
- $\bar{A}B\bar{C} \rightarrow 010 = m_2$

3. Truth Table Verification

We construct a truth table to double-check the logic levels for all 8 combinations of A, B, C .

Inputs			XNOR	NAND	Output	Canonical Forms	
A	B	C	$X = A \odot B$	$Y = \overline{BC}$	$F = X \oplus Y$	Minterm	Maxterm
0	0	0	1	1	0		M_0
0	0	1	1	1	0		M_1
0	1	0	0	1	1	m_2	
0	1	1	0	0	0		M_3
1	0	0	0	1	1	m_4	
1	0	1	0	1	1	m_5	
1	1	0	1	1	0		M_6
1	1	1	1	0	1	m_7	

4. Final Answer

From both the algebraic derivation and the truth table, the function equals 1 for minterms 2, 4, 5, and 7, and equals 0 for maxterms 0, 1, 3, and 6.

Sum of Minterms (Σm):

$$F(A, B, C) = \sum m(2, 4, 5, 7)$$

Product of Maxterms (ΠM):

$$F(A, B, C) = \prod M(0, 1, 3, 6)$$

Two-Level & Multi-Level Logic Implementation

Q1. Simplify the following function and draw the circuit using only NAND gates:

$$F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$$

(16 marks)

[CSE 205 | L-2/T-1 | 2023–24]

Answer

1. Karnaugh Map & Simplification

To simplify the Boolean function $F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$, we map the minterms onto a 4-variable Karnaugh Map.

		yz			
		00	01	11	10
wx	00	1	1		1
	01	1	1		1
	11	1	1		1
	10	1	1		

From the Karnaugh Map, we identify three essential prime implicants by forming the largest possible groups of 1s:

- **Group 1** (Red/Largest block): An octet spanning columns 00 and 01. The variables w, x , and z change state, leaving only $y = 0$. This yields the term y' .
- **Group 2** (Green/Top edges): A quad spanning rows 00, 01 and columns 00, 10. The variables x and y change state, leaving $w = 0$ and $z = 0$. This yields the term $w'z'$.
- **Group 3** (Blue/Middle edges): A quad spanning rows 01, 11 and columns 00, 10. The variables w and y change state, leaving $x = 1$ and $z = 0$. This yields the term xz' .

The minimized Sum of Products (SOP) expression is:

$$F = y' + w'z' + xz'$$

2. Algebraic Conversion to NAND-NAND Form

To implement the circuit using **only** NAND gates, we must convert the SOP expression by applying Boolean algebra rules—specifically the Involution Law (Double Negation) and De Morgan's Laws.

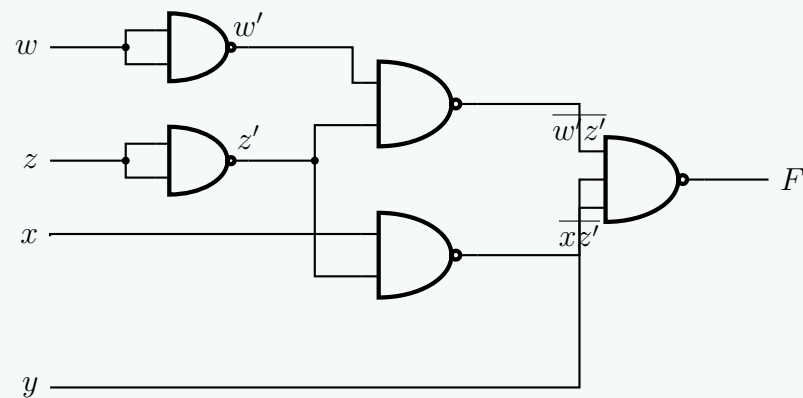
$$\begin{aligned}
 F &= y' + w'z' + xz' \\
 F &= \overline{\overline{y' + w'z' + xz'}} && \text{(Involution Law: } A = \overline{\overline{A}}\text{)} \\
 F &= \overline{\overline{y'} \cdot \overline{w'z'} \cdot \overline{xz'}} && \text{(De Morgan's Law: } \overline{A + B + C} = \overline{A} \cdot \overline{B} \cdot \overline{C}\text{)} \\
 F &= \overline{y \cdot w'z' \cdot xz'} && \text{(Involution Law: } \overline{\overline{y}} = y\text{)}
 \end{aligned}$$

This final expression indicates that F can be synthesized using:

- 2-input NAND gates with tied inputs to act as inverters for $w \rightarrow w'$ and $z \rightarrow z'$.
- 2-input NAND gates to generate the partial products $\overline{w'z'}$ and $\overline{xz'}$.
- A single 3-input NAND gate to combine y and the two partial products.

3. Logic Circuit Implementation (NAND Only)

Below is the structural circuit diagram derived directly from the factored NAND expression. Note how the uncomplemented input y is fed directly into the final stage, gracefully saving an inverter.



Q2. Implement $F(x, y, z) = x'y'z' + xyz'$ using two-level forms of logic by using NAND-AND gates. Assume that the complement of x , y and z are readily available. **(14 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Answer

1. Theoretical Basis for NAND-AND Implementation

A two-level **NAND-AND** logic circuit generates a Product of Sums (POS) conceptually, but more accurately, it implements the complement of a Sum of Products (SOP) expression. Let the target function be F . If we determine the simplified SOP expression for its complement, F' , such that $F' = P_1 + P_2 + \dots + P_k$, we can recover F by complementing F' :

$$F = (F')' = \overline{P_1 + P_2 + \dots + P_k} = \overline{P_1} \cdot \overline{P_2} \dots \overline{P_k}$$

By De Morgan's Law, this resulting expression is a product (AND operation at Level 2) of inverted product terms (NAND operations at Level 1).

2. Algebraic Derivation of F'

We are given the Boolean function $F(x, y, z) = x'y'z' + xyz'$. We first find and simplify its complement F' using Boolean algebra:

$$\begin{aligned} F' &= \overline{x'y'z' + xyz'} \\ &= \overline{(x'y'z')} \cdot \overline{(xyz')} && \text{(De Morgan's Law: } \overline{A + B} = \overline{A} \cdot \overline{B} \text{)} \\ &= (x + y + z) \cdot (x' + y' + z') && \text{(De Morgan's Law: } \overline{ABC} = \overline{A} + \overline{B} + \overline{C} \text{)} \\ &= (x + y + z) \cdot (x' + y' + z) && \text{(Correction: Complement of } z' \text{ is } z \text{)} \\ &= xx' + xy' + xz + yx' + yy' + yz + zx' + zy' + zz && \text{(Distributive Law)} \\ &= 0 + xy' + xz + x'y + 0 + yz + x'z + y'z + z && \text{(Complementarity: } AA' = 0 \text{, Idempotent: } AA = A \text{)} \\ &= xy' + x'y + xz + yz + x'z + y'z + z && \text{(Identity Law: } A + 0 = A \text{)} \\ &= xy' + x'y + z(x + y + x' + y' + 1) && \text{(Distributive Law / Factoring out } z \text{)} \\ &= xy' + x'y + z(1) && \text{(Dominance Law: } A + 1 = 1 \text{)} \\ &= z + x'y + xy' && \text{(Identity Law: } A \cdot 1 = A \text{)} \end{aligned}$$

The simplified Sum of Products for the complement is $F' = z + x'y + xy'$.

3. Conversion to NAND-AND Form

Now, we complement F' to obtain F in the desired NAND-AND structure:

$$\begin{aligned} F &= (F')' \\ &= \overline{z + x'y + xy'} \\ &= \overline{z} \cdot \overline{x'y} \cdot \overline{xy'} && \text{(De Morgan's Law)} \end{aligned}$$

Given the problem statement that the complement of z (z') is readily available, we can substitute $\overline{z} = z'$ directly. This prevents the need for a 1-input NAND gate to act as an inverter. The final expression perfectly matches a two-level NAND-AND architecture:

$$F = z' \cdot \overline{x'y} \cdot \overline{xy'}$$

- **Level 1 (NAND):** Two 2-input NAND gates generating $\overline{x'y}$ and $\overline{xy'}$.
- **Level 2 (AND):** One 3-input AND gate multiplying z' , $\overline{x'y}$, and $\overline{xy'}$.

4. Logic Circuit Diagram

Q3. Express $f(A, B, C, D) = \Sigma(0, 4, 8, 9, 10, 11, 12, 14)$ with the two-level forms of logic “NAND-AND” and “OR-NAND”. (6 marks)
[CSE 205 | L-2/T-1 | 2018–19]

Answer

Two-Level Logic Design: NAND-AND and OR-NAND Forms

To implement a logic function in specific two-level forms, we must first derive the simplified Boolean expressions for the function f and its complement f' . We achieve this using Karnaugh Maps.

The given Boolean function is:

$$f(A, B, C, D) = \Sigma(0, 4, 8, 9, 10, 11, 12, 14)$$

The unlisted minterms form the complement function f' :

$$f'(A, B, C, D) = \Sigma(1, 2, 3, 5, 6, 7, 13, 15)$$

—

1. Karnaugh Map Simplification

Minimizing f (Sum of Products for f): We plot the minterms of f on a 4-variable K-Map to find the minimal Sum of Products (SOP).

		CD			
		00	01	11	10
AB	00	1	0	0	0
	01	1	0	0	0
	11	1	0	0	1
	10	1	1	1	1

From the groupings above, we extract three essential prime implicants:

- Left column group (0, 4, 12, 8): $C'D'$
- Bottom row group (8, 9, 11, 10): AB'
- Wrapped edge quad (12, 14, 8, 10): AD'

Thus, the simplified SOP expression for f is:

$$f = C'D' + AB' + AD' \tag{1}$$

Minimizing f' (Sum of Products for f'): We plot the minterms of f' to find its minimal SOP.

		CD			
		00	01	11	10
AB	00	0	1	1	1
	01	0	1	1	1
	11	0	1	1	0
	10	0	0	0	0

From the K-Map for f' , we obtain:

- Group (1, 3, 5, 7): $A'D$
- Group (2, 3, 6, 7): $A'C$
- Group (5, 7, 13, 15): BD

Thus, the simplified SOP expression for f' is:

$$f' = A'D + A'C + BD \quad (2)$$

—

2. NAND-AND Logic Form

The **NAND-AND** form requires the logic equation to be a product of inverted terms (First Level: NAND, Second Level: AND). We derive this naturally from the SOP of the complemented function f' .

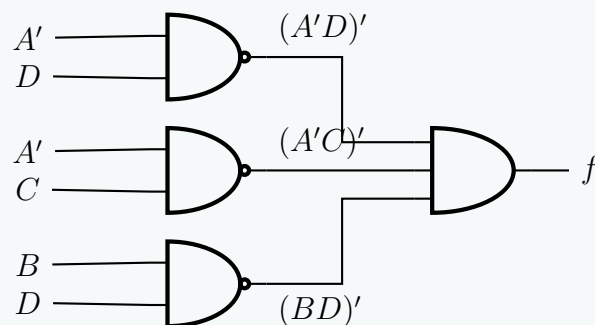
By taking the complement of both sides of Equation (2):

$$\begin{aligned} f &= (f')' \\ f &= (A'D + A'C + BD)' \end{aligned}$$

Applying **De Morgan's Law** to distribute the inversion over the OR terms:

$$f = (A'D)' \cdot (A'C)' \cdot (BD)'$$

This equation maps directly to a two-level NAND-AND circuit:



—

3. OR-NAND Logic Form

The **OR-NAND** form requires the logic equation to be an inverted product of sums (First Level: OR, Second Level: NAND). This is obtained by expressing f' as a Product of Sums (POS), and then taking its complement.

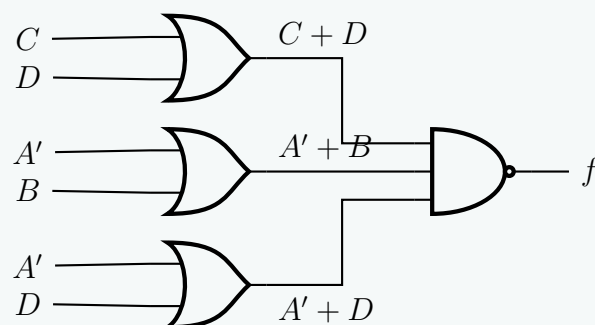
The POS of f' is the logical complement of the SOP of f (Equation 1). Applying **De Morgan's Law** to Equation (1):

$$\begin{aligned} f' &= (C'D' + AB' + AD')' \\ f' &= (C + D) \cdot (A' + B) \cdot (A' + D) \end{aligned}$$

To find f , we complement f' again:

$$\begin{aligned} f &= (f')' \\ f &= ((C + D) \cdot (A' + B) \cdot (A' + D))' \end{aligned}$$

This equation translates perfectly into a two-level OR-NAND circuit:



Q4. Implement the function $F(w, x, y, z) = w'x' + w'x'z + w'yz'$ using two-level forms of logic by

- NOR gates only
- AND-NOR
- NAND gates only
- OR-NAND

(20 marks)

[CSE 205 | L-2/T-1 | 2014–15]

Answer

Simplification and Two-Level Logic Implementations

First, we simplify the given Boolean function $F(w, x, y, z)$ to its minimal Sum of Products (SOP) form. This will serve as the foundation for our derivations.

$$\begin{aligned}
 F &= w'x' + w'x'z + w'yz' \\
 F &= w'x'(1 + z) + w'yz' && \text{(Distributive Law)} \\
 F &= w'x'(1) + w'yz' && \text{(Null Element Law: } 1 + X = 1\text{)} \\
 F &= w'x' + w'yz' && \text{(Identity Law: } X \cdot 1 = X\text{)}
 \end{aligned}$$

1. NOR Gates Only (NOR-NOR Form)

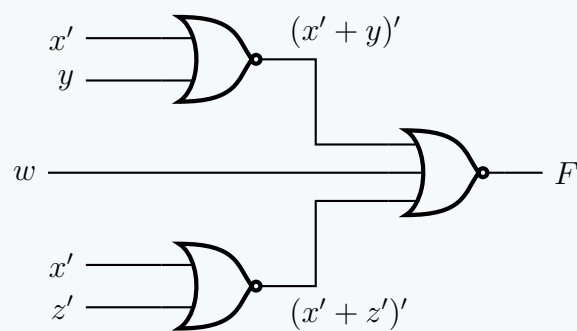
A two-level NOR-NOR circuit requires the function to be in the form of a complemented sum of negated sums. This is derived from the Product of Sums (POS) form of F .

Step 1: Find the POS of F

$$\begin{aligned}
 F &= w'x' + w'yz' \\
 F &= w'(x' + yz') && \text{(Distributive Law)} \\
 F &= w'(x' + y)(x' + z') && \text{(Distributive Law: } A + BC = (A + B)(A + C)\text{)}
 \end{aligned}$$

Step 2: Convert to NOR-NOR logic

$$\begin{aligned}
 F &= \overline{\overline{w'(x' + y)(x' + z')}} && \text{(Double Negation Law)} \\
 F &= \overline{w + (x' + y)' + (x' + z')'} && \text{(De Morgan's Law)}
 \end{aligned}$$



2. AND-NOR Form

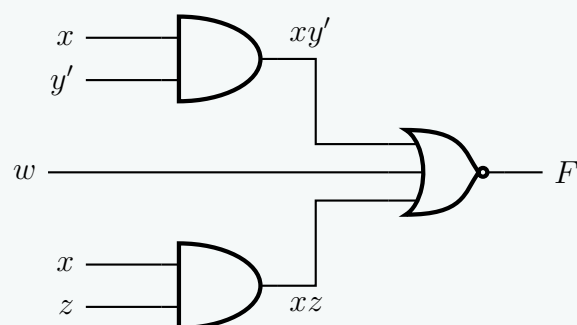
A two-level AND-NOR circuit is structurally equivalent to the complement of the SOP of F' . Therefore, we first find the SOP of the complemented function F' .

Step 1: Find the SOP of F'

$$\begin{aligned}
 F' &= (w'(x' + y)(x' + z'))' && \text{(Complementing the POS of } F\text{)} \\
 F' &= w + (x' + y)' + (x' + z')' && \text{(De Morgan's Law)} \\
 F' &= w + xy' + xz && \text{(De Morgan's Law and Double Negation)}
 \end{aligned}$$

Step 2: Express F as an AND-NOR equation

$$\begin{aligned}
 F &= (F')' \\
 F &= \overline{w + xy' + xz} && \text{(Substitution)}
 \end{aligned}$$



3. NAND Gates Only (NAND-NAND Form)

A two-level NAND-NAND circuit translates directly from the SOP form of F .

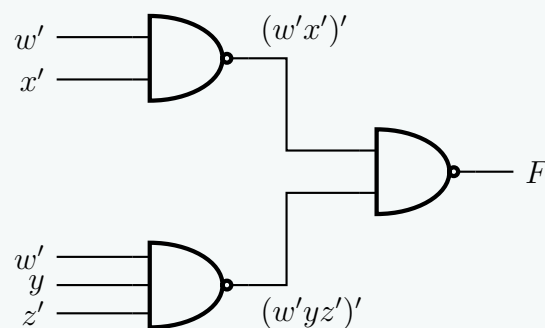
Step 1: Start with SOP of F

$$F = w'x' + w'yz'$$

Step 2: Convert to NAND-NAND logic

$$F = \overline{\overline{w'x' + w'yz'}} \quad (\text{Double Negation Law})$$

$$F = \overline{(w'x')' \cdot (w'yz')'} \quad (\text{De Morgan's Law})$$



4. OR-NAND Form

A two-level OR-NAND circuit corresponds to the inversion of the POS form of F' . We must derive the POS of the complement function.

Step 1: Find the POS of F' Using the SOP of F' determined previously ($F' = w + xy' + xz$):

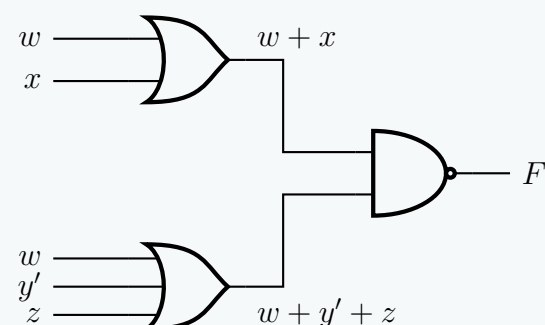
$$F' = w + x(y' + z) \quad (\text{Distributive Law})$$

$$F' = (w + x)(w + y' + z) \quad (\text{Distributive Law: } A + BC = (A + B)(A + C))$$

Step 2: Express F as an OR-NAND equation

$$F = (F')'$$

$$F = \overline{(w + x)(w + y' + z)} \quad (\text{Substitution})$$



Q5. Show the truth table for the following function:

$$F(x, y, z) = (x + y + z)(x + y + z')(x + y' + z)(x' + y + z)$$

(8 marks)

[CSE 205 | L-2/T-1 | 2014–15]

Answer

Truth Table Derivation

The given Boolean function is presented in its canonical **Product of Sums (POS)** form:

$$F(x, y, z) = (x + y + z)(x + y + z')(x + y' + z)(x' + y + z)$$

In a POS expression, each sum term is called a **maxterm** (M_i). A maxterm evaluates to logic 0 for exactly one combination of the input variables. To determine the binary value of each maxterm, we map uncomplemented variables to 0 and complemented variables to 1.

Let us evaluate each term to identify its corresponding row in the truth table where the function outputs a 0:

$$(x + y + z) \implies 000_2 \implies M_0$$

$$(x + y + z') \implies 001_2 \implies M_1$$

$$(x + y' + z) \implies 010_2 \implies M_2$$

$$(x' + y + z) \implies 100_2 \implies M_4$$

Thus, the function can be compactly written using the Pi-notation for maxterms:

$$F(x, y, z) = \Pi(0, 1, 2, 4)$$

This analysis establishes that the function F will output **0** for the decimal input combinations 0, 1, 2, and 4. For all remaining input combinations (3, 5, 6, and 7), the function will output **1**.

Truth Table

Constructing the table based on our derived maxterms:

Decimal (i)	x	y	z	$F(x, y, z)$
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

- Q6.** Given function $F = A'B + BC + AC$:
- (a) Implement F using AND, OR and NOT gates.
 - (b) Minimize F in SOP form using K-Map.
 - (c) Implement the minimized F using only NAND gates.

(15 marks)

[CSE 205 | L-2/T-1 | 2013–14]

Answer

Logic Circuit Design and Minimization

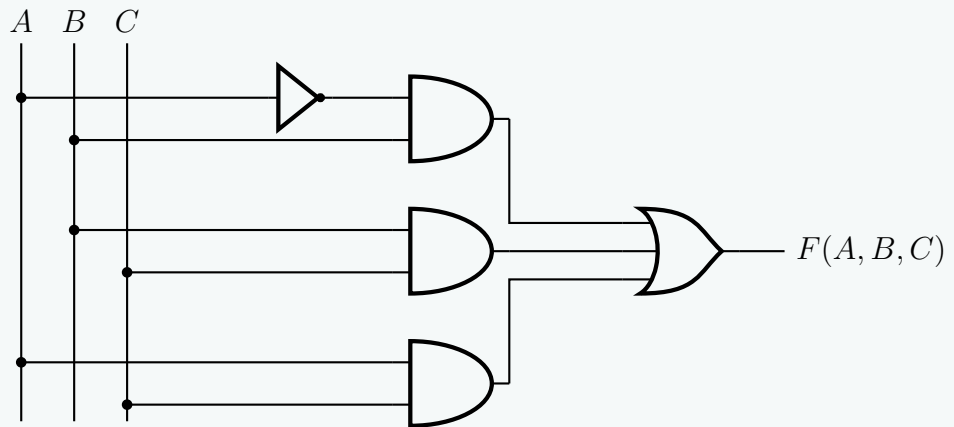
Given the Boolean function:

$$F(A, B, C) = A'B + BC + AC$$

1. Implementation using AND, OR, and NOT gates

To implement the unminimized function F , we analyze its structure:

- One **NOT** gate is required to generate the complement of A (A').
- Three 2-input **AND** gates are required to generate the product terms $A'B$, BC , and AC .
- One 3-input **OR** gate is required to sum the product terms together.



2. K-Map Minimization in SOP Form

To plot the function on a Karnaugh Map, we first expand the terms into canonical minterms (Sum of Minterms) by introducing missing variables:

$$\begin{aligned} A'B &= A'B(C + C') = A'BC + A'BC' \implies m_3 + m_2 \\ BC &= (A + A')BC = ABC + A'BC \implies m_7 + m_3 \\ AC &= A(B + B')C = ABC + AB'C \implies m_7 + m_5 \end{aligned}$$

Thus, the canonical function is $F(A, B, C) = \Sigma m(2, 3, 5, 7)$. Let us map these 1s onto a 3-variable K-Map.

		BC			
		00	01	11	10
A	0	0	0	1	1
	1	0	1	1	0

Group Extraction:

- **Group 1** (Cells 2, 3): This horizontal pair yields the term **A'B**.
- **Group 2** (Cells 5, 7): This horizontal pair yields the term **AC**.

Notice that cells 3 and 7 form a vertical pair (BC), but this is a *redundant group* since all its 1s are already covered by essential prime implicants. This visual redundancy perfectly illustrates the **Consensus Theorem**.

The minimized SOP function is:

$$F_{\min} = A'B + AC$$

3. Implementation of Minimized F using only NAND gates

To construct the circuit using strictly NAND gates, we must convert the minimized SOP expression into a NAND-NAND form. We achieve this by applying the **Double Negation Law** and **De Morgan's Laws**:

$$F_{\min} = A'B + AC$$

$$F_{\min} = \overline{\overline{A'B + AC}}$$

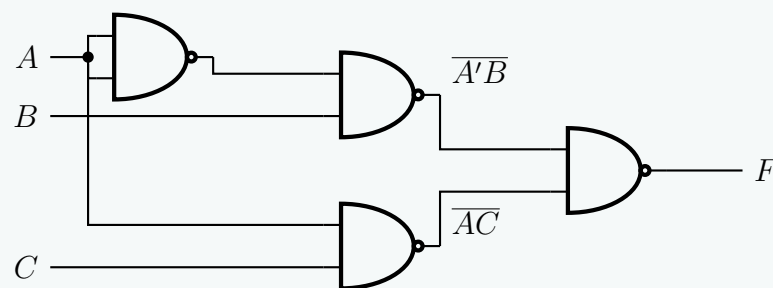
(Double Negation Law)

$$F_{\min} = \overline{(A'B) \cdot (AC)}$$

(De Morgan's Law: $\overline{X + Y} = \overline{X} \cdot \overline{Y}$)

This equation dictates a two-level NAND structure:

- The first level generates the terms $\overline{A'B}$ and $\overline{AC}$. Note that A' must also be created by tying the inputs of a NAND gate together (since $\overline{A \cdot A} = A'$).
- The second level NAND gate takes these inverted products and produces the final output F .



Q7. Given the Boolean function $F = xy + x'y' + y'z$, implement it with (i) AND, OR and NOT gates, and (ii) only NAND gates. **(5+5 marks)** [CSE 205 | L-2/T-1 | 2012–13]

Answer

Logic Circuit Implementations

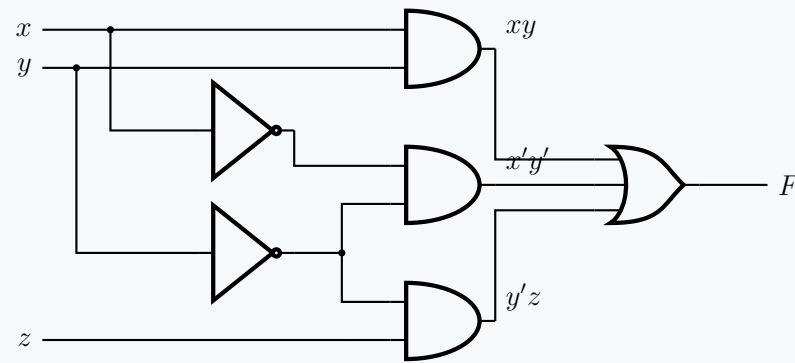
We are given the Boolean function in Sum of Products (SOP) form:

$$F(x, y, z) = xy + x'y' + y'z$$

(i) Implementation using AND, OR, and NOT gates

To implement F directly using standard logic gates, we require:

- **NOT Gates:** To generate the complements x' and y' .
- **AND Gates:** Three 2-input AND gates to compute the product terms (xy) , $(x'y')$, and $(y'z)$.
- **OR Gate:** One 3-input OR gate to sum the product terms together.



(ii) Implementation using ONLY NAND gates

To implement the circuit exclusively with NAND gates, we must convert the SOP expression into a **NAND-NAND** structure. We achieve this by applying the *Double Negation Law* and *De Morgan's Laws*:

$$F = xy + x'y' + y'z$$

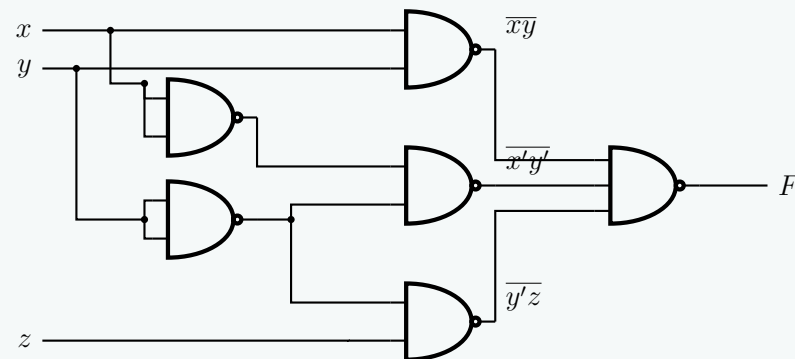
$$F = \overline{\overline{xy + x'y' + y'z}}$$

(Double Negation Law)

$$F = \overline{(xy) \cdot (x'y') \cdot (y'z)}$$

(De Morgan's Law: $\overline{A + B + C} = \overline{A} \cdot \overline{B} \cdot \overline{C}$)

This equation dictates a two-level NAND structure. Additionally, the complements x' and y' must be generated using NAND gates with their inputs tied together (since $\overline{A \cdot A} = A'$).



Q8. Using the two-level forms of logic (i) NOR-OR, and (ii) OR-NAND, implement the following Boolean function F :

$$F(A, B, C, D) = \Sigma(0, 4, 8, 9, 10, 11, 12, 14)$$

Assume that both the normal and complement inputs are available. **(6+6 marks)**

[CSE 205 | L-2/T-1 | 2011–12]

Answer

Step 1: K-Map Minimization in SOP Form

Before implementing the function in the required two-level forms, we must first minimize the Boolean function $F(A, B, C, D) = \Sigma(0, 4, 8, 9, 10, 11, 12, 14)$ into its minimal Sum of Products (SOP) form. We map the given minterms onto a 4-variable Karnaugh Map:

		CD			
		00	01	11	10
AB	00	1	0	0	0
	01	1	0	0	0
	11	1	0	0	1
	10	1	1	1	1

Group Extraction:

- **Group 1 (Vertical Col 00):** Minterms (0, 4, 12, 8). This eliminates A and B , yielding the prime implicant $C'D'$.
- **Group 2 (Horizontal Row 10):** Minterms (8, 9, 11, 10). This eliminates C and D , yielding the prime implicant AB' .
- **Group 3 (Edges):** Minterms (12, 14, 8, 10). This forms a 2×2 block wrapping around the horizontal edges, eliminating B and C , yielding the prime implicant AD' .

All three groups are essential prime implicants. The minimized SOP expression is:

$$F = AB' + AD' + C'D' \quad (1)$$

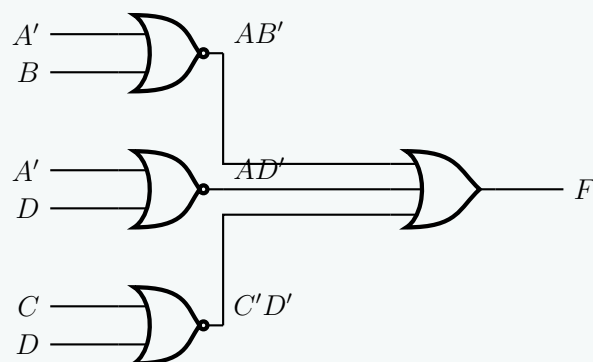
Step 2: Implementation (i) NOR-OR Logic

A **NOR-OR** implementation produces a Sum of Products (SOP) expression, but requires specific algebraic manipulation of the inputs. We analyze a single NOR gate: it computes the complement of an OR sum, which maps to a product term via De Morgan's Law ($\overline{X + Y} = X'Y'$).

To generate our required product terms using NOR gates, we complement the literals of each product term and feed them into the NOR gates:

$$\begin{aligned} AB' &= \overline{A' + B} && \text{(NOR with inputs } A', B) \\ AD' &= \overline{A' + D} && \text{(NOR with inputs } A', D) \\ C'D' &= \overline{C + D} && \text{(NOR with inputs } C, D) \end{aligned}$$

We then sum these terms together using a second-level OR gate. Since both normal and complement inputs are available, we construct the circuit directly:



Step 3: Implementation (ii) OR-NAND Logic

An **OR-NAND** structure is logically equivalent to the SOP form. To convert the SOP expression into an OR-NAND format, we apply the **Double Negation Law** followed by **De Morgan's Law** on the entire function:

$$\begin{aligned} F &= AB' + AD' + C'D' \\ F &= \overline{\overline{AB' + AD' + C'D'}} && \text{(Double Negation Law)} \\ F &= \overline{(\overline{AB'}) \cdot (\overline{AD'}) \cdot (\overline{C'D'})} && \text{(De Morgan's Law)} \end{aligned}$$

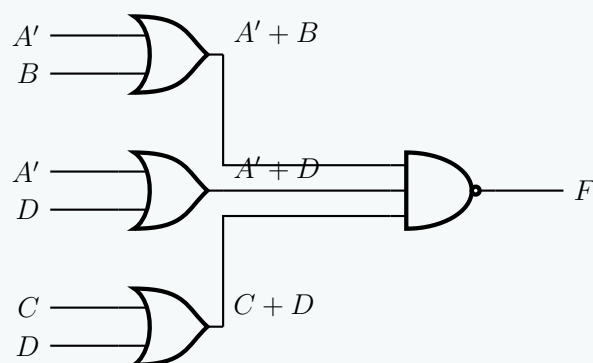
Next, we expand the inverted product terms internally using De Morgan's Law ($\overline{XY} = X' + Y'$):

$$\begin{aligned} \overline{AB'} &= A' + B && \text{(OR with inputs } A', B) \\ \overline{AD'} &= A' + D && \text{(OR with inputs } A', D) \\ \overline{C'D'} &= C + D && \text{(OR with inputs } C, D) \end{aligned}$$

Substituting these back yields the final OR-NAND equation:

$$F = \overline{(A' + B) \cdot (A' + D) \cdot (C + D)}$$

This dictates a first level of OR gates generating sums, feeding into a final NAND gate:



NOR Gate Implementation

Q1. Express $f(A, B, C) = \Pi(0, 2, 3, 5, 7)$ with NOR gates. (5 marks)

[CSE 205 | L-2/T-1 | 2020–21]

Answer

Step 1: K-Map Minimization in POS Form

We are given the Boolean function in the Product of Maxterms (POS) form:

$$f(A, B, C) = \Pi(0, 2, 3, 5, 7)$$

To express this function using **NOR gates**, it is most efficient to first minimize the function into its minimal Product of Sums (POS) form. We achieve this by mapping the Maxterms (0s) onto a 3-variable Karnaugh Map and grouping them.

		BC			
		00	01	11	10
A	0	0	1	0	0
	1	1	0	0	1

Maxterm Group Extraction (Grouping 0s):

- **Group 1 (Cells 0, 2):** Located in row $A = 0$, spanning columns $BC = 00$ and 10 . The variable B changes, while $A = 0$ and $C = 0$ remain constant. This yields the sum term $(A + C)$.
- **Group 2 (Cells 3, 7):** Located in column $BC = 11$, spanning both rows. The variable A changes, while $B = 1$ and $C = 1$ remain constant. This yields the sum term $(B' + C')$.
- **Group 3 (Cells 5, 7):** Located in row $A = 1$, spanning columns $BC = 01$ and 11 . The variable B changes, while $A = 1$ and $C = 1$ remain constant. This yields the sum term $(A' + C')$.

The minimized function in POS form is:

$$f(A, B, C) = (A + C)(B' + C')(A' + C') \quad (1)$$

Step 2: Conversion to NOR Logic

A **NOR-NOR** architecture naturally implements a Product of Sums (POS) expression. To mathematically map the POS equation to NOR gates, we apply the **Double Negation Law** and **De Morgan's Laws**:

$$f = (A + C)(B' + C')(A' + C')$$

$$f = \overline{\overline{(A + C)} \cdot \overline{(B' + C')} \cdot \overline{(A' + C')}} \quad (\text{Double Negation Law: } \overline{\overline{X}} = X)$$

$$f = \overline{\overline{(A + C)} + \overline{(B' + C')} + \overline{(A' + C')}} \quad (\text{De Morgan's Law: } \overline{X \cdot Y \cdot Z} = \overline{X} + \overline{Y} + \overline{Z})$$

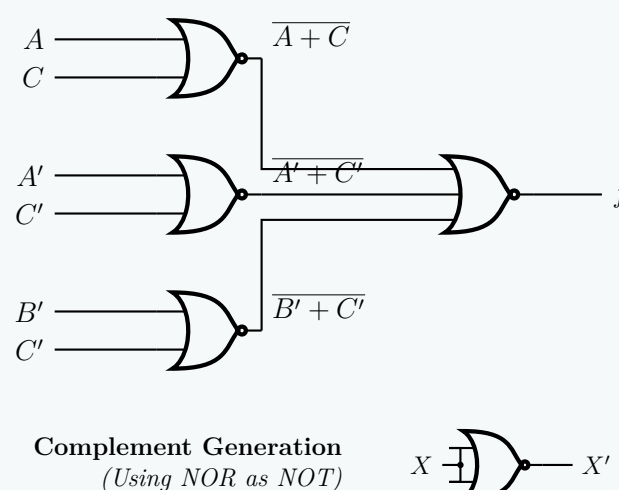
This final equation dictates a two-level NOR structure:

- The first level consists of three 2-input NOR gates generating the terms $\overline{A + C}$, $\overline{B' + C'}$, and $\overline{A' + C'}$.
- The second level consists of a single 3-input NOR gate that sums and inverts these terms to produce the final output f .

Step 3: Circuit Diagram

Below is the complete implementation of f using exclusively NOR gates.

Note: If the complemented literals (A' , B' , C') are not directly available from the inputs, they can be generated using a 2-input NOR gate with its inputs tied together (e.g., $A' = \overline{A + A}$).



Q2. Express $f(A, B, C, D) = \text{SOP}(0, 4, 8, 9, 10, 11, 12, 14)$ with the two-level forms of logic “NOR-OR” and “AND-NOR”. **(8 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

Step 1: K-Map Minimization for f and f'

To implement the target Boolean function using the required two-level forms, we need the minimal Sum of Products (SOP) for both the function f and its complement f' .

The given function is $f(A, B, C, D) = \Sigma(0, 4, 8, 9, 10, 11, 12, 14)$. Its complement is $f'(A, B, C, D) = \Sigma(1, 2, 3, 5, 6, 7, 13, 15)$.

We map both sets onto 4-variable Karnaugh Maps.

1.1 Minimized SOP for f

We group the 1s to find the SOP expression for f :

		CD			
		00	01	11	10
AB	00	1	0	0	0
	01	1	0	0	0
	11	1	0	0	1
	10	1	1	1	1

Group Extraction for f :

- **Group 1 (Col 00):** Minterms (0, 4, 12, 8) $\Rightarrow C'D'$
- **Group 2 (Row 10):** Minterms (8, 9, 11, 10) $\Rightarrow AB'$
- **Group 3 (Edges):** Minterms (12, 14, 8, 10) $\Rightarrow AD'$

$$f = C'D' + AB' + AD' \quad (1)$$

1.2 Minimized SOP for f'

We group the 0s of f (which are the 1s of f') to find the SOP expression for f' :

		CD			
		00	01	11	10
AB	00	0	1	1	1
	01	0	1	1	1
	11	0	1	1	0
	10	0	0	0	0

Group Extraction for f' :

- **Group 1:** Minterms (1, 3, 5, 7) $\Rightarrow A'D$
- **Group 2:** Minterms (2, 3, 6, 7) $\Rightarrow A'C$
- **Group 3:** Minterms (5, 7, 13, 15) $\Rightarrow BD$

$$f' = A'D + A'C + BD \quad (2)$$

Step 2: (i) NOR-OR Logic Implementation

A **NOR-OR** implementation generates a Sum of Products (SOP). We start with the SOP equation for f and apply the **Double Negation Law** and **De Morgan's Law** to convert the product terms into NOR operations.

$$f = AB' + AD' + C'D'$$

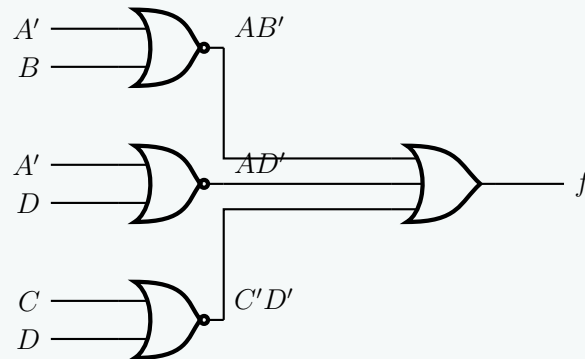
$$f = \overline{\overline{AB'}} + \overline{\overline{AD'}} + \overline{\overline{C'D'}}$$

(Double Negation Law)

$$f = \overline{(A' + B)} + \overline{(A' + D)} + \overline{(C + D)}$$

 (De Morgan's Law: $\overline{XY} = \overline{X} + \overline{Y}$)

This gives us a first level of NOR gates generating the required product terms, which are then summed using a second-level OR gate.



Step 3: (ii) AND-NOR Logic Implementation

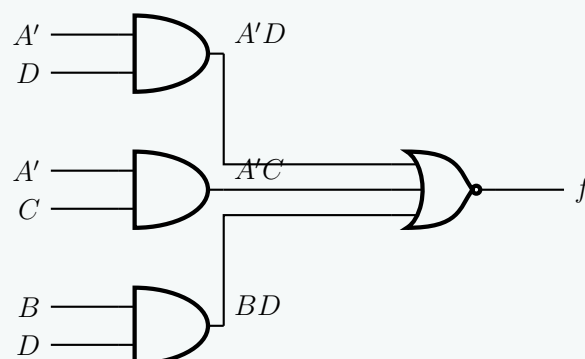
An **AND-NOR** structure essentially performs an AND-OR-INVERT (AOI) operation. It computes the complement of the SOP expression fed into it. Therefore, to output f , we must implement f' using a first level of AND gates and a second level NOR gate. Taking the complement of both sides of our f' equation:

$$f' = A'D + A'C + BD$$

$$\overline{f'} = \overline{A'D + A'C + BD}$$

$$f = \overline{(A' \cdot D) + (A' \cdot C) + (B \cdot D)}$$

This equation maps exactly to three AND gates (computing the products) feeding into a single NOR gate (computing the sum and inverting the result).



Q3. Using NOR gates, evaluate the following expression:

$$p \oplus (q \vee r) \oplus ((p \wedge q) \rightarrow r)$$

(5 marks)

[CSE 205 | L-2/T-1 | 2016–17]

Answer

Step 1: Formal Algebraic Simplification

We begin by evaluating and simplifying the given propositional expression E :

$$E = p \oplus (q \vee r) \oplus ((p \wedge q) \rightarrow r)$$

First, we apply the **Material Implication** identity to rewrite the conditional term in terms of basic Boolean operators:

$$\begin{aligned} (p \wedge q) \rightarrow r &= \neg(p \wedge q) \vee r \\ &= \bar{p} \vee \bar{q} \vee r \end{aligned}$$

(Material Implication: $X \rightarrow Y = \bar{X} \vee Y$)

(De Morgan's Law)

Substituting this back into our primary expression yields:

$$E = p \oplus (q \vee r) \oplus (\bar{p} \vee \bar{q} \vee r) \quad (1)$$

Step 2: Truth Table Evaluation

To systematically evaluate the expression for all possible combinations of the input variables p , q , and r , we construct a truth table. This enables us to determine the canonical minterms of the system.

p	q	r	$q \vee r$	$p \oplus (q \vee r)$	$\bar{p} \vee \bar{q} \vee r$	E	Minterm
0	0	0	0	0	1	1	m_0
0	0	1	1	1	1	0	—
0	1	0	1	1	1	0	—
0	1	1	1	1	1	0	—
1	0	0	0	1	1	0	—
1	0	1	1	0	1	1	m_5
1	1	0	1	0	0	0	—
1	1	1	1	0	1	1	m_7

From the truth table, the function evaluates to 1 at minterms 0, 5, and 7. Thus, the canonical Sum-of-Products (SOP) expression is:

$$E(p, q, r) = \Sigma(0, 5, 7) = \bar{p}\bar{q}\bar{r} + p\bar{q}r + pqr$$

(2)

Step 3: Karnaugh Map Minimization

We map the extracted minterms onto a 3-variable Karnaugh Map to determine the minimal SOP expression.

		qr			
		00	01	11	10
p	0	1	0	0	0
	1	0	1	1	0

Prime Implicant Extraction:

- **Group 1 (Cell 0):** Isolated minterm yields $\bar{p}\bar{q}\bar{r}$.
- **Group 2 (Cells 5, 7):** Combines to eliminate the variable q , yielding the product term pr .

The minimized SOP expression is:

$$E = \bar{p}\bar{q}\bar{r} + pr$$

(3)

Step 4: Mathematical Conversion to NOR-Only Logic

To realize the function using exclusively NOR gates, we must recall that a standard NOR gate computes the negated sum: $\text{NOR}(X, Y) = \overline{X + Y}$. We use Boolean algebra laws to massage our minimized equation into nested negated sums. First, observe that our product terms can be rewritten via **De Morgan's Law** as purely inverted sums:

$$\begin{aligned}\bar{p}\bar{q}\bar{r} &= \overline{p + q + r} \\ pr &= \overline{\bar{p} + \bar{r}}\end{aligned}$$

Substituting these back into E :

$$E = \overline{(p + q + r)} + \overline{(\bar{p} + \bar{r})}$$

Currently, the outer operation is an OR. To convert this final step into NOR gates, we apply the **Double Negation Law** ($X = \overline{\overline{X}}$) to the entire expression:

$$E = \overline{\overline{\overline{(p + q + r)} + \overline{(\bar{p} + \bar{r})}}}$$

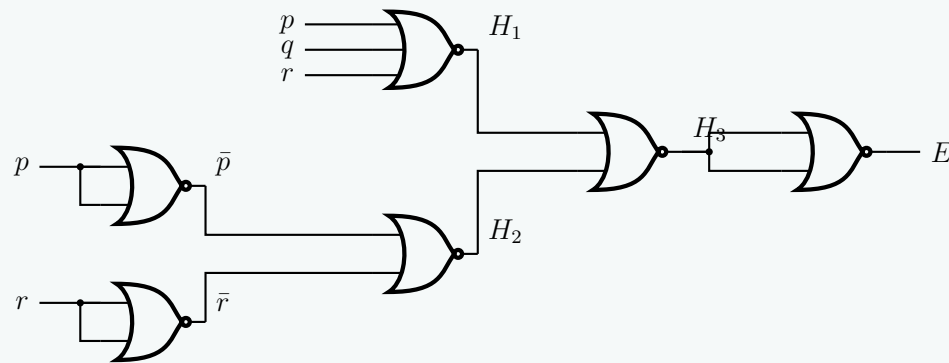
(4)

This maps perfectly to a multi-stage NOR architecture:

- (a) $H_1 = \text{NOR}(p, q, r) = \overline{p + q + r}$
- (b) $H_2 = \text{NOR}(\bar{p}, \bar{r}) = \overline{\bar{p} + \bar{r}}$
- (c) $H_3 = \text{NOR}(H_1, H_2) = \overline{H_1 + H_2}$
- (d) $E = \text{NOT}(H_3) = \text{NOR}(H_3, H_3)$ (Applying a final NOR gate as an inverter)

Step 5: Logic Gate Schematic

Below is the complete gate-level realization using only standard NOR gates. The required complements $\bar{p}$ and $\bar{r}$ are locally generated by using 2-input NOR gates with tied inputs.



Q4. Implement the following Boolean function using only NOR gates:

$$f = xy + yz$$

Your implementation should use as few NOR gates as possible. **(6 marks)**

[CSE 205 | L-2/T-1 | 2015–16]

Answer

Step 1: Boolean Function Simplification

We are given the Boolean function:

$$f = xy + yz$$

To minimize the number of required logic gates, we first simplify the expression by factoring out the common variable y using the **Distributive Law**:

$$f = y(x + z) \quad (1)$$

Step 2: Conversion to NOR Logic

A standard NOR gate performs the negated sum of its inputs: $\text{NOR}(A, B) = \overline{A + B}$. To implement our factored function using exclusively NOR gates, we must manipulate the equation using the **Double Negation Law** and **De Morgan's Laws** until all operations are in the form of negated sums.

Starting with the simplified function:

$$\begin{aligned} f &= y(x + z) \\ &= \overline{\overline{y(x + z)}} && \text{(Double Negation Law: } A = \overline{\overline{A}} \text{)} \\ &= \overline{\overline{y} + \overline{(x + z)}} && \text{(De Morgan's Law: } \overline{A \cdot B} = \overline{A} + \overline{B} \text{)} \end{aligned}$$

Let us analyze the components of this final equation to map them to NOR gates:

(a) $\overline{x + z}$: This is the direct output of a NOR gate with inputs x and z . Let $N_1 = \overline{x + z}$.

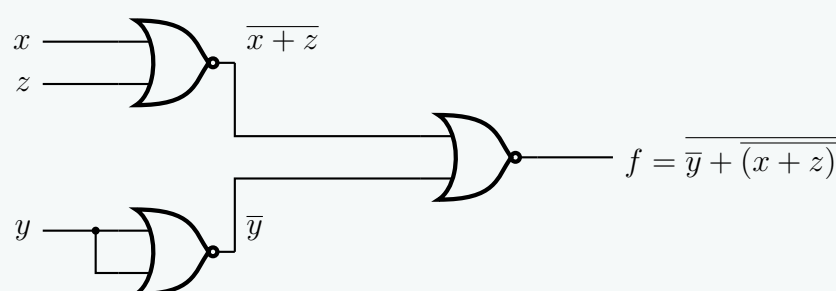
(b) $\overline{y}$: We can generate the complement of y by tying the two inputs of a NOR gate together. $\text{NOR}(y, y) = \overline{y + y} = \overline{y}$. Let $N_2 = \overline{y}$.

(c) $\overline{N_2 + N_1}$: This is the final NOR gate that takes the outputs of N_1 and N_2 as its inputs.

Thus, the function can be perfectly realized using exactly **three NOR gates**.

Step 3: Logic Gate Implementation

Below is the corresponding schematic drawing using the minimum number of NOR gates (3 gates).



Q5. Implement the following Boolean function together with the don't-care conditions using only two-input NOR gates (complements of literals are available as input):

$$F(A, B, C, D) = \Sigma(0, 1, 9, 11), \quad d(A, B, C, D) = \Sigma(2, 8, 10, 14, 15)$$

(15 marks)

[CSE 205 | L-2/T-1 | 2013–14]

Answer

Step 1: Karnaugh Map Minimization

We are given the minterms and don't-care conditions for a 4-variable function:

$$F(A, B, C, D) = \Sigma(0, 1, 9, 11)$$

$$d(A, B, C, D) = \Sigma(2, 8, 10, 14, 15)$$

We plot these on a 4-variable Karnaugh map to find the minimal Sum-of-Products (SOP) expression.

		CD			
		00	01	11	10
AB	00	1	1	0	-
	01	0	0	0	0
	11	0	0	-	-
	10	-	1	1	-

Prime Implicant Extraction:

- **Group 1** (Cells 0, 1, 8, 9): Covers the top and bottom rows of the first two columns. The variables that remain constant are $B = 0$ and $C = 0$. This yields the term $\bar{B}\bar{C}$.
- **Group 2** (Cells 8, 9, 10, 11): Covers the entire $AB = 10$ row. The variables that remain constant are $A = 1$ and $B = 0$. This yields the term $A\bar{B}$.

The minimized SOP expression is:

$$F = \bar{B}\bar{C} + A\bar{B} \quad (1)$$

Step 2: Boolean Factoring & NOR Conversion

Our goal is to implement this function using **only two-input NOR gates**. A NOR gate performs the negated sum of its inputs: $\text{NOR}(X, Y) = \overline{X + Y}$. To minimize the gate count, we first factor the SOP expression and then convert it into a nested negated-sum format.

1. Factoring out common literals:

$$F = \bar{B}\bar{C} + A\bar{B}$$

$$F = \bar{B}(A + \bar{C}) \quad (\text{Distributive Law})$$

2. Applying Involution and De Morgan's Law:

$$F = \overline{\overline{\bar{B}(A + \bar{C})}} \quad (\text{Double Negation Law: } X = \overline{\overline{X}})$$

$$F = \overline{\bar{B} + \overline{A + \bar{C}}} \quad (\text{De Morgan's Law: } \overline{X \cdot Y} = \bar{X} + \bar{Y})$$

$$F = \overline{B + A + \bar{C}}$$

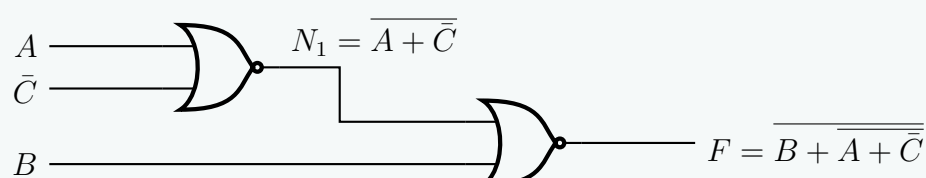
Let us analyze the required operations mapping perfectly to two-input NOR gates:

- (a) $N_1 = \overline{A + \bar{C}} \implies$ The output of a 2-input NOR gate with inputs A and $\bar{C}$.
- (b) $F = \overline{B + N_1} \implies$ The output of a second 2-input NOR gate with inputs B and N_1 .

Since we are given that the **complements of literals are available as input**, we do not need to use an additional NOR gate to generate $\bar{C}$. The complete function requires exactly **two** NOR gates.

Step 3: Logic Gate Implementation

Below is the highly optimized schematic using exclusively two-input NOR gates.



XOR Properties

Q1. Show that the dual of the exclusive-OR is equal to its complement. (7 marks)

[CSE 205 | L-2/T-1 | 2022–23, 2019–20]

Answer

Step 1: Definition of the Exclusive-OR (XOR) Operation

Let $f(A, B)$ represent the Boolean function for the Exclusive-OR (XOR) operation. By definition, the sum-of-products (SOP) expression for XOR is:

$$f(A, B) = A \oplus B = A\bar{B} + \bar{A}B \quad (1)$$

Step 2: Deriving the Dual of the XOR Function

The **Principle of Duality** states that the dual of a Boolean expression is obtained by:

- Interchanging OR (+) and AND (·) operators.
- Interchanging the constants 0 and 1 (though none are present in this specific equation).
- Leaving all variables and their complements unchanged.

Applying the duality principle to $f(A, B) = A\bar{B} + \bar{A}B$:

$$f^D(A, B) = (A + \bar{B}) \cdot (\bar{A} + B) \quad (2)$$

Now, we expand and simplify the dual expression using Boolean algebra laws:

$$\begin{aligned} f^D(A, B) &= (A + \bar{B}) \cdot (\bar{A} + B) \\ &= A\bar{A} + AB + \bar{B}\bar{A} + \bar{B}B && \text{(Distributive Law)} \\ &= 0 + AB + \bar{A}\bar{B} + 0 && \text{(Complementarity Law: } X\bar{X} = 0) \\ &= AB + \bar{A}\bar{B} && \text{(Identity Law: } X + 0 = X) \end{aligned}$$

Thus, the minimized dual of the XOR function is:

$$f^D(A, B) = AB + \bar{A}\bar{B} \quad (3)$$

Step 3: Deriving the Complement of the XOR Function

Now, we find the algebraic complement (NOT) of the original XOR function, denoted as $\overline{f(A, B)}$ or $\overline{A \oplus B}$. We apply **De Morgan's Laws** and Boolean simplification:

$$\begin{aligned} \overline{f(A, B)} &= \overline{A\bar{B} + \bar{A}B} \\ &= (\overline{A\bar{B}}) \cdot (\overline{\bar{A}B}) && \text{(De Morgan's Law: } \overline{X + Y} = \bar{X} \cdot \bar{Y}) \\ &= (\bar{A} + B) \cdot (A + \bar{B}) && \text{(De Morgan's Law: } \overline{X \cdot Y} = \bar{X} + \bar{Y}) \\ &= (\bar{A} + B) \cdot (A + \bar{B}) && \text{(Double Negation Law: } \overline{\bar{X}} = X) \end{aligned}$$

Notice that this expression is structurally identical to the unexpanded dual we found in Step 2. Expanding it yields:

$$\begin{aligned} \overline{f(A, B)} &= \bar{A}A + \bar{A}\bar{B} + BA + B\bar{B} && \text{(Distributive Law)} \\ &= 0 + \bar{A}\bar{B} + AB + 0 && \text{(Complementarity Law)} \\ &= AB + \bar{A}\bar{B} && \text{(Commutative Law & Identity Law)} \end{aligned}$$

Thus, the complement of the XOR function is the Exclusive-NOR (XNOR) function:

$$\overline{f(A, B)} = AB + \bar{A}\bar{B} \quad (4)$$

Step 4: Conclusion

Comparing the results from Step 2 and Step 3:

$$\begin{aligned} f^D(A, B) &= AB + \bar{A}\bar{B} \\ \overline{f(A, B)} &= AB + \bar{A}\bar{B} \end{aligned}$$

Since $f^D(A, B) = \overline{f(A, B)}$, we have successfully demonstrated that the **dual of the Exclusive-OR operation is mathematically identical to its complement**.

Q2. Show that XOR operation is not distributive over AND operation. (7 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

Step 1: Formulating the Distributive Property

To determine if an operation $*$ is distributive over another operation $\circ$, the following property must hold true for all possible values of the variables:

$$A * (B \circ C) = (A * B) \circ (A * C)$$

Therefore, for the Exclusive-OR ($\oplus$) operation to be distributive over the AND ($\cdot$) operation, the following Boolean equation must be an identity:

$$A \oplus (B \cdot C) \stackrel{?}{=} (A \oplus B) \cdot (A \oplus C) \quad (1)$$

We will test this hypothesis using both algebraic expansion and a truth table.

Step 2: Algebraic Expansion and Comparison

We will expand both the Left-Hand Side (LHS) and the Right-Hand Side (RHS) of the proposed identity into Sum-of-Products (SOP) form to see if they are mathematically equivalent.

Expanding the LHS:

$$\begin{aligned} \text{LHS} &= A \oplus (BC) \\ &= A(\overline{BC}) + \bar{A}(BC) && \text{(Definition of XOR: } X \oplus Y = X\bar{Y} + \bar{X}Y\text{)} \\ &= A(\bar{B} + \bar{C}) + \bar{A}BC && \text{(De Morgan's Law: } \overline{XY} = \bar{X} + \bar{Y}\text{)} \\ \text{LHS} &= A\bar{B} + A\bar{C} + \bar{A}BC && \text{(Distributive Law)} \end{aligned}$$

Expanding the RHS:

$$\begin{aligned} \text{RHS} &= (A \oplus B) \cdot (A \oplus C) \\ &= (A\bar{B} + \bar{A}B) \cdot (A\bar{C} + \bar{A}C) && \text{(Definition of XOR)} \\ &= (A\bar{B})(A\bar{C}) + (A\bar{B})(\bar{A}C) + (\bar{A}B)(A\bar{C}) + (\bar{A}B)(\bar{A}C) && \text{(Distributive Law / FOIL)} \\ &= (AA)\bar{B}\bar{C} + (A\bar{A})\bar{B}C + (\bar{A}A)B\bar{C} + (\bar{A}\bar{A})BC && \text{(Commutative & Associative Laws)} \\ &= A\bar{B}\bar{C} + (0)\bar{B}C + (0)B\bar{C} + \bar{A}BC && \text{(Idempotent } AA = A, \text{ Complementarity } A\bar{A} = 0\text{)} \\ \text{RHS} &= A\bar{B}\bar{C} + \bar{A}BC && \text{(Identity Law: } X + 0 = X\text{)} \end{aligned}$$

Comparison: Comparing the minimized forms, we clearly see that:

$$A\bar{B} + A\bar{C} + \bar{A}BC \neq A\bar{B}\bar{C} + \bar{A}BC$$

Because the expressions are not identical, the XOR operation is **not** distributive over the AND operation.

Step 3: Proof by Truth Table & Counterexample

To provide absolute proof, we evaluate both sides of the equation for all $2^3 = 8$ combinations of the variables A, B , and C .

A	B	C	$B \cdot C$	$A \oplus (B \cdot C)$ (LHS)	$A \oplus B$	$A \oplus C$	$(A \oplus B) \cdot (A \oplus C)$ (RHS)	Match?
0	0	0	0	0	0	0	0	Yes
0	0	1	0	0	0	1	0	Yes
0	1	0	0	0	1	0	0	Yes
0	1	1	1	1	1	1	1	Yes
1	0	0	0	1	1	1	1	Yes
1	0	1	0	1	1	0	0	No
1	1	0	0	1	0	1	0	No
1	1	1	1	0	0	0	0	Yes

Counterexample Analysis: As highlighted in the truth table, consider the case where $A = 1, B = 0$, and $C = 1$:

- **LHS:** $1 \oplus (0 \cdot 1) = 1 \oplus 0 = 1$
- **RHS:** $(1 \oplus 0) \cdot (1 \oplus 1) = 1 \cdot 0 = 0$

Since $1 \neq 0$, the equality fails. **Thus, the XOR operation does not distribute over the AND operation.**

Q3. Why is the XOR operation called an odd function? Convert the following Boolean expression to a canonical maxterm form:

$$a \oplus b \oplus c \oplus d$$

(10 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

Part 1: Why the XOR Operation is an Odd Function

The Exclusive-OR (XOR) operation, denoted by $\oplus$, is called an **odd function** (or an odd parity function) because its output evaluates to 1 (TRUE) *if and only if an odd number of its input variables are equal to 1*. Conversely, if the number of inputs containing 1 is even (including zero), the output evaluates to 0 (FALSE). Mathematically, for a multiple-variable XOR function:

$$f(x_1, x_2, \dots, x_n) = x_1 \oplus x_2 \oplus \dots \oplus x_n = \begin{cases} 1, & \text{if } \sum_{i=1}^n x_i \text{ is odd} \\ 0, & \text{if } \sum_{i=1}^n x_i \text{ is even} \end{cases} \tag{1}$$

This property makes XOR gates the fundamental building blocks of parity generators and checkers in digital circuits.

Part 2: Conversion to Canonical Maxterm Form

We are tasked with converting the 4-variable Boolean expression into its canonical maxterm form (Product of Maxterms, ΠM):

$$F(a, b, c, d) = a \oplus b \oplus c \oplus d$$

Since XOR is an odd function, $F(a, b, c, d) = 1$ when an odd number of variables are 1 (i.e., 1 or 3 inputs are 1). The maxterms correspond to the input combinations where the function evaluates to 0. Therefore, $F = 0$ when an **even number** of variables are 1 (i.e., 0, 2, or 4 inputs are 1).

Truth Table Analysis

Let us evaluate all 16 possible combinations for the 4 variables to explicitly extract the maxterms (M_i).

Dec	a	b	c	d	Count of 1s	F = a $\oplus$ b $\oplus$ c $\oplus$ d	Maxterm (M_i)
0	0	0	0	0	0 (Even)	0	$M_0 = a + b + c + d$
1	0	0	0	1	1 (Odd)	1	-
2	0	0	1	0	1 (Odd)	1	-
3	0	0	1	1	2 (Even)	0	$M_3 = a + b + \bar{c} + \bar{d}$
4	0	1	0	0	1 (Odd)	1	-
5	0	1	0	1	2 (Even)	0	$M_5 = a + \bar{b} + c + \bar{d}$
6	0	1	1	0	2 (Even)	0	$M_6 = a + \bar{b} + \bar{c} + d$
7	0	1	1	1	3 (Odd)	1	-
8	1	0	0	0	1 (Odd)	1	-
9	1	0	0	1	2 (Even)	0	$M_9 = \bar{a} + b + c + \bar{d}$
10	1	0	1	0	2 (Even)	0	$M_{10} = \bar{a} + b + \bar{c} + d$
11	1	0	1	1	3 (Odd)	1	-
12	1	1	0	0	2 (Even)	0	$M_{12} = \bar{a} + \bar{b} + c + d$
13	1	1	0	1	3 (Odd)	1	-
14	1	1	1	0	3 (Odd)	1	-
15	1	1	1	1	4 (Even)	0	$M_{15} = \bar{a} + \bar{b} + \bar{c} + \bar{d}$

Final Canonical Maxterm Expression

By gathering the indices where $F = 0$, we form the canonical Product of Maxterms (ΠM):

$$F(a, b, c, d) = \prod M(0, 3, 5, 6, 9, 10, 12, 15) \tag{2}$$

Expanded into its pure algebraic form (Product of Sums), this is:

$$F(a, b, c, d) = (a + b + c + d) \cdot (a + b + \bar{c} + \bar{d}) \cdot (a + \bar{b} + c + \bar{d}) \cdot (a + \bar{b} + \bar{c} + d) \cdot (\bar{a} + b + c + \bar{d}) \cdot (\bar{a} + b + \bar{c} + d) \cdot (\bar{a} + \bar{b} + c + d) \cdot (\bar{a} + \bar{b} + \bar{c} + \bar{d})$$

Parity Checker and Generator

Q1. In a network, for a 3-bit data transmission, an error will be generated if there is an odd parity. Design this logical circuit. **(5 marks)**
[CSE 205 | L-2/T-1 | 2020–21]

Answer

Step 1: Problem Analysis and Truth Table

We need to design a logic circuit for a 3-bit data transmission system that generates an error signal when the data contains an **odd parity** (i.e., an odd number of 1s in the 3-bit sequence).

Let the 3-bit input data be A , B , and C , and let the error output be E . The output E will be 1 when the number of 1s in the input is exactly 1 or 3.

A	B	C	Number of 1s	Error Output (E)
0	0	0	0 (Even)	0
0	0	1	1 (Odd)	1
0	1	0	1 (Odd)	1
0	1	1	2 (Even)	0
1	0	0	1 (Odd)	1
1	0	1	2 (Even)	0
1	1	0	2 (Even)	0
1	1	1	3 (Odd)	1

From the truth table, the Sum-of-Minterms expression for the error function is:

$$E(A, B, C) = \sum m(1, 2, 4, 7)$$

(1)

Step 2: Karnaugh Map Minimization

We map the canonical expression onto a 3-variable Karnaugh Map to explore potential simplifications.

		BC			
		00	01	11	10
A	0	0	1	0	1
	1	1	0	1	0

The K-map reveals a classic **checkerboard pattern**. None of the 1s are logically adjacent, which implies that no prime implicants can be grouped to eliminate variables. This specific diagonal pattern is the definitive signature of a parity function, indicating an Exclusive-OR (XOR) or Exclusive-NOR (XNOR) relationship.

Step 3: Algebraic Derivation

To implement this with standard logic gates, we will factor the unsimplified Sum-of-Products (SOP) expression algebraically:

$$E = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$$

(2)

Factor out $\bar{A}$ from the first two terms and A from the last two terms:

$$E = \bar{A}(\bar{B}C + B\bar{C}) + A(\bar{B}\bar{C} + BC)$$

(Distributive Law)

Recognizing the fundamental identities for XOR ($B \oplus C = \bar{B}C + B\bar{C}$) and XNOR ($\overline{B \oplus C} = \bar{B}\bar{C} + BC$), we substitute them into the equation:

$$E = \bar{A}(B \oplus C) + A(\overline{B \oplus C})$$

(Definition of XOR and XNOR)

Let $X = B \oplus C$. The equation simplifies structurally to:

$$E = \bar{A}X + A\bar{X}$$

$$E = A \oplus X$$

(Definition of XOR)

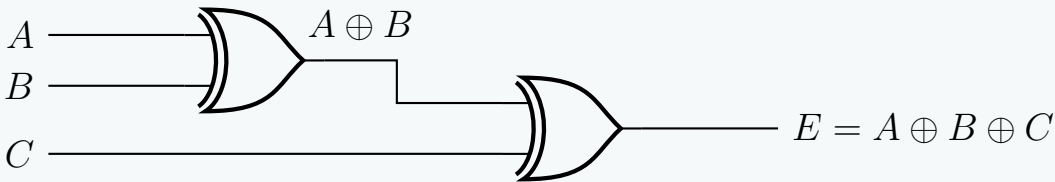
Substituting X back into the expression gives the final minimized logic equation:

$$E = A \oplus B \oplus C$$

(3)

Step 4: Logic Circuit Implementation

The odd parity error-detection circuit can be cleanly implemented using a cascade of two 2-input XOR gates.



Q2. What are the differences between an odd parity checker and an odd parity generator? (8 marks) [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Conceptual Differences

Both parity generators and parity checkers are fundamental combinational logic circuits used for error detection in digital communication. However, they serve complementary roles at opposite ends of a transmission channel.

- An **Odd Parity Generator** is located at the *transmitter*. It evaluates the original data bits and generates an additional bit (the parity bit) to ensure that the total number of 1s in the transmitted message (data + parity) is always **odd**.
- An **Odd Parity Checker** is located at the *receiver*. It evaluates the incoming data alongside the received parity bit to verify if the total number of 1s remains odd. If it is even, it flags an error.

2. Comparative Analysis

Feature	Odd Parity Generator	Odd Parity Checker
Location	Transmitter side.	Receiver side.
Primary Purpose	To <i>create</i> the parity bit before transmission.	To <i>validate</i> the received bits and detect single-bit errors.
Input Data	n bits (Original message).	$n + 1$ bits (Message + Parity bit).
Output Data	1 bit (The calculated Parity bit, P).	1 bit (The Error flag, E).
Output Condition	Outputs 1 if the message has an <i>even</i> number of 1s. Outputs 0 if it is <i>odd</i> .	Outputs 1 (Error) if the received frame has an <i>even</i> number of 1s.
Logic Equivalent	Exclusive-NOR (XNOR) of the n data bits.	Exclusive-NOR (XNOR) of all $n + 1$ received bits.

3. Mathematical Formulation (3-bit Example)

Consider a 3-bit data message A, B, C .

The Odd Parity Generator

The generator produces a parity bit P such that the group $\{A, B, C, P\}$ contains an odd number of 1s.

$$P = \begin{cases} 1 & \text{if } A + B + C \text{ contains an even number of 1s} \\ 0 & \text{if } A + B + C \text{ contains an odd number of 1s} \end{cases}$$
$$P = \overline{A \oplus B \oplus C} \quad \text{(XNOR Function)}$$

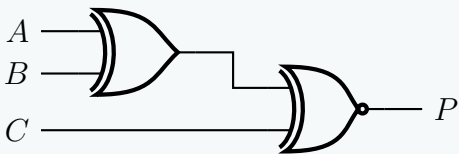
The Odd Parity Checker

The checker receives the bits A, B, C and the parity bit P . It produces an error signal E that is 1 if an error occurred (i.e., the total number of 1s is now even).

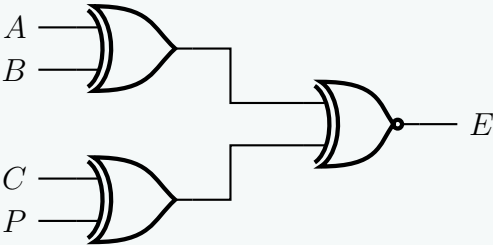
$$E = \begin{cases} 1 \text{ (Error)} & \text{if } A + B + C + P \text{ contains an even number of 1s} \\ 0 \text{ (Valid)} & \text{if } A + B + C + P \text{ contains an odd number of 1s} \end{cases}$$
$$E = \overline{A \oplus B \oplus C \oplus P} \quad \text{(XNOR Function)}$$

4. Logic Circuit Implementations

Odd Parity Generator



Odd Parity Checker



Q3. For a 3-bit Excess-3 code, design an odd parity checker and odd parity generator. (8 marks) [CSE 205 | L-2/T-1 | 2017–18]

Answer

Conceptual Note on Data Encoding and Parity

A critical principle in Digital Logic Design is that **parity generation and checking are independent of the semantic meaning of the data**. Whether the 3-bit data represents a standard binary number, a Gray code, or an **Excess-3 code**, the parity circuitry evaluates only the physical logic levels (0s and 1s) of the bits. Therefore, we will define our 3-bit Excess-3 code word as X, Y , and Z , and design the standard 3-bit odd parity logic.

Part 1: Odd Parity Generator Design (Transmitter Side)

The odd parity generator evaluates the 3-bit data (X, Y, Z) and produces a parity bit P such that the total number of 1s in the transmitted 4-bit message (X, Y, Z, P) is **odd**.

1.1 Truth Table for Odd Parity Generator

If the number of 1s in the input data is even (0 or 2), P must be 1 to make the total count odd. If the number of 1s in the input is odd (1 or 3), P must be 0.

X	Y	Z	Number of 1s in Data	Generated Parity Bit (P)
0	0	0	0 (Even)	1
0	0	1	1 (Odd)	0
0	1	0	1 (Odd)	0
0	1	1	2 (Even)	1
1	0	0	1 (Odd)	0
1	0	1	2 (Even)	1
1	1	0	2 (Even)	1
1	1	1	3 (Odd)	0

1.2 Boolean Algebra Derivation

From the truth table, the Sum-of-Minterms expression is:

$$\begin{aligned} P(X, Y, Z) &= \sum m(0, 3, 5, 6) \\ &= \bar{X}\bar{Y}\bar{Z} + \bar{X}YZ + X\bar{Y}Z + XY\bar{Z} \end{aligned}$$

Factoring out $\bar{X}$ and X :

$$\begin{aligned} P &= \bar{X}(\bar{Y}\bar{Z} + YZ) + X(\bar{Y}Z + Y\bar{Z}) && \text{(Distributive Law)} \\ P &= \bar{X}(\bar{Y} \oplus Z) + X(Y \oplus Z) && \text{(Definition of XOR and XNOR)} \end{aligned}$$

Let $W = Y \oplus Z$. Substituting W into the equation yields the standard definition of the XNOR operation:

$$\begin{aligned} P &= \bar{X}\bar{W} + XW \\ P &= \overline{X \oplus W} \\ P &= \overline{X \oplus Y \oplus Z} && \text{(Odd Parity Generator Equation)} \end{aligned}$$

Part 2: Odd Parity Checker Design (Receiver Side)

The checker receives the 4-bit sequence (X, Y, Z, P) . If no error occurred, the total number of 1s must be odd. If an error alters a bit, the total number of 1s becomes **even**, and the circuit must flag an error ($E = 1$).

2.1 Boolean Algebraic Logic

An even parity condition across 4 variables (X, Y, Z, P) triggers the error. The Boolean function that outputs 1 for an even number of 1s across n variables is the generalized XNOR function.

$$E = \overline{X \oplus Y \oplus Z \oplus P} \tag{1}$$

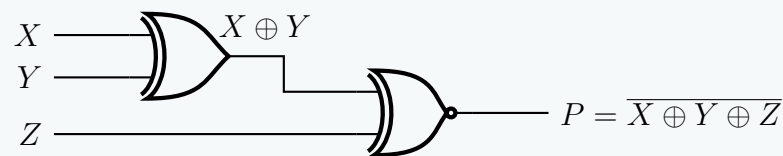
To optimize the logic circuit implementation, we can group the terms:

$$E = \overline{(X \oplus Y) \oplus (Z \oplus P)} \tag{2}$$

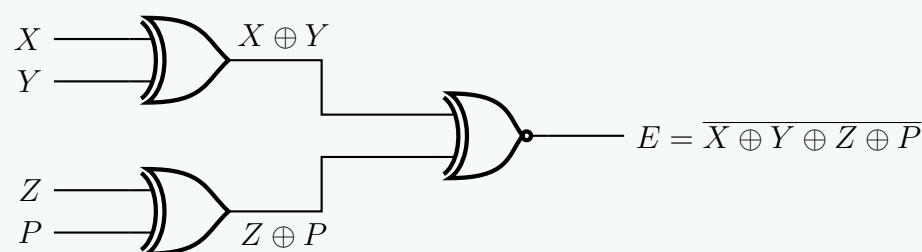
Part 3: Logic Circuit Implementations

Below are the highly optimized schematic diagrams for both the generator and the checker using minimum logic gates.

Odd Parity Generator (Transmitter)



Odd Parity Checker (Receiver)



Q4. Design a negative logic 4-bit parity checker and generator. (6 marks)

[CSE 205 | L-2/T-1 | 2016–17]

Answer

System Assumptions & Negative Logic Principles

We will design an **Even Parity** generator and checker for a **4-bit data word** (A, B, C, D).

In **negative logic**, the logical states (True/False or 1/0) are inversely mapped to physical voltage levels compared to standard positive logic.

- **Logical 1** is represented by a **Low Voltage** (0).
- **Logical 0** is represented by a **High Voltage** (1).

Mathematically, if X_L represents the logical Boolean variable and X_V represents the physical voltage state (the hardware gate behavior), their relationship is defined by:

$$X_L = \overline{X_V} \quad (1)$$

Part 1: Negative Logic 4-Bit Parity Generator

The generator calculates a parity bit P such that the total number of logical 1s in the 5-bit word (A, B, C, D, P) is even. The logical Boolean equation for an even parity generator is the continuous XOR of the data bits:

$$P_L = A_L \oplus B_L \oplus C_L \oplus D_L \quad (2)$$

To determine the required physical logic gates (which are defined by positive voltage truth tables), we substitute the negative logic definition $X_L = \overline{X_V}$ into the equation:

$$\overline{P_V} = \overline{A_V} \oplus \overline{B_V} \oplus \overline{C_V} \oplus \overline{D_V}$$

We apply the Boolean XOR property $\bar{X} \oplus \bar{Y} = X \oplus Y$:

$$\overline{P_V} = (\overline{A_V} \oplus \overline{B_V}) \oplus (\overline{C_V} \oplus \overline{D_V})$$

$$\overline{P_V} = (A_V \oplus B_V) \oplus (C_V \oplus D_V)$$

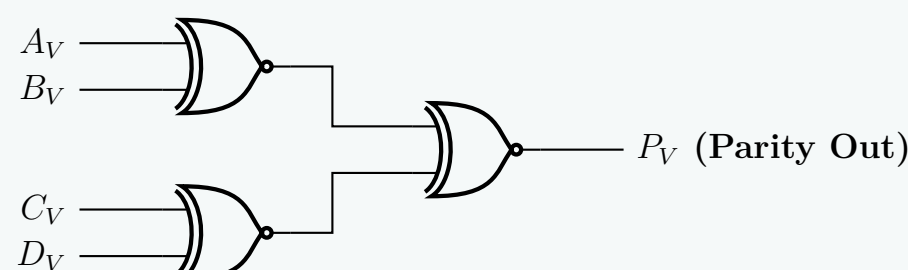
$$\overline{P_V} = A_V \oplus B_V \oplus C_V \oplus D_V$$

To isolate the physical output P_V , we invert both sides:

$$P_V = \overline{A_V \oplus B_V \oplus C_V \oplus D_V}$$

$$P_V = (A_V \odot B_V) \odot (C_V \odot D_V) \quad (\text{Hardware Equation}) \quad (3)$$

Conclusion for Generator: Due to the dual nature of negative logic, an even parity generator for an **even** number of input bits requires physical **XNOR** gates.



Part 2: Negative Logic 4-Bit Parity Checker

The parity checker receives the 5 bits (A, B, C, D, P). If the total number of logical 1s is odd, it outputs an error flag $E_L = 1$. The logical equation is:

$$E_L = A_L \oplus B_L \oplus C_L \oplus D_L \oplus P_L \quad (4)$$

Substituting the negative logic mappings ($X_L = \overline{X_V}$):

$$\overline{E_V} = \overline{A_V} \oplus \overline{B_V} \oplus \overline{C_V} \oplus \overline{D_V} \oplus \overline{P_V}$$

We group the first four terms and apply $\bar{X} \oplus \bar{Y} = X \oplus Y$ as derived previously:

$$\overline{E_V} = (A_V \oplus B_V \oplus C_V \oplus D_V) \oplus \overline{P_V}$$

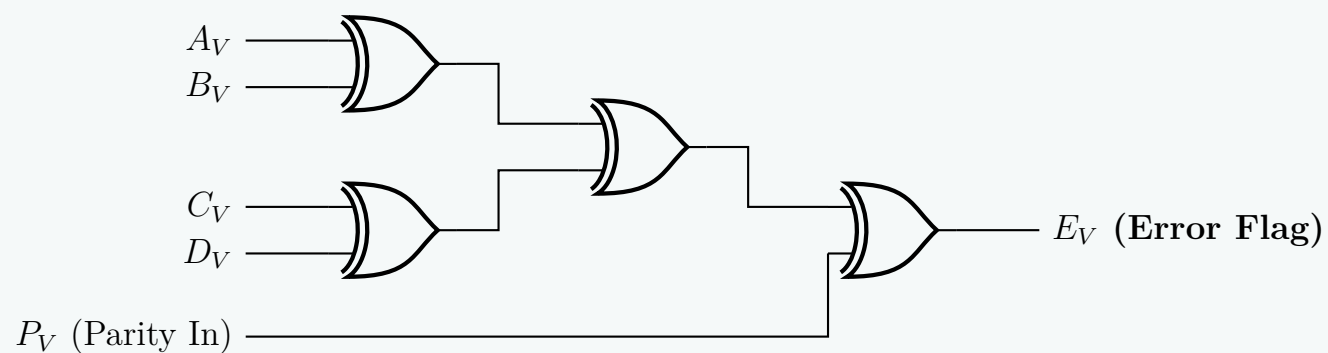
Next, we apply the property $X \oplus \bar{Y} = \overline{X \oplus Y}$:

$$\begin{aligned} \overline{E_V} &= \overline{(A_V \oplus B_V \oplus C_V \oplus D_V) \oplus P_V} \\ \overline{E_V} &= \overline{A_V \oplus B_V \oplus C_V \oplus D_V \oplus P_V} \end{aligned}$$

Inverting both sides to isolate the physical voltage output E_V :

$$E_V = A_V \oplus B_V \oplus C_V \oplus D_V \oplus P_V \quad (\text{Hardware Equation}) \quad (5)$$

Conclusion for Checker: Interestingly, because the checker evaluates an **odd** number of total variables (5), the negative logic inversions cancel out across the XOR boundaries. The physical implementation uses standard **XOR gates**.



BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Canonical & Standard Forms

Minterms, Maxterms, SOP, POS, Conversion

S2 Canonical Forms & Boolean Functions

Canonical vs Standard Forms

Q1. Compare the different types of canonical and standard forms used in Boolean Algebra with examples. (8 marks) [CSE 205 | L-2/T-1 | 2023–24]

Answer

Overview of Boolean Algebraic Forms

In Boolean Algebra, logic functions can be expressed in various algebraic formats. These formats are broadly classified into two main categories: **Canonical Forms** and **Standard Forms**. Understanding the distinction between them is crucial for both theoretical analysis and practical logic gate minimization.

1. Canonical Forms

In a canonical form, every individual term in the expression **must contain all the variables** present in the function, either in their normal (uncomplemented) or complemented state. Canonical forms are unique for a given Boolean function, meaning a specific logic function has exactly one canonical Sum of Products and one canonical Product of Sums.

A. Sum of Minterms (Canonical SOP)

A minterm is a product (AND) term that contains all variables of the function exactly once. The Canonical SOP form is the logical OR (Sum) of these minterms. It represents all the input combinations for which the function evaluates to 1.

- Example:** For a 3-variable function $F(A, B, C)$, a canonical SOP expression is:

$$F(A, B, C) = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + ABC$$

$$F(A, B, C) = m_2 + m_5 + m_7$$

$$F(A, B, C) = \sum m(2, 5, 7)$$

B. Product of Maxterms (Canonical POS)

A maxterm is a sum (OR) term that contains all variables of the function exactly once. The Canonical POS form is the logical AND (Product) of these maxterms. It represents all the input combinations for which the function evaluates to 0.

- Example:** For the same 3-variable function, the canonical POS expression consists of the missing terms from the sum of minterms:

$$F(A, B, C) = (A + B + C) \cdot (A + B + \bar{C}) \cdot (A + \bar{B} + \bar{C}) \cdot (\bar{A} + B + C) \cdot (\bar{A} + \bar{B} + C)$$

$$F(A, B, C) = M_0 \cdot M_1 \cdot M_3 \cdot M_4 \cdot M_6$$

$$F(A, B, C) = \prod M(0, 1, 3, 4, 6)$$

2. Standard Forms

In a standard form, the terms in the expression **do not necessarily contain all the variables**. Standard forms are generally the simplified or minimized versions of canonical forms (e.g., using Karnaugh Maps or Boolean algebraic theorems). They are heavily preferred for actual hardware implementation because they require fewer logic gates and inputs.

A. Sum of Products (Standard SOP)

An expression consisting of two or more AND terms that are ORed together. Unlike minterms, these product terms can have any number of variables.

- Example:** A minimized version of a 3-variable function might look like:

$$F(A, B, C) = AB + \bar{A}C$$

Note that the first term lacks C, and the second term lacks B.

B. Product of Sums (Standard POS)

An expression consisting of two or more OR terms that are ANDed together. These sum terms can contain any number of literals.

- Example:** A minimized POS expression for a function:

$$F(A, B, C) = (A + C) \cdot (\bar{A} + B)$$

3. Summary Comparison Table		
Feature	Canonical Form	Standard Form
Variable Presence	Every term must contain all variables of the function.	Terms can contain one, some, or all variables of the function.
Uniqueness	Unique for a specific Boolean function.	Not necessarily unique (multiple equivalent simplified forms can exist).
Hardware Implementation	Requires maximum hardware (more gates and more inputs per gate). Highly inefficient.	Requires minimized hardware. Highly efficient and practical for IC design.
Derivation Method	Derived directly from the Truth Table.	Derived using minimization techniques like K-Maps or Quine-McCluskey.
Notation	Expressed easily using $\sum m$ (minterms) or $\prod M$ (maxterms).	Expressed purely as algebraic equations.

Q2. (a) What are the differences between canonical form and standard form?
(b) Which form is preferable when implementing a Boolean function with gates? Why?
(c) Which form is obtained when reading a function from a truth table?
(8+4+3 marks)

[CSE 205 | L-2/T-1 | 2012–13]

Answer

(a) Differences between Canonical Form and Standard Form

In Boolean Algebra, logic expressions can be written in different algebraic formats. The fundamental differences between **Canonical** and **Standard** forms revolve around variable presence, uniqueness, and structural complexity.

Criteria	Canonical Form	Standard Form
Variable Inclusion	Every individual term must contain all variables of the Boolean function (either in normal or complemented form).	Terms do not need to contain all variables. They can consist of one, some, or all variables.
Terminology	Composed of <i>Minterms</i> (Canonical Sum of Products) or <i>Maxterms</i> (Canonical Product of Sums).	Composed of standard product terms (SOP) or standard sum terms (POS).
Uniqueness	Strictly Unique for a given Boolean function. There is exactly one canonical SOP and one canonical POS.	Not unique. A function can have multiple equivalent standard forms depending on how it is simplified.
Complexity	Generally longer, redundant, and more complex.	Generally simpler, often representing the minimized version of the function.
Example (for 3 variables A, B, C)	$F = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}BC + A\bar{B}\bar{C} + A\bar{B}C + AB\bar{C} + ABC$	$F = B\bar{C} + AC$

(b) Preferable Form for Logic Gate Implementation

When implementing a Boolean function with physical hardware (logic gates), the **Standard Form** (specifically the *minimized* standard form) is vastly preferable.

Reasons for preferring the Standard Form:

- **Reduced Gate Count:** Standard forms eliminate redundant terms and literals using Boolean simplification techniques (like K-Maps or Quine-McCluskey). Fewer terms directly translate to fewer logic gates in the physical circuit.
- **Lower Fan-in:** Because standard terms contain fewer literals, the corresponding logic gates require fewer input pins. High fan-in gates are slower, consume more area, and are more difficult to manufacture reliably.

- **Cost and Area Efficiency:** Minimizing the gate count and internal wire routing drastically reduces the silicon real estate required on an Integrated Circuit (IC), which lowers the overall manufacturing cost.
- **Improved Performance:** A simpler circuit with fewer components generally exhibits lower power consumption and shorter propagation delays.

(c) Form Obtained Directly from a Truth Table

When extracting a Boolean function directly from a truth table, the **Canonical Form** is inherently obtained.

Explanation: A truth table exhaustively lists every possible combination of the input variables. Each individual row dictates a specific, unique state for *all* variables.

- **Sum of Minterms (Canonical SOP):** If we write the function by taking the logical OR of all the rows where the output is 1, we generate an AND term containing all variables for each row.
- **Product of Maxterms (Canonical POS):** If we write the function by taking the logical AND of all the rows where the output is 0, we generate an OR term containing all variables for each row.

Because every row explicitly defines the state of every single variable in the system, the resulting algebraic expression is strictly canonical prior to any simplification.

Minterms, Maxterms

Q1. Express the following function in terms of minterms:

$$F = (x' + y)(x + z)(y + z)$$

(10 marks)

[CSE 205 | L-2/T-1 | 2023–24]

Answer

Step 1: Simplify the Boolean Function

We are given the Boolean function in a mixed form:

$$F(x, y, z) = (x' + y)(x + z)(y + z)$$

To express the function in terms of minterms (Canonical Sum of Products), it is often easier to first expand and simplify the expression into a Standard Sum of Products (SOP) form.

We can apply the Distributive Law to the last two terms, observing that $(A + B)(A + C) = A + BC$, where $A = z$, $B = x$, and $C = y$:

$$F = (x' + y)(z + xy) \quad (\text{Distributive Law})$$

Next, we distribute $(x' + y)$ across $(z + xy)$:

$$F = x'z + x'xy + yz + y(xy) \quad (\text{Distributive Law})$$

Now, we apply fundamental Boolean algebraic theorems to simplify the terms:

$$\begin{aligned} F &= x'z + (x'x)y + yz + x(yy) && (\text{Associative \& Commutative Laws}) \\ F &= x'z + (0)y + yz + xy && (\text{Complementarity Law: } x'x = 0; \text{ Idempotent Law: } yy = y) \\ F &= x'z + 0 + yz + xy && (\text{Null Law: } 0 \cdot y = 0) \\ F &= xy + x'z + yz && (\text{Identity Law}) \end{aligned}$$

Notice the presence of the term yz . According to the **Consensus Theorem** ($AB + A'C + BC = AB + A'C$), the term yz is redundant and can be eliminated:

$$F = xy + x'z \quad (\text{Consensus Theorem})$$

Step 2: Expand to Canonical Sum of Minterms

The function $F = xy + x'z$ is currently in standard SOP form. To convert it into canonical form (where every term contains all three variables x, y, z), we multiply each term by $(V + V') = 1$, where V is the missing variable.

- The first term xy is missing z . We multiply by $(z + z')$.
- The second term $x'z$ is missing y . We multiply by $(y + y')$.

$$F = xy(z + z') + x'z(y + y') \quad (\text{Identity Law \& Complementarity Law})$$

$$F = xyz + xyz' + x'yz + x'y'z \quad (\text{Distributive Law})$$

Rearranging the terms in logical order of their binary equivalents (x as MSB, z as LSB):

$$F = x'y'z + x'yz + xyz' + xyz \quad (\text{Commutative Law})$$

Step 3: Convert to Minterm Notation

Each individual product term containing all variables is a minterm m_i , where i is the decimal equivalent of the binary string formed by the variables (Uncomplemented = 1, Complemented = 0).

Product Term	Binary Value (x, y, z)	Decimal Index	Minterm
$x'y'z$	001	1	m_1
$x'yz$	011	3	m_3
xyz'	110	6	m_6
xyz	111	7	m_7

By summing these minterms, we arrive at the final canonical expression:

$$F(x, y, z) = \sum m(1, 3, 6, 7) \quad (1)$$

Q2. Convert the Boolean function $F = xy + x'z$ as a product of maxterms using the distributive law. **(10 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Answer

Step 1: Convert to Standard Product of Sums (POS) Form

We begin with the Boolean function in Sum of Products (SOP) form:

$$F(x, y, z) = xy + x'z$$

To convert this into a product form, we apply the **Distributive Law** ($A + BC = (A + B)(A + C)$). Let us treat xy as a single term A , and distribute it over $x'z$:

$$F = (xy + x')(xy + z) \quad (\text{Distributive Law})$$

Next, we apply the Distributive Law again to both of the resulting terms:

$$F = (x' + x)(x' + y)(z + x)(z + y) \quad (\text{Distributive Law})$$

Now, we simplify using fundamental Boolean theorems:

$$F = (1)(x' + y)(x + z)(y + z) \quad (\text{Complementarity Law: } x' + x = 1)$$

$$F = (x' + y)(x + z)(y + z) \quad (\text{Identity Law: } 1 \cdot A = A)$$

The function is now in a standard Product of Sums (POS) form.

Step 2: Expand to Canonical Product of Maxterms

A canonical maxterm must contain all three variables (x, y, z). We expand the standard POS terms by adding 0 in the form of $VV' = 0$ for any missing variable V , and then distributing.

- **First term** ($x' + y$): Missing z . We add zz' .

$$x' + y = x' + y + 0 \quad (\text{Identity Law})$$

$$= x' + y + zz' \quad (\text{Complementarity Law: } zz' = 0)$$

$$= (x' + y + z)(x' + y + z') \quad (\text{Distributive Law})$$

- **Second term** ($x + z$): Missing y . We add yy' .

$$x + z = x + z + yy' \quad (\text{Identity Law})$$

$$= (x + y + z)(x + y' + z) \quad (\text{Distributive Law})$$

- **Third term** ($y + z$): Missing x . We add xx' .

$$y + z = y + z + xx' \quad (\text{Identity Law})$$

$$= (x + y + z)(x' + y + z) \quad (\text{Distributive Law})$$

Substituting these expanded terms back into the function F :

$$F = [(x' + y + z)(x' + y + z')] \cdot [(x + y + z)(x + y' + z)] \cdot [(x + y + z)(x' + y + z)]$$

By the **Idempotent Law** ($A \cdot A = A$), we can eliminate duplicate terms. The term $(x + y + z)$ appears twice, and $(x' + y + z)$ appears twice:

$$F = (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z')$$

Step 3: Convert to Maxterm Notation

Each sum term containing all variables is a maxterm M_i . For maxterms, an uncomplemented variable represents 0, and a complemented variable represents 1. We find the binary and decimal equivalents:

Sum Term	Binary Value (x, y, z)	Decimal Index	Maxterm
$(x + y + z)$	000	0	M_0
$(x + y' + z)$	010	2	M_2
$(x' + y + z)$	100	4	M_4
$(x' + y + z')$	101	5	M_5

Taking the logical product of these maxterms gives the final canonical expression:

$$F(x, y, z) = \prod M(0, 2, 4, 5) \tag{1}$$

Q3. Using Boolean Algebra, express the following Boolean function as a sum of minterms:

$$F(A, B, C) = A + B'C$$

(7 marks) [CSE 205 | L-2/T-1 | 2021–22]

Answer

Step 1: Identify Missing Variables in Each Term

We are given the Boolean function in Standard Sum of Products (SOP) form:

$$F(A, B, C) = A + B'C$$

To express this function as a Sum of Minterms (Canonical SOP form), every term must contain all three variables (A, B , and C).

- The first term, A , is missing variables B and C .
- The second term, $B'C$, is missing variable A .

Step 2: Algebraic Expansion into Canonical Form

We will expand each term by multiplying it by 1 in the form of $(V + V') = 1$, where V is the missing variable. This is justified by the **Identity Law** ($X \cdot 1 = X$) and the **Complementarity Law** ($X + X' = 1$).

Expanding the first term (A):

First, we introduce the missing variable B :

$$\begin{aligned} A &= A \cdot (B + B') && \text{(Identity \& Complementarity Laws)} \\ &= AB + AB' && \text{(Distributive Law)} \end{aligned}$$

Next, we introduce the missing variable C to both resulting terms:

$$\begin{aligned} A &= AB(C + C') + AB'(C + C') && \text{(Identity \& Complementarity Laws)} \\ &= ABC + ABC' + AB'C + AB'C' && \text{(Distributive Law)} \end{aligned}$$

Expanding the second term ($B'C$):

We introduce the missing variable A :

$$\begin{aligned} B'C &= (A + A')B'C && \text{(Identity \& Complementarity Laws)} \\ &= AB'C + A'B'C && \text{(Distributive Law)} \end{aligned}$$

Step 3: Combine and Simplify

Now, substitute the expanded terms back into the original function F :

$$F = (ABC + ABC' + AB'C + AB'C') + (AB'C + A'B'C)$$

Rearrange the terms (Commutative Law) and apply the **Idempotent Law** ($X + X = X$) to eliminate the duplicate term $AB'C$:

$$F = A'B'C + AB'C' + AB'C + ABC' + ABC$$

Step 4: Convert to Minterm Notation

Each product term now contains all three variables, making them valid minterms (m_i). We determine the index i by treating uncomplemented variables as 1 and complemented variables as 0:

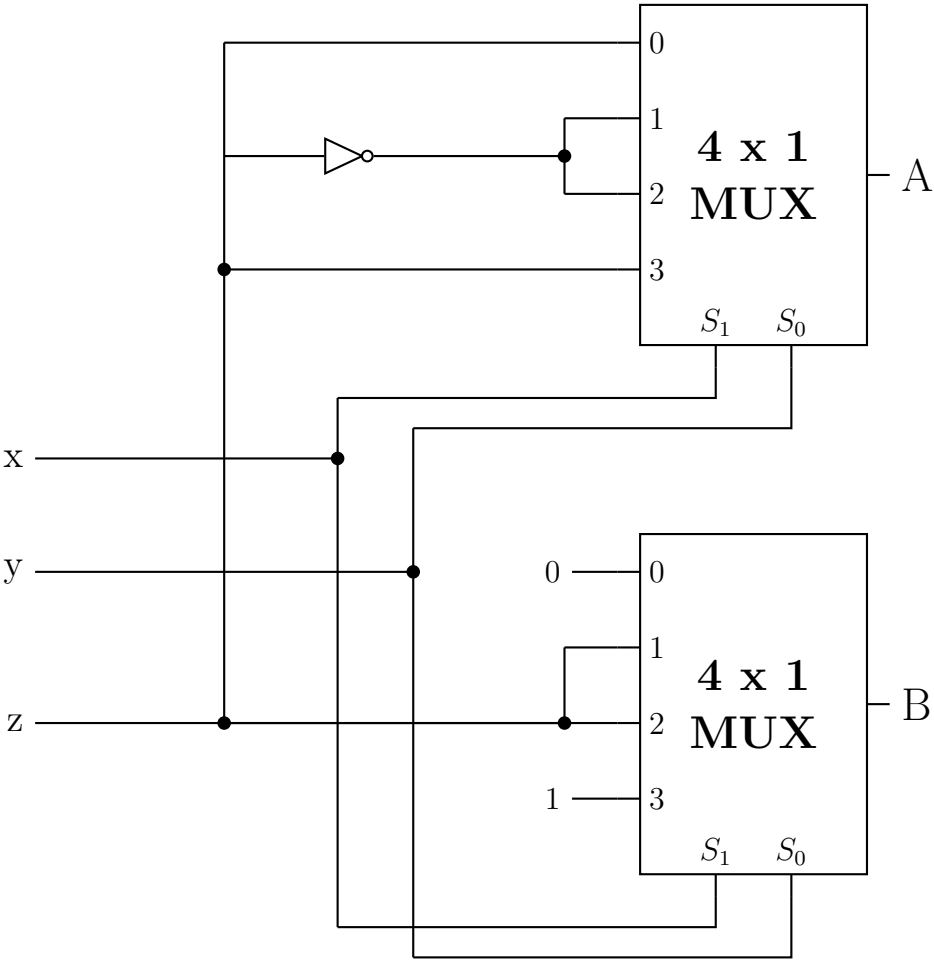
Product Term	Binary Equivalent (A, B, C)	Decimal Index	Minterm Notation
$A'B'C$	001	1	m_1
$AB'C'$	100	4	m_4
$AB'C$	101	5	m_5
ABC'	110	6	m_6
ABC	111	7	m_7

By summing these designated minterms, we arrive at the final canonical expression:

$$F(A, B, C) = \sum m(1, 4, 5, 6, 7)$$

(1)

Q4. Determine the output functions A as the sum of minterms and the output function B as the product of maxterms:



(16 marks)

[CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Circuit Analysis and Multiplexer Foundation

A 4×1 Multiplexer (MUX) routes one of its four data inputs (I_0, I_1, I_2, I_3) to the output based on the value of its two selection lines (S_1, S_0). The canonical logic equation for a 4×1 MUX is:

$$Y = S_1' S_0' I_0 + S_1' S_0 I_1 + S_1 S_0' I_2 + S_1 S_0 I_3 \tag{1}$$

By tracing the provided logic circuit, we can determine the global inputs applied to the select lines of both MUX A and MUX B:

- The Most Significant Select Bit (S_1) is connected directly to variable x .

- The Least Significant Select Bit (S_0) is connected directly to variable y .

Therefore, for both multiplexers, the select lines are $(S_1, S_0) = (x, y)$.

2. Derivation of Output Function A (Sum of Minterms)

By observing the connections to the data pins of **MUX A**, we identify the following inputs from the z bus:

- I_0 is connected directly to $z \implies I_0 = z$
- I_1 is connected through a NOT gate from $z \implies I_1 = z'$
- I_2 is connected through the same NOT gate from $z \implies I_2 = z'$
- I_3 is connected directly to $z \implies I_3 = z$

Substituting these into the MUX equation gives the Boolean function for A :

$$A(x, y, z) = x'y'(z) + x'y(z') + xy'(z') + xy(z)$$

Because each term contains all three variables (x, y, z) , the expression is already in canonical Sum of Products (SOP) form. We can map each product term to its corresponding minterm index (where uncomplemented = 1, complemented = 0):

$$\begin{aligned} x'y'z &\implies 001 \implies m_1 \\ x'yz' &\implies 010 \implies m_2 \\ xy'z' &\implies 100 \implies m_4 \\ xyz &\implies 111 \implies m_7 \end{aligned}$$

Thus, the function A as a sum of minterms is:

$$A(\mathbf{x}, \mathbf{y}, \mathbf{z}) = \sum \mathbf{m}(1, 2, 4, 7) \quad (2)$$

3. Derivation of Output Function B (Product of Maxterms)

By observing the connections to the data pins of **MUX B**, we identify:

- I_0 is connected to Logic 0 $\implies I_0 = 0$
- I_1 is connected directly to the z bus $\implies I_1 = z$
- I_2 is connected directly to the z bus $\implies I_2 = z$
- I_3 is connected to Logic 1 $\implies I_3 = 1$

Substituting these into the MUX equation gives the Boolean function for B :

$$\begin{aligned} B(x, y, z) &= x'y'(0) + x'y(z) + xy'(z) + xy(1) \\ B(x, y, z) &= x'yz + xy'z + xy \end{aligned}$$

To find the canonical maxterms, it is easiest to first expand the standard SOP expression into canonical minterms. We expand the term xy by introducing the missing variable z :

$$\begin{aligned} B(x, y, z) &= x'yz + xy'z + xy(z + z') && \text{(Identity \& Complementarity Laws)} \\ B(x, y, z) &= x'yz + xy'z + xyz + xyz' && \text{(Distributive Law)} \end{aligned}$$

Now we identify the minterm indices for B :

$$\begin{aligned} x'yz &\implies 011 \implies m_3 \\ xy'z &\implies 101 \implies m_5 \\ xyz' &\implies 110 \implies m_6 \\ xyz &\implies 111 \implies m_7 \end{aligned}$$

So, the minterms for B are $\sum m(3, 5, 6, 7)$.

In Boolean algebra, any term that is *not* a minterm is a maxterm (where the function evaluates to 0). For a 3-variable system, the universal set of indices is 0 through 7. The missing indices are 0, 1, 2, and 4.

Thus, the function B as a product of maxterms is:

$$B(\mathbf{x}, \mathbf{y}, \mathbf{z}) = \prod \mathbf{M}(0, 1, 2, 4) \quad (3)$$

4. Comprehensive Truth Table Verification

The derivations can be validated by mapping the entire system to a truth table.

Select Inputs			MUX A		MUX B	
$x (S_1)$	$y (S_0)$	z	Active Input	Output A	Active Input	Output B
0	0	0	$I_0 = z$	0	$I_0 = 0$	0
0	0	1	$I_0 = z$	1	$I_0 = 0$	0
0	1	0	$I_1 = z'$	1	$I_1 = z$	0
0	1	1	$I_1 = z'$	0	$I_1 = z$	1
1	0	0	$I_2 = z'$	1	$I_2 = z$	0
1	0	1	$I_2 = z'$	0	$I_2 = z$	1
1	1	0	$I_3 = z$	0	$I_3 = 1$	1
1	1	1	$I_3 = z$	1	$I_3 = 1$	1

Final Verification:

- A outputs 1 at decimal rows 1, 2, 4, 7 $\implies A = \sum m(1, 2, 4, 7)$
- B outputs 0 at decimal rows 0, 1, 2, 4 $\implies B = \prod M(0, 1, 2, 4)$

BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Minimization Techniques

K-Maps (2/3/4/5-variable), Quine-McCluskey, Don't-Care Conditions

S3Minimization Techniques

Prime Implicants

Q1. Differentiate between prime implicant and essential prime implicant of a K-Map. (7 marks) [CSE 205 | L-2/T-1 | 2023–24, 2011–12]

Answer

Core Definitions

Before differentiating the two, we must define an **Implicant**: In Boolean algebra, an implicant is any valid single group of 1s (minterms) in a Karnaugh Map that can be expressed as a product term.

1. Prime Implicant (PI)

A **Prime Implicant** is an implicant that is as large as possible. It represents a group of 2^n minterms that cannot be completely entirely encompassed by any larger valid group.

- Rule of Thumb:** If you can double the size of the group, the current group is *not* a Prime Implicant.
- Hardware Meaning:** It represents a logic gate with the absolute minimum number of inputs necessary for that specific grouping.

2. Essential Prime Implicant (EPI)

An **Essential Prime Implicant** is a specific type of Prime Implicant that covers **at least one minterm that is not covered by any other Prime Implicant**.

- Rule of Thumb:** If removing the group leaves a minterm stranded (uncovered), that group is essential.
- Hardware Meaning:** Because it is the *only* way to cover certain logic conditions, an EPI **must** be included in the final minimized hardware circuit.

Key Differences

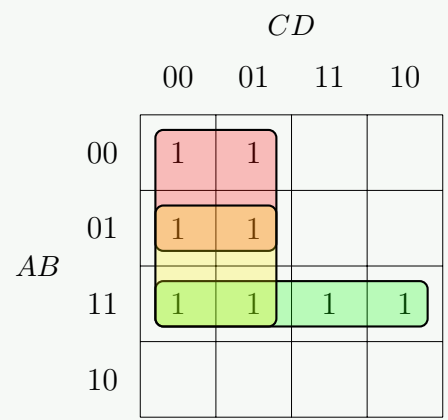
Feature	Prime Implicant (PI)	Essential Prime Implicant (EPI)
Definition	The largest possible grouping of adjacent minterms.	A PI that contains at least one unique, isolated minterm.
Necessity in Final Expression	Optional. It may or may not be included in the final minimized Boolean expression.	Mandatory. It <i>must</i> be included in the final minimized Boolean expression.
Coverage	All its minterms might be overlapped by other PIs.	Has at least one minterm that cannot be grouped by any other PI.
Subsets	All EPIs are PIs, but not all PIs are EPIs.	It is a strict subset of Prime Implicants.

Illustrative Example

Consider the 4-variable Boolean function:

$$F(A, B, C, D) = \sum m(0, 1, 4, 5, 12, 13, 14, 15)$$

Let us map this onto a Karnaugh Map to identify the PIs and EPIs.



Analysis of the K-Map:

- (a) **Group 1 (Red/Green Border typically):** Minterms (0, 1, 4, 5).
- It is a maximally sized 2×2 square. It is a **Prime Implicant**.
 - Notice minterms 0 and 1. They cannot be covered by any other valid grouping.
 - Therefore, this is an **Essential Prime Implicant (EPI)**. (Equation: $A'C'$)
- (b) **Group 2 (Red/Green Border typically):** Minterms (12, 13, 14, 15).
- It is a maximally sized 1×4 row. It is a **Prime Implicant**.
 - Notice minterms 14 and 15. They cannot be covered by any other valid grouping.
 - Therefore, this is an **Essential Prime Implicant (EPI)**. (Equation: AB)
- (c) **Group 3 (Overlapping Middle):** Minterms (4, 5, 12, 13).
- It is a maximally sized 2×2 square. It is a **Prime Implicant**.
 - However, look closely at its minterms: 4 and 5 are already covered by Group 1. 12 and 13 are already covered by Group 2.
 - It does *not* contain any unique minterm.
 - Therefore, this is a **Prime Implicant, but NOT an Essential Prime Implicant**. It represents redundant hardware and is discarded in the final answer. (Equation: BC')

Final Minimized Function (Using only EPIs):

$$F_{minimized} = A'C' + AB$$

(1)

Q2. Find all the prime implicants for the following Boolean function and determine which are essential:

$$F(A, B, C, D) = \Sigma(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$$

(11 marks)

[CSE 205 | L-2/T-1 | 2014–15]

Answer

Step 1: Plotting the Karnaugh Map

We are given the 4-variable Boolean function:

$$F(A, B, C, D) = \Sigma(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$$

We plot the given minterms (1s) onto a 4-variable Karnaugh Map. Empty cells are assumed to be 0. To avoid visual clutter from overlapping groups, only the **Essential Prime Implicants** are graphically highlighted on the map below.

		CD			
		00	01	11	10
AB	00	1		1	1
	01		1	1	
	11		1	1	
	10	1	1	1	1

Step 2: Identifying All Prime Implicants (PIs)

A **Prime Implicant** is a maximally sized grouping of 1s (size 2^n) that cannot be entirely encompassed by any larger grouping. By inspecting all possible adjacent rectangular groups on the K-map, we identify **six** distinct Prime Implicants:

- PI_1 : Grouping the four corners (0, 2, 8, 10) $\implies \mathbf{B'D'}$
- PI_2 : Grouping the center-right square (5, 7, 13, 15) $\implies \mathbf{BD}$
- PI_3 : Grouping the entire bottom row (8, 9, 10, 11) $\implies \mathbf{AB'}$
- PI_4 : Grouping the entire third column (3, 7, 11, 15) $\implies \mathbf{CD}$
- PI_5 : Grouping the bottom-right 2×2 square (9, 11, 13, 15) $\implies \mathbf{AD}$
- PI_6 : Grouping the top and bottom edge cells in columns 11 and 10 (2, 3, 10, 11) $\implies \mathbf{B'C}$

Step 3: Prime Implicant Coverage Chart

To systematically determine which of these PIs are **essential**, we construct a Prime Implicant Chart. An Essential Prime Implicant (EPI) must cover at least one minterm that is not covered by any other PI.

PI	Term	0	2	3	5	7	8	9	10	11	13	15
PI_1 (EPI)	$\mathbf{B'D'}$	X	X				X		X			
PI_2 (EPI)	$\mathbf{BD}$				X	X					X	X
PI_3	$\mathbf{AB'}$						X	X	X	X		
PI_4	$\mathbf{CD}$			X		X				X		X
PI_5	$\mathbf{AD}$							X		X	X	X
PI_6	$\mathbf{B'C}$		X	X					X	X		

Step 4: Conclusion

By examining the columns of the Prime Implicant chart, we look for columns that contain only a single 'X'. These identify minterms that are uniquely covered by only one logic block.

- Minterm 0** is uniquely covered by PI_1 ($B'D'$).
- Minterm 5** is uniquely covered by PI_2 (BD).

All other minterms are covered by overlapping groups (two or more PIs). Therefore, PI_1 and PI_2 are strictly essential to the final minimized circuit.

Final Results:

All Prime Implicants: $\mathbf{B'D', BD, AB', CD, AD, B'C}$

Essential Prime Implicants: $\mathbf{B'D', BD}$

Karnaugh Map

Q1. Simplify the following Boolean function into (a) sum-of-products form and (b) product-of-sums form:

$$F(A, B, C, D) = \Sigma (0, 1, 2, 5, 8, 9, 10)$$

(14 marks)

[CSE 205 | L-2/T-1 | 2022–23]

Answer

Part (a): Sum-of-Products (SOP) Minimization

Step 1: Identify Minterms

The Boolean function is given directly as a sum of minterms:

$$F(A, B, C, D) = \Sigma m(0, 1, 2, 5, 8, 9, 10)$$

69

Step 2: Karnaugh Map Plotting and Grouping for SOP

We plot the 1s on a 4-variable K-map and group them to find the essential prime implicants.

		<i>CD</i>			
		00	01	11	10
<i>AB</i>	00	1	1		1
	01		1		
	11				
	10	1	1		1

Essential Prime Implicants Analysis:

- **Wrap-around Quad:** Cells 0, 1, 8, 9 form a quad spanning the top and bottom rows.
Constant variables: $B = 0, C = 0 \Rightarrow \mathbf{B'C'}$.
- **Corner Quad:** Cells 0, 2, 8, 10 form a quad at the four corners.
Constant variables: $B = 0, D = 0 \Rightarrow \mathbf{B'D'}$.
- **Vertical Pair:** Cells 1, 5 form a pair.
Constant variables: $A = 0, C = 0, D = 1 \Rightarrow \mathbf{A'C'D}$.

Final Simplified SOP Expression:

$$F_{SOP}(A, B, C, D) = \mathbf{B'C' + B'D' + A'C'D}$$

Part (b): Product-of-Sums (POS) Minimization**Step 1: Identify Maxterms**

To find the POS form, we identify the maxterms (0s) by finding the missing indices from the 16 possible combinations (0 to 15):

$$F(A, B, C, D) = \Pi M(3, 4, 6, 7, 11, 12, 13, 14, 15)$$

Step 2: Karnaugh Map Plotting and Grouping for POS

We plot the 0s on the K-map. For POS, each grouped block of 0s yields a sum term. Constant 0s yield uncomplemented variables, and constant 1s yield complemented variables.

		<i>CD</i>			
		00	01	11	10
<i>AB</i>	00	1	1	0	1
	01	0	1	0	0
	11	0	0	0	0
	10	1	1	0	1

Groupings for POS Sum Terms:

- **Horizontal Quad (Row 11):** Cells 12, 13, 14, 15 form a complete row.
Constant variables: $A = 1, B = 1 \Rightarrow (\mathbf{A' + B'})$.
- **Vertical Quad (Column 11):** Cells 3, 7, 11, 15 form a complete column.
Constant variables: $C = 1, D = 1 \Rightarrow (\mathbf{C' + D'})$.
- **Side-Edge Quad:** Cells 4, 6, 12, 14 form a quad by wrapping the left and right edges across two rows.
Constant variables: $B = 1, D = 0 \Rightarrow (\mathbf{B' + D})$.

Final Simplified POS Expression:

$$F_{POS}(A, B, C, D) = (\mathbf{A' + B'})(\mathbf{C' + D'})(\mathbf{B' + D})$$

Q2. Using K-Map, minimize $f(v, w, x, y, z) = \Sigma(2, 7, 8, 15, 22, 23, 25, 29) + d(6, 9, 13, 18, 24, 31)$. Find the Prime Implicants and Essential Prime Implicants, and the Minimal Sums. Write down the responsible minterm for each Essential Prime Implicant. **(15 marks)**
[CSE 205 | L-2/T-1 | 2020–21]

Answer

1. 5-Variable Karnaugh Map Setup

For a 5-variable function $f(v, w, x, y, z)$, we construct two 4-variable K-maps. The first map corresponds to $v = 0$ (minterms 0 to 15) and the second map corresponds to $v = 1$ (minterms 16 to 31).

Given:

Minterms (1s) : $\Sigma(2, 7, 8, 15, 22, 23, 25, 29)$
Don't Cares (Xs) : $d(6, 9, 13, 18, 24, 31)$

We distribute these into the respective maps. For the $v = 1$ map, the physical cell locations correspond to the given minterm/don't care index minus 16.

To maintain visual clarity, the K-maps below explicitly show the groupings for **one** of the valid Minimal Sums (MS_1). All Prime Implicants will be listed systematically in the next section.

Map for $v = 0$

yz

00 01 11 10

wx 00				1
01			1	-
11		-	1	
10	1	-		

Map for $v = 1$

yz

00 01 11 10

wx 00				-
01			1	1
11		1	-	
10	-	1		

2. Identification of Prime Implicants (PIs)

By systematically grouping the maximum number of adjacent 1s and Xs (in sizes of 2^n), we identify the following **six** Prime Implicants. Notice that these groups span symmetrically across both the $v = 0$ and $v = 1$ maps, thereby eliminating the variable v .

- PI_1 : Grouping (2, 6, 18, 22) $\implies \mathbf{w'yz'}$
- PI_2 : Grouping (8, 9, 24, 25) $\implies \mathbf{wx'y'}$
- PI_3 : Grouping (7, 15, 23, 31) $\implies \mathbf{xyz}$
- PI_4 : Grouping (9, 13, 25, 29) $\implies \mathbf{wy'z}$
- PI_5 : Grouping (13, 15, 29, 31) $\implies \mathbf{wxz}$
- PI_6 : Grouping (6, 7, 22, 23) $\implies \mathbf{w'xy}$

3. Essential Prime Implicants (EPIs) and Coverage

To determine the Essential Prime Implicants, we construct a PI Coverage Chart. We only list the true minterms (excluding don't cares) because don't cares do not strictly need to be covered.

PI	Term	2	7	8	15	22	23	25	29
PI_1	$\mathbf{w'yz'}$	X				X			
PI_2	$\mathbf{wx'y'}$			X				X	
PI_3	$\mathbf{xyz}$		X		X		X		
PI_4	$\mathbf{wy'z}$							X	X
PI_5	$\mathbf{wxz}$				X				X
PI_6	$\mathbf{w'xy}$		X			X	X		

By scanning the columns for single 'X' marks, we find the EPIs and their responsible minterms:

- **Minterm 2** is covered *only* by PI_1 . Thus, $\mathbf{w'yz'}$ is an EPI.
- **Minterm 8** is covered *only* by PI_2 . Thus, $\mathbf{wx'y'}$ is an EPI.

Summary of EPIs & Responsible Minterms:

$$\begin{aligned} \text{EPI: } \mathbf{w'yz'} &\longrightarrow \text{Responsible minterm: } \mathbf{m_2} \\ \text{EPI: } \mathbf{wx'y'} &\longrightarrow \text{Responsible minterm: } \mathbf{m_8} \end{aligned}$$

4. Minimal Sums

Selecting the two EPIs (PI_1 and PI_2) automatically covers minterms 2, 8, 22, and 25. The minterms remaining to be covered are: {7, 15, 23, 29}.

We must cover these remaining minterms using the fewest possible remaining PIs. Since all PIs are 3-literal terms, cost is identical across choices. Let us evaluate combinations of 2 additional PIs that satisfy the remaining cover:

- (a) Selecting PI_3 covers {7, 15, 23}. We still need 29, which is covered by PI_4 . $\implies \{PI_3, PI_4\}$
- (b) Selecting PI_5 covers {15, 29}. We still need {7, 23}, which is covered by PI_6 . $\implies \{PI_5, PI_6\}$
- (c) Selecting PI_3 covers {7, 15, 23}. Selecting PI_5 covers {15, 29}. Together they cover all remaining. $\implies \{PI_3, PI_5\}$

Therefore, there are exactly **three** valid Minimal Sums for this function:

$$\begin{aligned} \text{Minimal Sum 1: } f &= \mathbf{w'yz'} + \mathbf{wx'y'} + \mathbf{xyz} + \mathbf{wy'z} \\ \text{Minimal Sum 2: } f &= \mathbf{w'yz'} + \mathbf{wx'y'} + \mathbf{w'xy} + \mathbf{wxz} \\ \text{Minimal Sum 3: } f &= \mathbf{w'yz'} + \mathbf{wx'y'} + \mathbf{xyz} + \mathbf{wxz} \end{aligned}$$

Q3. Convert the following Boolean function from sum-of-products form to a simplified product-of-sums form, then draw a logic circuit for the POS form:

$$F(w, x, y, z) = \Sigma(0, 1, 2, 5, 7, 11, 13)$$

(20 marks)

[CSE 205 | L-2/T-1 | 2019–20]

Answer

Step 1: Identify the Maxterms

The Boolean function is given in canonical Sum-of-Products (SOP) form with minterms:

$$F(w, x, y, z) = \Sigma(0, 1, 2, 5, 7, 11, 13)$$

For a 4-variable function, the total possible minterms range from 0 to 15. The minterms that are *not* present in F correspond to the maxterms (zeros) of the function.

$$F(w, x, y, z) = \Pi(3, 4, 6, 8, 9, 10, 12, 14, 15)$$

Step 2: Karnaugh Map for Product-of-Sums (POS) Minimization

To find the simplified POS expression, we map the zeros (maxterms) of F on a Karnaugh map. Grouping the zeros directly yields the minimal sum terms.

		yz			
		00	01	11	10
wx	00	1	1	0	1
	01	0	1	1	0
	11	0	1	0	0
	10	0	0	1	0

Step 3: Deriving the Sum Terms

From the groups of zeros, we extract the corresponding sum terms. Remember that for zeros (POS), an input value of 0 is represented by the uncomplemented variable, and an input value of 1 is represented by the complemented variable.

- **Group 1 (Single 0 at m_3):** Fixed $w = 0, x = 0, y = 1, z = 1 \implies (\mathbf{w + x + y' + z'})$
- **Group 2 (Pair at 14, 15):** Fixed $w = 1, x = 1, y = 1 \implies (\mathbf{w' + x' + y'})$
- **Group 3 (Pair at 8, 9):** Fixed $w = 1, x = 0, y = 0 \implies (\mathbf{w' + x + y})$

- **Group 4 (Quad at 4, 6, 12, 14):** Fixed $x = 1, z = 0 \implies (\mathbf{x'} + \mathbf{z})$
- **Group 5 (Quad at 8, 10, 12, 14):** Fixed $w = 1, z = 0 \implies (\mathbf{w'} + \mathbf{z})$

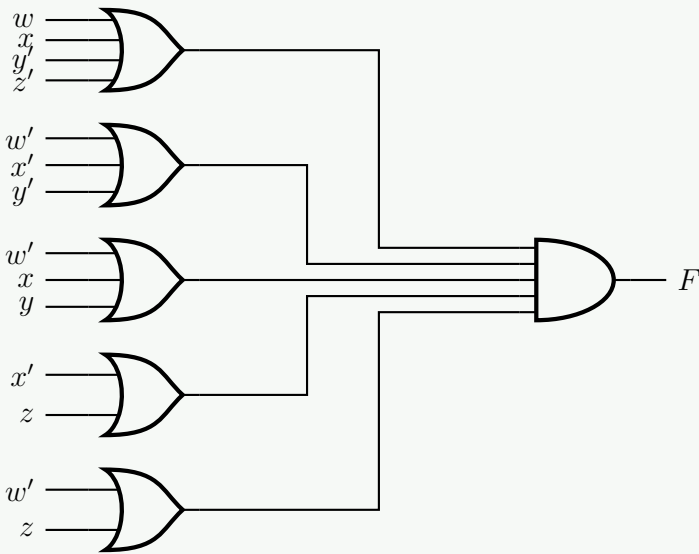
Algebraic Verification (De Morgan's Law): If we grouped the same cells as 1s for the complement function F' , we would get $F' = w'x'yz + wxy + wx'y' + xz' + wz'$. Taking the complement of both sides and applying De Morgan's Law gives:

$$\begin{aligned} F &= (F')' \\ &= (w'x'yz + wxy + wx'y' + xz' + wz')' \\ &= (w'x'yz)' \cdot (wxy)' \cdot (wx'y')' \cdot (xz')' \cdot (wz')' \\ &= (\mathbf{w + x + y' + z'})(\mathbf{w' + x' + y})(\mathbf{w' + x + y})(\mathbf{x' + z})(\mathbf{w' + z}) \end{aligned}$$

(De Morgan's Law: $(A + B)' = A'B'$)

(De Morgan's Law: $(AB)' = A' + B'$)

Step 4: Logic Circuit Diagram for POS Form
The POS function is implemented using a 2-level OR-AND logic structure.

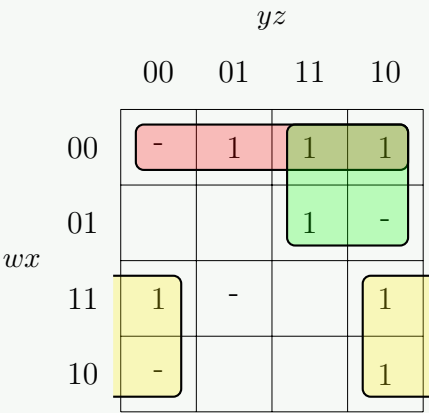


Q4. Using K-Map, minimize $f(w, x, y, z) = \Sigma (1, 2, 3, 7, 10, 12, 14) + d(0, 6, 8, 13)$. Find the Prime Implicants and Essential Prime Implicants. Write down the responsible minterm for each Essential Prime Implicant. **(12 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Karnaugh Map Construction

The Boolean function is given as $f(w, x, y, z) = \Sigma (1, 2, 3, 7, 10, 12, 14) + d(0, 6, 8, 13)$. We map the minterms as '1', the don't cares as 'd', and the remaining cells as '0'.



2. Identifying Prime Implicants (PIs)

A Prime Implicant is a maximal grouping of 1s and don't cares (i.e., a group that cannot be entirely consumed by a larger valid group). By analyzing the K-Map, we extract all possible maximal groupings:

Prime Implicant	Cells Covered	Minterms Covered (excluding 'd')
$w'x'$	0, 1, 3, 2	m_1, m_2, m_3
$w'y$	3, 2, 7, 6	m_2, m_3, m_7
$x'z'$	0, 2, 8, 10	m_2, m_{10}
yz'	2, 6, 14, 10	m_2, m_{10}, m_{14}
wz'	12, 14, 8, 10	m_{10}, m_{12}, m_{14}
wxy'	12, 13	m_{12}

3. Essential Prime Implicants (EPIs)

An Essential Prime Implicant is a PI that covers at least one minterm (1) that is *not* covered by any other Prime Implicant. This exclusively covered minterm is known as the *responsible minterm* or *distinguished minterm*.

By cross-referencing our PIs:

- Minterm m_1 is covered **only** by the PI $w'x'$.
- Minterm m_7 is covered **only** by the PI $w'y$.
- All other minterms ($m_2, m_{10}, m_{12}, m_{14}$) are covered by at least two different PIs.

Essential Prime Implicant	Responsible Minterm
$w'x'$	m_1
$w'y$	m_7

4. Minimal Sum-of-Products (SOP) Expression

To find the final minimized expression, we must first include all Essential Prime Implicants, which cover minterms m_1, m_2, m_3 , and m_7 . The remaining uncovered minterms are m_{10}, m_{12} , and m_{14} . Looking at our list of available PIs, the single implicant wz' perfectly covers all three of these remaining minterms simultaneously, making it the most optimal choice to complete the function.

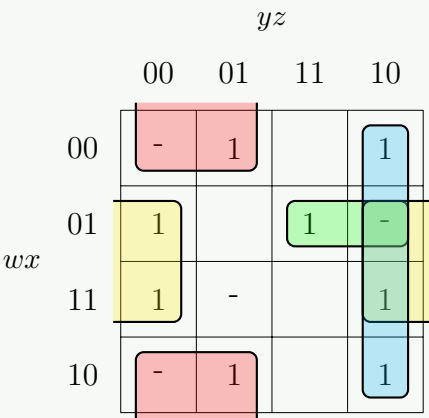
$$f(w, x, y, z) = \text{EPIs} + \text{Remaining Minimal Cover}$$
$$f(w, x, y, z) = w'x' + w'y + wz'$$

Q5. Using K-Map, minimize $f(w, x, y, z) = \text{SOP}(1, 2, 4, 7, 9, 10, 12, 14) + d(0, 6, 8, 13)$. Find the Prime Implicants and Essential Prime Implicants. Write down the responsible minterm for each Essential Prime Implicant. **(12 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

1. Karnaugh Map Construction

The Boolean function is given as $f(w, x, y, z) = \text{SOP}(1, 2, 4, 7, 9, 10, 12, 14) + d(0, 6, 8, 13)$. We plot the minterms as '1', the don't cares as 'd', and leave the remaining cells as '0'.



2. Identifying Prime Implicants (PIs)

A Prime Implicant is a maximal grouping of 1s and don't cares (i.e., a group that cannot be entirely consumed by a larger valid group). By thoroughly analyzing the K-Map, we extract all possible maximal groupings:

Prime Implicant	Cells Covered	Minterms Covered (excluding 'd')
$x'y'$	0, 1, 8, 9	m_1, m_9
wy'	8, 9, 12, 13	m_9, m_{12}
$y'z'$	0, 4, 8, 12	m_4, m_{12}
xz'	4, 6, 12, 14	m_4, m_{12}, m_{14}
yz'	2, 6, 10, 14	m_2, m_{10}, m_{14}
$x'z'$	0, 2, 8, 10	m_2, m_{10}
$w'xy$	6, 7	m_7

3. Essential Prime Implicants (EPIs)

An Essential Prime Implicant is a PI that covers at least one minterm (1) that is *not* covered by any other Prime Implicant. This exclusively covered minterm is known as the *responsible minterm*.

By cross-referencing our PIs with the minterms they cover:

- Minterm m_1 is covered **only** by the PI $x'y'$.
- Minterm m_7 is covered **only** by the PI $w'xy$.
- All other minterms ($m_2, m_4, m_9, m_{10}, m_{12}, m_{14}$) are covered by at least two different PIs.

Essential Prime Implicant	Responsible Minterm
$x'y'$	m_1
$w'xy$	m_7

4. Minimal Sum-of-Products (SOP) Expression

To find the final minimized expression, we must first include all Essential Prime Implicants, which cover minterms m_1, m_7 , and m_9 (since m_9 is absorbed by $x'y'$). The remaining uncovered minterms are m_2, m_4, m_{10}, m_{12} , and m_{14} . Looking at our list of available PIs, we want to select the minimum number of implicants to cover these five minterms:

- Selecting yz' covers m_2, m_{10} , and m_{14} .
- Selecting xz' covers m_4, m_{12} , and m_{14} .

These two PIs together perfectly cover all of the remaining minterms. Adding them to our EPIs yields the optimal and complete sum-of-products equation.

$$f(w, x, y, z) = \text{EPIs} + \text{Remaining Minimal Cover}$$
$$f(w, x, y, z) = x'y' + w'xy + xz' + yz'$$

Q6. Using K-map, solve $f(x_1, x_2, x_3, x_4, x_5) = \Sigma(0, 2, 4, 5, 6, 7, 8, 10, 14, 17, 18, 21, 29, 31) + d\Sigma(11, 20, 22)$. Find the essential prime implicants. (Show the reduction steps.) (15 marks) [CSE 205 | L-2/T-1 | 2016–17]

Answer

1. Problem Statement & Setup

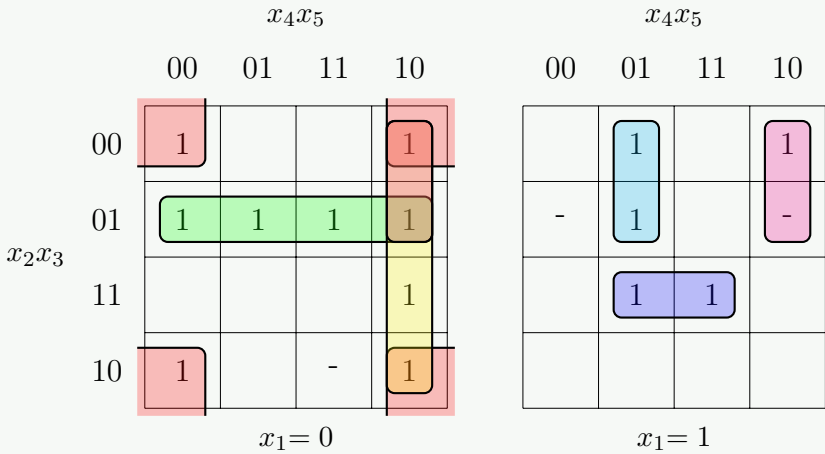
We are given a 5-variable Boolean function defined by its minterms (Sum of Products) and don't care conditions:

$$f(x_1, x_2, x_3, x_4, x_5) = \Sigma(0, 2, 4, 5, 6, 7, 8, 10, 14, 17, 18, 21, 29, 31) + d\Sigma(11, 20, 22)$$

To minimize this function, we construct a 5-variable Karnaugh Map. A 5-variable K-map is visually represented as two adjacent 4-variable K-maps. The left map corresponds to $x_1 = 0$ and the right map corresponds to $x_1 = 1$. The cells represent the remaining variables x_2, x_3, x_4, x_5 .

2. Karnaugh Map Construction

We populate the map with 1 for the minterms, d for the don't care conditions, and 0 for the remaining cells. Groupings are made to cover all 1s using the largest possible combinations (sizes of 2^n), utilizing don't cares (d) only when they help to enlarge a group.



3. Algebraic Foundation of Reduction Steps

Visual grouping on a K-map is mathematically grounded in the **Adjacency Theorem** ($XY + XY' = X$). When we group cells, we systematically eliminate variables that exist in both their true and complemented forms. Let us explicitly demonstrate the Boolean theorems applied during these reductions:

Example 1: A Pair (Group 5 - Minterms 17 and 21)

$$\begin{aligned} m_{17} + m_{21} &= x_1x_2'x_3'x_4'x_5 + x_1x_2'x_3x_4'x_5 \\ &= x_1x_2'x_4'x_5(x_3' + x_3) \\ &= x_1x_2'x_4'x_5(1) \\ &= x_1x_2'x_4'x_5 \end{aligned}$$

(Distributive Law)

(Complementarity Law: $A + A' = 1$)

(Identity Law: $A \cdot 1 = A$)

Example 2: A Quad (Group 2 - Minterms 4, 5, 6, 7)

$$\begin{aligned} m_4 + m_5 + m_6 + m_7 &= x_1'x_2'x_3x_4'x_5' + x_1'x_2'x_3x_4'x_5 + x_1'x_2'x_3x_4x_5' + x_1'x_2'x_3x_4x_5 \\ &= x_1'x_2'x_3x_4'(x_5' + x_5) + x_1'x_2'x_3x_4(x_5' + x_5) && \text{(Distributive Law)} \\ &= x_1'x_2'x_3x_4'(1) + x_1'x_2'x_3x_4(1) && \text{(Complementarity Law)} \\ &= x_1'x_2'x_3(x_4' + x_4) && \text{(Distributive \& Identity Laws)} \\ &= x_1'x_2'x_3 && \text{(Complementarity \& Identity Laws)} \end{aligned}$$

4. Identification of Essential Prime Implicants (EPIs)

A Prime Implicant (PI) becomes an **Essential Prime Implicant** if it is the *only* possible group that covers a specific minterm. To find the minimal Sum of Products (SOP), we must include all EPIs. Let us evaluate the mapped groups to find the essential ones:

Group Identifier	Simplified Term	Minterms Covered	Distinguishing Minterm
G_1 (4-corners of $x_1 = 0$)	$x_1'x_3'x_5'$	0, 2, 8, 10	8
G_2 (Full row in $x_1 = 0$)	$x_1'x_2'x_3$	4, 5, 6, 7	7
G_3 (Full column in $x_1 = 0$)	$x_1'x_4x_5'$	2, 6, 10, 14	14
G_4 (Spans both maps)	$x_2'x_4x_5'$	2, 6, 18, 22 (d)	18
G_5 (Pair in $x_1 = 1$)	$x_1x_2'x_4'x_5$	17, 21	17
G_6 (Pair in $x_1 = 1$)	$x_1x_2x_3x_5$	29, 31	31

Every group listed above contains at least one minterm (highlighted in the final column) that cannot be covered by any other valid grouping. Therefore, **all six of these groups are Essential Prime Implicants**.

5. Final Minimal SOP Expression

We must now verify if any valid 1s remain uncovered by the established EPIs. The set of minterms covered by the union of G_1 through G_6 is:

$$\begin{aligned} &\{0, 2, 8, 10\} \cup \{4, 5, 6, 7\} \cup \{2, 6, 10, 14\} \cup \{2, 6, 18\} \cup \{17, 21\} \cup \{29, 31\} \\ &= \{0, 2, 4, 5, 6, 7, 8, 10, 14, 17, 18, 21, 29, 31\} \end{aligned}$$

This perfectly matches our original function’s minterm list. Because there are no remaining uncovered minterms, the minimal expression is simply the logical OR (sum) of all the Essential Prime Implicants.

Minimal SOP Expression:

$$\begin{aligned} f(x_1, x_2, x_3, x_4, x_5) &= \mathbf{x_1'x_3'x_5'} + \mathbf{x_1'x_2'x_3} + \mathbf{x_1'x_4x_5'} \\ &\quad + \mathbf{x_2'x_4x_5'} + \mathbf{x_1x_2'x_4'x_5} + \mathbf{x_1x_2x_3x_5} \end{aligned}$$

Q7. Using K-maps, find the simplified product of sums form of the following Boolean function:

$$f(w, x, y, z) = \Sigma(1, 5, 8, 12, 14, 15), \quad \text{don't-care minterms: } d = \Sigma(3, 11)$$

(10 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

1. Problem Statement & Maxterm Identification

We are tasked with finding the simplified **Product of Sums (POS)** form for the 4-variable Boolean function:

$$f(w, x, y, z) = \Sigma(1, 5, 8, 12, 14, 15), \quad d = \Sigma(3, 11)$$

To find the POS form, we must group the **Maxterms** (the 0s) in the Karnaugh map. The total number of combinations for 4 variables is 16 (from 0 to 15). The Maxterms are the missing combinations not covered by the given minterms (1s) or don’t-care conditions (ds):

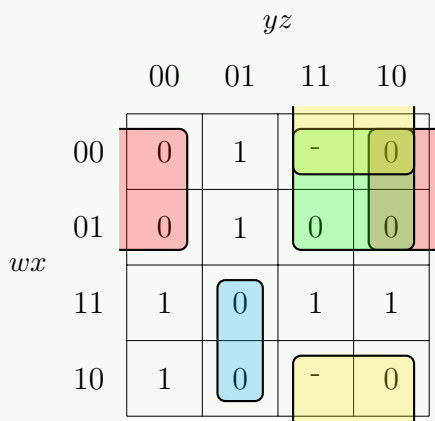
$$\begin{aligned} \text{Total Set} &= \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15\} \\ \text{Minterms (1s)} &= \{1, 5, 8, 12, 14, 15\} \\ \text{Don't-cares (ds)} &= \{3, 11\} \\ \text{Maxterms (}\Pi\text{)} &= \text{Total Set} - (\text{Minterms} \cup \text{Don't-cares}) \\ \Pi &= \{0, 2, 4, 6, 7, 9, 10, 13\} \end{aligned}$$

Thus, the function in standard POS form (with don’t-cares) is:

$$f(w, x, y, z) = \Pi(0, 2, 4, 6, 7, 9, 10, 13) \cdot d(3, 11)$$

2. Karnaugh Map Construction

We populate the K-map by placing 0s at the Maxterm locations, *ds* at the don't-care locations, and 1s elsewhere. To simplify for POS, we group the 0s. We can use the *ds* to make our groupings larger, which further minimizes the Boolean terms.



3. Derivation of Sum Terms (POS Logic)

By grouping the 0s, we are fundamentally finding the minimal Sum of Products (SOP) for the inverted function, f' . Applying De Morgan's Law $(f')' = f$ yields the Product of Sums. A direct rule for reading POS from a K-map of 0s is: **Read the common variables for a group, but invert the literals** (i.e., if a common variable is 0, write it as an uncomplemented variable; if it is 1, write it as a complemented variable), and sum them together.

Let us evaluate each essential group of 0s:

Group	Cells Covered	Common Variables	Resulting Sum Term
G_1 (Quad)	0, 2, 4, 6 (Left & Right edges)	$w = 0, z = 0$	$(w + z)$
G_2 (Quad)	2, 3, 6, 7 (Top-right block)	$w = 0, y = 1$	$(w + y')$
G_3 (Quad)	2, 3, 10, 11 (Top & Bottom edges)	$x = 0, y = 1$	$(x + y')$
G_4 (Pair)	9, 13 (Vertical pair)	$w = 1, y = 0, z = 1$	$(w' + y + z')$

Verification Step: All 0s (0, 2, 4, 6, 7, 9, 10, 13) have been successfully covered by these four groups. The don't-cares (3, 11) were strategically utilized to expand pairs into quads (G_2 and G_3), simplifying the resulting terms by dropping one additional variable.

4. Final Minimal POS Expression

Combining the derived sum terms together via logical AND (product), we arrive at the simplified Product of Sums expression for the Boolean function:

Simplified POS Expression:

$$f(w, x, y, z) = (w + z)(w + y')(x + y')(w' + y + z')$$

Q8. Using K-maps, find the simplified sum of products form of the following Boolean function:

$$f(a, b, c, d, e, f) = \Sigma (1, 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31)$$

Don't-care minterms: $d = \Sigma (6, 20, 24, 25, 26, 27)$. (12 marks) [CSE 205 | L-2/T-1 | 2015–16]

Answer

1. Variable Analysis & Dimension Reduction

We are given a 6-variable Boolean function $f(a, b, c, d, e, f)$ with minterms up to 31. The total number of minterms for a 6-variable function spans from 0 to 63. Since the highest minterm (31) corresponds to the binary representation 011111_2 , the most significant bit (MSB), variable a , is 0 for all active minterms and don't-care conditions.

Consequently, we can factor out a' and express the function as:

$$f(a, b, c, d, e, f) = a' \cdot g(b, c, d, e, f)$$

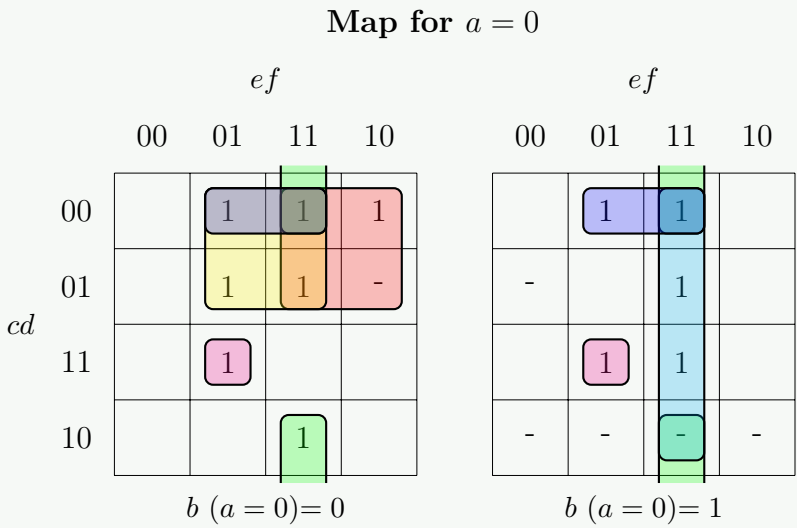
where $g(b, c, d, e, f)$ is a 5-variable function. We can plot g using two 4x4 Karnaugh maps representing the $b = 0$ and $b = 1$ planes. The variables c, d map to the rows and e, f map to the columns.

2. 5-Variable Karnaugh Map Construction

For the $b = 0$ map, the cell indices match the global minterm values directly (0 to 15). For the $b = 1$ map, the local indices are shifted by 16 (i.e., local cell i corresponds to global minterm $i + 16$).

- Map 0 ($b = 0$):** Minterms: 1, 2, 3, 5, 7, 11, 13. Don't cares: 6.

- **Map 1 ($b = 1$):** Minterms: 17, 19, 23, 29, 31. Don't cares: 20, 24, 25, 26, 27.
(Local map indices for $b = 1$: minterms 1, 3, 7, 13, 15; don't cares 4, 8, 9, 10, 11).



3. Prime Implicants & Boolean Term Derivation

We identify the optimal groupings to cover all 1s. Groups spanning across both Map 0 and Map 1 (3D groups) eliminate the b variable. Note that every derived term is fundamentally intersected with a' because $a = 0$.

Group	Spanning Cells (Global Index)	Constant Variables	Term
G_1 (Quad)	Map 0: 2, 3, d_6 , 7	$a = 0, b = 0, c = 0, e = 1$	$a'b'c'e$
G_2 (3D Quad)	Map 0: 3, 11 & Map 1: 19, d_{27}	$a = 0, d = 0, e = 1, f = 1$	$a'd'ef$
G_3 (3D Quad)	Map 0: 1, 3 & Map 1: 17, 19	$a = 0, c = 0, d = 0, f = 1$	$a'c'd'f$
G_4 (Quad)	Map 0: 1, 3, 5, 7	$a = 0, b = 0, c = 0, f = 1$	$a'b'c'f$
G_5 (3D Pair)	Map 0: 13 & Map 1: 29	$a = 0, c = 1, d = 1, e = 0, f = 1$	$a'cde'f$
G_6 (Quad)	Map 1: 19, 23, d_{27} , 31	$a = 0, b = 1, e = 1, f = 1$	$a'bef$

Note: There are alternative minimal choices for covering isolated minterms like m_5 and m_{13} (e.g., pairing m_5 with m_{13}), but the selected combination maintains mathematical minimalism with 6 product terms.

4. Final Simplified SOP Expression

Combining all the identified prime implicants, we yield the fully minimized Sum of Products (SOP) expression for the Boolean function:

Simplified SOP Expression:

$f(a, b, c, d, e, f) = a'b'c'e + a'd'ef + a'c'd'f + a'b'c'f + a'cde'f + a'bef$

Q9. Simplify the following Boolean functions using K-map:

- (a) $F(A, B, C, D) = AB'C + A'B'D' + ABD + ACD' + AB'C' + A'BC'D$
- (b) $F(A, B, C, D) = \Sigma(0, 2, 5, 7, 8, 15)$ with don't-care conditions $d(A, B, C, D) = \Sigma(10, 14)$

(15 marks)

[CSE 205 | L-2/T-1 | 2013–14]

Answer

Part 1: Simplification of the first Boolean function

Step 1: Expand terms to standard minterms (Canonical SOP)

Given the function:

$F(A, B, C, D) = AB'C + A'B'D' + ABD + ACD' + AB'C' + A'BC'D$

We expand each product term to its full minterms by applying the Boolean property $X + X' = 1$:

$AB'C = AB'C(D + D') = AB'CD + AB'CD'$

$A'B'D' = A'B'(C + C')D' = A'B'CD' + A'B'C'D'$

$ABD = AB(C + C')D = ABCD + ABC'D$

$ACD' = A(B + B')CD' = ABCD' + AB'CD'$

$AB'C' = AB'C'(D + D') = AB'C'D + AB'C'D'$

$A'BC'D = A'BC'D$

$\rightarrow m_{11}(1011), m_{10}(1010)$

$\rightarrow m_2(0010), m_0(0000)$

$\rightarrow m_{15}(1111), m_{13}(1101)$

$\rightarrow m_{14}(1110), (m_{10} \text{ already listed})$

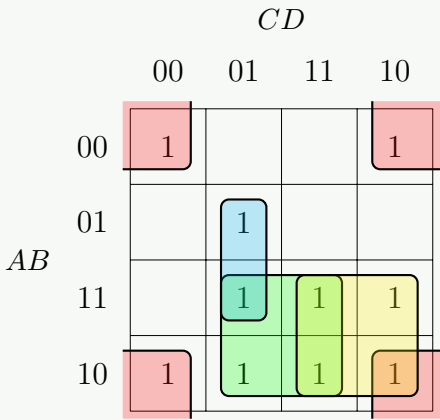
$\rightarrow m_9(1001), m_8(1000)$

$\rightarrow m_5(0101)$

Combining these, the function can be expressed as a sum of minterms:

$$F(A,B,C,D) = \Sigma(0,2,5,8,9,10,11,13,14,15)$$

Step 2: Karnaugh Map Plotting and Grouping



Essential Prime Implicants Analysis:

- **Corners (Quad):** Cells 0, 2, 8, 10 form a quad. Constants: $B = 0, D = 0 \implies \mathbf{B'D'}$.
- **Central Quad:** Cells 9, 11, 13, 15 form a quad. Constants: $A = 1, D = 1 \implies \mathbf{AD}$.
- **Right-side Quad:** Cells 10, 11, 14, 15 form a quad. Constants: $A = 1, C = 1 \implies \mathbf{AC}$.
- **Vertical Pair:** Cells 5, 13 form a pair. Constants: $B = 1, C = 0, D = 1 \implies \mathbf{BC'D}$.

Note: The potential quad of 8, 9, 10, 11 (AB') is redundant because all its 1s are already covered by the $B'D'$, AD , and AC groups.

Final Simplified Expression 1:

$$F(A,B,C,D) = \mathbf{B'D' + AD + AC + BC'D}$$

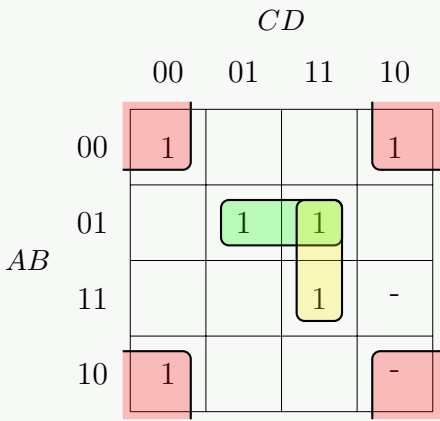
Part 2: Simplification with Don't-Care Conditions

Step 1: K-Map Formulation

We are given:

$$F(A,B,C,D) = \Sigma(0,2,5,7,8,15)$$
$$d(A,B,C,D) = \Sigma(10,14)$$

We plot the 1s for the minterms and Xs for the don't-cares on the 4-variable K-map. Don't-cares are strategically included in groupings only if they help yield larger groups, thereby further reducing literal counts.



Step 2: Grouping Strategy

- **Corners (Quad):** We group m_0, m_2, m_8 , and use the don't-care d_{10} . This forms a quad. Constants: $B = 0, D = 0 \implies \mathbf{B'D'}$.
- **Horizontal Pair:** m_5 and m_7 form an isolated pair. Constants: $A = 0, B = 1, D = 1 \implies \mathbf{A'BD}$.
- **Vertical Pair:** m_{15} must be covered. We can pair it with m_7 vertically. Constants: $B = 1, C = 1, D = 1 \implies \mathbf{BCD}$.
(Alternatively, m_{15} could pair with the don't care d_{14} to yield $\mathbf{ABC}$. Both solutions are minimal and equally valid).

Final Simplified Expression 2:

$$F(A,B,C,D) = \mathbf{B'D' + A'BD + BCD}$$

Q10. Minimize the following function in both SOP and POS forms using Karnaugh map:

$$f(A, B, C, D) = \Sigma m(0, 4, 7, 9, 13) + d(5, 8, 15)$$

(12 marks)

[CSE 205 | L-2/T-1 | 2012–13]

Answer

Part 1: Sum of Products (SOP) Minimization

Step 1: Identify Minterms and Don't Cares

The Boolean function is given in terms of minterms (1s) and don't-care conditions (Xs):

$$f(A, B, C, D) = \Sigma m(0, 4, 7, 9, 13) + d(5, 8, 15)$$

Step 2: Karnaugh Map Plotting and Grouping for SOP

We plot the 1s for the minterms and Xs for the don't-cares on a 4-variable K-map. We aim to form the largest possible groups (powers of 2) of 1s, incorporating Xs only if they help increase the group size.

		CD			
		00	01	11	10
AB	00	1			
	01	1	-	1	
	11		1	-	
	10	-	1		

Essential Prime Implicants Analysis:

- **Quad:** Cells 5, 7, 13, 15 form a 2×2 square.
Constant variables: $B = 1, D = 1 \implies \mathbf{BD}$.

- **Vertical Pair 1:** Cells 0, 4 form a pair.
Constant variables: $A = 0, C = 0, D = 0 \implies \mathbf{A'C'D'}$.

- **Vertical Pair 2:** Cells 9, 13 form a pair.
Constant variables: $A = 1, C = 0, D = 1 \implies \mathbf{AC'D}$.

(Note: Minterm m_9 could alternatively be paired with d_8 to yield $AB'C'$. Both result in a minimal 3-literal term, making either choice perfectly valid.)

Final Simplified SOP Expression:

$$f_{SOP}(A, B, C, D) = \mathbf{BD + A'C'D' + AC'D}$$

Part 2: Product of Sums (POS) Minimization

Step 1: Identify Maxterms

To find the POS form, we focus on the 0s of the function. The maxterms (0s) are the missing indices from the universal set of 16 cells, excluding the don't-cares:

$$f(A, B, C, D) = \Pi M(1, 2, 3, 6, 10, 11, 12, 14) \cdot \Pi_d(5, 8, 15)$$

Step 2: Karnaugh Map Plotting and Grouping for POS

We plot the 0s and Xs. For POS minimization, each grouped block of 0s directly yields a sum term. A variable that remains constant as a 0 gives the uncomplemented literal (e.g., A), and a 1 gives the complemented literal (e.g., A').

		CD			
		00	01	11	10
AB	00	1	0	0	0
	01	1	-	1	0
	11	0	1	-	0
	10	-	1	0	0

Groupings for POS Sum Terms:

- Wrap-around Quad:** Cells 2, 3, 10, 11 form a quad by wrapping the top and bottom rows.
Constants: $B = 0, C = 1 \implies (\mathbf{B} + \mathbf{C}')$.
- Column Quad:** Cells 2, 6, 14, 10 form a full vertical column.
Constants: $C = 1, D = 0 \implies (\mathbf{C}' + \mathbf{D})$.
- Vertical Pair 1:** Cells 1, 5 form a pair (using d_5).
Constants: $A = 0, C = 0, D = 1 \implies (\mathbf{A} + \mathbf{C} + \mathbf{D}')$.
(Alternatively, M_1 could pair with M_3 to yield $(A + B + D')$. Both are valid 3-literal sums.)
- Vertical Pair 2:** Cells 8, 12 form a pair (using d_8).
Constants: $A = 1, C = 0, D = 0 \implies (\mathbf{A}' + \mathbf{C} + \mathbf{D})$.
(Alternatively, M_{12} could pair with M_{14} to yield $(A' + B' + D)$. Both are valid.)

Final Simplified POS Expression:

$$f_{POS}(A, B, C, D) = (\mathbf{B} + \mathbf{C}')(\mathbf{C}' + \mathbf{D})(\mathbf{A} + \mathbf{C} + \mathbf{D}')(\mathbf{A}' + \mathbf{C} + \mathbf{D})$$

Tabulation Method (Quine-McCluskey)

Q1. Simplify the following Boolean function to product-of-sums form:

$$F = \Sigma(3, 4, 6, 7, 9, 11, 12, 14, 15)$$

What is the main purpose of the tabulation method (Quine-McCluskey method) in digital logic design, and how does it systematically help in minimizing Boolean expressions compared to the Karnaugh map method? **(15 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Answer

Part 1: Product-of-Sums (POS) Simplification

Step 1: Identify the Maxterms

The Boolean function is given in Sum of Minterms (SOP) format:

$$F(A, B, C, D) = \Sigma m(3, 4, 6, 7, 9, 11, 12, 14, 15)$$

To simplify into Product-of-Sums (POS) form, we must focus on the maxterms (0s) of the function. The maxterms are the indices missing from the universal set of 16 cells (from 0 to 15):

$$F(A, B, C, D) = \Pi M(0, 1, 2, 5, 8, 10, 13)$$

Step 2: Karnaugh Map Plotting and Grouping

We plot the 0s on a 4-variable K-map. For POS minimization, each valid group of 0s directly translates to a sum term. A variable that remains constant as a 0 yields the uncomplemented literal (e.g., A), and a 1 yields the complemented literal (e.g., A').

		CD			
		00	01	11	10
AB	00	0	0	1	0
	01	1	0	1	1
	11	1	0	1	1
	10	0	1	1	0

Essential Prime Implicant Analysis for Maxterms:

- Corner Quad:** Cells 0, 2, 8, 10 form a quad at the four corners.
Constant variables: $B = 0, D = 0 \implies (\mathbf{B} + \mathbf{D})$.
- Vertical Pair 1:** Cell 13 is isolated and can only group with cell 5.
Constant variables: $B = 1, C = 0, D = 1 \implies (\mathbf{B}' + \mathbf{C} + \mathbf{D}')$.
- Vertical Pair 2:** Cell 1 must be covered. It can group with cell 5 (or cell 0). Grouping with 5 forms the pair (1, 5).
Constant variables: $A = 0, C = 0, D = 1 \implies (\mathbf{A} + \mathbf{C} + \mathbf{D}')$.
(Note: Grouping cell 1 with cell 0 yields $(A + B + C)$, which is an equally valid minimal term. Both selections produce a minimal POS expression.)

Final Simplified POS Expression:

$$F_{POS}(A, B, C, D) = (B + D)(B' + C + D')(A + C + D')$$

Part 2: The Quine-McCluskey (Tabulation) Method

Main Purpose

The Quine-McCluskey (Q-M) method is an exact, algorithmic tabular technique used for minimizing Boolean functions. Its primary purpose is to systematically find the most simplified Boolean expression, particularly when functions contain more than four or five variables.

Systematic Advantages over the Karnaugh Map Method:

(a) Algorithmic vs. Visual:

The K-map relies heavily on human visual pattern recognition to identify adjacent cells. While efficient for up to 4 variables, it becomes extremely error-prone and visually unwieldy for 5 or more variables. The Q-M method replaces graphical grouping with a strict algebraic tabulation, eliminating visual errors.

(b) Exhaustive Prime Implicant Generation:

The Q-M method systematically groups minterms by their number of 1s (index) and compares every term against others differing by exactly one bit. This ensures *every single* Prime Implicant is discovered without relying on geometric intuition.

(c) Prime Implicant Chart (Coverage Table):

Once all Prime Implicants are found, the method uses a coverage chart to cleanly separate *Essential Prime Implicants* from redundant ones. It systematically resolves overlapping terms using techniques like row/column dominance and Petrick’s Method to guarantee an absolute minimal SOP or POS equation.

(d) Computer Programmability:

Because it is a deterministic, step-by-step mathematical algorithm, the tabulation method is highly suitable for software implementation. Modern logic synthesis tools (e.g., in EDA software for FPGA/ASIC design) rely on algorithms derived from Quine-McCluskey and Espresso rather than heuristic graphical maps.

Q2. Using tabular method, find the prime implicants and essential prime implicants of the following Boolean function:

$$F(A, B, C, D) = \Sigma (0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$$

(15 marks)

[CSE 205 | L-2/T-1 | 2022–23]

Answer

Step 1: Binary Representation and Initial Grouping (Column I)

First, we list all the given minterms and convert them to their 4-bit binary representations (A, B, C, D). We then group them based on the number of 1s in their binary form (their index).

Group	Minterm	A	B	C	D	Used
0	m_0	0	0	0	0	✓
1	m_2	0	0	1	0	✓
	m_8	1	0	0	0	✓
2	m_3	0	0	1	1	✓
	m_5	0	1	0	1	✓
	m_9	1	0	0	1	✓
	m_{10}	1	0	1	0	✓
3	m_7	0	1	1	1	✓
	m_{11}	1	0	1	1	✓
	m_{13}	1	1	0	1	✓
4	m_{15}	1	1	1	1	✓

Step 2: Combine Pairs of Minterms (Column II)

We compare minterms from adjacent groups (Group i with Group $i + 1$). If they differ by exactly one bit, we combine them, place a dash (-) in the differing bit position, and mark the original minterms as used (✓).

Group	Combined Minterms	A B C D	Used
0-1	(0, 2)	0 0 - 0	✓
	(0, 8)	- 0 0 0	✓
1-2	(2, 3)	0 0 1 -	✓
	(2, 10)	- 0 1 0	✓
	(8, 9)	1 0 0 -	✓
	(8, 10)	1 0 - 0	✓
2-3	(3, 7)	0 - 1 1	✓
	(3, 11)	- 0 1 1	✓
	(5, 7)	0 1 - 1	✓
	(5, 13)	- 1 0 1	✓
	(9, 11)	1 0 - 1	✓
	(9, 13)	1 - 0 1	✓
	(10, 11)	1 0 1 -	✓
3-4	(7, 15)	- 1 1 1	✓
	(11, 15)	1 - 1 1	✓
	(13, 15)	1 1 - 1	✓

Step 3: Combine Quads of Minterms (Column III)

Now, we compare the terms from adjacent groups in Column II. They can be combined if their dashes align and they differ by exactly one numeric bit. Uncombined terms from Column II would be Prime Implicants, but here all terms find a match.

Group	Combined Minterms	A B C D	Prime Implicant (Literal)
0-1-2	(0, 2, 8, 10)	- 0 - 0	$\text{PI}_1 = B'D'$
	(0, 8, 2, 10)	- 0 - 0	Duplicate
1-2-3	(2, 3, 10, 11)	- 0 1 -	$\text{PI}_2 = B'C$
	(2, 10, 3, 11)	- 0 1 -	Duplicate
	(8, 9, 10, 11)	1 0 - -	$\text{PI}_3 = AB'$
	(8, 10, 9, 11)	1 0 - -	Duplicate
2-3-4	(3, 7, 11, 15)	- - 1 1	$\text{PI}_4 = CD$
	(3, 11, 7, 15)	- - 1 1	Duplicate
	(5, 7, 13, 15)	- 1 - 1	$\text{PI}_5 = BD$
	(5, 13, 7, 15)	- 1 - 1	Duplicate
	(9, 11, 13, 15)	1 - - 1	$\text{PI}_6 = AD$
	(9, 13, 11, 15)	1 - - 1	Duplicate

Since no further combinations can be made, the unique terms in Column III represent all the **Prime Implicants**.

Step 4: Prime Implicant Chart

We construct a chart mapping the Prime Implicants to the minterms they cover. We look for columns containing a single 'X'. These minterms can only be covered by one specific Prime Implicant, making that Prime Implicant **essential**.

PI	Term	0	2	3	5	7	8	9	10	11	13	15
PI_1^*	$B'D'$	X	X				X		X			
PI_2	$B'C$		X	X					X	X		
PI_3	AB'						X	X	X	X		
PI_4	CD			X		X				X		X
PI_5^*	BD				X	X					X	X
PI_6	AD							X		X	X	X

Observation from the Chart:

- Minterm **0** is covered *only* by PI_1 ($B'D'$).
- Minterm **5** is covered *only* by PI_5 (BD).

These isolated columns signify that PI_1 and PI_5 are absolutely required to cover those minterms.

Final Result Summary

All Prime Implicants: $B'D'$ (PI_1), $B'C$ (PI_2), AB' (PI_3), CD (PI_4), BD (PI_5), AD (PI_6).

Essential Prime Implicants: $B'D'$ and BD

Q3. Determine the prime implicants of the following function using the tabulation (tabular) method:

$F(w, x, y, z) = \Sigma (0, 1, 2, 8, 10, 11, 14, 15)$

(15 marks)

[CSE 205 | L-2/T-1 | 2021–22]

Answer

Tabulation (Quine-McCluskey) Method

To determine the prime implicants of the Boolean function $F(w, x, y, z) = \Sigma (0, 1, 2, 8, 10, 11, 14, 15)$, we will systematically group the minterms based on the number of 1s in their binary representation and then progressively combine them.

Step 1: Binary Representation and Initial Grouping (Column I)

We list all the given minterms, represent them in 4-bit binary format (w, x, y, z) , and group them by their index (number of 1s).

Group	Minterm	w x y z	Used
0	m_0	0 0 0 0	✓
1	m_1	0 0 0 1	✓
	m_2	0 0 1 0	✓
	m_8	1 0 0 0	✓
2	m_{10}	1 0 1 0	✓
3	m_{11}	1 0 1 1	✓
	m_{14}	1 1 1 0	✓
4	m_{15}	1 1 1 1	✓

Step 2: Combine Pairs of Minterms (Column II)

We compare minterms from adjacent groups (Group i and Group $i + 1$). If they differ by exactly one bit, we combine them, place a dash (-) in the differing position, and mark the original minterms as used (✓). Terms that cannot be combined with any term in the next stage are marked as Prime Implicants (PI).

Group	Combined Minterms	w x y z	Status / PI
0-1	(0, 1)	0 0 0 -	PI₁ (Uncombined)
	(0, 2)	0 0 - 0	✓
	(0, 8)	- 0 0 0	✓
1-2	(2, 10)	- 0 1 0	✓
	(8, 10)	1 0 - 0	✓
2-3	(10, 11)	1 0 1 -	✓
	(10, 14)	1 - 1 0	✓
3-4	(11, 15)	1 - 1 1	✓
	(14, 15)	1 1 1 -	✓

Note: The pair (0,1) cannot be combined with any term in the next stage because there is no matching term in Group 1-2 with a dash in the last position. Therefore, it becomes our first Prime Implicant.

Step 3: Combine Quads of Minterms (Column III)

Now, we compare the paired terms from adjacent groups in Column II. They can be combined if their dashes align and they differ by exactly one numeric bit.

Group	Combined Minterms	w x y z	Status / PI
0-1-2	(0, 2, 8, 10)	- 0 - 0	PI₂
	(0, 8, 2, 10)	- 0 - 0	Duplicate
2-3-4	(10, 11, 14, 15)	1 - 1 -	PI₃
	(10, 14, 11, 15)	1 - 1 -	Duplicate

Since no further groups can be compared, the process stops here. The unique terms in Column III, along with the uncombined term from Column II, form the complete set of Prime Implicants.

Final List of Prime Implicants

Translating the binary combinations back to Boolean variables ($0 \rightarrow$ complemented, $1 \rightarrow$ uncomplemented, $- \rightarrow$ omitted variable):

PI₁ from (0,1) $\implies 000- \implies \mathbf{w'x'y'}$

PI₂ from (0,2,8,10) $\implies -0-0 \implies \mathbf{x'z'}$

PI₃ from (10,11,14,15) $\implies 1-1- \implies \mathbf{wy}$

Optional Verification (Prime Implicant Chart):

Although the question only asks for the prime implicants, constructing the PI chart verifies coverage and essentiality:

PI	Term	0	1	2	8	10	11	14	15
PI₁*	w'x'y'	X	X						
PI₂*	x'z'	X		X	X	X			
PI₃*	wy					X	X	X	X

Every column has at least one minterm covered by only one PI (denoted by bold **Xs** in isolated columns). Thus, all three Prime Implicants are also Essential Prime Implicants.

The Prime Implicants are:
w'x'y', x'z', wy

Q4. Compare the Tabulation method and the Karnaugh map for minimizing Boolean functions based on their merits and demerits. (15 marks)

[CSE 205 | L-2/T-1 | 2019–20]

Answer

Comparison of Karnaugh Map and Tabulation (Quine-McCluskey) Methods

Both the Karnaugh Map (K-map) and the Tabulation method are widely used techniques for simplifying Boolean expressions to their minimal Sum-of-Products (SOP) or Product-of-Sums (POS) forms. However, they operate on entirely different principles—one being graphical and heuristic, the other being algebraic and algorithmic.

1. Karnaugh Map (K-map) Method

The Karnaugh map is a graphical representation of a truth table where adjacent cells differ by exactly one variable (Gray code sequence).

Merits:

- Visual Simplicity:** It relies on the human brain’s ability to recognize geometric patterns, making it highly intuitive and fast for manual solving.
- Immediate Results:** Grouping directly yields the simplified prime implicants without tedious intermediate algebraic steps.
- Don’t Care Handling:** “Don’t care” conditions can be effortlessly visualized and included in groups to maximize their size.
- Optimal for Small Variables:** It is generally the fastest and most efficient manual method for logic functions with 2, 3, or 4 variables.

Demerits:

- Scalability Issues:** It becomes exceedingly complex and visualization breaks down for 5 variables (requiring 3D visualization) and is practically unusable for 6 or more variables.
- Human Error:** Because it relies on human visual inspection, it is easy to miss optimal groupings or redundant terms, potentially yielding a sub-optimal result.
- Not Programmable:** The graphical, heuristic nature of K-maps makes them highly unsuitable for implementation in computer software.

2. Tabulation (Quine-McCluskey) Method

The Tabulation method is a systematic, step-by-step algorithmic procedure based on the exhaustive algebraic application of the theorem $XY + XY' = X$.

Merits:

- Highly Scalable:** It can seamlessly handle any number of variables (5, 6, or more) without a breakdown in methodology.
- Algorithmic and Programmable:** Because it uses strict tabular procedures, it is easily coded into software. Modern Electronic Design Automation (EDA) synthesis tools utilize algorithms derived from this method (e.g., the Espresso algorithm).
- Guaranteed Minimization:** It exhaustively generates all prime implicants and systematically uses a prime implicant chart to guarantee the absolute minimal expression. Visual errors are completely eliminated.

Demerits:

- Tedious Manual Process:** For a human, the exhaustive listing, comparing, and checking of minterms is heavily time-consuming and prone to arithmetic or clerical errors.
- Computational Complexity:** As the number of variables n increases, the number of minterms grows exponentially (2^n). The algorithm’s memory and processing requirements expand rapidly, making it computationally expensive for very large circuits.

Summary Comparison Table		
Feature	Karnaugh Map	Tabulation Method
Approach	Graphical / Visual mapping	Algorithmic / Tabular step-by-step
Ideal Number of Variables	Up to 4 (maximum 5)	5 or more (scales to any number)
Error Susceptibility	Prone to visual omission errors	Prone to clerical tracking errors manually
Computer Automation	Highly difficult / Impractical	Highly suitable and widely implemented
Guarantee of Minimum	Relies on user's visual accuracy	Mathematically guaranteed
Process Speed (Manual)	Very fast for small functions	Very slow and tedious

Q5. Using the Tabulation method, simplify the following Boolean function by first finding the essential prime implicants:

$$F(A, B, C, D) = \Sigma (1, 3, 4, 5, 10, 11, 12, 13, 14)$$

(28 marks)

[CSE 205 | L-2/T-1 | 2019–20]

Answer

Tabulation (Quine-McCluskey) Method

Step 1: Binary Representation and Initial Grouping (Column I)
List all minterms, convert them to 4-bit binary format (A, B, C, D), and group them by the number of 1s (index).

Group	Minterm	A	B	C	D	Used
1	m_1	0	0	0	1	✓
	m_4	0	1	0	0	✓
2	m_3	0	0	1	1	✓
	m_5	0	1	0	1	✓
	m_{10}	1	0	1	0	✓
	m_{12}	1	1	0	0	✓
3	m_{11}	1	0	1	1	✓
	m_{13}	1	1	0	1	✓
	m_{14}	1	1	1	0	✓

Step 2: Combine Pairs of Minterms (Column II)
Compare minterms from adjacent groups. If they differ by exactly one bit, combine them, place a dash (-) in the differing position, and mark the original minterms as used (✓).

Group	Combined Minterms	A	B	C	D	Status / PI
1-2	(1, 3)	0	0	-	1	PI ₂ ($A'B'D$)
	(1, 5)	0	-	0	1	PI ₃ ($A'C'D$)
	(4, 5)	0	1	0	-	✓
	(4, 12)	-	1	0	0	✓
2-3	(3, 11)	-	0	1	1	PI ₄ ($B'CD$)
	(5, 13)	-	1	0	1	✓
	(10, 11)	1	0	1	-	PI ₅ ($AB'C$)
	(10, 14)	1	-	1	0	PI ₆ (ACD')
	(12, 13)	1	1	0	-	✓
	(12, 14)	1	1	-	0	PI ₇ (ABD')

Terms that do not combine further into Column III are marked as Prime Implicants (PIs).

Step 3: Combine Quads of Minterms (Column III)
Compare the paired terms from adjacent groups in Column II.

Group	Combined Minterms	A	B	C	D	Status / PI
1-2-3	(4, 5, 12, 13)	-	1	0	-	PI ₁ (BC')
	(4, 12, 5, 13)	-	1	0	-	Duplicate

Step 4: Prime Implicant Chart
Construct a chart to map the Prime Implicants to the minterms they cover. Look for columns with a single 'X' to find the **Essential Prime Implicants (EPIs)**.

PI	Term	1	3	4	5	10	11	12	13	14
PI ₁ *	BC'			X	X			X	X	
PI ₂	A'B'D	X	X							
PI ₃	A'C'D	X			X					
PI ₄	B'CD		X				X			
PI ₅	AB'C					X	X			
PI ₆	ACD'					X				X
PI ₇	ABD'							X		X

Analysis of the Chart:

- Minterms 4 and 13 are covered *only* by PI₁. Thus, BC' is an Essential Prime Implicant.
- Selecting PI₁ covers minterms 4, 5, 12, and 13.

Step 5: Reduced Prime Implicant Chart

After removing the covered minterms (4, 5, 12, 13) and PI₁, we construct a reduced chart for the remaining minterms (1, 3, 10, 11, 14):

PI	Term	1	3	10	11	14
PI ₂	A'B'D	X	X			
PI ₃	A'C'D	X				
PI ₄	B'CD		X		X	
PI ₅	AB'C			X	X	
PI ₆	ACD'			X		X
PI ₇	ABD'					X

Applying Row Dominance:

- PI₂ dominates PI₃: PI₂ covers 1, 3 while PI₃ covers only 1. We can safely discard PI₃.
- PI₆ dominates PI₇: PI₆ covers 10, 14 while PI₇ covers only 14. We can safely discard PI₇.

After discarding PI₃ and PI₇, minterm 1 is covered only by PI₂, and minterm 14 is covered only by PI₆. Therefore, PI₂ and PI₆ must be selected.

- Selecting PI₂ (A'B'D) and PI₆ (ACD') covers minterms 1, 3, 10, and 14.
- The only remaining uncovered minterm is 11.
- Minterm 11 can be covered by either PI₄ (B'CD) OR PI₅ (AB'C). Both options yield a minimal expression with the same number of literals.

Final Minimized Boolean Expression

There are two equally valid minimal Sum-of-Products (SOP) expressions:

$$F(A, B, C, D) = \textcolor{green}{BC'} + \textcolor{green}{A'B'D} + \textcolor{green}{ACD'} + \textcolor{green}{B'CD}$$

OR

$$F(A, B, C, D) = \textcolor{green}{BC'} + \textcolor{green}{A'B'D} + \textcolor{green}{ACD'} + \textcolor{green}{AB'C}$$

Q6. Minimize the following function using the Quine-McCluskey (Q-M) method. Find essential prime implicants and all prime implicants:

$f(A, B, C, D, E) = \Sigma (0, 1, 2, 8, 9, 15, 17, 21, 24, 25, 27, 31)$

(20 marks)

[CSE 205 | L-2/T-1 | 2016–17]

Answer

Quine-McCluskey (Tabulation) Method

We are given a 5-variable Boolean function:

$$f(A, B, C, D, E) = \Sigma (0, 1, 2, 8, 9, 15, 17, 21, 24, 25, 27, 31)$$

Step 1: Binary Representation and Grouping (Column I)

We list all minterms, convert them to 5-bit binary representations, and group them according to the number of 1s (index) they contain.

Group	Minterm	A B C D E	Used
0	m_0	0 0 0 0 0	✓
1	m_1	0 0 0 0 1	✓
	m_2	0 0 0 1 0	✓
	m_8	0 1 0 0 0	✓
2	m_9	0 1 0 0 1	✓
	m_{17}	1 0 0 0 1	✓
	m_{24}	1 1 0 0 0	✓
3	m_{21}	1 0 1 0 1	✓
	m_{25}	1 1 0 0 1	✓
4	m_{15}	0 1 1 1 1	✓
	m_{27}	1 1 0 1 1	✓
5	m_{31}	1 1 1 1 1	✓

Step 2: Combine Pairs of Minterms (Column II)

We compare minterms from adjacent groups. If they differ by exactly one bit, we combine them, place a dash (-) in the differing position, and mark the original minterms as used (✓). Terms that cannot be combined further are Prime Implicants (PIs).

Group	Combined Minterms	A B C D E	Status / PI
0-1	(0, 1)	0 0 0 0 -	✓
	(0, 2)	0 0 0 - 0	PI₁ ($A'B'C'E'$)
	(0, 8)	0 - 0 0 0	✓
1-2	(1, 9)	0 - 0 0 1	✓
	(1, 17)	- 0 0 0 1	✓
	(8, 9)	0 1 0 0 -	✓
	(8, 24)	- 1 0 0 0	✓
2-3	(9, 25)	- 1 0 0 1	✓
	(17, 21)	1 0 - 0 1	PI₂ ($AB'D'E$)
	(17, 25)	1 - 0 0 1	✓
	(24, 25)	1 1 0 0 -	✓
3-4	(25, 27)	1 1 0 - 1	PI₃ ($ABC'E$)
4-5	(15, 31)	- 1 1 1 1	PI₄ ($BCDE$)
	(27, 31)	1 1 - 1 1	PI₅ ($ABDE$)

Step 3: Combine Quads of Minterms (Column III)

We now compare the paired terms from adjacent groups in Column II.

Group	Combined Minterms	A B C D E	Status / PI
0-1-2	(0, 1, 8, 9)	0 - 0 0 -	PI₆ ($A'C'D'$)
	(0, 8, 1, 9)	0 - 0 0 -	Duplicate
1-2-3	(1, 9, 17, 25)	- - 0 0 1	PI₇ ($C'D'E$)
	(1, 17, 9, 25)	- - 0 0 1	Duplicate
	(8, 9, 24, 25)	- 1 0 0 -	PI₈ ($BC'D'$)
	(8, 24, 9, 25)	- 1 0 0 -	Duplicate

Step 4: Prime Implicant Chart

We construct a chart to map all Prime Implicants (PIs) to the minterms they cover. Columns with a single ‘X’ denote **Essential Prime Implicants (EPIs)**.

PI	Term	0	1	2	8	9	15	17	21	24	25	27	31
PI₁*	A'B'C'E'	X		X									
PI₂*	AB'D'E							X	X				
PI ₃	ABC'E										X	X	
PI₄*	BCDE						X						X
PI ₅	ABDE											X	X
PI ₆	A'C'D'	X	X		X	X							
PI ₇	C'D'E		X			X		X			X		
PI₈*	BC'D'				X	X				X	X		

Analysis of the Chart:

- **Minterm 2** is covered only by PI_1 .
- **Minterm 21** is covered only by PI_2 .
- **Minterm 15** is covered only by PI_4 .
- **Minterm 24** is covered only by PI_8 .

Thus, **PI_1 , PI_2 , PI_4 , and PI_8** are **Essential Prime Implicants**. Selecting these covers minterms 0, 2, 8, 9, 15, 17, 21, 24, 25, and 31.

Step 5: Covering Remaining Minterms

The only uncovered minterms are **1** and **27**.

- Minterm **1** can be covered by either PI_6 ($A'C'D'$) or PI_7 ($C'D'E$). Both have 3 literals.
- Minterm **27** can be covered by either PI_3 ($ABC'E$) or PI_5 ($ABDE$). Both have 4 literals.

Final Results

1. List of All Prime Implicants:

$PI_1 = A'B'C'E'$

$PI_2 = AB'D'E$

$PI_3 = ABC'E$

$PI_4 = BCDE$

$PI_5 = ABDE$

$PI_6 = A'C'D'$

$PI_7 = C'D'E$

$PI_8 = BC'D'$

2. List of Essential Prime Implicants:

$A'B'C'E', AB'D'E, BCDE, BC'D'$

3. Final Minimized Boolean Function (One of the 4 optimal solutions):

$f(A, B, C, D, E) = A'B'C'E' + AB'D'E + BCDE + BC'D' + A'C'D' + ABC'E$

Note: Replacing $A'C'D'$ with $C'D'E$ and/or replacing $ABC'E$ with $ABDE$ yields three other equally minimal, mathematically correct expressions.

Q7. When is the tabulation method more preferable than the K-Map method for simplifying a Boolean function? **(5 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Answer

Preferability of the Tabulation (Quine-McCluskey) Method

While the Karnaugh Map (K-Map) is an excellent intuitive tool for Boolean simplification, the Tabulation method (also known as the Quine-McCluskey method) becomes significantly more preferable—and often mandatory—under the following distinct scenarios in digital logic design:

1. Functions with a Large Number of Variables ($n \geq 5$)

- **K-Map Limitation:** K-Maps rely entirely on human visual pattern recognition. They are highly efficient and reliable for 2, 3, and 4 variables. However, a 5-variable K-Map requires visualizing relationships across two separate 16-cell sub-maps, and a 6-variable map requires four 16-cell sub-maps. Beyond 4 variables, human spatial visualization breaks down, making the process painfully slow and highly prone to oversight.
- **Tabulation Advantage:** The Tabulation method does not depend on graphical or geometric visualization. It utilizes a rigorous, tabular, algebraic algorithm that scales identically regardless of the number of variables. It is the definitive choice for manual simplification of 5 or more variables.

2. Requirement for Computer Automation (Programmability)

- **K-Map Limitation:** Because the K-Map is a heuristic, graphical method dependent on human spatial reasoning (looping adjacent 1s), it is inherently difficult to translate directly into efficient computer code.
- **Tabulation Advantage:** The Quine-McCluskey method is strictly algorithmic, systematic, and deterministic. It consists of well-defined, repetitive procedural steps (exhaustive list generation, binary matching, and prime implicant chart reduction). This makes it perfectly suited for software implementation. Modern Electronic Design Automation (EDA) software and logic synthesizers rely heavily on algorithms derived from this method (such as the Espresso algorithm).

3. Guaranteeing Absolute Minimization without Human Error

- **K-Map Limitation:** For moderately complex functions (especially those heavily populated with 1s and “don’t care” conditions), a human designer using a K-Map might easily overlook the optimal grouping. Selecting a redundant group or missing a larger, non-obvious geometric grouping results in a sub-optimal circuit.

- **Tabulation Advantage:** The Tabulation method exhaustively generates *all* possible Prime Implicants (PIs) and then systematically isolates the Essential Prime Implicants (EPIs) using a rigorous cross-referencing chart. This mathematical rigor guarantees that the final Boolean Sum-of-Products (SOP) or Product-of-Sums (POS) expression is the absolute minimum, entirely eliminating the variable of human visual error.

Summary Conclusion

In professional and academic environments, the K-Map remains an excellent pedagogical tool and is preferred for quick, manual simplification of small logic circuits ($n \leq 4$). However, the **Tabulation method is explicitly preferable whenever a function exceeds 4 variables, when mathematical proof of minimal form is required, or when the minimization process is handled by a computer algorithm.**

Q8. Simplify the following Boolean function by using the tabulation method:

$F(A, B, C, D) = \Sigma (0, 1, 2, 8, 10, 11, 14, 15)$

(15 marks)

[CSE 205 | L-2/T-1 | 2011–12]

Answer

Tabulation (Quine-McCluskey) Method

We are given the 4-variable Boolean function:

$F(A, B, C, D) = \Sigma (0, 1, 2, 8, 10, 11, 14, 15)$

Step 1: Binary Representation and Initial Grouping (Column I)

Group the minterms based on the number of 1s (index) in their binary representation.

Group	Minterm	A	B	C	D	Used
0	m_0	0	0	0	0	✓
1	m_1	0	0	0	1	✓
	m_2	0	0	1	0	✓
	m_8	1	0	0	0	✓
2	m_{10}	1	0	1	0	✓
3	m_{11}	1	0	1	1	✓
	m_{14}	1	1	1	0	✓
4	m_{15}	1	1	1	1	✓

Step 2: Combine Pairs of Minterms (Column II)

Combine minterms from adjacent groups that differ by exactly one bit. Place a dash (-) in the position of change and check off (✓) the combined minterms.

Group	Combined Minterms	A	B	C	D	Status / PI
0-1	(0, 1)	0	0	0	-	PI₁ ($A'B'C'$)
	(0, 2)	0	0	-	0	✓
	(0, 8)	-	0	0	0	✓
1-2	(2, 10)	-	0	1	0	✓
	(8, 10)	1	0	-	0	✓
2-3	(10, 11)	1	0	1	-	✓
	(10, 14)	1	-	1	0	✓
3-4	(11, 15)	1	0	1	1	is 1 - 11 with 15 $\implies$ 1 - 1 1 ✓
	(14, 15)	1	1	1	-	✓

Note: Minterm pair (0, 1) cannot combine further and forms Prime Implicant 1.

Step 3: Combine Quads of Minterms (Column III)

Combine pairs from Column II that have matching dash (-) positions and differ by exactly one bit.

Group	Combined Minterms	A	B	C	D	Status / PI
0-1-2	(0, 2, 8, 10)	-	0	-	0	PI₂ ($B'D'$)
	(0, 8, 2, 10)	-	0	-	0	Duplicate
1-2-3	(2, 10, 8, 10)	No valid distinct match with same dash				
	(8, 10, 10, 14)	Does not match dash position				
2-3-4	(10, 11, 14, 15)	1	-	1	-	PI₃ (AC)
	(10, 14, 11, 15)	1	-	1	-	Duplicate

Let us review any uncombined pairs from Column II:

- $(2, 10) \rightarrow (-010)$ combined with $(8, 10) \rightarrow (10-0)$ is invalid because dashes are in different positions. However, $(0, 2) \rightarrow (00-0)$ and $(8, 10) \rightarrow (10-0)$ combine to form $(-0-0)$.
- $(2, 10) \rightarrow (-010)$ combines with $(14, 15)$? No. Let's check $(2, 10)$ and $(14, 15)$. No.
- Are there any leftover terms from Column II? Every single pair except $(0,1)$ was covered by the quads. Let's double check:
 - $(0,2), (0,8), (2,10), (8,10)$ form the quad $(0,2,8,10)$.
 - $(10,11), (10,14), (11,15), (14,15)$ form the quad $(10,11,14,15)$.

Thus, we have successfully identified all Prime Implicants.

List of Prime Implicants (PIs):

- $PI_1 = 000- \implies A'B'C'$
- $PI_2 = -0-0 \implies B'D'$
- $PI_3 = 1-1- \implies AC$

Step 4: Prime Implicant Chart

Construct the selection chart to identify Essential Prime Implicants (EPIs).

PI	Term	0	1	2	8	10	11	14	15
PI ₁ *	A'B'C'	X	X						
PI ₂ *	B'D'	X		X	X	X			
PI ₃ *	AC					X	X	X	X

EPI Analysis:

- Minterm **1** is covered uniquely by **PI₁**. Thus, $A'B'C'$ is an EPI.
- Minterms **11** and **14** are covered uniquely by **PI₃**. Thus, AC is an EPI.
- Selecting PI_1 and PI_3 leaves minterms **0, 2, 8, 10** uncovered.
- Minterms **0, 2, 8, 10** are all covered by **PI₂** ($B'D'$). Therefore, **PI₂** is also essential to complete the coverage of the function.

Since all minterms are covered by selecting all three Prime Implicants, they are all Essential Prime Implicants.

Final Minimized Boolean Expression

$$F(A, B, C, D) = A'B'C' + B'D' + AC$$

BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Combinational Logic

Adders, Subtractors, Comparators, Decoders, Encoders, MUX, DEMUX

S4 Combinational Logic

Number System & Arithmetic Operations

Q1. Describe the steps to find $M - N$ using $(r - 1)$'s complement, where M and N are two n -digit unsigned numbers and r is the base. (13 marks) [CSE 205 | L-2/T-1 | 2022-23, 2021-22]

Answer

Theoretical Foundation

Subtraction of two n -digit unsigned numbers, M (the minuend) and N (the subtrahend), in base r can be efficiently performed using the $(r - 1)$'s **complement** method. The $(r - 1)$'s complement of a number N is mathematically defined as $(r^n - 1) - N$. This method simplifies hardware implementation by replacing subtraction operations with addition and simple complementation logic.

Algorithmic Steps for computing $M - N$

- (a) **Find the $(r - 1)$'s complement of the subtrahend N :** Compute the $(r - 1)$'s complement of N . For a base r and n digits, this is algebraically represented as:

$$N_{(r-1)'s} = (r^n - 1) - N$$

Note: In practice (e.g., base 2 or base 10), this is achieved by subtracting each digit of N from $(r - 1)$. In binary (1 's complement), it is simply bitwise inversion.

- (b) **Add the minuend M to the $(r - 1)$'s complement of N :** Perform standard base- r addition of M and the result from Step 1. Let this intermediate sum be S :

$$\begin{aligned} S &= M + N_{(r-1)'s} \\ S &= M + (r^n - 1) - N \\ S &= (M - N) + r^n - 1 \end{aligned}$$

- (c) **Inspect the result for an end carry (carry-out from the most significant digit):** The evaluation of the intermediate sum S splits into two distinct cases depending on whether an end carry is produced.

- **Case A: An end carry occurs (implies $M \geq N$)**

If an end carry of 1 is generated, it indicates that the r^n term is present and the overall result is positive.

- Remove the end carry (which mathematically subtracts r^n).
- Add 1 to the least significant digit (LSD) of the result. This operation is known as an **end-around carry**.

Proof:

$$\begin{aligned} \text{Final Result} &= (S - r^n) + 1 \\ &= ((M - N) + r^n - 1 - r^n) + 1 \\ &= (M - N) - 1 + 1 \\ &= M - N \end{aligned}$$

- **Case B: No end carry occurs (implies $M < N$)**

If no end carry is generated, it indicates that the result is inherently negative, and the sum S is currently stored in its $(r - 1)$'s complement form.

- Take the $(r - 1)$'s complement of the intermediate sum S .
- Affix a negative sign to the final magnitude.

Proof:

$$\begin{aligned} \text{Final Magnitude} &= (r^n - 1) - S \\ &= (r^n - 1) - ((M - N) + r^n - 1) \\ &= -(M - N) \\ &= N - M \end{aligned}$$

Affixing the negative sign correctly yields $-(N - M) = M - N$.

Q2. Explain the difference between binary and gray encoding and the benefits of each. (9 marks) [CSE 205 | L-2/T-1 | 2022-23]

Answer

1. Fundamental Differences

The primary difference between Standard Binary and Gray encoding lies in their structural properties regarding bit weighting and bit transitions between consecutive numbers.

- Standard Binary Code:** This is a *weighted* (positional) number system. Each bit position has a specific mathematical weight (e.g., $2^0, 2^1, 2^2, \dots$). When transitioning between consecutive numbers, multiple bits can change simultaneously. For example, changing from decimal 3 to 4 requires three bit changes (**011** $\rightarrow$ **100**).
- Gray Code:** This is an *unweighted* number system. It is a cyclic code where successive values differ in exactly **one bit position**. For example, transitioning from decimal 3 to 4 requires only one bit change (**010** $\rightarrow$ **110**).

Decimal	Binary Code	Gray Code	Bits Changed (Gray)
0	000	000	-
1	001	001	1
2	010	011	1
3	011	010	1
4	100	110	1
5	101	111	1
6	110	101	1
7	111	100	1

2. Benefits of Binary Encoding

Binary encoding is the foundational language of digital logic systems due to its direct mapping to base-2 mathematics.

- Mathematical Computations:** Because it is a weighted system, standard arithmetic operations (addition, subtraction, multiplication, division) are natively straightforward to implement using standard combinational logic (e.g., adders and multipliers).
- Magnitude Comparison:** Comparing two binary numbers (greater than, less than) is simple because the position of the bits directly indicates their magnitude.
- Interfacing:** Most microprocessors, memories, and digital buses inherently process data in standard binary formats, requiring no conversion overhead for internal data routing.

3. Benefits of Gray Encoding

Gray code is primarily used to prevent spurious outputs from electromechanical switches and digital logic elements during state transitions.

- Glitch and Error Minimization:** Since only one bit changes between adjacent values, the system is immune to intermediate transient states (glitches). In standard binary, varying signal propagation delays during the transition from 011 to 100 might momentarily read as 111 or 000. Gray code entirely eliminates this multi-bit race condition.
- Electromechanical Rotary Encoders:** Gray code absolute encoders prevent angular position read errors. If the sensor is exactly on the boundary of two positions, the physical uncertainty only affects one bit, ensuring the read value is at most one position off.
- Logic Minimization (K-Maps):** Karnaugh Maps utilize Gray code ordering for their axes. The single-bit change ensures that physically adjacent cells represent logically adjacent minterms, allowing for visual identification of prime implicants.
- Clock Domain Crossing (CDC):** In asynchronous FIFO designs, read and write pointers are often converted to Gray code before being synchronized into different clock domains. Because only one bit changes at a time, the receiving clock domain will either see the old pointer value or the new pointer value, preventing data corruption.

Q3. Write the rules of subtraction for two n -digit numbers M and N in base r . Consider two cases: $M \geq N$ and $M < N$. Find the value of $9 - 6.9 - 9.8$ using the complement rule where the base is Hexadecimal. **(2+5 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

1. Rules of Subtraction using r 's Complement

To subtract two n -digit unsigned numbers, $M - N$, in a given base r , the r 's complement method replaces subtraction with addition. The r 's complement of the subtrahend N is mathematically defined as $r^n - N$. The subtraction procedure follows these steps:

(a) **Addition Phase:** Add the minuend M to the r 's complement of the subtrahend N .

$$S = M + (r^n - N) = r^n + (M - N)$$

(b) **Case 1:** $M \geq N$

94

- The addition will produce a sum $S \geq r^n$, generating an **end carry** out of the most significant digit.
- **Rule:** Discard the end carry (which effectively subtracts r^n). The remaining result is the positive value $(M - N)$.

(c) **Case 2:** $M < N$

- The addition will produce a sum $S < r^n$, and **no end carry** is generated.
- **Rule:** The absence of a carry indicates a negative result. To find the magnitude, take the r 's complement of the sum S and place a negative sign in front of it.

$$- [r^n - S] = - [r^n - (r^n + M - N)] = -(N - M) = M - N$$

2. Evaluation of $9 - 6.9 - 9.8$ in Hexadecimal (Base 16)

We evaluate the expression strictly left-to-right: $(9_{16} - 6.9_{16}) - 9.8_{16}$.

To ensure proper alignment for carries, we pad the numbers to 2 integer digits and 1 fractional digit (XX.Y).

- $M = 09.0_{16}$
- $N_1 = 06.9_{16}$
- $N_2 = 09.8_{16}$

Step 2A: First Subtraction, $A = 09.0_{16} - 06.9_{16}$

1. Find the 16's complement of $N_1 = 06.9_{16}$.

$$15\text{'s complement of } 06.9_{16} = (F - 0)(F - 6).(F - 9) = F9.6_{16}$$

$$16\text{'s complement of } 06.9_{16} = F9.6_{16} + 00.1_{16} = F9.7_{16}$$

2. Add M to the 16's complement of N_1 :

$$09.0_{16} + F9.7_{16} = 102.7_{16}$$

3. An end carry of 1 is generated (Case $M \geq N$). Discard the carry.

$$A = 02.7_{16}$$

Step 2B: Second Subtraction, $B = A - N_2 = 02.7_{16} - 09.8_{16}$

1. Find the 16's complement of $N_2 = 09.8_{16}$.

$$15\text{'s complement of } 09.8_{16} = (F - 0)(F - 9).(F - 8) = F6.7_{16}$$

$$16\text{'s complement of } 09.8_{16} = F6.7_{16} + 00.1_{16} = F6.8_{16}$$

2. Add A to the 16's complement of N_2 :

$$02.7_{16} + F6.8_{16} = F8.F_{16} \quad (\text{Note: } 7 + 8 = F_{16}, 2 + 6 = 8_{16})$$

3. No end carry is generated (Case $M < N$), which signifies a negative result. To find the final magnitude, we must take the 16's complement of the sum $F8.F_{16}$.

$$15\text{'s complement of } F8.F_{16} = (F - F)(F - 8).(F - F) = 07.0_{16}$$

$$16\text{'s complement of } F8.F_{16} = 07.0_{16} + 00.1_{16} = 07.1_{16}$$

4. Affix the negative sign to the final magnitude:

$$B = -07.1_{16} = -7.1_{16}$$

Final Answer: The value of $9_{16} - 6.9_{16} - 9.8_{16}$ is **-7.1₁₆**.

Q4. What is the difference between r 's complement and $(r - 1)$'s complement (where r is the base)? **(2 marks)** [CSE 205 | L-2/T-1 | 2016–17]

Answer

1. Mathematical Definitions

Let N be a positive number in base r having n integer digits and m fractional digits.

- **$(r - 1)$'s Complement (Diminished Radix Complement):** Mathematically, it is defined as:

$$(r - 1)\text{'s Complement} = r^n - r^{-m} - N$$

In practice, this is obtained by subtracting every single digit of the number N from the maximum possible digit value in that base, which is $(r - 1)$.

- **r 's Complement (Radix Complement):** Mathematically, it is defined as:

$$r\text{'s Complement} = r^n - N$$

It can be straightforwardly obtained by finding the $(r - 1)$'s complement and then adding 1 to the least significant digit (LSD), represented by r^{-m} .

$$r\text{'s Complement} = (r - 1)\text{'s Complement} + r^{-m}$$

2. Key Operational Differences

Characteristic	$(r - 1)$'s Complement	r 's Complement
Computation Method	Simple digit-by-digit operation. For base-2, it is a simple bitwise inversion (NOT operation).	Requires generating the $(r - 1)$'s complement first, then mathematically adding 1 to the LSD.
Hardware Complexity	Generating the complement is fast (no carry propagation), but arithmetic operations require an extra adder step.	Slower to generate (requires carry propagation through an adder), but simplifies arithmetic logic later.
Representation of Zero	Has two representations of zero: Positive Zero (+0) and Negative Zero (−0).	Has a unique , single representation of zero.
Subtraction Logic ($M - N$)	If an end carry is generated, it must be added back to the LSB. This is called an end-around carry .	If an end carry is generated, it is simply discarded (ignored).

3. Illustrative Example ($r = 2$, Binary Base)

Consider the binary number $N = 10110_2$. Here, the base is $r = 2$, the number of integer digits is $n = 5$, and there are no fractional digits ($m = 0$, so $r^{-m} = 1$).

- **Finding the 1's Complement ($(r - 1)$'s Complement):**
Subtract each bit from 1 (which is equivalent to flipping every bit):

$$1\text{'s Complement} = 01001_2$$

- **Finding the 2's Complement (r 's Complement):**
Add 1 to the Least Significant Bit (LSB) of the 1's complement:

$$\begin{aligned} 2\text{'s Complement} &= 01001_2 + 1_2 \\ &= 01010_2 \end{aligned}$$

Q5. Convert $(0.625)_{10}$ to binary. Show every step of your calculation. (4 marks)

[CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Method: Successive Multiplication by 2

To convert a decimal fraction to binary, we use the method of successive multiplication by the target base (which is 2 for binary). The procedure is as follows:

- (a) Multiply the decimal fractional part by 2.
- (b) Record the integer part of the resulting product. This integer becomes a digit in the binary fraction.
- (c) Take the new fractional part and multiply it by 2 again.
- (d) Repeat the process until the fractional part becomes 0.00 (exact conversion) or until the desired precision is reached.
- (e) Read the recorded integer parts from **top to bottom** (first generated to last generated) to form the binary fraction.

2. Step-by-Step Calculation

We apply the successive multiplication method to $(0.625)_{10}$:

$0.625 \times 2 = 1.25$

$\implies$ Integer part: **1**,

Remaining fraction: 0.25

$0.25 \times 2 = 0.50$

$\implies$ Integer part: **0**,

Remaining fraction: 0.50

$0.50 \times 2 = 1.00$

$\implies$ Integer part: **1**,

Remaining fraction: 0.00

The fractional part has now reached 0.00, indicating that the conversion terminates precisely.

3. Final Result

Reading the generated integer parts from the first step down to the last step (top to bottom), we get the sequence **1, 0, 1**.

Therefore, the decimal fraction $(0.625)_{10}$ is exactly equal to the binary fraction:

$$(0.625)_{10} = (0.101)_2$$

Q6. A decimal number N is represented in a weighted binary code as 0011 0100 1010 1100. Write the decimal digits of N , if the following weights are used for the weighted binary code: 6, 3, 1, 1. **(4 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Method of Decoding

In a weighted binary code, each bit position in a sequence is assigned a specific numerical weight. For a 4-bit group $B = b_3b_2b_1b_0$ with corresponding weights $W = w_3, w_2, w_1, w_0$, the decimal equivalent digit D is calculated by computing the sum of the products of each bit and its respective weight:

$$D = (b_3 \times w_3) + (b_2 \times w_2) + (b_1 \times w_1) + (b_0 \times w_0)$$

Given the weights are **6, 3, 1, 1**, the formula becomes:

$$D = (b_3 \times 6) + (b_2 \times 3) + (b_1 \times 1) + (b_0 \times 1)$$

The given 16-bit binary sequence is 0011 0100 1010 1100. We will divide this sequence into four 4-bit nibbles, with each nibble representing a single decimal digit.

2. Step-by-Step Evaluation

We evaluate each 4-bit group from left to right:

- First Digit (D_1): 0011**
$$\begin{aligned} D_1 &= (0 \times 6) + (0 \times 3) + (1 \times 1) + (1 \times 1) \\ &= 0 + 0 + 1 + 1 \\ &= \mathbf{2} \end{aligned}$$
- Second Digit (D_2): 0100**
$$\begin{aligned} D_2 &= (0 \times 6) + (1 \times 3) + (0 \times 1) + (0 \times 1) \\ &= 0 + 3 + 0 + 0 \\ &= \mathbf{3} \end{aligned}$$
- Third Digit (D_3): 1010**
$$\begin{aligned} D_3 &= (1 \times 6) + (0 \times 3) + (1 \times 1) + (0 \times 1) \\ &= 6 + 0 + 1 + 0 \\ &= \mathbf{7} \end{aligned}$$
- Fourth Digit (D_4): 1100**
$$\begin{aligned} D_4 &= (1 \times 6) + (1 \times 3) + (0 \times 1) + (0 \times 1) \\ &= 6 + 3 + 0 + 0 \\ &= \mathbf{9} \end{aligned}$$

3. Summary Table

4-bit Group ($b_3b_2b_1b_0$)	Calculation ($6 \cdot b_3 + 3 \cdot b_2 + 1 \cdot b_1 + 1 \cdot b_0$)	Decimal Digit
0011	$0 + 0 + 1 + 1$	2
0100	$0 + 3 + 0 + 0$	3
1010	$6 + 0 + 1 + 0$	7
1100	$6 + 3 + 0 + 0$	9

Final Answer

The decimal digits of N are 2, 3, 7, and 9. Therefore, the decimal number N is **2379**.

Q7. You are given two numbers: $(305.E)_{16}$ and $(109.6875)_{10}$.

- (a) Convert the given numbers into binary.
- (b) Subtract the latter number from the former using binary arithmetic.
- (c) Convert the result of the subtraction into octal.

(15 marks)

[CSE 205 | L-2/T-1 | 2013–14]

Answer

Part (a): Conversion to Binary

1. Converting $(305.E)_{16}$ to binary: Each hexadecimal digit corresponds directly to a 4-bit binary sequence.

$$3_{16} \longrightarrow 0011_2$$

$$0_{16} \longrightarrow 0000_2$$

$$5_{16} \longrightarrow 0101_2$$

$$E_{16} \longrightarrow 1110_2$$

Combining these nibbles gives:

$$(305.E)_{16} = (0011\ 0000\ 0101.1110)_2 = (\mathbf{1100000101.1110})_2$$

2. Converting $(109.6875)_{10}$ to binary: We split the decimal number into its integer and fractional parts.

Integer Part (109_{10}) : Successive division by 2.

$$109 \div 2 = 54 \quad \text{Remainder: } \mathbf{1} \text{ (LSB)}$$

$$54 \div 2 = 27 \quad \text{Remainder: } \mathbf{0}$$

$$27 \div 2 = 13 \quad \text{Remainder: } \mathbf{1}$$

$$13 \div 2 = 6 \quad \text{Remainder: } \mathbf{1}$$

$$6 \div 2 = 3 \quad \text{Remainder: } \mathbf{0}$$

$$3 \div 2 = 1 \quad \text{Remainder: } \mathbf{1}$$

$$1 \div 2 = 0 \quad \text{Remainder: } \mathbf{1} \text{ (MSB)}$$

Reading from bottom to top, the integer part is $(1101101)_2$.

Fractional Part (0.6875_{10}) : Successive multiplication by 2.

$$0.6875 \times 2 = \mathbf{1.375} \quad \implies \text{Integer: } \mathbf{1} \text{ (MSB)}$$

$$0.3750 \times 2 = \mathbf{0.750} \quad \implies \text{Integer: } \mathbf{0}$$

$$0.7500 \times 2 = \mathbf{1.500} \quad \implies \text{Integer: } \mathbf{1}$$

$$0.5000 \times 2 = \mathbf{1.000} \quad \implies \text{Integer: } \mathbf{1} \text{ (LSB)}$$

Reading from top to bottom, the fractional part is $(.1011)_2$.

Combining both parts gives:

$$(109.6875)_{10} = (\mathbf{1101101.1011})_2$$

Part (b): Binary Subtraction

We need to compute $X - Y$, where:

$$X = 1100000101.1110_2$$

$$Y = 1101101.1011_2$$

To perform the subtraction using the **2's complement method**, we first equalize the number of bits in X and Y by padding with leading zeros for the integer part and trailing zeros for the fractional part (if necessary). We will use a 14-bit representation (10 integer bits, 4 fractional bits).

$$X = 1100000101.1110$$

$$Y = 0001101101.1011$$

Step 1: Find the 2's complement of Y :

$$1\text{'s complement of } Y = 1110010010.0100$$

$$\text{Add 1 to LSB (0.0001)} = +0000000000.0001$$

$$2\text{'s complement of } Y = 1110010010.0101$$

Step 2: Add X to the 2's complement of Y :

$$\begin{array}{r} 1100000101.1110 \quad (X) \\ + 1110010010.0101 \quad (2\text{'s complement of } Y) \\ \hline 1\ 1010011000.0011 \quad (\text{Sum}) \end{array}$$

Step 3: Discard the end carry: Since an end carry is generated (1), the result is positive, and the carry is discarded.

$$X - Y = 1010011000.0011_2$$

Part (c): Conversion of Result to Octal

To convert the binary result 1010011000.0011_2 to octal, we group the bits into sets of three, starting from the binary point and moving outwards (left for the integer part, right for the fractional part). Pad with zeros where necessary.

Integer part grouping: $\underbrace{001}_1 \underbrace{010}_2 \underbrace{011}_3 \underbrace{000}_0$

Fractional part grouping: $.\underbrace{001}_1 \underbrace{100}_4$

Replacing each 3-bit group with its corresponding octal digit yields the final answer:

$$(1010011000.0011)_2 = (1230.14)_8$$

Q8. Perform the following binary subtraction: $11000011 - 10101010$. (6 marks)

[CSE 205 | L-2/T-1 | 2012–13]

Answer

1. Method A: 2's Complement Subtraction (Standard Digital Logic Approach)

In digital hardware, subtraction $M - N$ is typically implemented using 2's complement addition: $M + (2\text{'s complement of } N)$. Given:

$$\begin{aligned} M &= 11000011_2 \\ N &= 10101010_2 \end{aligned}$$

Step 1: Find the 2's complement of the subtrahend N

$$\begin{aligned} \text{1's complement of } N \text{ (invert all bits)} &= 01010101_2 \\ \text{Add 1 to the LSB} &= \underline{+0000001_2} \\ \text{2's complement of } N &= 01010110_2 \end{aligned}$$

Step 2: Add M and the 2's complement of N

$$\begin{array}{r} 11000011_2 \quad (M) \\ + 01010110_2 \quad (2\text{'s complement of } N) \\ \hline 1\,00011001_2 \quad (\text{Sum}) \end{array}$$

Step 3: Evaluate the end carry An end carry of 1 is generated, indicating that the result is positive. We discard the carry to obtain the final result.

$$\text{Result} = 00011001_2$$

2. Method B: Direct Binary Subtraction (Borrow Method)

We can also perform direct bit-by-bit subtraction. When subtracting 1 from 0, a borrow is required from the next most significant bit that contains a 1.

$$\begin{array}{r} 1\cancel{1}^0\cancel{0}^1\cancel{0}^10011_2 \quad (\text{Borrows propagate from bit 6 down to bit 3}) \\ - 10101010_2 \\ \hline 00011001_2 \end{array}$$

Bit-by-bit breakdown:

- **Bit 0:** $1 - 0 = 1$
- **Bit 1:** $1 - 1 = 0$
- **Bit 2:** $0 - 0 = 0$
- **Bit 3:** $0 - 1 \implies$ Requires a borrow. The nearest 1 is at Bit 6. Borrowing leaves Bit 6 as 0, makes Bits 5 and 4 as 1, and gives Bit 3 a value of 2_{10} (10_2). $2 - 1 = 1$.
- **Bit 4:** $1(\text{borrowed}) - 0 = 1$
- **Bit 5:** $1(\text{borrowed}) - 1 = 0$
- **Bit 6:** $0(\text{after borrow out}) - 0 = 0$
- **Bit 7:** $1 - 1 = 0$

3. Verification (Decimal Conversion)

To ensure mathematical accuracy, we can verify the result using base-10 equivalent values:

$$\begin{aligned} M &= 11000011_2 = 128 + 64 + 2 + 1 = 195_{10} \\ N &= 10101010_2 = 128 + 32 + 8 + 2 = 170_{10} \\ M - N &= 195_{10} - 170_{10} = \mathbf{25_{10}} \end{aligned}$$

Converting the result back to binary verifies our answer:

$$25_{10} = 16 + 8 + 1 = \mathbf{00011001_2}$$

Final Answer: $11000011_2 - 10101010_2 = \mathbf{00011001_2}$

Q9. Using 2's complement, subtract decimal -70 from decimal -50 . **(9 marks)**

[CSE 205 | L-2/T-1 | 2012–13]

Answer

1. Problem Formulation and Bit Length

We are asked to evaluate $M - N$, where:

$$\begin{aligned} M &= -50_{10} \\ N &= -70_{10} \end{aligned}$$

The mathematical operation is $(-50_{10}) - (-70_{10})$, which is mathematically equivalent to $(-50_{10}) + (+70_{10})$.

To perform this in binary, we must determine the necessary bit width. The expected result is $+20_{10}$. The operands and the result all fall within the range $[-128, +127]$, which is exactly the representable range for **8-bit signed 2's complement** numbers. Therefore, we will use an 8-bit representation.

2. 8-Bit 2's Complement Representation of Operands

Step 2A: Representing $M = -50_{10}$

- Find the binary magnitude of $+50$: $50 = 32 + 16 + 2 \Rightarrow \mathbf{00110010_2}$
- Find the 1's complement (invert bits): $\mathbf{11001101_2}$
- Find the 2's complement (add 1):

$$11001101_2 + 1_2 = \mathbf{11001110_2} \quad (\text{This is } -50_{10})$$

Step 2B: Representing $N = -70_{10}$

- Find the binary magnitude of $+70$: $70 = 64 + 4 + 2 \Rightarrow \mathbf{01000110_2}$
- Find the 1's complement (invert bits): $\mathbf{10111001_2}$
- Find the 2's complement (add 1):

$$10111001_2 + 1_2 = \mathbf{10111010_2} \quad (\text{This is } -70_{10})$$

3. Subtraction using 2's Complement Addition

In digital logic, the subtraction $M - N$ is performed by adding the 2's complement of the subtrahend (N) to the minuend (M).

$$M - N = M + (\text{2's complement of } N)$$

Since $N = -70_{10}$, its 2's complement is simply $+70_{10}$, which we already found to be $\mathbf{01000110_2}$.

Now, we add M and the 2's complement of N :

$$\begin{array}{r} 11001110_2 \quad (M = -50_{10}) \\ + \quad 01000110_2 \quad (2's \text{ comp of } N = +70_{10}) \\ \hline \mathbf{1 \quad 00010100_2} \quad (\text{Sum}) \end{array}$$

4. Evaluation of Result

- End Carry:** An end carry of **1** is generated out of the Most Significant Bit (MSB). In signed 2's complement arithmetic, when adding numbers of opposite signs, overflow is impossible, and any carry generated out of the sign bit is safely **discarded**.
- Final Binary Result:** $\mathbf{00010100_2}$

Verification: Since the MSB (sign bit) of the result is 0, the number is positive. We convert the binary magnitude directly to decimal to verify correctness:

$$\begin{aligned} 00010100_2 &= (1 \times 2^4) + (1 \times 2^2) \\ &= 16 + 4 \\ &= \mathbf{+20_{10}} \end{aligned}$$

This mathematically matches the expected decimal calculation: $(-50) - (-70) = +20$.

Q10. Convert $(41.6875)_{10}$ to binary. (5 marks)

[CSE 205 | L-2/T-1 | 2011–12]

Answer

1. Integer Part Conversion: Successive Division

To convert the integer part $(41)_{10}$ to binary, we repeatedly divide by the target base (2) and record the remainders. The first remainder represents the Least Significant Bit (LSB) and the final remainder represents the Most Significant Bit (MSB).

$$\begin{aligned} 41 \div 2 &= 20 & \text{Remainder: } 1 & \text{ (LSB)} \\ 20 \div 2 &= 10 & \text{Remainder: } 0 & \\ 10 \div 2 &= 5 & \text{Remainder: } 0 & \\ 5 \div 2 &= 2 & \text{Remainder: } 1 & \\ 2 \div 2 &= 1 & \text{Remainder: } 0 & \\ 1 \div 2 &= 0 & \text{Remainder: } 1 & \text{ (MSB)} \end{aligned}$$

Reading the remainders from bottom to top (MSB to LSB), we obtain:

$$(41)_{10} = (101001)_2$$

2. Fractional Part Conversion: Successive Multiplication

To convert the fractional part $(0.6875)_{10}$ to binary, we repeatedly multiply by the target base (2) and record the integer portion of the product. The first generated integer represents the Most Significant Bit (MSB) of the fraction, closest to the radix point.

$$\begin{aligned} 0.6875 \times 2 &= 1.3750 & \Rightarrow \text{Integer part: } 1 & \text{ (MSB)} \\ 0.3750 \times 2 &= 0.7500 & \Rightarrow \text{Integer part: } 0 & \\ 0.7500 \times 2 &= 1.5000 & \Rightarrow \text{Integer part: } 1 & \\ 0.5000 \times 2 &= 1.0000 & \Rightarrow \text{Integer part: } 1 & \text{ (LSB)} \end{aligned}$$

The fractional part has reached exactly .0000, so the conversion terminates. Reading the integer parts from top to bottom, we obtain:

$$(0.6875)_{10} = (0.1011)_2$$

3. Final Result

Combining the binary integer and fractional parts together:

$$\begin{aligned} (41)_{10} + (0.6875)_{10} &= (101001)_2 + (0.1011)_2 \\ (41.6875)_{10} &= (101001.1011)_2 \end{aligned}$$

Q11. Perform the subtraction with the following binary unsigned numbers using (i) 2's complement, (ii) 1's complement: $11010 - 1101$. (10 marks)

[CSE 205 | L-2/T-1 | 2011–12]

Answer

1. Initialization and Bit Equalization

We are required to perform the subtraction $M - N$, where:

$$\begin{aligned} M &= 11010_2 & \text{(5-bit number)} \\ N &= 1101_2 & \text{(4-bit number)} \end{aligned}$$

Before applying complement arithmetic, both numbers must have the same number of bits. We pad the subtrahend N with a leading zero to equalize the bit lengths to 5 bits.

$$\begin{aligned} M &= 11010_2 \\ N &= 01101_2 \end{aligned}$$

Part (i): Subtraction using 2's Complement

The operation $M - N$ is evaluated as: $M + (2\text{'s complement of } N)$.

Step 1: Find the 2's complement of N

$$\begin{aligned} \text{1's complement of } N \text{ (invert all bits)} &= 10010_2 \\ \text{Add 1 to the LSB} &= \underline{+00001_2} \\ \text{2's complement of } N &= 10011_2 \end{aligned}$$

Step 2: Add M to the 2's complement of N

$$\begin{array}{r} 11010_2 \quad (M) \\ + 10011_2 \quad (2\text{'s complement of } N) \\ \hline 1 \quad 01101_2 \quad (\text{Sum}) \end{array}$$

Step 3: Evaluate the end carry An end carry of **1** is generated. In 2’s complement unsigned subtraction, a generated end carry indicates that the result is strictly positive and the carry is simply **discarded**.

Final Result = **01101₂**

Part (ii): Subtraction using 1’s Complement

The operation $M - N$ is evaluated as: $M + (1\text{'s complement of } N)$, followed by an end-around carry logic step.

Step 1: Find the 1’s complement of N

1’s complement of $N = 10010_2$

Step 2: Add M to the 1’s complement of N

11010₂ (M)

+ 10010₂ (1’s complement of N)

1 01100₂ (Intermediate Sum)

Step 3: Evaluate the end carry and perform End-Around Carry An end carry of **1** is generated. In 1’s complement unsigned subtraction, this indicates a positive result, but the carry must be added back to the Least Significant Bit (LSB) of the intermediate sum. This is known as an **end-around carry**.

01100₂ (Intermediate Sum)

+ 1₂ (End-Around Carry)

01101₂ (Final Result)

3. Verification (Decimal Translation)

We can mathematically verify both methods by converting to decimal:

$M = 11010_2 = 16 + 8 + 2 = 26_{10}$

$N = 01101_2 = 8 + 4 + 1 = 13_{10}$

$M - N = 26_{10} - 13_{10} = \mathbf{13_{10}}$

Converting the decimal result back to binary:

$13_{10} = 8 + 4 + 1 = \mathbf{01101_2}$

Both methods successfully yield the correct binary result.

Code Converter

Q1. A consensus circuit is a combinational circuit whose output is equal to 1 if exactly two inputs are 1, and 0 otherwise. Design a 3-input consensus circuit by finding the circuit’s truth table, simplified Boolean equation and a circuit diagram. **(12 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Answer

1. Problem Formulation and Truth Table

A consensus circuit evaluates its inputs and produces a logic HIGH (1) if and only if exactly two of its inputs are HIGH. For a 3-input circuit with inputs A , B , and C , and output Y , we systematically evaluate all $2^3 = 8$ possible input combinations.

Inputs			Output	
A	B	C	Y	Condition Met?
0	0	0	0	None are 1
0	0	1	0	Only one is 1
0	1	0	0	Only one is 1
0	1	1	1	Exactly two are 1 (B, C)
1	0	0	0	Only one is 1
1	0	1	1	Exactly two are 1 (A, C)
1	1	0	1	Exactly two are 1 (A, B)
1	1	1	0	Three are 1

From the truth table, the sum-of-minterms expression is:

$Y(A, B, C) = \sum m(3, 5, 6)$

2. Boolean Equation Simplification

To find the simplified Boolean equation, we map the minterms onto a 3-variable Karnaugh Map.

		BC			
		00	01	11	10
A	0			1	
	1		1		1

Analysis of the K-Map: Observing the plotted 1s at cells 3 (011), 5 (101), and 6 (110), there are no adjacent cells containing a 1. Because no groups of two, four, or eight can be formed, the Boolean expression cannot be further reduced using standard Sum-of-Products (SOP) grouping.

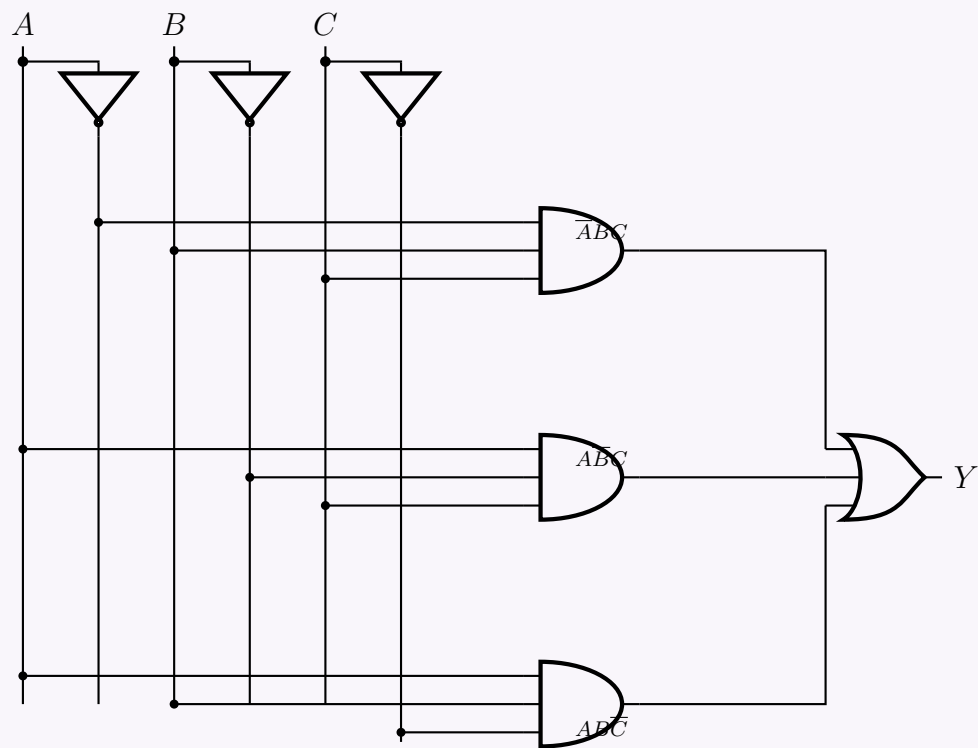
Thus, the minimal SOP Boolean equation is directly the canonical sum of products:

$$Y = m_3 + m_5 + m_6$$
$$Y = \overline{A}BC + A\overline{B}C + AB\overline{C}$$

Note: While not reducible in pure SOP form, algebraic manipulation using XOR logic is possible (e.g., $Y = C(A \oplus B) + AB\overline{C}$), but standard combinational design typically defaults to the minimal SOP utilizing AND, OR, and NOT gates.

3. Logic Circuit Diagram

We implement the minimal SOP expression using a standard logic bus architecture. Three 3-input AND gates generate the required product terms, which are then summed by a 3-input OR gate.



Q2. Design a combinational circuit that converts a three-bit gray code to a three-bit binary number. Use only a minimum number of gates in your design. (20 marks)

CSE 205 | L-2/T-1 | 2022–23

Answer

1. Truth Table Formulation

A Gray code to binary converter takes a Gray code input (G_2, G_1, G_0) and produces a binary number output (B_2, B_1, B_0). We evaluate all 8 possible combinations in standard binary order for the inputs to easily derive our K-maps.

Gray Code Input			Binary Output		
G_2	G_1	G_0	B_2	B_1	B_0
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	1	1
1	0	1	1	1	0
1	1	0	1	0	0
1	1	1	1	0	1

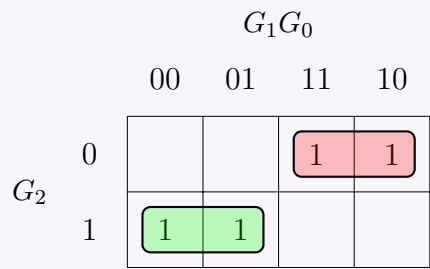
2. Karnaugh Map Analysis & Boolean Simplification

We derive the simplified Boolean expressions for each output bit (B_2, B_1, B_0).

For MSB (B_2): By direct inspection of the truth table, B_2 exactly matches the input G_2 .

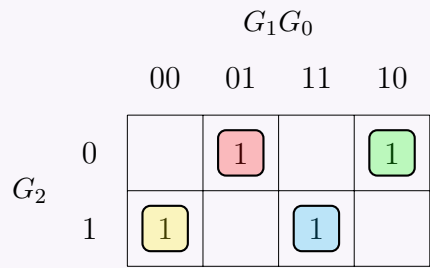
$$B_2 = G_2$$

For Middle Bit (B_1): We plot the minterms for B_1 ($\Sigma m(2, 3, 4, 5)$) on a K-map.



$$B_1 = \overline{G_2}G_1 + G_2\overline{G_1}$$
$$B_1 = G_2 \oplus G_1 \quad (\text{XOR Definition})$$

For LSB (B_0): We plot the minterms for B_0 ($\Sigma m(1, 2, 4, 7)$) on a K-map.



The resulting “checkerboard” pattern indicates a multi-variable XOR relationship:

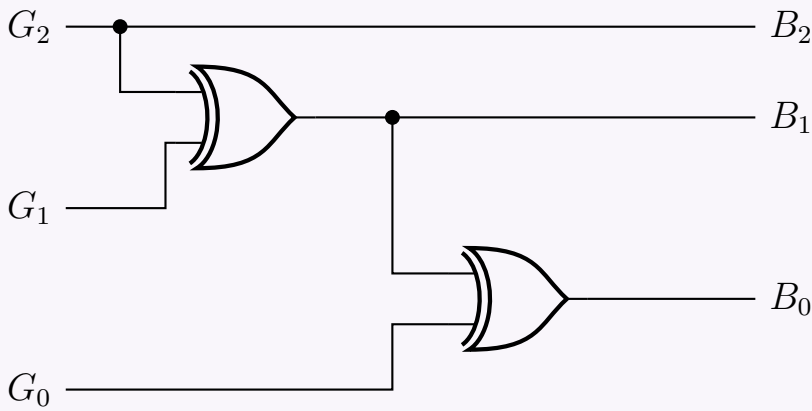
$$B_0 = \overline{G_2}\overline{G_1}G_0 + \overline{G_2}G_1\overline{G_0} + G_2\overline{G_1}\overline{G_0} + G_2G_1G_0$$
$$B_0 = \overline{G_2}(\overline{G_1}G_0 + G_1\overline{G_0}) + G_2(\overline{G_1}\overline{G_0} + G_1G_0) \quad (\text{Distributive Law})$$
$$B_0 = \overline{G_2}(G_1 \oplus G_0) + G_2(\overline{G_1} \oplus \overline{G_0}) \quad (\text{XOR and XNOR Definitions})$$
$$B_0 = G_2 \oplus G_1 \oplus G_0 \quad (\text{XOR Definition})$$

3. Logic Circuit Implementation (Minimized Gate Count)

To minimize the number of gates, we can cascade 2-input XOR gates by reusing the generated B_1 signal for the calculation of B_0 :

$$B_0 = (G_2 \oplus G_1) \oplus G_0 = B_1 \oplus G_0$$

This reduces the circuit requirement to exactly **two 2-input XOR gates**.



Q3. Design a combinational circuit that converts a decimal digit represented with “Random Code” to “BCD Code”. Use the K-Map method for logic simplification and show the circuit diagram.

Decimal Digit	Random Code ABCD	BCD Code wxyz
0	0000	0000
1	0011	0001
2	0101	0010
3	0110	0011
4	1010	0100
5	1011	0101
6	1100	0110
7	1101	0111
8	1110	1000
9	1111	1001

(20 marks)

[CSE 205 | L-2/T-1 | 2021–22]

Answer

1. Identification of Minterms and Don't Cares

From the provided table, we define the combinational circuit with inputs A, B, C, D and outputs w, x, y, z . The input is a 4-bit "Random Code". Only 10 specific input combinations are valid (representing decimal digits 0-9). The remaining 6 combinations are invalid and will be treated as **Don't Cares** (d).

• **Valid Inputs (Minterms):** $m_0, m_3, m_5, m_6, m_{10}, m_{11}, m_{12}, m_{13}, m_{14}, m_{15}$

• **Don't Cares** (d): $\sum d(1, 2, 4, 7, 8, 9)$

We can now list the sum of minterms for each output bit of the BCD code (w, x, y, z) by observing when each output is '1' in the truth table:

$$w(A, B, C, D) = \sum m(14, 15) + \sum d(1, 2, 4, 7, 8, 9)$$
$$x(A, B, C, D) = \sum m(10, 11, 12, 13) + \sum d(1, 2, 4, 7, 8, 9)$$
$$y(A, B, C, D) = \sum m(5, 6, 12, 13) + \sum d(1, 2, 4, 7, 8, 9)$$
$$z(A, B, C, D) = \sum m(3, 6, 11, 13, 15) + \sum d(1, 2, 4, 7, 8, 9)$$

2. K-Map Simplification

We use 4-variable Karnaugh Maps to find the minimal Sum of Products (SOP) expressions for each output.

K-Map for w

CD

00011110

00

01

11

10

00

01

11

10

-

-

-

-

1

1

-

-

Group 1 (Red): ABC

Simplified: $w = ABC$

K-Map for x

CD

00011110

00

01

11

10

00

01

11

10

-

-

-

-

1

1

-

-

1

1

Group 1 (Red): $A\bar{C}$

Group 2 (Green): $A\bar{B}$

Simplified: $x = A\bar{C} + A\bar{B}$

K-Map for y

CD

00011110

00

01

11

10

00

01

11

10

-

-

-

1

-

1

1

1

-

-

Group 1 (Red): $B\bar{C}$

Group 2 (Green): $\bar{A}B$

Simplified: $y = B\bar{C} + \bar{A}B$

K-Map for z

CD

00011110

00

01

11

10

00

01

11

10

-

1

-

-

-

1

1

1

-

-

1

Group 1 (Red): AD

Group 2 (Green): $\bar{A}C$

Simplified: $z = AD + \bar{A}C$

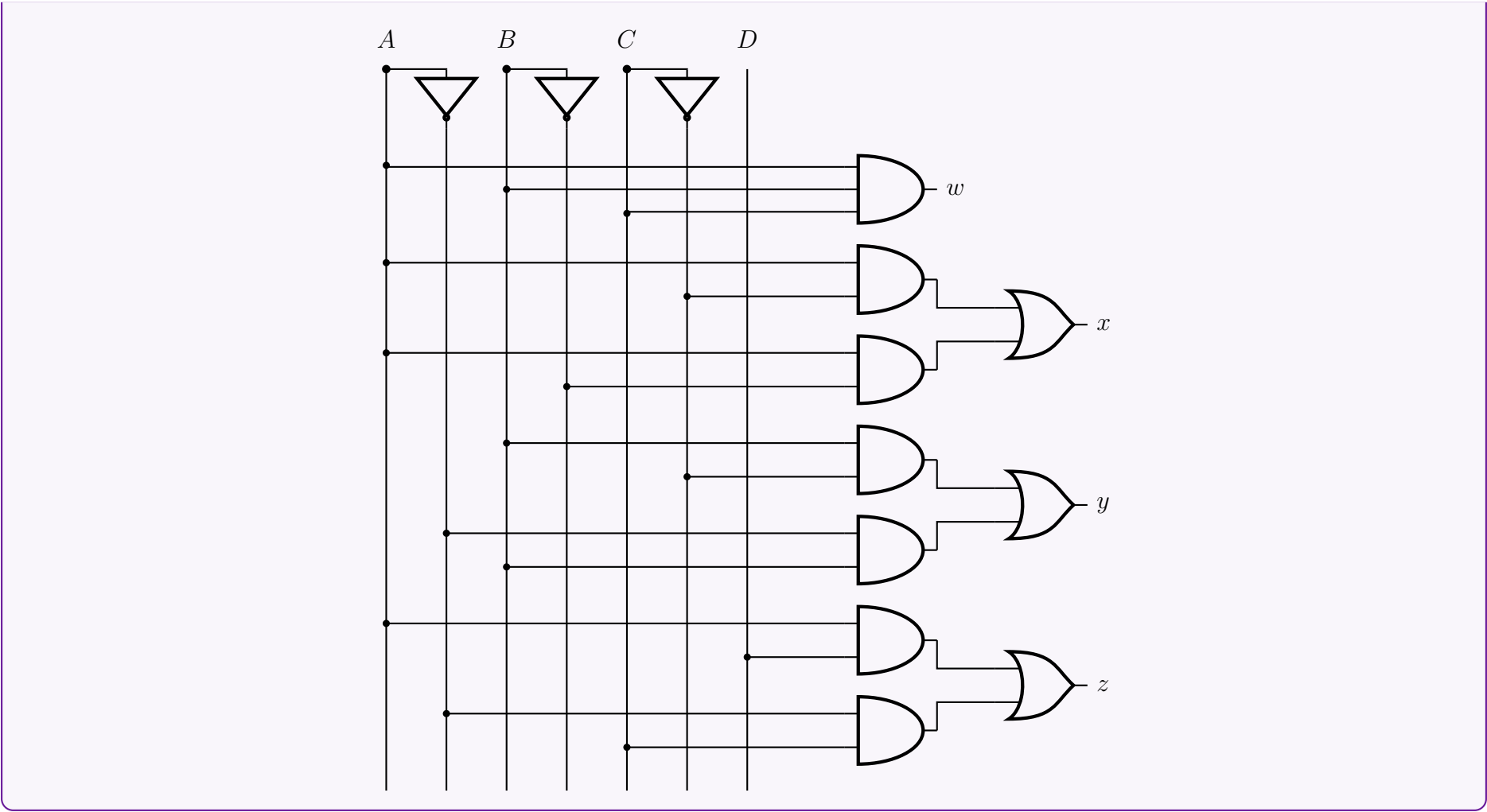
3. Logic Circuit Implementation

Using the derived minimized equations:

$$w = ABC$$
$$x = A\bar{C} + A\bar{B}$$
$$y = B\bar{C} + \bar{A}B$$
$$z = AD + \bar{A}C$$

We construct the circuit diagram utilizing logic gates. The input bus is organized with vertical lines for each variable and their inverted forms.

105



Binary Adder & Subtractor

Q1. Design a 4-bit binary subtractor circuit using full adders.

(a) Construct and draw the logic diagram of the subtractor circuit.

(b) Explain how subtraction is performed using 2's complement representation.(Understand level)

(15 marks)

[CSE 205 | L-2/T-1 | 2024–25]

Answer

Part (a): Logic Diagram of a 4-Bit Binary Subtractor

To design a 4-bit binary subtractor using Full Adders (FAs), we utilize the principle of 2's complement addition. The circuit consists of four Full Adders in a ripple-carry configuration. The minuend bits (A_0 to A_3) are fed directly into the adders, while the subtrahend bits (B_0 to B_3) are passed through NOT gates to produce their 1's complement. The initial carry-in is set to logic 1 to complete the 2's complement representation.

The diagram shows a 4-bit binary subtractor using four Full Adders (FA 0 to FA 3) in a ripple-carry configuration. The inputs are A_3, B_3 for FA 3, A_2, B_2 for FA 2, A_1, B_1 for FA 1, and A_0, B_0 for FA 0. The carry-in for FA 0 is $C_0 = 1$. The carry-out of FA 3 is $C_4 (C_{out})$. The outputs are D_3, D_2, D_1, D_0 .

Part (b): Explanation of Subtraction using 2's Complement

In digital systems, subtraction is typically implemented using addition hardware to reduce complexity. The operation $A - B$ is mathematically equivalent to adding the negative value of the subtrahend (B) to the minuend (A).

$$\begin{aligned} \text{Difference } (D) &= A - B \\ &= A + (-B) \end{aligned}$$

In the binary number system, the negative of a number is represented by its **2's complement**. The computation proceeds in the following logical steps:

(a) **Generating the 1's Complement:** The subtrahend bits ($B_3B_2B_1B_0$) are passed through NOT gates. This logically inverts every bit (changing 1s to 0s and 0s to 1s), producing the 1's complement of B , denoted as $\overline{B}$.

- (b) **Forming the 2's Complement:** To convert the 1's complement to the 2's complement, we must mathematically add 1. In the circuit design above, this is achieved by permanently wiring the input carry (C_0) of the least significant Full Adder (FA 0) to a logic high (1).

$$-B = \overline{B} + 1$$

- (c) **Parallel Addition:** The cascaded Full Adders now perform a standard parallel addition of three components: the minuend A , the inverted subtrahend $\overline{B}$, and the logic 1 from C_0 .

$$D = A + \overline{B} + 1$$

Because $\overline{B} + 1$ is the 2's complement of B , the output naturally evaluates to the difference $A - B$.

- (d) **Interpreting the Final Carry (C_4):**

- If $C_4 = 1$: The result is positive and is represented in its true binary form. The end carry is ignored/discarded.
- If $C_4 = 0$: The result is negative and is natively generated in its 2's complement form. To determine its true magnitude, one would need to take the 2's complement of the result.

Q2. Design a full subtractor circuit with three inputs x , y , B_{in} and two outputs Diff and B_{out} . The circuit subtracts $x - y - B_{in}$, where B_{in} is the input borrow and B_{out} is the output borrow. (18 marks) [CSE 205 | L-2/T-1 | 2019–20]

Answer

1. Truth Table Formulation

A full subtractor is a combinational circuit that performs subtraction between two bits (x and y), taking into account a borrow-in (B_{in}) from a lower significant stage. It produces two outputs: the Difference (Diff) and the Borrow-out (B_{out}). The mathematical operation is $x - y - B_{in}$.

Inputs			Outputs	
x (Minuend)	y (Subtrahend)	B_{in} (Borrow In)	Diff	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

2. Boolean Expression Derivation

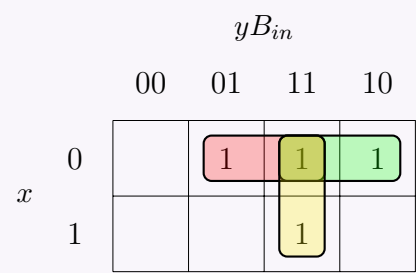
For Difference (Diff): The minterms for Diff are $\sum m(1, 2, 4, 7)$.

		yB_{in}			
		00	01	11	10
x	0		1		1
	1	1		1	

No adjacent cells can be grouped in the K-map. We write the sum of products (SOP) expression and simplify using Boolean algebra:

$$\begin{aligned}
 \text{Diff} &= \bar{x}\bar{y}B_{in} + \bar{x}y\bar{B}_{in} + x\bar{y}\bar{B}_{in} + xyB_{in} \\
 &= \bar{x}(\bar{y}B_{in} + y\bar{B}_{in}) + x(\bar{y}\bar{B}_{in} + yB_{in}) && \text{(Distributive Law)} \\
 &= \bar{x}(y \oplus B_{in}) + x(\overline{y \oplus B_{in}}) && \text{(XOR and XNOR Definitions)} \\
 &= x \oplus (y \oplus B_{in}) && \text{(XOR Definition)} \\
 &= \mathbf{x \oplus y \oplus B_{in}}
 \end{aligned}$$

For Borrow-out (B_{out}): The minterms for B_{out} are $\sum m(1, 2, 3, 7)$.



By grouping the 1s in the K-map, we obtain three pairs:

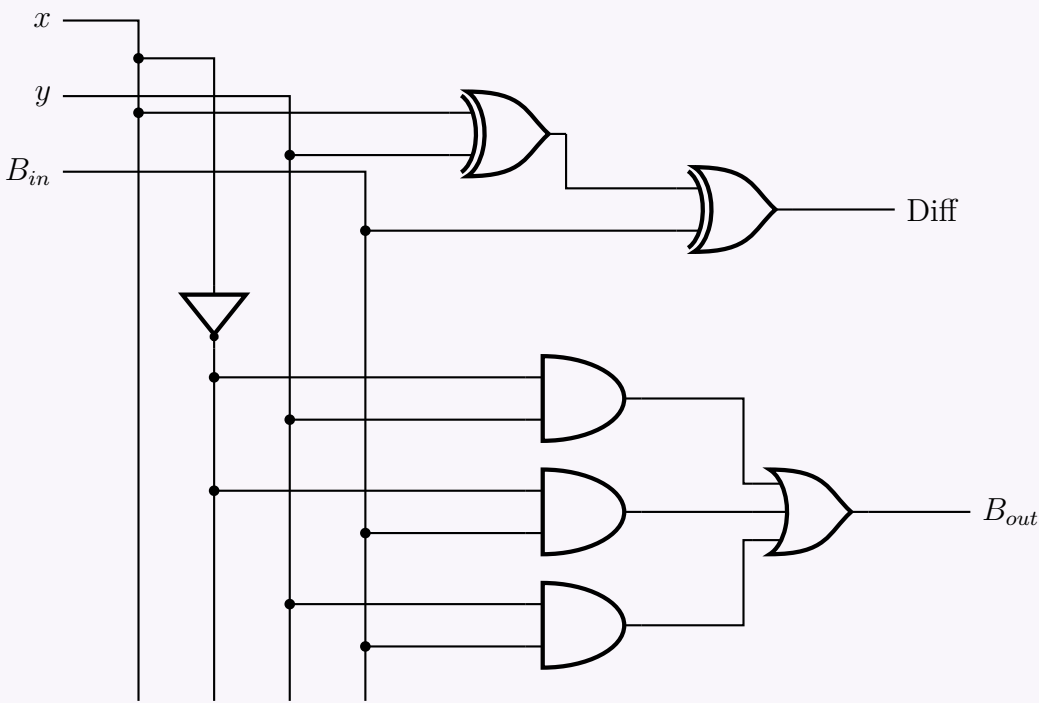
- Group 1 (m_3, m_2): yields $\bar{x}y$
- Group 2 (m_1, m_3): yields $\bar{x}B_{in}$
- Group 3 (m_3, m_7): yields yB_{in}

Thus, the simplified Boolean expression is:

$$B_{out} = \bar{x}y + \bar{x}B_{in} + yB_{in}$$

3. Logic Circuit Diagram

Using the simplified Boolean expressions, we can draw the full subtractor circuit using XOR, AND, OR, and NOT gates.



Q3. Using 4-bit binary adder and XOR gate, design a circuit which takes two 4-bit BCD inputs A and B , and returns $A + 1$ if the control input is 00, A' if the control input is 01, $A' + 1$ when the control input is 10, and A when the control input is 11.**(13 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Problem Interpretation and Strategy

The problem asks to design a circuit that performs four specific operations on a 4-bit input A based on a 2-bit condition ("input is 00, 01, 10, 11"). Although the prompt mentions "two 4-bit BCD inputs A and B ", the requested operations ($A + 1$, A' , $A' + 1$, A) do not utilize the 4-bit input B . Therefore, we assume the condition is driven by a 2-bit control signal, which we will designate as S_1 and S_0 .

We are constrained to use a **4-bit binary adder** and **XOR gates**. A 4-bit binary adder computes $F = X + Y + C_{in}$. We can permanently ground the second input ($Y = 0000$) and selectively modify the first input (X) and the carry-in (C_{in}) using XOR gates to achieve the desired operations.

2. Operation Analysis and Truth Table

We analyze the required operations to determine the necessary values for the adder's inputs X and C_{in} .

Control Input		Desired Output (F)	Adder Inputs		Operation Result
S_1	S_0		X	C_{in}	$X + Y + C_{in}$
0	0	$A + 1$	A	1	$A + 0 + 1 = A + 1$
0	1	A'	A'	0	$A' + 0 + 0 = A'$
1	0	$A' + 1$	A'	1	$A' + 0 + 1 = A' + 1$ (2's Complement)
1	1	A	A	0	$A + 0 + 0 = A$

3. Boolean Logic Derivation for Adder Inputs

From the truth table, we can derive the logic expressions for X_i (each bit of X) and C_{in} in terms of A_i , S_1 , and S_0 .

A. Deriving X_i : Notice that $X = A$ when the control inputs S_1S_0 are equal (00 or 11), and $X = A'$ when they are different (01 or 10). This behavior perfectly matches the Exclusive-OR (XOR) logic for the control bits. Let the inversion control signal be $Inv = S_1 \oplus S_0$.

$$Inv = S_1 \oplus S_0$$

$$X_i = A_i \oplus Inv = A_i \oplus (S_1 \oplus S_0)$$

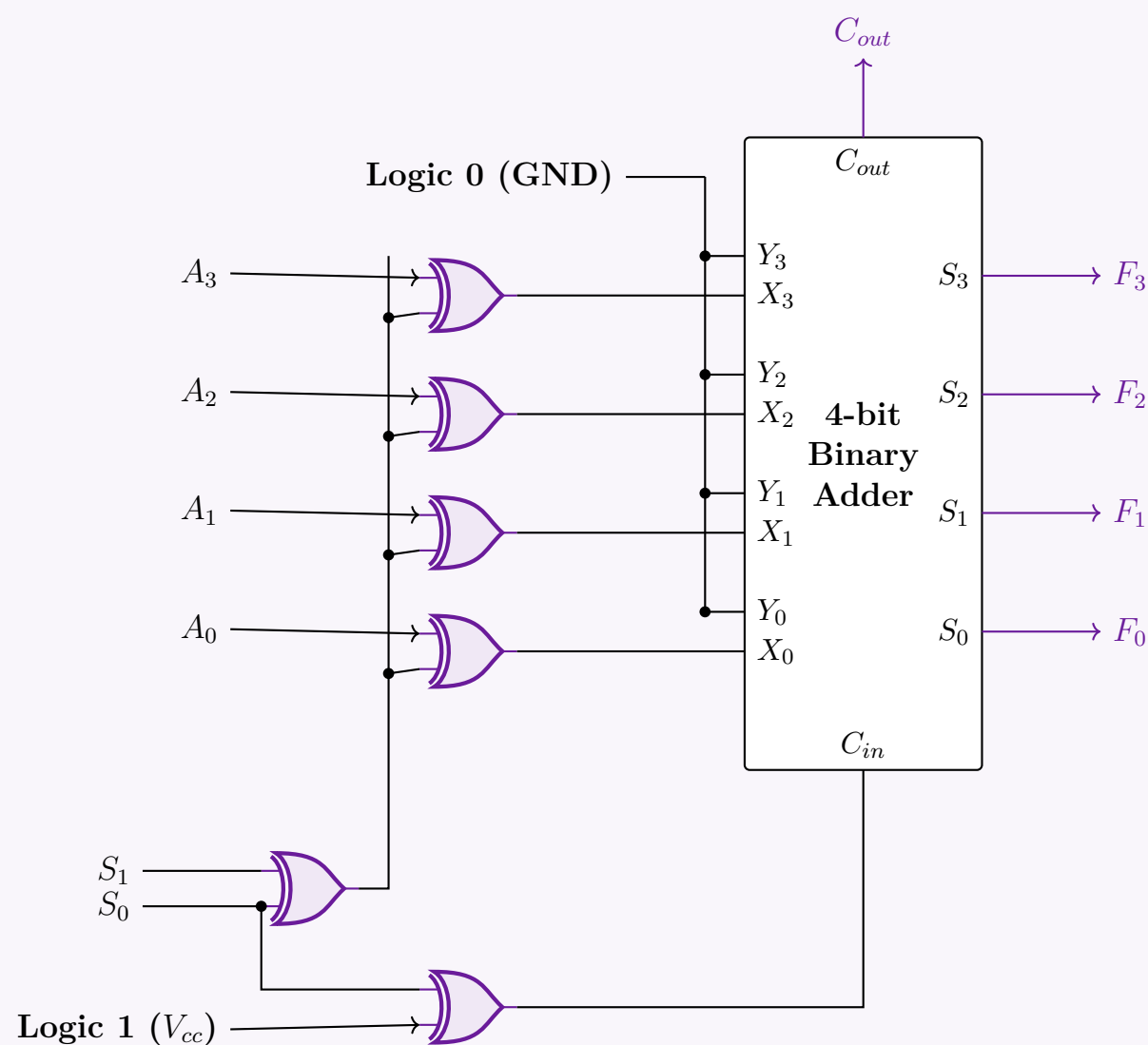
If $Inv = 0$, $X_i = A_i \oplus 0 = A_i$.

If $Inv = 1$, $X_i = A_i \oplus 1 = A'_i$.

B. Deriving C_{in} : Looking at the table, $C_{in} = 1$ when $S_0 = 0$, and $C_{in} = 0$ when $S_0 = 1$. Thus, $C_{in} = \overline{S_0}$. Since we are restricted to using only XOR gates, we can implement the NOT operation by XORing S_0 with Logic 1 (V_{cc}).

$$C_{in} = S_0 \oplus 1 = \overline{S_0}$$

4. Logic Circuit Diagram



Q4. Write the advantages and disadvantages of serial and parallel adders. (4 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

1. Serial Adders

A serial adder processes binary addition one bit at a time, sequentially. It typically uses a single Full Adder (FA), a flip-flop (to store the carry bit for the next clock cycle), and shift registers to hold the data.

Advantages:

- **Minimal Hardware Complexity:** Requires only a single Full Adder and one D flip-flop, regardless of the size (n -bits) of the operands.
- **Low Cost and Reduced Area:** Because of the minimal gate count, it occupies very little space on a silicon die and is inexpensive to manufacture.
- **High Scalability:** Expanding the adder to handle larger operands does not require adding more logic gates; the system only requires running the clock for more cycles.

Disadvantages:

- **Slow Operational Speed:** The addition operation is inherently slow. To add two n -bit numbers, the system requires n clock cycles.

- **Complex Control Mechanism:** Requires additional control logic, shift registers, and a synchronized clock to feed the bits sequentially.

2. Parallel Adders

A parallel adder processes all bits of the corresponding operands simultaneously. Common implementations include the Ripple Carry Adder (RCA) and the Carry Lookahead Adder (CLA).

Advantages:

- **High Operational Speed:** Performs the arithmetic operation extremely fast, as all bits are added concurrently in a single operation phase.
- **Immediate Results:** The final sum bits and the final carry-out are available almost immediately, subject only to the combinational propagation delay of the logic gates.
- **Simpler Control Logic:** Does not depend on sequential shifting or multiple clock cycles to yield a single addition result.

Disadvantages:

- **High Hardware Overhead:** Requires n Full Adders to compute the sum of two n -bit operands, substantially increasing the gate count.
- **Increased Cost and Power Consumption:** Larger silicon area translates to higher production costs and greater power dissipation.
- **Propagation Delay (in RCA):** In basic parallel architectures like the Ripple Carry Adder, the carry bit must propagate from the Least Significant Bit (LSB) to the Most Significant Bit (MSB), creating a delay bottleneck for large bit-widths (though this is solved by using more hardware-intensive Lookahead architectures).

3. Summary Comparison Table

Parameter	Serial Adder	Parallel Adder
Operating Speed	Slow (Takes n clock cycles)	Fast (Combinational delay time)
Hardware Required	1 Full Adder, 1 Flip-Flop	n Full Adders (for n -bit addition)
Circuit Cost	Low	High
Silicon Area	Very small	Large
Data Application	Bits are fed serially (one by one)	Bits are fed simultaneously
Scalability overhead	Time (More cycles needed)	Hardware (More gates needed)

Q5. Design a four-bit binary adder using a 1-bit full-adder circuit. How can it be modified to use as a four-bit subtractor? (15 marks)
[CSE 205 | L-2/T-1 | 2013–14]

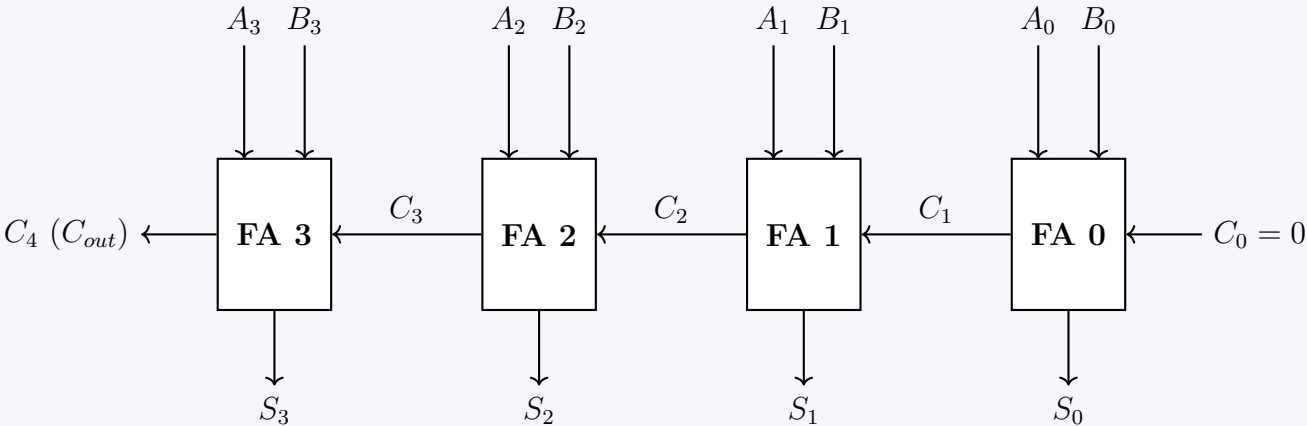
Answer

1. Design of a Four-Bit Binary Adder

A 4-bit binary adder can be constructed by cascading four 1-bit Full Adder (FA) circuits in a configuration known as a **Ripple Carry Adder (RCA)**. Each full adder computes the addition of two corresponding bits of the operands (A_i and B_i) along with the carry generated from the previous, less significant stage (C_i).

For two 4-bit numbers $A = A_3A_2A_1A_0$ and $B = B_3B_2B_1B_0$:

- The Carry-In to the Least Significant Bit (LSB) stage, C_0 , is tied to 0.
- The Carry-Out of each FA (C_{i+1}) is directly connected to the Carry-In of the next FA.
- The final Carry-Out (C_4) indicates an overflow or the 5th bit of the sum.



2. Modification to a Four-Bit Adder-Subtractor

To modify the binary adder to perform both addition and subtraction, we utilize the principle of **2’s Complement Arithmetic**. Subtraction $A - B$ is mathematically equivalent to adding the 2’s complement of B to A :

$$\begin{aligned} A - B &= A + (-B) \\ &= A + (1\text{'s complement of } B) + 1 \\ &= A + \bar{B} + 1 \end{aligned}$$

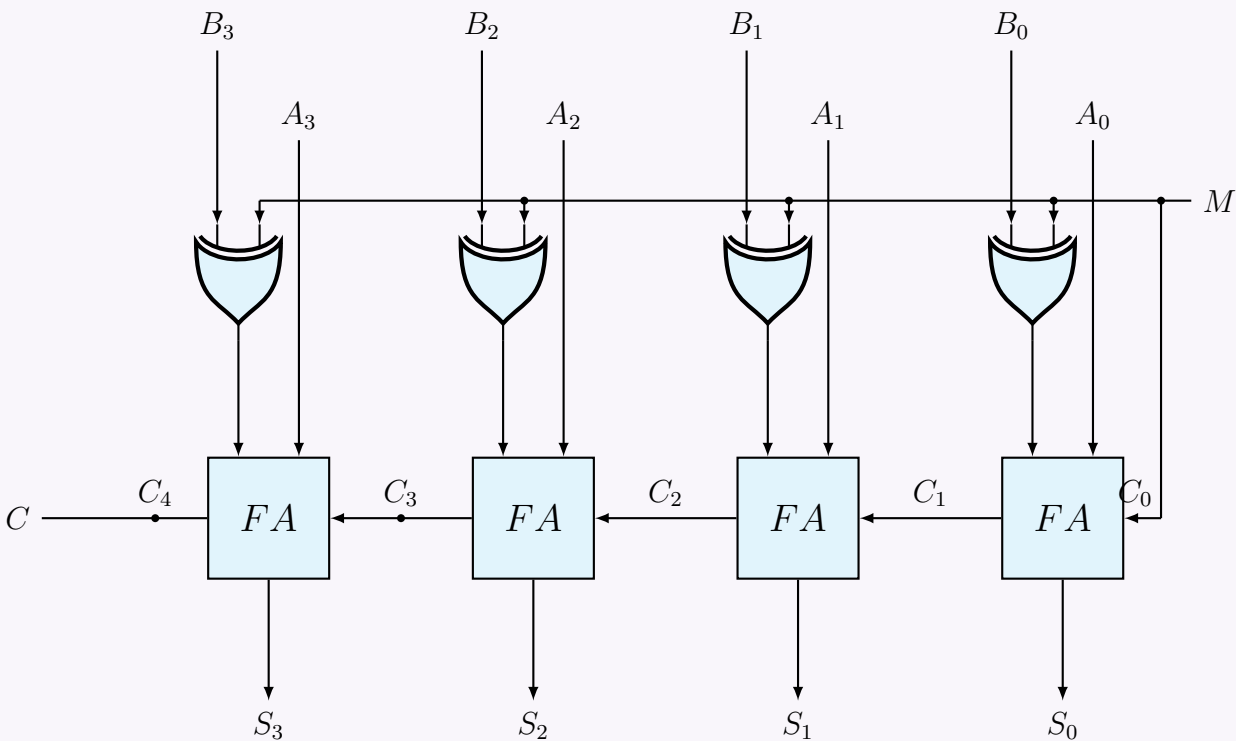
We introduce a control mode line, M , and add an **XOR gate** at the B input of each full adder. The properties of the XOR gate dictate:

$$\begin{aligned} B \oplus 0 &= B \quad (\text{Passes the bit unchanged}) \\ B \oplus 1 &= \bar{B} \quad (\text{Inverts the bit}) \end{aligned}$$

Operation Modes:

- **Addition** ($M = 0$): The XOR gates output $B \oplus 0 = B$. The initial carry-in C_0 receives $M = 0$. The circuit computes $S = A + B$.
- **Subtraction** ($M = 1$): The XOR gates output $B \oplus 1 = \bar{B}$. The initial carry-in C_0 receives $M = 1$. The circuit computes $S = A + \bar{B} + 1$, yielding the correct $A - B$.

3. Four-Bit Adder-Subtractor Circuit Diagram



Q6. Starting with a truth table, design a full subtractor. (20 marks)

[CSE 205 | L-2/T-1 | 2012–13]

Answer

1. Truth Table

A Full Subtractor is a combinational circuit that performs subtraction between two bits (A and B), taking into account a borrow-in (B_{in}) from a lower significant stage. It produces two outputs: the Difference (D) and the Borrow-out (B_{out}).

Inputs			Outputs	
Minuend (A)	Subtrahend (B)	Borrow-in (B_{in})	Difference (D)	Borrow-out (B_{out})
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

2. Karnaugh Maps and Boolean Derivation

Using the truth table, we can plot the Karnaugh maps for the outputs D and B_{out} .

For Difference (D):

The minterms are $\sum m(1, 2, 4, 7)$.

		BB_{in}			
		00	01	11	10
A	0		1		1
	1	1		1	

No simplifications (groupings) are possible. We derive the expression mathematically:

$$\begin{aligned}
 D &= \bar{A}\bar{B}B_{in} + \bar{A}B\bar{B}_{in} + A\bar{B}\bar{B}_{in} + AB B_{in} \\
 &= \bar{A}(\bar{B}B_{in} + B\bar{B}_{in}) + A(\bar{B}\bar{B}_{in} + B B_{in}) && \text{(Distributive Law)} \\
 &= \bar{A}(B \oplus B_{in}) + A(\overline{B \oplus B_{in}}) && \text{(XOR and XNOR definitions)} \\
 &= A \oplus (B \oplus B_{in}) && \text{(XOR definition)} \\
 &= A \oplus B \oplus B_{in}
 \end{aligned}$$

For Borrow-out (B_{out}):

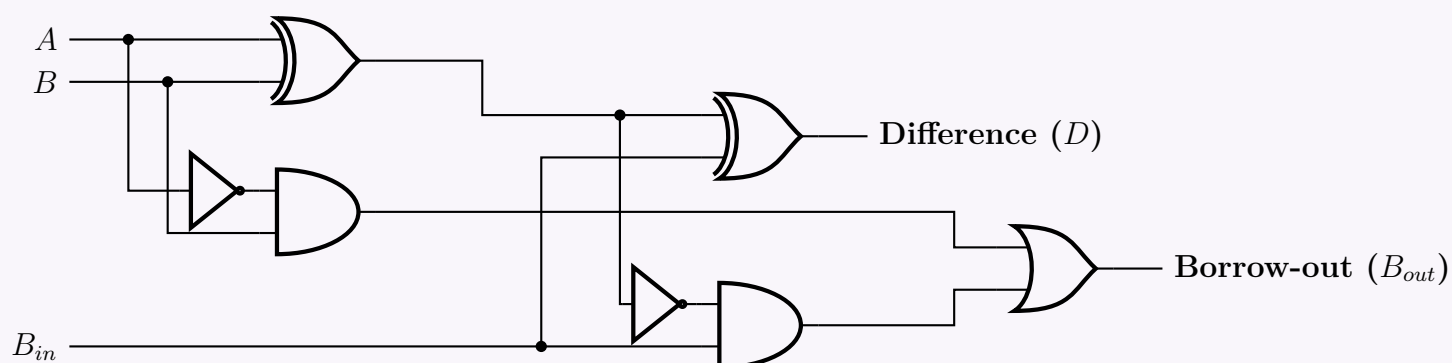
The minterms are $\sum m(1, 2, 3, 7)$.

		BB_{in}			
		00	01	11	10
A	0		1	1	1
	1			1	

From the K-map groupings, the standard Sum of Products (SOP) is $B_{out} = \bar{A}B + \bar{A}B_{in} + BB_{in}$. To implement this efficiently using XOR gates (sharing logic with the Difference circuit), we algebraically manipulate the unsimplified minterms:

$$\begin{aligned}
 B_{out} &= \bar{A}\bar{B}B_{in} + \bar{A}B\bar{B}_{in} + \bar{A}B B_{in} + AB B_{in} \\
 &= \bar{A}B(\bar{B}_{in} + B_{in}) + \bar{A}\bar{B}B_{in} + AB B_{in} && \text{(Distributive Law)} \\
 &= \bar{A}B(1) + B_{in}(\bar{A}\bar{B} + AB) && \text{(Complement Law & Distributive Law)} \\
 &= \bar{A}B + B_{in}(\bar{A} \oplus \bar{B}) && \text{(XNOR Equivalence)}
 \end{aligned}$$

3. Logic Circuit Implementation



BCD Adder-Subtractor

Q1. Design a BCD adder-subtractor which takes two 4-bit BCD numbers A and B . Addition and subtraction is selected by a selector bit S : when $S = 0$ the circuit produces $A + B$; when $S = 1$ it produces $A - B$. Explain the operations $9 + 7$ and $9 - 7$ with your designed circuit. (12 marks)

[CSE 205 | L-2/T-1 | 2020–21, 2017–18, 2016–17]

Answer

1. Principle of Operation

This circuit functions as a modified Arithmetic logic unit where the mode selector S dictates whether it performs addition or subtraction.

- **When $S = 0$ (Addition):** The input y_i passes through the XOR gates unchanged. With the initial carry-in $C_i = 0$, the first stage computes the standard binary sum of $A + B$.
- **When $S = 1$ (Subtraction):** The XOR gates invert all bits of the y_i input, producing the 1's complement ($\bar{y}_i$). Since

$C_i = S = 1$, the first stage computes $A + \bar{y} + 1$, effectively performing a **2's complement subtraction** ($A - B$) at the binary level.

2. Input Modification Logic (XOR Gates)

Instead of a complex combinational circuit, this design utilizes bitwise inversion using 2-input XOR gates to handle the Y_i inputs fed into the top adder:

$$Y_i = y_i \oplus S$$

If $S = 0$, $Y_i = y_i$ (Unchanged). If $S = 1$, $Y_i = \bar{y}_i$ (Inverted).

3. Derivation of BCD Correction Logic (Err)

When adding two 4-bit BCD numbers (maximum $9 + 9 + 1 = 19$), the top binary adder produces a 5-bit result: a carry-out C_y and a 4-bit sum $S_3S_2S_1S_0$. Since a binary adder operates in base-2, but BCD operates in base-10, any sum exceeding 9 is an **invalid BCD digit** and requires adding a correction factor of 6 (0110_2).

Let's analyze the conditions where the sum requires correction ($Err = 1$):

Condition 1: Sum is between 16 and 19.

In this case, the binary adder automatically generates a carry-out. So, if $C_y = 1$, correction is needed.

Condition 2: Sum is between 10 and 15.

Here, $C_y = 0$, but the 4-bit sum is an invalid BCD digit. Let's look at the truth table for sums 10 to 15:

Decimal Sum	C_y	S_3	S_2	S_1	S_0	Correction Needed (Err)
0 to 9	0	(Valid BCD range)				0
10	0	1	0	1	0	1
11	0	1	0	1	1	1
12	0	1	1	0	0	1
13	0	1	1	0	1	1
14	0	1	1	1	0	1
15	0	1	1	1	1	1
16 to 19	1	(Don't cares for S_{0-3})				1

From the truth table, we can extract the Boolean expression for the invalid states where $C_y = 0$:

- Notice that for Decimal 10 and 11, both $S_3 = 1$ and $S_1 = 1$. This gives the term $(S_3 \cdot S_1)$.
- For Decimal 12, 13, 14, and 15, both $S_3 = 1$ and $S_2 = 1$. This gives the term $(S_3 \cdot S_2)$.

Therefore, if the sum is between 10 and 15, the condition is $(S_3 \cdot S_2) + (S_3 \cdot S_1)$.

Final Correction Equation:

Combining Condition 1 ($C_y = 1$) and Condition 2, the final logic equation for the Error signal becomes:

$$Err = C_y + (S_3 \cdot S_2) + (S_3 \cdot S_1)$$

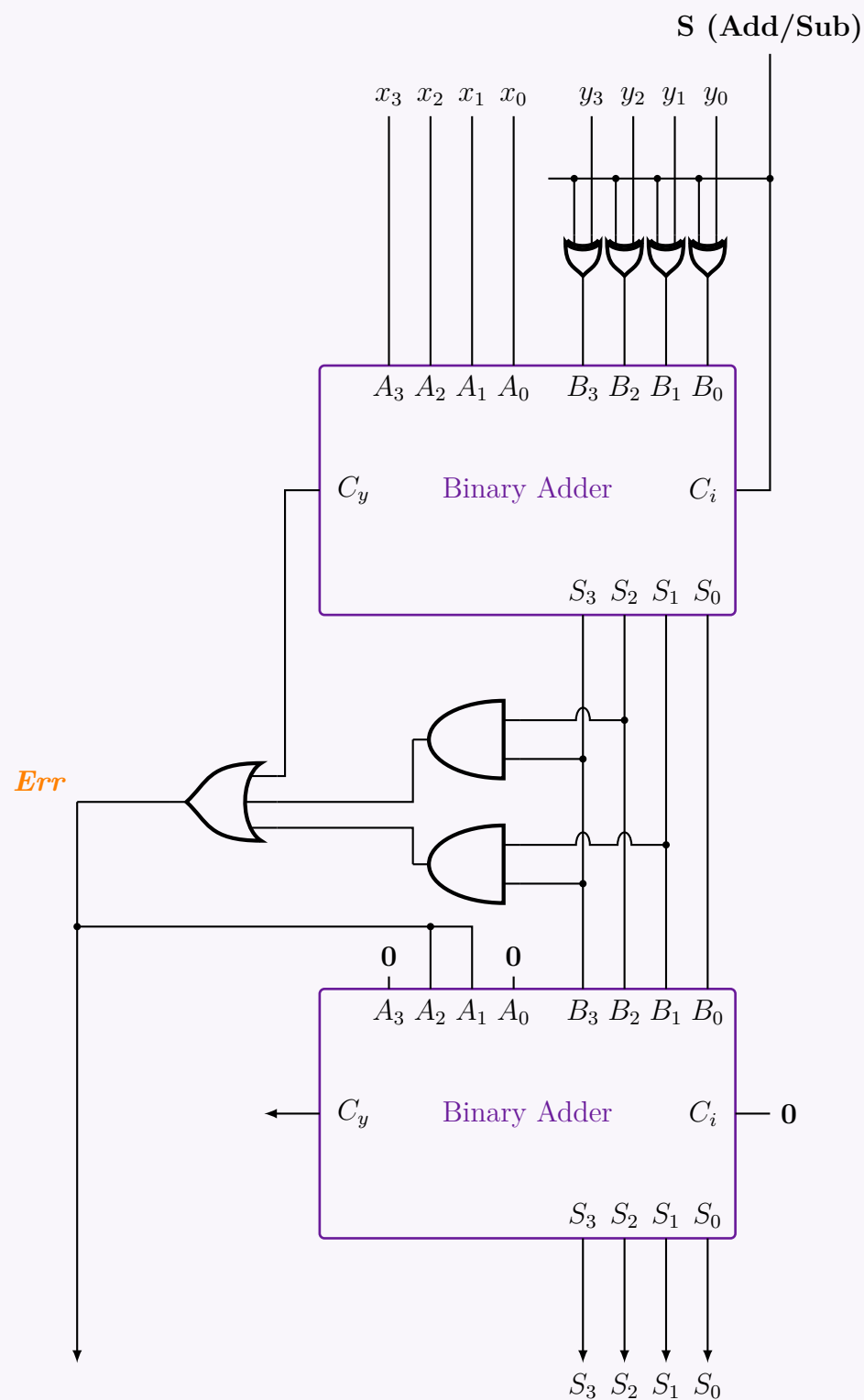
This logic is implemented in the circuit using two AND gates and one 3-input OR gate.

4. Bottom Adder (Correction Execution)

The bottom binary adder takes the uncorrected sum from the top adder and adds the correction factor based on the Err signal:

- The inputs to the bottom adder are hardwired as: $A_3 = 0, A_2 = Err, A_1 = Err, A_0 = 0$.
- If $Err = 0$, it adds 0000_2 (no correction applied).
- If $Err = 1$, it adds 0110_2 (Decimal 6), which adjusts the sum to the correct BCD format and generates the proper BCD carry for the next stage.

5. Complete System Block Diagram



6. Explanation of Operations

Operation 1: Compute $9 + 7$ ($x = 0$)

- **Inputs:** $A = 1001_2$ (9), $B = 0111_2$ (7), $x = 0$.
- **Complementer Output:** Since $x = 0$, $Y = B = 0111_2$.
- **Adder 1:** Computes $A + Y + x \Rightarrow 1001_2 + 0111_2 + 0 = 10000_2$.
 - Intermediate sum $Z = 0000_2$.
 - Carry $C_{out1} = 1$.
- **Correction Logic:** $C_{BCD} = C_{out1} + Z_3Z_2 + Z_3Z_1 = 1 + 0 + 0 = 1$. A correction is required.
- **Adder 2:** Adds Z and the correction vector $0C_{BCD}C_{BCD}0$ (0110_2).
 - $S = 0000_2 + 0110_2 = 0110_2$ (6_{10}).
- **Final Result:** The carry to the next BCD digit is $C_{BCD} = 1$, and the sum is $S = 0110_2$ (6). Thus, the final BCD representation is 16, which is strictly correct ($9 + 7 = 16$).

Operation 2: Compute $9 - 7$ ($x = 1$)

- **Inputs:** $A = 1001_2$ (9), $B = 0111_2$ (7), $x = 1$.
- **Complementer Output:** Since $x = 1$, the logic outputs the 9's complement of B . $Y = 9 - 7 = 2 \Rightarrow 0010_2$.
- **Adder 1:** Computes $A + Y + x$ (adding 1 implements the 10's complement).
 - $1001_2 + 0010_2 + 1 = 1100_2$.
 - Intermediate sum $Z = 1100_2$ (12_{10}).

- Carry $C_{out1} = 0$.
- **Correction Logic:** Check if $Z > 9$.
 - $C_{BCD} = C_{out1} + Z_3Z_2 + Z_3Z_1 = 0 + (1 \cdot 1) + (1 \cdot 0) = 1$. A correction is required.
- **Adder 2:** Adds Z and the correction vector 0110_2 .
 - $1100_2 + 0110_2 = 10010_2$.
 - The lower 4 bits are $S = 0010_2$ (2_{10}).
- **Final Result:** The sum $S = 0010_2$ gives the correct BCD difference of 2. The $C_{BCD} = 1$ acts as a no-borrow indicator (signifying a positive result in a 10's complement subtractor), which is discarded for single-digit subtraction. Result is correctly 2.

Q2. Design a 4-bit BCD adder using 4-bit binary adders and basic logic gates. Use a block diagram to draw a binary adder. **(15 marks)**
 [CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Principle of Operation

A 4-bit Binary Coded Decimal (BCD) adder performs the addition of two BCD digits (ranging from 0 to 9) and produces a valid BCD sum. Since a 4-bit binary adder natively computes base-16 (from 0000 to 1111), its output must be corrected if the sum exceeds the valid BCD range (> 9) or if a carry is generated out of the most significant bit.

The conditions for a valid BCD sum requiring **no correction** are:

- The binary sum is less than or equal to 9 (1001_2).
- No final carry out is generated ($K = 0$).

The condition requiring **correction** (adding 6_{10} or 0110_2 to the sum) is met if the sum is invalid (> 9). This happens under three specific scenarios:

- (a) A carry (K) is generated out of the MSB of the binary adder (e.g., $9 + 9 = 18$, carry generated).
- (b) The sum bits $Z_3Z_2Z_1Z_0$ fall in the range 1010 to 1111. This occurs when:
 - $Z_3 = 1$ and $Z_2 = 1$ (Values 12, 13, 14, 15).
 - $Z_3 = 1$ and $Z_1 = 1$ (Values 10, 11).

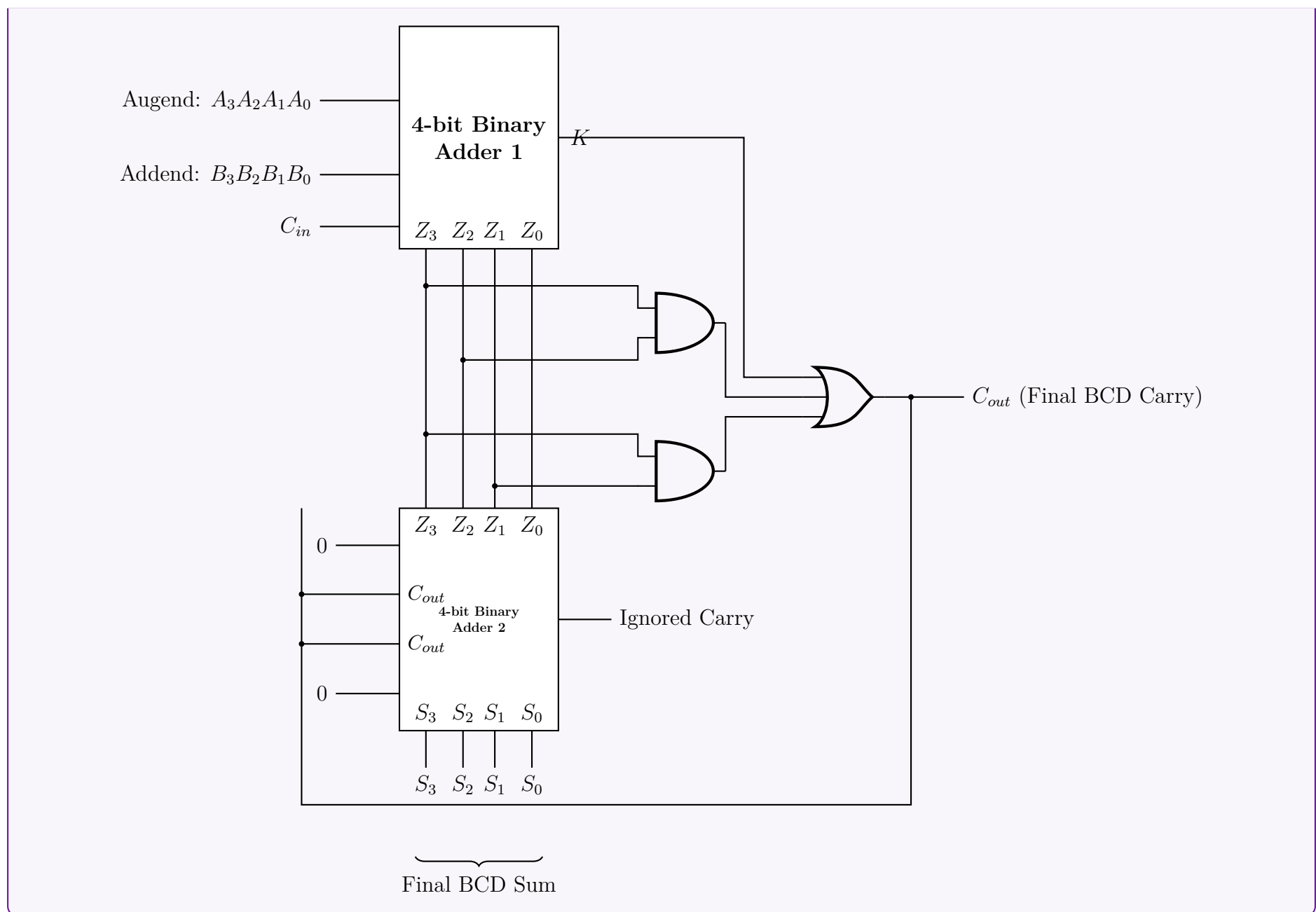
2. Correction Logic Derivation

Based on the conditions above, we can express the logic for the final carry-out (C_{out}) which triggers the addition of 0110_2 :

$$C_{out} = K + (Z_3 \cdot Z_2) + (Z_3 \cdot Z_1)$$

When $C_{out} = 1$, the second 4-bit binary adder adds 0110_2 to the intermediate sum $Z_3Z_2Z_1Z_0$ to produce the final valid BCD sum $S_3S_2S_1S_0$. If $C_{out} = 0$, it adds 0000_2 , leaving the valid sum unchanged.

3. BCD Adder Block Diagram



Carry Look-Ahead Adder (CLA)

Q1. Explain the working principle of a Carry Look-Ahead (CLA) adder. Derive the expressions for carry generation (C_1, C_2, C_3, C_4) using generate and propagate functions. (12 marks) [CSE 205 | L-2/T-1 | 2024–25]

Answer

Working Principle of Carry Look-Ahead (CLA) Adder

In a standard Ripple Carry Adder (RCA), the carry bit from each full adder stage must propagate serially to the next stage before the subsequent addition can complete. This serial propagation introduces a significant time delay proportional to the number of bits in the adders.

The Carry Look-Ahead (CLA) adder overcomes this limitation by calculating the carry signals in advance, simultaneously for all stages, rather than waiting for them to ripple through. It achieves this by examining the inputs to determine whether a particular bit position will *generate* a carry on its own or *propagate* an incoming carry to the next stage. By decoupling the carry computation from the intermediate sum stages using high-speed two-level logic, the total addition time is vastly reduced.

Generate and Propagate Functions

For the i -th stage of the adder with individual inputs A_i and B_i , we define two Boolean functions:

- **Carry Generate (G_i):** A carry is unconditionally generated out of the i -th stage if both inputs A_i and B_i are logic 1, regardless of the incoming carry.

$$G_i = A_i \cdot B_i$$

- **Carry Propagate (P_i):** An incoming carry C_i is propagated to the next stage if either A_i or B_i is logic 1 (exclusive-OR is commonly used to simultaneously satisfy the standard sum definition).

$$P_i = A_i \oplus B_i$$

Using these definitions, the sum (S_i) and the outgoing carry (C_{i+1}) for the i -th stage can be elegantly expressed as:

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + (P_i \cdot C_i)$$

Derivation of Carry Expressions (C_1, C_2, C_3, C_4)

To eliminate the serial dependency, we recursively expand the general carry equation $C_{i+1} = G_i + P_i \cdot C_i$. This allows us to express all future carries strictly in terms of the parallel generation functions (G_i), propagation functions (P_i), and the initial system carry-in (C_0).

$$C_1 = G_0 + P_0 \cdot C_0$$

$$\begin{aligned} C_2 &= G_1 + P_1 \cdot C_1 \\ &= G_1 + P_1 \cdot (G_0 + P_0 \cdot C_0) && \text{(Substitute expression for } C_1) \\ &= G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0 && \text{(Distributive Law)} \end{aligned}$$

$$\begin{aligned} C_3 &= G_2 + P_2 \cdot C_2 \\ &= G_2 + P_2 \cdot (G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0) && \text{(Substitute expression for } C_2) \\ &= G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0 && \text{(Distributive Law)} \end{aligned}$$

$$\begin{aligned} C_4 &= G_3 + P_3 \cdot C_3 \\ &= G_3 + P_3 \cdot (G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0) && \text{(Substitute expression for } C_3) \\ &= G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0 && \text{(Distributive Law)} \end{aligned}$$

Conclusion

The expanded expressions clearly demonstrate that C_1, C_2, C_3 , and C_4 do not depend on the resolution of immediately preceding carry outputs (for instance, C_4 does not have to wait for C_3 to be physically calculated by the previous adder block). Instead, they depend entirely on the initial input carry C_0 and the external inputs (represented by P_i and G_i).

Because P_i and G_i are generated in parallel with a single gate delay, and the expanded carry expressions form a Sum-of-Products (SOP), the entire Look-Ahead Carry block can be implemented directly using a two-level AND-OR logic circuit. As a result, all intermediate carries are available simultaneously after just a few gate delays, drastically accelerating binary arithmetic operations in modern microprocessors.

Q2. What is the benefit of using an adder with carry lookahead logic? Derive expressions of different carry of a 4-bit adder with carry lookahead logic. **(15 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Answer

1. Benefit of Using an Adder with Carry Lookahead Logic

In a conventional n -bit Ripple Carry Adder (RCA), the computation of the final sum requires the carry bit to propagate sequentially through every full-adder stage. This creates a linear time delay of $\mathcal{O}(n)$, which significantly slows down arithmetic operations in processing units.

The primary **benefit of a Carry Lookahead Adder (CLA)** is its ability to vastly reduce this computation time by calculating the carry-in signals for all stages simultaneously. Instead of waiting for the carry to "ripple" from the Least Significant Bit (LSB) to the Most Significant Bit (MSB), the CLA logic utilizes the input bits to dynamically predict whether a carry will be generated or propagated. This parallel evaluation using two-level logic (AND-OR) reduces the carry propagation delay to a constant theoretical time of $\mathcal{O}(1)$ (typically 2 to 3 gate delays), independent of the number of bits within the lookahead block, thereby maximizing the arithmetic processing speed.

2. Derivation of Carry Expressions for a 4-Bit CLA

To compute the carries independently of the sequential ripple, we first define two intermediate signals for the i -th bit stage based on inputs A_i and B_i :

- **Carry Generate (G_i):** Produces a carry regardless of the incoming carry.

$$G_i = A_i \cdot B_i$$

- **Carry Propagate (P_i):** Propagates an incoming carry to the next stage.

$$P_i = A_i \oplus B_i$$

The general Boolean equation for the outgoing carry of the i -th stage is:

$$C_{i+1} = G_i + (P_i \cdot C_i) \tag{1}$$

For a 4-bit adder, the bit stages are $i = 0, 1, 2, 3$. We iteratively substitute the previous stage's carry into equation (1) to express each carry purely in terms of the initial carry-in (C_0) and the parallel G_i and P_i signals.

Carry 1 (C_1): For $i = 0$

$$C_1 = G_0 + P_0 \cdot C_0$$

Carry 2 (C_2): For $i = 1$

$$\begin{aligned} C_2 &= G_1 + P_1 \cdot C_1 \\ &= G_1 + P_1 \cdot (G_0 + P_0 \cdot C_0) && \text{(Substitute } C_1) \\ \mathbf{C_2} &= \mathbf{G_1} + \mathbf{P_1} \cdot \mathbf{G_0} + \mathbf{P_1} \cdot \mathbf{P_0} \cdot \mathbf{C_0} && \text{(Distributive Law)} \end{aligned}$$

Carry 3 (C_3): For $i = 2$

$$\begin{aligned} C_3 &= G_2 + P_2 \cdot C_2 \\ &= G_2 + P_2 \cdot (G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0) && \text{(Substitute } C_2) \\ \mathbf{C_3} &= \mathbf{G_2} + \mathbf{P_2} \cdot \mathbf{G_1} + \mathbf{P_2} \cdot \mathbf{P_1} \cdot \mathbf{G_0} + \mathbf{P_2} \cdot \mathbf{P_1} \cdot \mathbf{P_0} \cdot \mathbf{C_0} && \text{(Distributive Law)} \end{aligned}$$

Carry 4 (C_4): For $i = 3$

$$\begin{aligned} C_4 &= G_3 + P_3 \cdot C_3 \\ &= G_3 + P_3 \cdot (G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0) && \text{(Substitute } C_3) \\ \mathbf{C_4} &= \mathbf{G_3} + \mathbf{P_3} \cdot \mathbf{G_2} + \mathbf{P_3} \cdot \mathbf{P_2} \cdot \mathbf{G_1} + \mathbf{P_3} \cdot \mathbf{P_2} \cdot \mathbf{P_1} \cdot \mathbf{G_0} + \mathbf{P_3} \cdot \mathbf{P_2} \cdot \mathbf{P_1} \cdot \mathbf{P_0} \cdot \mathbf{C_0} && \text{(Distributive Law)} \end{aligned}$$

Summary of Results: The derivations yield the final expressions mathematically structured as Sum-of-Products (SOP). None of the carry signals (C_1, C_2, C_3, C_4) depend sequentially on the preceding full-adder outputs; they only require the base input signals (G_i, P_i , and C_0), allowing them to be synthesized simultaneously via rapid two-level logic hardware.

Q3. Design a 4 bit Carry Look Ahead (CLA) adder which will take $X_3X_2X_1X_0$ and $Y_3Y_2Y_1Y_0$ as input, and produce their sum. Show the steps and the role of different *carries* of this CLA. **(13 marks)** [CSE 205 | L-2/T-1 | 2020–21]

Answer

1. Design Overview of a 4-bit Carry Look-Ahead (CLA) Adder

A standard ripple-carry adder suffers from propagation delay because each stage must wait for the carry from the previous stage. A Carry Look-Ahead (CLA) adder eliminates this sequential dependency by calculating all carry bits simultaneously. The 4-bit CLA takes inputs $X_3X_2X_1X_0$ and $Y_3Y_2Y_1Y_0$ along with an initial carry C_0 , and produces the sum $S_3S_2S_1S_0$ and the final carry-out C_4 . The design follows three distinct stages:

- (a) **Generate and Propagate Logic:** Computes the auxiliary signals for each bit.
- (b) **Carry Look-Ahead Logic:** Computes the intermediate carries (C_1, C_2, C_3, C_4) in parallel.
- (c) **Sum Logic:** Computes the final sum bits (S_0, S_1, S_2, S_3).

2. Step-by-Step Derivation and Logic Design

Step A: Propagate (P_i) and Generate (G_i) Functions

For each bit position $i \in \{0, 1, 2, 3\}$, we define two signals:

$$\begin{aligned} G_i &= X_i \cdot Y_i && \text{(Carry Generate: produces a carry if both inputs are 1)} \\ P_i &= X_i \oplus Y_i && \text{(Carry Propagate: passes an incoming carry to the next stage)} \end{aligned}$$

Step B: The Role and Derivation of Different Carries

The core equation for any carry is $C_{i+1} = G_i + P_i \cdot C_i$. The fundamental *role* of the different carries (C_1, C_2, C_3, C_4) in a CLA is to pre-compute the carry status for all stages independently of the intermediate sums, functioning strictly on the basis of P_i, G_i , and the initial C_0 . By expanding the core equation, we express every carry solely as a two-level Sum-of-Products (SOP), reducing the delay to just $\mathcal{O}(1)$ gate levels.

The mathematical derivations are as follows:

$$\begin{aligned} \mathbf{C_1} &= \mathbf{G_0} + \mathbf{P_0} \cdot \mathbf{C_0} \\ \\ C_2 &= G_1 + P_1 \cdot C_1 \\ \mathbf{C_2} &= \mathbf{G_1} + \mathbf{P_1} \cdot \mathbf{G_0} + \mathbf{P_1} \cdot \mathbf{P_0} \cdot \mathbf{C_0} && \text{(Substitute } C_1 \text{ \& Apply Distributive Law)} \\ \\ C_3 &= G_2 + P_2 \cdot C_2 \\ \mathbf{C_3} &= \mathbf{G_2} + \mathbf{P_2} \cdot \mathbf{G_1} + \mathbf{P_2} \cdot \mathbf{P_1} \cdot \mathbf{G_0} + \mathbf{P_2} \cdot \mathbf{P_1} \cdot \mathbf{P_0} \cdot \mathbf{C_0} && \text{(Substitute } C_2 \text{ \& Apply Distributive Law)} \\ \\ C_4 &= G_3 + P_3 \cdot C_3 \\ \mathbf{C_4} &= \mathbf{G_3} + \mathbf{P_3} \cdot \mathbf{G_2} + \mathbf{P_3} \cdot \mathbf{P_2} \cdot \mathbf{G_1} + \mathbf{P_3} \cdot \mathbf{P_2} \cdot \mathbf{P_1} \cdot \mathbf{G_0} + \mathbf{P_3} \cdot \mathbf{P_2} \cdot \mathbf{P_1} \cdot \mathbf{P_0} \cdot \mathbf{C_0} && \text{(Substitute } C_3 \text{ \& Distribute)} \end{aligned}$$

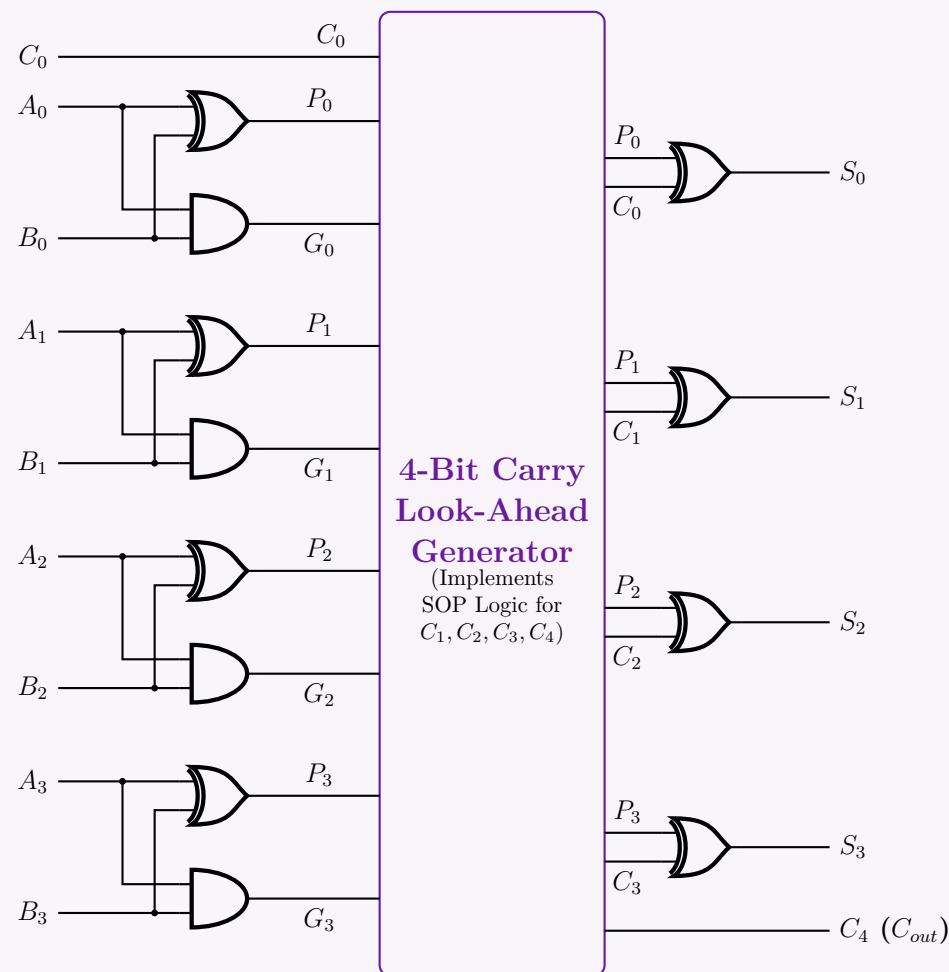
Step C: Sum Generation

Once the carries are evaluated in parallel, the final sum is calculated as:

$$S_i = P_i \oplus C_i \quad \text{for } i = 0, 1, 2, 3$$

3. Structural Block Diagram of the 4-Bit CLA Adder

To implement this design reliably, the hardware is structured into three parallel modules based on the equations above.



Conclusion on the Role of Carries: In this architecture, the C_1, C_2, C_3 , and C_4 logic gates (comprising Stage 2) are completely decoupled from the S_i calculations of preceding bits. They serve the critical role of flattening the serial propagation path. By utilizing direct inputs and pre-computed P_i and G_i values, they feed the correct carry bit into Stage 3 almost instantaneously, empowering the adder to process the 4-bit numbers at a fraction of the time required by standard adders.

Q4. Design a 3-bit CLA circuit and show all types of “carry” for the addition $3 + 4$ with your designed CLA. **(8+4 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

A Carry Lookahead Adder (CLA) significantly reduces carry propagation delay by computing carries in parallel using early generated and propagated carry signals.

1. Mathematical Derivation for a 3-bit CLA

For any bit position i , the logic evaluates two intermediate signals:

- **Propagate Carry (P_i):** Indicates if an incoming carry will be passed to the next bit. $P_i = A_i \oplus B_i$
- **Generate Carry (G_i):** Indicates if a carry is directly created at this bit regardless of the incoming carry. $G_i = A_i \cdot B_i$

The sum and lookahead carry equations are given by:

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i \cdot C_i$$

Unrolling the carry equations for a 3-bit adder yields the **Carry Lookahead logic**:

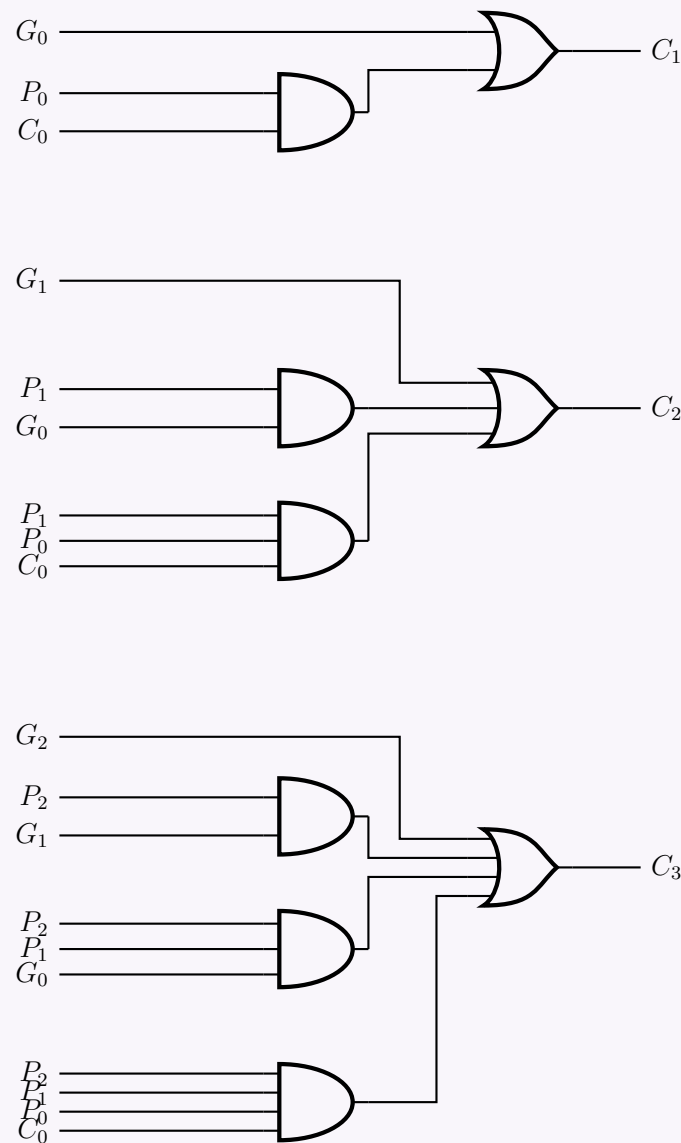
Carry 1: $C_1 = G_0 + P_0 C_0$

Carry 2: $C_2 = G_1 + P_1 C_1$
 $= G_1 + P_1(G_0 + P_0 C_0)$
 $= G_1 + P_1 G_0 + P_1 P_0 C_0$

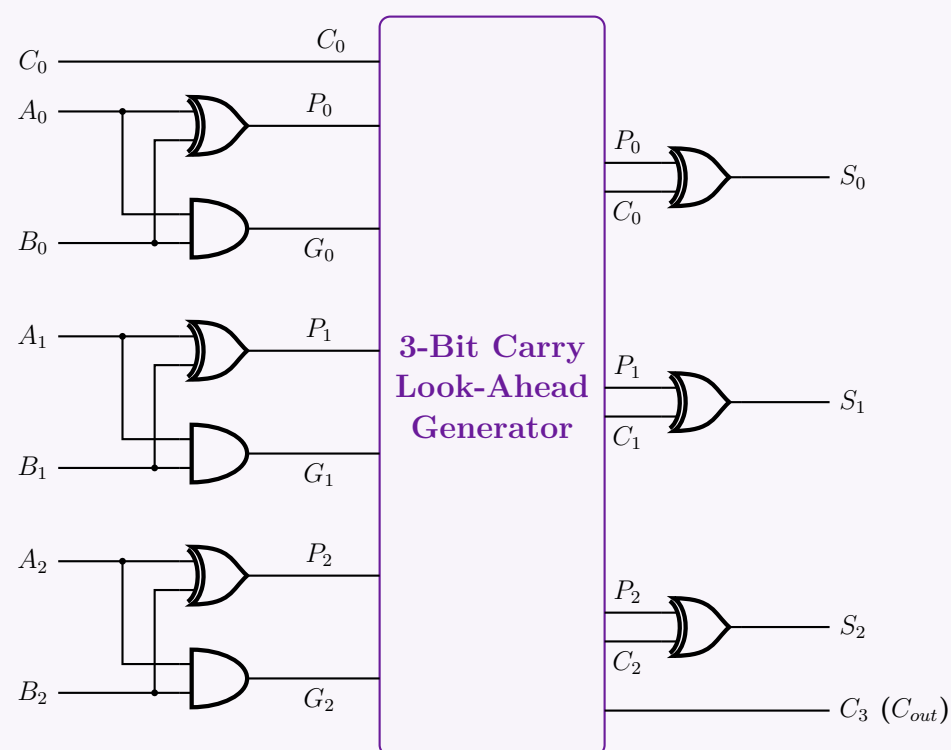
Carry 3: $C_3 = G_2 + P_2 C_2$
 $= G_2 + P_2(G_1 + P_1 G_0 + P_1 P_0 C_0)$
 $= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$

2. 3-Bit CLA Circuit Design

Below is the gate-level implementation of the **Carry Lookahead Generator** that computes C_1, C_2 , and C_3 in parallel.



The fully integrated structural top-level design incorporates the initial PG Generators, the CLA unit (designed above), and the final Sum XOR gates:



3. Application: Adding 3 + 4

We are performing the addition $A + B$, where:

- $A = 3_{10} = 011_2 \implies A_2 = 0, A_1 = 1, A_0 = 1$
- $B = 4_{10} = 100_2 \implies B_2 = 1, B_1 = 0, B_0 = 0$
- Assume an initial carry input $C_0 = 0$.

Step 1: Calculate Generate (G_i) and Propagate (P_i) Carries

Bit 0: $P_0 = A_0 \oplus B_0 = 1 \oplus 0 = 1$

Bit 1: $P_1 = A_1 \oplus B_1 = 1 \oplus 0 = 1$

Bit 2: $P_2 = A_2 \oplus B_2 = 0 \oplus 1 = 1$

$G_0 = A_0 \cdot B_0 = 1 \cdot 0 = 0$

$G_1 = A_1 \cdot B_1 = 1 \cdot 0 = 0$

$G_2 = A_2 \cdot B_2 = 0 \cdot 1 = 0$

Step 2: Calculate Internal Lookahead Carries (C_i)

$C_1 = G_0 + P_0C_0 = 0 + (1 \cdot 0) = 0$

$C_2 = G_1 + P_1C_1 = 0 + (1 \cdot 0) = 0$

$C_3 = G_2 + P_2C_2 = 0 + (1 \cdot 0) = 0$

Note: Because bits of A and B do not simultaneously contain ‘1’ in any position, $G_i = 0$ across all bits. As the starting carry $C_0 = 0$, there are no incoming carries to propagate either. Thus, all internal lookahead carries evaluate to zero.

Step 3: Calculate Final Sum (S_i)

$S_0 = P_0 \oplus C_0 = 1 \oplus 0 = 1$

$S_1 = P_1 \oplus C_1 = 1 \oplus 0 = 1$

$S_2 = P_2 \oplus C_2 = 1 \oplus 0 = 1$

Summary of all signal types (Showing all “carries”):

Bit (i)	A_i	B_i	Generate (G_i)	Propagate (P_i)	Carry In (C_i)	Sum (S_i)
0	1	0	0	1	$C_0 = 0$	1
1	1	0	0	1	$C_1 = 0$	1
2	0	1	0	1	$C_2 = 0$	1
Final Carry Out (C_3)						0

Final Result: $C_3S_2S_1S_0 = 0111_2 = 7_{10}$.

Q5. For a carry look-ahead adder circuit, what advantage do we get if we use it instead of a 4-bit simple full adder? For a NOT gate with 2 ns propagation delay and other basic gates with 4 ns propagation delay, what will be the total propagation delay? **(5 marks)**
[CSE 205 | L-2/T-1 | 2016–17]

Answer

Part 1: Advantage of a Carry Look-Ahead Adder (CLA)

In a simple 4-bit full adder (often called a **Ripple Carry Adder** or RCA), the carry bit ripples from the least significant bit (LSB) to the most significant bit (MSB). Consequently, the sum for bit i cannot be computed until the carry-out from bit $i - 1$ is ready. This creates a linear propagation delay, $O(n)$, which becomes unacceptably slow for larger bit-widths.

The primary advantage of the **Carry Look-Ahead Adder (CLA)** is that it completely eliminates this ripple effect. By calculating intermediary **Generate** (G_i) and **Propagate** (P_i) signals, the CLA computes all carry signals (C_1, C_2, C_3, C_4) **in parallel** using 2-level Sum-of-Products (SOP) logic.

- Speed:** The carry generation delay becomes constant (theoretical $O(1)$ gate levels), drastically reducing the overall addition time.
- Independence:** Each carry bit is calculated directly from the inputs (A, B) and the initial carry-in (C_0), without waiting for the intermediate sum/carry of preceding bits.

Part 2: Total Propagation Delay Calculation

The problem explicitly provides the delay of a NOT gate (2 ns) and other basic gates (4 ns). Standard “basic gates” include AND and OR gates. Because the NOT gate delay is distinctly specified, we must decompose the XOR gate (which is a derived gate) into its constituent basic gates to accurately trace the delay.

1. Delay of an XOR Gate

The XOR operation $X \oplus Y = X\bar{Y} + \bar{X}Y$ is constructed using NOT, AND, and OR gates:

NOT gates ($\bar{X}, \bar{Y}$ delay): 2 ns

AND layer ($X\bar{Y}, \bar{X}Y$ delay): 2 ns + 4 ns = 6 ns

OR layer (Final XOR delay): 6 ns + 4 ns = **10 ns**

Thus, any XOR operation introduces a 10 ns delay from the time its latest input is available.

2. Stage-by-Stage Delay Trace for the CLA

Assume all primary inputs (A_i, B_i, C_0) arrive simultaneously at $t = 0$ ns.

Stage 1: Generate (G_i) and Propagate (P_i) Signals

The signals are computed in parallel for all bits:

$$\begin{aligned} G_i &= A_i \cdot B_i & (1 \text{ AND gate}) \implies \text{Valid at } t = 0 + 4 = \mathbf{4 \text{ ns}} \\ P_i &= A_i \oplus B_i & (1 \text{ XOR logic}) \implies \text{Valid at } t = 0 + 10 = \mathbf{10 \text{ ns}} \end{aligned}$$

Stage 2: Carry Look-Ahead Logic (C_i)

The carry bits are evaluated using 2-level AND-OR logic. For example, C_4 :

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

- The inputs to this block are the G_i signals (arriving at 4 ns) and P_i signals (arriving at 10 ns).
- AND Layer:** Must wait for the slowest input, which is P_i at 10 ns. Outputs of the AND gates are valid at $t = 10 \text{ ns} + 4 \text{ ns} = \mathbf{14 \text{ ns}}$.
- OR Layer:** Takes the outputs of the AND layer. Outputs of the OR gates (all C_i) are valid at $t = 14 \text{ ns} + 4 \text{ ns} = \mathbf{18 \text{ ns}}$.

Stage 3: Final Sum Generation (S_i)

The sum is computed as $S_i = P_i \oplus C_i = P_i\bar{C}_i + \bar{P}_iC_i$.

We track the arrival times of the inputs to this final XOR logic:

- P_i is available at 10 ns.
- C_i is available at 18 ns.
- NOT Layer:** Computes $\bar{P}_i$ and $\bar{C}_i$.
 - $\bar{P}_i$ is valid at $10 \text{ ns} + 2 \text{ ns} = 12 \text{ ns}$.
 - $\bar{C}_i$ is valid at $18 \text{ ns} + 2 \text{ ns} = \mathbf{20 \text{ ns}}$.
- AND Layer ($P_i\bar{C}_i$ and $\bar{P}_iC_i$):** The slowest input is $\bar{C}_i$ at 20 ns. The AND gates evaluate at $t = 20 \text{ ns} + 4 \text{ ns} = \mathbf{24 \text{ ns}}$.
- OR Layer (Final Sum):** The final OR gate evaluates at $t = 24 \text{ ns} + 4 \text{ ns} = \mathbf{28 \text{ ns}}$.

Conclusion: The total propagation delay to compute the final sum and carry-out in this Carry Look-Ahead Adder is **28 ns**.

Note: If a simpler architecture is assumed where the XOR gate is treated intrinsically as a standard basic gate with a 4 ns delay (ignoring the provided NOT gate delay parameter), the calculation simplifies to: P_i ready at 4 ns $\rightarrow$ Carry ANDs at 8 ns $\rightarrow$ Carry ORs at 12 ns $\rightarrow$ Final Sum XOR at 16 ns. However, utilizing the explicitly provided NOT delay requires the standard decomposition shown above.

Q6. What is the problem with the binary parallel adder circuit? Design and implement a 4-bit carry look-ahead adder circuit. If the propagation delay is 1 ns for a NOT gate and 2 ns for other basic gates, what is the propagation delay of the CLA adder to produce its final output? **(18 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Answer

1. The Problem with the Binary Parallel Adder

A standard Binary Parallel Adder (also known as a **Ripple Carry Adder** or RCA) cascades multiple 1-bit full adders to add n -bit numbers. The primary problem with this architecture is the **carry propagation delay**. In an RCA, the sum S_i and the carry-out C_{i+1} for any bit position i cannot be correctly computed until the carry-in C_i from the previous stage has been evaluated. Consequently, the carry must “ripple” sequentially from the least significant bit (LSB) to the most significant bit (MSB). For an n -bit adder, the worst-case propagation delay is $O(n)$, which acts as a major bottleneck in high-speed computing systems.

2. Design and Implementation of a 4-Bit Carry Look-Ahead Adder

A Carry Look-Ahead Adder (CLA) solves the ripple delay by computing all carry signals simultaneously using two intermediate signals for each bit:

- Generate ($G_i = A_i \cdot B_i$):** Produces a carry out regardless of the carry in.
- Propagate ($P_i = A_i \oplus B_i$):** Propagates a carry in to the carry out.

The standard sum and carry formulas are:

$$\begin{aligned} S_i &= P_i \oplus C_i \\ C_{i+1} &= G_i + P_i \cdot C_i \end{aligned}$$

By unrolling the carry equations, we derive independent carry logic for the 4-bit CLA:

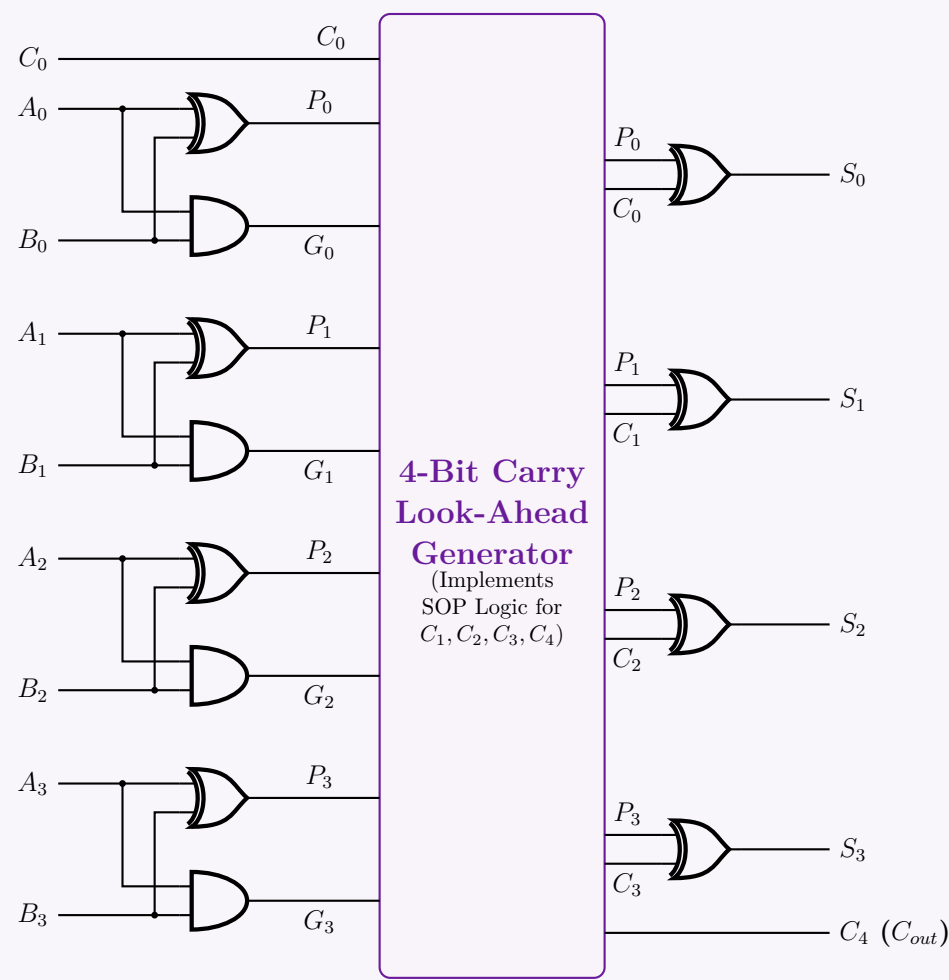
$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

Structural Block Diagram of the 4-Bit CLA:



3. Propagation Delay Calculation

We are given the delay of a **NOT gate** (1 ns) and **other basic gates** (2 ns) (i.e., AND/OR gates). Because the NOT gate delay is specifically provided, we must decompose the complex XOR gates into their basic gate equivalents: $X \oplus Y = X\bar{Y} + \bar{X}Y$.

NOT layer ($\bar{X}, \bar{Y}$): 1 ns

AND layer ($X\bar{Y}, \bar{X}Y$): 1 ns + 2 ns = 3 ns

OR layer (Final XOR delay): 3 ns + 2 ns = **5 ns**

Thus, an XOR operation yields a delay of 5 ns. Now, assuming all inputs (A_i, B_i, C_0) arrive at $t = 0$ ns, we trace the CLA critical path:

Stage 1: Generate (G_i) and Propagate (P_i) Signals

- $G_i = A_i \cdot B_i$ uses 1 AND gate $\Rightarrow$ Ready at $t = 0 + 2 = \mathbf{2}$ ns.
- $P_i = A_i \oplus B_i$ uses 1 XOR gate $\Rightarrow$ Ready at $t = 0 + 5 = \mathbf{5}$ ns.

Stage 2: Carry Look-Ahead Logic (C_1, C_2, C_3, C_4) The logic evaluates 2-level Sum-of-Products (SOP) equations (AND gates followed by OR gates).

- **AND Layer:** Takes inputs G_i (2 ns), P_i (5 ns), and C_0 (0 ns). It must wait for the slowest input (P_i at 5 ns).
Output of AND layer $\Rightarrow 5$ ns + 2 ns = **7 ns**.
- **OR Layer:** Takes the output of the AND layer.
Output of OR layer (all C_i and C_4) $\Rightarrow 7$ ns + 2 ns = **9 ns**.

(Note: The final carry out C_4 is completely evaluated and valid at **9 ns**.)

Stage 3: Final Sum Generation (S_i) The final sum is $S_i = P_i \oplus C_i = P_i\bar{C}_i + \bar{P}_iC_i$.

- **Inputs Available:** P_i is ready at 5 ns, and C_i is ready at 9 ns.
- **NOT Layer:** $\bar{P}_i$ is ready at $5 + 1 = 6$ ns, and $\bar{C}_i$ is ready at $9 + 1 = 10$ ns.
- **AND Layer:** Must wait for the slowest input ($\bar{C}_i$ at 10 ns).
Evaluation finishes at $t = 10 \text{ ns} + 2 \text{ ns} = 12 \text{ ns}$.
- **OR Layer:** Generates the final Sum.
Evaluation finishes at $t = 12 \text{ ns} + 2 \text{ ns} = 14 \text{ ns}$.

Conclusion: The propagation delay for the CLA adder to produce its final output (the final Sum bit) is **14 ns**.

Q7. What are the advantages of a CLA over a normal binary four-bit adder? Assume that the exclusive-OR gate has a propagation delay of 10 ns and that the AND or OR gates have a propagation delay of 5 ns. What is the total propagation delay time in the four-bit CLA adder? (10 marks) [CSE 205 | L-2/T-1 | 2013–14]

Answer

1. Advantages of a CLA over a Normal Binary Four-Bit Adder

A “normal” binary four-bit adder, typically implemented as a **Ripple Carry Adder (RCA)**, cascades single-bit full adders in a series. The primary advantages of replacing this with a **Carry Look-Ahead Adder (CLA)** are:

- **Elimination of the Ripple Effect:** In an RCA, the carry propagates sequentially from the LSB to the MSB, resulting in a linear delay $O(n)$. A CLA precomputes the carry signals in parallel using Generate (G_i) and Propagate (P_i) logic, ensuring that the MSB does not have to wait for the carry to ripple through all preceding bits.
- **Constant Carry Delay:** Regardless of the bit-width of the adder block (up to a fan-in limit), the carry outputs (C_1, C_2, C_3, C_4) are generated simultaneously via a 2-level Sum-of-Products (SOP) circuit. This significantly boosts the clock speed capability of the CPU’s Arithmetic Logic Unit (ALU).
- **Faster Overall Addition:** Because the carries arrive much earlier, the final Sum ($S_i = P_i \oplus C_i$) bits are also evaluated much faster compared to a standard RCA.

2. Total Propagation Delay Calculation

We are given the following inherent gate delays:

- **XOR gate delay:** 10 ns
- **AND gate delay:** 5 ns
- **OR gate delay:** 5 ns

Assume all primary inputs (A_i, B_i , and initial carry C_0) are applied simultaneously at $t = 0$ ns. The calculation proceeds stage by stage based on the critical path of the CLA architecture.

Stage 1: Generate (G_i) and Propagate (P_i) Signals For each bit i , these signals are calculated directly from the inputs in parallel:

$$\begin{aligned} G_i &= A_i \cdot B_i & (1 \text{ AND gate}) \implies \text{Valid at } t = 0 + 5 = 5 \text{ ns} \\ P_i &= A_i \oplus B_i & (1 \text{ XOR gate}) \implies \text{Valid at } t = 0 + 10 = 10 \text{ ns} \end{aligned}$$

Stage 2: Carry Look-Ahead Logic (C_1, C_2, C_3, C_4) The look-ahead logic utilizes a 2-level AND-OR structure (Sum-of-Products) for all carries. For example, the longest expression is:

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

- **AND Layer:** The inputs to the AND gates are G_i (arriving at 5 ns), P_i (arriving at 10 ns), and C_0 (0 ns). The AND gates must wait for the slowest input, which is P_i at 10 ns.
Evaluating the AND gates adds 5 ns: $t = 10 \text{ ns} + 5 \text{ ns} = 15 \text{ ns}$.
- **OR Layer:** The OR gates sum the outputs of the AND layer (which arrive at 15 ns) and the standalone G_i signals (which arrive early at 5 ns). It waits for the slowest input at 15 ns.
Evaluating the OR gates adds 5 ns: $t = 15 \text{ ns} + 5 \text{ ns} = 20 \text{ ns}$.

All internal carry bits (C_1, C_2, C_3) and the final carry-out (C_4) are valid at 20 ns.

Stage 3: Final Sum Generation (S_i) The final sum for each bit is calculated using the equation:

$$S_i = P_i \oplus C_i$$

- **Inputs:** The XOR gate receives P_i (ready at 10 ns) and C_i (ready at 20 ns).
- **Evaluation:** The XOR gate must wait for the slowest input, which is C_i at 20 ns. Evaluating the XOR gate adds 10 ns: $t = 20 \text{ ns} + 10 \text{ ns} = \text{30 ns}$.

Conclusion:
The total propagation delay time for the four-bit CLA adder to stabilize all outputs (including the Sum bits and the final Carry-out) is **30 ns**.

2’s Complement Circuit

Q1. Using only half adders, design a circuit that outputs the 1’s complement of a 4-bit input when the control input C is 1, and passes the input unchanged when C is 0. **(6 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Answer

1. Logical Analysis and Formulation

Let the 4-bit input be $A = A_3A_2A_1A_0$ and the required 4-bit output be $Y = Y_3Y_2Y_1Y_0$. We are given a control signal C which determines the operation:

- When $C = 0$, the output passes the input unchanged: $Y_i = A_i$.
- When $C = 1$, the output is the 1’s complement of the input: $Y_i = A'_i$.

We can formulate the truth table for a single bit i (where $i \in \{0, 1, 2, 3\}$):

Control (C)	Input (A_i)	Desired Output (Y_i)	Description
0	0	0	Pass Unchanged ($Y_i = A_i$)
0	1	1	
1	0	1	1’s Complement ($Y_i = A'_i$)
1	1	0	

Extracting the Boolean expression from the truth table via sum-of-products for Y_i , we get:

$$Y_i = C' A_i + C A'_i$$
$$Y_i = C \oplus A_i \quad (\text{Exclusive-OR Operation})$$

2. Implementation using Half Adders

The problem restricts us to using **only Half Adders (HA)**. Let’s review the Boolean equations for a standard Half Adder with inputs X and Y :

$$S \text{ (Sum)} = X \oplus Y$$
$$C_{out} \text{ (Carry)} = X \cdot Y$$

Notice that the Sum (S) output of a Half Adder inherently performs an XOR operation on its two inputs. Therefore, if we assign the Half Adder inputs as:

- $X = A_i$ (Data bit)
- $Y = C$ (Control signal)

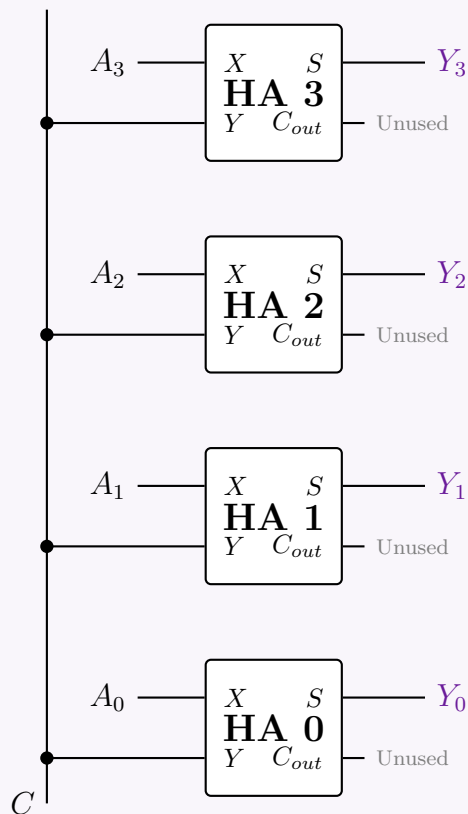
Then the Sum output becomes:

$$S_i = A_i \oplus C = Y_i$$

The Carry output ($C_{out} = A_i \cdot C$) is not needed for this operation and will simply be left unconnected (unused). To process a 4-bit input, we will employ exactly four Half Adders in parallel.

3. Schematic Diagram

The following circuit diagram illustrates the 4-bit conditional complements constructed entirely from Half Adders.



Q2. Design a circuit that takes a 4-bit number as input and produces the 2's complement of the given number. **(20 marks)** [CSE 205 | L-2/T-1 | 2011–12]

Answer

1. Logical Formulation and Algorithm

To design a circuit that generates the 2's complement of a 4-bit binary input $A = A_3A_2A_1A_0$, we can utilize the well-known “look-ahead” or bit-scan algorithm for 2's complement:

Starting from the least significant bit (LSB) up to the most significant bit (MSB), leave all bits unchanged until the first ‘1’ is encountered. Leave that first ‘1’ unchanged, and then invert all subsequent higher-order bits.

We can implement this conditionally using Exclusive-OR (XOR) gates. An XOR gate acts as a controlled inverter:

- $X \oplus 0 = X$ (Passes unchanged)
- $X \oplus 1 = X'$ (Inverts)

We will generate a sequence of “Enable Inversion” signals, denoted as E_i , where $E_i = 1$ if any bit from position 0 to $i - 1$ is 1.

2. Boolean Equations

Let the 4-bit input be $A_3A_2A_1A_0$ and the 2's complement output be $Y_3Y_2Y_1Y_0$.

- **Bit 0 (LSB):** Never inverted.

$$E_0 = 0$$

$$Y_0 = A_0 \oplus 0 = A_0$$

- **Bit 1:** Inverted only if $A_0 = 1$.

$$E_1 = A_0$$

$$Y_1 = A_1 \oplus E_1$$

- **Bit 2:** Inverted if either A_0 or A_1 is 1.

$$E_2 = E_1 \vee A_1 \quad (\text{Generated using an OR gate})$$

$$Y_2 = A_2 \oplus E_2$$

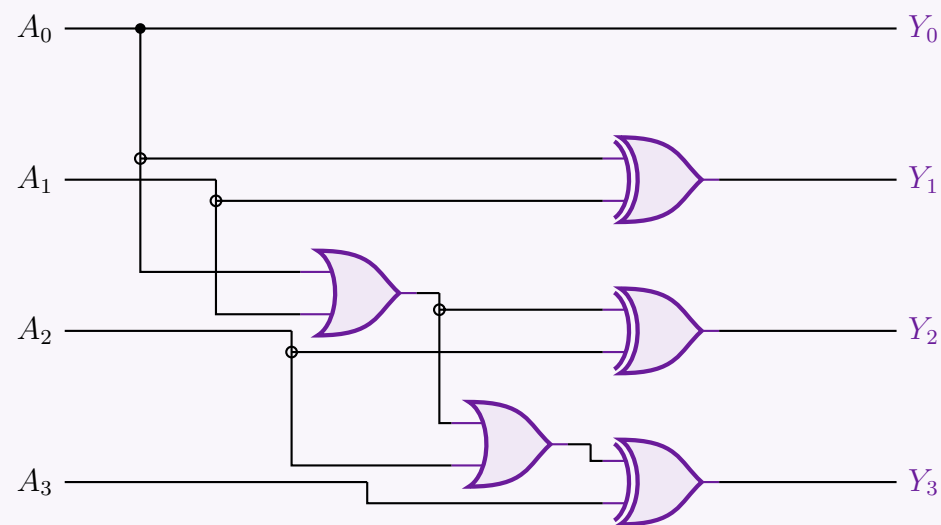
- **Bit 3 (MSB):** Inverted if A_0, A_1 , or A_2 is 1.

$$E_3 = E_2 \vee A_2 \quad (\text{Generated using an OR gate})$$

$$Y_3 = A_3 \oplus E_3$$

3. Logic Circuit Diagram

Using the derived Boolean equations, we can construct the logic circuit using cascading OR gates to propagate the E_i signals, and XOR gates to generate the final Y_i output bits.



Note: An alternative method is to simply use four NOT gates to find the 1's complement of the inputs and then feed those inverted signals into a 4-bit Parallel Adder integrated circuit (such as the 7483) with the initial carry-in (C_{in}) permanently tied to logic 1. However, the dedicated combinatorial network shown above directly achieves the target with minimal propagation delay and gate count.

Overflow Detection

Q1. What is overflow in binary arithmetic? Explain how overflow is detected in signed addition and signed subtraction. Provide suitable examples. (11 marks) [CSE 205 | L-2/T-1 | 2024–25]

Answer

1. Definition of Overflow

In binary arithmetic, **overflow** is an error condition that occurs when the true mathematical result of an arithmetic operation exceeds the maximum or minimum numerical value that the designated fixed-width hardware register can represent.

In an n -bit signed 2's complement representation system, the range of valid integers is bounded by:

$$[-2^{n-1}, 2^{n-1} - 1]$$

When an operation yields a result outside this window, an overflow occurs, leading to corrupted data (e.g., adding two positive numbers yields a negative value, or vice versa).

Crucial Distinction (Carry vs. Overflow): A *carry-out* from the most significant bit (MSB) simply indicates that the unsigned magnitude exceeded the capacity, which is expected and normal in signed arithmetic. An *overflow* indicates that the *signed* value is mathematically invalid for the given bit-width.

2. Overflow Detection Methods

A. Signed Addition

There are two primary techniques used to detect overflow during the addition of two signed binary numbers ($A + B = Y$):

(a) **Sign Bit Analysis Rule:** Overflow can *only* happen when adding two numbers of the same sign.

- Positive + Positive = Negative $\implies$ **Overflow**
- Negative + Negative = Positive $\implies$ **Overflow**

Adding a positive number and a negative number can never cause an overflow because the absolute magnitude of the sum is guaranteed to be smaller than or equal to the largest operand.

(b) **The Carry XOR Rule (Hardware Implementation):** Let C_{n-1} be the internal carry generated *into* the MSB (sign bit position), and C_n be the final carry-out generated *out* of the MSB. The overflow hardware status flag (V) is directly computed via an Exclusive-OR operation:

$$V = C_{n-1} \oplus C_n$$

If $V = 1$, an overflow has occurred.

B. Signed Subtraction

Signed subtraction ($A - B$) is conventionally converted into an addition problem by taking the 2's complement of the subtrahend (B) and adding it to the minuend (A), such that $A - B = A + (-B)$.

(a) **Sign Bit Analysis Rule:** Overflow occurs if and only if the operands have opposite signs and the result matches the sign of the subtrahend:

- Positive − Negative = Negative $\implies$ **Overflow**
- Negative − Positive = Positive $\implies$ **Overflow**

(b) **Hardware Implementation:** Because the subtraction is physically transformed into an addition operation inside the Arithmetic Logic Unit (ALU), the exact same hardware check ($V = C_{n-1} \oplus C_n$) evaluates overflow seamlessly using the intermediate carries of the adder block.

—

3. Illustrative Examples (4-Bit Signed System)

In a 4-bit signed 2’s complement system, the range of valid numbers is $[-2^3, 2^3 - 1] = [-8, +7]$.

Example 1: Signed Addition Overflow (Positive + Positive)

Compute $(+5) + (+4)$. The true mathematical answer is $+9$, which is outside the range $[-8, +7]$.

Step	Carry Line	Bit 3 (MSB)	Bit 2	Bit 1	Bit 0	Signed Value
Carries:	C_4 = 0	C_3 = 1	0	0		
Operand 1 (A):		0	1	0	1	+5
Operand 2 (B):	+	0	1	0	0	+4
Result (Y):		1	0	0	1	−7 (Incorrect)

- **Sign Check:** Two positive numbers ($A_3 = 0, B_3 = 0$) yielded a negative result ($Y_3 = 1$). This reveals an overflow condition.
- **XOR Rule Evaluation:**

$$V = C_3 \oplus C_4$$
$$V = 1 \oplus 0 = 1 \text{ (Overflow Detected)}$$

Example 2: Signed Subtraction Overflow (Negative - Positive)

Compute $(-6) - (+3)$. The true mathematical answer is -9 , which falls outside $[-8, +7]$. We transform this into addition: $(-6) + (-3)$.

- -6 in 4-bit 2’s complement form is 1010_2 .
- -3 in 4-bit 2’s complement form is 1101_2 .

Step	Carry Line	Bit 3 (MSB)	Bit 2	Bit 1	Bit 0	Signed Value
Carries:	C_4 = 1	C_3 = 0	0	0		
Minuend (A):		1	0	1	0	−6
−Subtrahend (−B):	+	1	1	0	1	−3
Result (Y):		0	1	1	1	+7 (Incorrect)

- **Sign Check:** A negative number minus a positive number yielded a positive result ($Y_3 = 0$). This reveals an overflow condition.
- **XOR Rule Evaluation:**

$$V = C_3 \oplus C_4$$
$$V = 0 \oplus 1 = 1 \text{ (Overflow Detected)}$$

Q2. How do you check for overflow in the case of addition of two binary numbers in both signed and unsigned representations? (5 marks)
[CSE 205 | L-2/T-1 | 2013–14]

Answer

1. **Overflow in Unsigned Binary Addition**

In an n -bit unsigned binary system, all bits are used to represent the magnitude of a positive number. The valid range of representable integers is:

$$[0, \quad 2^n - 1]$$

Overflow Condition: An overflow occurs when the sum of two unsigned numbers exceeds the maximum capacity ($2^n - 1$), meaning the result requires $n + 1$ bits to be represented correctly.

Detection Mechanism: In hardware, unsigned overflow is detected simply by examining the **final carry-out** (C_{out}) from the Most Significant Bit (MSB) position.

- If $C_{out} = 1$, an **overflow has occurred**.

- If $C_{out} = 0$, the sum is valid and bounded within n bits.

Example (4-bit Unsigned, Range: 0 to 15): Add 12_{10} and 5_{10} .

Step	Carry	Bit 3	Bit 2	Bit 1	Bit 0	Decimal
Carries:	1	0	0	0		
Operand A:		1	1	0	0	12
Operand B:	+	0	1	0	1	5
Result (S):	$C_{out} = 1$	0	0	0	1	1 (Incorrect)

The true sum is 17, which exceeds 15. The $C_{out} = 1$ correctly flags this overflow.

2. Overflow in Signed Binary Addition (2’s Complement)

In an n -bit signed 2’s complement system, the MSB acts as the sign bit (0 for positive, 1 for negative). The valid range of representable integers is:

$$[-2^{n-1}, \quad 2^{n-1} - 1]$$

Overflow Condition: An overflow occurs when the sum of two numbers with the same sign yields a result that falls outside this valid range, causing the sign bit of the sum to flip incorrectly. *Note: Adding a positive and a negative number can NEVER cause an overflow.*

Detection Mechanisms: There are two primary ways to detect signed overflow:

Method A: Sign-Bit Observation (Logical Rule) Let the sign bits of the addends be A_{n-1} and B_{n-1} , and the sign bit of the sum be S_{n-1} . Overflow occurs if the addends have the same sign, but the sum has a different sign:

$$\text{Overflow } (V) = A_{n-1}B_{n-1}S'_{n-1} + A'_{n-1}B'_{n-1}S_{n-1}$$

Method B: Carry XOR (Hardware Implementation) Let C_{n-1} be the internal carry *into* the MSB, and C_n be the carry *out* of the MSB. The overflow flag (V) is strictly the Exclusive-OR (XOR) of these two carries:

$$V = C_{n-1} \oplus C_n$$

If $V = 1$, an overflow has occurred. (The carry out is ignored for the actual arithmetic value, but crucial for this XOR check).

Example (4-bit Signed, Range: -8 to +7): Add $+5_{10}$ and $+4_{10}$.

Step	Carry	Bit 3 (Sign)	Bit 2	Bit 1	Bit 0	Decimal
Carries:	$C_4 = 0$	$C_3 = 1$	0	0		
Operand A:		0	1	0	1	+5
Operand B:	+	0	1	0	0	+4
Result (S):		1	0	0	1	-7 (Incorrect)

The true sum is +9, which exceeds +7. We evaluate the hardware overflow condition:

$$\begin{aligned} V &= C_3 \oplus C_4 \\ V &= 1 \oplus 0 = \mathbf{1} \quad (\text{Overflow Detected}) \end{aligned}$$

Additionally, observing the sign bits (Method A): +5 and +4 are positive (0), but the sum appears negative (1). This confirms the overflow.

Binary Multiplier

Q1. Discuss and design a 2-bit by 2-bit binary multiplier. (15 marks)

[CSE 205 | L-2/T-1 | 2021–22]

Answer

1. Theoretical Foundation and Logical Formulation

A 2-bit by 2-bit binary multiplier takes two 2-bit unsigned inputs:

- **Multiplicand:** $A = A_1A_0$
- **Multiplier:** $B = B_1B_0$

The maximum possible value for both A and B is 3_{10} (11_2). Therefore, the maximum product is $3 \times 3 = 9_{10}$, which requires 4 bits to represent (1001_2). Let the 4-bit product be $P = P_3P_2P_1P_0$.

The binary multiplication process is identical to decimal multiplication, relying on the generation of *partial products* and their subsequent addition. In binary, a partial product is simply the logical AND of two bits.

		A_1		A_0
$\times$		B_1		B_0
		A_1B_0	A_0B_0	(Partial Product 1)
$+$	A_1B_1	A_0B_1		(Partial Product 2, shifted)
	P_3	P_2	P_1	P_0 (Final Product)

2. Boolean Equations for Product Bits

From the columnar addition in the tableau above, we can derive the exact logical requirements for each product bit P_i .

Bit 0 (P_0): The least significant bit is formed directly by the AND operation of the LSBs.
 $P_0 = A_0 \cdot B_0$

Bit 1 (P_1): Requires adding two partial product bits: A_1B_0 and A_0B_1 . This is done using a Half Adder.
 $P_1 = (A_1 \cdot B_0) \oplus (A_0 \cdot B_1)$ (Sum output of HA1)
 $C_1 = (A_1 \cdot B_0) \cdot (A_0 \cdot B_1)$ (Carry output of HA1)

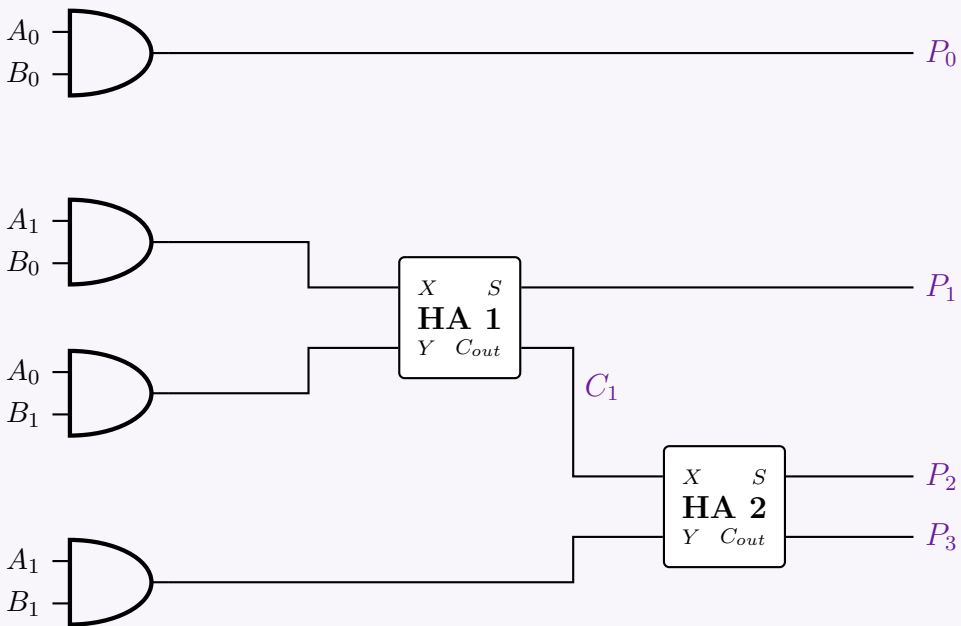
Bit 2 (P_2): Requires adding the next partial product bit (A_1B_1) and the carry C_1 from the previous stage.
We use a second Half Adder.
 $P_2 = (A_1 \cdot B_1) \oplus C_1$ (Sum output of HA2)
 $C_2 = (A_1 \cdot B_1) \cdot C_1$ (Carry output of HA2)

Bit 3 (P_3): The most significant bit is simply the final carry out.
 $P_3 = C_2$

3. Circuit Implementation

To implement this design, we require:

- **Four 2-input AND gates** to generate the partial products: A_0B_0 , A_1B_0 , A_0B_1 , and A_1B_1 .
- **Two Half Adders (HA)** to sum the aligned columns and propagate carries.



Q2. Design a circuit with a 4-bit binary adder which returns $A \times B$ where A and B are 3-bit binary numbers. (10 marks) [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Logical Formulation of 3×3 Multiplication

Let the two 3-bit unsigned binary numbers be the Multiplicand $A = A_2A_1A_0$ and the Multiplier $B = B_2B_1B_0$. The maximum value for each is 7_{10} (111_2), meaning the maximum product is 49_{10} (110001_2). Thus, the final product requires 6 bits: $P = P_5P_4P_3P_2P_1P_0$. The multiplication is achieved by generating three *partial products* (PP_0, PP_1, PP_2) using 2-input AND gates, and then summing

them based on their positional weights.

			A_2	A_1	A_0	
	$\times$		B_2	B_1	B_0	
			A_2B_0	A_1B_0	A_0B_0	(PP_0)
		A_2B_1	A_1B_1	A_0B_1		$(PP_1, \text{shifted})$
	$+$	A_2B_2	A_1B_2	A_0B_2		$(PP_2, \text{shifted})$
		P_5	P_4	P_3	P_2	P_1
					P_0	(Final Product)

Because there are three partial products to add, a purely combinational implementation requires **two addition stages**. We will construct this using **two 4-bit binary parallel adders**.

2. Stage-by-Stage Adder Configuration

Stage 1 (Adder 1): Adds the upper bits of PP_0 with PP_1 .

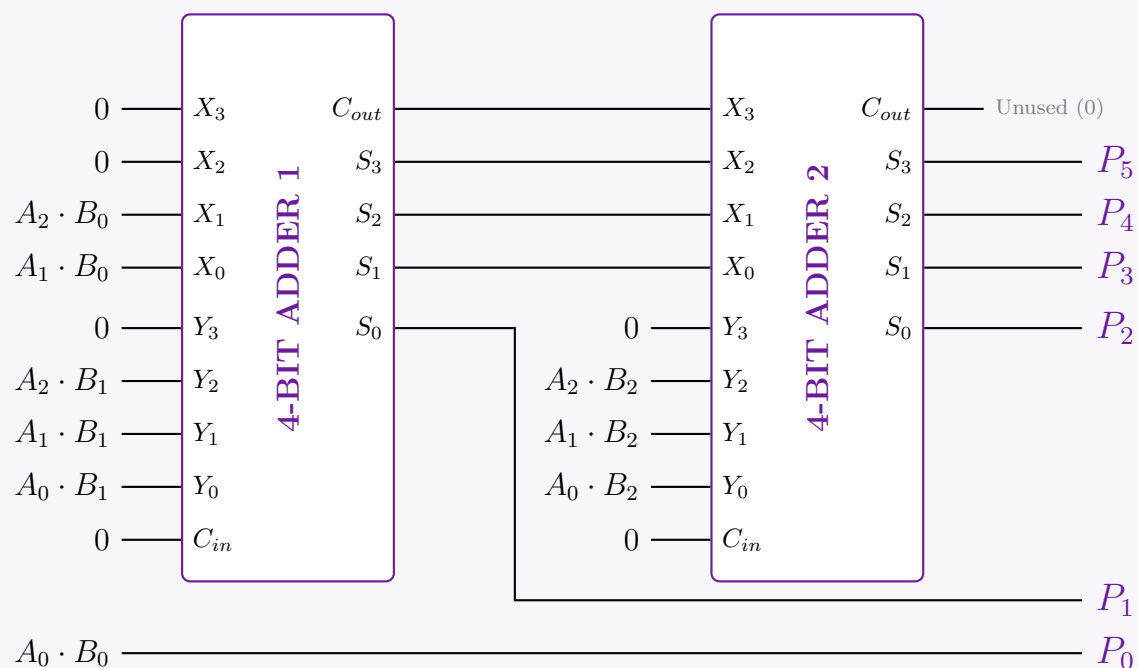
- P_0 is simply A_0B_0 and requires no addition. It bypasses the adders.
- Input X** (PP_0 components): $X_3 = 0, X_2 = 0, X_1 = A_2B_0, X_0 = A_1B_0$.
- Input Y** (PP_1 components): $Y_3 = 0, Y_2 = A_2B_1, Y_1 = A_1B_1, Y_0 = A_0B_1$.
- Output:** Yields sum bits S_3, S_2, S_1, S_0 and carry C_{out1} .
- The least significant sum bit represents $P_1 = S_0$.

Stage 2 (Adder 2): Adds the shifted intermediate sum from Stage 1 to PP_2 .

- Input X** (Shifted Sum 1): $X_3 = C_{out1}, X_2 = S_3, X_1 = S_2, X_0 = S_1$.
- Input Y** (PP_2 components): $Y_3 = 0, Y_2 = A_2B_2, Y_1 = A_1B_2, Y_0 = A_0B_2$.
- Output:** Yields sum bits S'_3, S'_2, S'_1, S'_0 . The carry C_{out2} is always 0.
- These sum bits directly form the remaining product bits: $P_5 = S'_3, P_4 = S'_2, P_3 = S'_1, P_2 = S'_0$.

3. Logic Circuit Schematic

The following diagram illustrates the complete 3x3 multiplier utilizing two 4-bit adders. (Note: The 9 logical products like $A_0 \cdot B_0$ are assumed to be pre-generated via a parallel bank of standard 2-input AND gates).



Q3. Using 4-bit adders, design a 3-bit multiplier for 3-bit variables $X(X_1, X_2, X_3)$ and $Y(Y_1, Y_2, Y_3)$. Explain the design. **(15 marks)**
[CSE 205 | L-2/T-1 | 2016–17]

Answer

1. Mathematical Formulation of the 3×3 Multiplier

Let the two 3-bit variables be represented as:

- Multiplicand:** $X = X_3X_2X_1$ (where X_3 is the MSB and X_1 is the LSB)
- Multiplier:** $Y = Y_3Y_2Y_1$ (where Y_3 is the MSB and Y_1 is the LSB)

Multiplying a 3-bit number by a 3-bit number yields a maximum product of $7 \times 7 = 49_{10}$ (110001_2), which requires 6 bits. Let the

final product be $P = P_6P_5P_4P_3P_2P_1$.

We generate partial products by multiplying the multiplicand by each bit of the multiplier using AND gates, and then we sum these partial products according to their binary weights.

				X_3	X_2	X_1	
				Y_3	Y_2	Y_1	
				X_3Y_1	X_2Y_1	X_1Y_1	(PP_1 , weight 2^0)
				X_3Y_2	X_2Y_2	X_1Y_2	(PP_2 , weight 2^1)
				X_3Y_3	X_2Y_3	X_1Y_3	(PP_3 , weight 2^2)
+							
	P_6	P_5	P_4	P_3	P_2	P_1	(Final Product)

2. Design Strategy Using 4-Bit Adders

To sum these three rows of partial products, we will cascade **two 4-bit binary adders**. A standard 4-bit adder has inputs $A_3A_2A_1A_0$ and $B_3B_2B_1B_0$, generating sums $S_3S_2S_1S_0$ and a carry-out C_{out} . We map the partial products to the adder inputs strictly based on their positional weights.

Stage 1: Adding PP_1 and PP_2 (Adder 1)

We align PP_1 and PP_2 into the first 4-bit adder.

- **Weight 2^0 :** $A_0 = X_1Y_1$, $B_0 = 0$. The sum S_0 becomes the final product bit **P_1** .
- **Weight 2^1 :** $A_1 = X_2Y_1$, $B_1 = X_1Y_2$. The sum S_1 becomes the final product bit **P_2** .
- **Weight 2^2 :** $A_2 = X_3Y_1$, $B_2 = X_2Y_2$. The sum is S_2 .
- **Weight 2^3 :** $A_3 = 0$, $B_3 = X_3Y_2$. The sum is S_3 .
- **Carry Out:** C_{out1} has a positional weight of 2^4 .

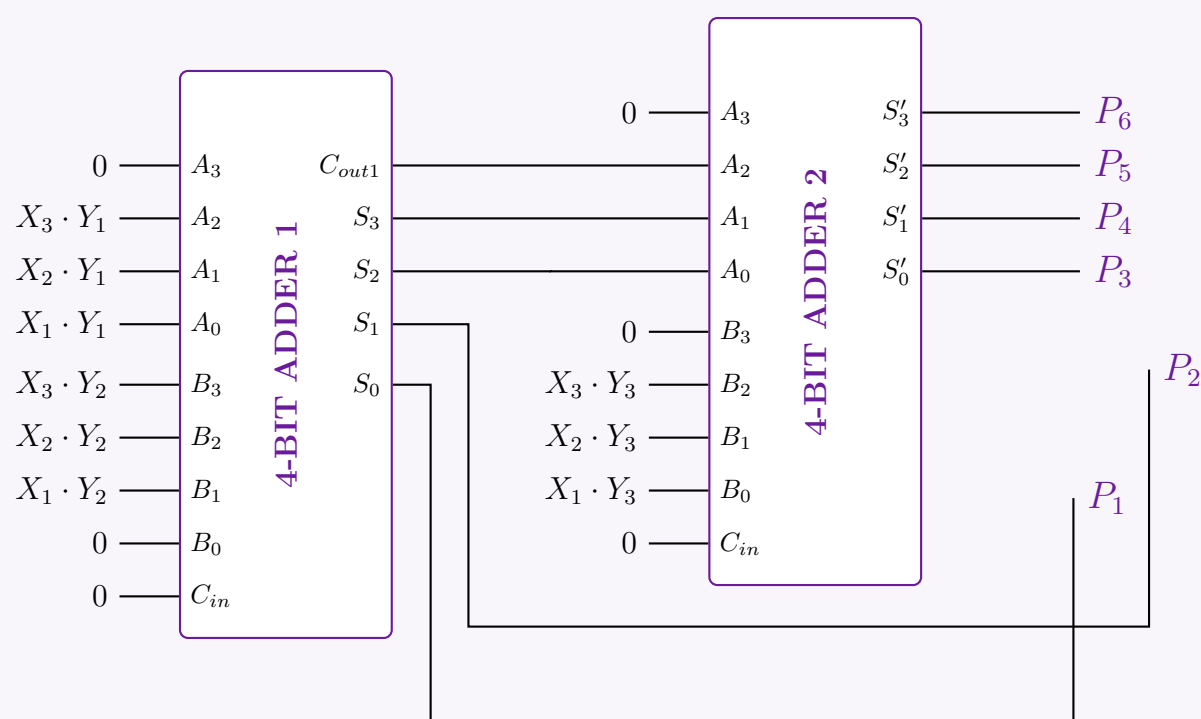
Stage 2: Adding the Intermediate Sum to PP_3 (Adder 2)

We take the unresolved upper bits from Adder 1 (S_2, S_3, C_{out1}) and add them to PP_3 . Notice that PP_3 starts at weight 2^2 .

- **Weight 2^2 :** $A_0 = S_2$ (from Adder 1), $B_0 = X_1Y_3$. The sum S'_0 becomes **P_3** .
- **Weight 2^3 :** $A_1 = S_3$ (from Adder 1), $B_1 = X_2Y_3$. The sum S'_1 becomes **P_4** .
- **Weight 2^4 :** $A_2 = C_{out1}$ (from Adder 1), $B_2 = X_3Y_3$. The sum S'_2 becomes **P_5** .
- **Weight 2^5 :** $A_3 = 0$, $B_3 = 0$. The sum S'_3 absorbs the carry and becomes **P_6** .

3. Logic Circuit Schematic

The circuit diagram below illustrates the exact routing of the partial products into the two cascaded 4-bit adders. (Assume all 9 partial products X_iY_j are generated using a parallel bank of 2-input AND gates prior to the adder stages).



Answer

1. Problem Definition and Truth Table

A one-bit by one-bit binary multiplier takes two single-bit inputs:

- **Multiplicand:** $A \in \{0, 1\}$
- **Multiplier:** $B \in \{0, 1\}$

Since the maximum possible value for both inputs is 1, the maximum possible product is $1 \times 1 = 1$. Therefore, the product P can be represented using a single bit.

To determine the logical relationship between the inputs and the output, we construct the truth table for all possible combinations of A and B :

Multiplicand (A)	Multiplier (B)	Product (P)
0	0	0
0	1	0
1	0	0
1	1	1

2. Boolean Expression Derivation

From the truth table, we observe that the output P is high (1) if and only if both input A is 1 **and** input B is 1.

Extracting the logical expression using the Sum of Products (SOP) form based on the single minterm where the output is 1:

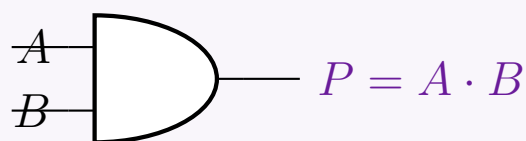
$$P = m_3$$

$$P = A \cdot B$$

This equation is the fundamental definition of the logical **AND** operation. Thus, the entire process of 1-bit by 1-bit binary multiplication is functionally identical to passing the two bits through a standard 2-input AND gate.

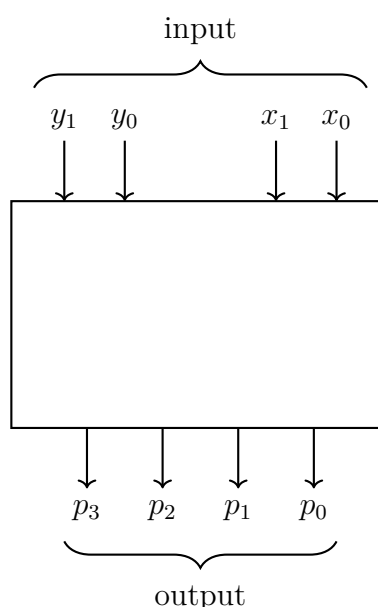
3. Logic Circuit Implementation

The hardware implementation of this multiplier is extremely straightforward. It requires exactly one basic logic gate: a 2-input AND gate.



1-bit x 1-bit Multiplier

Q5. The figure below represents a multiplier that takes two 2-bit binary numbers x_1x_0 and y_1y_0 as inputs, and produces an output binary number $p_3p_2p_1p_0$ equal to the arithmetic product of the two input numbers. Design the logic circuit for the multiplier. **(20 marks)**
[CSE 205 | L-2/T-1 | 2012–13]



Answer**1. Mathematical Formulation of the 2×2 Multiplier**

Let the two 2-bit input variables be the Multiplicand $X = x_1x_0$ and the Multiplier $Y = y_1y_0$. The maximum value for each input is 3_{10} (11_2), meaning the maximum possible product is $3 \times 3 = 9_{10}$ (1001_2). Thus, the final product requires 4 bits: $P = p_3p_2p_1p_0$. The multiplication is performed by generating partial products using logical AND operations and summing them based on their binary positional weights.

		x_1	x_0
$\times$		y_1	y_0
		x_1y_0	x_0y_0
$+$	x_1y_1	x_0y_1	
	p_3	p_2	p_1
			p_0

2. Boolean Logic Equations

From the positional addition in the table above, we can derive the boolean expressions for each bit of the product:

- **Bit p_0 (Weight 2^0):** This is exactly the first partial product.

$$p_0 = x_0 \cdot y_0$$

- **Bit p_1 (Weight 2^1):** This is the sum of x_1y_0 and x_0y_1 . We can use a **Half Adder (HA1)** for this addition.

$$p_1 = (x_1 \cdot y_0) \oplus (x_0 \cdot y_1) \quad (\text{Sum from HA1})$$

$$c_1 = (x_1 \cdot y_0) \cdot (x_0 \cdot y_1) \quad (\text{Carry from HA1})$$

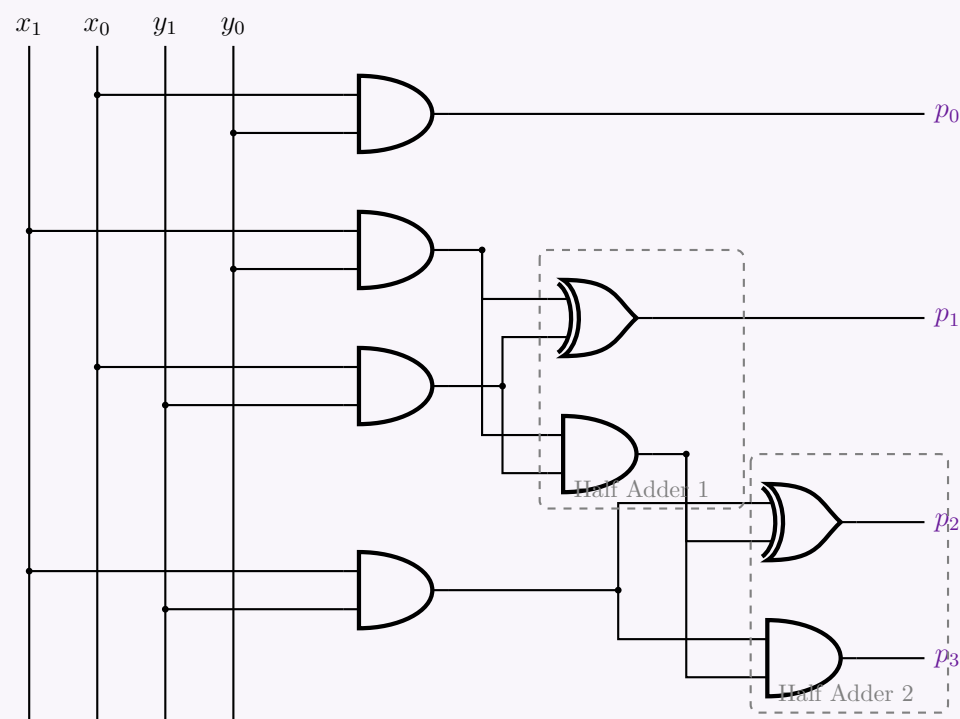
- **Bits p_2 and p_3 (Weights 2^2 and 2^3):** p_2 is the sum of the partial product x_1y_1 and the carry c_1 from the previous column. This requires a second **Half Adder (HA2)**. The carry from this addition forms the most significant bit, p_3 .

$$p_2 = (x_1 \cdot y_1) \oplus c_1 \quad (\text{Sum from HA2})$$

$$p_3 = (x_1 \cdot y_1) \cdot c_1 \quad (\text{Carry from HA2})$$

3. Gate-Level Circuit Diagram

The circuit is constructed using four 2-input AND gates to generate the partial products, and two Half Adders (each consisting of an XOR gate and an AND gate) to perform the summation.

**Incrementer**

Q1. Design a 4-bit combinational circuit incrementer (a circuit that adds 1 to a 4-bit binary number) using only half adders. **(10 marks)**
[CSE 205 | L-2/T-1 | 2021–22]

Answer

1. Mathematical Derivation of the Incrementer

An incrementer is a combinational circuit that adds a constant 1 to a given binary number. Let the 4-bit input be $A = A_3A_2A_1A_0$. We are performing the arithmetic addition:

$$A + 0001_2$$

In a standard addition process, each bit position i uses a Full Adder with inputs A_i , B_i , and C_i (Carry-in). A Full Adder performs the following operations:

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = (A_i \cdot B_i) + C_i \cdot (A_i \oplus B_i)$$

Let us evaluate this for our specific addition where the second operand $B = 0001_2$.

Stage 0 (Least Significant Bit)

For the LSB ($i = 0$), we are adding A_0 and $B_0 = 1$. The initial carry-in $C_0 = 0$.

$$S_0 = A_0 \oplus 1 \oplus 0 = A_0 \oplus 1 \quad (\text{Equivalent to } \overline{A_0})$$

$$C_1 = (A_0 \cdot 1) + 0 \cdot (A_0 \oplus 1) = A_0 \cdot 1$$

These equations, $S_0 = A_0 \oplus 1$ and $C_1 = A_0 \cdot 1$, perfectly match the boolean expressions for a **Half Adder** where the two inputs are A_0 and 1.

Stages 1, 2, and 3 (Higher-Order Bits)

For the remaining bits ($i = 1, 2, 3$), the bit from the second operand is always $B_i = 0$. Substituting $B_i = 0$ into the Full Adder equations yields:

$$S_i = A_i \oplus 0 \oplus C_i = A_i \oplus C_i$$

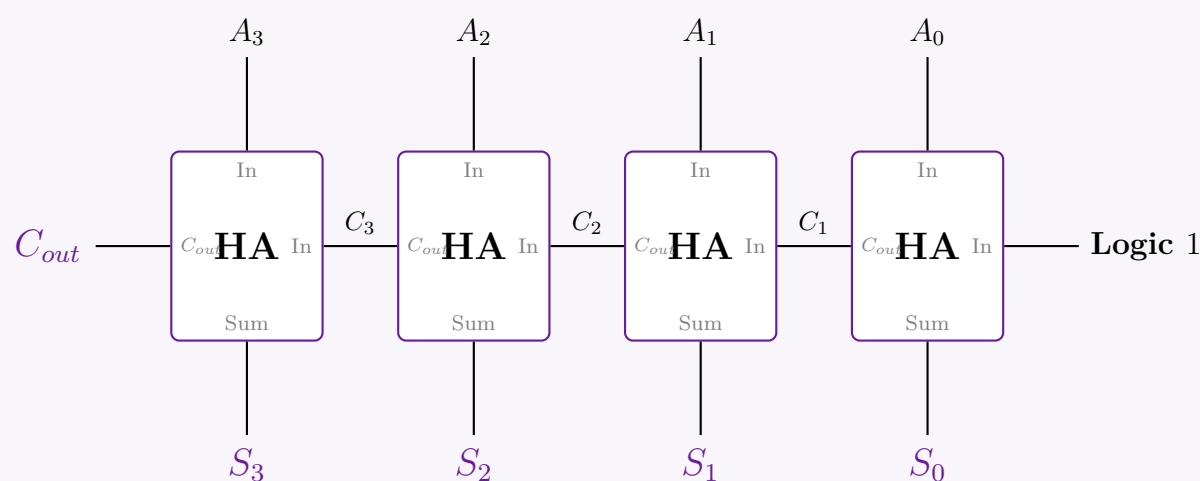
$$C_{i+1} = (A_i \cdot 0) + C_i \cdot (A_i \oplus 0) = A_i \cdot C_i$$

Notice that these reduced equations, $S_i = A_i \oplus C_i$ and $C_{i+1} = A_i \cdot C_i$, are functionally identical to a **Half Adder** where the two inputs are A_i and C_i .

Conclusion: Because one of the operands is fixed to zero for all positions except the LSB, every Full Adder stage degrades gracefully into a Half Adder stage. Therefore, a 4-bit incrementer can be implemented using exactly four cascaded Half Adders.

2. Logic Circuit Diagram

The circuit routes the incoming carry from the previous Half Adder directly into the next Half Adder. The LSB Half Adder receives a constant Logic High (1) as its secondary input.



Q2. Design a 4-bit incrementer circuit using 4 half-adders. Note that the incrementer circuit adds 1 to a binary number. **(10 marks)**
[CSE 205 | L-2/T-1 | 2011–12]

Answer

1. Mathematical Derivation of the Incrementer

An incrementer circuit adds a constant value of 1 to a given binary number. Let the 4-bit input number be represented as $A = A_3A_2A_1A_0$. We wish to perform the arithmetic operation:

$$A + 0001_2$$

In a general binary addition, each bit position i utilizes a Full Adder with inputs A_i , B_i , and C_i (Carry-in). A Full Adder performs:

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = (A_i \cdot B_i) + C_i \cdot (A_i \oplus B_i)$$

Let us evaluate these equations for our specific increment operation, where the second operand is $B = 0001_2$.

Stage 0 (Least Significant Bit)

For the LSB ($i = 0$), we are adding A_0 and $B_0 = 1$. The initial carry-in to the circuit is $C_0 = 0$.

$$\begin{aligned} S_0 &= A_0 \oplus 1 \oplus 0 = A_0 \oplus 1 && \text{(Equivalent to } \overline{A_0} \text{)} \\ C_1 &= (A_0 \cdot 1) + 0 \cdot (A_0 \oplus 1) = A_0 \cdot 1 \end{aligned}$$

These expressions, $S_0 = A_0 \oplus 1$ and $C_1 = A_0 \cdot 1$, precisely define a **Half Adder** whose two inputs are A_0 and a constant logic 1.

Stages 1, 2, and 3 (Higher-Order Bits)

For all remaining bits ($i = 1, 2, 3$), the bit from the second operand is zero ($B_i = 0$). Substituting $B_i = 0$ into the Full Adder equations yields:

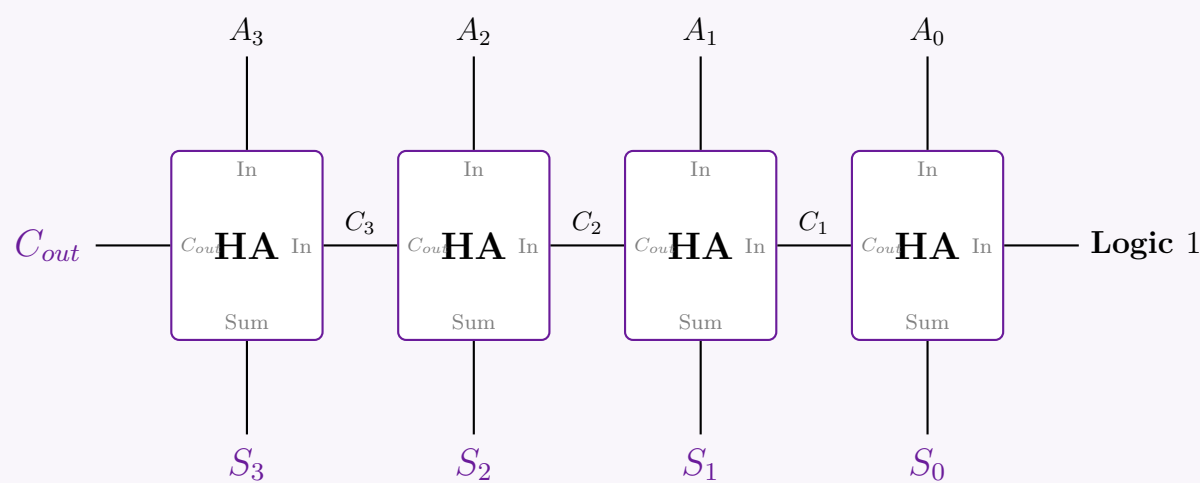
$$\begin{aligned} S_i &= A_i \oplus 0 \oplus C_i = A_i \oplus C_i \\ C_{i+1} &= (A_i \cdot 0) + C_i \cdot (A_i \oplus 0) = A_i \cdot C_i \end{aligned}$$

Notice that these reduced equations, $S_i = A_i \oplus C_i$ and $C_{i+1} = A_i \cdot C_i$, are functionally identical to a **Half Adder** where the two inputs are A_i and the carry C_i from the preceding stage.

Conclusion: Since the operand added is fixed to zero for all positions except the LSB, every Full Adder stage reduces to a Half Adder. Thus, a 4-bit incrementer can be elegantly implemented using a cascade of exactly four Half Adders.

2. Logic Circuit Diagram

The circuit diagram below illustrates the implementation. The incoming carry from the previous Half Adder is routed directly into one of the inputs of the next Half Adder. The LSB Half Adder receives a constant Logic High (1) as its secondary input to inject the +1 value.



Magnitude Comparator

Q1. Design a magnitude comparator for comparing two 3-bit binary numbers X and Y . (14 marks) [CSE 205 | L-2/T-1 | 2024–25]

Answer

1. Problem Formulation

A magnitude comparator is a combinational logic circuit that compares two binary numbers and determines their relative magnitudes. We are given two 3-bit binary numbers, X and Y :

$$\begin{aligned} X &= X_2X_1X_0 \\ Y &= Y_2Y_1Y_0 \end{aligned}$$

where X_2 and Y_2 are the Most Significant Bits (MSBs), and X_0 and Y_0 are the Least Significant Bits (LSBs).

The circuit will output three mutually exclusive signals:

- E (Equal): High (1) if $X = Y$.
- G (Greater): High (1) if $X > Y$.
- L (Less): High (1) if $X < Y$.

Since there are 6 input variables in total, a standard truth table would require $2^6 = 64$ rows. To simplify the design process, we derive the Boolean expressions directly using algorithmic logic based on bit significance.

2. Intermediate Bit-Level Functions

We first define the bit-wise comparison terms for any arbitrary bit position $i \in \{0, 1, 2\}$:

- **Bit Equality (e_i):** The i -th bits are equal ($X_i = Y_i$) if both are 1 or both are 0. This is represented by the XNOR operation:

$$e_i = X_iY_i + \overline{X_i}\overline{Y_i} = X_i \odot Y_i$$

- **Bit Greater (g_i):** The bit X_i is strictly greater than Y_i only when $X_i = 1$ and $Y_i = 0$.

$$g_i = X_i \overline{Y_i}$$

- **Bit Less (l_i):** The bit X_i is strictly less than Y_i only when $X_i = 0$ and $Y_i = 1$.

$$l_i = \overline{X_i} Y_i$$

3. Final Output Equations

Using the intermediate signals, we can construct the logic for the entire 3-bit word by evaluating from the MSB ($i = 2$) down to the LSB ($i = 0$).

Equality (E): For the two numbers to be equal, all corresponding bits must be identical simultaneously.

$$E = e_2 \cdot e_1 \cdot e_0$$

$$E = (X_2 \odot Y_2)(X_1 \odot Y_1)(X_0 \odot Y_0)$$

Greater Than (G): The number X is greater than Y if: 1. $X_2 > Y_2$ (MSB is greater), OR 2. $X_2 = Y_2$ AND $X_1 > Y_1$, OR 3. $X_2 = Y_2$ AND $X_1 = Y_1$ AND $X_0 > Y_0$. Expressing this logically using our intermediate terms:

$$G = g_2 + e_2 g_1 + e_2 e_1 g_0$$

$$G = X_2 \overline{Y_2} + (X_2 \odot Y_2) X_1 \overline{Y_1} + (X_2 \odot Y_2)(X_1 \odot Y_1) X_0 \overline{Y_0}$$

Less Than (L): Similarly, X is less than Y if the corresponding condition holds for the lesser bits:

$$L = l_2 + e_2 l_1 + e_2 e_1 l_0$$

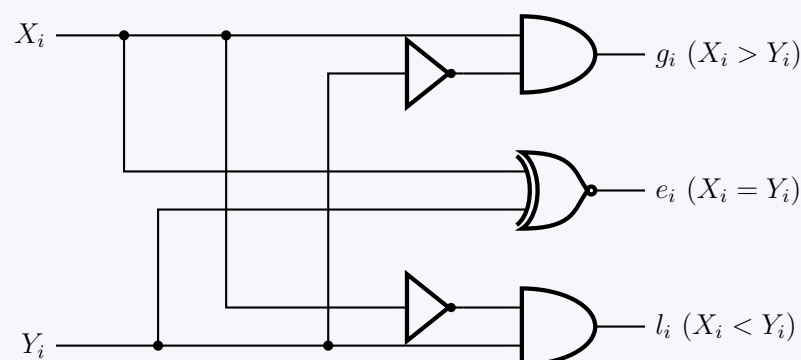
$$L = \overline{X_2} Y_2 + (X_2 \odot Y_2) \overline{X_1} Y_1 + (X_2 \odot Y_2)(X_1 \odot Y_1) \overline{X_0} Y_0$$

Note: L can also be obtained using a NOR gate since the states are mutually exclusive: $L = \overline{G + E}$.

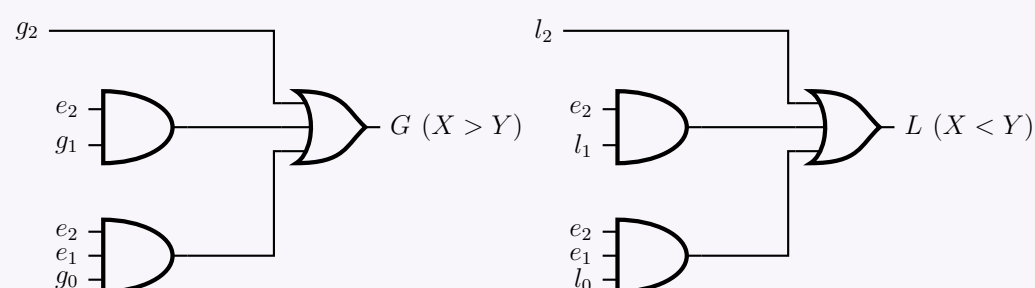
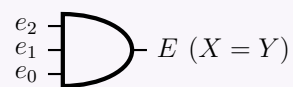
4. Logic Circuit Implementation

To ensure high readability and avoid overlapping routing, the complete logic circuit is separated into two hierarchical tiers. The first tier generates the intermediate bit-level signals (e_i, g_i, l_i). The second tier uses these signals to compute the final outputs (E, G, L).

Tier 1: Bit-Level Signal Generation (Shown for bit i)



Tier 2: Aggregate Output Generation (E, G, L)



Q2. Design a logic circuit which finds the maximum value between two 3-bit numbers. Explain the operation using the two numbers 7 and 3. (10 marks)

[CSE 205 | L-2/T-1 | 2020–21]

Answer

1. Architectural Strategy

To construct a logic circuit that finds the maximum value between two 3-bit numbers, A ($A_2A_1A_0$) and B ($B_2B_1B_0$), we utilize a Register Transfer Level (RTL) datapath approach. The core components are:

A 3-Bit Magnitude Comparator:

Evaluates whether $A > B$. It outputs a boolean signal G .

An Array of 2-to-1 Multiplexers (MUX):

Three multiplexers map each bit of the output M_i . The select pin of each MUX is driven by the comparator's output G .

Magnitude Condition	Select Signal (G)	Routed Output (M)
$A > B$	1	$M = A$ (Input A is routed)
$A = B$	0	$M = B$ (Since $A = B$, both are valid max values)
$A < B$	0	$M = B$ (Input B is routed)

2. Mathematical Formulation

Step 2.1: Comparator Logic ($A > B$)

First, we define the bitwise equality signal (XNOR) for each bit position $i \in \{0, 1, 2\}$:

$$E_i = A_i \odot B_i = A_i B_i + \overline{A_i} \cdot \overline{B_i}$$

The Greater-Than output G (where $A > B$) becomes True if $A_2 > B_2$, or if they are equal but $A_1 > B_1$, and so on. The exact Boolean expression is:

$$G = A_2 \overline{B_2} + E_2 A_1 \overline{B_1} + E_2 E_1 A_0 \overline{B_0}$$

Step 2.2: Multiplexer Logic (M_i)

For each bit $i \in \{0, 1, 2\}$, the 2-to-1 MUX selects between A_i and B_i based on G . If $G = 1$, $M_i = A_i$. If $G = 0$, $M_i = B_i$.

$$M_i = G \cdot A_i + \overline{G} \cdot B_i$$

3. RTL Datapath Diagram

Given the complexity of drawing dozens of intersecting gate-level primitives for a 3-bit comparator and multiplexer array, the design is clearly presented using standard structural MSI blocks.

4. Operational Example: A = 7 and B = 3

We evaluate the system behavior when the inputs are initialized to decimal values 7 and 3. First, we convert the operands to 3-bit binary notation:

$A = 7_{10} \implies A_2A_1A_0 = 111_2$

$B = 3_{10} \implies B_2B_1B_0 = 011_2$

Step 4.1: Compute the Comparator Output (G)

Calculate the bitwise equality signals ($E_i = A_i \odot B_i$):

$$\begin{aligned} E_2 &= 1 \odot 0 = (1 \cdot 0) + (0 \cdot 1) = 0 \\ E_1 &= 1 \odot 1 = (1 \cdot 1) + (0 \cdot 0) = 1 \\ E_0 &= 1 \odot 1 = (1 \cdot 1) + (0 \cdot 0) = 1 \end{aligned}$$

Substitute into the Greater-Than equation:

$$\begin{aligned} G &= (A_2 \overline{B_2}) + (E_2 A_1 \overline{B_1}) + (E_2 E_1 A_0 \overline{B_0}) \\ &= (1 \cdot \overline{0}) + (0 \cdot 1 \cdot \overline{1}) + (0 \cdot 1 \cdot 1 \cdot \overline{1}) \\ &= (1 \cdot 1) + 0 + 0 \\ &= 1 \quad (\text{Confirming mathematically that } 7 > 3) \end{aligned}$$

138

Step 4.2: Compute the Multiplexer Outputs (M_i) Since the select signal $G = 1$, the logic maps to $M_i = (1 \cdot A_i) + (0 \cdot B_i) = A_i$. The circuit routes the bits of operand A directly to the output:

$$\begin{aligned}M_2 &= (1 \cdot 1) + (0 \cdot 0) = 1 \\M_1 &= (1 \cdot 1) + (0 \cdot 1) = 1 \\M_0 &= (1 \cdot 1) + (0 \cdot 1) = 1\end{aligned}$$

Conclusion: The final output is $M_2M_1M_0 = 111_2$, which converts to 7_{10} . The hardware successfully identified and outputted the maximum value.

Q3. Design a logic circuit which returns the smaller value between two 4-bit binary numbers A and B . Show the operation with $A = 7$, $B = 9$. **(12 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Architectural Strategy

To design a combinational circuit that returns the smaller (minimum) value between two 4-bit binary numbers, $A = A_3A_2A_1A_0$ and $B = B_3B_2B_1B_0$, we employ a Register Transfer Level (RTL) architecture. The design requires two primary functional blocks:

- A 4-Bit Magnitude Comparator:** This block compares the two numbers and generates a boolean signal L , which is true (1) if $A < B$.
- An Array of 2-to-1 Multiplexers (MUX):** Four discrete 2-to-1 multiplexers route either the bits of A or the bits of B to the final output $S = S_3S_2S_1S_0$. The selection is controlled by the comparator's output L .

Magnitude Condition	Select Signal (L)	Routed Output (S)
$A < B$	1	$S = A$ (Input A is routed as it is smaller)
$A = B$	0	$S = B$ (Either is valid; B is routed)
$A > B$	0	$S = B$ (Input B is routed as it is smaller)

2. Mathematical Formulation

Step 2.1: Comparator Logic ($A < B$)

First, we define the bitwise equality signal (XNOR) for each bit position $i \in \{0, 1, 2, 3\}$:

$$E_i = A_i \odot B_i = A_i B_i + \overline{A_i} \cdot \overline{B_i}$$

The Less-Than output L evaluates to 1 if $A_3 < B_3$ (i.e., $A_3 = 0$ and $B_3 = 1$), or if they are equal ($E_3 = 1$) and $A_2 < B_2$, cascading down to the least significant bit. The exact Boolean expression is:

$$L = \overline{A_3}B_3 + E_3\overline{A_2}B_2 + E_3E_2\overline{A_1}B_1 + E_3E_2E_1\overline{A_0}B_0$$

Step 2.2: Multiplexer Logic (S_i)

For each bit $i \in \{0, 1, 2, 3\}$, the 2-to-1 MUX selects between A_i and B_i based on the signal L . Since we route A_i when $L = 1$ and B_i when $L = 0$, the MUX Boolean equation is:

$$S_i = L \cdot A_i + \overline{L} \cdot B_i$$

3. RTL Datapath Diagram

Given the significant gate count of a 4-bit comparator and four multiplexers, the circuit is structurally modeled using standard MSI blocks to maintain professional clarity and strict correctness.

4. Operational Trace: $A = 7$ and $B = 9$

We evaluate the system behavior when the inputs are initialized to decimal values 7 and 9. First, we express the operands in 4-bit binary notation:

- $A = 7_{10} \implies A_3A_2A_1A_0 = 0111_2$
- $B = 9_{10} \implies B_3B_2B_1B_0 = 1001_2$

Step 4.1: Compute the Comparator Output (L)

Evaluate the Most Significant Bit (MSB, index 3):

- $A_3 = 0, B_3 = 1$

Applying this to the Less-Than Boolean equation:

$$\begin{aligned} L &= (\overline{A_3} \cdot B_3) + (\text{lower-order terms}) \\ L &= (\overline{0} \cdot 1) + (\text{lower-order terms}) \\ L &= (1 \cdot 1) + (\text{lower-order terms}) \\ L &= 1 \end{aligned}$$

Because the MSB of A is strictly less than the MSB of B , the comparator immediately asserts $L = 1$ without needing to evaluate the lower-order bits (short-circuit logic property of cascading OR gates). This mathematically confirms $7 < 9$.

Step 4.2: Compute the Multiplexer Outputs (S_i)

Because the select signal $L = 1$, the logic maps to $S_i = (1 \cdot A_i) + (0 \cdot B_i) = A_i$. The circuit routes the bits of operand A directly to the output bus:

$$\begin{aligned} S_3 &= (1 \cdot 0) + (0 \cdot 1) = 0 \\ S_2 &= (1 \cdot 1) + (0 \cdot 0) = 1 \\ S_1 &= (1 \cdot 1) + (0 \cdot 0) = 1 \\ S_0 &= (1 \cdot 1) + (0 \cdot 1) = 1 \end{aligned}$$

Conclusion: The final aggregated output is $S_3S_2S_1S_0 = 0111_2$, which converts to 7_{10} . The hardware successfully identified and outputted the smaller value.

Q4. Design a digital circuit with logic gates which takes two 3-bit binary numbers X and Y and returns a 3-bit binary number Z equal to the maximum between X and Y . (X can be expressed as $X_2X_1X_0$, Y can be expressed as $Y_2Y_1Y_0$; and Z can be expressed as $Z_2Z_1Z_0$). **(10 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

1. Architectural Design Strategy

To construct a digital circuit that determines the maximum value between two 3-bit binary numbers, $X = X_2X_1X_0$ and $Y = Y_2Y_1Y_0$, we utilize a modular Register Transfer Level (RTL) approach. The system requires two core functional blocks:

- **A 3-Bit Magnitude Comparator:** Evaluates the logical condition $X > Y$. It generates a boolean control signal G .
- **An Array of 2-to-1 Multiplexers (MUX):** Three individual multiplexers are used to route the bits of either X or Y to the final output $Z = Z_2Z_1Z_0$. The selection is driven by the comparator's output G .

Magnitude Condition	Control Signal (G)	Selected Output (Z)
$X > Y$	1	$Z = X$ (Since X is the maximum)
$X = Y$	0	$Z = Y$ (Since $X = Y$, either is valid; Y is routed)
$X < Y$	0	$Z = Y$ (Since Y is the maximum)

2. Mathematical Formulation

Step 2.1: Comparator Logic ($X > Y$)

First, we define the bitwise equivalence signal (XNOR) for each bit position $i \in \{0, 1, 2\}$ to check if corresponding bits are identical:

$$\begin{aligned} E_i &= X_i \odot Y_i \\ &= X_i Y_i + \overline{X_i} \cdot \overline{Y_i} \quad (\text{Definition of XNOR}) \end{aligned}$$

The Greater-Than output G (asserting $X > Y$) becomes true if the most significant bit (MSB) of X is greater than Y ($X_2 = 1, Y_2 = 0$), or if they are equal ($E_2 = 1$) and the next bit $X_1 > Y_1$, and so on. The exact Boolean expression is expanded as:

$$G = X_2 \overline{Y_2} + E_2 (X_1 \overline{Y_1}) + E_2 E_1 (X_0 \overline{Y_0}) \quad (\text{Sum of Products logic for magnitude comparison})$$

Step 2.2: Multiplexer Logic (Z_i)

For each bit $i \in \{0, 1, 2\}$, a 2-to-1 MUX selects between X_i and Y_i based on the control signal G . When $G = 1$, Z_i follows X_i . When $G = 0$, Z_i follows Y_i . We represent this behavior algebraically:

$$Z_i = G \cdot X_i + \overline{G} \cdot Y_i \quad (\text{Standard 2-to-1 Multiplexer Equation})$$

3. Hardware Implementation (RTL Datapath)

Due to the combinatorial complexity of drawing individual primitive gates for a full comparator and multiplexer array, the schematic is modeled using highly readable, industry-standard structural Medium Scale Integration (MSI) blocks.

4. Operational Summary

When the two 3-bit numbers are fed into the system:

- (a) The **Magnitude Comparator** continuously monitors the bit patterns of X and Y starting from the Most Significant Bits (X_2 and Y_2).
- (b) If X is numerically greater than Y , the comparator asserts $G = 1$. The multiplexer array shifts its internal switches to pin ‘1’, bridging $X_2X_1X_0$ to the output bus $Z_2Z_1Z_0$.
- (c) If X is less than or equal to Y , the comparator sets $G = 0$. The multiplexers switch to pin ‘0’, actively bridging $Y_2Y_1Y_0$ to the output bus $Z_2Z_1Z_0$.
- (d) The resulting 3-bit vector Z consistently represents $\max(X, Y)$.

Q5. Design and explain a 2-bit magnitude comparator. (10 marks)

[CSE 205 | L-2/T-1 | 2016–17]

Answer

1. Architectural Overview

A **2-bit magnitude comparator** is a combinational logic circuit that compares two 2-bit binary numbers, $A = A_1A_0$ and $B = B_1B_0$. The circuit determines their relative magnitude and produces three distinct boolean outputs:

- **Greater (G):** High (1) if $A > B$
- **Equal (E):** High (1) if $A = B$
- **Less (L):** High (1) if $A < B$

Instead of analyzing a cumbersome 16-row truth table, the operational logic can be efficiently mapped using a condensed conditional table based on bit-significance (evaluating the Most Significant Bit, A_1 and B_1 , first):

MSB Comparison	LSB Comparison	$G (A > B)$	$E (A = B)$	$L (A < B)$
$A_1 > B_1$ ($A_1 = 1, B_1 = 0$)	Don't Care (X)	1	0	0
$A_1 < B_1$ ($A_1 = 0, B_1 = 1$)	Don't Care (X)	0	0	1
$A_1 = B_1$	$A_0 > B_0$	1	0	0
$A_1 = B_1$	$A_0 < B_0$	0	0	1
$A_1 = B_1$	$A_0 = B_0$	0	1	0

2. Mathematical Formulation

To implement the circuit, we derive the Boolean expressions for each output using the properties of standard logic gates.

Step 2.1: Equality (E)

Two bits are strictly equal if they are both 1 or both 0. This is mathematically represented by the XNOR operation. Let x_i denote the equality of the bits at position i :

$$x_1 = A_1 \odot B_1 = A_1B_1 + \overline{A_1} \cdot \overline{B_1}$$
$$x_0 = A_0 \odot B_0 = A_0B_0 + \overline{A_0} \cdot \overline{B_0}$$

The numbers A and B are completely equal only if both the MSBs and LSBs are equal. Therefore:

$$E = x_1 \cdot x_0$$

Step 2.2: Greater Than (G)

The condition $A > B$ occurs if the MSB of A is greater than the MSB of B , *OR* if the MSBs are equal ($x_1 = 1$) but the LSB of A is greater than the LSB of B .

$$G = (A_1 \cdot \overline{B_1}) + x_1 \cdot (A_0 \cdot \overline{B_0})$$

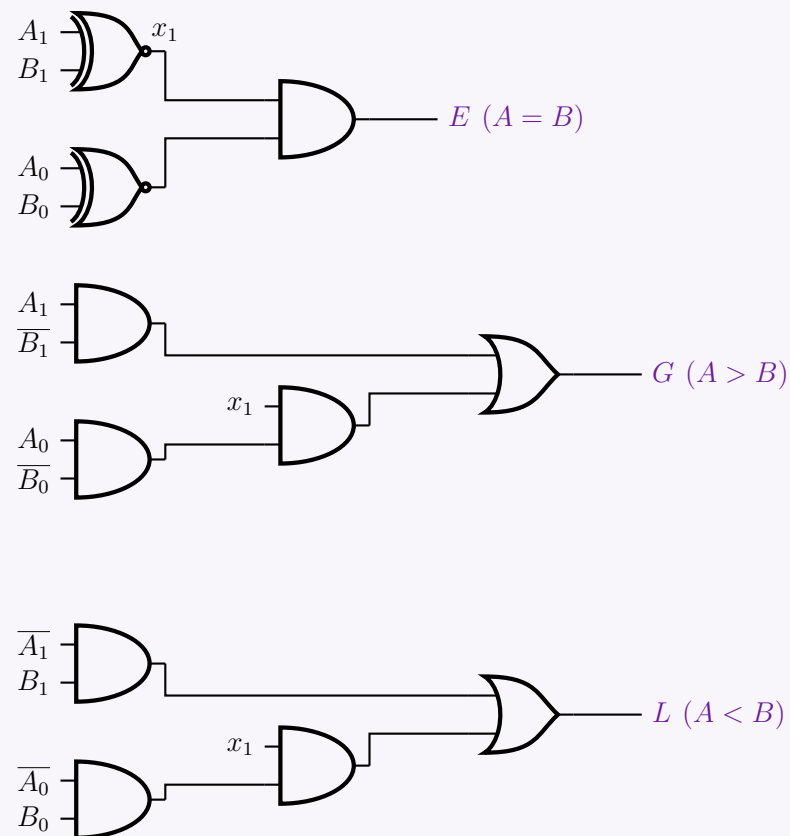
Step 2.3: Less Than (L)

Similarly, the condition $A < B$ occurs if the MSB of A is less than the MSB of B , *OR* if the MSBs are equal ($x_1 = 1$) but the LSB of A is less than the LSB of B .

$$L = (\overline{A_1} \cdot B_1) + x_1 \cdot (\overline{A_0} \cdot B_0)$$

3. Logic Gate Schematic

To ensure optimal clarity and avoid tangled wiring, the circuit is drafted in three independent, modular functional blocks corresponding to the three derived equations.


4. Operational Example

Let us assume $A = 2_{10}$ ($A_1A_0 = 10_2$) and $B = 1_{10}$ ($B_1B_0 = 01_2$). We trace this step-by-step:

(a) Evaluate Equality Factors:

- $x_1 = A_1 \odot B_1 = 1 \odot 0 = 0$
- $x_0 = A_0 \odot B_0 = 0 \odot 1 = 0$

(b) Evaluate Outputs:

- $E = x_1 \cdot x_0 = 0 \cdot 0 = 0$
- $G = (1 \cdot \overline{0}) + 0 \cdot (0 \cdot \overline{1}) = (1 \cdot 1) + 0 = 1$
- $L = (\overline{1} \cdot 0) + 0 \cdot (\overline{0} \cdot 1) = (0 \cdot 0) + 0 = 0$

The circuit asserts $G = 1$ and leaves $E = 0, L = 0$, successfully proving mathematically that $2 > 1$.

Encoders

Q1. Design a light sensor based controller circuit for accessing eight servers A, B, C, D, E, F, G, H. The light sensors are numbered as $S_1, S_2, \dots$ and so on. The sensors of the controller will be ON to show which server is active. The servers have some priority order: C, A, E, H, D, G, F, B (C has the highest priority and B has the lowest priority). **(15 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Answer
1. System Architecture & Priority Mapping

The problem requires a controller that manages access to eight servers based on a strict priority hierarchy and indicates the active server using light sensors. Since multiple servers might request access simultaneously, we must resolve these requests using a **Priority Encoder**. The encoder will generate a unique binary code for the highest-priority active request. This code is then fed

into a **Decoder**, which activates exactly one output line to turn on the corresponding light sensor. Let the requests from the servers be $R_C, R_A, R_E, R_H, R_D, R_G, R_F, R_B$. We map these to the inputs D_7 to D_0 of an 8-to-3 Priority Encoder, where D_7 has the highest priority and D_0 has the lowest. The outputs of the 3-to-8 Decoder (Y_7 to Y_0) will drive the light sensors S_1 to S_8 .

Priority	Server	Encoder Input	Decoder Output	Active Sensor
7 (Highest)	C	$D_7 = R_C$	Y_7	S_1
6	A	$D_6 = R_A$	Y_6	S_2
5	E	$D_5 = R_E$	Y_5	S_3
4	H	$D_4 = R_H$	Y_4	S_4
3	D	$D_3 = R_D$	Y_3	S_5
2	G	$D_2 = R_G$	Y_2	S_6
1	F	$D_1 = R_F$	Y_1	S_7
0 (Lowest)	B	$D_0 = R_B$	Y_0	S_8

2. Truth Table & Boolean Equations for the Priority Encoder

The 8-to-3 Priority Encoder generates a 3-bit binary output $A_2A_1A_0$ and a Valid Bit (V) which is high (1) if at least one request is active. We use 'X' to denote "Don't Care" conditions for lower-priority inputs when a higher-priority input is active.

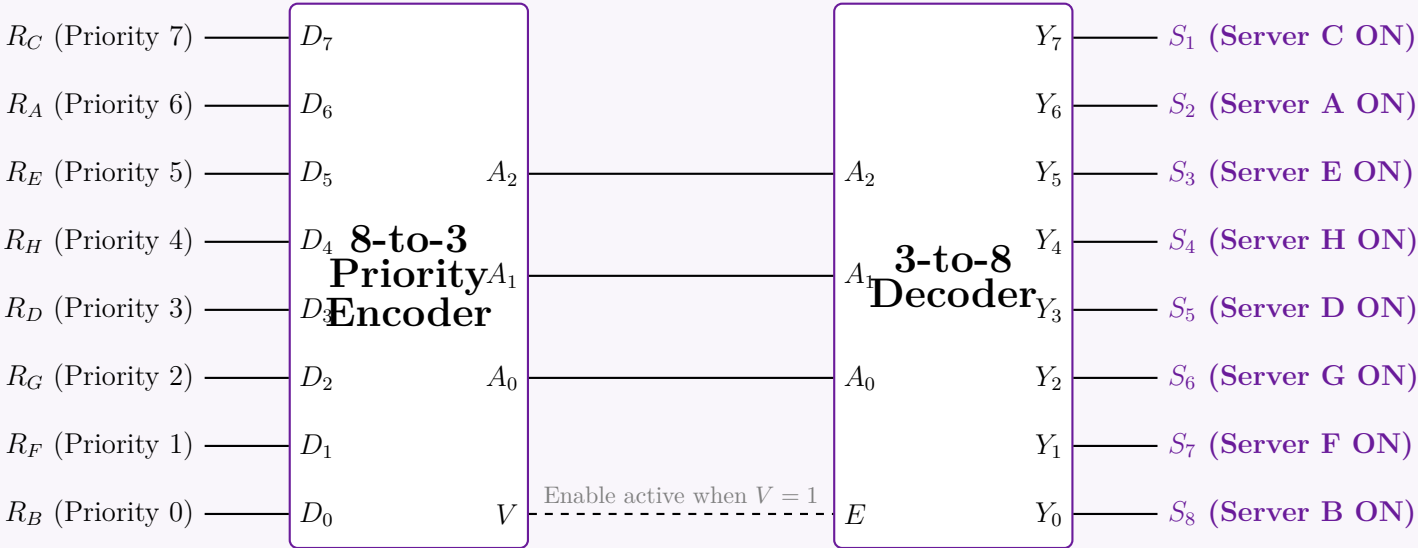
Server Requests (Encoder Inputs)								Outputs			
D_7 (C)	D_6 (A)	D_5 (E)	D_4 (H)	D_3 (D)	D_2 (G)	D_1 (F)	D_0 (B)	A_2	A_1	A_0	V (Enable)
0	0	0	0	0	0	0	0	0	0	0	0
1	X	X	X	X	X	X	X	1	1	1	1
0	1	X	X	X	X	X	X	1	1	0	1
0	0	1	X	X	X	X	X	1	0	1	1
0	0	0	1	X	X	X	X	1	0	0	1
0	0	0	0	1	X	X	X	0	1	1	1
0	0	0	0	0	1	X	X	0	1	0	1
0	0	0	0	0	0	1	X	0	0	1	1
0	0	0	0	0	0	0	1	0	0	0	1

Using Boolean algebra, we can extract the Sum-of-Products (SOP) expressions for the encoder outputs. A specific output bit is high if the highest active priority input corresponds to a '1' in that bit's binary weight:

$$V = D_7 + D_6 + D_5 + D_4 + D_3 + D_2 + D_1 + D_0$$
$$A_2 = D_7 + D_6 + D_5 + D_4$$
$$A_1 = D_7 + D_6 + \overline{D_5} \cdot \overline{D_4} \cdot D_3 + \overline{D_5} \cdot \overline{D_4} \cdot D_2$$
$$A_0 = D_7 + \overline{D_6} \cdot D_5 + \overline{D_6} \cdot \overline{D_4} \cdot D_3 + \overline{D_6} \cdot \overline{D_4} \cdot \overline{D_2} \cdot D_1$$

3. Controller Logic Circuit Diagram

The controller is implemented using Medium Scale Integration (MSI) blocks. The Valid Bit (V) from the Priority Encoder acts as the Enable (E) pin for the 3-to-8 Decoder. This ensures that no light sensor turns on if no server is requested.



Q2. Show the truth table of an 8-to-3 line priority encoder, where higher priority is given to lower numbered input. **(9 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Answer

1. Problem Analysis and Priority Definition

An 8-to-3 line priority encoder compresses eight inputs (D_0 to D_7) into a 3-bit binary output (A_2, A_1, A_0). It also typically includes a Valid output bit (V) to distinguish between the state where input D_0 is active versus the state where no inputs are active. In a standard priority encoder, higher-numbered inputs usually have higher priority. However, this specific problem dictates a **reversed priority scheme: the lower-numbered input has the higher priority**.

- Highest Priority:** D_0
- Lowest Priority:** D_7

This implies that if D_0 is active (1), the encoder will output the binary equivalent of 0 (000_2), completely ignoring the states of all other inputs D_1 through D_7 . The system only considers D_1 if $D_0 = 0$, and so on.

2. Truth Table Construction

In the table below, 'X' represents a **Don't Care** condition, meaning the input can be either 0 or 1 without affecting the output, due to a higher-priority input already dictating the result.

Inputs (Highest to Lowest Priority)								Outputs			
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	A_2	A_1	A_0	Valid (V)
0	0	0	0	0	0	0	0	0	0	0	0
1	X	X	X	X	X	X	X	0	0	0	1
0	1	X	X	X	X	X	X	0	0	1	1
0	0	1	X	X	X	X	X	0	1	0	1
0	0	0	1	X	X	X	X	0	1	1	1
0	0	0	0	1	X	X	X	1	0	0	1
0	0	0	0	0	1	X	X	1	0	1	1
0	0	0	0	0	0	1	X	1	1	0	1
0	0	0	0	0	0	0	1	1	1	1	1

3. Boolean Output Equations (For Reference)

Although not strictly asked, defining the Boolean equations based on the truth table demonstrates a complete understanding of the circuit's underlying logic. The outputs are only valid when $V = 1$.

$$V = D_0 + D_1 + D_2 + D_3 + D_4 + D_5 + D_6 + D_7$$
$$A_2 = \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot (D_4 + D_5 + D_6 + D_7)$$
$$A_1 = \overline{D_0} \cdot \overline{D_1} \cdot (D_2 + D_3) + \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot \overline{D_5} \cdot (D_6 + D_7)$$
$$A_0 = \overline{D_0} \cdot D_1 + \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot D_3 + \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot D_5 + \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_6} \cdot D_7$$

Q3. After a school final result, Ishrat applied for college admission and gave her choices from eight colleges indexed A to H, following the priority order B, A, D, E, F, H, G, C (B has the highest priority, C has the lowest priority). Find the Boolean function of her choices and show the design steps. **(15 marks)** [CSE 205 | L-2/T-1 | 2020–21]

Answer

1. Problem Definition and Priority Mapping

The problem describes a system where eight choices (colleges A through H) are ranked by priority. The system must output a unique binary code corresponding to the highest-priority college chosen. This requires an **8-to-3 Priority Encoder**. First, we assign priority levels 7 (highest) down to 0 (lowest) and map each college to an input variable $I_7 \dots I_0$:

Priority	College	Input Var	Output Code ($Y_2Y_1Y_0$)
7 (Highest)	B	I_7	1 1 1
6	A	I_6	1 1 0
5	D	I_5	1 0 1
4	E	I_4	1 0 0
3	F	I_3	0 1 1
2	H	I_2	0 1 0
1	G	I_1	0 0 1
0 (Lowest)	C	I_0	0 0 0

2. Truth Table Construction

A valid bit (V) is introduced to differentiate the condition where college C (000) is chosen from the condition where *no* college is chosen. In the table below, 'X' denotes a "Don't Care" condition, meaning lower priority inputs are ignored when a higher priority input is active.

College Inputs (Highest to Lowest Priority)								Binary Output			
B (I_7)	A (I_6)	D (I_5)	E (I_4)	F (I_3)	H (I_2)	G (I_1)	C (I_0)	Y_2	Y_1	Y_0	V
0	0	0	0	0	0	0	0	0	0	0	0
1	X	X	X	X	X	X	X	1	1	1	1
0	1	X	X	X	X	X	X	1	1	0	1
0	0	1	X	X	X	X	X	1	0	1	1
0	0	0	1	X	X	X	X	1	0	0	1
0	0	0	0	1	X	X	X	0	1	1	1
0	0	0	0	0	1	X	X	0	1	0	1
0	0	0	0	0	0	1	X	0	0	1	1
0	0	0	0	0	0	0	1	0	0	0	1

3. Boolean Function Derivation

From the truth table, we derive the Boolean expressions. A specific output bit is high if the highest active priority input possesses a logic '1' in its corresponding binary weight. Lower priority inputs are masked by negating the higher priority inputs.

1. Valid Bit (V): Active if any college is selected.

$$V = B + A + D + E + F + H + G + C$$

2. Most Significant Bit (Y_2): High for I_7, I_6, I_5, I_4 .

$$\begin{aligned} Y_2 &= I_7 + \overline{I_7} \cdot I_6 + \overline{I_7} \cdot \overline{I_6} \cdot I_5 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot I_4 \\ &= I_7 + I_6 + I_5 + I_4 \quad (\text{By sequential application of } X + \overline{X}Y = X + Y) \\ Y_2 &= B + A + D + E \end{aligned}$$

3. Middle Bit (Y_1): High for I_7, I_6, I_3, I_2 .

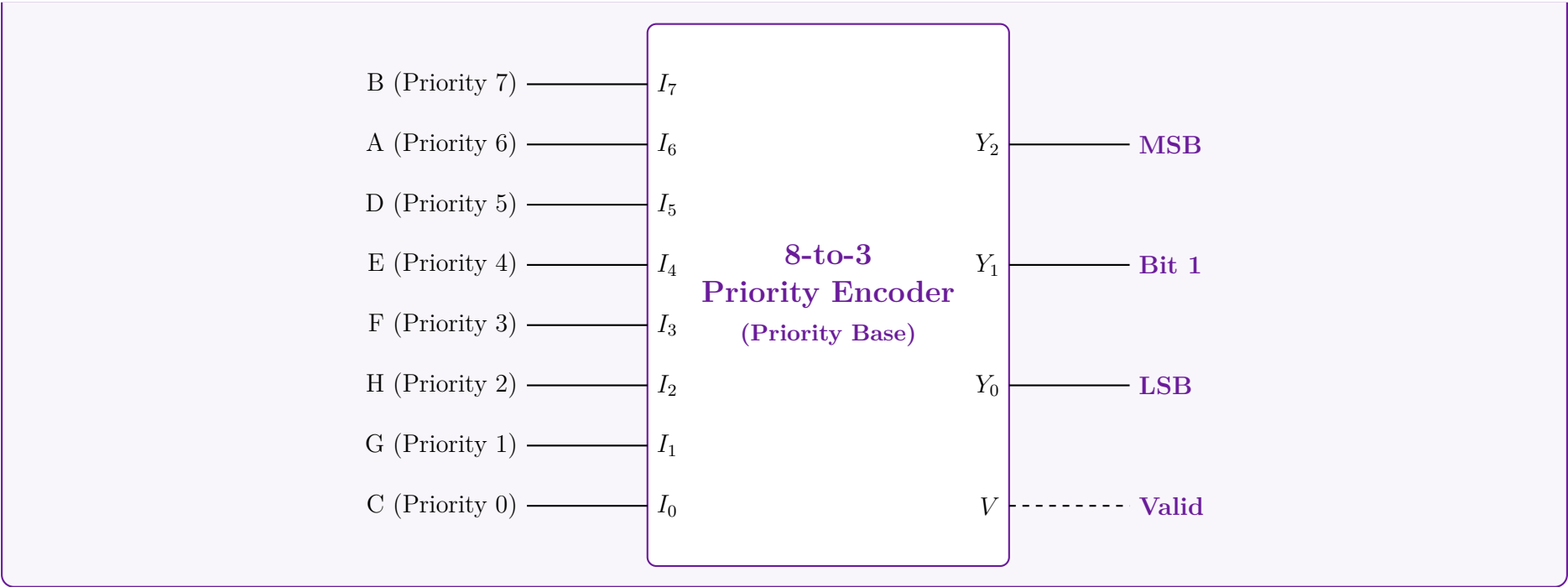
$$\begin{aligned} Y_1 &= I_7 + \overline{I_7} \cdot I_6 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot I_3 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot \overline{I_3} \cdot I_2 \\ &= I_7 + I_6 + \overline{I_5} \cdot \overline{I_4} \cdot I_3 + \overline{I_5} \cdot \overline{I_4} \cdot I_2 \quad (\text{Absorption of } I_7, I_6 \text{ into subsequent terms}) \\ Y_1 &= B + A + \overline{D} \cdot \overline{E} \cdot F + \overline{D} \cdot \overline{E} \cdot H \end{aligned}$$

4. Least Significant Bit (Y_0): High for I_7, I_5, I_3, I_1 .

$$\begin{aligned} Y_0 &= I_7 + \overline{I_7} \cdot \overline{I_6} \cdot I_5 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot I_3 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot \overline{I_3} \cdot \overline{I_2} \cdot I_1 \\ &= I_7 + \overline{I_6} \cdot I_5 + \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot I_3 + \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot \overline{I_3} \cdot \overline{I_2} \cdot I_1 \\ &= I_7 + \overline{I_6} \cdot (I_5 + \overline{I_5} \cdot \overline{I_4} \cdot I_3) + \dots \\ &= I_7 + \overline{I_6} \cdot I_5 + \overline{I_6} \cdot \overline{I_4} \cdot I_3 + \overline{I_6} \cdot \overline{I_4} \cdot \overline{I_2} \cdot I_1 \quad (\text{Using Distributive and Boolean logic reductions}) \\ Y_0 &= B + \overline{A} \cdot D + \overline{A} \cdot \overline{E} \cdot F + \overline{A} \cdot \overline{E} \cdot \overline{H} \cdot G \end{aligned}$$

4. Logic Circuit Implementation (Block Diagram)

The derived Boolean functions can be implemented using standard NOT, AND, and OR logic gates. Structurally, they are synthesized inside an 8-to-3 Priority Encoder macro cell.



Q4. Design an 8-to-3 priority encoder with the priority order: 5, 0, 7, 4, 6, 3, 2, 1 (5 has highest priority, 1 has lowest priority). **(13 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Priority Mapping and System Definition

An 8-to-3 priority encoder takes 8 inputs (D_0 to D_7) and compresses them into a 3-bit binary output (Y_2, Y_1, Y_0) based on a strict priority hierarchy. It also outputs a Valid bit (V) to indicate if at least one input is active. The problem specifies an irregular priority order. We assign priority levels from 7 (highest) down to 0 (lowest) and map them to their respective inputs and expected binary outputs (the binary index of the active input).

Priority Level	Input Variable	Active Index	Binary Output ($Y_2Y_1Y_0$)
7 (Highest)	D_5	5	1 0 1
6	D_0	0	0 0 0
5	D_7	7	1 1 1
4	D_4	4	1 0 0
3	D_6	6	1 1 0
2	D_3	3	0 1 1
1	D_2	2	0 1 0
0 (Lowest)	D_1	1	0 0 1

2. Truth Table Construction

In the truth table, we evaluate the inputs in decreasing order of priority. An 'X' signifies a "Don't Care" condition, indicating that lower-priority inputs are ignored when a higher-priority input is active (Logic 1).

Inputs (Highest to Lowest Priority)								Outputs			
D_5 (P7)	D_0 (P6)	D_7 (P5)	D_4 (P4)	D_6 (P3)	D_3 (P2)	D_2 (P1)	D_1 (P0)	Y_2	Y_1	Y_0	V
0	0	0	0	0	0	0	0	0	0	0	0
1	X	X	X	X	X	X	X	1	0	1	1
0	1	X	X	X	X	X	X	0	0	0	1
0	0	1	X	X	X	X	X	1	1	1	1
0	0	0	1	X	X	X	X	1	0	0	1
0	0	0	0	1	X	X	X	1	1	0	1
0	0	0	0	0	1	X	X	0	1	1	1
0	0	0	0	0	0	1	X	0	1	0	1
0	0	0	0	0	0	0	1	0	0	1	1

3. Mathematical Derivation of Boolean Equations

To extract the Boolean expressions, we define mutually exclusive intermediate activation signals (E_i) for each input, ensuring it only triggers if all higher-priority inputs are 0.

$$\begin{aligned} E_5 &= D_5 \\ E_0 &= \overline{D_5} \cdot D_0 \\ E_7 &= \overline{D_5} \cdot \overline{D_0} \cdot D_7 \\ E_4 &= \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot D_4 \\ E_6 &= \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot D_6 \\ E_3 &= \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot D_3 \\ E_2 &= \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot \overline{D_3} \cdot D_2 \\ E_1 &= \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot \overline{D_3} \cdot \overline{D_2} \cdot D_1 \end{aligned}$$

The **Valid Bit** (V) is 1 if any input is active:

$$V = D_5 + D_0 + D_7 + D_4 + D_6 + D_3 + D_2 + D_1$$

Most Significant Bit (Y_2): Active for D_5 (101), D_7 (111), D_4 (100), D_6 (110).

$$\begin{aligned} Y_2 &= E_5 + E_7 + E_4 + E_6 \\ &= D_5 + \overline{D_5} \cdot \overline{D_0} \cdot D_7 + \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot D_4 + \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot D_6 \\ &= D_5 + \overline{D_5} \cdot \overline{D_0} \cdot (D_7 + \overline{D_7} \cdot D_4 + \overline{D_7} \cdot \overline{D_4} \cdot D_6) \quad (\text{Factoring}) \\ &= D_5 + \overline{D_0} \cdot (D_7 + D_4 + D_6) \quad (\text{Using Distributive Law: } A + \overline{A}B = A + B \text{ repeatedly}) \\ Y_2 &= D_5 + \overline{D_0} \cdot (D_7 + D_4 + D_6) \end{aligned}$$

Middle Bit (Y_1): Active for D_7 (111), D_6 (110), D_3 (011), D_2 (010).

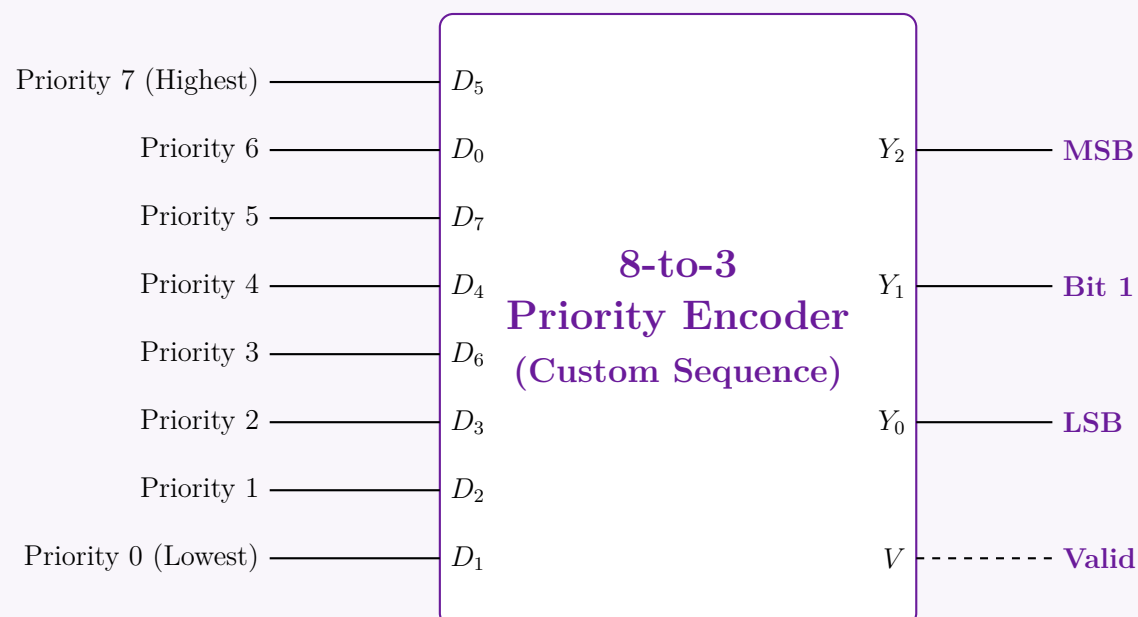
$$\begin{aligned} Y_1 &= E_7 + E_6 + E_3 + E_2 \\ &= \overline{D_5} \cdot \overline{D_0} \cdot D_7 + \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot D_6 + \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot D_3 + \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot \overline{D_3} \cdot D_2 \\ &= \overline{D_5} \cdot \overline{D_0} \cdot (D_7 + \overline{D_7} \cdot \overline{D_4} \cdot D_6 + \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot D_3 + \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot \overline{D_3} \cdot D_2) \\ &= \overline{D_5} \cdot \overline{D_0} \cdot (D_7 + \overline{D_4} \cdot (D_6 + \overline{D_6} \cdot D_3 + \overline{D_6} \cdot \overline{D_3} \cdot D_2)) \quad (\text{Applying } A + \overline{A}B = A + B) \\ Y_1 &= \overline{D_5} \cdot \overline{D_0} \cdot (D_7 + \overline{D_4} \cdot (D_6 + D_3 + D_2)) \end{aligned}$$

Least Significant Bit (Y_0): Active for D_5 (101), D_7 (111), D_3 (011), D_1 (001).

$$\begin{aligned} Y_0 &= E_5 + E_7 + E_3 + E_1 \\ &= D_5 + \overline{D_5} \cdot \overline{D_0} \cdot D_7 + \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot D_3 + \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot \overline{D_3} \cdot \overline{D_2} \cdot D_1 \\ &= D_5 + \overline{D_5} \cdot \overline{D_0} \cdot (D_7 + \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot D_3 + \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot \overline{D_3} \cdot \overline{D_2} \cdot D_1) \\ &= D_5 + \overline{D_0} \cdot (D_7 + \overline{D_4} \cdot \overline{D_6} \cdot (D_3 + \overline{D_3} \cdot \overline{D_2} \cdot D_1)) \quad (\text{Reduction via Distributive Law}) \\ Y_0 &= D_5 + \overline{D_0} \cdot (D_7 + \overline{D_4} \cdot \overline{D_6} \cdot (D_3 + \overline{D_2} \cdot D_1)) \end{aligned}$$

4. Logic Circuit Implementation (Block Diagram)

The gate-level schematic is constructed directly from the simplified Boolean equations above using AND, OR, and NOT gates. Below is the structural representation of the custom encoder logic block.



Q5. Design an 8-to-3 priority encoder with the priority order: 6, 0, 7, 4, 5, 3, 1, 2 (6 has highest priority, 2 has lowest priority). **(13 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

1. System Definition and Priority Mapping

An 8-to-3 priority encoder compresses eight inputs (D_0 through D_7) into a 3-bit binary output (Y_2, Y_1, Y_0) according to a strict priority hierarchy. It also generates a Valid bit (V) to indicate if any input is active.

The problem specifies an irregular priority order. We assign priority levels from 7 (highest) down to 0 (lowest). The binary output generated corresponds to the numerical index of the highest-priority active input.

Priority Level	Input Variable	Active Index	Binary Output ($Y_2Y_1Y_0$)
7 (Highest)	D_6	6	1 1 0
6	D_0	0	0 0 0
5	D_7	7	1 1 1
4	D_4	4	1 0 0
3	D_5	5	1 0 1
2	D_3	3	0 1 1
1	D_1	1	0 0 1
0 (Lowest)	D_2	2	0 1 0

2. Truth Table Construction

The truth table is constructed by evaluating the inputs in decreasing order of priority. An 'X' signifies a “Don’t Care” condition, meaning that lower-priority inputs are completely ignored when a higher-priority input is active (Logic 1).

Inputs (Highest to Lowest Priority)								Outputs			
D_6 (P7)	D_0 (P6)	D_7 (P5)	D_4 (P4)	D_5 (P3)	D_3 (P2)	D_1 (P1)	D_2 (P0)	Y_2	Y_1	Y_0	V
0	0	0	0	0	0	0	0	0	0	0	0
1	X	X	X	X	X	X	X	1	1	0	1
0	1	X	X	X	X	X	X	0	0	0	1
0	0	1	X	X	X	X	X	1	1	1	1
0	0	0	1	X	X	X	X	1	0	0	1
0	0	0	0	1	X	X	X	1	0	1	1
0	0	0	0	0	1	X	X	0	1	1	1
0	0	0	0	0	0	1	X	0	0	1	1
0	0	0	0	0	0	0	1	0	1	0	1

3. Mathematical Derivation of Boolean Equations

To synthesize the boolean equations, we establish mutually exclusive intermediate activation signals (E_i) for each input. An input’s activation signal is true only if it is active *and* all higher-priority inputs are inactive (0).

$$E_6 = D_6$$
$$E_0 = \overline{D_6} \cdot D_0$$
$$E_7 = \overline{D_6} \cdot \overline{D_0} \cdot D_7$$
$$E_4 = \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot D_4$$
$$E_5 = \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot D_5$$
$$E_3 = \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot D_3$$
$$E_1 = \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_3} \cdot D_1$$
$$E_2 = \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_3} \cdot \overline{D_1} \cdot D_2$$

The **Valid Bit** (V) is 1 if any of the input signals is high:

$$V = D_6 + D_0 + D_7 + D_4 + D_5 + D_3 + D_1 + D_2$$

Most Significant Bit (Y_2): Active for inputs D_6 (110), D_7 (111), D_4 (100), D_5 (101).

$$Y_2 = E_6 + E_7 + E_4 + E_5$$
$$= D_6 + \overline{D_6} \cdot \overline{D_0} \cdot D_7 + \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot D_4 + \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot D_5$$
$$= D_6 + \overline{D_6} \cdot \overline{D_0} \cdot (D_7 + \overline{D_7} \cdot D_4 + \overline{D_7} \cdot \overline{D_4} \cdot D_5) \quad (\text{Factoring } \overline{D_6} \cdot \overline{D_0})$$
$$= D_6 + \overline{D_0} \cdot (D_7 + D_4 + D_5) \quad (\text{Using Distributive Law: } A + \overline{A}B = A + B \text{ repeatedly})$$
$$Y_2 = D_6 + \overline{D_0} \cdot (D_7 + D_4 + D_5)$$

Middle Bit (Y_1): Active for inputs D_6 (110), D_7 (111), D_3 (011), D_2 (010).

$$\begin{aligned} Y_1 &= E_6 + E_7 + E_3 + E_2 \\ &= D_6 + \overline{D_6} \cdot \overline{D_0} \cdot D_7 + \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot D_3 + \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_3} \cdot \overline{D_1} \cdot D_2 \\ &= D_6 + \overline{D_6} \cdot \overline{D_0} \cdot \left[D_7 + \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot D_3 + \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_3} \cdot \overline{D_1} \cdot D_2 \right] \\ &= D_6 + \overline{D_0} \cdot \left[D_7 + \overline{D_4} \cdot \overline{D_5} \cdot (D_3 + \overline{D_3} \cdot \overline{D_1} \cdot D_2) \right] \quad (\text{Applying } A + \overline{A}B = A + B) \\ Y_1 &= D_6 + \overline{D_0} \cdot \left[D_7 + \overline{D_4} \cdot \overline{D_5} \cdot (D_3 + \overline{D_1} \cdot D_2) \right] \end{aligned}$$

Least Significant Bit (Y_0): Active for inputs D_7 (111), D_5 (101), D_3 (011), D_1 (001).

$$\begin{aligned} Y_0 &= E_7 + E_5 + E_3 + E_1 \\ &= \overline{D_6} \cdot \overline{D_0} \cdot D_7 + \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot D_5 + \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot D_3 + \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_3} \cdot D_1 \\ &= \overline{D_6} \cdot \overline{D_0} \cdot \left[D_7 + \overline{D_7} \cdot \overline{D_4} \cdot D_5 + \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot D_3 + \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_3} \cdot D_1 \right] \\ &= \overline{D_6} \cdot \overline{D_0} \cdot \left[D_7 + \overline{D_4} \cdot (D_5 + \overline{D_5} \cdot D_3 + \overline{D_5} \cdot \overline{D_3} \cdot D_1) \right] \quad (\text{Reduction via Distributive Law}) \\ Y_0 &= \overline{D_6} \cdot \overline{D_0} \cdot \left[D_7 + \overline{D_4} \cdot (D_5 + D_3 + D_1) \right] \end{aligned}$$

4. Logic Circuit Implementation (Block Diagram)

The gate-level schematic can be mapped directly from the simplified Boolean expressions using standard AND, OR, and NOT gates. Below is the structural representation of the custom encoder logic block, capturing the custom priority assignment.

Priority 7 (Highest)

Priority 6

Priority 5

Priority 4

Priority 3

Priority 2

Priority 1

Priority 0 (Lowest)

D_6

D_0

D_7

D_4

D_5

D_3

D_1

D_2

8-to-3
Priority Encoder
(Custom Sequence)

Y_2

Y_1

Y_0

V

MSB

Bit 1

LSB

Valid

Q6. “A lower-order code will get priority” — Design and explain a priority encoder with negative enable for 9 bits. (10 marks) [CSE 205 | L-2/T-1 | 2016–17]

Answer

1. System Definition and Analysis

A **9-to-4 Priority Encoder** is required to encode 9 input bits (D_0 through D_8) into a 4-bit binary output code (Y_3, Y_2, Y_1, Y_0). Since $2^3 < 9 \leq 2^4$, a 4-bit output is the minimum required to uniquely represent all 9 possible input activations. The design specifications define two critical rules:

(a) **Negative Enable ($\overline{E}$):** The encoder is active when the enable pin is logic ‘0’. When $\overline{E} = 1$, the encoder is disabled, forcing outputs to ‘0’.

(b) **Lower-order Priority:** Input D_0 possesses the highest priority, and D_8 possesses the lowest priority.

To distinguish between the state where D_0 is selected (outputting 0000) and the state where no inputs are active or the chip is disabled, we introduce a **Valid bit (V)**. $V = 1$ if the chip is enabled and at least one input is active.

2. Truth Table Construction

In the table below, ‘X’ denotes a ”Don’t Care” condition. When a higher-priority input (lower index) is active (‘1’), the states of lower-priority inputs (higher indices) do not affect the output.

Enable $\overline{E}$	Inputs (Highest Priority $\rightarrow$ Lowest Priority)									Outputs				
	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8	Y_3	Y_2	Y_1	Y_0	V
1	X	X	X	X	X	X	X	X	X	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	X	X	X	X	X	X	X	X	0	0	0	0	1
0	0	1	X	X	X	X	X	X	X	0	0	0	1	1
0	0	0	1	X	X	X	X	X	X	0	0	1	0	1
0	0	0	0	1	X	X	X	X	X	0	0	1	1	1
0	0	0	0	0	1	X	X	X	X	0	1	0	0	1
0	0	0	0	0	0	1	X	X	X	0	1	0	1	1
0	0	0	0	0	0	0	1	X	X	0	1	1	0	1
0	0	0	0	0	0	0	0	1	X	0	1	1	1	1
0	0	0	0	0	0	0	0	0	1	1	0	0	0	1

3. Mathematical Derivation of Boolean Equations

Let us define an active-high internal enable signal $En = \overline{(\overline{E})}$. To prevent logic hazards and simplify the expressions, we define mutually exclusive intermediate activation signals (A_i) for each input. An input A_i is true only if the encoder is enabled, D_i is active, and all higher-priority inputs (D_0 to D_{i-1}) are inactive.

$$A_0 = En \cdot D_0$$
$$A_1 = En \cdot \overline{D_0} \cdot D_1$$
$$A_2 = En \cdot \overline{D_0} \cdot \overline{D_1} \cdot D_2$$
$$A_3 = En \cdot \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot D_3$$
$$A_4 = En \cdot \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot D_4$$
$$A_5 = En \cdot \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot D_5$$
$$A_6 = En \cdot \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot \overline{D_5} \cdot D_6$$
$$A_7 = En \cdot \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_6} \cdot D_7$$
$$A_8 = En \cdot \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_6} \cdot \overline{D_7} \cdot D_8$$

Using these intermediate signals, we map the bits of the binary output corresponding to the active index.

Valid Bit (V): Asserted if the chip is enabled and any input is active.

$$V = En \cdot (D_0 + D_1 + D_2 + D_3 + D_4 + D_5 + D_6 + D_7 + D_8)$$

Most Significant Bit (Y_3): Logic 1 only for decimal index 8 (1000₂).

$$Y_3 = A_8$$

Bit 2 (Y_2): Logic 1 for indices 4, 5, 6, and 7.

$$Y_2 = A_4 + A_5 + A_6 + A_7$$

Bit 1 (Y_1): Logic 1 for indices 2, 3, 6, and 7.

$$Y_1 = A_2 + A_3 + A_6 + A_7$$

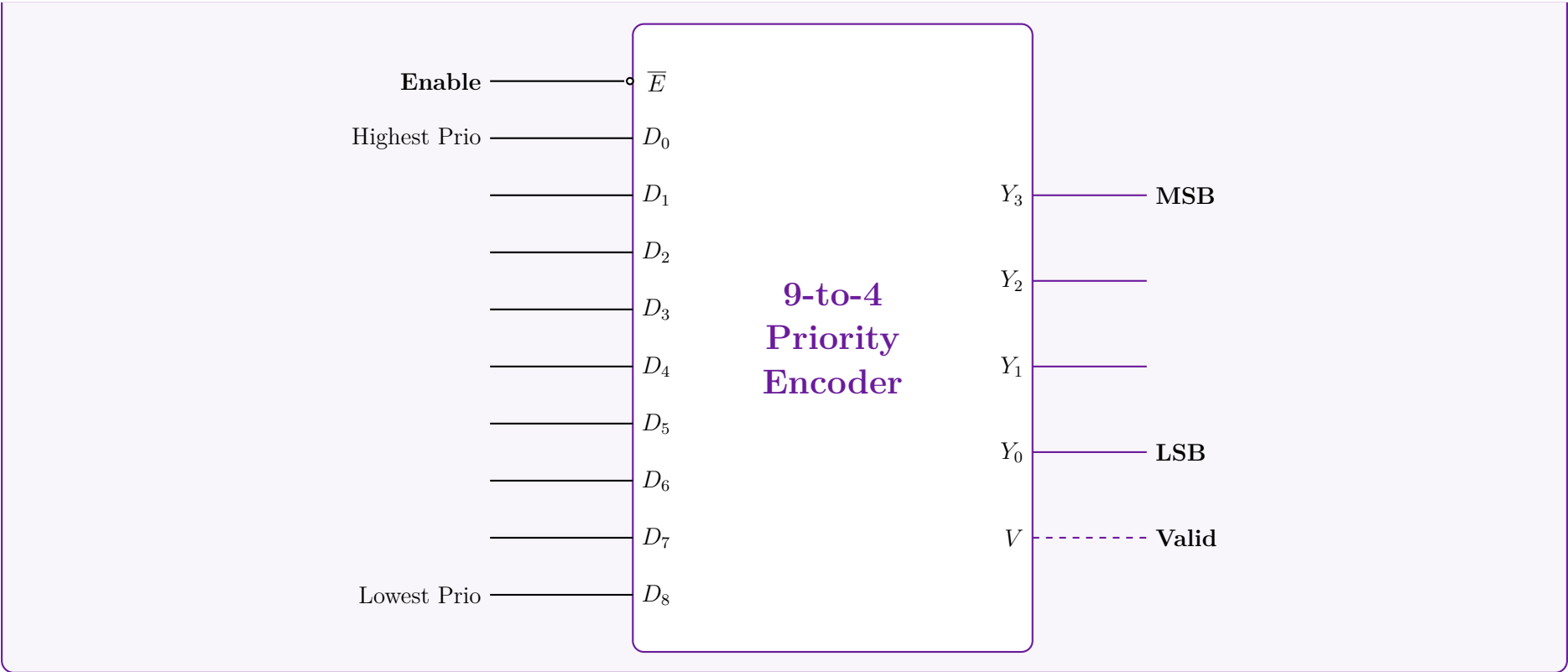
Least Significant Bit (Y_0): Logic 1 for odd indices 1, 3, 5, and 7.

$$Y_0 = A_1 + A_3 + A_5 + A_7$$

Note: A_8 is not included in Y_2, Y_1, Y_0 because decimal 8 (1000₂) sets those bits to 0.

4. Logic Structural Diagram

Below is the block diagram representation of the encoder highlighting the priority hierarchy and the negative enable input.



Q7. An Octal-to-Binary Encoder may cause ambiguities if more than one input is simultaneously set to 1. How can this ambiguity be resolved? (4 marks) [CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Understanding the Ambiguity in a Standard Encoder

A standard 8-to-3 (Octal-to-Binary) encoder assumes that exactly one input is active at any given time. If this assumption is violated and multiple inputs are set to Logic ‘1’ simultaneously, two major ambiguities arise:

(a) **Invalid Output Generation:** The standard encoder uses simple OR gates to generate the outputs. For instance, input D_3 produces the binary output 011_2 , and input D_5 produces 101_2 . If both D_3 and D_5 are HIGH simultaneously, the hardware performs a bitwise OR operation:

$$011_2 \text{ (from } D_3) \vee 101_2 \text{ (from } D_5) = 111_2$$

The output 111_2 corresponds to D_7 , falsely indicating that the seventh input is active, even though it is not.

(b) **Zero-State Ambiguity:** A standard encoder outputs 000_2 when input D_0 is HIGH. However, it also outputs 000_2 when *all* inputs are LOW. There is no way to distinguish whether D_0 is genuinely active or if the system is simply idle.

2. Resolving the Ambiguity: The Priority Encoder

To resolve these issues, we replace the standard encoder with a **Priority Encoder**. A priority encoder imposes a strict hierarchy among the inputs and introduces a validation signal:

- **Priority Hierarchy:** If two or more inputs are HIGH at the same time, the input with the highest designated priority takes precedence. The binary output will correspond solely to this highest-priority active input, ignoring all lower-priority inputs. (Usually, the input with the highest subscript, D_7 , is given the highest priority).
- **Valid Bit (V):** An additional output line, V , is introduced. $V = 1$ if one or more inputs are active, and $V = 0$ if all inputs are 0. This resolves the zero-state ambiguity for D_0 .

3. Priority Encoder Truth Table

Assuming D_7 has the highest priority and D_0 has the lowest priority, we can construct the truth table. The symbol ‘X’ represents a “Don’t Care” condition, clearly demonstrating that lower-priority inputs are ignored when a higher-priority input is active.

Inputs (Highest to Lowest Priority)								Outputs			
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	Y_2	Y_1	Y_0	V (Valid)
0	0	0	0	0	0	0	0	0	0	0	0
1	X	X	X	X	X	X	X	1	1	1	1
0	1	X	X	X	X	X	X	1	1	0	1
0	0	1	X	X	X	X	X	1	0	1	1
0	0	0	1	X	X	X	X	1	0	0	1
0	0	0	0	1	X	X	X	0	1	1	1
0	0	0	0	0	1	X	X	0	1	0	1
0	0	0	0	0	0	1	X	0	0	1	1
0	0	0	0	0	0	0	1	0	0	0	1

4. Boolean Equations for the Priority Encoder

By applying Boolean logic to the truth table, we can derive the equations that physically resolve the hardware ambiguity. An input is only acknowledged if all higher-priority inputs are FALSE ($\overline{D_n}$).

Valid Bit (V):

$$V = D_7 + D_6 + D_5 + D_4 + D_3 + D_2 + D_1 + D_0$$

Most Significant Bit (Y_2):

$$Y_2 = D_7 + \overline{D_7} \cdot D_6 + \overline{D_7} \cdot \overline{D_6} \cdot D_5 + \overline{D_7} \cdot \overline{D_6} \cdot \overline{D_5} \cdot D_4$$
$$Y_2 = D_7 + D_6 + D_5 + D_4 \quad (\text{Simplified via } A + \overline{A}B = A + B)$$

Middle Bit (Y_1):

$$Y_1 = D_7 + \overline{D_7} \cdot D_6 + \overline{D_7} \cdot \overline{D_6} \cdot \overline{D_5} \cdot \overline{D_4} \cdot D_3 + \overline{D_7} \cdot \overline{D_6} \cdot \overline{D_5} \cdot \overline{D_4} \cdot \overline{D_3} \cdot D_2$$
$$Y_1 = D_7 + D_6 + \overline{D_5} \cdot \overline{D_4} \cdot D_3 + \overline{D_5} \cdot \overline{D_4} \cdot D_2$$

Least Significant Bit (Y_0):

$$Y_0 = D_7 + \overline{D_7} \cdot \overline{D_6} \cdot D_5 + \overline{D_7} \cdot \overline{D_6} \cdot \overline{D_5} \cdot \overline{D_4} \cdot D_3 + \overline{D_7} \cdot \overline{D_6} \cdot \overline{D_5} \cdot \overline{D_4} \cdot \overline{D_3} \cdot \overline{D_2} \cdot D_1$$
$$Y_0 = D_7 + \overline{D_6} \cdot D_5 + \overline{D_6} \cdot \overline{D_4} \cdot D_3 + \overline{D_6} \cdot \overline{D_4} \cdot \overline{D_2} \cdot D_1$$

By incorporating these negated terms ($\overline{D_n}$), the combinational circuit inherently locks out lower-order inputs whenever a higher-order input is detected, successfully resolving both multiple-input and zero-state ambiguities.

Q8. What is a priority encoder? Design a four-input priority encoder with inputs I_0, I_1, I_2, I_3 , where the priority order is $I_2 > I_3 > I_1 > I_0$. (10 marks) [CSE 205 | L-2/T-1 | 2013–14]

- **Lowest Priority (I_0):** Outputs 00₂ (decimal 0)

We construct the truth table listing inputs in descending order of priority. An ‘X’ represents a “Don’t Care” condition, emphasizing that lower-order inputs are ignored.

Inputs (Highest → Lowest)				Outputs		
I_2 (P3)	I_3 (P2)	I_1 (P1)	I_0 (P0)	Y_1	Y_0	V
0	0	0	0	0	0	0
1	X	X	X	1	0	1
0	1	X	X	1	1	1
0	0	1	X	0	1	1
0	0	0	1	0	0	1

3. Mathematical Derivation of Boolean Equations

Valid Bit (V): The valid bit is HIGH if any of the inputs are HIGH.

$$V = I_2 + I_3 + I_1 + I_0$$

Most Significant Bit (Y_1): From the truth table, Y_1 is ‘1’ for the rows where I_2 is active or I_3 is active (and I_2 is not).

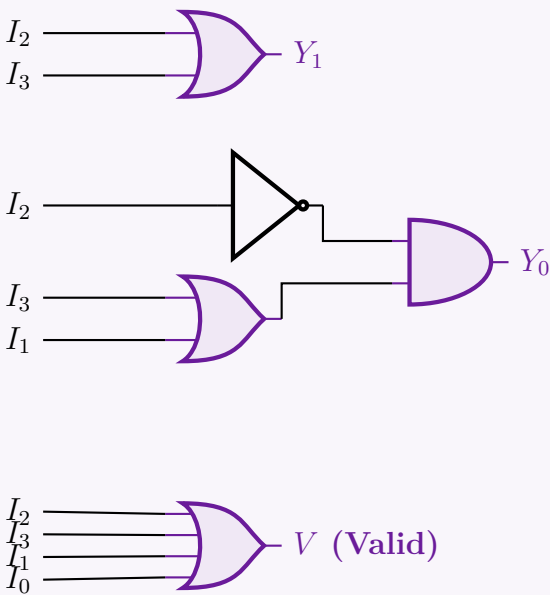
$$Y_1 = I_2 + \overline{I_2}I_3$$
$$Y_1 = I_2 + I_3 \quad (\text{Applying the Distributive Law: } A + \overline{A}B = A + B)$$

Least Significant Bit (Y_0): Y_0 is ‘1’ for the row where I_3 is active (with $I_2 = 0$), and the row where I_1 is active (with $I_2 = 0$ and $I_3 = 0$).

$$Y_0 = \overline{I_2}I_3 + \overline{I_2} \cdot \overline{I_3}I_1$$
$$= \overline{I_2}(I_3 + \overline{I_3}I_1) \quad (\text{Factoring out } \overline{I_2})$$
$$Y_0 = \overline{I_2}(I_3 + I_1) \quad (\text{Applying } A + \overline{A}B = A + B \text{ inside the parenthesis})$$

4. Logic Circuit Implementation

The derived equations use minimal standard logic gates. We require one NOT gate, two OR gates, and one AND gate for the output mapping, alongside a 4-input OR gate for the valid bit.



Q9. Design a 4-input priority encoder and show the gate-level circuit diagram. (12 marks) [CSE 205 | L-2/T-1 | 2011–12, 2013–14]

Answer

1. Priority Specification and Truth Table

A 4-input priority encoder has four inputs (D_3, D_2, D_1, D_0) and outputs a binary code (Y_1, Y_0) indicating the active input with the highest priority. It also includes a Valid bit (V) that is logic ‘1’ when at least one input is active. We will assume the standard priority ordering where D_3 has the highest priority and D_0 has the lowest priority. If a higher-priority input is active, the states of all lower-priority inputs are treated as “Don’t Cares” (denoted by ‘X’).

Inputs				Outputs		
D_3 (Highest)	D_2	D_1	D_0 (Lowest)	Y_1	Y_0	V (Valid)
0	0	0	0	X	X	0
1	X	X	X	1	1	1
0	1	X	X	1	0	1
0	0	1	X	0	1	1
0	0	0	1	0	0	1

2. Boolean Logic Derivation

We derive the Boolean equations from the truth table by identifying the conditions where each output is ‘1’.
Valid Bit (V): The valid bit is HIGH if any of the four inputs is HIGH.

$V = D_3 + D_2 + D_1 + D_0$

Most Significant Output Bit (Y_1): Y_1 is HIGH when D_3 is HIGH, or when D_2 is HIGH (provided D_3 is LOW).

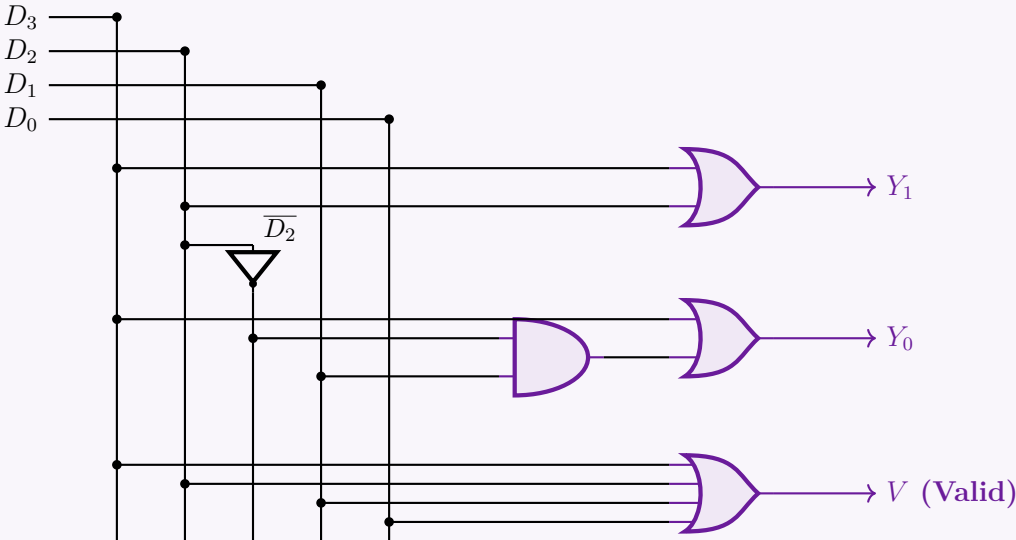
$Y_1 = D_3 + \overline{D_3}D_2$
 $Y_1 = D_3 + D_2$ (Applied Distributive Law: $A + \overline{A}B = A + B$)

Least Significant Output Bit (Y_0): Y_0 is HIGH when D_3 is HIGH, or when D_1 is HIGH (provided both D_3 and D_2 are LOW).

$Y_0 = D_3 + \overline{D_3} \cdot \overline{D_2}D_1$
 $Y_0 = D_3 + \overline{D_2}D_1$ (Applied Distributive Law: $A + \overline{A}X = A + X$, where $X = \overline{D_2}D_1$)

3. Gate-Level Circuit Diagram

Using the simplified Boolean equations, the priority encoder can be realized using basic logic gates (NOT, AND, OR).



Decoders

Q1. Design a 1-bit full subtractor using a 4 to 16 line decoder that has active low outputs and two active low enables $\overline{EN_1}$ and $\overline{EN_2}$.
(12 marks) [CSE 205 | L-2/T-1 | 2024–25]

Answer

1. Full Subtractor Truth Table and Minterm Extraction

A 1-bit full subtractor subtracts a subtrahend bit (Y) and a borrow-in bit (B_{in}) from a minuend bit (X), producing a Difference (D) and a Borrow-out (B_{out}). We begin by defining its truth table.

Inputs			Outputs		Minterm
X (Minuend)	Y (Subtrahend)	B_{in} (Borrow In)	Difference (D)	Borrow Out (B_{out})	
0	0	0	0	0	m_0
0	0	1	1	1	m_1
0	1	0	1	1	m_2
0	1	1	0	1	m_3
1	0	0	1	0	m_4
1	0	1	0	0	m_5
1	1	0	0	0	m_6
1	1	1	1	1	m_7

2. Boolean Expressions and De Morgan’s Law Application

From the truth table, we extract the canonical Sum of Products (SOP) expressions for D and B_{out} :

$$D(X,Y,B_{in}) = \sum m(1,2,4,7) = m_1 + m_2 + m_4 + m_7$$

$$B_{out}(X,Y,B_{in}) = \sum m(1,2,3,7) = m_1 + m_2 + m_3 + m_7$$

The problem specifies that the decoder has **active-low outputs**. This means the decoder outputs are the complements of the standard minterms ($\overline{m_0}, \overline{m_1}, \dots, \overline{m_{15}}$).
To implement a logical OR operation using active-low inputs, we apply **De Morgan’s Law** ($\overline{A \cdot B} = \overline{A} + \overline{B}$). By feeding the active-low outputs into a **NAND gate**, we reconstruct the SOP form:

$$D = m_1 + m_2 + m_4 + m_7 = \overline{\overline{m_1} \cdot \overline{m_2} \cdot \overline{m_4} \cdot \overline{m_7}}$$

$$\Rightarrow D = \text{NAND}(\overline{m_1}, \overline{m_2}, \overline{m_4}, \overline{m_7})$$

$$B_{out} = m_1 + m_2 + m_3 + m_7 = \overline{\overline{m_1} \cdot \overline{m_2} \cdot \overline{m_3} \cdot \overline{m_7}}$$

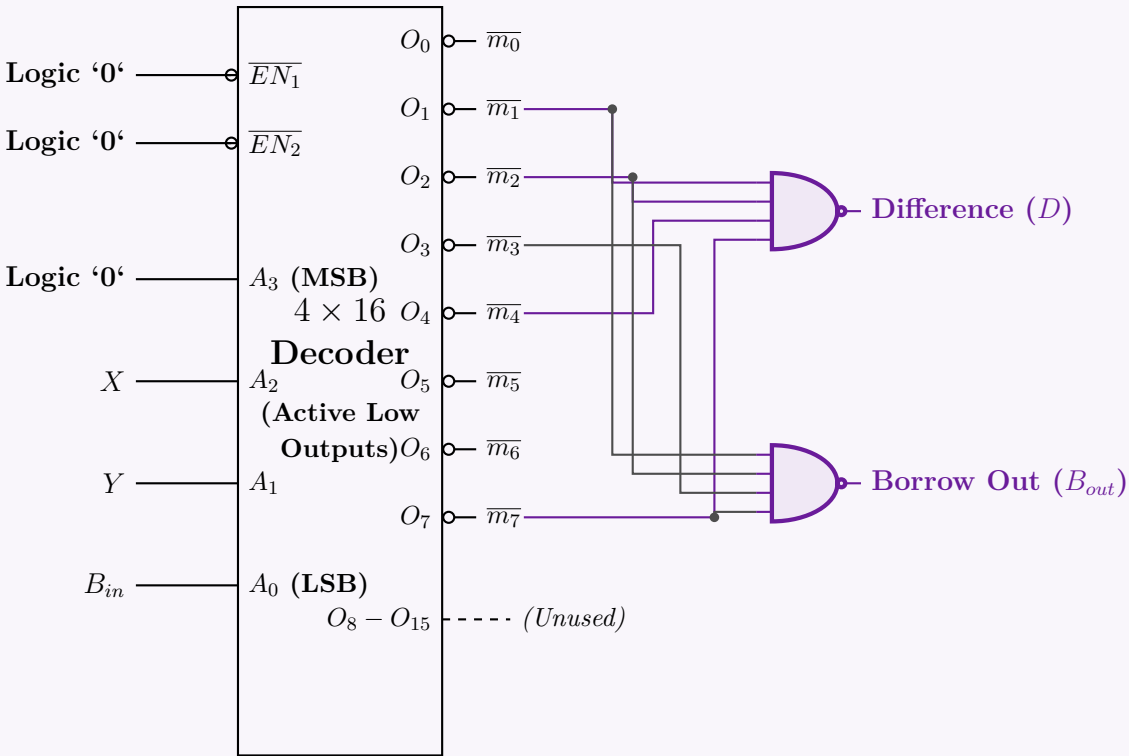
$$\Rightarrow B_{out} = \text{NAND}(\overline{m_1}, \overline{m_2}, \overline{m_3}, \overline{m_7})$$

3. Decoder Configuration

To adapt a 4×16 decoder into our required 3×8 configuration:

- **Data Inputs:** Connect the subtractor inputs X, Y, B_{in} to the lower three decoder inputs A_2, A_1, A_0 .
- **MSB Grounding:** Tie the most significant bit input (A_3) to Logic ‘0’ (Ground). This disables outputs $\overline{m_8}$ through $\overline{m_{15}}$, restricting operation to the first 8 outputs.
- **Enable Pins:** Connect the two active-low enables ($\overline{EN_1}$ and $\overline{EN_2}$) to Logic ‘0’ (Ground) to keep the chip permanently active.

4. Logic Circuit Implementation



Q2. Design a 4-to-16-line decoder by using only the minimum number of 2-to-4-line decoders. Assume that the 2-to-4-line decoders have an enable input (‘1’ = enabled) but the designed 4-to-16-line decoder does not have an enable input. **(12 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Answer

1. Design Strategy and Component Count

To design a 4-to-16-line decoder, we must decode a 4-bit binary input (A_3, A_2, A_1, A_0) into 16 distinct outputs (D_0 to D_{15}). Given that we are restricted to 2-to-4-line decoders (which have 2 inputs, 4 outputs, and 1 enable pin), we need to determine the minimum number of components:

- Output Stage:** To generate 16 outputs, we need at least $16 \div 4 = 4$ decoders.
- Input/Decoding Stage (Root):** To select which of these 4 output decoders is active at any given time, we need a mechanism to decode the two Most Significant Bits (MSBs). This requires exactly **1** additional 2-to-4-line decoder.

Therefore, the **minimum number of 2-to-4-line decoders required is 5**.

2. Input and Output Mapping

We partition the 4-bit input $A_3A_2A_1A_0$ as follows:

- Least Significant Bits (A_1, A_0):** These are connected in parallel to the I_1 and I_0 inputs of all four output-stage decoders. They determine which specific line (0 to 3) is activated *within* the enabled decoder.
- Most Significant Bits (A_3, A_2):** These are connected to the I_1 and I_0 inputs of the root decoder. The root decoder will assert exactly one of its outputs based on A_3 and A_2 .
- Enable Logic (E):** The outputs of the root decoder (O_0, O_1, O_2, O_3) are connected to the Enable (E) inputs of the four output-stage decoders, respectively. Because the final 4-to-16-line decoder is specified to have **no enable input**, the root decoder's enable input is permanently tied to **Logic '1' (HIGH)**.

3. Truth Table Partitioning

The functional partition behaves according to the state of the MSBs:

A_3	A_2	Active Output Decoder	Active Output Range (based on A_1, A_0)
0	0	Decoder 0 (Root $O_0 = 1$)	D_0, D_1, D_2, D_3
0	1	Decoder 1 (Root $O_1 = 1$)	D_4, D_5, D_6, D_7
1	0	Decoder 2 (Root $O_2 = 1$)	D_8, D_9, D_{10}, D_{11}
1	1	Decoder 3 (Root $O_3 = 1$)	$D_{12}, D_{13}, D_{14}, D_{15}$

4. Logic Circuit Implementation

Note: All 2-to-4-line decoders shown below feature active-high inputs and outputs, and an active-high Enable (E) pin as per the problem description.

The diagram illustrates the logic circuit implementation of a 4-to-16-line decoder using five 2-to-4-line decoders. A root decoder (labeled "Root 2x4 I1Decoder") has inputs A_3 (MSB) and A_2 , and its enable E is tied to Logic '1'. Its outputs O_0, O_1, O_2, O_3 enable four output-stage decoders (labeled "2x4 Dec 0", "2x4 Dec 1", "2x4 Dec 2", and "2x4 Dec 3"). The inputs A_1 and A_0 (LSB) are connected to the I_1 and I_0 inputs of all four output-stage decoders. The outputs of the decoders are $D_0-D_3, D_4-D_7, D_8-D_{11},$ and $D_{12}-D_{15}$ respectively.

156

Q3. Design a full adder circuit using a 3×8 decoder and two OR gates. (10 marks)

[CSE 205 | L-2/T-1 | 2022–23]

Answer

1. Full Adder Truth Table and Minterm Extraction

A Full Adder is a combinational logic circuit that performs the arithmetic sum of three bits: two significant bits (A and B) and a carry-in bit (C_{in}). It produces two outputs: a Sum (S) and a Carry-out (C_{out}). To design the circuit using a decoder, we first map the input-output relationship using a truth table and extract the canonical minterms.

Inputs			Outputs		Minterm
A	B	C_{in}	Sum (S)	Carry (C_{out})	
0	0	0	0	0	m_0
0	0	1	1	0	m_1
0	1	0	1	0	m_2
0	1	1	0	1	m_3
1	0	0	1	0	m_4
1	0	1	0	1	m_5
1	1	0	0	1	m_6
1	1	1	1	1	m_7

2. Boolean Logic Derivation

A 3×8 decoder generates all $2^3 = 8$ possible minterms for 3 input variables. Each output pin of the decoder represents exactly one minterm. Any Boolean function can be expressed in the Canonical Sum of Products (SOP) form. Thus, by identifying the minterms where the outputs are logic ‘1’, we can form the functions:
Sum Output (S):
$$S(A, B, C_{in}) = \sum m(1, 2, 4, 7)$$
$$S = m_1 + m_2 + m_4 + m_7$$

Carry Output (C_{out}):
$$C_{out}(A, B, C_{in}) = \sum m(3, 5, 6, 7)$$
$$C_{out} = m_3 + m_5 + m_6 + m_7$$

3. Circuit Implementation Strategy

The implementation requires a 3×8 decoder and two 4-input OR gates:

- Connect the inputs A, B , and C_{in} to the select lines I_2, I_1 , and I_0 of the decoder, respectively.
- Tie the enable pin (E) of the decoder to logic HIGH (‘1’).
- Route the outputs corresponding to m_1, m_2, m_4 , and m_7 to the first OR gate to yield the **Sum** (S).
- Route the outputs corresponding to m_3, m_5, m_6 , and m_7 to the second OR gate to yield the **Carry** (C_{out}).

4. Gate-Level Circuit Diagram

Q4. Implement $F(x, y, z) = \Sigma(3, 5, 6, 7)$ using an active low 3×8 decoder and AND gates. **(10 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Answer

1. Truth Table and Boolean Function Analysis

We are given the Boolean function $F(x, y, z) = \Sigma(3, 5, 6, 7)$. Let us first analyze the truth table to identify the minterms (where $F = 1$) and maxterms (where $F = 0$).

Inputs			Output F	Minterm Notation
x	y	z		
0	0	0	0	m_0
0	0	1	0	m_1
0	1	0	0	m_2
0	1	1	1	m_3
1	0	0	0	m_4
1	0	1	1	m_5
1	1	0	1	m_6
1	1	1	1	m_7

2. Mathematical Derivation for Active-Low Decoder

A standard sum-of-products (SOP) expression for F uses the active minterms:

$$\begin{aligned} F(x, y, z) &= \Sigma m(3, 5, 6, 7) \\ &= m_3 + m_5 + m_6 + m_7 \end{aligned}$$

However, we must implement this using a decoder with **active-low outputs** and **AND gates**. An active-low decoder outputs the *complement* of the minterms (i.e., $O_i = \overline{m_i}$).

To match the available hardware (AND gates and complemented minterms), we can express the function F using its Product of Sums (POS) form, which groups the *maxterms* (the rows where $F = 0$):

$$\begin{aligned} F(x, y, z) &= \Pi M(0, 1, 2, 4) \\ &= M_0 \cdot M_1 \cdot M_2 \cdot M_4 \quad (\text{Product of Maxterms}) \end{aligned}$$

By definition, a maxterm is the exact complement of a minterm ($M_i = \overline{m_i}$). Substituting this relationship into our equation yields:

$$F(x, y, z) = \overline{m_0} \cdot \overline{m_1} \cdot \overline{m_2} \cdot \overline{m_4}$$

Conclusion: Since the active-low decoder directly outputs $\overline{m_0}, \overline{m_1}, \overline{m_2}$, and $\overline{m_4}$, we simply connect these specific four output lines to a single **4-input AND gate** to realize the function F .

3. Logic Circuit Implementation

The diagram shows a 3x8 decoder with inputs x (MSB), y , and z (LSB). The enable input $\overline{E}$ is connected to Logic '0'. The decoder has eight active-low outputs labeled O_0 through O_7 , which are also labeled $\overline{m_0}$ through $\overline{m_7}$. A 4-input AND gate is connected to the outputs $O_0, O_1, O_2,$ and O_4 . The output of the AND gate is the function $F(x, y, z)$.

Q5. Design a logic circuit which takes two 2-bit numbers X_1X_0 and Y_1Y_0 and finds their sum using a 2×4 line decoder with inputs I_0, I_1 , active-low outputs O_3, O_2, O_1, O_0 , and active-low enables $\overline{EN}_1$ and $\overline{EN}_2$. (Use the minimum number of extra logic gates.) **(15 marks)** [CSE 205 | L-2/T-1 | 2020–21]

Answer

1. Functional Partitioning of the 2-Bit Adder

The addition of two 2-bit numbers, X_1X_0 and Y_1Y_0 , yields a 3-bit sum $S_2S_1S_0$. We can partition this operation into two distinct stages:

- **LSB Stage (Half Adder):** Adds X_0 and Y_0 to produce Sum S_0 and Carry-out C_1 . We will implement this stage using the provided 2×4 decoder to minimize extra gates.
- **MSB Stage (Full Adder):** Adds X_1 , Y_1 , and C_1 to produce Sum S_1 and the final Carry-out S_2 . This will be implemented using standard logic gates.

2. LSB Implementation using the 2×4 Decoder

The 2×4 decoder has inputs I_0, I_1 and active-low outputs O_0, O_1, O_2, O_3 . To enable the decoder, both active-low enables ($\overline{EN}_1$ and $\overline{EN}_2$) must be tied to Logic '0' (Ground).

We connect the LSBs to the decoder inputs: $I_0 = X_0$ and $I_1 = Y_0$. The decoder generates the active-low minterms of these variables: $O_i = \overline{m}_i$.

The Half Adder Boolean equations are:

$$\begin{aligned} \text{Sum: } S_0 &= X_0 \oplus Y_0 = m_1 + m_2 \\ \text{Carry: } C_1 &= X_0 \cdot Y_0 = m_3 \end{aligned}$$

Using De Morgan's Law ($\overline{A \cdot B} = \overline{A} + \overline{B}$), we can synthesize these using the active-low decoder outputs with minimum extra gates:

$$\begin{aligned} S_0 &= \overline{\overline{m}_1 \cdot \overline{m}_2} = \text{NAND}(O_1, O_2) \quad (\text{Requires 1 NAND gate}) \\ C_1 &= \overline{\overline{m}_3} = \text{NOT}(O_3) \quad (\text{Requires 1 NOT gate}) \end{aligned}$$

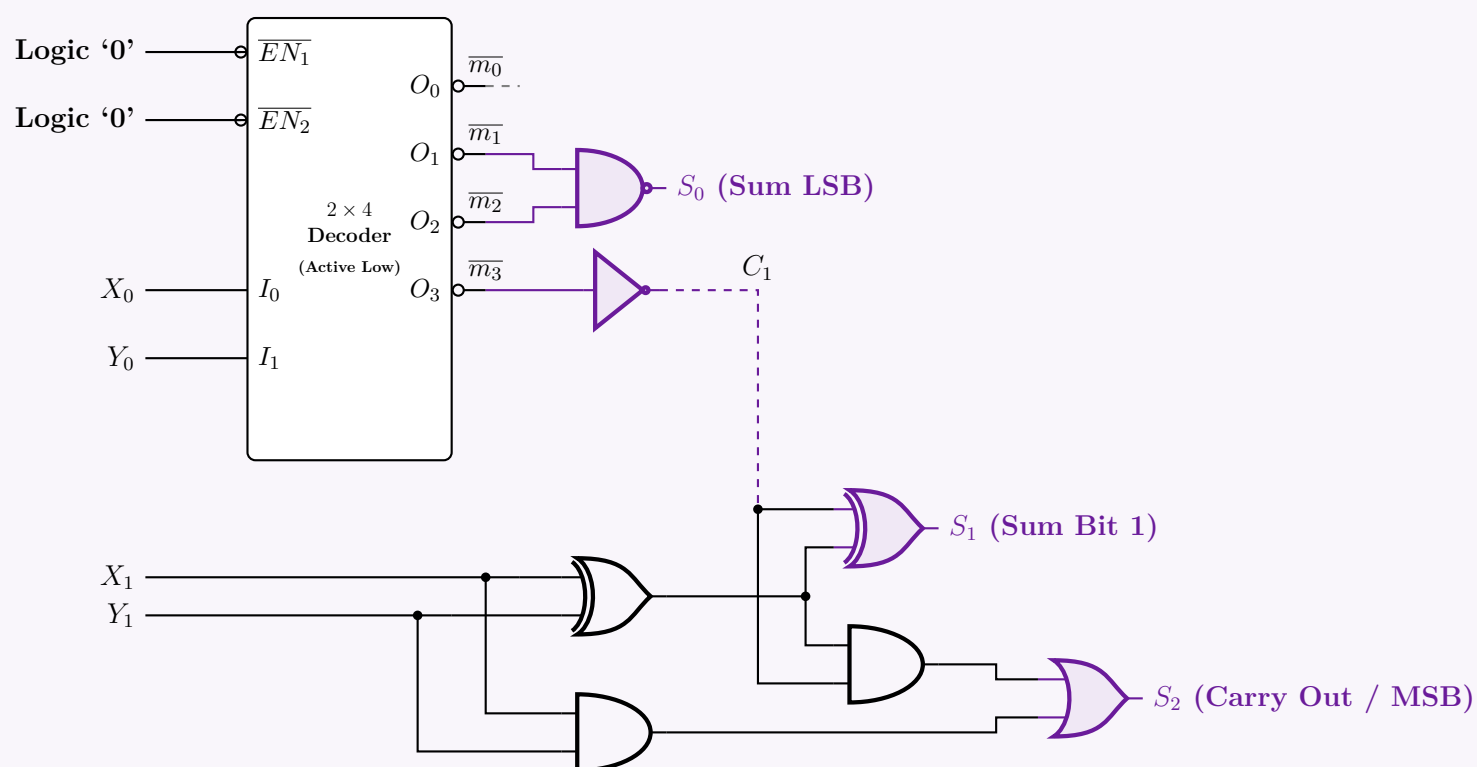
3. MSB Implementation (Full Adder)

For the MSB, we use standard logic gates. To achieve minimum gate count, we share the XOR product ($X_1 \oplus Y_1$):

$$\begin{aligned} S_1 &= (X_1 \oplus Y_1) \oplus C_1 \\ S_2 &= \text{Carry-out} = (X_1 \cdot Y_1) + C_1 \cdot (X_1 \oplus Y_1) \end{aligned}$$

This requires 2 XOR gates, 2 AND gates, and 1 OR gate.

4. Complete Logic Circuit Diagram



Q6. Design a logic circuit for $f(w, x, y, z) = \Sigma(0, 2, 6, 7, 10, 11, 12, 14)$ with a 2×4 line decoder that has inputs A, B , active-low enables $\overline{EN}_1$ and $\overline{EN}_2$. (Use the minimum number of extra logic gates.) **(14 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Shannon Expansion & Functional Partitioning

To implement the 4-variable function $f(w, x, y, z) = \Sigma(0, 2, 6, 7, 10, 11, 12, 14)$ using a 2×4 line decoder, we must strategically partition the function. A 2×4 decoder yields the four minterms of its two input variables. By assigning inputs $A = y$ and $B = z$, the decoder's active-high outputs will generate $\overline{y}z$, $\overline{y}\overline{z}$, $y\overline{z}$, and yz .

We apply **Shannon's Expansion Theorem** to expand $f(w, x, y, z)$ around variables y and z :

$$f(w, x, y, z) = \overline{y}z \cdot f(w, x, 0, 0) + \overline{y}z \cdot f(w, x, 0, 1) + y\overline{z} \cdot f(w, x, 1, 0) + yz \cdot f(w, x, 1, 1)$$

2. Evaluating the Residual Functions

We extract the active minterms from the function for each combination of (y, z) :

- **For $y = 0, z = 0$ (Decoder Output O_0):**

Minterms where $y = 0, z = 0$ are 0, 4, 8, 12. In our function, only minterms 0(0000) and 12(1100) are present.

$$f(w, x, 0, 0) = \overline{w}x + wx = w \odot x \quad (\text{XNOR})$$

- **For $y = 0, z = 1$ (Decoder Output O_1):**

Minterms where $y = 0, z = 1$ are 1, 5, 9, 13. None of these exist in f .

$$f(w, x, 0, 1) = 0$$

- **For $y = 1, z = 0$ (Decoder Output O_2):**

Minterms where $y = 1, z = 0$ are 2, 6, 10, 14. All of these minterms (0010, 0110, 1010, 1110) are present in f .

$$f(w, x, 1, 0) = 1$$

- **For $y = 1, z = 1$ (Decoder Output O_3):**

Minterms where $y = 1, z = 1$ are 3, 7, 11, 15. In our function, only minterms 7(0111) and 11(1011) are present.

$$f(w, x, 1, 1) = \overline{w}x + w\overline{x} = w \oplus x \quad (\text{XOR})$$

3. Boolean Logic Minimization

Substituting these residual functions back into the Shannon expansion gives:

$$f(w, x, y, z) = (\overline{y}z) \cdot (w \odot x) + (\overline{y}z) \cdot (0) + (y\overline{z}) \cdot (1) + (yz) \cdot (w \oplus x)$$

Let the outputs of the 2×4 decoder be $O_0 = \overline{y}z$, $O_1 = \overline{y}z$, $O_2 = y\overline{z}$, and $O_3 = yz$. The equation maps directly to the decoder pins:

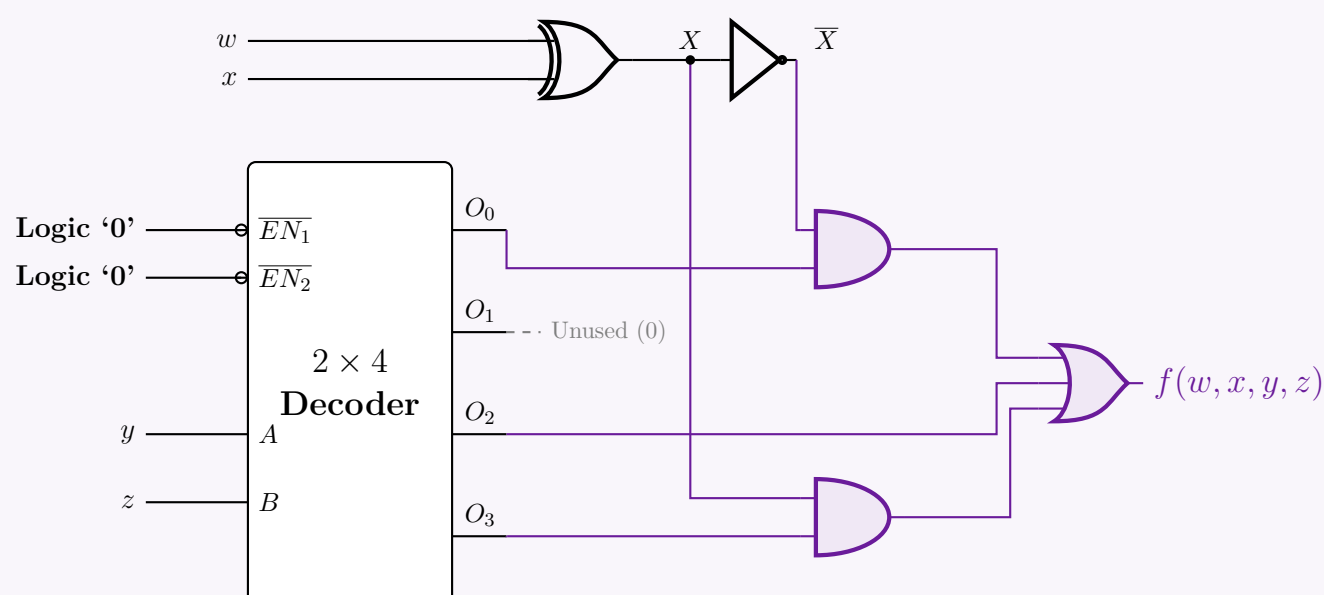
$$f(w, x, y, z) = O_0 \cdot (w \odot x) + O_2 + O_3 \cdot (w \oplus x)$$

To use the **minimum number of extra logic gates**, we note the complementary relationship: $(w \odot x) = \overline{w \oplus x}$. We can generate intermediate variable $X = w \oplus x$ using one XOR gate, and branch it through a NOT gate to yield $\overline{X} = w \odot x$.

$$f_{final} = O_0 \cdot \overline{X} + O_2 + O_3 \cdot X$$

Gate Count: 1 XOR, 1 NOT, 2 ANDs, and 1 3-input OR. The active-low enables ($\overline{EN}_1, \overline{EN}_2$) must be tied to Ground ('Logic 0') to keep the decoder permanently active.

4. Logic Circuit Implementation



Another Solution(Not a minimum gate solution) :

1. Shannon Expansion with Selection Inputs $A = w, B = x$

We expand $f(w, x, y, z) = \Sigma(0, 2, 6, 7, 10, 11, 12, 14)$ around variables w and x :

$$f(w, x, y, z) = \overline{w}x \cdot f(0, 0, y, z) + \overline{w}x \cdot f(0, 1, y, z) + w\overline{x} \cdot f(1, 0, y, z) + wx \cdot f(1, 1, y, z)$$

2. Evaluating Simplified Residual Functions

We extract the residual functions for y and z from the minterm definitions:

- **For $w = 0, x = 0$ (Decoder Output $O_0 = \overline{wx}$):**
Contains minterms 0(0000) and 2(0010).

$$f(0, 0, y, z) = \overline{y}z + y\overline{z} = \overline{z}(\overline{y} + y) = \overline{z}$$

- **For $w = 0, x = 1$ (Decoder Output $O_1 = \overline{w}x$):**
Contains minterms 6(0110) and 7(0111).

$$f(0, 1, y, z) = y\overline{z} + yz = y(\overline{z} + z) = y$$

- **For $w = 1, x = 0$ (Decoder Output $O_2 = w\overline{x}$):**
Contains minterms 10(1010) and 11(1011).

$$f(1, 0, y, z) = y\overline{z} + yz = y(\overline{z} + z) = y$$

- **For $w = 1, x = 1$ (Decoder Output $O_3 = wx$):**
Contains minterms 12(1100) and 14(1110).

$$f(1, 1, y, z) = \overline{y}z + y\overline{z} = \overline{z}(\overline{y} + y) = \overline{z}$$

3. Simplified Boolean Expression

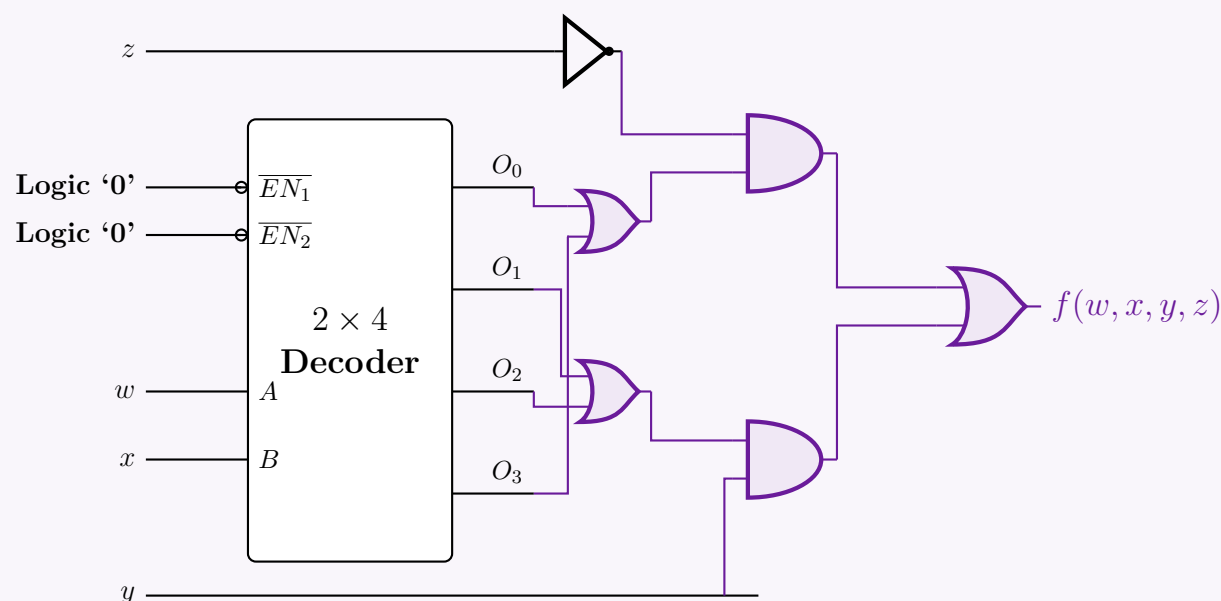
Substituting the simplified literal outputs back into the expansion equation:

$$f(w, x, y, z) = O_0 \cdot \overline{z} + O_1 \cdot y + O_2 \cdot y + O_3 \cdot \overline{z}$$

Factoring common literals y and $\overline{z}$:

$$f(w, x, y, z) = (O_0 + O_3) \cdot \overline{z} + (O_1 + O_2) \cdot y$$

4. Logic Circuit Implementation



Q7. Design a logic circuit for $f(w, x, y, z) = \text{SOP}(0, 2, 6, 7, 8, 10, 11, 13, 14)$ with a 2×4 line decoder that has inputs A, B , active-low enable $\overline{EN}$ and active-low outputs D'_0, D'_1, D'_2, D'_3 . (Use the minimum number of extra logic gates.) **(12 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

1. Functional Partitioning via Shannon's Expansion

To implement the 4-variable boolean function $f(w, x, y, z)$ using a 2×4 line decoder, we partition the function using two variables. We choose the least significant variables, y and z , as the decoder inputs ($A = y, B = z$).

The decoder has active-low outputs D'_0, D'_1, D'_2, D'_3 . These correspond to the complemented minterms of (y, z) :

$$D'_0 = \overline{y}\overline{z} \quad D'_1 = \overline{y}z \quad D'_2 = y\overline{z} \quad D'_3 = yz$$

Applying Shannon's Expansion Theorem around y and z :

$$\begin{aligned} f(w, x, y, z) &= (\overline{y}\overline{z}) \cdot I_0(w, x) + (\overline{y}z) \cdot I_1(w, x) + (y\overline{z}) \cdot I_2(w, x) + (yz) \cdot I_3(w, x) \\ &= \overline{D'_0} \cdot I_0(w, x) + \overline{D'_1} \cdot I_1(w, x) + \overline{D'_2} \cdot I_2(w, x) + \overline{D'_3} \cdot I_3(w, x) \end{aligned}$$

2. Evaluating the Residual Functions $I_i(w, x)$

We evaluate $f(w, x, y, z) = \sum(0, 2, 6, 7, 8, 10, 11, 13, 14)$ for each combination of y and z :

- **For $y = 0, z = 0$ (Minterms 0, 4, 8, 12):**
Given in f : $\{0, 8\}$.

$$I_0(w, x) = \bar{w}\bar{x} + w\bar{x} = \bar{x}(\bar{w} + w) = \bar{x} \quad (\text{Distributive \& Complement Laws})$$

- **For $y = 0, z = 1$ (Minterms 1, 5, 9, 13):**
Given in f : $\{13\}$.

$$I_1(w, x) = wx$$

- **For $y = 1, z = 0$ (Minterms 2, 6, 10, 14):**
Given in f : $\{2, 6, 10, 14\}$.

$$I_2(w, x) = \bar{w}\bar{x} + \bar{w}x + w\bar{x} + wx = 1 \quad (\text{Identity})$$

- **For $y = 1, z = 1$ (Minterms 3, 7, 11, 15):**
Given in f : $\{7, 11\}$.

$$I_3(w, x) = \bar{w}x + w\bar{x} = w \oplus x \quad (\text{XOR definition})$$

3. Gate Minimization using De Morgan's Laws

Substitute the residual functions back into the expanded equation:

$$f(w, x, y, z) = \bar{D}'_0 \cdot \bar{x} + \bar{D}'_1 \cdot (wx) + \bar{D}'_2 \cdot (1) + \bar{D}'_3 \cdot (w \oplus x)$$

To minimize hardware with active-low decoder outputs, we apply **De Morgan's Involution Law** ($\bar{\bar{f}} = f$) to convert the outer Sum-of-Products into a NAND-NAND equivalent structure:

$$f(w, x, y, z) = \overline{\bar{D}'_0 \cdot \bar{x} \cdot \bar{D}'_1 \cdot (wx) \cdot \bar{D}'_2 \cdot \bar{D}'_3 \cdot (w \oplus x)}$$

Applying De Morgan's Law ($\overline{A \cdot B} = \bar{A} + \bar{B}$) to the inner terms:

$$\overline{\bar{D}'_0 \cdot \bar{x}} = D'_0 + x$$

$$\overline{\bar{D}'_1 \cdot (wx)} = D'_1 + \bar{w}\bar{x} = D'_1 + (w \text{ NAND } x)$$

$$\overline{\bar{D}'_2} = D'_2$$

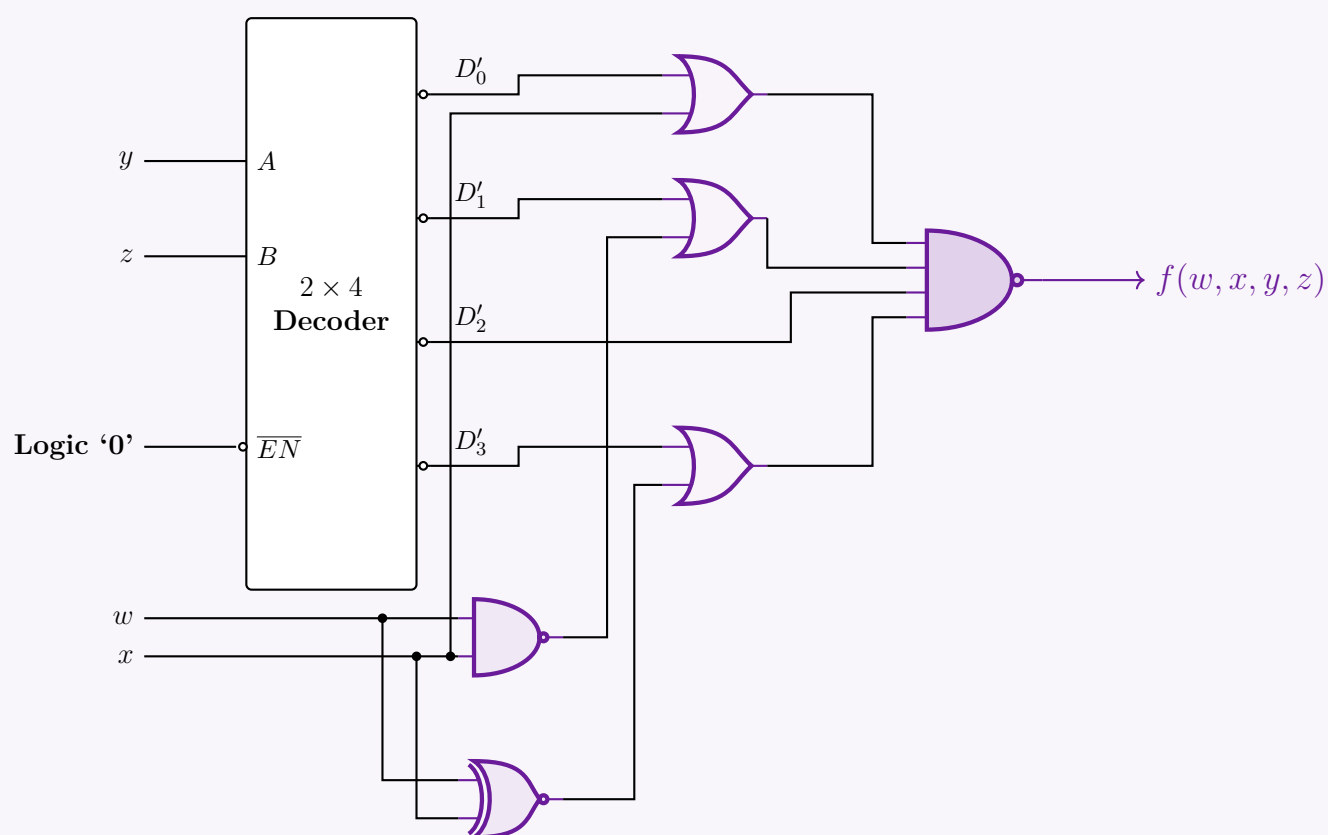
$$\overline{\bar{D}'_3 \cdot (w \oplus x)} = D'_3 + \overline{w \oplus x} = D'_3 + (w \text{ XNOR } x)$$

Final optimized Boolean equation mapping directly to gates:

$$f(w, x, y, z) = \text{NAND}_4 \left((D'_0 + x), (D'_1 + \text{NAND}(w, x)), D'_2, (D'_3 + \text{XNOR}(w, x)) \right)$$

Gate Count: 1 NAND2, 1 XNOR2, 3 OR2, and 1 NAND4 gate.

4. Logic Circuit Implementation



Another Solution(Not a minimum gate solution) :

1. Functional Partitioning via Shannon's Expansion

We partition the function $f(w, x, y, z) = \sum(0, 2, 6, 7, 8, 10, 11, 13, 14)$ around variables w and x . The 2×4 decoder yields active-low outputs corresponding to the complemented minterms of (w, x) :

$$D'_0 = \overline{w} \overline{x}$$

$$D'_1 = \overline{w} x$$

$$D'_2 = w \overline{x}$$

$$D'_3 = w x$$

The Shannon expansion becomes:

$$f(w, x, y, z) = \overline{D'_0} \cdot I_0(y, z) + \overline{D'_1} \cdot I_1(y, z) + \overline{D'_2} \cdot I_2(y, z) + \overline{D'_3} \cdot I_3(y, z)$$

2. Evaluating the Residual Functions $I_i(y, z)$

We extract the active minterms from f for each combination of (w, x) :

- **For $w = 0, x = 0$ (Decoder Output D'_0):**
Minterms 0, 1, 2, 3. Present in f : $\{0, 2\}$.

$$I_0(y, z) = \overline{y} \overline{z} + y \overline{z} = \overline{z}(\overline{y} + y) = \overline{z}$$

- **For $w = 0, x = 1$ (Decoder Output D'_1):**
Minterms 4, 5, 6, 7. Present in f : $\{6, 7\}$.

$$I_1(y, z) = y \overline{z} + y z = y(\overline{z} + z) = y$$

- **For $w = 1, x = 0$ (Decoder Output D'_2):**
Minterms 8, 9, 10, 11. Present in f : $\{8, 10, 11\}$.

$$I_2(y, z) = \overline{y} \overline{z} + y \overline{z} + y z = \overline{z}(\overline{y} + y) + y z = \overline{z} + y z = \overline{z} + y$$

- **For $w = 1, x = 1$ (Decoder Output D'_3):**
Minterms 12, 13, 14, 15. Present in f : $\{13, 14\}$.

$$I_3(y, z) = \overline{y} z + y \overline{z} = y \oplus z$$

3. Gate Minimization using De Morgan's Laws

Substitute the residual functions back into the expanded equation:

$$f(w, x, y, z) = \overline{D'_0} \cdot \overline{z} + \overline{D'_1} \cdot y + \overline{D'_2} \cdot (y + \overline{z}) + \overline{D'_3} \cdot (y \oplus z)$$

Because the decoder outputs D'_i are active-low, we apply **De Morgan's Involution Law** ($\overline{\overline{f}} = f$) to convert the outer Sum-of-Products into a NAND-NAND structure compatible with OR gates:

$$f(w, x, y, z) = \overline{\overline{\overline{D'_0} \cdot \overline{z}} \cdot \overline{\overline{D'_1} \cdot y} \cdot \overline{\overline{D'_2} \cdot (y + \overline{z})} \cdot \overline{\overline{D'_3} \cdot (y \oplus z)}}$$

Applying De Morgan's Law ($\overline{A \cdot B} = \overline{A} + \overline{B}$) to the inner terms:

$$\overline{\overline{D'_0} \cdot \overline{z}} = D'_0 + z$$

$$\overline{\overline{D'_1} \cdot y} = D'_1 + \overline{y}$$

$$\overline{\overline{D'_2} \cdot (y + \overline{z})} = D'_2 + \overline{y + \overline{z}} = D'_2 + \overline{y} z \quad (\text{De Morgan's again})$$

$$\overline{\overline{D'_3} \cdot (y \oplus z)} = D'_3 + \overline{y \oplus z} = D'_3 + (y \odot z) \quad (\text{XNOR})$$

Final logic equation mapping directly to gates:

$$f(w, x, y, z) = \text{NAND}_4 \left((D'_0 + z), (D'_1 + \overline{y}), (D'_2 + \overline{y} z), (D'_3 + (y \odot z)) \right)$$

4. Logic Circuit Implementation

The diagram shows a 2×4 decoder with inputs w (labeled A) and x (labeled B). The enable pin $\overline{EN}$ is connected to Logic '0'. The decoder has four active-high outputs: D'_0 , D'_1 , D'_2 , and D'_3 . The output $f(w, x, y, z)$ is implemented using OR gates for D'_0 , D'_1 , and D'_2 , and an AND gate for D'_3 . The inputs y and z are connected to the OR gates for D'_0 , D'_1 , and D'_2 , and to the AND gate for D'_3 . The output $f(w, x, y, z)$ is the output of the AND gate for D'_3 .

Q8. Using a 2×4 line decoder, design a half subtractor. (Decoder outputs D_0, D_1, D_2, D_3 are active high; enable $\overline{EN}$ is active low.) (10 marks) [CSE 205 | L-2/T-1 | 2017–18]

Answer

1. Half Subtractor Functional Analysis

A half subtractor is a combinational circuit that performs the subtraction of two single-bit numbers, yielding a Difference (D) and a Borrow (B). Let the minuend be X and the subtrahend be Y .

Truth Table:

Inputs		Outputs	
X	Y	Difference (D)	Borrow (B)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

From the truth table, we can extract the standard Sum-of-Products (SOP) expressions in terms of minterms $m(X, Y)$:

$$D(X, Y) = \overline{X}Y + X\overline{Y} = \Sigma m(1, 2)$$
$$B(X, Y) = \overline{X}Y = \Sigma m(1)$$

2. Decoder Implementation Strategy

A 2×4 line decoder with active-high outputs takes two inputs (let's map $A = X, B = Y$) and asserts exactly one of its four outputs D_0, D_1, D_2, D_3 corresponding to the decimal equivalent of the binary input. The active-high outputs generate the canonical minterms of the input variables:

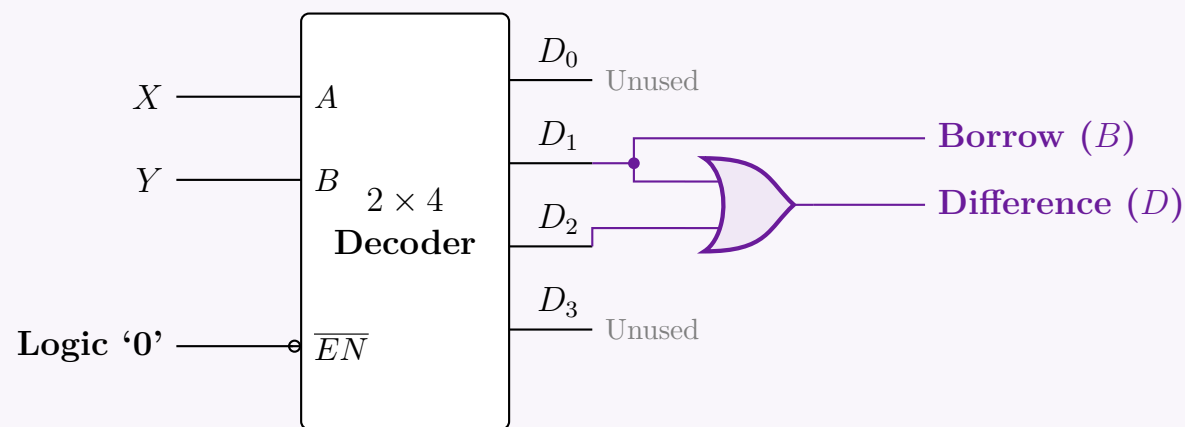
$$D_0 = m_0 = \overline{X}\overline{Y}$$
$$D_1 = m_1 = \overline{X}Y$$
$$D_2 = m_2 = X\overline{Y}$$
$$D_3 = m_3 = XY$$

By substituting the decoder outputs into the boolean expressions for the half subtractor, we obtain:

$$\text{Difference}(D) = D_1 + D_2 \quad \text{(Requires one 2-input OR gate)}$$
$$\text{Borrow}(B) = D_1 \quad \text{(Direct connection, no gates required)}$$

The decoder has an active-low enable ($\overline{EN}$). For the decoder to function normally, it must be permanently enabled by tying this pin to Logic '0' (Ground).

3. Logic Circuit Implementation



Q9. Using only a 2×4 line decoder, solve the function $f(A, B, C, D) = \pi(3, 5, 9, 15, 12, 4, 1)$. The 2×4 line decoder has an active-low enable and active-low outputs. (Use the minimum number of gates if required.) **(10 marks)** [CSE 205 | L-2/T-1 | 2016–17]

Answer

1. Standard Form Conversion (POS to SOP)

The given function is in Product of Sums (POS) form, specified by its maxterms:

$$f(A, B, C, D) = \pi(1, 3, 4, 5, 9, 12, 15)$$

For a 4-variable boolean function, the possible minterms/maxterms range from 0 to 15. The set of maxterms and minterms are mutually exclusive and collectively exhaustive. Therefore, we can express the function in its Sum of Products (SOP) form by listing the missing indices:

$$f(A, B, C, D) = \Sigma m(0, 2, 6, 7, 8, 10, 11, 13, 14)$$

2. Functional Partitioning via Shannon's Expansion

We use a 2×4 line decoder, requiring two select inputs. Let us partition the function using the least significant variables, C and D , as the decoder inputs ($S_1 = C$, $S_0 = D$).

The decoder features an active-low enable ($\overline{EN}$) and active-low outputs (D'_0, D'_1, D'_2, D'_3). When enabled ($\overline{EN} = 0$), the active-low outputs correspond to the complemented minterms of (C, D) :

$$D'_0 = \overline{C}\overline{D} \quad D'_1 = \overline{C}D \quad D'_2 = C\overline{D} \quad D'_3 = CD$$

Applying Shannon's Expansion Theorem with respect to C and D :

$$\begin{aligned} f(A, B, C, D) &= (\overline{C}\overline{D}) \cdot I_0(A, B) + (\overline{C}D) \cdot I_1(A, B) + (C\overline{D}) \cdot I_2(A, B) + (CD) \cdot I_3(A, B) \\ &= \overline{D'_0} \cdot I_0(A, B) + \overline{D'_1} \cdot I_1(A, B) + \overline{D'_2} \cdot I_2(A, B) + \overline{D'_3} \cdot I_3(A, B) \end{aligned}$$

3. Evaluating the Residual Functions $I_i(A, B)$

We group the SOP minterms $\{0, 2, 6, 7, 8, 10, 11, 13, 14\}$ based on the states of C and D :

- **For $C = 0, D = 0$ (D'_0 active):** Corresponds to minterms $\{0, 4, 8, 12\}$. Present in f : $\{0, 8\}$.

$$I_0(A, B) = \overline{A}\overline{B} + A\overline{B} = \overline{B}(\overline{A} + A) = \overline{B} \quad (\text{Distributive \& Complement Laws})$$

- **For $C = 0, D = 1$ (D'_1 active):** Corresponds to minterms $\{1, 5, 9, 13\}$. Present in f : $\{13\}$.

$$I_1(A, B) = AB$$

- **For $C = 1, D = 0$ (D'_2 active):** Corresponds to minterms $\{2, 6, 10, 14\}$. Present in f : $\{2, 6, 10, 14\}$.

$$I_2(A, B) = \overline{A}\overline{B} + \overline{A}B + A\overline{B} + AB = 1 \quad (\text{Universal Coverage})$$

- **For $C = 1, D = 1$ (D'_3 active):** Corresponds to minterms $\{3, 7, 11, 15\}$. Present in f : $\{7, 11\}$.

$$I_3(A, B) = \overline{A}B + A\overline{B} = A \oplus B \quad (\text{XOR Definition})$$

4. Gate Minimization via De Morgan's Laws

Substitute the derived residual functions back into the expanded equation:

$$f(A, B, C, D) = \overline{D'_0} \cdot \overline{B} + \overline{D'_1} \cdot (AB) + \overline{D'_2} \cdot (1) + \overline{D'_3} \cdot (A \oplus B)$$

Because our decoder outputs are active-low, we use **De Morgan's Involution Law** ($\overline{\overline{f}} = f$) to shift the logic into an active-low/NAND framework, minimizing extra gates:

$$f(A, B, C, D) = \overline{\overline{D'_0} \cdot \overline{B} \cdot \overline{D'_1} \cdot (AB) \cdot \overline{D'_2} \cdot \overline{D'_3} \cdot (A \oplus B)}$$

Applying De Morgan's Law ($\overline{X \cdot Y} = \overline{X} + \overline{Y}$) term by term:

$$\overline{\overline{D'_0} \cdot \overline{B}} = D'_0 + B$$

$$\overline{\overline{D'_1} \cdot (AB)} = D'_1 + \overline{AB} = D'_1 + \text{NAND}(A, B)$$

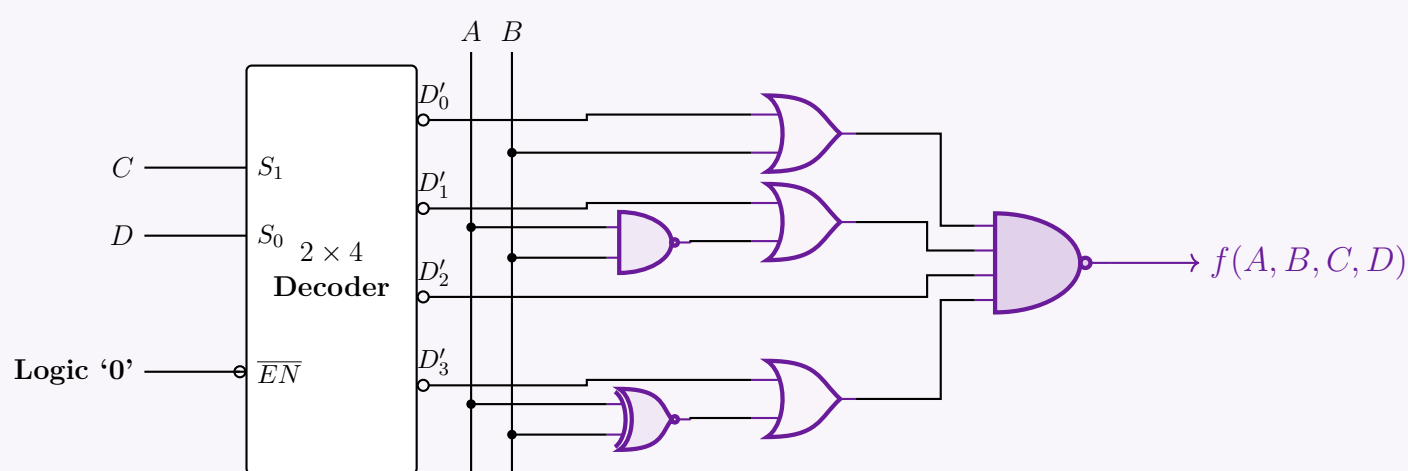
$$\overline{\overline{D'_2}} = D'_2$$

$$\overline{\overline{D'_3} \cdot (A \oplus B)} = D'_3 + \overline{A \oplus B} = D'_3 + \text{XNOR}(A, B)$$

Final optimized logic expression:

$$f(A, B, C, D) = \text{NAND}_4\left((D'_0 + B), (D'_1 + \text{NAND}(A, B)), D'_2, (D'_3 + \text{XNOR}(A, B))\right)$$

5. Logic Circuit Implementation



Q10. Consider a 3×8 decoder whose outputs are active low and which has one active high enable signal. Implement the following functions using the above-mentioned decoder:

(a) $f(a, b, c, d) = \Pi(3, 5, 14)$

(b) $f(a, b, c, d) = \Sigma(1, 6, 8, 13)$

Note that you will need to implement both functions in a single circuit. You may use as many decoders as required, keeping the implementation as efficient as possible. Basic gates are also allowed. **(13 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Answer

1. 4-to-16 Line Decoder Expansion

To implement 4-variable functions (a, b, c, d) , we must first construct a 4×16 decoder using two available 3×8 decoders. We partition the variables as follows:

- The three least significant bits (**LSBs**), b, c , and d , act as the common select inputs (S_2, S_1, S_0) for both decoders.
- The most significant bit (**MSB**), a , acts as the enable controller.
- Decoder 0** is enabled when $a = 0$ (using an inverter since the enable is active-high) and generates minterms m_0 to m_7 .
- Decoder 1** is enabled when $a = 1$ (connected directly) and generates minterms m_8 to m_{15} .

Because the decoders have **active-low outputs**, they will output $\overline{m_i}$ for the selected minterm index i , and Logic '1' for all other unselected lines.

2. Mathematical Derivation and Minimization

Function 1: $f_1(a, b, c, d) = \Pi(3, 5, 14)$

The function is provided in Product of Sums (POS) form as the product of maxterms M_i . By definition, a maxterm is the exact

complement of a minterm ($M_i = \overline{m_i}$).

$$\begin{aligned} f_1(a, b, c, d) &= M_3 \cdot M_5 \cdot M_{14} \\ &= \overline{m_3} \cdot \overline{m_5} \cdot \overline{m_{14}} \quad (\text{Definition of Maxterms}) \end{aligned}$$

Since the decoder inherently produces the complemented minterms ($\overline{m_i}$) on its active-low output pins (D'_3 from Dec 0, D'_5 from Dec 0, and D'_6 from Dec 1, which corresponds to overall index 14), we simply need to **AND** these active-low signals together.

$$f_1(a, b, c, d) = \text{AND}(\overline{m_3}, \overline{m_5}, \overline{m_{14}})$$

Function 2: $f_2(a, b, c, d) = \Sigma(1, 6, 8, 13)$

The function is given in Sum of Products (SOP) form.

$$f_2(a, b, c, d) = m_1 + m_6 + m_8 + m_{13}$$

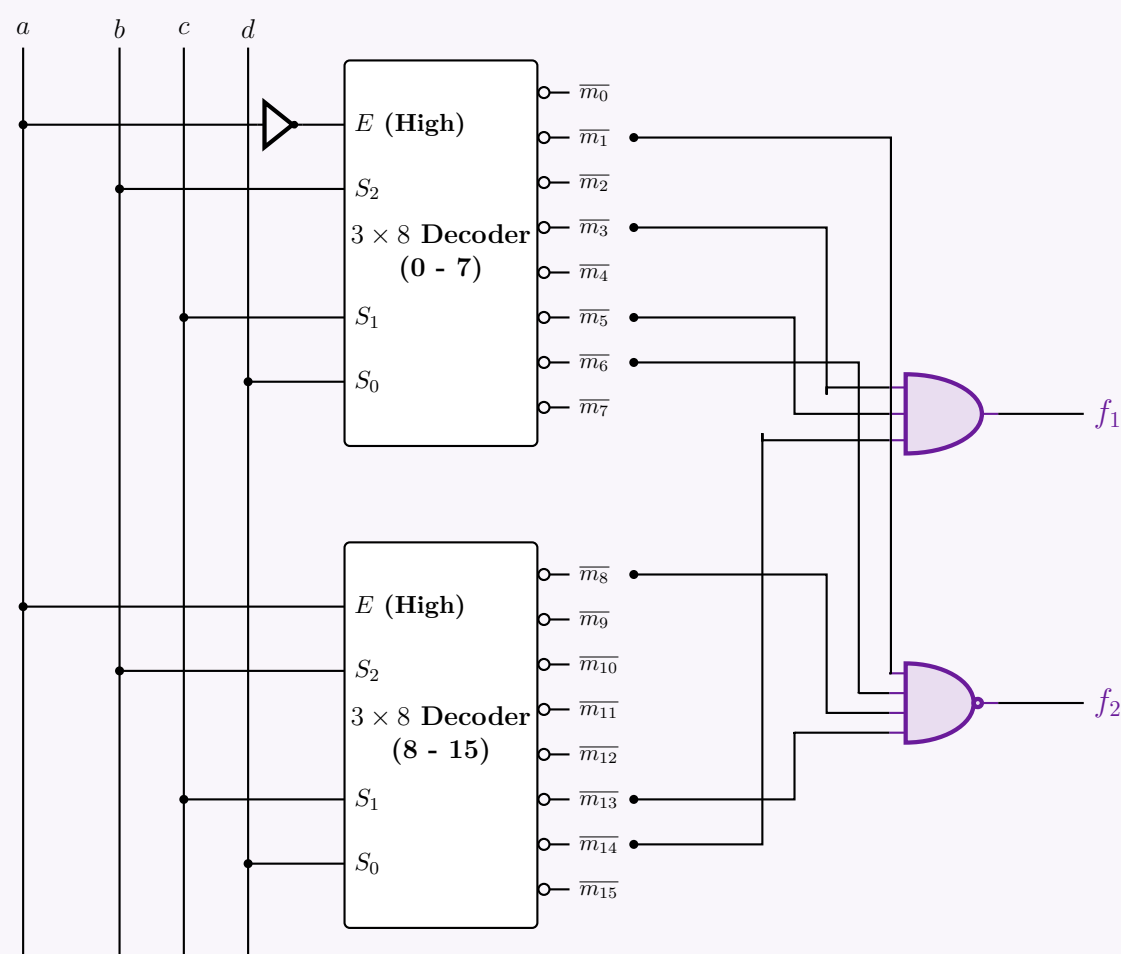
To map this to our active-low decoder outputs ($\overline{m_i}$), we apply **De Morgan's Involution Law** ($\overline{\overline{f}} = f$) followed by De Morgan's Law for addition ($\overline{X + Y} = \overline{X} \cdot \overline{Y}$):

$$\begin{aligned} f_2(a, b, c, d) &= \overline{\overline{m_1 + m_6 + m_8 + m_{13}}} \\ &= \overline{\overline{m_1} \cdot \overline{m_6} \cdot \overline{m_8} \cdot \overline{m_{13}}} \quad (\text{By De Morgan's Law}) \end{aligned}$$

This translates directly into a **NAND** operation applied to the active-low decoder outputs.

$$f_2(a, b, c, d) = \text{NAND}(\overline{m_1}, \overline{m_6}, \overline{m_8}, \overline{m_{13}})$$

3. Integrated Circuit Implementation



Q11. Design a full adder with a decoder and basic logic gates. (12 marks)

[CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Problem Analysis and Truth Table

A Full Adder is a combinational arithmetic circuit that adds three single-bit numbers: A , B , and a carry-in C_{in} . It produces two outputs: a Sum (S) and a Carry-out (C_{out}).

To design this using a decoder, we first construct the truth table to determine the active minterms for each output.

Inputs			Outputs		Minterm (m_i)
A	B	C_{in}	S (Sum)	C_{out} (Carry)	
0	0	0	0	0	m_0
0	0	1	1	0	m_1
0	1	0	1	0	m_2
0	1	1	0	1	m_3
1	0	0	1	0	m_4
1	0	1	0	1	m_5
1	1	0	0	1	m_6
1	1	1	1	1	m_7

2. Boolean Expressions in Minterm Form

From the truth table, we extract the standard Sum of Products (SOP) form for both outputs by identifying the rows where the output is logic ‘1’.

$$S(A, B, C_{in}) = \sum m(1, 2, 4, 7)$$

$$C_{out}(A, B, C_{in}) = \sum m(3, 5, 6, 7)$$

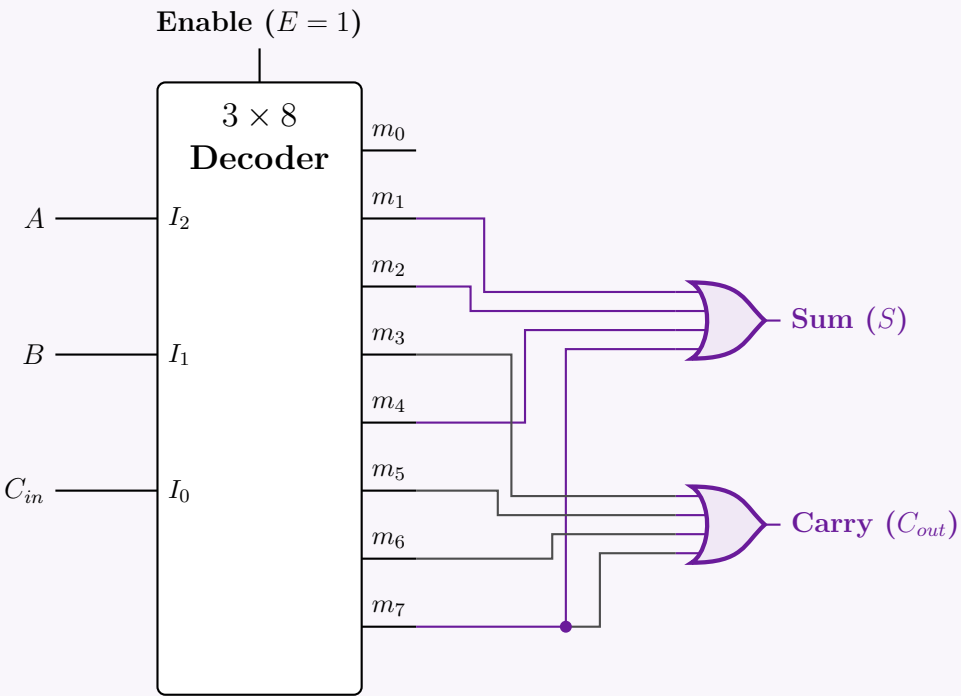
3. Decoder Implementation Strategy

A 3-to-8 line decoder activates exactly one of its 8 output lines (representing minterms m_0 to m_7) based on the 3-bit binary input. By connecting the inputs A, B, C_{in} to the select lines of the decoder, the decoder automatically generates all 8 minterms. To implement the Full Adder logic:

- We use a **4-input OR gate** to logically sum the activated minterms m_1, m_2, m_4 , and m_7 to generate the **Sum (S)**.
- We use a second **4-input OR gate** to sum the activated minterms m_3, m_5, m_6 , and m_7 to generate the **Carry (C_{out})**.

4. Logic Circuit Diagram

Below is the complete schematic utilizing a 3-to-8 decoder and standard OR gates.



Q12. You need a full adder for an experiment but in the lab you could only find plenty of 3×8 decoders. Can any such decoder be used as a full adder? Explain your answer with a truth table. **(15 marks)** [CSE 205 | L-2/T-1 | 2012–13]

Answer

1. Direct Answer and Theoretical Justification

Yes, a 3×8 decoder can be used to implement a full adder, provided that we also have access to basic logic gates (specifically, OR gates) to combine the outputs.

Reasoning: An $n \times 2^n$ decoder functions as a universal minterm generator for n input variables. Since a full adder is a combinational circuit with 3 inputs (A, B, C_{in}), a 3×8 decoder will generate all $2^3 = 8$ possible minterms (m_0 to m_7). Any Boolean function of 3 variables can be expressed in standard Sum of Products (SOP) form. Therefore, by logically ORing the specific minterms that

produce a logic ‘1’ for the Sum and Carry outputs, we can fully realize the full adder circuit.

2. Truth Table Analysis

To determine which minterms to combine, we construct the truth table for a full adder. It adds two significant bits (A, B) and a carry-in bit (C_{in}), producing a Sum (S) and a Carry-out (C_{out}).

Inputs			Outputs		Minterm Designation
A	B	C _{in}	Sum (S)	Carry (C _{out})	
0	0	0	0	0	m ₀
0	0	1	1	0	m ₁
0	1	0	1	0	m ₂
0	1	1	0	1	m ₃
1	0	0	1	0	m ₄
1	0	1	0	1	m ₅
1	1	0	0	1	m ₆
1	1	1	1	1	m ₇

3. Boolean Minterm Equations

From the truth table, we extract the Canonical Sum of Products (SOP) form by identifying the rows where the respective outputs evaluate to logic ‘1’.

Sum Output (S):

$$S(A, B, C_{in}) = \sum m(1, 2, 4, 7)$$
$$S = m_1 + m_2 + m_4 + m_7$$

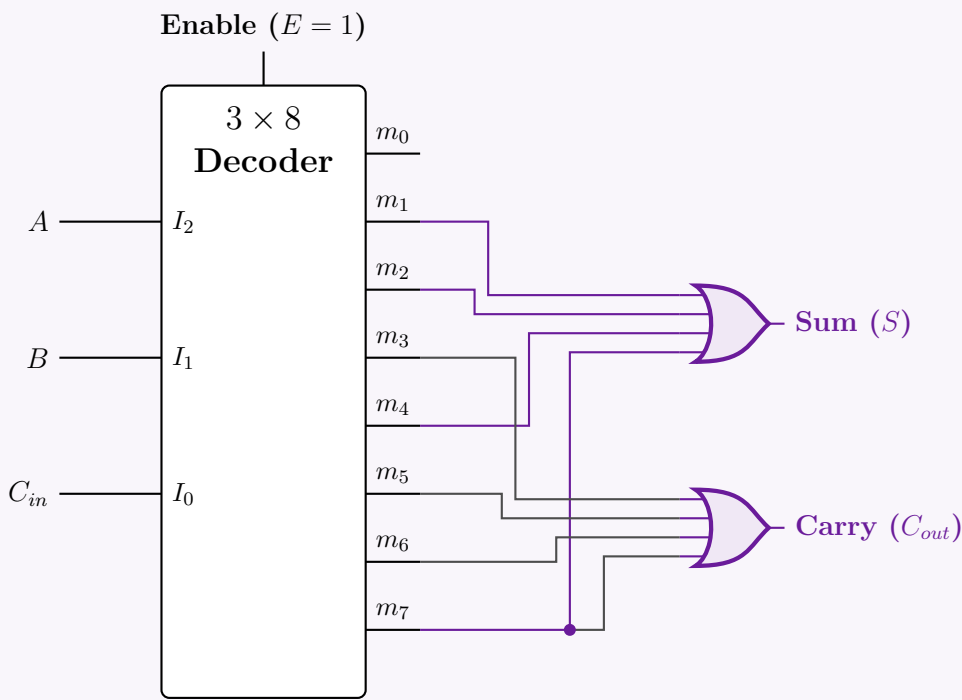
Carry Output (C_{out}):

$$C_{out}(A, B, C_{in}) = \sum m(3, 5, 6, 7)$$
$$C_{out} = m_3 + m_5 + m_6 + m_7$$

4. Logic Circuit Implementation

To implement the circuit:

- (a) Connect the inputs A , B , and C_{in} to the select lines I_2 , I_1 , and I_0 of the 3×8 decoder.
- (b) Assume the decoder has an active-high enable ($E = 1$) and active-high outputs.
- (c) Route the output pins corresponding to m_1, m_2, m_4 , and m_7 into a 4-input OR gate to generate the **Sum** (S).
- (d) Route the output pins corresponding to m_3, m_5, m_6 , and m_7 into a second 4-input OR gate to generate the **Carry** (C_{out}).



Note: If the 3×8 decoder in the lab features active-low outputs (such as the standard 74LS138), NAND gates must be used in place of the OR gates to properly assert the Sum and Carry logic levels according to De Morgan’s Laws.

Answer

1. Design Principle

A 3×8 decoder takes 3 input lines (A_2, A_1, A_0) and activates exactly one of its 8 output lines (Y_0 to Y_7) based on the binary value of the inputs. We can construct this by cascading two 2×4 decoders (each possessing an Enable pin, E) using the following logic partition:

- Common Inputs (LSBs):** The two least significant bits, A_1 and A_0 , are connected in parallel to the selection inputs of both 2×4 decoders. They dictate which of the four outputs within an active decoder is asserted.
- Selection Input (MSB):** The most significant bit, A_2 , acts as a block selector. We connect A_2 directly to the enable pin of the second decoder and route it through a NOT gate to the enable pin of the first decoder.
- Operation:**
 - When $A_2 = 0$, the **Top Decoder** is enabled. It handles input combinations 000_2 to 011_2 , asserting outputs Y_0 through Y_3 .
 - When $A_2 = 1$, the **Bottom Decoder** is enabled. It handles input combinations 100_2 to 111_2 , asserting outputs Y_4 through Y_7 .

2. Truth Table Partitioning

The truth table clearly demonstrates how the MSB (A_2) partitions the outputs into two mutually exclusive sets, mapping perfectly to the two 2×4 decoders.

Inputs			Outputs								Active Hardware Block
A_2	A_1	A_0	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7	
0	0	0	1	0	0	0	0	0	0	0	Decoder 0 ($E = \overline{A_2}$)
0	0	1	0	1	0	0	0	0	0	0	
0	1	0	0	0	1	0	0	0	0	0	
0	1	1	0	0	0	1	0	0	0	0	
1	0	0	0	0	0	0	1	0	0	0	Decoder 1 ($E = A_2$)
1	0	1	0	0	0	0	0	1	0	0	
1	1	0	0	0	0	0	0	0	1	0	
1	1	1	0	0	0	0	0	0	0	1	

3. Circuit Diagram

Assumption: The 2×4 decoders utilize active-high Enable (E) pins and active-high outputs.

The diagram illustrates the hardware implementation of a 3-to-8 decoder using two 2-to-4 decoders. The MSB, A_2 , serves as the block selector. It is connected to the enable pin (E) of Decoder 1 and, through an inverter, to the enable pin of Decoder 0. The two least significant bits, A_1 and A_0 , are connected to the selection inputs (I_1 and I_0) of both decoders. Decoder 0, enabled when $A_2 = 0$, produces outputs Y_0 through Y_3 . Decoder 1, enabled when $A_2 = 1$, produces outputs Y_4 through Y_7 .

Decoder as Demultiplexer

Q1. What is a demultiplexer? Give an example. (8 marks)

[CSE 205 | L-2/T-1 | 2021–22]

Answer

1. Definition of a Demultiplexer

A **Demultiplexer** (often abbreviated as **DeMUX**) is a combinational logic circuit that performs the reverse operation of a multiplexer. It takes a single input data line and routes it to one of 2^n possible output lines.

The specific output line that receives the input data is determined by the binary value present on the n select (or control) lines. Because it distributes a single input across multiple outputs, a demultiplexer is frequently referred to as a **data distributor**.

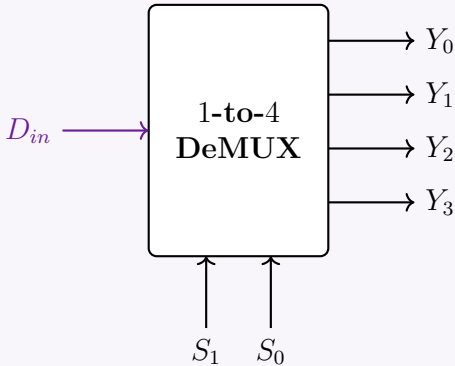
Key Characteristics:

- **Data Inputs:** 1
- **Select Lines:** n
- **Data Outputs:** 2^n

2. Example: A 1-to-4 Line Demultiplexer

To illustrate, consider a 1-to-4 demultiplexer. It features 1 data input (D_{in}), $n = 2$ select lines (S_1, S_0), and $2^2 = 4$ outputs (Y_0, Y_1, Y_2, Y_3).

Block Diagram



Truth Table

The output that mirrors the input D_{in} is determined by the binary combination of S_1 and S_0 . All other unselected outputs are held at logic 0.

Select Lines		Outputs			
S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	D_{in}
0	1	0	0	D_{in}	0
1	0	0	D_{in}	0	0
1	1	D_{in}	0	0	0

Boolean Expressions

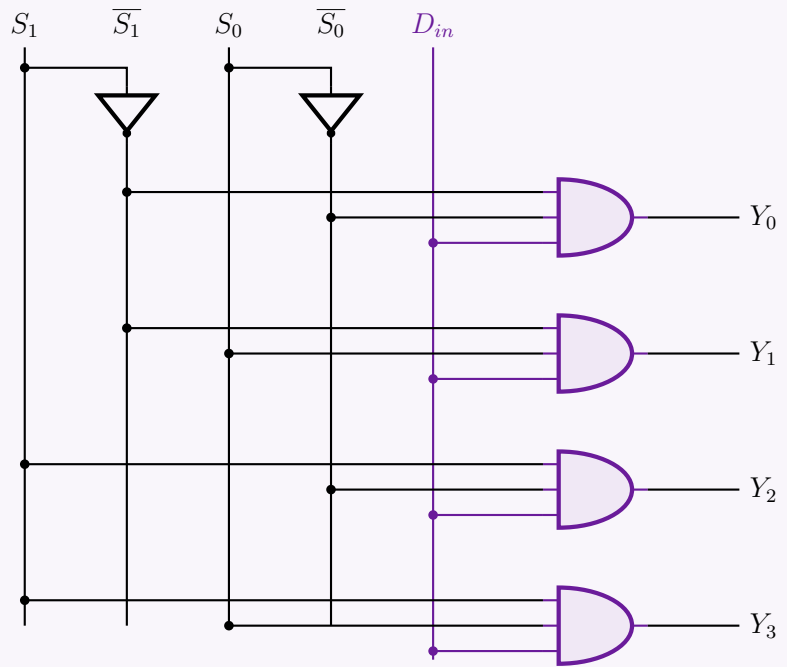
From the truth table, we can directly extract the Sum-of-Products (SOP) expressions for each output line:

$$Y_0 = \overline{S_1} \cdot \overline{S_0} \cdot D_{in}$$
$$Y_1 = \overline{S_1} \cdot S_0 \cdot D_{in}$$
$$Y_2 = S_1 \cdot \overline{S_0} \cdot D_{in}$$
$$Y_3 = S_1 \cdot S_0 \cdot D_{in}$$

Logic Circuit Implementation

Using basic logic gates (NOT gates for the select lines and 3-input AND gates for the outputs), the 1-to-4 demultiplexer is implemented as follows:

171



Q2. Can a decoder act like a demultiplexer? If yes, explain how. (5 marks)

[CSE 205 | L-2/T-1 | 2020–21]

Answer

1. Direct Answer

Yes, a decoder can act as a demultiplexer. Specifically, any $N \times 2^N$ line decoder equipped with an **Enable** (EN) input can be directly used as a 1-to- 2^N line demultiplexer without requiring any additional logic gates or structural modifications.

2. Conceptual Explanation & Pin Mapping

To understand how this conversion works, we must compare the functional definitions of both circuits:

- A **Decoder** activates one out of 2^N output lines based on the binary combination applied to its N input lines, provided the chip is enabled.
- A **Demultiplexer (DeMUX)** routes a single data input to one of 2^N output lines based on the binary combination applied to its N select lines.

By cleverly reassigning the roles of the decoder’s pins, it functions perfectly as a demultiplexer:

Decoder Enable Input (EN) $\implies$ Demux Data Input (D_{in})

Decoder Input Lines ($A_{n-1} \dots A_0$) $\implies$ Demux Select Lines ($S_{n-1} \dots S_0$)

Decoder Output Lines ($Y_{2^n-1} \dots Y_0$) $\implies$ Demux Output Lines ($Y_{2^n-1} \dots Y_0$)

Mechanism of Action: If the Enable pin (EN) is used as the data input (D_{in}):

(a) When $D_{in} = 0$: The decoder is disabled. All outputs are forced to logic ‘0’, regardless of the select line inputs. The ‘0’ from the data input has been successfully routed to the output.

(b) When $D_{in} = 1$: The decoder is enabled. The single output specified by the select lines goes to logic ‘1’ (while all others remain ‘0’). The ‘1’ from the data input has been successfully routed to the selected output.

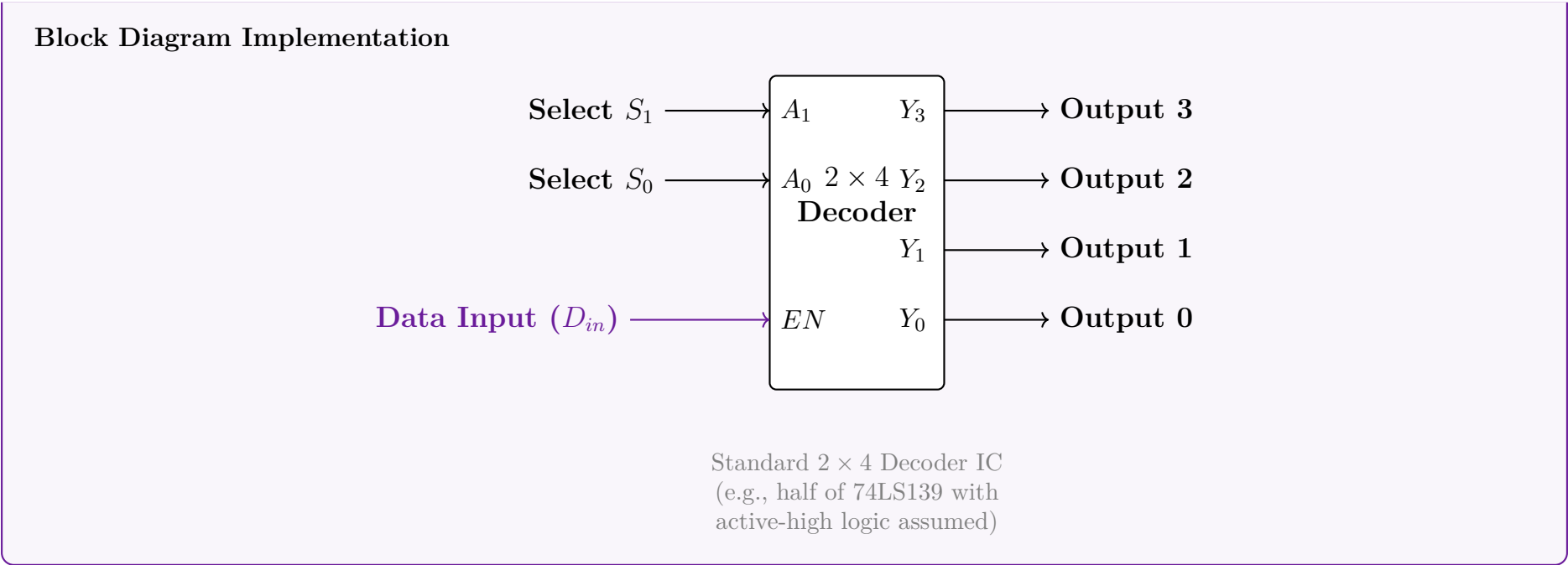
In both cases, the value of D_{in} is accurately transferred to the selected output channel.

3. Example: 2×4 Decoder as a 1-to-4 Demultiplexer

Truth Table Verification

The following truth table demonstrates a 2×4 active-high decoder operating as a Demultiplexer. Notice how the targeted output perfectly mirrors the state of D_{in} .

Data Input $EN (D_{in})$	Select Lines $A_1 (S_1) \quad A_0 (S_0)$		Outputs $Y_3 \quad Y_2 \quad Y_1 \quad Y_0$			
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0



Q3. Show that a decoder can be used as a demultiplexer. (8 marks)

[CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Theoretical Equivalence

A decoder with an **Enable** input is functionally and structurally identical to a demultiplexer. We can show this by directly mapping the terminals of an $N \times 2^N$ decoder to a 1-to- 2^N demultiplexer:

- **Data Input Mapping:** The **Enable** (E) pin of the decoder is used as the single **Data Input** (D_{in}) of the demultiplexer.
- **Select Line Mapping:** The N **Input lines** ($A_{N-1}, \dots, A_0$) of the decoder are used as the N **Select lines** ($S_{N-1}, \dots, S_0$) of the demultiplexer.
- **Output Mapping:** The 2^N **Output lines** ($Y_{2^N-1}, \dots, Y_0$) serve as the data outputs for both circuits.

When $E = 0$, the decoder is disabled, forcing all outputs to 0. This corresponds to routing a data bit of ‘0’ to the output. When $E = 1$, the decoder is enabled, and the specific output addressed by the input lines goes to 1. This corresponds to routing a data bit of ‘1’ to the selected output.

2. Mathematical and Truth Table Proof

Let us examine a 2-to-4 line decoder with an active-high Enable pin. The Boolean expressions for its outputs are:

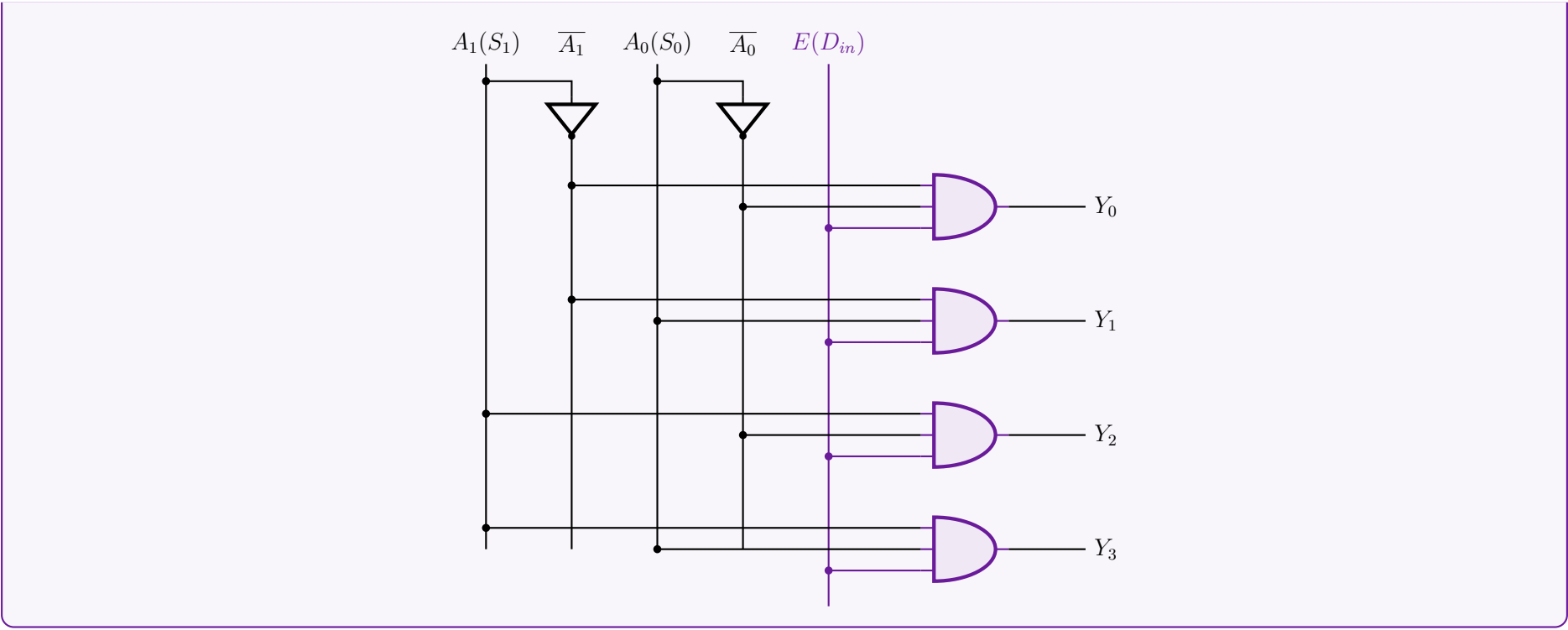
$$\begin{aligned} Y_0 &= E \cdot \overline{A_1} \cdot \overline{A_0} \\ Y_1 &= E \cdot \overline{A_1} \cdot A_0 \\ Y_2 &= E \cdot A_1 \cdot \overline{A_0} \\ Y_3 &= E \cdot A_1 \cdot A_0 \end{aligned}$$

If we substitute $E \rightarrow D_{in}$, $A_1 \rightarrow S_1$, and $A_0 \rightarrow S_0$, we get the exact Boolean expressions of a 1-to-4 demultiplexer. The truth table below confirms this dual functionality:

Enable / Data In E (D_{in})	Inputs / Select A_1 (S_1) A_0 (S_0)		Outputs Y_3 Y_2 Y_1 Y_0				Operation Description
0	0	0	0	0	0	0	$D_{in} = 0$ routed to Y_0
1	0	0	0	0	0	1	$D_{in} = 1$ routed to Y_0
0	0	1	0	0	0	0	$D_{in} = 0$ routed to Y_1
1	0	1	0	0	1	0	$D_{in} = 1$ routed to Y_1
0	1	0	0	0	0	0	$D_{in} = 0$ routed to Y_2
1	1	0	0	1	0	0	$D_{in} = 1$ routed to Y_2
0	1	1	0	0	0	0	$D_{in} = 0$ routed to Y_3
1	1	1	1	0	0	0	$D_{in} = 1$ routed to Y_3

3. Logic Circuit Demonstration

The underlying gate-level architecture perfectly illustrates how the Enable pin distributes its signal to the selected logic pathway.



Q4. How can a decoder be used as a demultiplexer? Explain with an example. (5 marks) [CSE 205 | L-2/T-1 | 2016–17]

Answer

1. The Core Principle

A standard $n \times 2^n$ line decoder can be used directly as a 1-to- 2^n line demultiplexer simply by utilizing its **Enable** (E) input as the **Data Input** (D_{in}). No internal structural changes or extra logic gates are required.

To achieve this, the following pin mapping is established:

- **Enable Pin** (E): Acts as the **Data Input** (D_{in}). The signal applied here is the data to be distributed.
- **Decoder Inputs** ($A_{n-1} \dots A_0$): Act as the **Select Lines** ($S_{n-1} \dots S_0$). They determine which output line will receive the data.
- **Decoder Outputs** ($Y_{2^n-1} \dots Y_0$): Serve directly as the **Demultiplexer Outputs**.

How it works: When the decoder is disabled (e.g., $E = 0$ for active-high enable), all outputs are forced to logic ‘0’, successfully transferring the ‘0’ from the input to the output. When the decoder is enabled ($E = 1$), the specific output addressed by the input lines goes to logic ‘1’, while all other lines remain ‘0’. Thus, the logic ‘1’ is successfully transferred to the selected output line.

2. Example: 2 × 4 Decoder as a 1-to-4 Demultiplexer

Let us configure a standard 2 × 4 decoder with an active-high Enable pin to function as a 1-to-4 demultiplexer.

Mathematical Verification

The Boolean equations for a 2 × 4 active-high decoder are:

$$\begin{aligned} Y_0 &= E \cdot \overline{A_1} \cdot \overline{A_0} \\ Y_1 &= E \cdot \overline{A_1} \cdot A_0 \\ Y_2 &= E \cdot A_1 \cdot \overline{A_0} \\ Y_3 &= E \cdot A_1 \cdot A_0 \end{aligned}$$

By substituting the variables $E \rightarrow D_{in}$, $A_1 \rightarrow S_1$, and $A_0 \rightarrow S_0$, the equations perfectly match the standard Sum-of-Products (SOP) expressions of a 1-to-4 demultiplexer:

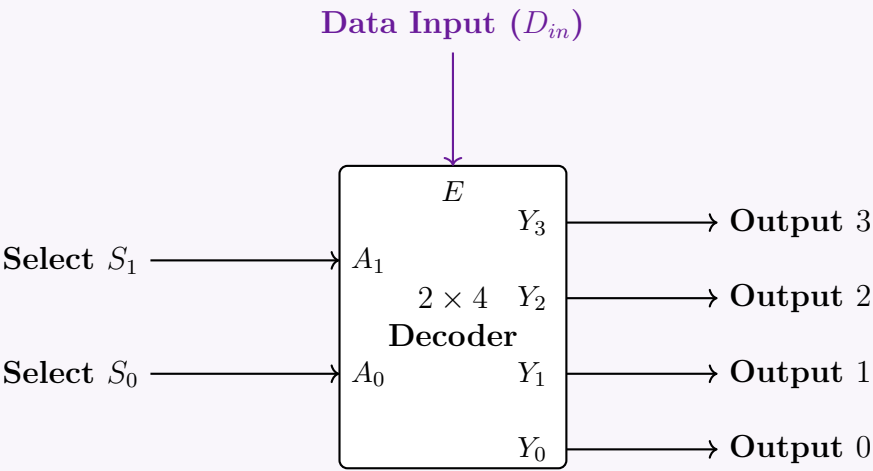
$$\begin{aligned} Y_0 &= D_{in} \cdot \overline{S_1} \cdot \overline{S_0} \\ Y_3 &= D_{in} \cdot S_1 \cdot S_0 \quad (\text{and similarly for } Y_1, Y_2) \end{aligned}$$

Truth Table

The following truth table demonstrates that the selected output perfectly mirrors the state of the Enable / Data Input pin.

Enable / Data Input $E (D_{in})$	Decoder Inputs / Select		Outputs			
	$A_1 (S_1)$	$A_0 (S_0)$	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0
1	0	0	0	0	0	1
0	0	1	0	0	0	0
1	0	1	0	0	1	0
0	1	0	0	0	0	0
1	1	0	0	1	0	0
0	1	1	0	0	0	0
1	1	1	1	0	0	0

Logic Block Diagram Configuration



BCD to 7-Segment Decoder & BCD Error Detector

Q1. Explain how the Ripple-blanking input (RBI) and Ripple-blanking output (RBO) pins of the BCD to 7-segment decoder (IC 7447) should be connected such that all but the LSB show blank for leading zeros. **(6 marks)** [CSE 205 | L-2/T-1 | 2012–13]

Answer

1. Concept of Ripple Blanking in IC 7447

The IC 7447 is a BCD to 7-segment decoder/driver that includes built-in logic for zero suppression (blanking leading or trailing zeros). This is managed by two active-low pins:

- $\overline{RBI}$ (Ripple Blanking Input):** If $\overline{RBI}$ is LOW (Logic ‘0’) and the BCD input is 0000, the display is completely blanked (all segments OFF), and the IC forces its $\overline{RBO}$ pin LOW.
- $\overline{RBO}$ (Ripple Blanking Output):** Outputs a LOW (Logic ‘0’) only if the display is blanked (i.e., $\overline{RBI}$ is LOW and the BCD input is 0000). Otherwise, it outputs a HIGH (Logic ‘1’).

2. Connection Strategy for Leading Zero Suppression

To suppress leading zeros in a multi-digit display (e.g., displaying ‘0007’ as ‘7’), while ensuring that an absolute zero value (‘0000’) displays as ‘0’ (the LSB must not be blanked), the pins must be connected in a cascaded “ripple” configuration:

(a) **Most Significant Digit (MSD):** The $\overline{RBI}$ pin of the MSD is hardwired to **GND (Logic ‘0’)**. If the MSD BCD input is zero, the digit is blanked, and its $\overline{RBO}$ becomes ‘0’. If the input is non-zero, it displays the digit, and its $\overline{RBO}$ becomes ‘1’.

(b) **Intermediate Digits:** The $\overline{RBI}$ of each subsequent digit is connected directly to the $\overline{RBO}$ of the next most significant digit.

- If the previous higher-order digit was blanked, it passes a ‘0’ to the current digit, allowing it to blank if its BCD input is also zero.
- If any higher-order digit was non-zero, it passes a ‘1’ down the chain, forcing all subsequent digits to display normally (e.g., ‘0105’ becomes ‘105’, preserving the inner zero).

(c) **Least Significant Digit (LSB):** The $\overline{RBI}$ pin of the LSB is hardwired to **V_{cc} (Logic ‘1’)**. This guarantees that the LSB will never be blanked, ensuring that a completely zero value will display a single ‘0’. The $\overline{RBO}$ of the previous digit is left unconnected (NC).

3. Block Diagram of a 4-Digit Implementation

The following logic circuit illustrates the connections for a 4-digit display system:

4. Truth Table Verification (Cascading Logic)

Consider the behavior of the chain for the number **0030**:

Stage	BCD Input	$\overline{RBI}$ (In)	$\overline{RBO}$ (Out)	Display Status
Digit 3 (MSD)	0000	0 (GND)	0	Blanked
Digit 2	0000	0 (from MSD)	0	Blanked
Digit 1	0011	0 (from Dig 2)	1	Displays '3' (forces $\overline{RBO}$ High)
Digit 0 (LSB)	0000	1 (Vcc)	1	Displays '0' (never blanks)

Q2. You are receiving BCD codes sent by a remote transmitter. The bits are $A_3A_2A_1A_0$ (A_0 being the LSB). Design a BCD-error-detector (BED) for your receiver that will produce a HIGH for any received code that is not a valid BCD code. (For example, if you receive 1010, the output of BED will be HIGH.) **(15 marks)** [CSE 205 | L-2/T-1 | 2012–13]

Answer

1. Problem Analysis and Truth Table

A valid Binary-Coded Decimal (BCD) digit is represented by a 4-bit sequence ranging from 0000_2 (0) to 1001_2 (9). Any 4-bit sequence from 1010_2 (10) to 1111_2 (15) is considered an invalid BCD code.

Let the 4-bit input be $A_3A_2A_1A_0$, where A_3 is the Most Significant Bit (MSB) and A_0 is the Least Significant Bit (LSB). We need to design a BCD Error Detector (BED) whose output E is HIGH (Logic '1') for invalid BCD codes and LOW (Logic '0') for valid BCD codes.

Input Bits				Output	Condition
A_3	A_2	A_1	A_0	E	
0	0	0	0	0	Valid (0)
0	0	0	1	0	Valid (1)
0	0	1	0	0	Valid (2)
0	0	1	1	0	Valid (3)
0	1	0	0	0	Valid (4)
0	1	0	1	0	Valid (5)
0	1	1	0	0	Valid (6)
0	1	1	1	0	Valid (7)
1	0	0	0	0	Valid (8)
1	0	0	1	0	Valid (9)
1	0	1	0	1	Invalid (10)
1	0	1	1	1	Invalid (11)
1	1	0	0	1	Invalid (12)
1	1	0	1	1	Invalid (13)
1	1	1	0	1	Invalid (14)
1	1	1	1	1	Invalid (15)

2. Karnaugh Map Minimization

We can extract the minterms for the error condition from the truth table:

$$E(A_3, A_2, A_1, A_0) = \sum m(10, 11, 12, 13, 14, 15)$$

We plot these minterms on a 4-variable Karnaugh Map to find the minimized Sum of Products (SOP) expression.

		A_1A_0			
		00	01	11	10
A_3A_2	00	0	0	0	0
	01	0	0	0	0
	11	1	1	1	1
	10	0	0	1	1

Group 1: A_3A_2 (Minterms 12, 13, 14, 15)

Group 2: A_3A_1 (Minterms 10, 11, 14, 15)

3. Logic Circuit Implementation

The optimized expression requires only a single 2-input OR gate and a single 2-input AND gate.

A_0 (Unused)

Multiplexers & Demultiplexers

Q1. There is a 4×4 smart grid to serve as the electricity transmission facility. The grids will be activated based on the logic function $g(a, b, c, d) = \Sigma(1, 2, 5, 7, 12, 14)$. Design the logic circuit using a switching circuit that has 3-bit selector bits S_2, S_1, S_0 to select any of the eight input lines. The switching circuit is activated by two Enables — one active low $\overline{EN}_0$ and another active high EN_1 . (Design with the minimum number of extra logic gates if required.) **(13 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Answer

1. Problem Analysis and Mux Configuration

The problem requires us to design a logic circuit for the function $g(a, b, c, d) = \Sigma(1, 2, 5, 7, 12, 14)$ using a switching circuit with 3 select bits. A switching circuit with 3 select lines is an **8-to-1 Multiplexer (MUX)**. Since there are 4 input variables (a, b, c, d) and the multiplexer has 3 select lines (S_2, S_1, S_0) , we will use the **variable mapping method**. We map the three most significant variables to the select lines:

$$\begin{aligned} S_2 &= a \\ S_1 &= b \\ S_0 &= c \end{aligned}$$

The remaining variable, d , (along with its complement $\bar{d}$, Logic 0, and Logic 1) will be applied to the data input lines D_0 through D_7 . The MUX features two enable pins which control its active state:

177

- $\overline{EN_0}$ is an **active-low** enable. It must be connected to **Logic 0 (Ground)** to be activated.
- EN_1 is an **active-high** enable. It must be connected to **Logic 1 (V_{cc})** to be activated.

2. Multiplexer Implementation Table

To determine the inputs for $D_0 \dots D_7$, we construct an implementation table. The 16 minterms are distributed across the 8 data lines based on the decimal equivalent of the select bits a, b, c .

- The top row contains minterms where $d = 0$ (Even minterms).
- The bottom row contains minterms where $d = 1$ (Odd minterms).

We circle/highlight the minterms present in the given function $\Sigma(1, 2, 5, 7, 12, 14)$ and deduce the data line input.

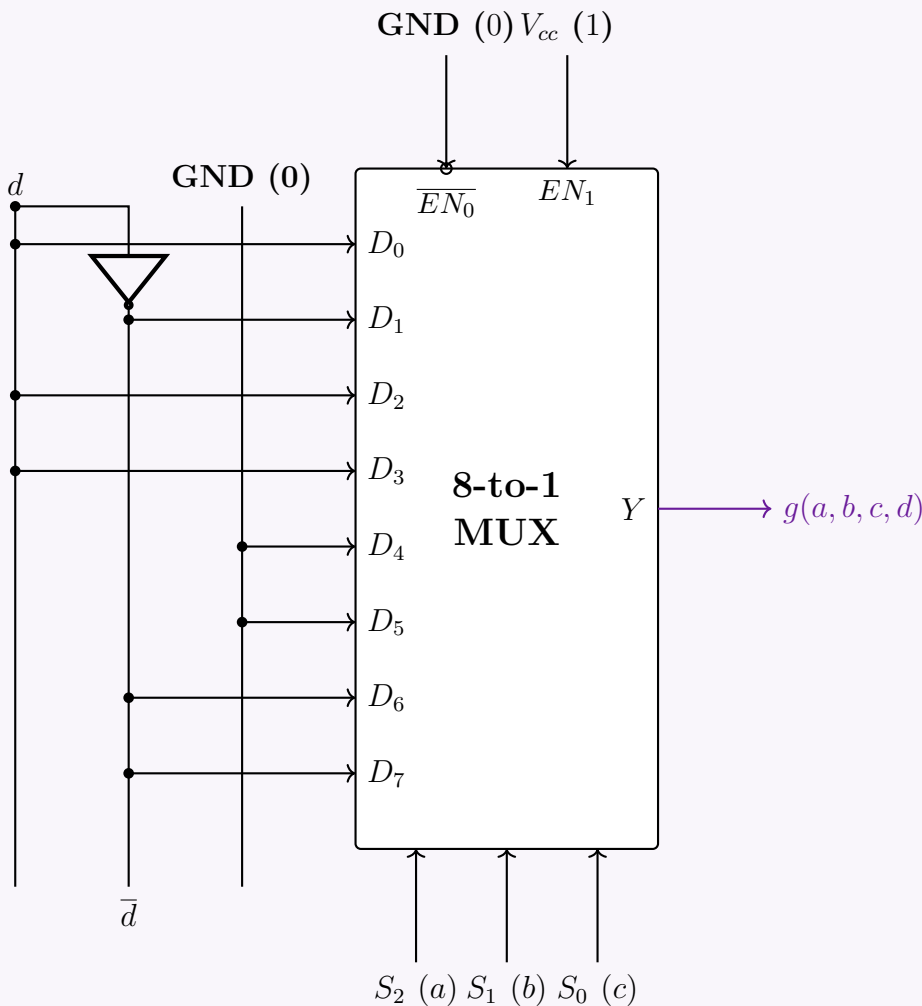
Variable	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
$\bar{d} \ (d = 0)$	0	2	4	6	8	10	12	14
$d \ (d = 1)$	1	3	5	7	9	11	13	15
MUX Input	d	$\bar{d}$	d	d	0	0	$\bar{d}$	$\bar{d}$

Derivation rules applied:

- For D_0, D_2, D_3 : Only the minterm corresponding to $d = 1$ is present. Input is d .
- For D_1, D_6, D_7 : Only the minterm corresponding to $d = 0$ is present. Input is $\bar{d}$.
- For D_4, D_5 : Neither minterm is present. Input is Logic 0.

3. Circuit Diagram

The final 8-to-1 multiplexer circuit is implemented below. Only one NOT gate is required as the extra logic to generate $\bar{d}$.



Q2. Using the switching circuit defined above (5-bit selector lines A, B, C, D, E where A is MSB and E is LSB), design a 32×1 multiplexer. (Design with the minimum number of logic gates if required.) **(10 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Answer

1. System Architecture

A 32×1 Multiplexer requires 32 data inputs, 5 select lines (A, B, C, D, E), and 1 output. Based on the provided switching circuit (an 8-to-1 MUX with 3 select lines and two enable pins: active-low $\overline{EN}_0$ and active-high EN_1), we can cascade **four 8-to-1 MUXes** to construct the 32×1 MUX.

- Lower Order Select Lines (C, D, E):** These represent the 3 Least Significant Bits (LSBs) and are connected directly to the selector bits S_2, S_1, S_0 of all four MUXes simultaneously. They choose 1 out of 8 inputs within an enabled MUX.
- Higher Order Select Lines (A, B):** These represent the 2 Most Significant Bits (MSBs) and act as chip-select signals. They ensure that exactly one of the four MUXes is activated at any given time.
- Output Stage:** Since only one MUX is active at a time (and disabled MUXes output Logic ‘0’), their outputs can be combined using a single **4-input OR gate** to produce the final 32×1 MUX output.

2. Enable Logic Design (Minimizing Gate Count)

Instead of using a separate 2-to-4 decoder for the MSBs, we can cleverly apply A, B , and their complements directly to the built-in $\overline{EN}_0$ and EN_1 pins of the MUXes. A MUX is active only when $\overline{EN}_0 = 0$ **AND** $EN_1 = 1$.

MUX Block	Data Range	A	B	$\overline{EN}_0$ Connection	EN_1 Connection	Activation Logic Check
MUX 0	$D_0 - D_7$	0	0	A (which is 0)	$\overline{B}$ (which is 1)	Active when $A = 0, B = 0$
MUX 1	$D_8 - D_{15}$	0	1	A (which is 0)	B (which is 1)	Active when $A = 0, B = 1$
MUX 2	$D_{16} - D_{23}$	1	0	B (which is 0)	A (which is 1)	Active when $A = 1, B = 0$
MUX 3	$D_{24} - D_{31}$	1	1	$\overline{A}$ (which is 0)	B (which is 1)	Active when $A = 1, B = 1$

By utilizing the internal enables, the extra decoding logic is reduced to strictly **two NOT gates** (to generate $\overline{A}$ and $\overline{B}$).

3. Circuit Diagram

The diagram illustrates a circuit with four 8-to-1 multiplexers (MUX 0, MUX 1, MUX 2, MUX 3) arranged vertically. Each MUX has an 8-bit data input (D₀₋₇, D₈₋₁₅, D₁₆₋₂₃, D₂₄₋₃₁), two 3-bit select inputs (C, D, E), and two enable inputs (EN₀, EN₁). The outputs of MUX 0, MUX 1, and MUX 2 are connected to a 3-input OR gate, which produces the final output Y. The enable inputs are connected such that only one MUX is active at a time based on the select inputs C, D, and E.

Q3. Design a full adder using NOT gates and two 4 to 1 multiplexers. (12 marks)

[CSE 205 | L-2/T-1 | 2023–24]

Answer

1. Problem Analysis and Truth Table

A Full Adder adds three 1-bit binary numbers (A , B , and C_{in}) and outputs a Sum (S) and a Carry (C_{out}). To implement a 3-variable Boolean function using a 4-to-1 multiplexer (which has 2 select lines), we map the two most significant variables (A and B) to the select lines S_1 and S_0 . The remaining variable (C_{in}) and constants (0, 1) will be applied to the data input lines (I_0, I_1, I_2, I_3).

The truth table for the Full Adder is grouped by the combinations of A and B to determine the relationship between C_{in} and the outputs.

Select Lines		Input	Outputs		MUX Input Evaluation	
$A (S_1)$	$B (S_0)$	C_{in}	$Sum (S)$	$Carry (C_{out})$	Sum MUX Input	C_{out} MUX Input
0	0	0	0	0	$I_0 = C_{in}$	$I_0 = 0$
0	0	1	1	0		
0	1	0	1	0	$I_1 = \overline{C_{in}}$	$I_1 = C_{in}$
0	1	1	0	1		
1	0	0	1	0	$I_2 = \overline{C_{in}}$	$I_2 = C_{in}$
1	0	1	0	1		
1	1	0	0	1	$I_3 = C_{in}$	$I_3 = 1$
1	1	1	1	1		

2. Multiplexer Implementation Strategy

Based on the grouped truth table, we extract the boolean expressions for the MUX data inputs:
For the Sum (S) MUX:

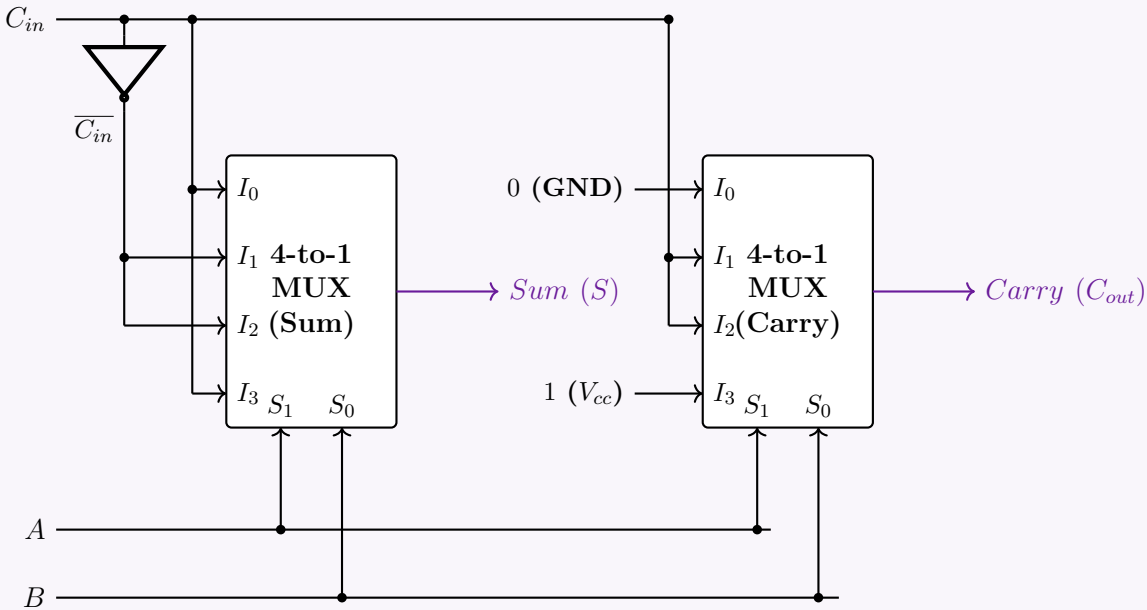
$$I_0 = C_{in}$$
$$I_1 = \overline{C_{in}}$$
$$I_2 = \overline{C_{in}}$$
$$I_3 = C_{in}$$

For the Carry (C_{out}) MUX:

$$I_0 = 0 \text{ (Logic 0/GND)}$$
$$I_1 = C_{in}$$
$$I_2 = C_{in}$$
$$I_3 = 1 \text{ (Logic 1/V}_{cc}\text{)}$$

We will require a single **NOT** gate to generate the $\overline{C_{in}}$ signal for the Sum MUX. Both MUXes will share the A and B inputs on their select lines.

3. Circuit Diagram



Q4. Implement the following Boolean function with a 4×1 multiplexer and external gates:

$$F(A, B, C, D) = \Sigma (1, 2, 4, 7, 8, 9, 10, 11, 13, 15)$$

(13 marks)

[CSE 205 | L-2/T-1 | 2022–23]

Answer

1. Problem Analysis and Implementation Strategy

We are required to implement a 4-variable Boolean function $F(A, B, C, D) = \Sigma (1, 2, 4, 7, 8, 9, 10, 11, 13, 15)$ using a 4×1 Multiplexer. A 4×1 MUX has 2 select lines and 4 data inputs. To implement a 4-variable function, we will apply the **variable mapping method**:

- We connect the two Most Significant Bits (MSBs), A and B , to the select lines: $S_1 = A$ and $S_0 = B$.

- The remaining variables, C and D , will be used to determine the logic expressions applied to the four data input lines (I_0, I_1, I_2, I_3).

2. Multiplexer Implementation Table

We construct the implementation table by listing all 16 minterms of the 4 variables. The minterms are arranged in 4 columns corresponding to the 4 combinations of the select lines (A, B). The rows correspond to the 4 combinations of the data variables (C, D). We circle/highlight the minterms present in the given function.

Variables		I_0	I_1	I_2	I_3
C	D	$(A = 0, B = 0)$	$(A = 0, B = 1)$	$(A = 1, B = 0)$	$(A = 1, B = 1)$
0	0	0	4	8	12
0	1	1	5	9	13
1	0	2	6	10	14
1	1	3	7	11	15
SOP Expression		$\overline{C}D + C\overline{D}$	$\overline{C}\overline{D} + CD$	$\overline{C}\overline{D} + \overline{C}D + C\overline{D} + CD$	$\overline{C}D + CD$
MUX Input		$\mathbf{C \oplus D}$	$\mathbf{\overline{C \oplus D}}$	$\mathbf{1}$	$\mathbf{D}$

3. Boolean Derivations for MUX Inputs

Based on the minterms present in each column, we derive the minimized Boolean expressions for the MUX inputs:

$$I_0 = \Sigma(1, 2) = \overline{C}D + C\overline{D} = \mathbf{C \oplus D} \quad (\text{XOR operation})$$

$$I_1 = \Sigma(4, 7) = \overline{C}\overline{D} + CD = \mathbf{C \odot D} = \mathbf{\overline{C \oplus D}} \quad (\text{XNOR operation})$$

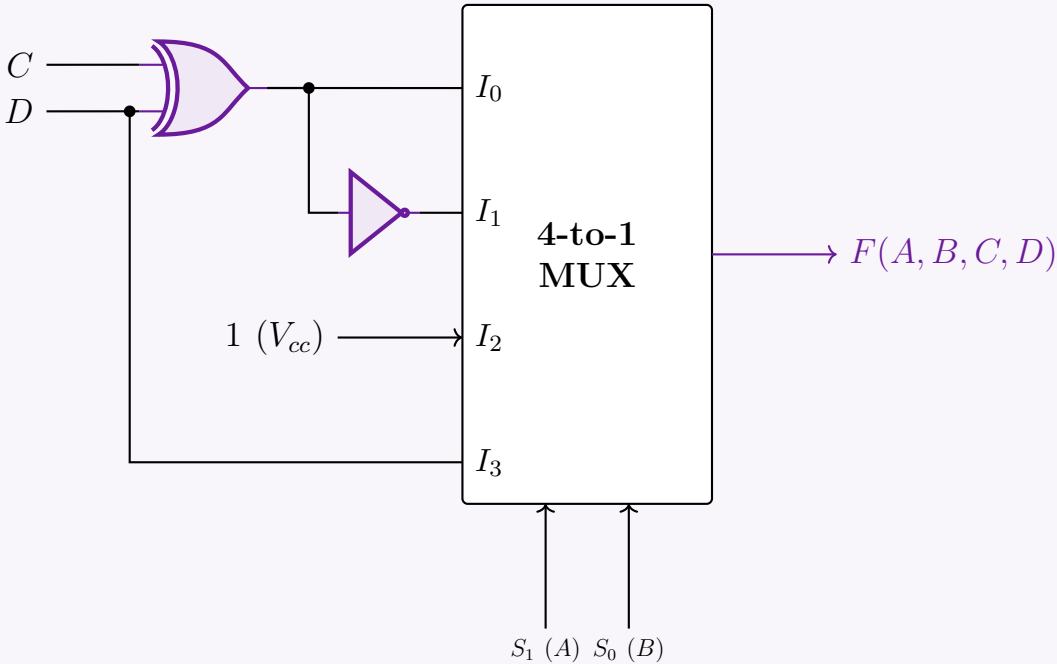
$$I_2 = \Sigma(8, 9, 10, 11) = 1 \quad (\text{All minterms present, connects to Logic 1})$$

$$I_3 = \Sigma(13, 15) = \overline{C}D + CD$$

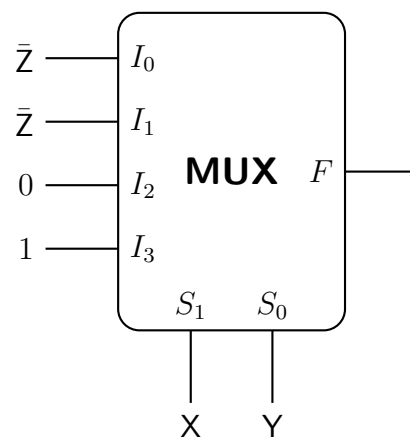
$$= D(\overline{C} + C) \quad (\text{Distributive Law})$$
$$= D(1) = \mathbf{D} \quad (\text{Complement Law and Identity Law})$$

We can generate I_1 simply by passing the output of the XOR gate (I_0) through a NOT gate.

4. Logic Circuit Implementation



Q5. What will be the output function $F(X, Y, Z)$ in the product of maxterms form in the given 4×1 MUX circuit? Assume X as the most significant bit and Z as the least significant bit. (The MUX has inputs $Z', Z, 0, 1$ and selection lines $S_1 S_0$ connected to X, Y .)



(12 marks)

[CSE 205 | L-2/T-1 | 2021–22]

Answer

1. Problem Analysis & Characteristic Equation

The logic function implemented by a 4×1 Multiplexer is defined by its standard characteristic equation:

$$F(S_1, S_0) = \overline{S_1}\overline{S_0}I_0 + \overline{S_1}S_0I_1 + S_1\overline{S_0}I_2 + S_1S_0I_3$$

Based on the provided problem description, we map the variables to the MUX ports as follows:

- **Selection Lines:** $S_1 = X$ (MSB) and $S_0 = Y$
- **Data Inputs:** $I_0 = \overline{Z}$, $I_1 = Z$, $I_2 = 0$, and $I_3 = 1$

Substituting these mappings into the characteristic equation yields the Boolean function in terms of X, Y , and Z :

$$\begin{aligned} F(X, Y, Z) &= \overline{X}\overline{Y}(\overline{Z}) + \overline{X}Y(Z) + X\overline{Y}(0) + XY(1) \\ &= \overline{X}\overline{Y}\overline{Z} + \overline{X}YZ + XY \quad (\text{Applying Zero Property and Identity Law}) \end{aligned}$$

2. Canonical Sum of Products (SOP) Expansion

To find the maxterms reliably, we first determine the active minterms by expanding the function into its canonical Sum of Products (SOP) form. The term XY is missing the variable Z , so we expand it using the Boolean rule $Z + \overline{Z} = 1$:

$$\begin{aligned} F(X, Y, Z) &= \overline{X}\overline{Y}\overline{Z} + \overline{X}YZ + XY(Z + \overline{Z}) \\ &= \overline{X}\overline{Y}\overline{Z} + \overline{X}YZ + XYZ + XY\overline{Z} \end{aligned}$$

Next, we translate each product term into its corresponding binary combination and minterm notation (m_i):

- $\overline{X}\overline{Y}\overline{Z} \rightarrow 000_2 \rightarrow m_0$
- $\overline{X}YZ \rightarrow 011_2 \rightarrow m_3$
- $XY\overline{Z} \rightarrow 110_2 \rightarrow m_6$
- $XYZ \rightarrow 111_2 \rightarrow m_7$

Thus, the canonical SOP form is:

$$F(X, Y, Z) = \Sigma m(0, 3, 6, 7)$$

3. Conversion to Product of Maxterms (POS)

For any Boolean function of n variables, the maxterms (M_j) represent the input combinations for which the function evaluates to logic 0. The set of maxterms is the exact complement of the set of minterms with respect to the universal set of indices 0 to $2^n - 1$. Since $n = 3$ (for variables X, Y, Z), the indices range from 0 to 7. Extracting the missing indices from our minterm set gives the maxterm set:

$$\text{Maxterms} = \{0, 1, 2, 3, 4, 5, 6, 7\} \setminus \{0, 3, 6, 7\} = \{1, 2, 4, 5\}$$

Therefore, the output function in Product of Maxterms (POS) form is:

$$\mathbf{F(X, Y, Z) = \Pi M(1, 2, 4, 5)}$$

4. Truth Table Verification

To guarantee mathematical flawlessness, we verify the output logic across all possible combinations of X, Y, Z .

$X (S_1)$	$Y (S_0)$	Z	Active MUX Input	F	Classification
0	0	0	$I_0 = \overline{Z}$	1	Minterm m_0
0	0	1		0	Maxterm M_1
0	1	0	$I_1 = Z$	0	Maxterm M_2
0	1	1		1	Minterm m_3
1	0	0	$I_2 = 0$	0	Maxterm M_4
1	0	1		0	Maxterm M_5
1	1	0	$I_3 = 1$	1	Minterm m_6
1	1	1		1	Minterm m_7

The truth table visually confirms that the function generates a ‘0’ exclusively at indices 1, 2, 4, and 5, perfectly validating the analytical POS derivation.

Q6. Math teacher Mina Rahman sent sixteen question choices $Q_0, \dots, Q_{15}$ to her students with a 2-bit binary code C_1C_0 . If $C_1C_0 = 11$, Q_0 is selected; $C_1C_0 = 10$, Q_1 is selected; ...and so on. The code is activated by active-low $\overline{EN}_1$ and active-high EN_2 . Design the multiplexing circuit. **(15 marks)** [CSE 205 | L-2/T-1 | 2020–21]

Answer

1. Problem Analysis & Clarification

The problem states that there are sixteen question choices ($Q_0, \dots, Q_{15}$) but provides only a 2-bit binary selection code (C_1C_0). In digital logic, an n -bit selection code can uniquely address exactly 2^n inputs. Therefore, a 2-bit code (C_1C_0) yields $2^2 = 4$ unique combinations.

To create a mathematically sound multiplexing circuit based on the specific selection sequence provided in the problem, we will design a **4-to-1 Multiplexer (MUX)** addressing the four specified choices (Q_0, Q_1, Q_2, Q_3). *Note: If a 16-to-1 selection was strictly required, the system would necessitate a 4-bit selection code ($C_3C_2C_1C_0$).*

2. Selection Logic & Variable Mapping

Based on the defined behavior, the selection sequence is inverted compared to standard binary weighting. We map the variables as follows:

- Selection Lines:** $S_1 = C_1$ (MSB) and $S_0 = C_0$ (LSB).
- Enable Logic:** The MUX is active only when $\overline{EN}_1 = 0$ (active-low) and $EN_2 = 1$ (active-high).
- Data Input Mapping:**
 - When $C_1C_0 = 11$ (Decimal 3), Q_0 is selected $\implies I_3 = Q_0$
 - When $C_1C_0 = 10$ (Decimal 2), Q_1 is selected $\implies I_2 = Q_1$
 - When $C_1C_0 = 01$ (Decimal 1), Q_2 is selected $\implies I_1 = Q_2$
 - When $C_1C_0 = 00$ (Decimal 0), Q_3 is selected $\implies I_0 = Q_3$

3. Truth Table and Boolean Expression

Let the global Enable signal be E . The circuit operates correctly only when $E = 1$.

$$E = (\overline{EN}_1)' \cdot EN_2$$

Enable Inputs		Selection Lines		Internal	Output
$\overline{EN}_1$	EN_2	$C_1 (S_1)$	$C_0 (S_0)$	Enable (E)	Y
1	X	X	X	0	0 (High-Z/Inactive)
X	0	X	X	0	0 (High-Z/Inactive)
0	1	0	0	1	Q_3
0	1	0	1	1	Q_2
0	1	1	0	1	Q_1
0	1	1	1	1	Q_0

The characteristic equation for the active multiplexer is:

$$Y = E \cdot (\overline{S_1}\overline{S_0}I_0 + \overline{S_1}S_0I_1 + S_1\overline{S_0}I_2 + S_1S_0I_3)$$
$$Y = ((\overline{EN}_1)' \cdot EN_2) \cdot (\overline{C_1}\overline{C_0}Q_3 + \overline{C_1}C_0Q_2 + C_1\overline{C_0}Q_1 + C_1C_0Q_0)$$

4. Logic Circuit Implementation

The diagram shows a 4-to-1 Multiplexer (MUX) with inputs I_3, I_2, I_1, I_0 and outputs Q_0, Q_1, Q_2, Q_3 respectively. The MUX is controlled by select lines $S_1 (C_1)$ and $S_0 (C_0)$. The enable input E is connected to the output of an AND gate with inputs $\overline{EN_1}$ and $\overline{EN_2}$. The MUX output is labeled **Output (Y)**.

Q7. There are 16 electronic turnstile gates in a cricket stadium. The gates are numbered from 0 to 15 denoted by four bits $T_3T_2T_1T_0$. You have to design a digital ticketing system which uses a switch, and the switch has four input lines I_0, I_1, I_2, I_3 . These lines can be selected by S_0, S_1 . The switch is activated by two active low enables $\overline{EN_1}$ and $\overline{EN_2}$. (The selection policy of the switch is as follows. If $S_1S_0 = 00$, I_0 is selected; if $S_1S_0 = 01$, I_1 is selected; if $S_1S_0 = 10$, I_2 is selected; ... so on.). You can have entry to the stadium if you get the ticket for any of the following gate numbers (1, 3, 4, 6, 7, 9, 12, 15). Design the ticketing circuit using the switch. **(15 marks)** [CSE 205 | L-2/T-1 | 2020–21]

Answer

1. Problem Formalization and Variable Mapping

The stadium gates are represented by a 4-bit number $T_3T_2T_1T_0$, where T_3 is the Most Significant Bit (MSB) and T_0 is the Least Significant Bit (LSB). The ticketing system allows entry for a specific set of gate numbers, which can be expressed as a Boolean function in canonical Sum of Products (SOP) form:

$$F(T_3, T_2, T_1, T_0) = \sum m(1, 3, 4, 6, 7, 9, 12, 15)$$

We are required to implement this function using a switch that acts as a **4-to-1 Multiplexer (MUX)**. The MUX has:

- Selection Lines:** S_1, S_0 . We map the most significant variables to these lines: $S_1 = T_3$ and $S_0 = T_2$.
- Data Input Lines:** I_0, I_1, I_2, I_3 . The logic for these will be derived using the remaining variables T_1 and T_0 .
- Enable Lines:** Two active-low enables, $\overline{EN_1}$ and $\overline{EN_2}$. The MUX is active only when both are connected to Logic 0 (Ground).

2. Multiplexer Implementation Table

We distribute the 16 minterms across 4 columns corresponding to the 4 combinations of the select lines (T_3, T_2). We then highlight the minterms present in the entry function to determine the boolean expression for each MUX input based on T_1 and T_0 .

Variables		I_0	I_1	I_2	I_3
T_1	T_0	$(T_3 = 0, T_2 = 0)$	$(T_3 = 0, T_2 = 1)$	$(T_3 = 1, T_2 = 0)$	$(T_3 = 1, T_2 = 1)$
0	0	0	4	8	12
0	1	1	5	9	13
1	0	2	6	10	14
1	1	3	7	11	15

3. Boolean Derivations for MUX Inputs

Based on the active minterms in each column, we extract and simplify the Boolean expressions for I_0, I_1, I_2 , and I_3 :

$$\begin{aligned} I_0 &= \Sigma(1, 3) = \overline{T_1}T_0 + T_1T_0 \\ &= T_0(\overline{T_1} + T_1) \\ &= T_0(1) = \mathbf{T_0} \end{aligned}$$
$$\begin{aligned} I_1 &= \Sigma(4, 6, 7) = \overline{T_1}\overline{T_0} + T_1\overline{T_0} + T_1T_0 \\ &= \overline{T_0}(\overline{T_1} + T_1) + T_1T_0 \\ &= \overline{T_0} + T_1T_0 \\ &= (\overline{T_0} + T_1)(\overline{T_0} + T_0) \\ &= \mathbf{T_1 + \overline{T_0}} \end{aligned}$$
$$I_2 = \Sigma(9) = \mathbf{\overline{T_1}T_0}$$
$$\begin{aligned} I_3 &= \Sigma(12, 15) = \overline{T_1}\overline{T_0} + T_1T_0 \\ &= \mathbf{T_1 \odot T_0} \end{aligned}$$

(Distributive Law)

(Complement and Identity Laws)

(Distributive Law)

(Complement Law)

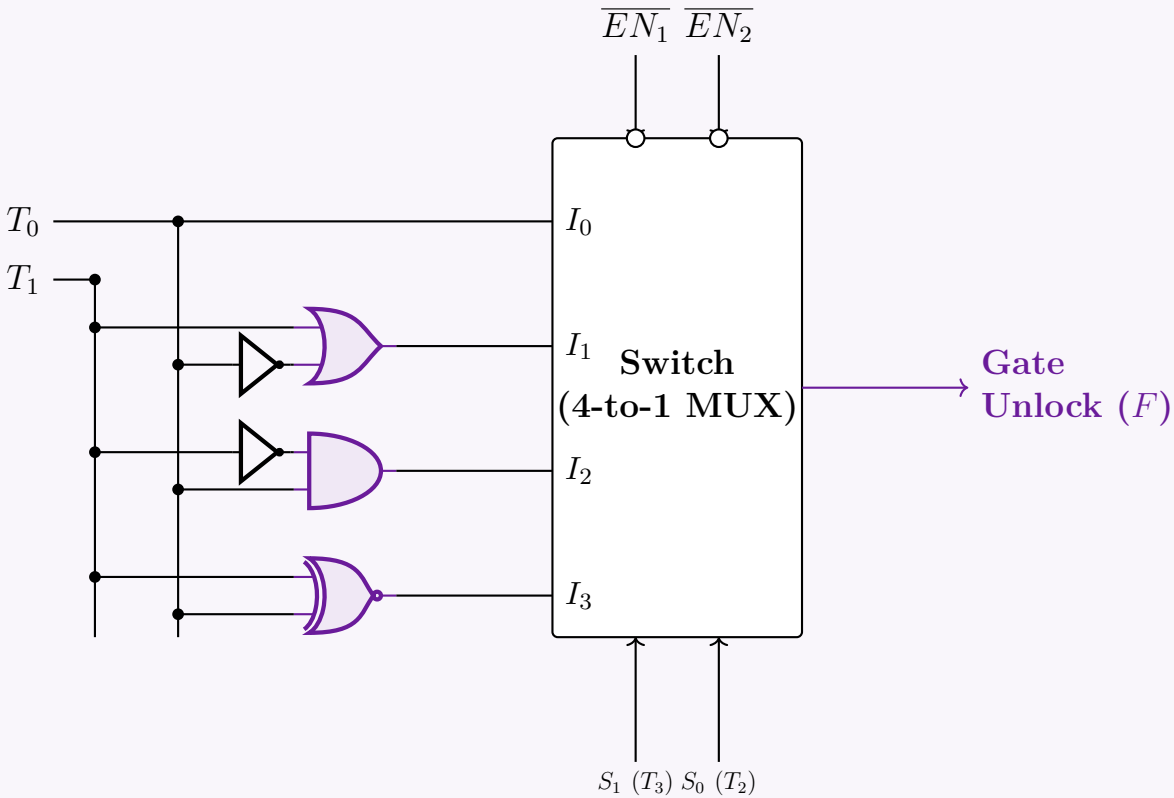
(Absorption / Distributive Law: $A + \overline{A}B = A + B$)

(OR operation)

(AND operation, minimal form)

(XNOR operation)

4. Logic Circuit Implementation



Q8. Implement $F(A, B, C, D) = \Sigma(1, 2, 4, 10, 12, 13, 14, 15)$ with a 4×1 multiplexer and external gates. Connect inputs A and B to the selection lines. Express F as a function of C and D for each of the four cases $AB = 00, 01, 10, 11$ to obtain the data-line requirements. (17 marks) [CSE 205 | L-2/T-1 | 2019–20]

Answer

1. Problem Analysis and Implementation Strategy

We are tasked with implementing a 4-variable Boolean function $F(A, B, C, D) = \Sigma(1, 2, 4, 10, 12, 13, 14, 15)$ using a 4×1 multiplexer. A 4×1 MUX has 2 select lines and 4 data inputs. The problem specifies connecting inputs A and B to the select lines:

- Selection Lines:** $S_1 = A$ (MSB) and $S_0 = B$ (LSB).
- Data Lines (I_0, I_1, I_2, I_3):** The remaining variables, C and D , will dictate the logic expressions applied to the input lines for each of the four combinations of AB .

2. Multiplexer Implementation Table

We map the 16 minterms of the 4 variables across 4 columns corresponding to the states of AB . We highlight the minterms specified in the function to determine the residual logic required from variables C and D .

Variables		I_0	I_1	I_2	I_3
C	D	$(A = 0, B = 0)$	$(A = 0, B = 1)$	$(A = 1, B = 0)$	$(A = 1, B = 1)$
0	0	0	4	8	12
0	1	1	5	9	13
1	0	2	6	10	14
1	1	3	7	11	15

3. Boolean Derivations for Data Lines

By analyzing the active minterms in each column as a function of C and D , we systematically derive the minimal Boolean expression for each MUX input:

$$\begin{aligned} I_0 \text{ (AB = 00)} &= \Sigma(1, 2) = \overline{C}D + C\overline{D} \\ &= \mathbf{C \oplus D} \end{aligned}$$

(Definition of XOR operation)

$$\begin{aligned} I_1 \text{ (AB = 01)} &= \Sigma(4) = \overline{C}\overline{D} \\ &= \mathbf{\overline{C + D}} \end{aligned}$$

(De Morgan's Law - NOR operation)

$$\begin{aligned} I_2 \text{ (AB = 10)} &= \Sigma(10) = \mathbf{C\overline{D}} \end{aligned}$$

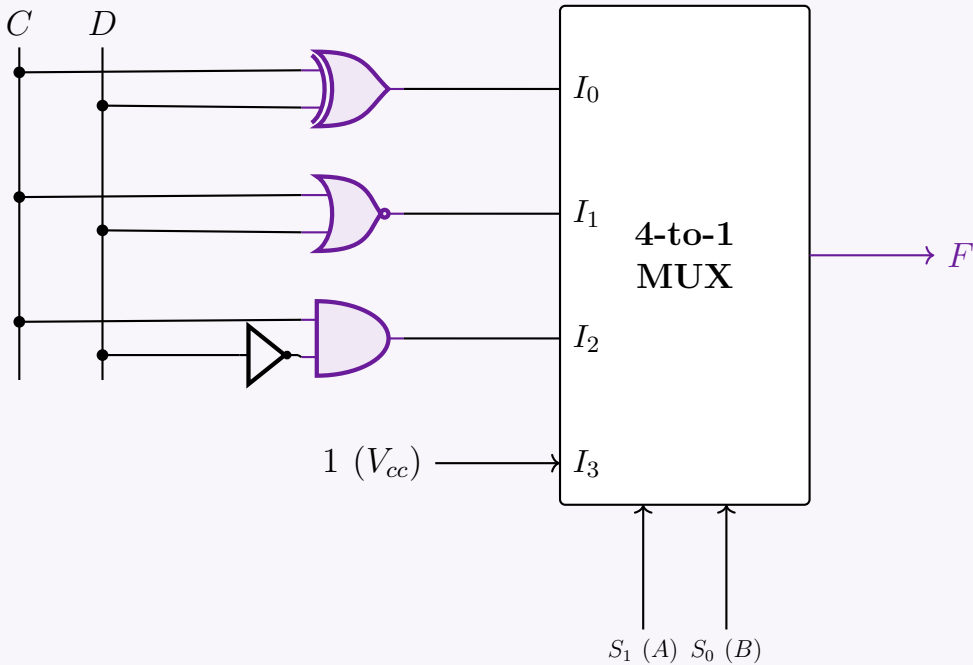
(AND operation with inverted D)

$$\begin{aligned} I_3 \text{ (AB = 11)} &= \Sigma(12, 13, 14, 15) = \overline{C}\overline{D} + \overline{C}D + C\overline{D} + CD \\ &= \overline{C}(\overline{D} + D) + C(\overline{D} + D) \\ &= \overline{C}(1) + C(1) \\ &= \mathbf{\overline{C} + C = 1} \end{aligned}$$

(Distributive Law)
(Complement Law: $X + \overline{X} = 1$)
(Identity and Complement Laws)

4. Logic Circuit Implementation

Using the derived input expressions, we construct the circuit with standard logic gates feeding into the 4×1 MUX.



Q9. Using a 4×1 line MUX, design a logic circuit for $f(w, x, y, z) = \Pi(0, 2, 5, 6, 7, 8, 9, 10)$, where the MUX has selector bits S_1S_0 , input lines $I_0 \dots I_3$, active-low enable $\overline{EN}_1$ and active-high enable EN_2 . (15 marks) [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Problem Analysis and Canonical Conversion

The given Boolean function is defined in Product of Maxterms (POS) form:

$$f(w, x, y, z) = \Pi(0, 2, 5, 6, 7, 8, 9, 10)$$

To implement this using a Multiplexer (MUX), it is more systematic to express the function in its canonical Sum of Minterms (SOP) form. For a 4-variable function, the universal set of indices is 0 to $2^4 - 1 = 15$. The minterms are the indices missing from the maxterm list:

$$\begin{aligned} \text{Minterms} &= \{0, 1, \dots, 15\} \setminus \{0, 2, 5, 6, 7, 8, 9, 10\} \\ &= \{1, 3, 4, 11, 12, 13, 14, 15\} \end{aligned}$$

Thus, the equivalent SOP expression is:

$f(w, x, y, z) = \Sigma (1, 3, 4, 11, 12, 13, 14, 15)$

2. Multiplexer Variable Mapping

A 4×1 multiplexer requires 2 selection lines (S_1, S_0) to address its 4 input lines (I_0, I_1, I_2, I_3). We map the most significant variables to the selection lines:

- **Selection Lines:** $S_1 = w$ and $S_0 = x$.
- **Data Lines:** The logic for inputs I_0, I_1, I_2, I_3 will be determined as functions of the remaining variables, y and z .

3. Implementation Table

We distribute the 16 possible minterms across 4 columns according to the combinations of w and x . We then highlight the active minterms belonging to the function to extract the logic expression for each MUX input.

Variables		I_0	I_1	I_2	I_3
y	z	$(w = 0, x = 0)$	$(w = 0, x = 1)$	$(w = 1, x = 0)$	$(w = 1, x = 1)$
0	0	0	4	8	12
0	1	1	5	9	13
1	0	2	6	10	14
1	1	3	7	11	15

4. Boolean Derivations for Input Lines

We formulate the minimum Boolean expression for each input column based on the active minterms:

$$\begin{aligned} I_0 \text{ (wx = 00)} &= \Sigma(1, 3) = \overline{y}z + yz \\ &= z(\overline{y} + y) \\ &= z(1) = \mathbf{z} \end{aligned}$$

(Distributive Law)
(Complement and Identity Laws)

$$\begin{aligned} I_1 \text{ (wx = 01)} &= \Sigma(4) = \overline{y}\overline{z} \\ &= \overline{\mathbf{y + z}} \end{aligned}$$

(De Morgan’s Law - NOR operation)

$$I_2 \text{ (wx = 10)} = \Sigma(11) = \mathbf{yz}$$

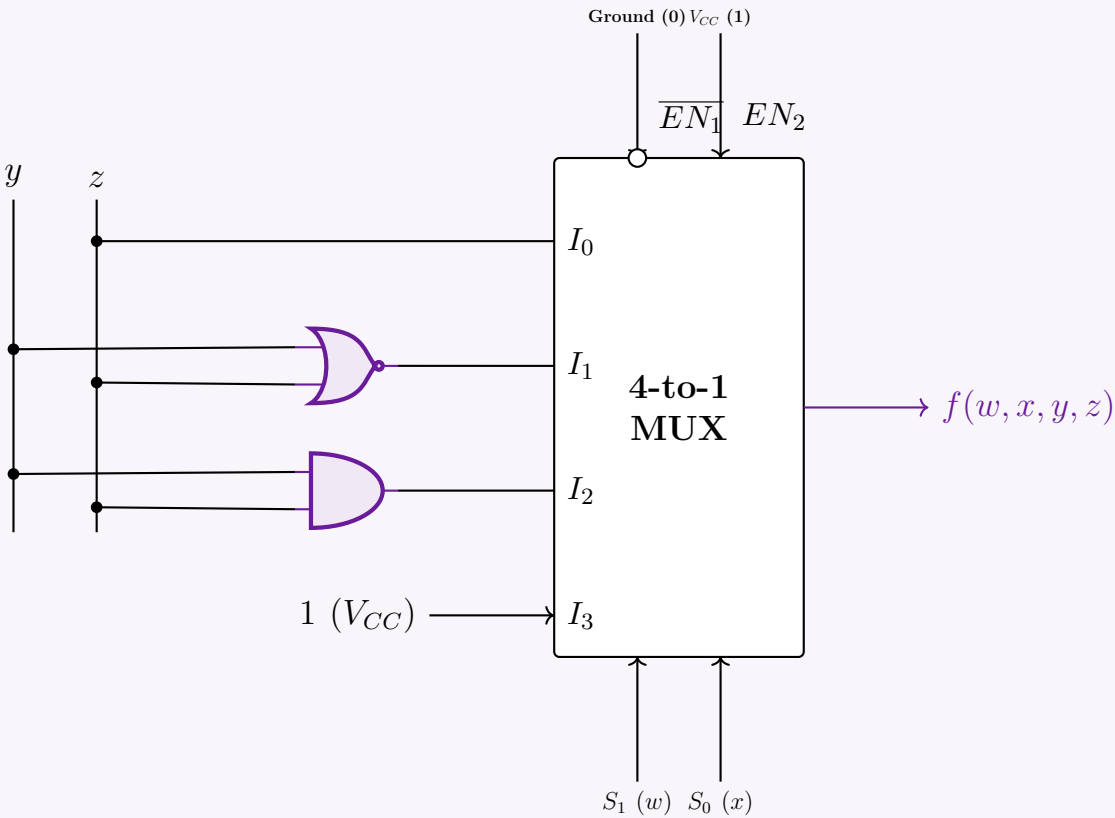
(AND operation)

$$\begin{aligned} I_3 \text{ (wx = 11)} &= \Sigma(12, 13, 14, 15) = \overline{y}\overline{z} + \overline{y}z + y\overline{z} + yz \\ &= \overline{y}(\overline{z} + z) + y(\overline{z} + z) \\ &= \overline{y}(1) + y(1) \\ &= \overline{y} + y = \mathbf{1} \end{aligned}$$

(Distributive Law)
(Complement Law: $A + \overline{A} = 1$)
(Identity and Complement Laws)

5. Logic Circuit Implementation

The circuit includes the MUX enables: $\overline{EN}_1$ (active-low) must be tied to Ground (0), and EN_2 (active-high) must be tied to V_{CC} (1) for normal operation.



Q10. Design a 1-bit Full Subtractor using a 4×1 multiplexer. The MUX has selector bits S_1S_0 , input lines I_0, I_1, I_2, I_3 , and active-low enable $\overline{EN}$. **(12 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Truth Table of a 1-Bit Full Subtractor

A 1-bit Full Subtractor subtracts a subtrahend (Y) and a borrow-in (B_{in}) from a minuend (X). It produces two outputs: Difference (D) and Borrow-out (B_{out}). The truth table defining this behavior is:

Inputs			Outputs	
X (Minuend)	Y (Subtrahend)	B_{in} (Borrow In)	D (Difference)	B_{out} (Borrow Out)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

2. Multiplexer Configuration Strategy

Since a Full Subtractor has two independent output functions (D and B_{out}), we require **two** 4×1 multiplexers.

- Selection Lines:** We assign the variables X and Y to the MUX selector bits: $S_1 = X$ and $S_0 = Y$.
- Data Lines (I_0, I_1, I_2, I_3):** The inputs for each MUX will be evaluated as a function of the remaining variable, B_{in} .
- Enable Line:** The problem specifies an active-low enable ($\overline{EN}$). For the MUX to function, this line must be tied to Logic 0 (Ground).

3. Boolean Derivation for Data Lines

We determine the input data lines algebraically by fixing the selection lines X and Y , and expressing the output solely in terms of B_{in} .

A. Difference Output (D)

The standard Boolean expression for the Difference is $D = X \oplus Y \oplus B_{in}$. Let's evaluate this for all four selector combinations:

I_0 ($XY = 00$):

$D = 0 \oplus 0 \oplus B_{in}$
 $= 0 \oplus B_{in} = \mathbf{B_{in}}$

(Identity Law of XOR)

I_1 ($XY = 01$):

$D = 0 \oplus 1 \oplus B_{in}$
 $= 1 \oplus B_{in} = \overline{\mathbf{B_{in}}}$

(Inversion Law of XOR)

I_2 ($XY = 10$):

$D = 1 \oplus 0 \oplus B_{in}$
 $= 1 \oplus B_{in} = \overline{\mathbf{B_{in}}}$

(Inversion Law of XOR)

I_3 ($XY = 11$):

$D = 1 \oplus 1 \oplus B_{in}$
 $= 0 \oplus B_{in} = \mathbf{B_{in}}$

(Identity Law of XOR)

B. Borrow-Out Output (B_{out})

The standard minimized Boolean expression for Borrow-out is $B_{out} = \overline{X}Y + \overline{X}B_{in} + YB_{in}$.

I_0 ($XY = 00$):

$B_{out} = \overline{0}(0) + \overline{0}B_{in} + 0(B_{in})$
 $= 0 + 1 \cdot B_{in} + 0 = \mathbf{B_{in}}$

(Identity and Annulment Laws)

I_1 ($XY = 01$):

$B_{out} = \overline{0}(1) + \overline{0}B_{in} + 1(B_{in})$
 $= 1 + B_{in} + B_{in} = \mathbf{1}$

(Annulment Law: $1 + A = 1$)

I_2 ($XY = 10$):

$B_{out} = \overline{1}(0) + \overline{1}B_{in} + 0(B_{in})$
 $= 0 + 0 \cdot B_{in} + 0 = \mathbf{0}$

(Annulment Law: $0 \cdot A = 0$)

I_3 ($XY = 11$):

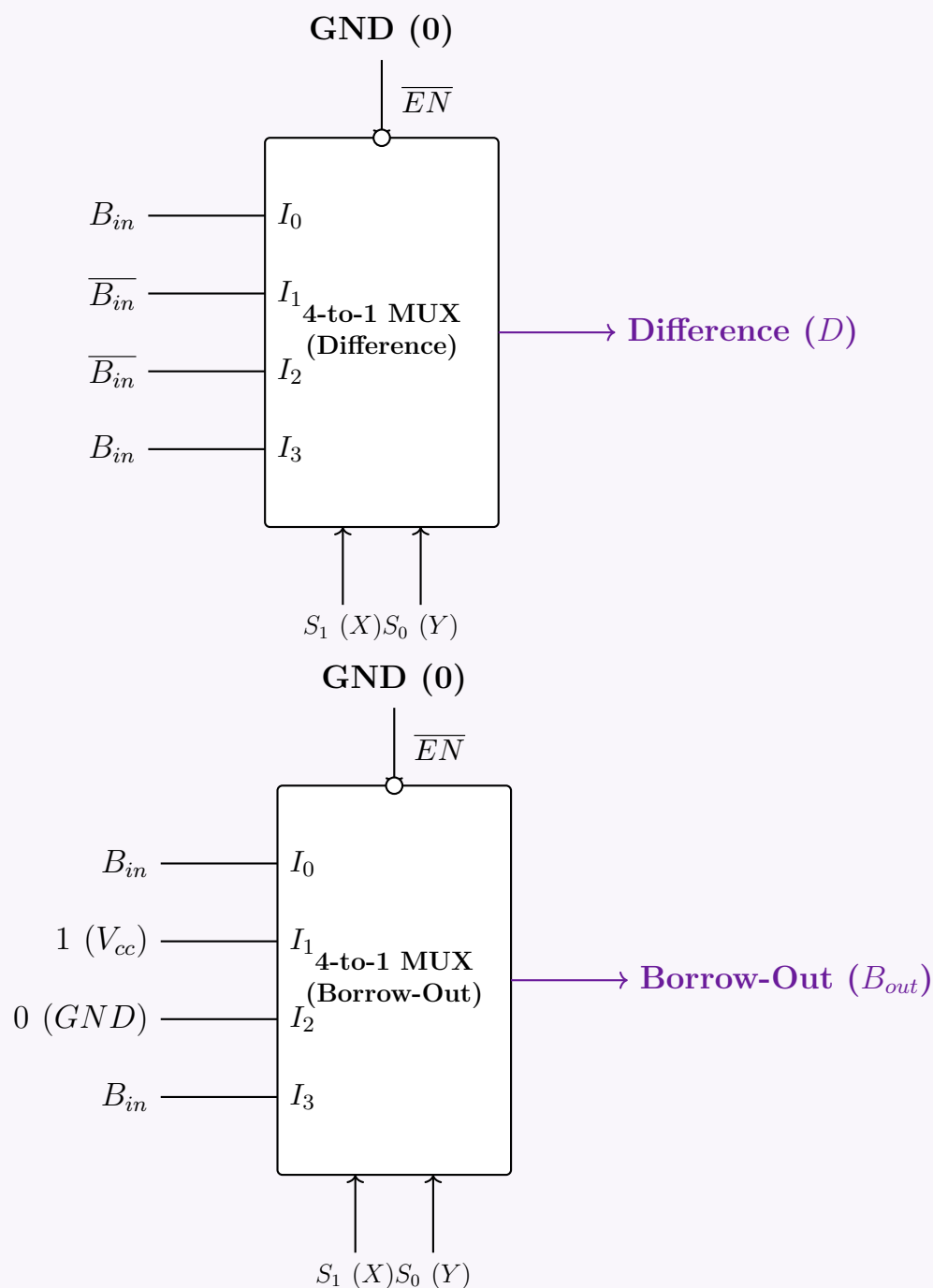
$B_{out} = \overline{1}(1) + \overline{1}B_{in} + 1(B_{in})$
 $= 0 \cdot 1 + 0 \cdot B_{in} + B_{in} = \mathbf{B_{in}}$

(Identity and Annulment Laws)

189

4. Logic Circuit Implementation

Using the derived data lines, we construct the twin 4×1 MUX circuit for the Full Subtractor.



Q11. Using an 8×1 line MUX, design a logic circuit for $f(w, x, y, z) = \text{POS}(0, 2, 5, 6, 7, 8, 9, 10)$, where the MUX has selector bits $S_2S_1S_0$, input lines $I_0 \dots I_7$, and active-low enable $\overline{EN}$. (If $S_2S_1S_0 = 000$, I_0 is selected; ...and so on.) (15 marks) [CSE 205 | L-2/T-1 | 2017–18]

Answer

Step 1: Convert the Function to Sum of Products (SOP) Form

The given boolean function is expressed as a Product of Sums (POS), characterized by its maxterms. Since there are 4 variables (w, x, y, z) , the total number of fundamental combinations is $2^4 = 16$, corresponding to indices 0 to 15.

We can easily switch from POS to SOP (Sum of Products) by finding the complementary indices, as a function contains either the minterm or the maxterm for every possible input combination:

$$\begin{aligned} f(w, x, y, z) &= \prod M(0, 2, 5, 6, 7, 8, 9, 10) \\ &= \sum m(1, 3, 4, 11, 12, 13, 14, 15) \end{aligned}$$

Step 2: Assign Variables to Multiplexer Control Lines

We must implement this 4-variable function using an 8×1 Multiplexer. An 8×1 MUX has 3 selector lines (S_2, S_1, S_0) . We partition our 4 variables such that the most significant variables are tied to the selectors, and the least significant variable acts as the data input to the MUX.

- **Selector Lines:** Connect $w \rightarrow S_2$, $x \rightarrow S_1$, and $y \rightarrow S_0$.
- **Data Input Variable:** The remaining variable is z . Thus, the data inputs $I_0, I_1, \dots, I_7$ will be derived as functions of z (i.e., 0, 1, z , or $\bar{z}$).

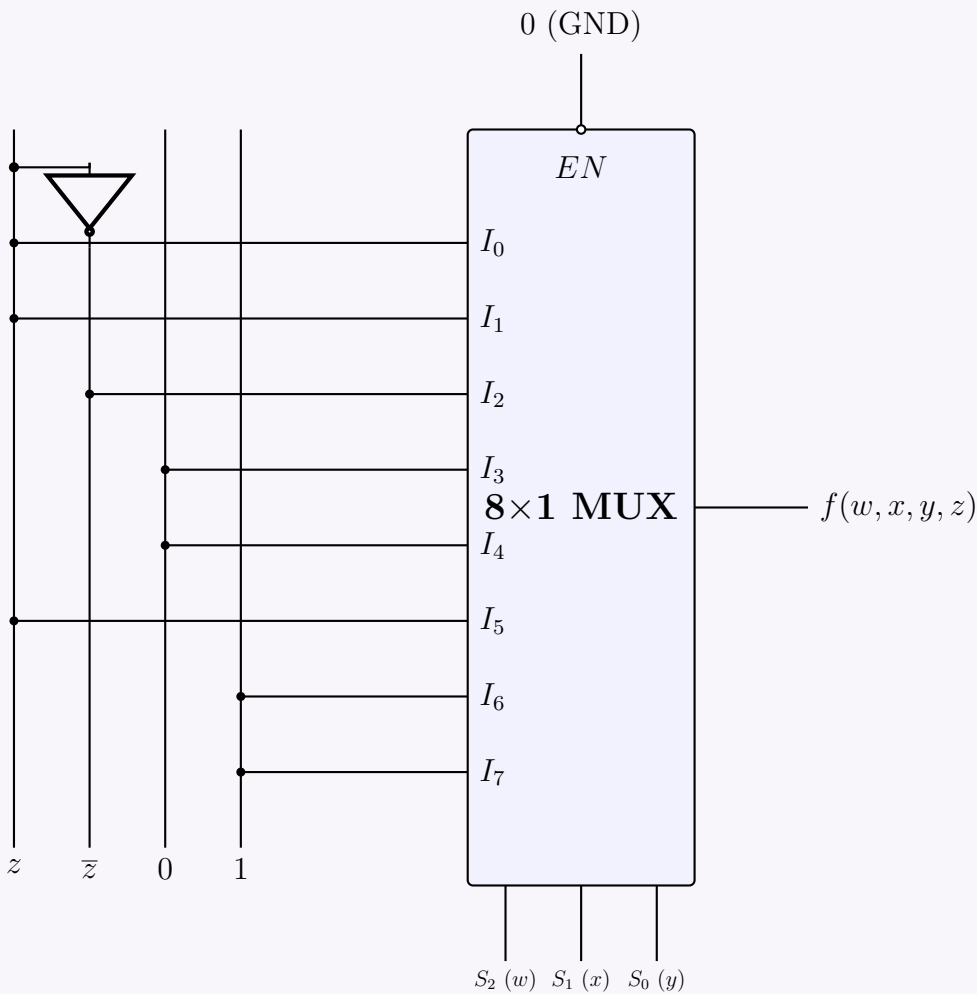
Step 3: Construct the Multiplexer Implementation Table

We create an implementation table mapping the 16 possible minterms into the 8 data inputs based on the state of z . Each data line I_k controls a pair of minterms: one where $z = 0$ (even index) and one where $z = 1$ (odd index). We circle or bold the minterms present in our SOP expression to determine the logical expression for each I_k .

$S_2(w)$	$S_1(x)$	$S_0(y)$	MUX Input	$z = 0$	$z = 1$	Logic Function
0	0	0	I_0	0	1	$I_0 = z$
0	0	1	I_1	2	3	$I_1 = z$
0	1	0	I_2	4	5	$I_2 = \bar{z}$
0	1	1	I_3	6	7	$I_3 = 0$
1	0	0	I_4	8	9	$I_4 = 0$
1	0	1	I_5	10	11	$I_5 = z$
1	1	0	I_6	12	13	$I_6 = 1$
1	1	1	I_7	14	15	$I_7 = 1$

Step 4: Circuit Diagram

The MUX has an active-low enable $\overline{EN}$. For the multiplexer to function normally, this pin must be tied to a logic low state (0 or GND).



Q12. Design a 1-bit Full Adder using two 4×1 multiplexers. The MUX has selector bits S_1S_0 , input lines I_0, I_1, I_2, I_3 , and active-low enable $\overline{EN}$. (If $S_1S_0 = 00$, I_0 is selected; $01 \rightarrow I_1$; $10 \rightarrow I_2$; $11 \rightarrow I_3$.) **(12 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

Step 1: Truth Table of a 1-bit Full Adder

A 1-bit full adder computes the sum of three bits: A, B , and C_{in} . It produces two outputs: Sum and C_{out} (Carry-out). The truth table for these operations is defined as follows:

Inputs			Outputs	
A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Step 2: Assign Variables to Multiplexer Selectors

We are tasked with implementing the two output functions (Sum and C_{out}) using two 4×1 multiplexers. A 4×1 MUX requires 2 selector lines (S_1, S_0). We will map the inputs A and B to the selector lines of both multiplexers:

- $S_1 = A$
- $S_0 = B$

The remaining input, C_{in} , will be used to derive the data inputs I_0, I_1, I_2, I_3 for each MUX.

Step 3: Deriving MUX Inputs

We evaluate the relationship between the outputs and C_{in} for every combination of A and B .

For the Sum Multiplexer (MUX 1):

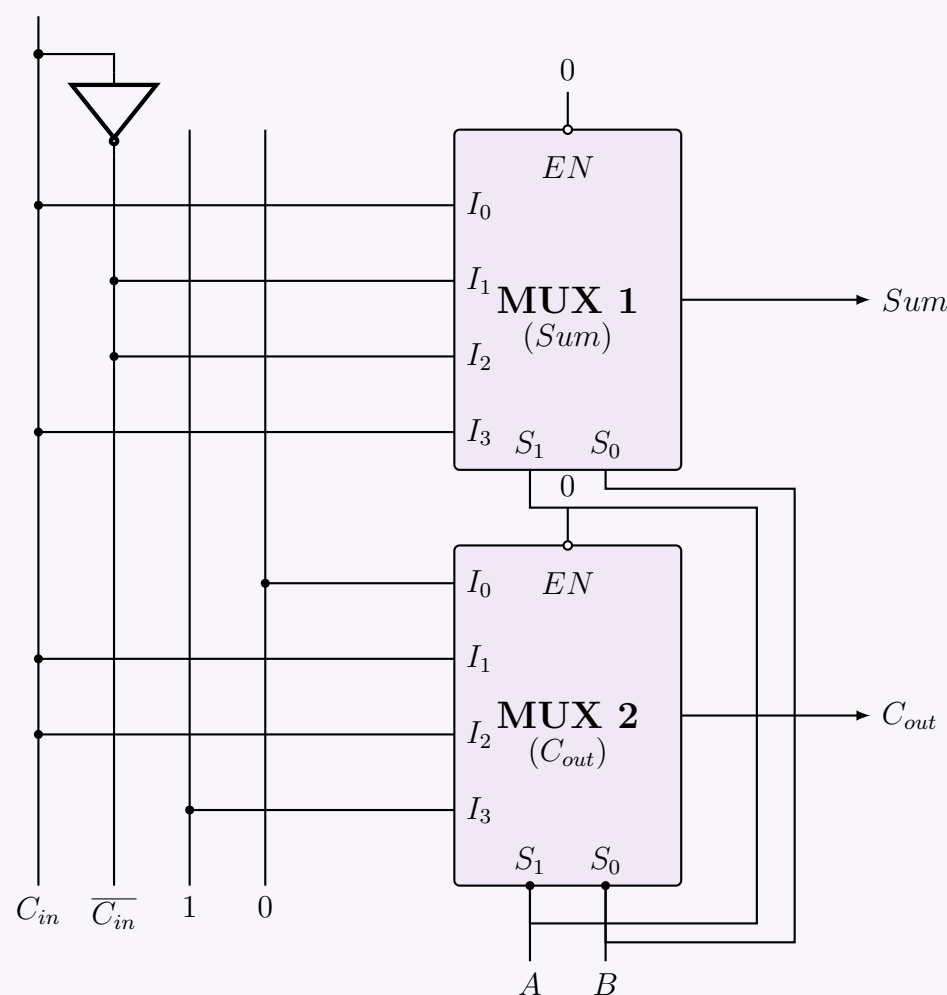
$$\begin{aligned} A = 0, B = 0 \ (S_1 S_0 = 00) &\implies Sum = C_{in} &\therefore I_0 = C_{in} \\ A = 0, B = 1 \ (S_1 S_0 = 01) &\implies Sum = \overline{C_{in}} &\therefore I_1 = \overline{C_{in}} \\ A = 1, B = 0 \ (S_1 S_0 = 10) &\implies Sum = \overline{C_{in}} &\therefore I_2 = \overline{C_{in}} \\ A = 1, B = 1 \ (S_1 S_0 = 11) &\implies Sum = C_{in} &\therefore I_3 = C_{in} \end{aligned}$$

For the Carry-out Multiplexer (MUX 2):

$$\begin{aligned} A = 0, B = 0 \ (S_1 S_0 = 00) &\implies C_{out} = 0 &\therefore I_0 = 0 \\ A = 0, B = 1 \ (S_1 S_0 = 01) &\implies C_{out} = C_{in} &\therefore I_1 = C_{in} \\ A = 1, B = 0 \ (S_1 S_0 = 10) &\implies C_{out} = C_{in} &\therefore I_2 = C_{in} \\ A = 1, B = 1 \ (S_1 S_0 = 11) &\implies C_{out} = 1 &\therefore I_3 = 1 \end{aligned}$$

Step 4: Circuit Diagram

Both multiplexers have an active-low enable $\overline{EN}$. This pin must be tied to a logic low state (0 or GND) for normal operation. The data rails provide 0, 1, C_{in} , and $\overline{C_{in}}$.



Q13. Using only 4×1 line, 2-bit multiplexers, design $f(w, x, y, z) = \pi(2, 4, 5, 7, 11, 12, 13)$. (Use minimum gates if required.) (12 marks)
[CSE 205 | L-2/T-1 | 2016–17]

Answer

Step 1: Convert the Function to Sum of Products (SOP) Form

The given boolean function is provided in Product of Sums (POS) form, specifying the maxterms. For a 4-variable function (w, x, y, z), the domain ranges from 0 to 15. We convert this to Sum of Products (SOP) by identifying the missing indices, which correspond to the minterms:

$$\begin{aligned} f(w, x, y, z) &= \prod M(2, 4, 5, 7, 11, 12, 13) \\ &= \sum m(0, 1, 3, 6, 8, 9, 10, 14, 15) \end{aligned}$$

Step 2: Multiplexer Tree Design Strategy (Shannon’s Expansion)

The prompt restricts us to using **only** 4×1 line multiplexers. A single 4×1 MUX has 2 selector lines and 4 data inputs, which inherently evaluates any 2-variable logic function by tying its inputs directly to 0 or 1.

Because we have a 4-variable function and cannot use external logic gates (AND, OR, NOT), we must construct a **Multiplexer Tree** to form an equivalent 16×1 multiplexer.

• **Level 2 (Output Stage):** A single 4×1 MUX controlled by the two Most Significant Bits (MSBs), w and x .

• **Level 1 (Input Stage):** Four 4×1 MUXes controlled by the two Least Significant Bits (LSBs), y and z . The outputs of these four MUXes feed directly into the Output Stage MUX.

Step 3: Deriving the MUX Data Inputs

We evaluate the function for all 16 combinations. The bits w and x select which Level 1 MUX is active, while y and z select the specific input (0, 1, 2, or 3) inside that Level 1 MUX.

Level 2 Selectors		Minterm Values for y, z (S_1, S_0)				Level 1 MUX Assignment
w (S_1)	x (S_0)	00 (I_0)	01 (I_1)	10 (I_2)	11 (I_3)	
0	0	$m_0 = 1$	$m_1 = 1$	$m_2 = 0$	$m_3 = 1$	MUX 0 inputs: 1, 1, 0, 1
0	1	$m_4 = 0$	$m_5 = 0$	$m_6 = 1$	$m_7 = 0$	MUX 1 inputs: 0, 0, 1, 0
1	0	$m_8 = 1$	$m_9 = 1$	$m_{10} = 1$	$m_{11} = 0$	MUX 2 inputs: 1, 1, 1, 0
1	1	$m_{12} = 0$	$m_{13} = 0$	$m_{14} = 1$	$m_{15} = 1$	MUX 3 inputs: 0, 0, 1, 1

Step 4: Circuit Diagram

Below is the complete MUX tree. Notice that no logic gates are required; all primary inputs to the Level 1 multiplexers are tied strictly to logic high (1) or logic low (0).

Q14. Design and implement the circuit diagram of a 1-bit full subtractor circuit using 2×1 MUXs only. You cannot use any other gates.
(12 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

Step 1: Truth Table of a 1-bit Full Subtractor

A full subtractor performs the operation $A - B - B_{in}$, producing a Difference (D) and a Borrow-out (B_{out}).

193

Inputs			Outputs	
A (Minuend)	B (Subtrahend)	B _{in} (Borrow-in)	D (Difference)	B _{out} (Borrow-out)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Step 2: Mathematical Derivation using Shannon’s Expansion

To implement these functions using **only** 2 × 1 multiplexers, we use Shannon’s Expansion Theorem to factor the expressions around the variables A and B, since a 2 × 1 MUX natively implements the logic $Y = \overline{S}I_0 + SI_1$.

1. Difference (D):

$$D(A, B, B_{in}) = \overline{A} \cdot [\overline{B}(B_{in}) + B(\overline{B_{in}})] + A \cdot [\overline{B}(\overline{B_{in}}) + B(B_{in})]$$

This directly implies a 2-level MUX tree:

- **MUX 1** (A = 0 branch, Selector=B): I₀ = B_{in}, I₁ = $\overline{B_{in}}$
- **MUX 2** (A = 1 branch, Selector=B): I₀ = $\overline{B_{in}}$, I₁ = B_{in}
- **MUX 3** (Output stage, Selector=A): I₀ = MUX 1_{out}, I₁ = MUX 2_{out}

2. Borrow-out (B_{out}):

$$B_{out}(A, B, B_{in}) = \overline{A} \cdot [\overline{B}(B_{in}) + B(1)] + A \cdot [\overline{B}(0) + B(B_{in})]$$

Similarly, this gives us:

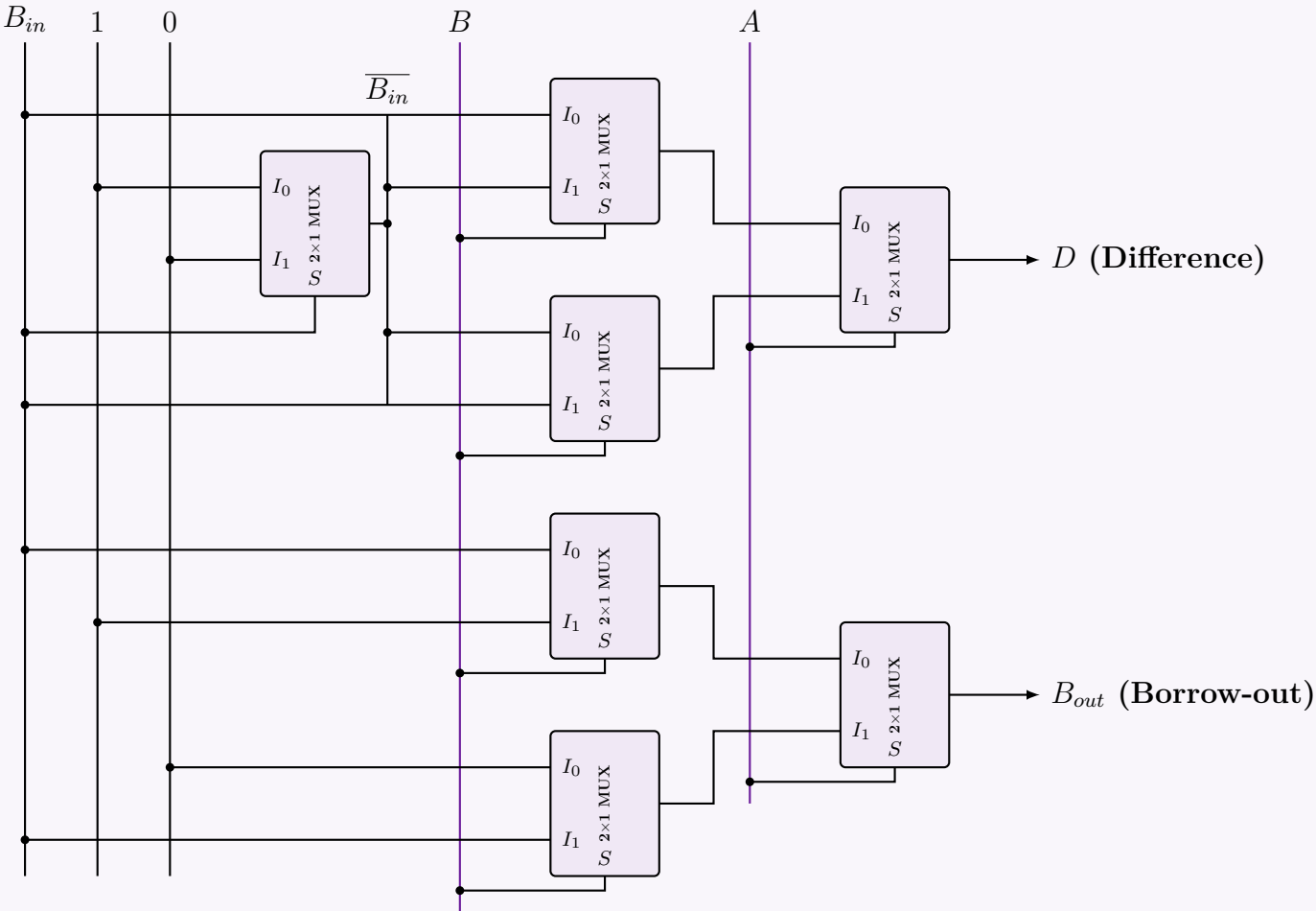
- **MUX 4** (A = 0 branch, Selector=B): I₀ = B_{in}, I₁ = 1
- **MUX 5** (A = 1 branch, Selector=B): I₀ = 0, I₁ = B_{in}
- **MUX 6** (Output stage, Selector=A): I₀ = MUX 4_{out}, I₁ = MUX 5_{out}

3. NOT Gate for $\overline{B_{in}}$: Because we cannot use standard logic gates, we must generate $\overline{B_{in}}$ using a 2 × 1 MUX:

$$\overline{B_{in}} = \overline{B_{in}} \cdot (1) + B_{in} \cdot (0)$$

- **MUX 0** (Inverter, Selector=B_{in}): I₀ = 1, I₁ = 0

Step 3: Circuit Implementation



Q15. Consider the Boolean function $f(w, x, y, z) = \Pi(2, 4, 5, 7, 11, 12, 13)$. Implement the function using a 4×1 MUX and other basic gates if necessary. **(10 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Answer

Step 1: Convert the Function to Sum of Products (SOP) Form

The given Boolean function is provided in Product of Sums (POS) form, specifying the maxterms. For a 4-variable function (w, x, y, z) , the domain ranges from 0 to 15. We first convert this to the Sum of Products (SOP) form by identifying the missing indices, which correspond to the minterms:

$$\begin{aligned} f(w, x, y, z) &= \prod M(2, 4, 5, 7, 11, 12, 13) \\ &= \sum m(0, 1, 3, 6, 8, 9, 10, 14, 15) \end{aligned}$$

Step 2: Assign Variables to the Multiplexer Selectors

We are required to implement the function using a 4×1 multiplexer. A 4×1 MUX requires exactly 2 selector lines (S_1, S_0) . We will assign the two Most Significant Bits (MSBs) to the selector lines:

- $S_1 = w$
- $S_0 = x$

This implies that the multiplexer's data inputs (I_0, I_1, I_2, I_3) will be derived as Boolean expressions in terms of the remaining variables, y and z .

Step 3: Deriving MUX Data Inputs

We evaluate the minterms corresponding to each selector combination to find the expressions for I_0, I_1, I_2, I_3 .

1. For $w = 0, x = 0$ (Selects I_0): This branch covers minterms m_0, m_1, m_2, m_3 . The minterms present in the function are m_0, m_1, m_3 .

$$\begin{aligned} I_0(y, z) &= \bar{y}\bar{z} + \bar{y}z + yz \\ &= \bar{y}(\bar{z} + z) + yz && \text{(Factoring out } \bar{y}) \\ &= \bar{y}(1) + yz && \text{(Complement Law: } \bar{z} + z = 1) \\ &= \bar{y} + yz \\ &= (\bar{y} + y)(\bar{y} + z) && \text{(Distributive Law)} \\ &= \bar{y} + z \end{aligned}$$

2. For $w = 0, x = 1$ (Selects I_1): This branch covers minterms m_4, m_5, m_6, m_7 . The only minterm present is m_6 .

$$I_1(y, z) = y\bar{z}$$

3. For $w = 1, x = 0$ (Selects I_2): This branch covers minterms m_8, m_9, m_{10}, m_{11} . The minterms present are m_8, m_9, m_{10} .

$$\begin{aligned} I_2(y, z) &= \bar{y}\bar{z} + \bar{y}z + y\bar{z} \\ &= \bar{y}(\bar{z} + z) + y\bar{z} \\ &= \bar{y} + y\bar{z} \\ &= (\bar{y} + y)(\bar{y} + \bar{z}) && \text{(Distributive Law)} \\ &= \bar{y} + \bar{z} \\ &= \overline{yz} && \text{(De Morgan's Law)} \end{aligned}$$

4. For $w = 1, x = 1$ (Selects I_3): This branch covers minterms $m_{12}, m_{13}, m_{14}, m_{15}$. The minterms present are m_{14}, m_{15} .

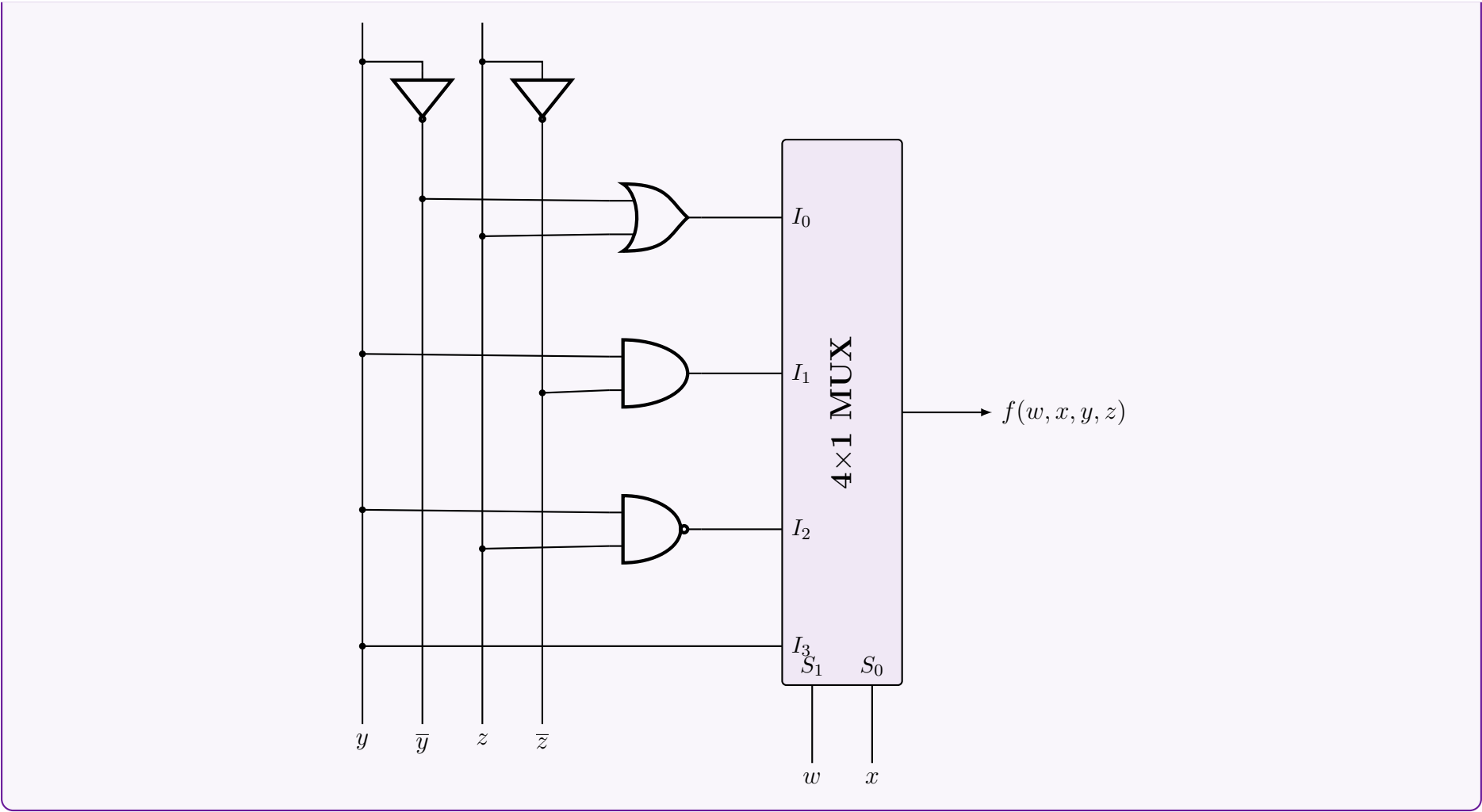
$$\begin{aligned} I_3(y, z) &= y\bar{z} + yz \\ &= y(\bar{z} + z) \\ &= y \end{aligned}$$

Summary of Inputs:

$w (S_1)$	$x (S_0)$	Active Minterms (y, z)	Input Equation
0	0	m_0, m_1, m_3	$I_0 = \bar{y} + z$
0	1	m_6	$I_1 = y\bar{z}$
1	0	m_8, m_9, m_{10}	$I_2 = \overline{yz}$
1	1	m_{14}, m_{15}	$I_3 = y$

Step 4: Circuit Diagram

The logical expressions for the inputs can be implemented using standard basic gates: an OR gate for I_0 , an AND gate for I_1 , and a NAND gate for I_2 (implementing $\bar{y} + \bar{z}$ directly via De Morgan's theorem). Inverters are used to generate $\bar{y}$ and $\bar{z}$.



Q16. Show the truth table for a three-state buffer. Construct a 2-to-1 line MUX with three-state buffers. **(3+5 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Truth Table for a Three-State Buffer

A three-state (or tri-state) buffer acts as a standard buffer but includes a third operational state: **High-Impedance** (Z). It is controlled by an Enable line (E).

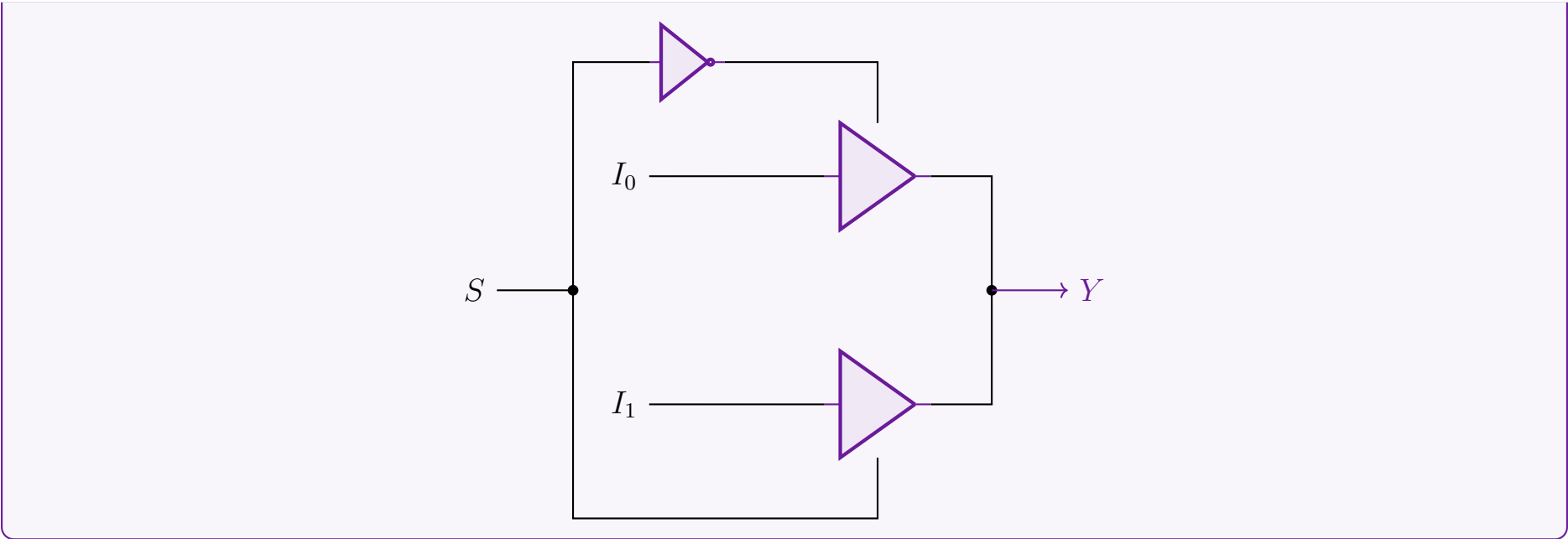
- When $E = 1$ (Active), the buffer acts normally, passing the input A directly to the output Y .
- When $E = 0$ (Inactive), the output is disconnected from the input, placing the output in a High-Impedance state (Z), effectively acting as an open circuit.

Inputs		Output
Enable (E)	Data (A)	Y
0	X (Don't Care)	Z (High-Impedance)
1	0	0
1	1	1

2. Construction of a 2-to-1 MUX using Three-State Buffers

A 2-to-1 Multiplexer routes one of two input signals (I_0 or I_1) to a single output (Y) based on the value of a Select line (S). Using three-state buffers, we can construct this by connecting the outputs of two buffers together. To avoid bus contention (short circuits), we must ensure that **only one buffer is enabled at any given time**.

- When $S = 0$:** The inverted signal $\overline{S} = 1$ enables the top buffer. Input I_0 is passed to Y . The bottom buffer is in state Z .
- When $S = 1$:** The direct signal $S = 1$ enables the bottom buffer. Input I_1 is passed to Y . The top buffer is in state Z .



Q17. An 8×1 multiplexer has inputs A, B, C, D where inputs A, B , and C are connected to selection inputs S_2, S_1 , and S_0 respectively. The data inputs I_0 through I_7 are: $I_1 = I_2 = 0$; $I_3 = I_7 = 1$; $I_4 = I_5 = D$; $I_0 = I_6 = D'$. Determine the Boolean function that the multiplexer implements. **(15 marks)** [CSE 205 | L-2/T-1 | 2013–14]

Answer

1. Characteristic Equation of the 8×1 Multiplexer

An 8×1 multiplexer outputs a Boolean function determined by the logical sum of the products of its selection line minterms and their corresponding data inputs. The general output equation F is given by:
$$F(S_2, S_1, S_0) = \sum_{k=0}^7 m_k \cdot I_k$$
where m_k represents the k -th minterm of the selection variables.
Given the selection inputs $S_2 = A, S_1 = B$, and $S_0 = C$, we can map the given data inputs I_0 through I_7 as follows:

Selection Lines			Data Input	Assigned Value
$A (S_2)$	$B (S_1)$	$C (S_0)$	I_k	$F(A, B, C, D)$
0	0	0	I_0	D'
0	0	1	I_1	0
0	1	0	I_2	0
0	1	1	I_3	1
1	0	0	I_4	D
1	0	1	I_5	D
1	1	0	I_6	D'
1	1	1	I_7	1

2. Algebraic Formulation

Substituting the given data inputs into the characteristic equation:
$$\begin{aligned} F(A, B, C, D) &= A'B'C'(D') + A'B'C(0) + A'BC'(0) + A'BC(1) \\ &\quad + AB'C'(D) + AB'C(D) + ABC'(D') + ABC(1) \\ &= A'B'C'D' + 0 + 0 + A'BC + AB'C'D + AB'CD + ABC'D' + ABC \\ &= A'B'C'D' + A'BC + AB'C'D + AB'CD + ABC'D' + ABC \end{aligned}$$

3. Conversion to Canonical Sum of Minterms (SOP)

To find the standard minterms Σm , we expand the terms missing the variable D by multiplying them by $(D + D') = 1$ (Complement Law):
$$\begin{aligned} A'BC &= A'BC(D + D') = A'BCD + A'BCD' && \text{(Minterms 7, 6)} \\ ABC &= ABC(D + D') = ABCD + ABCD' && \text{(Minterms 15, 14)} \end{aligned}$$
The remaining terms are already standard minterms:

- $A'B'C'D' \rightarrow m_0$

197

- $AB'C'D \rightarrow m_9$
- $AB'CD \rightarrow m_{11}$
- $ABC'D' \rightarrow m_{12}$

Thus, the canonical Boolean function implemented by the multiplexer is:

$$F(A, B, C, D) = \Sigma m(0, 6, 7, 9, 11, 12, 14, 15)$$

4. Logic Minimization using Karnaugh Map

We map the active minterms onto a 4-variable K-map to determine the simplified Boolean expression.

		CD			
		00	01	11	10
AB	00	1			
	01			1	1
	11	1		1	1
	10		1	1	

Group Extractions:

- **Red / 2×2 Block** (m_6, m_7, m_{14}, m_{15}): Variables A and D change, leaving BC .
- **Green / Horizontal Pair** (m_9, m_{11}): Variable C changes, leaving $AB'D$.
- **Blue / Edge Pair** (m_{12}, m_{14}): Variable C changes, leaving ABD' .
- **Yellow / Isolated Cell** (m_0): No variables change, leaving $A'B'C'D'$.

Final Minimal Boolean Function:

$$F(A, B, C, D) = BC + AB'D + ABD' + A'B'C'D'$$

Q18. Construct a 16×1 multiplexer with two 8×1 and one 2×1 multiplexers. (5 marks)

[CSE 205 | L-2/T-1 | 2013–14]

Answer

1. Design Strategy and Logic Formulation

A 16×1 multiplexer routes one of 16 data inputs (I_0 to I_{15}) to a single output (Y) based on a 4-bit selection code (S_3, S_2, S_1, S_0), where S_3 is the Most Significant Bit (MSB).

Since we have 8×1 multiplexers available (which accept 3 selection bits and 8 data inputs), we can partition the 16 inputs into two groups of 8:

- **Lower Half** (I_0 to I_7): Selected when the MSB $S_3 = 0$.
- **Upper Half** (I_8 to I_{15}): Selected when the MSB $S_3 = 1$.

Configuration Step-by-Step:

- (a) **First Stage (8×1 Multiplexers):** The three Least Significant Bits (LSBs) S_2, S_1, S_0 are applied simultaneously to the selection lines of both 8×1 multiplexers.

$$F_1 = \sum_{k=0}^7 m_k(S_2, S_1, S_0) \cdot I_k \quad (\text{Output of top MUX})$$

$$F_2 = \sum_{k=0}^7 m_k(S_2, S_1, S_0) \cdot I_{k+8} \quad (\text{Output of bottom MUX})$$

- (b) **Second Stage (2×1 Multiplexer):** The outputs F_1 and F_2 serve as the data inputs to the 2×1 multiplexer. The MSB S_3 acts as the selection line for this final multiplexer to toggle between the lower and upper data banks.

$$Y = \overline{S_3} \cdot F_1 + S_3 \cdot F_2 \quad (\text{Multiplexer logical expansion})$$

2. Truth Table Summary

Selection Lines				Active MUX in 1st Stage	Selected Output (Y)
S ₃	S ₂	S ₁	S ₀		
0	0	0	0	MUX 1 (Top)	I ₀
⋮	⋮	⋮	⋮		⋮
0	1	1	1		I ₇
1	0	0	0	MUX 2 (Bottom)	I ₈
⋮	⋮	⋮	⋮		⋮
1	1	1	1		I ₁₅

3. Logic Circuit Implementation

Q19. Implement $F = \Sigma(3, 5, 7, 10, 11, 13, 14, 15)$ using a 16-to-1 MUX. (10 marks)

[CSE 205 | L-2/T-1 | 2012–13]

Answer

1. MUX Implementation Strategy

A 16-to-1 Multiplexer has 4 selection lines (S_3, S_2, S_1, S_0) and 16 data inputs (I_0 to I_{15}). To implement a 4-variable Boolean function $F(A, B, C, D)$ directly using a 16-to-1 MUX, we follow a straightforward mapping technique:

- Connect the 4 Boolean variables directly to the 4 selection lines: $S_3 = A$, $S_2 = B$, $S_1 = C$, and $S_0 = D$.
- Connect the data inputs I_i corresponding to the active minterms (present in the function) to **Logic 1** (V_{cc}).
- Connect the data inputs I_i corresponding to the absent minterms to **Logic 0** (Ground).

Given the function:

$$F(A, B, C, D) = \Sigma(3, 5, 7, 10, 11, 13, 14, 15)$$

The active minterms are $m_3, m_5, m_7, m_{10}, m_{11}, m_{13}, m_{14}, m_{15}$.

2. Input Mapping Table

Minterm	$A(S_3)$	$B(S_2)$	$C(S_1)$	$D(S_0)$	Data Input	Assigned Value
m_0, m_1, m_2	Various combinations				I_0, I_1, I_2	0 (GND)
m_3	0	0	1	1	I_3	1 (V_{cc})
m_4	0	1	0	0	I_4	0 (GND)
m_5	0	1	0	1	I_5	1 (V_{cc})
m_6	0	1	1	0	I_6	0 (GND)
m_7	0	1	1	1	I_7	1 (V_{cc})
m_8, m_9	Various combinations				I_8, I_9	0 (GND)
m_{10}, m_{11}	Various combinations				I_{10}, I_{11}	1 (V_{cc})
m_{12}	1	1	0	0	I_{12}	0 (GND)
m_{13}, m_{14}, m_{15}	Various combinations				I_{13}, I_{14}, I_{15}	1 (V_{cc})

3. Circuit Diagram

Q20. Can the function $F = \Sigma(3, 5, 7, 10, 11, 13, 14, 15)$ be implemented using only one 4×1 MUX? If yes, show how; if not, explain why. (10 marks) [CSE 205 | L-2/T-1 | 2012–13]

Answer

Conclusion: Yes, the function $F(A, B, C, D) = \Sigma(3, 5, 7, 10, 11, 13, 14, 15)$ **can** be implemented using strictly one 4×1 multiplexer (MUX) and absolutely no additional logic gates. This is achieved by strategically choosing variables C and D as the selection lines.

1. Theoretical Foundation
A 4×1 MUX implements any logic function conforming to its characteristic equation:

$$Y = S'_1S'_0I_0 + S'_1S_0I_1 + S_1S'_0I_2 + S_1S_0I_3$$

where S_1, S_0 are the select lines and I_0, I_1, I_2, I_3 are the data inputs.
To implement a 4-variable function using only one 4×1 MUX (which has 2 select lines), we must map two variables to the select lines. The remaining two variables will dictate the logic required at the data inputs. To avoid using external logic gates (such as AND/OR/NOT gates), the data inputs must resolve entirely to constants (0 or 1) or single input variables (uncomplemented or complemented).

2. Mathematical Derivation (Shannon's Expansion)
We apply Shannon's Expansion Theorem to factor out the variables C and D , setting them as our select variables ($S_1 = C, S_0 = D$). First, let us expand the given function into its canonical sum of minterms:

$$\begin{aligned} F(A, B, C, D) &= \sum m(3, 5, 7, 10, 11, 13, 14, 15) \\ &= m_3 + m_5 + m_7 + m_{10} + m_{11} + m_{13} + m_{14} + m_{15} \\ &= A'B'CD + A'BC'D + A'BCD + AB'CD' + AB'CD + ABC'D + ABCD' + ABCD \end{aligned}$$

Next, we group the terms based on the four possible combinations of C and D :

$$\begin{aligned} F &= C'D'(0) \\ &+ C'D(A'B + AB) \\ &+ CD'(AB' + AB) \\ &+ CD(A'B' + A'B + AB' + AB) \end{aligned}$$

(No minterms end in $C = 0, D = 0$)
(From m_5 and m_{13})
(From m_{10} and m_{14})
(From m_3, m_7, m_{11}, m_{15})

Now, we systematically simplify the expressions for each MUX input using Boolean algebra:

- **For $C'D'$ (I_0):**
The coefficient is 0.
 $\therefore I_0 = 0$

- **For $C'D$ (I_1):**

$$\begin{aligned} I_1 &= A'B + AB \\ &= B(A' + A) \\ &= B(1) \end{aligned}$$

(Distributive Law)
(Complement Law: $A' + A = 1$)
(Identity Law)

$\therefore I_1 = B$

- **For CD' (I_2):**

$$\begin{aligned} I_2 &= AB' + AB \\ &= A(B' + B) \\ &= A(1) \end{aligned}$$

(Distributive Law)
(Complement Law: $B' + B = 1$)
(Identity Law)

$\therefore I_2 = A$

- **For CD (I_3):**

$$\begin{aligned} I_3 &= A'B' + A'B + AB' + AB \\ &= A'(B' + B) + A(B' + B) \\ &= A'(1) + A(1) \\ &= A' + A \end{aligned}$$

(Distributive Law)
(Complement Law)
(Identity Law)
(Complement Law)

$\therefore I_3 = 1$

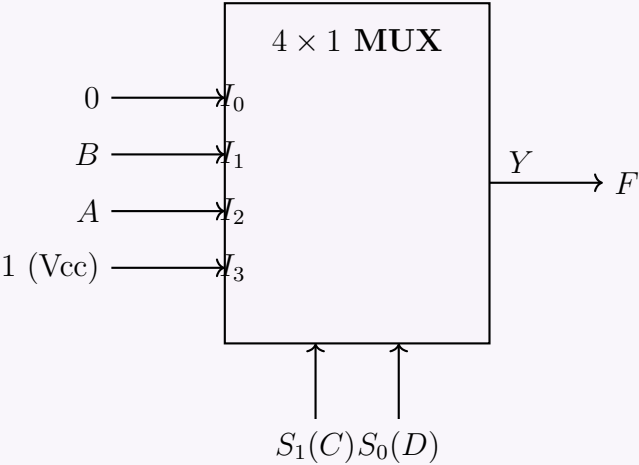
Since all derived inputs (0, B , A , and 1) can be obtained via direct wiring without any logic gates, the implementation is valid.

3. Implementation Table

The following table maps the state of the select lines (C, D) to the corresponding minterms and the necessary data inputs.

S_1 (C)	S_0 (D)	Data Input	Relevant Minterms in F	Simplified Input
0	0	I_0	None	0
0	1	I_1	$m_5(0101), m_{13}(1101)$	B
1	0	I_2	$m_{10}(1010), m_{14}(1110)$	A
1	1	I_3	$m_3(0011), m_7(0111), m_{11}(1011), m_{15}(1111)$	1

4. Circuit Diagram



Note: If we had chosen A and B as the select lines instead, the inputs I_0 through I_3 would have required complex expressions in terms of C and D (e.g., $I_3 = C + D$), which would necessitate additional logic gates. The choice of C and D perfectly avoids this constraint.

Q21. Implement the following function F using one 4×1 multiplexer and basic gates:

$$F(A, B, C, D) = \Sigma(1, 3, 4, 11, 12, 13, 14, 15)$$

(15 marks)

[CSE 205 | L-2/T-1 | 2011–12]

Answer

1. Multiplexer Sizing and Variable Selection

To implement a 4-variable Boolean function $F(A, B, C, D)$ using a 4×1 Multiplexer (MUX), we need to allocate the variables strategically. A 4×1 MUX has:

- 4 data inputs (I_0, I_1, I_2, I_3)
- 2 select lines (S_1, S_0)

We will connect the two most significant variables, A and B , to the select lines:

$$S_1 = A, \quad S_0 = B$$

The remaining variables, C and D , will be used to generate the data inputs I_0, I_1, I_2 , and I_3 .

2. Implementation Table

We map the given minterms $\Sigma(1, 3, 4, 11, 12, 13, 14, 15)$ into an implementation table based on the permutations of C and D for each combination of A and B . Minterms present in the function are highlighted in **bold**.

Data Variables	I ₀	I ₁	I ₂	I ₃
Select (AB)	(00)	(01)	(10)	(11)
$\bar{C}\bar{D}$ (00)	0	4	8	12
$\bar{C}D$ (01)	1	5	9	13
$C\bar{D}$ (10)	2	6	10	14
CD (11)	3	7	11	15
Boolean Sum	$I_0 = \bar{C}D + CD$	$I_1 = \bar{C}\bar{D}$	$I_2 = CD$	$I_3 = \bar{C}\bar{D} + \bar{C}D + C\bar{D} + CD$

3. Mathematical Derivation of Data Inputs

We simplify the logic expressions for each data input using fundamental Boolean algebra laws:

$$\begin{aligned} I_0 &= \bar{C}D + CD \\ &= D(\bar{C} + C) && \text{(Distributive Law)} \\ &= D(1) && \text{(Complementarity Law: } X + \bar{X} = 1) \\ &= D && \text{(Identity Law)} \end{aligned}$$

$$I_1 = \bar{C}\bar{D} \quad \text{(No further simplification needed)}$$

$$I_2 = CD \quad \text{(No further simplification needed)}$$

$$\begin{aligned} I_3 &= \bar{C}\bar{D} + \bar{C}D + C\bar{D} + CD \\ &= \bar{C}(\bar{D} + D) + C(\bar{D} + D) && \text{(Distributive Law)} \\ &= \bar{C}(1) + C(1) && \text{(Complementarity Law)} \\ &= \bar{C} + C \\ &= 1 && \text{(Complementarity Law)} \end{aligned}$$

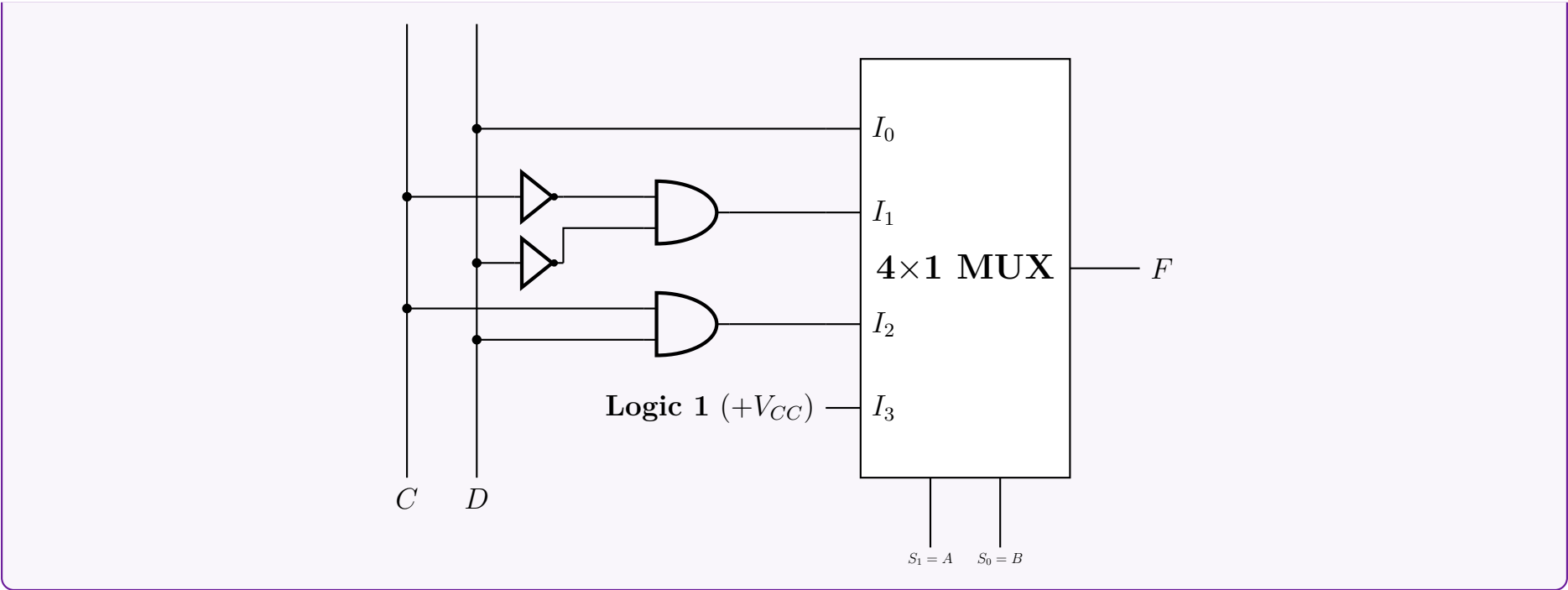
Summary of MUX Inputs:

- $I_0 = D$
- $I_1 = \bar{C}\bar{D}$ (Implemented using an AND gate with inverted inputs)
- $I_2 = CD$ (Implemented using an AND gate)
- $I_3 = 1$ (Connected to Logic HIGH / $+V_{CC}$)

4. Logic Circuit Diagram

Below is the complete hardware implementation using standard basic logic gates and the 4×1 MUX block:

202



Custom Combinational Circuit Design

Q1. Using a 4-bit binary adder and XOR gates, design a circuit which takes two 4-bit BCD inputs A and B and returns: $A + 1$ if input is 00; A' if input is 01; $A' + 1$ if input is 10; A if input is 11. **(13 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Problem Analysis & Interpretation

The problem requires designing an arithmetic logic circuit using only a **4-bit binary adder** and **XOR gates**. We are given a 4-bit data input A . *Note on Input B :* While the problem description mentions B as a 4-bit BCD input, the operational modes provided (00, 01, 10, 11) correspond to a 2-bit control space. Therefore, we interpret that the two least significant bits of B (let us denote them as B_1 and B_0) act as the control inputs to select the operation performed on A . A standard 4-bit binary adder computes the function:

$$F = X + Y + C_{in}$$

where X and Y are 4-bit data inputs, C_{in} is the carry-in bit, and F is the 4-bit sum. Our objective is to determine the appropriate signals for X , Y , and C_{in} using only A , B_1 , B_0 , and XOR gates to satisfy the four conditions.

2. Target Operations and Adder Input Mapping

Let us map the desired outputs to the adder's parameters. To either pass A directly or invert it (A'), we can use the fundamental property of the XOR gate:

$$A \oplus 0 = A \quad \text{and} \quad A \oplus 1 = A'$$

We can fix the second adder input to zero ($Y = 0$). We introduce a control signal C_X such that $X = A \oplus C_X$.

Control Input		Desired Output (F)	Required Adder Inputs		
B_1	B_0		X (Data)	Y (Data)	C_{in} (Carry)
0	0	$A + 1$	A (so $C_X = 0$)	0	1
0	1	A'	A' (so $C_X = 1$)	0	0
1	0	$A' + 1$	A' (so $C_X = 1$)	0	1
1	1	A	A (so $C_X = 0$)	0	0

3. Boolean Derivation for Control Signals

From the table above, we extract the boolean expressions for C_X and C_{in} in terms of B_1 and B_0 :

- Deriving C_{in} :** Observing the C_{in} column, the values are $\{1, 0, 1, 0\}$ corresponding to $B_0 = \{0, 1, 0, 1\}$.

$$C_{in} = \overline{B_0}$$

Since we are constrained to use *only* XOR gates, we can synthesize a NOT gate by XORing with Logic 1 (V_{CC}):

$$C_{in} = B_0 \oplus 1$$

- Deriving C_X :** Observing the required C_X values, we have $C_X = \{0, 1, 1, 0\}$. This is the exact truth table for an XOR operation between B_1 and B_0 .

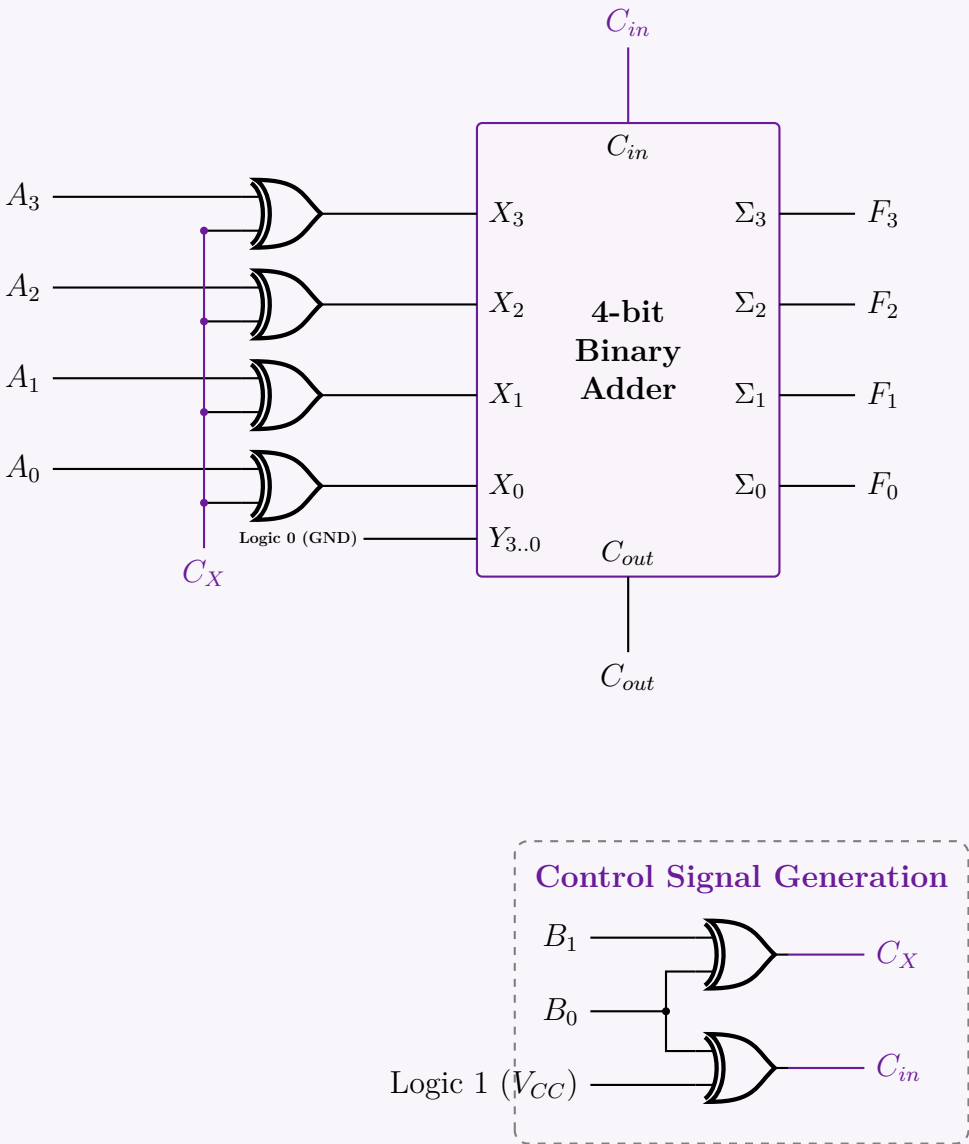
$$C_X = B_1 \oplus B_0$$

Thus, the inputs to the 4-bit adder will be:

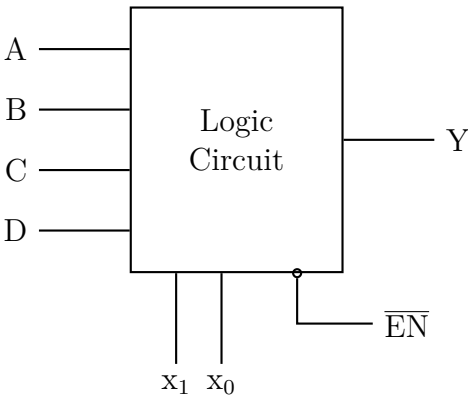
$$X_i = A_i \oplus C_X = A_i \oplus (B_1 \oplus B_0) \quad \text{for } i \in \{0, 1, 2, 3\}$$
$$Y_i = 0 \quad (\text{Grounded})$$
$$C_{in} = B_0 \oplus 1$$

4. Circuit Implementation

The full schematic utilizes one 4-bit binary adder block and six XOR gates (four for the data path, two for the control logic). Net labels (C_X and C_{in}) are utilized for clean routing.



Q2. Design a Binary-to-Excess-3 converter using a logic circuit block with inputs A, B, C, D , control lines x_1, x_0 , and active-low $\overline{EN}$. Output Y : if $x_1x_0 = 00$, $A \rightarrow Y$; if 01 , $B \rightarrow Y$; if 10 , $C \rightarrow Y$; if 11 , $D \rightarrow Y$.



(8 marks)

[CSE 205 | L-2/T-1 | 2016–17]

Answer

1. Component Analysis & Problem Formulation

The given logic circuit block functions as a **4-to-1 Multiplexer (MUX)** with an active-low enable ($\overline{EN}$).

- **Data Inputs:** A, B, C, D correspond to select line conditions $x_1x_0 = 00, 01, 10, 11$ respectively.
- **Select Inputs:** x_1, x_0 determine which data input is routed to output Y .
- **Enable Input:** $\overline{EN}$ must be tied to Logic 0 (Ground) for normal operation.

To design a **4-bit Binary-to-Excess-3 Converter**, we require four such blocks—one for each output bit of the Excess-3 code (E_3, E_2, E_1, E_0). We map the Binary Coded Decimal (BCD) input bits B_3, B_2, B_1, B_0 as follows:

- We connect the two most significant bits to the select lines of all blocks: $x_1 = B_3$ and $x_0 = B_2$.
- The block inputs A, B, C, D for each bit will be derived as Boolean functions of the remaining least significant bits: B_1 and B_0 .

2. Truth Table & Input Derivation

Excess-3 is formed by adding 3 (0011_2) to a valid BCD input (0 to 9). The input states 10 to 15 are invalid in BCD and are treated as **Don't Cares (X)** to simplify our hardware logic.

BCD Input				Excess-3 Output			
$B_3(x_1)$	$B_2(x_0)$	B_1	B_0	E_3	E_2	E_1	E_0
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	X	X	X	X	X	X

By grouping the truth table based on the select lines (B_3B_2), we inspect the behavior of each output with respect to B_1 and B_0 :

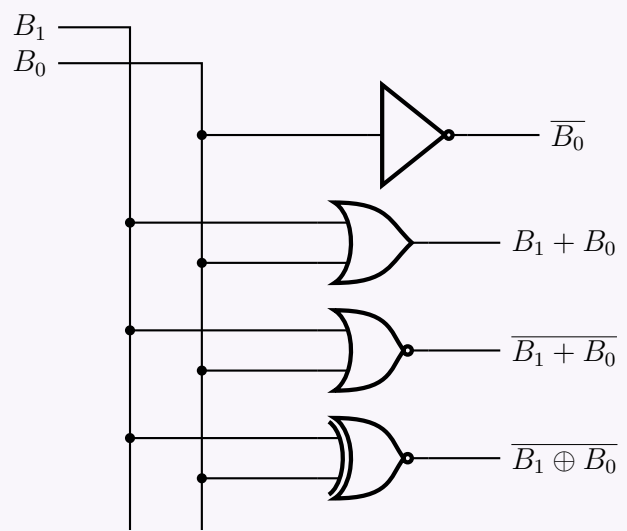
- **For E_0 :** The pattern is alternating (1,0,1,0) for all valid segments. This directly evaluates to the NOT function: $\overline{B_0}$.
- **For E_1 :** The pattern is symmetric (1,0,0,1) for valid segments. This matches the XNOR function: $\overline{B_1 \oplus B_0}$.
- **For E_2 :**
 - When 00, output is (0,1,1,1) $\implies$ OR function: $B_1 + B_0$
 - When 01, output is (1,0,0,0) $\implies$ NOR function: $\overline{B_1 + B_0}$
 - When 10, valid outputs are (0,1), matching directly to B_0 .
- **For E_3 :**
 - When 00, output is always (0,0,0,0) $\implies$ Logic 0
 - When 01, output is (0,1,1,1) $\implies$ OR function: $B_1 + B_0$
 - When 10, valid outputs are (1,1) $\implies$ Logic 1

Mapping the Don't Care states optimally to reuse existing signals, we formulate the final wiring table for our Logic Blocks:

MUX Input	Select (B_3B_2)	E_3 Block	E_2 Block	E_1 Block	E_0 Block
A	00	0	$B_1 + B_0$	$\overline{B_1 \oplus B_0}$	$\overline{B_0}$
B	01	$B_1 + B_0$	$\overline{B_1 + B_0}$	$\overline{B_1 \oplus B_0}$	$\overline{B_0}$
C	10	1	B_0	$\overline{B_1 \oplus B_0}$	$\overline{B_0}$
D	11	0	0	$\overline{B_1 \oplus B_0}$	$\overline{B_0}$

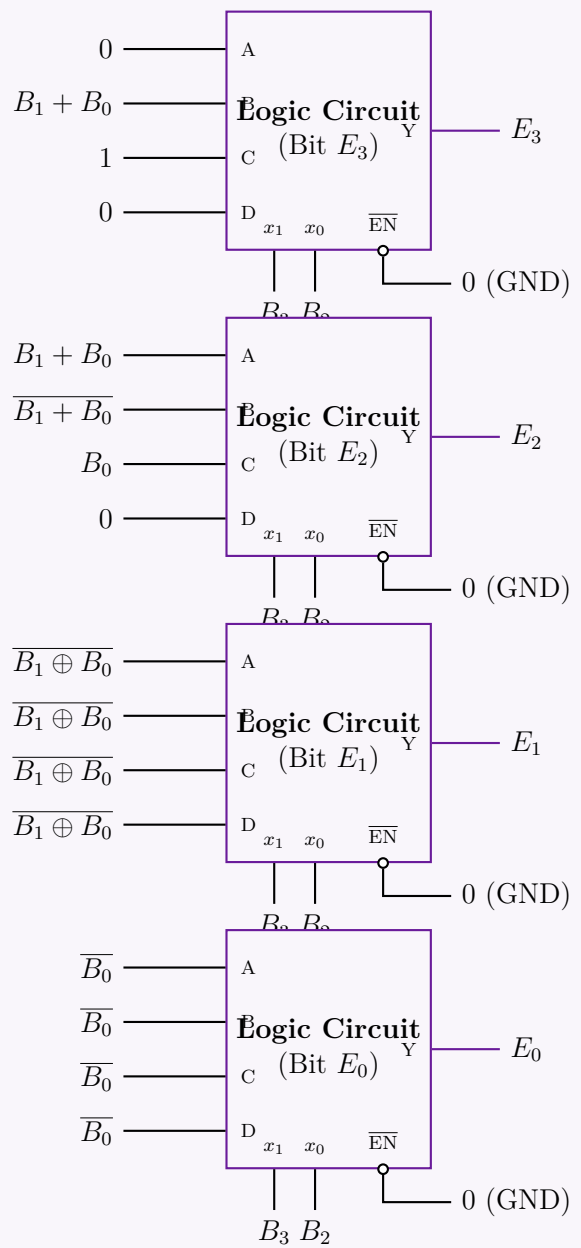
3. Signal Generation Sub-circuit

To avoid clutter in the main routing, we independently generate the intermediate logic functions from B_1 and B_0 using standard gates.



4. Complete System Diagram

The final implementation utilizing four instances of the given Logic Circuit block. The generated signals from the sub-circuit are applied directly to the A, B, C, D input nodes.



Q3. Design a combinational circuit with 3 inputs x, y, z and 3 outputs A, B, C . When the binary input is 0, 1, 2, or 3, the binary output is two greater than the input. When the binary input is 4, 5, 6, or 7, the binary output is three less than the input. **(18 marks)**
[CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Component Analysis & Problem Formulation

We are tasked with designing a combinational circuit with 3 inputs (x, y, z) and 3 outputs (A, B, C). The input x is the Most Significant Bit (MSB), and z is the Least Significant Bit (LSB). The operational conditions are defined as follows:

- **Condition 1:** If the input $\in \{0, 1, 2, 3\}$, the output is **Input + 2**.
- **Condition 2:** If the input $\in \{4, 5, 6, 7\}$, the output is **Input - 3**.

2. Truth Table Derivation

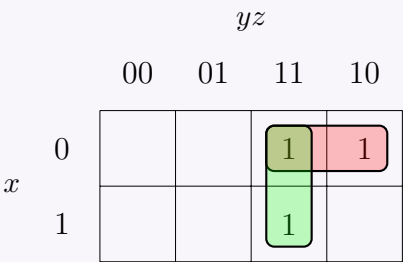
We compute the expected decimal outputs and map them to their corresponding 3-bit binary equivalents to populate the truth table.

Condition	Input			Input (Decimal)	Output (Decimal)	Output		
	<i>x</i>	<i>y</i>	<i>z</i>			<i>A</i>	<i>B</i>	<i>C</i>
Input +2	0	0	0	0	0 + 2 = 2	0	1	0
	0	0	1	1	1 + 2 = 3	0	1	1
	0	1	0	2	2 + 2 = 4	1	0	0
	0	1	1	3	3 + 2 = 5	1	0	1
Input -3	1	0	0	4	4 - 3 = 1	0	0	1
	1	0	1	5	5 - 3 = 2	0	1	0
	1	1	0	6	6 - 3 = 3	0	1	1
	1	1	1	7	7 - 3 = 4	1	0	0

3. Karnaugh Maps and Boolean Optimization

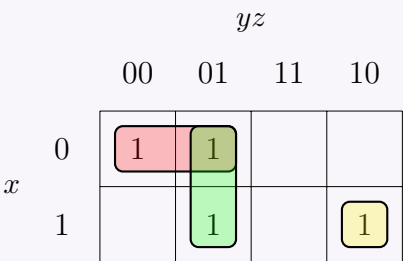
Using the truth table, we extract the minterms for each output variable and minimize the Boolean expressions using 3-variable Karnaugh Maps.

Output A: Minterms $\sum m(2, 3, 7)$



$$A = \bar{x}y + yz$$
$$= y(\bar{x} + z) \quad \text{(Optional factoring via Distributive Law)}$$

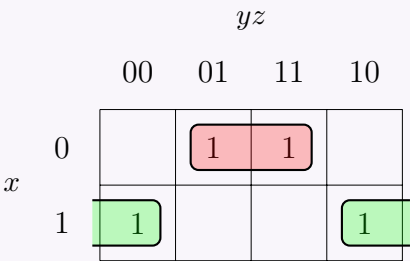
Output B: Minterms $\sum m(0, 1, 5, 6)$



$$B = \bar{x}\bar{y} + \bar{y}z + xy\bar{z}$$

(Note: Minterm m_6 forms an isolated cell, resulting in the 3-literal term $xy\bar{z}$).

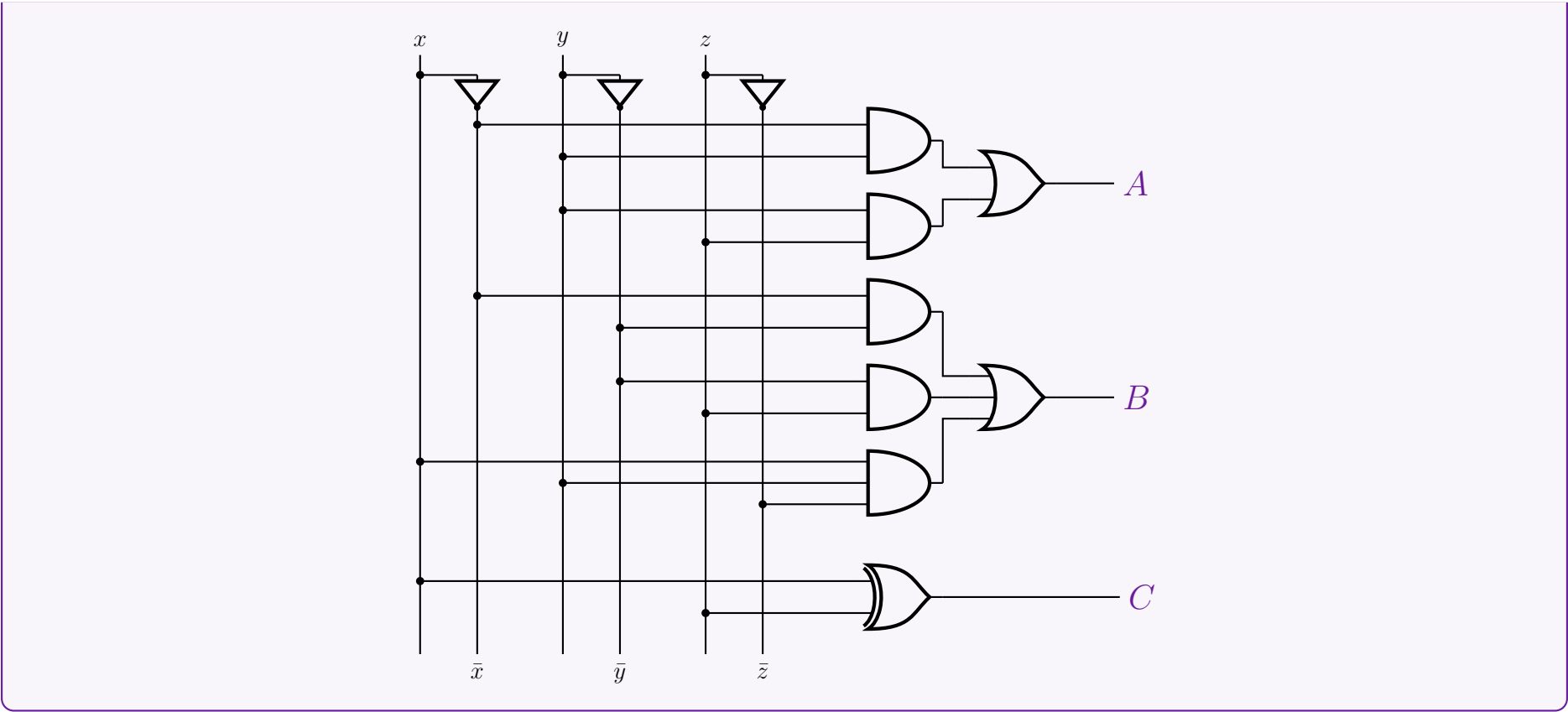
Output C: Minterms $\sum m(1, 3, 4, 6)$



$$C = \bar{x}z + x\bar{z}$$
$$= x \oplus z \quad \text{(Definition of XOR)}$$

4. Logic Circuit Implementation

Based on the optimized expressions, the combinational logic circuit is constructed below using standard logic gates. Input buses for x, y, z and their complements are provided for clean signal routing.



BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Flip-Flops, Latches & Race-Around Problems

SR, JK, D, T Flip-Flops, Excitation Tables, Race Conditions

S5 Flip-Flops, Latches & Race-Around Problems

SR Latch & SR/JK Undefined Condition

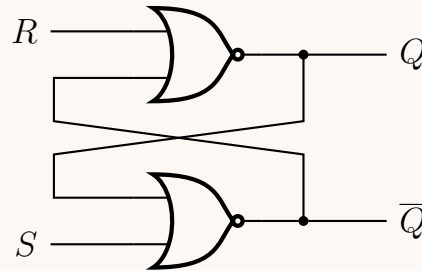
Q1. Discuss why the condition $S = R = 1$ leads to an unstable condition for an SR latch. (7 marks) [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Introduction to the SR Latch

The most common implementation of an Active-High Set-Reset (SR) latch utilizes two cross-coupled NOR gates. The defining characteristic of any fundamental latch is the principle of **complementarity**: the two outputs, Q and $\bar{Q}$, must always be logical inverses of each other.

Applying the input condition $S = 1$ and $R = 1$ is universally restricted (often labeled as "Forbidden" or "Invalid") because it violates this fundamental rule and sets up a catastrophic race condition if the inputs change simultaneously.



2. Mathematical Proof of the Complementarity Violation

Let us evaluate the Boolean logic of the cross-coupled NOR gates when $S = 1$ and $R = 1$. The standard characteristic equations for the NOR-based SR latch are:

$$Q_{next} = \overline{R + \bar{Q}_{prev}} \quad (\text{NOR operation on upper gate})$$

$$\bar{Q}_{next} = \overline{S + Q_{prev}} \quad (\text{NOR operation on lower gate})$$

Substituting $S = 1$ and $R = 1$ into the equations:

$$Q_{next} = \overline{1 + \bar{Q}_{prev}}$$

$$Q_{next} = \bar{1} \quad (\text{Applying Boolean Annulment/Domination Law: } 1 + x = 1)$$

$$Q_{next} = 0$$

Simultaneously, for the second gate:

$$\bar{Q}_{next} = \overline{1 + Q_{prev}}$$

$$\bar{Q}_{next} = \bar{1} \quad (\text{Applying Boolean Annulment/Domination Law: } 1 + x = 1)$$

$$\bar{Q}_{next} = 0$$

Result: Both outputs are forced to 0 ($Q = 0$ and $\bar{Q} = 0$). This directly violates the rule that Q and $\bar{Q}$ must be complements ($Q = \neg \bar{Q}$).

3. The Critical Race Condition (Instability)

While $Q = 0$ and $\bar{Q} = 0$ is a logic violation, the actual **instability** occurs when the inputs transition away from this state. Assume the circuit is in the $S = R = 1$ state (with $Q = \bar{Q} = 0$), and the user suddenly drops both inputs to $S = 0$ and $R = 0$ (the "Hold" state) simultaneously.

Evaluating the transition logically:

$$Q_{next} = \overline{0 + \bar{Q}_{prev}} = \overline{0 + 0} = \bar{0} = 1$$

$$\bar{Q}_{next} = \overline{0 + Q_{prev}} = \overline{0 + 0} = \bar{0} = 1$$

- Both NOR gates sense a 0 on both of their inputs and thus both attempt to output a 1 simultaneously.
- As soon as the outputs begin to transition to 1, the feedback loops cross-feed these 1s into the opposite gates.
- Because $1 + 0 = 1$, the outputs are immediately forced back to 0.

This creates a high-frequency oscillation or a **critical race condition**. Ultimately, because physical logic gates possess microscopic variations in propagation delay, one gate will switch slightly faster than the other. The faster gate will lock its output to 1, forcing the slower gate to remain at 0.

S	R	Q	$\bar{Q}$	Operational State
0	0	Q_{prev}	$\bar{Q}_{prev}$	Hold Memory
0	1	0	1	Reset
1	0	1	0	Set
1	1	0	0	Invalid / Unstable Race Condition

Conclusion: The state is deemed unstable because the final latched state ($Q = 1$ or $Q = 0$) is physically unpredictable and entirely dependent on manufacturing imperfections and thermal noise (metastability). Consequently, digital design explicitly prohibits the $S = R = 1$ condition.

Q2. Consider the following transition table:

$y \setminus x_1x_2$	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Design the circuit with NAND SR latches and gates. (10 marks)

[CSE 205 | L-2/T-1 | 2016–17]

Answer

1. State Excitation Table Formulation

To design the sequential circuit, we first need to determine the required inputs for the SR latch to achieve the specified state transitions. The standard excitation table for an SR latch determines the required Set (S) and Reset (R) signals to transition from a present state $y(t)$ to a next state $y(t + 1)$:

- $0 \rightarrow 0 \implies S = 0, R = X$ (Don't care)
- $0 \rightarrow 1 \implies S = 1, R = 0$
- $1 \rightarrow 0 \implies S = 0, R = 1$
- $1 \rightarrow 1 \implies S = X, R = 0$

Applying these rules to the given transition table, we formulate the combined excitation table:

Inputs & Present State			Next State	Required Latch Inputs	
y	x_1	x_2	$Y(y_{next})$	S	R
0	0	0	0	0	X
0	0	1	0	0	X
0	1	1	1	1	0
0	1	0	0	0	X
1	0	0	0	0	1
1	0	1	1	X	0
1	1	1	1	X	0
1	1	0	1	X	0

2. Karnaugh Map Minimization

We now plot the required values for S and R onto Karnaugh maps to extract the minimized Boolean expressions.

K-Map for S

		x_1x_2			
		00	01	11	10
y	0	0	0	1	0
	1	0	X	X	X

$S = x_1x_2$

K-Map for R

		x_1x_2			
		00	01	11	10
y	0	X	X	0	X
	1	1	0	0	0

$R = \overline{x_1} \cdot \overline{x_2}$

3. Adaptation for NAND SR Latch

The problem specifically requests the use of **NAND SR latches**. A fundamental cross-coupled NAND latch operates with active-low excitation inputs, mathematically denoted as $\overline{S}$ and $\overline{R}$. Therefore, we must invert the derived S and R Boolean equations to feed into the NAND latch:

$\overline{S} = \overline{x_1 \cdot x_2}$

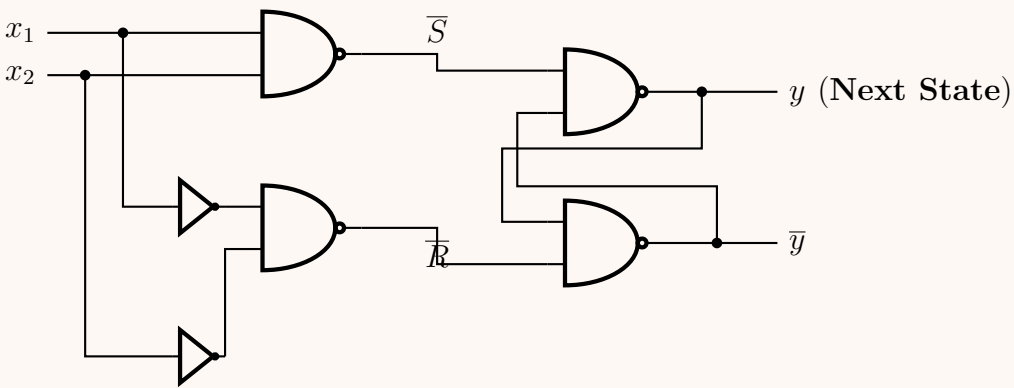
(Directly implementable with a 2-input NAND gate)

$\overline{R} = \overline{\overline{x_1} \cdot \overline{x_2}}$

(Implementable with a 2-input NAND gate fed by inverted inputs)

4. Hardware Implementation (Logic Circuit)

The synthesized sequential logic circuit uses standard logic gates for the combinatorial excitation logic and a cross-coupled dual-NAND gate configuration for the fundamental memory latch.



Q3. Write the excitation table for a SR latch constructed with NOR gates. (4 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

1. Overview of the NOR-Based SR Latch

An SR (Set-Reset) latch constructed with cross-coupled NOR gates is an active-high fundamental memory element. The excitation table of a sequential circuit specifies the required input values (S and R) to cause a desired transition from a given Present State, $Q(t)$, to a desired Next State, $Q(t + 1)$. To derive the excitation table, we first observe the standard truth table (characteristic table) for the NOR-based SR Latch:

S	R	$Q(t + 1)$	Operation
0	0	$Q(t)$	Hold (No Change)
0	1	0	Reset
1	0	1	Set
1	1	X	Invalid / Restricted

2. Derivation of the Excitation Table

We analyze each possible state transition from $Q(t)$ to $Q(t + 1)$ to determine the necessary S and R conditions. We use "X" to denote a "Don't Care" condition (where the input can be either 0 or 1 without affecting the desired outcome).

- Transition $0 \rightarrow 0$:** The state must remain at 0. This can be achieved either by the "Hold" operation ($S = 0, R = 0$) or the "Reset" operation ($S = 0, R = 1$). **Result:** S must be 0, and R can be 0 or 1. Thus, $S = 0, R = X$.
- Transition $0 \rightarrow 1$:** The state must change from 0 to 1. This is explicitly the "Set" operation. **Result:** $S = 1, R = 0$.
- Transition $1 \rightarrow 0$:** The state must change from 1 to 0. This is explicitly the "Reset" operation. **Result:** $S = 0, R = 1$.
- Transition $1 \rightarrow 1$:** The state must remain at 1. This can be achieved either by the "Hold" operation ($S = 0, R = 0$) or the "Set" operation ($S = 1, R = 0$). **Result:** S can be 0 or 1, and R must be 0. Thus, $S = X, R = 0$.

3. Final Excitation Table

Summarizing the derivations above yields the formal excitation table for the SR latch:

State Transition		Required Inputs	
Present State $Q(t)$	Next State $Q(t + 1)$	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

4. Reference Circuit Schematic

For completeness, the logic diagram of the active-high NOR SR Latch corresponding to this table is shown below:

Q4. What do you understand by mechanical switch contact bounce? Describe how the $\overline{S}\overline{R}$ latch can eliminate mechanical switch contact bounce. (10 marks)

[CSE 205 | L-2/T-1 | 2014–15]

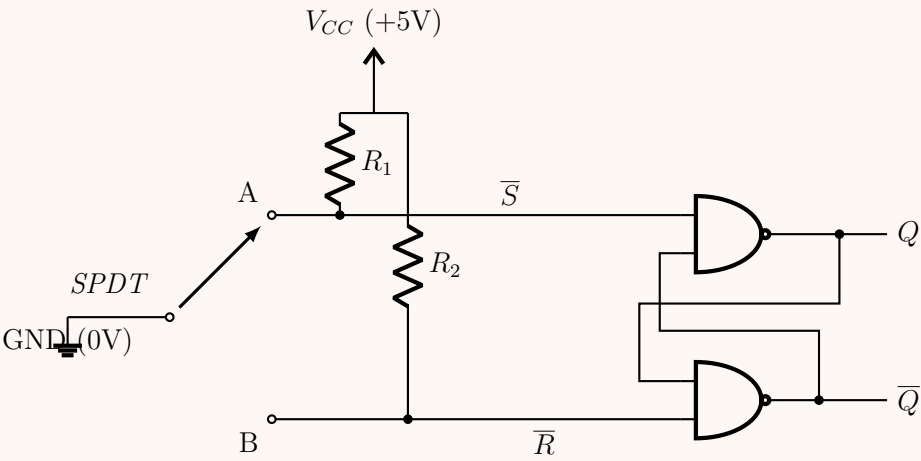
Answer

1. Mechanical Switch Contact Bounce: Definition

Mechanical contact bounce (or switch chatter) is a physical phenomenon that occurs when mechanical switches, relays, or push-buttons are actuated. When the metal contacts of a switch are forced together, their inherent elasticity and momentum cause them to rebound or "bounce" against each other several times before coming to a complete rest. In terms of electrical signals, instead of producing a single, clean step transition (e.g., from 0 V to 5 V), the switch generates a rapid, chaotic series of high and low voltage spikes during the settling time (typically 1 to 20 milliseconds). Because high-speed digital logic circuits operate in the nanosecond range, they will erroneously register these microscopic bounces as multiple distinct button presses, leading to severe logical errors in sequential circuits (like counters or state machines).

2. The $\overline{S}\overline{R}$ Latch Debouncing Circuit

To eliminate this hardware issue, an active-low Set-Reset ($\overline{S}\overline{R}$) latch constructed from two cross-coupled NAND gates is standardly used. A Single-Pole Double-Throw (SPDT) switch connects the logic circuit to ground, while pull-up resistors maintain a logical 1 on unconnected inputs.



3. Mathematical and Logical Operation

The fundamental principle of this debounce circuit relies on the **Memory (Hold) State** of the active-low $\overline{S}\overline{R}$ latch. The characteristic table of the NAND latch is:

$\overline{S}$	$\overline{R}$	Q_{next}	State Description
0	0	Invalid	(Restricted / Not Used)
0	1	1	Set
1	0	0	Reset
1	1	Q_{prev}	Hold (Memory) - Key to Debouncing

Let us trace the physical state sequence as the switch is flipped from terminal A to terminal B:

- Initial Steady State (At A):** The switch arm rests securely on A. Terminal A is grounded ($\overline{S} = 0$). Terminal B is pulled up by R_2 to V_{CC} ($\overline{R} = 1$).
Result: $Q = 1$ (Set State).
- Break Bounce (Leaving A):** As the user pushes the switch, the arm begins to separate from A. However, micro-bouncing causes it to rapidly break and reconnect with A. $\overline{S}$ oscillates between 1 (open) and 0 (connected), while $\overline{R}$ remains stably at 1.
Result: The inputs alternate between ($\overline{S} = 0, \overline{R} = 1 \implies Q = 1$) and the **Hold State** ($\overline{S} = 1, \overline{R} = 1 \implies Q$ stays 1). The output Q remains a clean, uninterrupted 1.
- Intermediate Travel Time:** The switch arm is completely suspended between A and B. Both terminals are physically open and pulled up to V_{CC} by their respective resistors.
Result: $\overline{S} = 1$ and $\overline{R} = 1$. The latch remains in the Hold State, ensuring Q remains 1.
- Make Bounce (Arriving at B):** The switch arm finally makes its first microscopic contact with B. At this exact instant, $\overline{R}$ drops to 0, while $\overline{S}$ is 1. The latch immediately Resets ($Q = 0$). As the arm bounces off B slightly before settling, $\overline{R}$ bounces back to 1.
Result: The inputs alternate between the Reset state ($\overline{S} = 1, \overline{R} = 0 \implies Q = 0$) and the Hold state ($\overline{S} = 1, \overline{R} = 1 \implies Q$ stays 0). Once again, the output Q is strictly locked to 0 upon the very first contact.

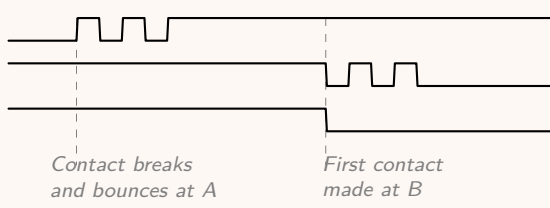
4. Timing Diagram Verification

The timing diagram visualizes how the latch absorbs the high-frequency chatter. Notice that the latch output transitions cleanly precisely upon the *first* contact with the destination terminal, completely ignoring all subsequent mechanical rebounds.

Switch Terminal A ($\overline{S}$)

Switch Terminal B ($\overline{R}$)

Latch Output (Q)



Conclusion:

By capitalizing on the memory retention capability ($\overline{S} = \overline{R} = 1$) of the cross-coupled logic gates, the SR latch perfectly masks mechanical uncertainties, guaranteeing a single, monolithic digital pulse per physical actuation.

213

Q5. Explain the undefined condition in SR flip-flops. How is this problem solved in JK flip-flops? (4+4 marks) [CSE 205 | L-2/T-1 | 2012–13]

Answer

1. The Undefined Condition in SR Flip-Flops

The SR (Set-Reset) flip-flop is a fundamental sequential logic element. Its state transitions are controlled by the inputs S and R . The characteristic equation for an SR flip-flop is given by:

$$Q_{next} = S + \overline{R}Q_{prev} \quad \text{with the constraint } S \cdot R = 0$$

The constraint $S \cdot R = 0$ explicitly forbids the condition where $S = 1$ and $R = 1$ simultaneously. If both inputs are asserted ($S = R = 1$), two severe problems occur:

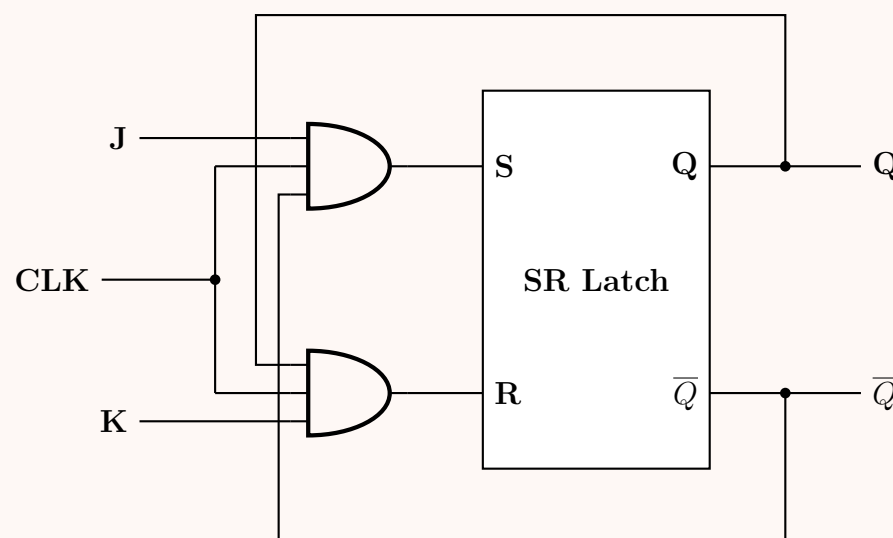
- Logical Violation:** In the internal cross-coupled gate structure (e.g., NOR-based latch), applying 1 to both inputs forces both outputs Q and $\overline{Q}$ to 0. This violates the fundamental complementarity principle of memory elements, which mandates that Q and $\overline{Q}$ must always be logical inverses.
- Metastability and Race Condition:** If the clock pulse ends or the inputs are simultaneously changed back to the holding state ($S = 0, R = 0$), the cross-coupled gates enter a critical race condition. Both gates simultaneously attempt to transition to 1, driving each other back to 0. The final resolved state becomes physically unpredictable (metastable), driven entirely by microscopic manufacturing variations and thermal noise.

S	R	Q_{next}	State Name
0	0	Q_{prev}	Hold / Memory
0	1	0	Reset
1	0	1	Set
1	1	?	Undefined / Forbidden

2. Resolution Using the JK Flip-Flop

The JK flip-flop was specifically designed to resolve the undefined state of the SR flip-flop without sacrificing any of its capabilities. It achieves this by introducing **cross-coupled feedback** from the outputs (Q and $\overline{Q}$) back to the input gating logic.

In a JK flip-flop, the J input acts like S (Set), and the K input acts like R (Reset). The internal logic dynamically masks the input that would attempt to drive the flip-flop into a state it is already in, thereby preventing the $S = R = 1$ condition internally.



3. Mathematical Derivation of the JK Characteristic Equation

From the logic diagram above, the internal SR inputs are governed by the outputs of the AND gates:

$$S = J \cdot \overline{Q}_{prev}$$

$$R = K \cdot Q_{prev}$$

We substitute these expressions into the characteristic equation of the underlying SR flip-flop ($Q_{next} = S + \overline{R}Q_{prev}$):

$$Q_{next} = (J \cdot \overline{Q}_{prev}) + \overline{(K \cdot Q_{prev})} \cdot Q_{prev} \quad \text{(Substitution)}$$

$$Q_{next} = J\overline{Q}_{prev} + (\overline{K} + \overline{Q}_{prev}) \cdot Q_{prev} \quad \text{(De Morgan's Law applied to } \overline{K \cdot Q})$$

$$Q_{next} = J\overline{Q}_{prev} + \overline{K}Q_{prev} + \overline{Q}_{prev}Q_{prev} \quad \text{(Distributive Law)}$$

$$Q_{next} = J\overline{Q}_{prev} + \overline{K}Q_{prev} + 0 \quad \text{(Complementarity Law: } X \cdot \overline{X} = 0)$$

$$Q_{next} = J\overline{Q}_{prev} + \overline{K}Q_{prev} \quad \text{(Identity Law)}$$

4. Behavior Under $J = 1, K = 1$

Let us evaluate the new characteristic equation, $Q_{next} = J\overline{Q} + \overline{K}Q$, for the previously problematic condition where both inputs are 1 ($J = 1, K = 1$):

$$Q_{next} = (1 \cdot \overline{Q}_{prev}) + (\overline{1} \cdot Q_{prev})$$

$$Q_{next} = \overline{Q}_{prev} + (0 \cdot Q_{prev})$$

$$Q_{next} = \overline{Q}_{prev}$$

Conclusion: By implementing cross-coupled feedback, the undefined state is completely eliminated. When $J = 1$ and $K = 1$, the flip-flop simply inverts its current state (a behavior known as **Toggling**).

J	K	Q_{next}	State Name
0	0	Q_{prev}	Hold / Memory
0	1	0	Reset
1	0	1	Set
1	1	$\overline{Q_{prev}}$	Toggle (Resolved)

Q6. Show the logic diagram and function table of an SR latch with NAND gates. (4 marks)

[CSE 205 | L-2/T-1 | 2011–12]

Answer

1. Overview of the NAND-Based SR Latch

An SR (Set-Reset) latch constructed with cross-coupled NAND gates operates as an **active-low** device. This means that a logic **0** is required to assert either the Set or Reset action, whereas a logic **1** is the passive (holding) state. For this reason, the inputs are traditionally denoted as $\overline{S}$ and $\overline{R}$.

2. Function Table (Truth Table)

The function table defines the next state (Q_{next}) based on the current inputs. Because it is an active-low latch, the quiescent (memory) state occurs when both inputs are driven high (1). Applying low (0) to both inputs simultaneously forces both outputs high, violating the complementarity rule ($Q \neq \overline{Q}$), making it an invalid or forbidden state.

Inputs		Outputs		State / Operation
$\overline{S}$	$\overline{R}$	Q_{next}	$\overline{Q}_{next}$	
1	1	Q_{prev}	$\overline{Q}_{prev}$	Hold (Memory / No Change)
0	1	1	0	Set (Q is forced to 1)
1	0	0	1	Reset (Q is forced to 0)
0	0	1	1	Invalid (Forbidden / Restricted)

3. Logic Diagram

The circuit consists of two 2-input NAND gates. The output of the top gate (Q) is cross-coupled to one of the inputs of the bottom gate, and the output of the bottom gate ($\overline{Q}$) is cross-coupled back to one of the inputs of the top gate.

Boolean Equations for the Latch: The outputs govern each other based on NAND logic (where $\overline{A \cdot B}$ is the NAND operation):
$$Q = \overline{\overline{S} \cdot \overline{Q}}$$
$$\overline{Q} = \overline{\overline{R} \cdot Q}$$

JK Flip-Flop Characteristics

Q1. What is the problem in the construction of a JK flip-flop? How can we overcome this problem? (6 marks)

[CSE 205 | L-2/T-1 | 2023–24]

Answer

1. The Problem: The Race-Around Condition

The primary problem encountered in the construction of a basic, level-triggered JK flip-flop is known as the **Race-Around Condition**. This anomaly occurs under a specific set of circumstances:

(a) The inputs are set to the toggle state: $J = 1$ and $K = 1$.

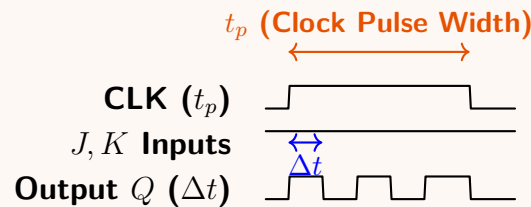
(b) The clock signal (CLK) is HIGH (level-triggered active state).

(c) The width of the clock pulse (t_p) is significantly longer than the propagation delay of the flip-flop (Δt).

Mechanism of the Problem: When $J = 1$ and $K = 1$, the JK flip-flop is designed to toggle its output ($Q_{next} = \overline{Q}_{prev}$). However, because the flip-flop is level-sensitive and the clock pulse remains HIGH for a relatively long time ($t_p > \Delta t$), the output Q will toggle, feed back to the input, and cause the flip-flop to toggle *again* while the clock is still HIGH.

This creates a "race" between the input and the feedback. The output oscillates ($1 \rightarrow 0 \rightarrow 1 \rightarrow 0 \dots$) continuously as long as the clock remains HIGH. When the clock finally goes LOW, the final state of Q is entirely unpredictable.

Timing Diagram of the Race-Around Condition



2. Mathematical Condition for Race-Around

For the race-around condition to occur, the timing relationship is:

$$t_p > \Delta t$$

Where:

- t_p = Clock pulse duration (HIGH time).
- Δt = Total propagation delay of the logic gates forming the flip-flop.

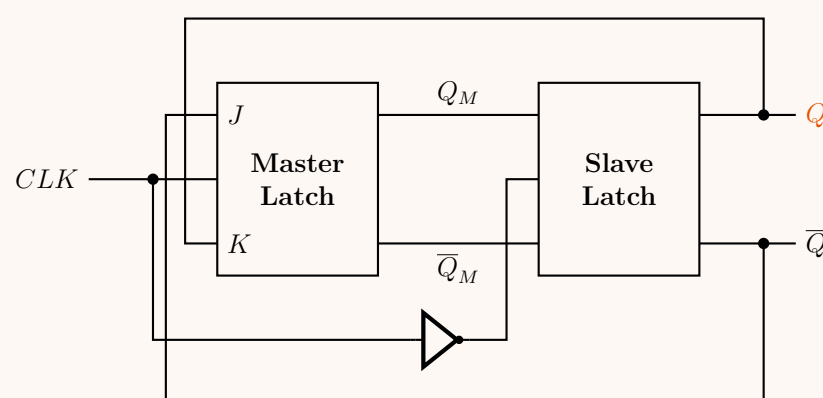
3. Solutions to Overcome the Problem

To eliminate the race-around condition, we must ensure that the flip-flop can only change state **once** per clock cycle, regardless of how long the clock remains HIGH. There are three primary methods to achieve this:

Solution A: Using a Master-Slave JK Flip-Flop The most robust solution is to construct a Master-Slave configuration. This involves cascading two flip-flops (a "Master" and a "Slave") and driving them with complementary clock signals.

- When $CLK = 1$: The Master is active and captures the J and K inputs. The Slave is disabled (since it receives $\overline{CLK} = 0$), so the final output Q remains completely isolated and unchanged.
- When $CLK = 0$: The Master is disabled (locking its state). The Slave becomes active and transfers the Master's state to the final output Q .

Because the feedback to the Master comes from the Slave's output, and the Slave only updates when the Master is locked, feedback oscillation is physically impossible.



Solution B: Edge-Triggered Flip-Flops Instead of making the flip-flop sensitive to the *level* of the clock (HIGH or LOW), edge-triggered flip-flops use specialized internal delay networks or edge-detector circuits (pulse transition detectors) to make the flip-flop active only during the microscopic transition of the clock (e.g., the positive edge from $0 \rightarrow 1$). Because the active window (transition time) is significantly shorter than the propagation delay (Δt), the flip-flop can only toggle once.

Solution C: Modifying Propagation Delay (Theoretical) By modifying the circuitry such that the propagation delay of the flip-flop (Δt) is greater than the clock pulse width (t_p), but less than the total clock period (T), the race-around condition is mathematically prevented:

$$t_p < \Delta t < T$$

However, this is generally impractical in modern high-speed digital designs, as maintaining such precise analog timing constraints across varying temperatures and voltages is unreliable. Therefore, Master-Slave or Edge-Triggering architectures are the industry standards.

Q2. Derive the characteristic table and the characteristic equation for a JK flip-flop. (4 marks)

[CSE 205 | L-2/T-1 | 2011–12]

Answer

1. The Characteristic Table

The characteristic table of a flip-flop describes its next state, $Q(t + 1)$, as a function of its current inputs and its present state, $Q(t)$. The JK flip-flop is a universal flip-flop that incorporates the behaviors of Set, Reset, and Hold, while resolving the undefined state of the SR flip-flop by introducing a Toggle state when both inputs are asserted ($J = 1, K = 1$). Based on its operational behavior, we construct the comprehensive characteristic table:

Inputs		Present State	Next State	Operation
J	K	$Q(t)$	$Q(t + 1)$	
0	0	0	0	Hold / No Change ($Q(t + 1) = Q(t)$)
0	0	1	1	
0	1	0	0	Reset ($Q(t + 1) = 0$)
0	1	1	0	
1	0	0	1	Set ($Q(t + 1) = 1$)
1	0	1	1	
1	1	0	1	Toggle ($Q(t + 1) = \overline{Q}(t)$)
1	1	1	0	

2. Karnaugh Map Simplification

To derive the characteristic equation, we extract the minterms from the characteristic table where $Q(t + 1) = 1$. Treating J, K , and $Q(t)$ as boolean variables (where J is the MSB and $Q(t)$ is the LSB), the minterms where the output is HIGH are:

$$m(J, K, Q(t)) = \sum m(1, 4, 5, 6)$$

We plot these minterms onto a 3-variable Karnaugh Map:

$KQ(t)$

0001

0111

1010

0

0

1

0

0

1

1

1

0

1

3. Derivation of the Characteristic Equation

From the Karnaugh Map, we can identify two prime implicants:

- Group 1 (Minterms 4, 6):** Spans the cells where $J = 1$ and $Q(t) = 0$. The variable K changes (from 0 to 1) and is therefore eliminated. This yields the term: $JQ(t)$.
- Group 2 (Minterms 1, 5):** Spans the cells where $K = 0$ and $Q(t) = 1$. The variable J changes (from 0 to 1) and is therefore eliminated. This yields the term: $\overline{K}Q(t)$.

Summing these product terms provides the minimized Boolean expression for the Next State.

Final Characteristic Equation:

$$Q(t + 1) = J\overline{Q}(t) + \overline{K}Q(t)$$

This mathematically confirms the structural behavior of the JK flip-flop: When $J = 1$ and $Q(t) = 0$, the device sets. When $K = 0$ and $Q(t) = 1$, the device holds its HIGH state. The undefined SR condition is naturally resolved.

Master-Slave D Flip-Flop

Q1. A clocked sequential circuit uses a master-slave configuration.

(a) Explain how signal propagation through the master and slave stages affects the timing behavior of the circuit.

(b) Analyze and justify the possibility of race-around under non-ideal clock conditions (finite rise and fall time).

(12 marks)

[CSE 205 | L-2/T-1 | 2024–25]

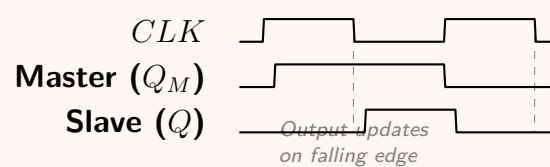
217

Answer**1. Timing Behavior and Signal Propagation in a Master-Slave Configuration**

A master-slave sequential circuit is constructed by cascading two level-sensitive latches: the **Master** and the **Slave**. They are driven by complementary clock signals (CLK for the master, and $\overline{CLK}$ for the slave, typically generated via an internal NOT gate). This configuration splits the clock cycle into two distinct propagation phases, effectively creating edge-triggered behavior and isolating the input from the output.

- **Phase 1: Master Active (Clock HIGH)** When $CLK = 1$, the Master latch is transparent (enabled). It samples the external inputs (e.g., J and K , or D) and its output (Q_M) updates accordingly. Concurrently, the Slave receives $\overline{CLK} = 0$, rendering it opaque (disabled). The Slave holds its previous value, isolating the final output (Q) from any input variations during this phase.
- **Phase 2: Slave Active (Clock LOW)** When $CLK = 0$, the Master latch is disabled, locking in the state it held just before the clock transitioned. The Slave now receives $\overline{CLK} = 1$, making it transparent. The signal propagates from the locked Master output (Q_M) through the Slave to the final output (Q).

Conclusion on Timing: Signal propagation takes exactly one full clock cycle to reach the final output and stabilize. Because the master and slave are never transparent simultaneously under ideal conditions, direct signal shoot-through (and thus, race-around) is prevented. State changes strictly occur on the clock edge (e.g., the falling edge for a positive-high master).

**2. Race-Around Under Non-Ideal Clock Conditions**

While the master-slave architecture mathematically prevents the race-around condition under ideal, zero-transition-time clocks, real physical circuits suffer from **finite rise (t_r) and fall (t_f) times**. A non-ideal clock can cause race-around due to a phenomenon known as **Clock Overlap**.

Mechanism of Failure:

- Inverter Delay ($t_{pd,inv}$):** The Slave's clock ($\overline{CLK}$) is generated by passing CLK through an inverter. This inverter has an inherent propagation delay.
- Finite Transition Time:** When CLK is transitioning from HIGH to LOW (falling edge), the voltage drops gradually rather than instantaneously.
- Simultaneous Conduction (Overlap):** Because CLK is falling slowly, there is a finite window where its voltage is still slightly above the logic threshold ($V_{th,L}$), keeping the Master partially enabled. Meanwhile, due to the inverter, $\overline{CLK}$ begins to rise and may cross the threshold ($V_{th,H}$), enabling the Slave.

During this critical overlap window ($t_{overlap}$), both the Master and the Slave latches are transparent at the exact same time. The isolation barrier is broken.

Mathematical Justification:

Let the propagation delay of the Master latch be $t_{pd(M)}$ and the Slave latch be $t_{pd(S)}$. If the clock overlap duration exceeds the combined propagation delay of the internal latches, a race-around will occur:

$$t_{overlap} > t_{pd(M)} + t_{pd(S)}$$

If this condition is met (especially in JK flip-flops where the output is fed back to the input), the newly toggled state from the Slave will feed back into the transparent Master, traverse the Slave again, and toggle the output multiple times within the same overlap window.

Resolution: To mitigate this in practical IC design, non-overlapping multi-phase clocks are used, or threshold voltages of the latches are intentionally skewed so that the Master disables well before the Slave enables.

Q2. Draw the block diagram of a negative edge triggered Master-Slave D flip-flop using D latches. **(10 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Answer**1. Architectural Overview**

A **Negative Edge-Triggered Master-Slave D Flip-Flop** is constructed by cascading two level-sensitive D latches. The first latch is designated as the **Master**, and the second as the **Slave**.

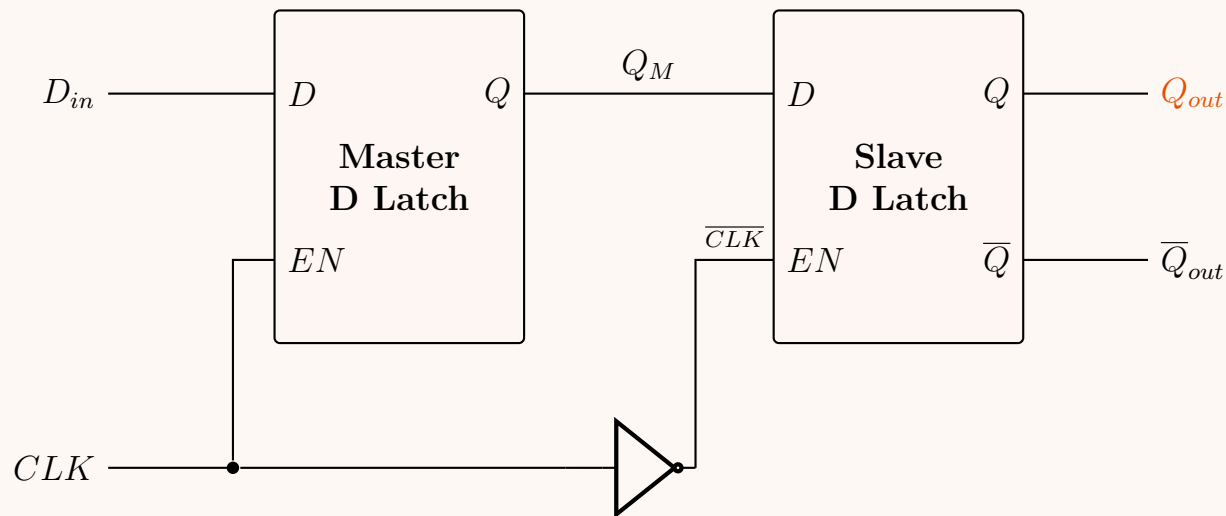
To achieve negative edge-triggering (where the output only changes on the falling edge of the clock signal, $1 \rightarrow 0$), the clock routing is designed as follows:

- The external CLK signal is connected directly to the Enable (EN) pin of the Master latch.
- The external CLK signal is inverted (via a NOT gate) before connecting to the Enable (EN) pin of the Slave latch.

Because of this complementary clocking scheme, the Master and Slave latches are never transparent (enabled) at the same time,

completely isolating the input D_{in} from the final output Q_{out} during static clock levels.

2. Logic Block Diagram



3. Operational Mechanism (State Analysis)

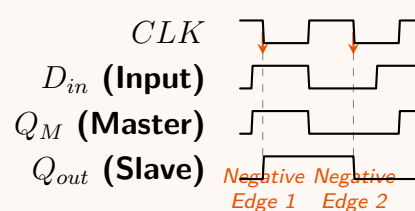
The operation can be broken down into two distinct phases of the clock cycle:

- **Phase 1: $CLK = 1$ (Clock HIGH)**
 - *Master Latch:* $EN = 1$. The Master is **transparent**. The intermediate output Q_M actively tracks the external input D_{in} .
 - *Slave Latch:* $EN = 0$ (due to the inverter). The Slave is **opaque** (locked). It holds its previous value, ensuring Q_{out} remains stable regardless of changes at D_{in} .
- **Phase 2: $CLK = 0$ (Clock LOW)**
 - *Master Latch:* $EN = 0$. The Master becomes **opaque**. It locks in the value of D_{in} that was present at the exact moment the clock transitioned from 1 to 0.
 - *Slave Latch:* $EN = 1$. The Slave becomes **transparent**. It reads the locked value from Q_M and immediately transfers it to the final output Q_{out} .

Conclusion: The final output Q_{out} is updated *only* at the exact instant the clock falls (the negative edge).

4. Timing Diagram Verification

The following timing diagram mathematically verifies the step-by-step logic. Notice how Q_{out} remains completely unaffected by any variations in D_{in} while the clock is steady, changing strictly at the designated negative edges (marked by dashed lines).



Q3. Draw the circuit diagram of a master-slave JK flip-flop with asynchronous preset and clear using only NOR gates. **(5 marks)** [CSE 205 | L-2/T-1 | 2021–22]

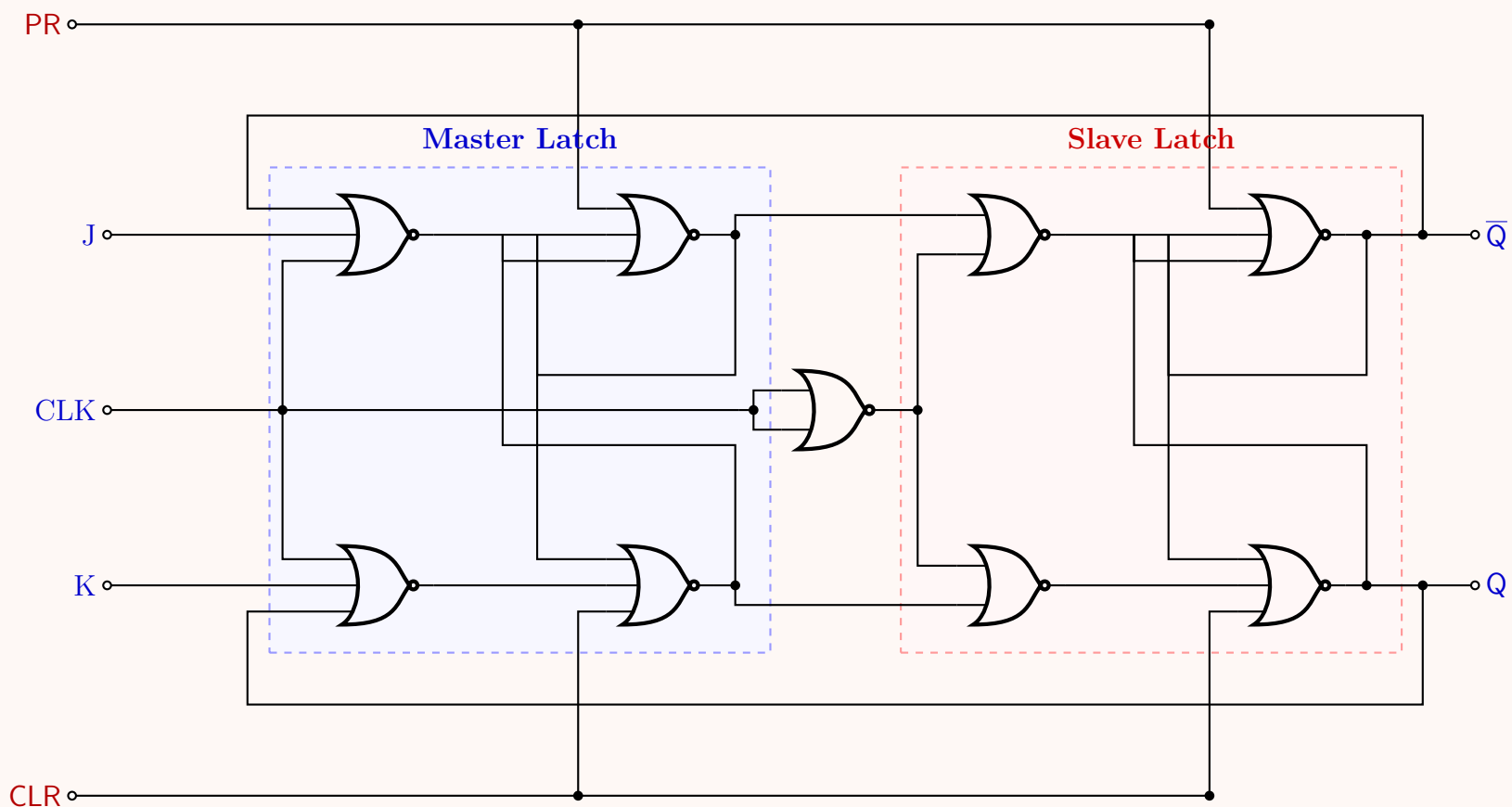
Answer

1. Structural Logic and Design Principles

Designing a Master-Slave JK flip-flop with asynchronous Preset (PR) and Clear (CLR) entirely out of NOR gates requires mapping the standard AND-OR-Latch topology into a NOR-only equivalent.

- **Latches:** The Master and Slave memory cores are formed by cross-coupled NOR gates. Because NOR logic is active-high, a 1 on the S input resets $\bar{Q}$ to 0 (which forces Q to 1). Asynchronous PR and CLR inputs are directly added to these latches by expanding them into 3-input NOR gates.
- **Steering Logic:** To satisfy the characteristic JK equation using NOR gates, we must invert the J and K inputs. The Master steering gates are 3-input NORs receiving $\overline{CLK}$, feedback from the Slave, and the inverted external inputs ($\bar{J}$ and $\bar{K}$).
- **Clock Isolation:** An inverter (created by tying the inputs of a NOR gate together) provides $\overline{CLK}$ to the Master, while the original CLK drives the Slave. This guarantees that Master and Slave are never transparent simultaneously.

2. Circuit Diagram



3. Mathematical & Operational Analysis

Asynchronous Override (PR & CLR): In NOR logic, an input of 1 forces the output to 0 regardless of the other inputs.

- If **CLR = 1**: The top latch gates of both Master and Slave are forced to output 0. Thus, $Q_M = 0$ and $Q = 0$. The flip-flop is unconditionally reset.
- If **PR = 1**: The bottom latch gates are forced to output 0. Thus, $\bar{Q}_M = 0$ and $\bar{Q} = 0$, logically forcing $Q_M = 1$ and $Q = 1$. The flip-flop is unconditionally set.

Synchronous Operation (when PR = 0, CLR = 0): The Master Steering gates control the Master Latch. A NOR gate outputs 1 only when all of its inputs are 0.

$$\begin{aligned}
 S_M \text{ (Set Master)} &= \overline{\bar{J} + Q_{prev} + \overline{CLK}} & \xrightarrow{\text{De Morgan's}} & J \cdot \bar{Q}_{prev} \cdot CLK \\
 R_M \text{ (Reset Master)} &= \overline{\bar{K} + \bar{Q}_{prev} + \overline{CLK}} & \xrightarrow{\text{De Morgan's}} & K \cdot Q_{prev} \cdot CLK
 \end{aligned}$$

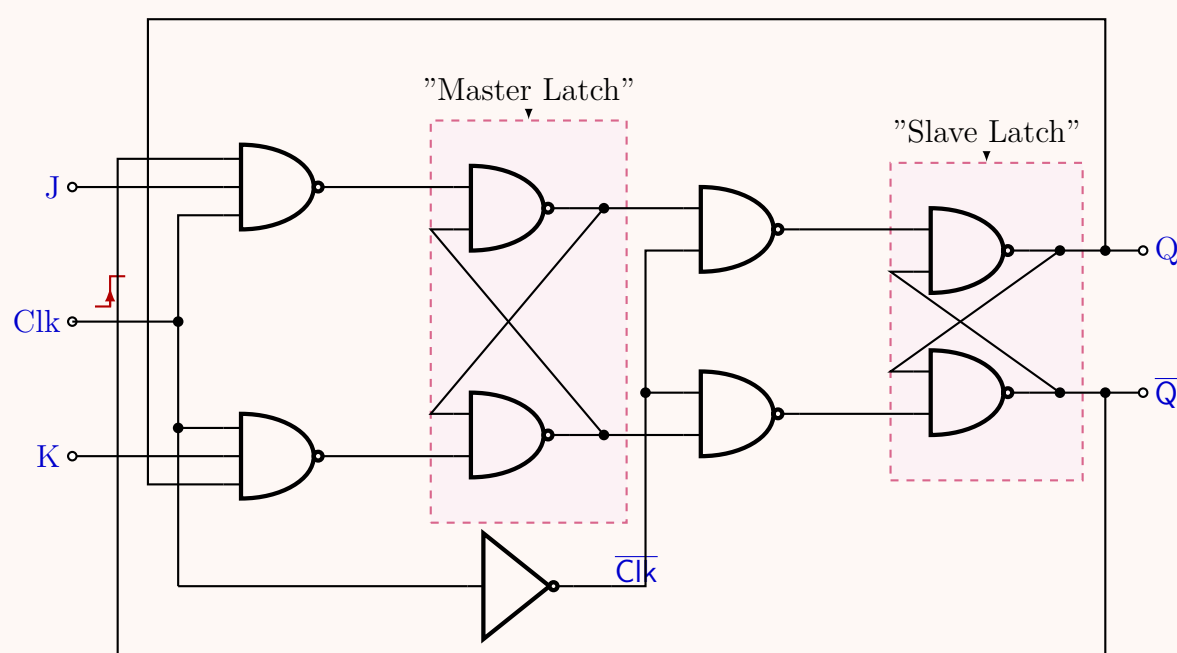
This demonstrates mathematically that placing the NOR-based inverters on J and K , combined with the Slave feedback loops (Q to bottom, $\bar{Q}$ to top), perfectly reproduces the classical JK flip-flop behavior, updating the state strictly when the clock allows.

Q4. Draw the circuit diagram of a master-slave JK Flip-Flop using only NAND gates. (8 marks) [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Circuit Diagram

The circuit diagram below shows a classic pulse-triggered (Master-Slave) JK flip-flop constructed entirely from standard NAND gates. The Master section consists of a 3-input steering stage followed by a cross-coupled NAND latch. The Slave section consists of a 2-input steering stage followed by another cross-coupled NAND latch. A single NAND gate with tied inputs serves as the clock inverter.



2. Mathematical Formulation and Operating Principles

The master-slave architecture separates state evaluation from state assignment by utilizing two distinct phases of the clock cycle, completely avoiding the classical race-around condition present in level-triggered JK latches.

Phase 1: Charging the Master ($CLK = 1$)

When the clock signal goes high ($CLK = 1$), the inverted clock driving the slave stage goes low ($\overline{CLK} = 0$).

- **Slave Behavior:** Since $\overline{CLK} = 0$, the outputs of the slave steering gates ('nand5' and 'nand6') are forced to a logic high state:

$$S_S = \overline{Q_M \cdot \overline{CLK}} = \overline{Q_M \cdot 0} = 1$$

$$R_S = \overline{\overline{Q_M} \cdot \overline{CLK}} = \overline{\overline{Q_M} \cdot 0} = 1$$

When both inputs to a cross-coupled NAND latch are 1, the latch retains its current state. Hence, the final outputs Q and $\overline{Q}$ remain locked.

- **Master Behavior:** Because $CLK = 1$, the master steering gates are enabled. Taking the global cross-coupled feedback loops into account, the intermediate excitation equations for the master latch inputs are evaluated as:

$$S_M = \overline{J \cdot CLK \cdot \overline{Q}} = \overline{J \cdot 1 \cdot \overline{Q}} = \overline{J \cdot \overline{Q}}$$

$$R_M = \overline{\overline{K} \cdot CLK \cdot Q} = \overline{\overline{K} \cdot 1 \cdot Q} = \overline{\overline{K} \cdot Q}$$

The master latch updates its state variables Q_M and $\overline{Q}_M$ based on the current inputs J, K and the existing feedback state of the device.

Phase 2: Transferring to the Slave ($CLK = 0$)

When the clock transitions to a low level ($CLK = 0$), the internal inverted clock switches high ($\overline{CLK} = 1$).

- **Master Behavior:** With $CLK = 0$, the outputs of both master steering gates are immediately forced to a logic high state ($S_M = 1, R_M = 1$). This completely isolates the master latch from any transient changes occurring at the external J and K inputs, preserving the evaluated values of Q_M and $\overline{Q}_M$.
- **Slave Behavior:** Since $\overline{CLK} = 1$, the slave steering gates act as transparent buffers tracking the state captured by the master:

$$S_S = \overline{Q_M \cdot 1} = \overline{Q_M}$$

$$R_S = \overline{\overline{Q}_M \cdot 1} = Q_M$$

The slave cross-coupled latch updates its state according to these equations, allowing the final output terminal Q to accept the value previously computed by the master latch during the high portion of the clock phase.

3. Truth Table and State Verification

The operational behavior described mathematically above perfectly matches the classic JK flip-flop functional profiles shown below.

Clock Phase	J	K	Q_M	$\overline{Q}_M$	Q_{next}	Functional Operation
$CLK = 1$	0	0	Q	$\overline{Q}$	Q	No Change (Hold State)
$CLK = 1$	0	1	0	1	0	Reset State Condition
$CLK = 1$	1	0	1	0	1	Set State Condition
$CLK = 1$	1	1	$\overline{Q}$	Q	$\overline{Q}$	Toggle State Condition

Q5. Why do we use master-slave flip-flops? (5 marks)

[CSE 205 | L-2/T-1 | 2017–18]

Answer

The primary reason we use **Master-Slave Flip-Flops** is to eradicate the **race-around condition** that inherently plagues standard level-triggered J-K flip-flops, thereby ensuring predictable and stable outputs.

1. The Problem: The Race-Around Condition

In a standard level-triggered J-K flip-flop, the toggle operation is activated when both inputs are HIGH ($J = 1, K = 1$). Under these conditions, the characteristic equation dictates that the next state should be the complement of the present state:

$$Q_{next} = J\overline{Q} + \overline{K}Q$$

$$= (1 \cdot \overline{Q}) + (0 \cdot Q)$$

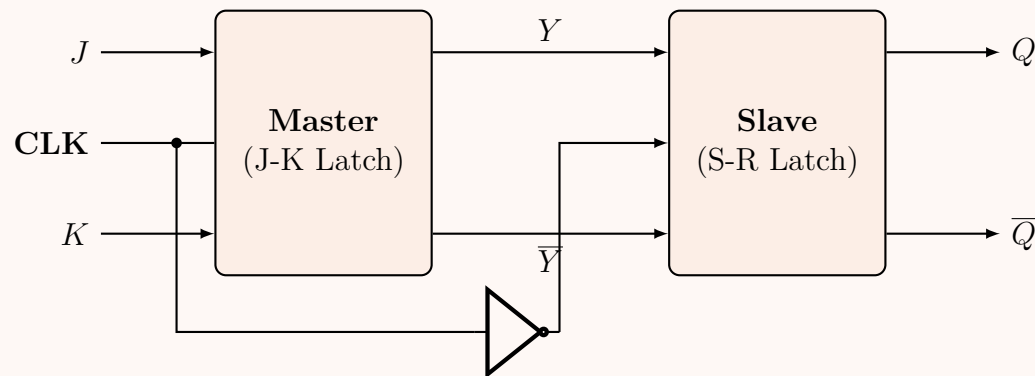
$$= \overline{Q} \quad (\text{Toggle State})$$

However, an issue arises due to timing. Let t_p be the width (duration) of the HIGH clock pulse, and t_{pd} be the propagation delay of the flip-flop. If the clock pulse remains HIGH longer than the propagation delay ($t_p > t_{pd}$), the flip-flop will toggle, feed the new inverted output back to its inputs, and toggle *again* while the clock is still HIGH.

Because the flip-flop toggles continuously (e.g., $0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \dots$) during a single clock pulse, the final state of Q at the falling edge of the clock becomes completely unpredictable. This rapid, uncontrolled toggling is known as the **race-around condition**.

2. The Solution: Master-Slave Architecture

To solve this, the master-slave configuration cascades two distinct latches—a **Master** and a **Slave**—driven by complementary clock signals. This design effectively converts a level-triggered device into a highly reliable **edge-triggered** device.



3. Mechanism of Action

The master-slave architecture successfully decouples the input sampling from the output transition through the following two-step phase:

- **Positive Clock Phase ($\text{CLK} = 1$):** The un-inverted clock enables the Master latch. It responds to the external J and K inputs and updates its intermediate outputs (Y and $\bar{Y}$). Simultaneously, the Slave latch receives an inverted clock ($\bar{\text{CLK}} = 0$) and is thereby disabled. The final outputs (Q and $\bar{Q}$) remain completely isolated from the Master's transitions.
- **Negative Clock Phase ($\text{CLK} = 0$):** The un-inverted clock drops to 0, disabling the Master latch and locking its state. The Master is now deaf to any changes in J and K . Concurrently, the Slave latch receives an inverted clock ($\bar{\text{CLK}} = 1$) and is enabled. The Slave transfers the stabilized intermediate outputs (Y and $\bar{Y}$) to the final outputs (Q and $\bar{Q}$).

Conclusion: By strictly ensuring that the Master and Slave are never active at the same time, the master-slave flip-flop guarantees that the output Q can only change state *once* per clock cycle (specifically at the falling edge of the clock). This provides strict data isolation and entirely eliminates the race-around condition.

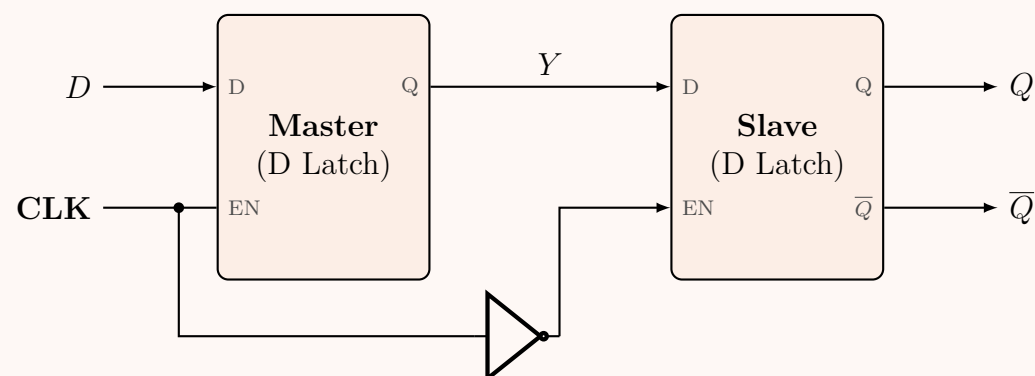
Q6. Draw the block diagram and explain the operation of a Master-Slave negative edge triggered D flip-flop. (12 marks) [CSE 205 | L-2/T-1 | 2015–16]

Answer

A **Master-Slave Negative Edge-Triggered D Flip-Flop** is constructed by cascading two level-sensitive D latches. The first latch is called the **Master**, and the second is the **Slave**. They are driven by complementary clock signals to ensure that they are never active at the same time.

1. Block Diagram

In this configuration, the Master latch is enabled when the clock is HIGH, and the Slave latch is enabled when the clock is LOW (achieved via an inverter on the clock line).



2. Principle of Operation

The operation of the flip-flop can be divided into three distinct phases based on the state of the CLK signal:

- **Positive Phase ($\text{CLK} = 1$):** The Master latch receives an Enable signal of 1 and becomes **transparent**. The intermediate output Y tracks the input D ($Y = D$). Concurrently, the Slave latch receives an Enable signal of 0 (due to the inverter) and becomes **opaque** (holds its state). The final outputs Q and $\bar{Q}$ remain unchanged, entirely isolated from any variations in D .
- **Negative Edge Transition ($\text{CLK} : 1 \rightarrow 0$):** As the clock transitions from HIGH to LOW, the Master latch is disabled. It samples and **locks** the value of D present exactly at the moment of the falling edge. Instantly, the Slave latch is enabled as its input becomes 1.

- **Negative Phase (CLK = 0):**
The Master is now opaque and ignores any further changes to the D input. The Slave latch is **transparent** to its input Y . It passes the locked value of Y to the final output Q ($Q = Y = D_{\text{sampled}}$).

By enforcing this strict "handshake" between the two latches, the input is only sampled at the **falling edge** of the clock, eliminating any transparency from input D to output Q during level phases.

3. Truth Table & Characteristic Equation

The behavior evaluates precisely to the definition of a D flip-flop, where the next state Q_{next} follows the D input, but strictly restricted to the negative clock edge.

Clock (CLK)	Data Input (D)	Next State (Q_{next})
0	X	Q (Hold)
1	X	Q (Hold)
↑ (Rising)	X	Q (Hold)
↓ (Falling)	0	0 (Reset)
↓ (Falling)	1	1 (Set)

Characteristic Equation:

$$Q_{\text{next}} = D \quad (\text{triggered exclusively on CLK } \downarrow)$$

Q7. Draw and explain the operation of a Master-Slave D flip-flop. (12 marks)

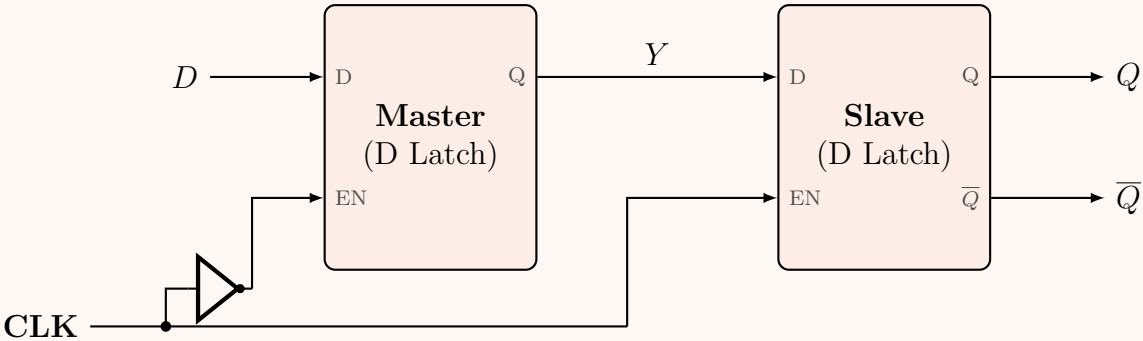
[CSE 205 | L-2/T-1 | 2011–12]

Answer

A **Master-Slave D Flip-Flop** is built by cascading two level-sensitive D latches. The first latch is called the **Master**, and the second is the **Slave**. By feeding them complementary clock signals, the circuit ensures that only one latch is active (transparent) at any given time, effectively creating an **edge-triggered** device that prevents transparency from input to output.

1. Block Diagram (Positive Edge-Triggered)

In this configuration, the Master latch receives an *inverted* clock signal, while the Slave latch receives the *direct* clock signal. This makes the entire flip-flop **positive edge-triggered**.



2. Principle of Operation

The circuit operates in three critical phases, decoupling the input reading from the output writing:

- **Clock LOW Phase (CLK = 0):**
The clock signal is inverted, providing an Enable (EN) of 1 to the Master latch. The Master becomes **transparent**, and its intermediate output tracks the input ($Y = D$). The Slave latch receives an Enable of 0, placing it in **hold (opaque) mode**. The final outputs Q and $\overline{Q}$ hold their previous states, completely isolated from any changes at the D input.
- **Positive Edge Transition (CLK : 0 → 1):**
At the exact moment the clock transitions from LOW to HIGH, the inverted signal to the Master latch falls to 0. The Master latch instantly becomes opaque, **sampling and locking** the value of D present just before the edge. Simultaneously, the direct clock to the Slave latch rises to 1.
- **Clock HIGH Phase (CLK = 1):**
The Master remains locked and ignores all subsequent changes on the D input. The Slave latch is now **transparent** and passes the locked intermediate signal Y to the final output ($Q = Y = D_{\text{sampled}}$).

Through this mechanism, data is exclusively transferred from input to output on the rising edge of the clock pulse.

3. Truth Table & Characteristic Equation

A Master-Slave D flip-flop functions equivalently to a standard edge-triggered D flip-flop, where the next state is purely dependent on the D input sampled at the active clock edge.

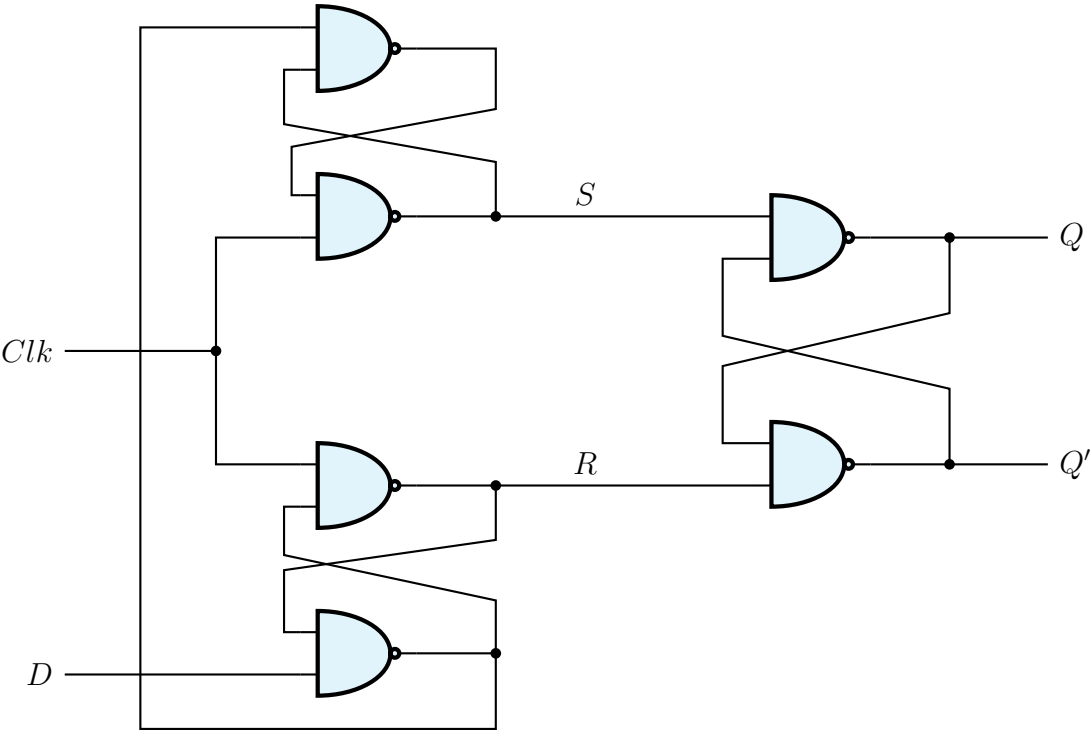
Clock (CLK)	Data Input (D)	Next State (Q_{next})
0	X	Q (Hold)
1	X	Q (Hold)
↓ (Falling)	X	Q (Hold)
↑ (Rising)	0	0 (Reset)
↑ (Rising)	1	1 (Set)

Characteristic Equation:

$Q_{next} = D$ (triggered strictly on CLK ↑)

Sequential Circuit Timing Analysis

Q1. Consider a sequential circuit. Let the current values be: CLK = 1, D = 1, Q = 1, Q' = 0. If the value of D changes from 1 to 0 after the “Hold Time” ends and CLK remains unchanged at 1, what will be the values of Q and Q'?



(6 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

To determine the final values of Q and Q' , we must rigorously trace the logic levels through the six NAND gates (labeled N_1 through N_6). The circuit represents a modified or faulty edge-triggered D flip-flop master-slave configuration, notably lacking the standard locking cross-coupling from N_2 to N_3 . We will analyze the circuit in three discrete steps:

Step 1: Establish the Initial Steady State

Given the initial conditions: CLK = 1, D = 1, Q = 1, and Q' = 0. The output stage consisting of gates N_5 and N_6 forms an active-low $\overline{S}$ - $\overline{R}$ latch. Let us define the intermediate signals driving this latch as $S = N_2$ and $R = N_3$. To hold the state $Q = 1, Q' = 0$, the inputs to the latch must be $S = 0$ and $R = 1$. We can verify the internal nodes (N_1, N_2, N_3, N_4) at this initial state:

$N_2 = 0 \implies$ Since $N_2 = \overline{N_1 \cdot \text{CLK}}$ and CLK = 1, we must have $N_1 = 1$.

$N_3 = 1 \implies$ Since $N_3 = \overline{N_4 \cdot \text{CLK}}$ and CLK = 1, we must have $N_4 = 0$.

$N_4 = 0 \implies$ Since $N_4 = \overline{N_3 \cdot D}$, checking with $N_3 = 1, D = 1 \implies \overline{1 \cdot 1} = 0$ (Consistent).

$N_1 = 1 \implies$ Since $N_1 = \overline{N_2 \cdot N_4}$, checking with $N_2 = 0, N_4 = 0 \implies \overline{0 \cdot 0} = 1$ (Consistent).

Initial Internal State: $N_1 = 1, N_2 = 0, N_3 = 1, N_4 = 0$.

Step 2: Evaluate the Transition (D changes 1 → 0)

The input D falls to 0 while CLK remains locked at 1. We propagate this change through the input gates:

$$N_4 = \overline{N_3 \cdot D}$$
$$= \overline{1 \cdot 0} = 1$$

(Output of N_4 transitions from 0 → 1)

This new value of $N_4 = 1$ simultaneously feeds into N_1 and N_3 . Let us re-evaluate them:

$$N_3 = \overline{\text{CLK} \cdot N_4}$$
$$= \overline{1 \cdot 1} = \mathbf{0}$$

(Output of N_3 , our R signal, transitions from $1 \rightarrow 0$)

$$N_1 = \overline{N_2 \cdot N_4}$$
$$= \overline{0 \cdot 1} = \mathbf{1}$$

(Output of N_1 holds its previous state)

$$N_2 = \overline{N_1 \cdot \text{CLK}}$$
$$= \overline{1 \cdot 1} = \mathbf{0}$$

(Output of N_2 , our S signal, holds its previous state)

Step 3: Resolve the Final Output Latch (Q, Q')

Following the transition, the signals driving the output $\overline{S}\text{-}\overline{R}$ latch are now $S(N_2) = 0$ and $R(N_3) = 0$. Applying these to the output NAND gates:

$$Q = \overline{S \cdot Q'} = \overline{0 \cdot Q'} = \mathbf{1}$$
$$Q' = \overline{R \cdot Q} = \overline{0 \cdot Q} = \mathbf{1}$$

Phase	D Input	N_4	N_1	$N_3 (R)$	$N_2 (S)$	Q	Q'
Initial (CLK = 1)	1	0	1	1	0	1	0
Post-Transition (CLK = 1)	0	1	1	0	0	1	1

Final Values: Both outputs are forced logically high, yielding $Q = 1$ and $Q' = 1$.

Latch vs Flip-Flop

Q1. Write the differences between a latch and a flip-flop. (5 marks)

[CSE 205 | L-2/T-1 | 2013–14]

Answer

Both latches and flip-flops are fundamental memory elements used in sequential logic circuits to store a single bit of information. However, they differ significantly in their operation, structure, and synchronization mechanics. The primary differences are detailed in the comparison table below:

Parameter	Latch	Flip-Flop
Triggering Mechanism	Level-triggered: The output continuously responds to input changes as long as the Enable/Clock signal is at its active level (High or Low).	Edge-triggered: The output responds to input changes only at a specific instant: the rising (positive) or falling (negative) edge of the clock.
Structural Composition	Simpler design; constructed directly using fundamental cross-coupled logic gates (e.g., NAND or NOR gates).	More complex design; typically constructed from a combination of latches (e.g., Master-Slave configuration) and an edge-detector circuit.
Sensitivity & Glitches	Highly sensitive to input changes during the active level. It is vulnerable to spurious glitches if inputs fluctuate while enabled.	Inputs are sampled only precisely at the clock edge. It is highly robust and inherently immune to glitches between clock edges.
Operating Speed	Generally faster operating speeds due to shorter propagation delays through fewer logic gates.	Comparatively slower due to the propagation delays introduced by the additional circuitry required for edge-triggering.
Hardware & Area	Requires fewer logic gates, thereby consuming less power and occupying less silicon area on an IC.	Requires more logic gates, leading to higher power consumption and a larger silicon footprint.
Usage & Applications	Best suited for asynchronous sequential circuits, simple buffers, and designs where strict timing synchronization is not critical.	The fundamental building block for synchronous sequential circuits, shift registers, counters, and microprocessor state machines.

Conceptual Analogy:

- A **Latch** behaves like a **transparent window**: as long as the window is open (enable is active), everything passing by outside (inputs) is immediately seen inside (outputs).
- A **Flip-Flop** behaves like a **camera shutter**: it captures a frozen snapshot of the outside (inputs) only at the exact fraction of a second the shutter clicks (clock edge), ignoring changes before and after.

Flip-Flop Conversion

Q1. Design a T flip-flop using a JK flip-flop and verify its operation using state transitions. **(10 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Answer

To design a **T (Toggle) flip-flop** using a **JK flip-flop**, we must determine how to drive the J and K inputs such that the JK flip-flop mimics the behavior of a T flip-flop. We begin by comparing their excitation tables.

1. Excitation Table Formulation

The T flip-flop has a single input T which toggles the state when $T = 1$ and retains the state when $T = 0$. The JK flip-flop requires specific J and K inputs to achieve these identical state transitions ($Q_n \rightarrow Q_{n+1}$).

State Transition		T Flip-Flop	JK Flip-Flop Inputs	
Q_n (Current)	Q_{n+1} (Next)	Required T	Required J	Required K
0	0	0	0	X
0	1	1	1	X
1	0	1	X	1
1	1	0	X	0

Note: 'X' denotes a "Don't Care" condition.

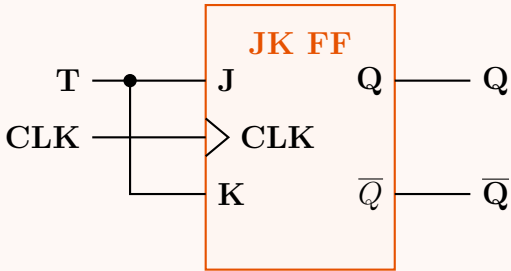
2. Boolean Expression Derivation

By observing the table above, we can directly derive the Boolean expressions for inputs J and K in terms of T :

- For the **J input**: Looking at the rows where J is not 'X' (rows 1 and 2), we see that $J = 0$ when $T = 0$, and $J = 1$ when $T = 1$. Thus, **$J = T$** .
- For the **K input**: Looking at the rows where K is not 'X' (rows 3 and 4), we see that $K = 1$ when $T = 1$, and $K = 0$ when $T = 0$. Thus, **$K = T$** .

Since $J = T$ and $K = T$, we simply tie the J and K inputs of the JK flip-flop together and connect them to the T input.

3. Logic Circuit Diagram



4. Mathematical Verification via State Transitions

To verify the operation mathematically, we analyze the characteristic equation of a standard JK flip-flop:

$$Q_{n+1} = J\overline{Q_n} + \overline{K}Q_n$$

(Characteristic equation of JK FF)

Substitute the derived relationships $J = T$ and $K = T$:

$$Q_{n+1} = T\overline{Q_n} + \overline{T}Q_n$$

(Substitution)

$$Q_{n+1} = T \oplus Q_n$$

(Definition of XOR operation)

The resulting equation, **$Q_{n+1} = T \oplus Q_n$** , is exactly the characteristic equation of a T flip-flop.

Conclusion:

- When $T = 0 \implies Q_{n+1} = 0 \oplus Q_n = Q_n$ (The state is **Held**).
- When $T = 1 \implies Q_{n+1} = 1 \oplus Q_n = \overline{Q_n}$ (The state is **Toggled**).

This logically confirms that tying the J and K inputs together creates a fully functional T flip-flop.

Q2. Construct a JK flip-flop from a T flip-flop and necessary gates. Show all the steps of this construction process. **(11 marks)** [CSE 205 | L-2/T-1 | 2023–24, 2022–23, 2016–17]

Answer

To construct a **JK flip-flop** (the target) using a **T flip-flop** (the available hardware) and basic logic gates, we must design a combinational circuit that generates the appropriate T input based on the J and K inputs and the current state Q_n . The construction process follows a systematic four-step methodology:

Step 1: Define Target and Available Flip-Flop Characteristics

First, we state the excitation requirement for the available **T flip-flop** and the state transition behavior of the target **JK flip-flop**.

- **T Flip-Flop Excitation:** To change state ($Q_{n+1} = \overline{Q_n}$), we need $T = 1$. To hold state ($Q_{n+1} = Q_n$), we need $T = 0$. Mathematically, $T = Q_n \oplus Q_{n+1}$.
- **JK Flip-Flop Behavior:** $Q_{n+1} = J\overline{Q_n} + \overline{K}Q_n$.

Step 2: Construct the Conversion Table

We merge the characteristic table of the JK flip-flop with the excitation requirements of the T flip-flop to determine the required T input for every combination of J , K , and Q_n .

Target JK Inputs & State			Next State	Required Input
J	K	Q_n (Current)	Q_{n+1}	$T = Q_n \oplus Q_{n+1}$
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	0
1	1	0	1	1
1	1	1	0	1

Step 3: Boolean Simplification using Karnaugh Map

From the conversion table, the minterms for T are $m(3, 4, 6, 7)$. We plot these on a 3-variable Karnaugh Map to find the minimized Sum-of-Products (SOP) expression for T .

		KQ_n			
		00	01	11	10
J	0	0	0	1	0
	1	1	0	1	1

Groupings:

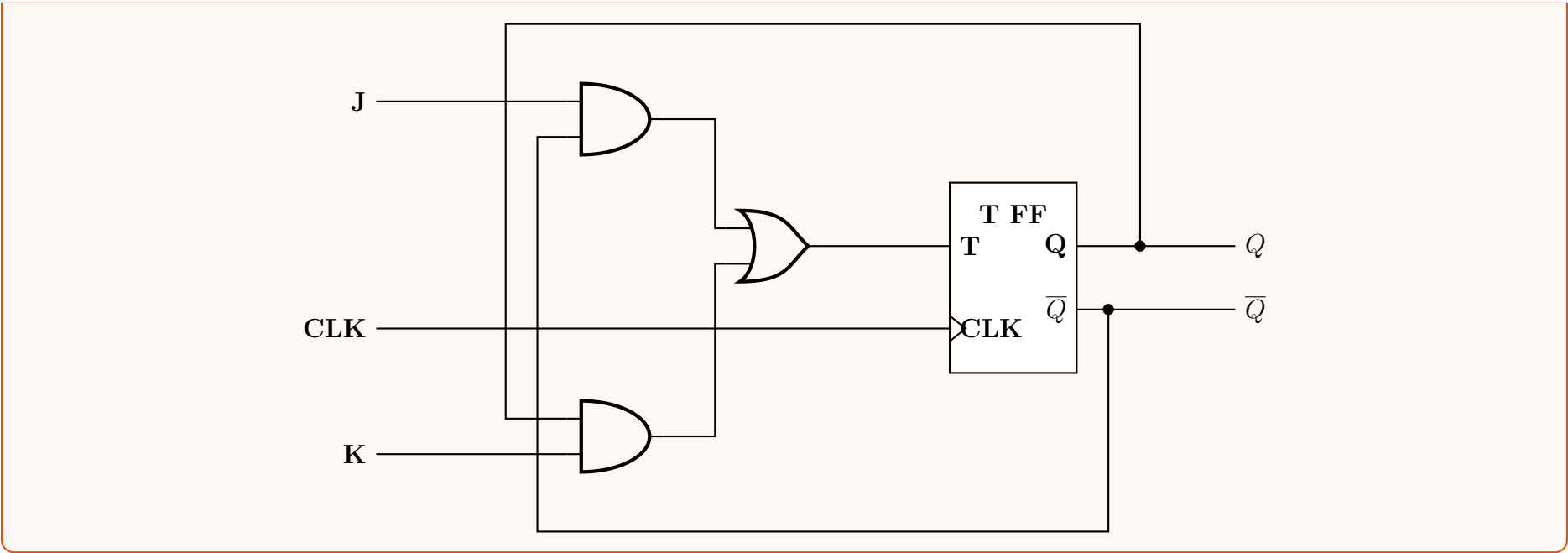
- The horizontal pair covering cells 4 and 6 yields the term: $J\overline{Q_n}$
- The vertical pair covering cells 3 and 7 yields the term: KQ_n

Applying the OR operation to these prime implicants provides the final combinational logic equation:

$$T = J\overline{Q_n} + KQ_n$$

Step 4: Logic Circuit Implementation

Using the derived Boolean equation, the combinational logic requires two AND gates and one OR gate. The outputs of the flip-flop (Q and $\overline{Q}$) are fed back into the AND gates to evaluate the current state.



Q3. Design a D flip-flop using a T flip-flop and gates. (5 marks)

[CSE 205 | L-2/T-1 | 2020–21]

↪ Similar: 2019–20

Answer

To design a **D flip-flop** (target) using a **T flip-flop** (available hardware) and basic logic gates, we must design a combinational circuit that determines the appropriate T input based on the target D input and the current state Q_n .

1. Excitation and Conversion Table Formulation

We begin by establishing the required state transitions for a D flip-flop and mapping them to the excitation requirements of a T flip-flop.

- **D Flip-Flop Target Behavior:** The next state simply follows the input, meaning $Q_{n+1} = D$.
- **T Flip-Flop Excitation Rule:** The input T must be 1 to toggle the state ($Q_n \neq Q_{n+1}$), and 0 to hold the state ($Q_n = Q_{n+1}$). Mathematically, $T = Q_n \oplus Q_{n+1}$.

Target D Input & State		Next State	Required T Input
D	Q_n (Current)	Q_{n+1} (Target, = D)	$T = Q_n \oplus Q_{n+1}$
0	0	0	0
0	1	0	1
1	0	1	1
1	1	1	0

2. Boolean Expression Derivation

From the conversion table, we can directly write the Sum-of-Products (SOP) expression for the required T input by identifying the rows where $T = 1$:

$$T = \Sigma m(1, 2)$$
$$T = \overline{D}Q_n + D\overline{Q}_n$$
$$\mathbf{T = D \oplus Q_n}$$

(Standard SOP form extracted from the table)
(Definition of the XOR operation)

Alternative Algebraic Verification:

$$Q_{n+1} \text{ (T FF)} = T \oplus Q_n$$

Substitute $T = D \oplus Q_n$:

$$Q_{n+1} = (D \oplus Q_n) \oplus Q_n$$
$$Q_{n+1} = D \oplus (Q_n \oplus Q_n)$$
$$Q_{n+1} = D \oplus 0$$
$$Q_{n+1} = D$$

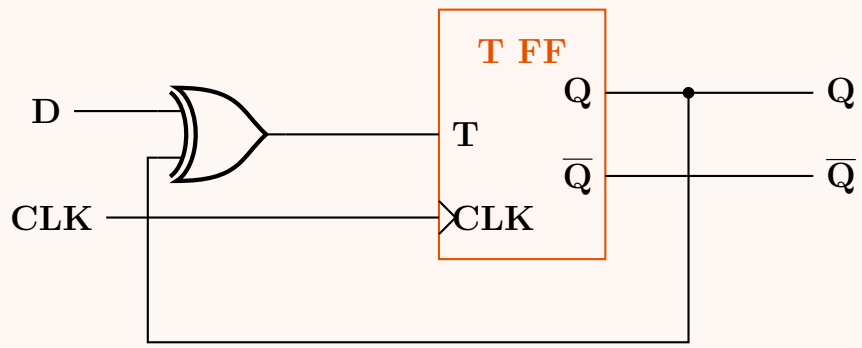
(Characteristic equation of T FF)

(Substitution)
(Associative Law of XOR)
(Self-Inverse Law: $A \oplus A = 0$)
(Identity Law of XOR: $A \oplus 0 = A$)

This confirms that driving the T input with $D \oplus Q_n$ flawlessly replicates the behavior of a D flip-flop ($Q_{n+1} = D$).

3. Logic Circuit Realization

Based on the derived equation $\mathbf{T = D \oplus Q_n}$, the circuit requires only a single 2-input XOR gate connected to the T input. The current state output Q is fed back to serve as one of the inputs to this XOR gate.



Q4. Design a SR Flip-Flop using a D Flip-Flop and a 4×1 MUX. (10 marks)

[CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Design Objective

To design a Synchronous SR Flip-Flop using a single D Flip-Flop and a 4×1 Multiplexer (MUX).

2. Characteristic Equations

The relationship between the present state (Q_n), next state (Q_{n+1}), and the inputs determines the behavior of the flip-flops:

D Flip-Flop:

The next state simply follows the D input.

$$Q_{n+1} = D$$

SR Flip-Flop:

The next state is defined by the S (Set) and R (Reset) inputs. The state equation is:

$$Q_{n+1} = S + \bar{R}Q_n \quad (\text{Valid when } SR = 0)$$

Because $Q_{n+1} = D$, the required D input at any time is exactly the required next state of the SR flip-flop. We can use the 4×1 MUX to implement this next-state logic.

3. Truth Table & MUX Implementation Logic

We will use the S and R inputs as the selection lines for the 4×1 MUX. Let $S_1 = S$ and $S_0 = R$. The MUX will select one of its four data inputs (I_0, I_1, I_2, I_3) and route it to the output Y , which is connected to the D input of the flip-flop.

The logical function of the 4×1 MUX is:

$$Y = \bar{S}_1\bar{S}_0I_0 + \bar{S}_1S_0I_1 + S_1\bar{S}_0I_2 + S_1S_0I_3$$

Let's derive the inputs I_0 to I_3 from the SR Flip-Flop characteristic table:

Select Lines	Next State	Required Input	MUX Data Input
$S (S_1)$ $R (S_0)$	Q_{n+1}	$D = Q_{n+1}$	Assignment
0 0	Q_n (Hold)	Q_n	$I_0 = Q_n$
0 1	0 (Reset)	0	$I_1 = 0$
1 0	1 (Set)	1	$I_2 = 1$
1 1	X (Invalid)	0 (or X)	$I_3 = 0$ (Grounding to avoid floating state)

Substituting these inputs into the MUX equation gives us the exact D input required:

$$D = \bar{S}\bar{R}(Q_n) + \bar{S}R(0) + S\bar{R}(1) + SR(0)$$
$$D = \bar{S}\bar{R}Q_n + S\bar{R}$$

Note: This simplifies to $S + \bar{R}Q_n$, which perfectly matches the SR flip-flop characteristic equation.

4. Circuit Diagram

The circuit below visualizes the implementation. The 4×1 MUX generates the next-state logic based on S and R , feeding into the D Flip-Flop. The present state output Q is fed back into the I_0 input of the MUX to enable the "Hold" state.

229

Q5. Convert: (i) a JK flip-flop into a T flip-flop, and (ii) a D flip-flop into a JK flip-flop. **(8 marks)** [CSE 205 | L-2/T-1 | 2013–14]

Answer

Flip-flop conversion involves designing combinational logic that transforms the available (target) flip-flop’s behavior into the desired (source) flip-flop’s behavior. The general procedure involves expressing the desired inputs in terms of the available flip-flop’s excitation requirements.

(i) Conversion of a JK Flip-Flop into a T Flip-Flop

1. Conversion Table

We aim to construct a T (Toggle) flip-flop using an available JK flip-flop. We use the Truth Table of the T flip-flop and map the required next state (Q_{n+1}) to the Excitation Table of the JK flip-flop to determine the required inputs J and K .

Desired T-FF		Next State Q_{n+1}	Required JK Inputs	
T	Present State (Q_n)		J	K
0	0	0	0	X
0	1	1	X	0
1	0	1	1	X
1	1	0	X	1

2. Boolean Minimization

From the conversion table, we can directly extract the minimal Boolean equations for J and K in terms of the input T and present state Q_n .

$$J = \sum m(2) + d(1, 3) \implies J = T$$
$$K = \sum m(3) + d(0, 2) \implies K = T$$

Both J and K are directly connected to the T input.

3. Logic Circuit Diagram

(ii) Conversion of a D Flip-Flop into a JK Flip-Flop

1. Conversion Table

We need to design a circuit where the overall behavior matches a JK flip-flop, but the memory element is a D (Data) flip-flop. The characteristic equation of a D flip-flop is simple: $D = Q_{n+1}$. Therefore, the required D input is exactly identical to the required next state of the JK flip-flop.

Desired JK-FF Inputs			Next State Q_{n+1}	Required D Input D
J	K	Present State (Q_n)		
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

2. Boolean Derivation

Extracting the minterms for the required D input where $D = 1$:

$$D = \sum m(1, 4, 5, 6)$$
$$D = \overline{J}KQ_n + \overline{J}K\overline{Q_n} + J\overline{K}Q_n + J\overline{K}\overline{Q_n}$$

We can simplify this equation utilizing standard Boolean theorems:

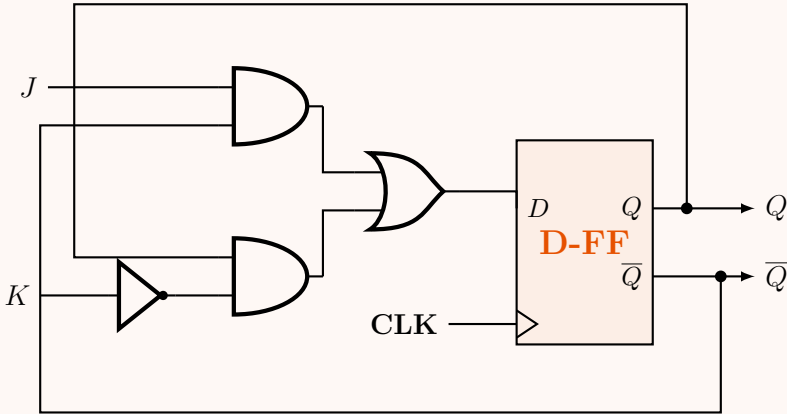
$$D = \overline{K}Q_n(\overline{J} + J) + J\overline{Q_n}(\overline{K} + K)$$
$$D = \overline{K}Q_n(1) + J\overline{Q_n}(1)$$
$$D = J\overline{Q_n} + \overline{K}Q_n$$

(Distributive Law)
(Complementarity Law: $A + \overline{A} = 1$)
(Identity Law)

Note: This is precisely the characteristic equation of a JK flip-flop.

3. Logic Circuit Diagram

To build this, we feed the output of the combinational logic $J\overline{Q_n} + \overline{K}Q_n$ into the D input.



Q6. Convert a T flip-flop into a JK flip-flop and vice versa. (4+4 marks)

[CSE 205 | L-2/T-1 | 2012–13]

Answer

Flip-flop conversion requires designing a combinational logic circuit that translates the desired inputs into the excitation inputs of the available flip-flop. We will address both conversions below.

Part 1: Conversion of a T Flip-Flop into a JK Flip-Flop

1. Conversion Table

We want the circuit to behave like a JK flip-flop, using a T (Toggle) flip-flop as the actual memory element. The excitation equation for a T flip-flop is $T = Q_n \oplus Q_{n+1}$. We list the JK flip-flop’s truth table and determine the required T input to achieve the desired next state (Q_{n+1}).

Desired JK Inputs & State			Next State	Required T Input
J	K	Present State (Q_n)	Q_{n+1}	$T = Q_n \oplus Q_{n+1}$
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	0
1	1	0	1	1
1	1	1	0	1

2. Boolean Minimization

From the table, we extract the minterms where $T = 1$:

$$T = \sum m(3, 4, 6, 7)$$
$$T = \overline{J}KQ_n + J\overline{K}\overline{Q_n} + JK\overline{Q_n} + JKQ_n$$

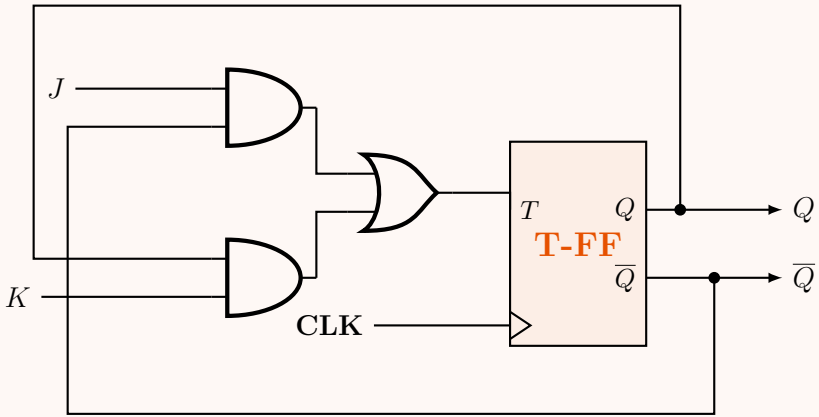
Simplifying using Boolean algebra theorems:

$$T = \overline{J}KQ_n + JKQ_n + J\overline{K}\overline{Q_n} + JK\overline{Q_n}$$
$$T = KQ_n(\overline{J} + J) + J\overline{Q_n}(\overline{K} + K)$$
$$T = KQ_n(1) + J\overline{Q_n}(1)$$
$$T = J\overline{Q_n} + KQ_n$$

(Commutative Law)
(Distributive Law)
(Complementarity Law: $A + \overline{A} = 1$)
(Identity Law)

3. Logic Circuit Diagram

We implement the Boolean equation $T = J\overline{Q_n} + KQ_n$ using standard logic gates feeding into the T input of the flip-flop.



Part 2: Conversion of a JK Flip-Flop into a T Flip-Flop

1. Conversion Table

Here, we want the circuit to behave like a T flip-flop using a JK flip-flop as the memory element. We take the T flip-flop state table and map it to the JK flip-flop excitation table.

Desired T-FF		Next State Q_{n+1}	Required JK Inputs	
T	Present State (Q_n)		J	K
0	0	0	0	X
0	1	1	X	0
1	0	1	1	X
1	1	0	X	1

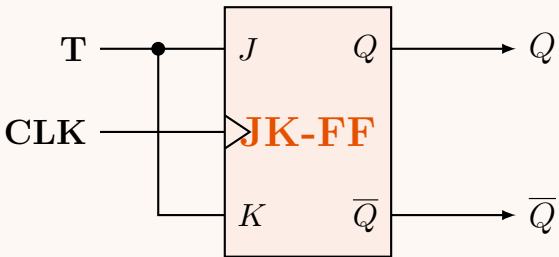
2. Boolean Minimization

Using K-maps or direct observation from the conversion table for the minterms of J and K in terms of input T and state Q_n :

$$J(T, Q_n) = \sum m(2) + d(1, 3) \implies J = T$$
$$K(T, Q_n) = \sum m(3) + d(0, 2) \implies K = T$$

Therefore, tying both J and K inputs of the JK flip-flop together and connecting them to T achieves the required toggle behavior.

3. Logic Circuit Diagram



Q7. Convert a D flip-flop to a T flip-flop. Use necessary gates. (4 marks)

[CSE 205 | L-2/T-1 | 2011–12]

Answer

Flip-flop conversion requires designing a combinational logic circuit that translates the desired input behavior (T flip-flop) into the excitation input of the available memory element (D flip-flop).

1. Conversion Table

We need the overall circuit to act as a T (Toggle) flip-flop. The characteristic equation of a D (Data) flip-flop is straightforward: $D = Q_{n+1}$. This means whatever next state (Q_{n+1}) we desire must be directly applied to the D input. We use the truth table of the T flip-flop to determine this required next state.

Desired T-FF Inputs		Next State Q_{n+1}	Required D-FF Input $D = Q_{n+1}$
T	Present State (Q_n)		
0	0	0 (Hold)	0
0	1	1 (Hold)	1
1	0	1 (Toggle)	1
1	1	0 (Toggle)	0

2. Boolean Minimization

From the conversion table, we extract the minterms where the required input $D = 1$:

$$D(T, Q_n) = \sum m(1, 2)$$

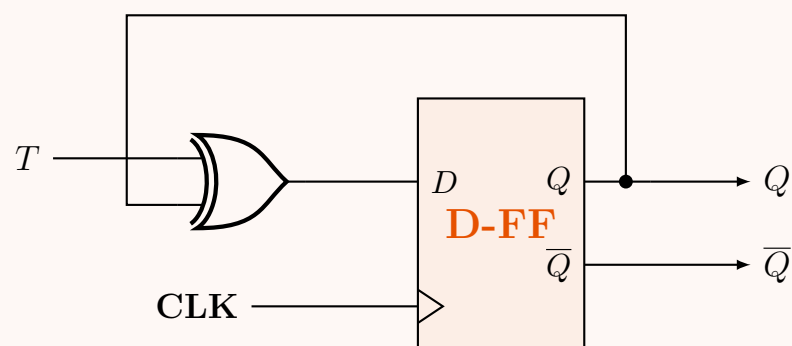
$$D = \bar{T}Q_n + T\bar{Q}_n$$

This Boolean expression matches the standard Exclusive-OR (XOR) operation. Therefore, we can simplify the excitation equation to:

$$D = T \oplus Q_n$$

3. Logic Circuit Diagram

To implement this conversion, an XOR gate is required. The T input serves as one input to the XOR gate, and the present state output Q from the D flip-flop is fed back as the second input. The output of the XOR gate is connected to the D input of the flip-flop.



BUET · CSE 205 · Digital Logic Design

Digital Logic Design

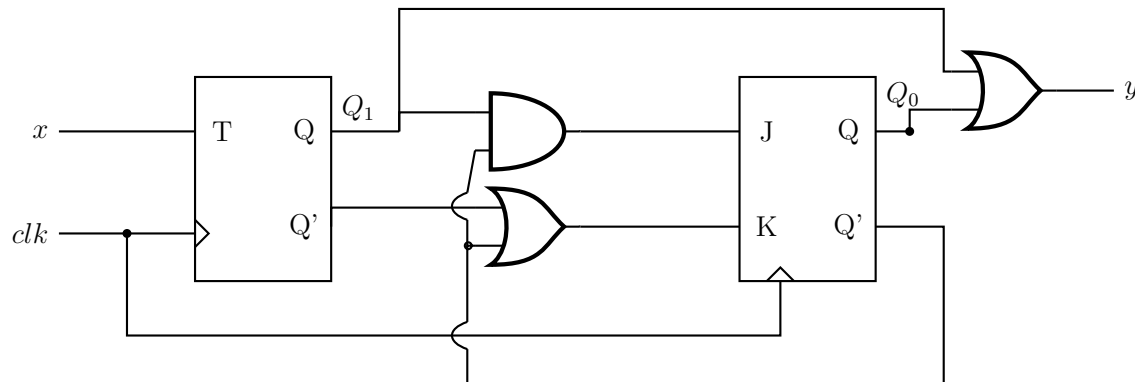
Sequential Circuits, State Diagrams & FSM

Mealy & Moore Machines, State Tables, State Minimization, Pulse/Fundamental Mode

S6 Sequential Circuits, State Diagrams & FSM

State Diagram Derivation from Circuit

Q1. Analyze the sequential circuit in the figure below. Derive the transition table, excitation table and state table.



(12 marks)

[CSE 205 | L-2/T-1 | 2023–24]

Answer

1. Boolean Equations for Flip-Flop Inputs and Circuit Output

By analyzing the provided sequential circuit diagram, we can determine the excitation equations (the inputs to the flip-flops) and the output equation as functions of the external input x and the present states of the flip-flops Q_1 and Q_0 .

- **T Flip-Flop (Q_1):** The input is directly connected to the external input x .

$$T_1 = x$$

- **JK Flip-Flop (Q_0):** The J input is driven by an AND gate with inputs from Q_1 and Q_0' . The K input is driven by an OR gate with inputs from Q_1' and Q_0' .

$$J_0 = Q_1 \cdot Q_0'$$

$$K_0 = Q_1' + Q_0'$$

- **Circuit Output (y):** The output is driven by an OR gate with inputs from Q_1 and Q_0 . Since the output depends only on the present states, this is a **Moore machine**.

$$y = Q_1 + Q_0$$

2. Next-State Equations

We use the characteristic equations of the T and JK flip-flops to derive the next-state formulas, denoted as $Q_1(t+1)$ and $Q_0(t+1)$.

- **Next State for Q_1 :** The characteristic equation for a T flip-flop is $Q(t+1) = T \oplus Q$.

$$Q_1(t+1) = T_1 \oplus Q_1$$

$$Q_1(t+1) = x \oplus Q_1$$

- **Next State for Q_0 :** The characteristic equation for a JK flip-flop is $Q(t+1) = JQ' + K'Q$. First, we compute K'_0 using De Morgan's Law:

$$K'_0 = (Q_1' + Q_0')'$$

$$K'_0 = (Q_1')' \cdot (Q_0')'$$

$$K'_0 = Q_1 \cdot Q_0$$

(De Morgan's Law)
(Involution Law)

Now, substituting J_0 and K'_0 into the characteristic equation:

$$Q_0(t+1) = J_0Q_0' + K'_0Q_0$$

$$Q_0(t+1) = (Q_1 \cdot Q_0')Q_0' + (Q_1 \cdot Q_0)Q_0$$

$$Q_0(t+1) = Q_1 \cdot (Q_0' \cdot Q_0') + Q_1 \cdot (Q_0 \cdot Q_0)$$

(Associative Law)

$$Q_0(t+1) = Q_1 \cdot Q_0' + Q_1 \cdot Q_0$$

(Idempotent Law)

$$Q_0(t+1) = Q_1 \cdot (Q_0' + Q_0)$$

(Distributive Law)

$$Q_0(t+1) = Q_1 \cdot (1)$$

(Complementarity Law)

$$Q_0(t+1) = Q_1$$

(Identity Law)

3. Combined Excitation and Transition Table

Using the derived equations, we evaluate the flip-flop inputs (excitation) and the next states (transition) for all possible combinations of the present state and input x .

Present State		Input	FF Inputs (Excitation)			Next State		Output
Q_1	Q_0	x	$T_1 = x$	$J_0 = Q_1Q'_0$	$K_0 = Q'_1 + Q'_0$	$Q_1(t+1)$	$Q_0(t+1)$	$y = Q_1 + Q_0$
0	0	0	0	0	1	0	0	0
0	0	1	1	0	1	1	0	0
0	1	0	0	0	1	0	0	1
0	1	1	1	0	1	1	0	1
1	0	0	0	1	1	1	1	1
1	0	1	1	1	1	0	1	1
1	1	0	0	0	0	1	1	1
1	1	1	1	0	0	0	1	1

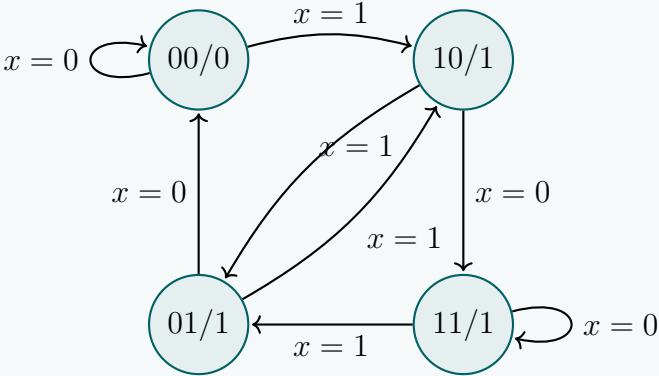
4. State Table

The state table summarizes the transition behavior in a compact format, mapping present states to next states based on the input x . Since this is a Moore machine, the output y is listed as a function of the present state only.

Present State Q_1Q_0	Next State ($Q_1(t+1)Q_0(t+1)$)		Output y
	$x = 0$	$x = 1$	
00	00	10	0
01	00	10	1
10	11	01	1
11	11	01	1

5. State Diagram (Supplementary Analysis)

To fully visualize the sequential behavior, we can construct the state diagram. Note how the evaluation of $Q_0(t+1) = Q_1$ simplifies the transitions: the next state's LSB always inherits the present state's MSB.



Q2. A sequential circuit has two flip-flops A and B , two inputs x_1 and x_2 , and one output z . The flip-flop input equations and circuit output equations are:

$$J_A = y_Ax_1 + y_Bx_2,$$
$$J_B = y_Ax_1,$$
$$z = y_Ax_1x_2 + y_Bx_1x_2$$

$$K_A = y_Ay_B + x_1x_2$$
$$K_B = y_Ay_Bx_1 + y_Bx_2$$

- (a) Draw the logic diagram of the circuit.
- (b) Derive the transition table, excitation table and state table.
- (c) Draw the state diagram.

(20 marks)

[CSE 205 | L-2/T-1 | 2021–22, 2019–20]

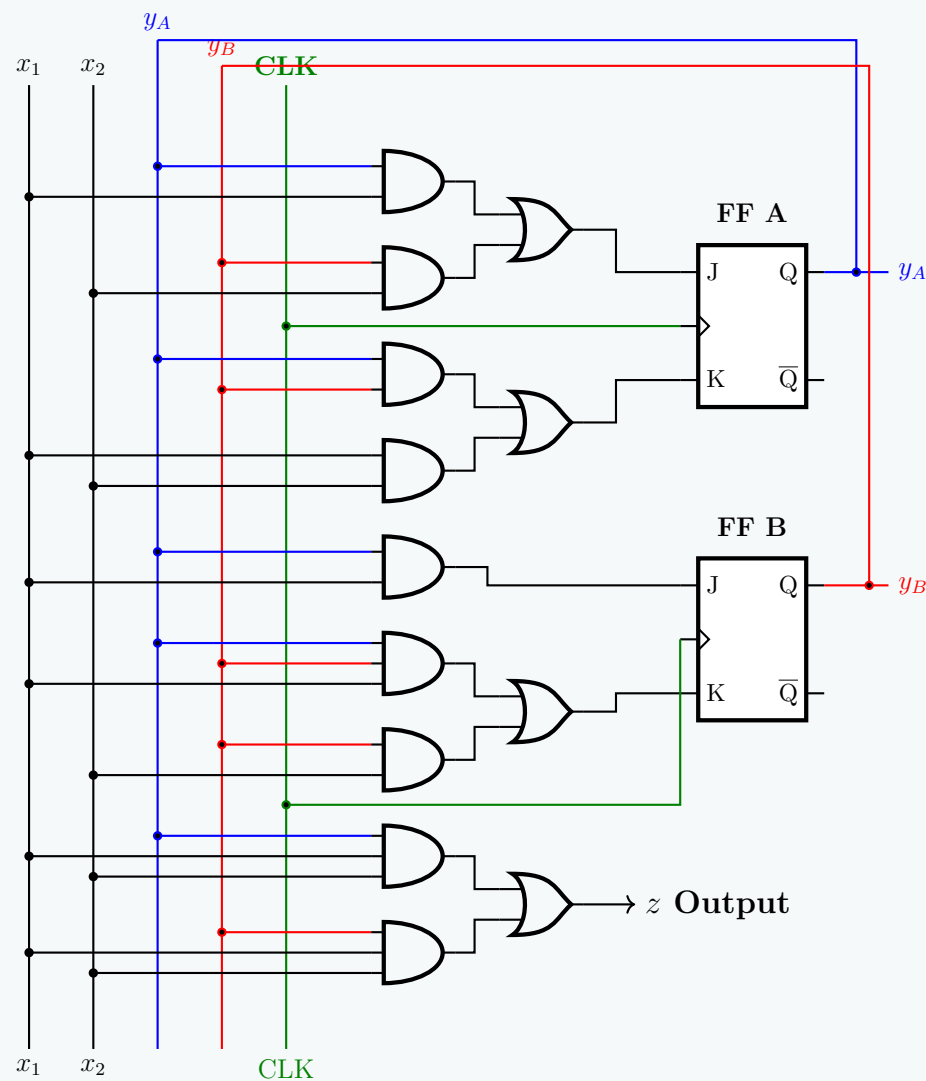
- **K_A Logic:** A 2-input AND gate computes $y_A y_B$, and another 2-input AND gate computes $x_1 x_2$. The outputs feed into a 2-input OR gate connected to the K_A terminal.

(c) **Excitation Network for Flip-Flop B:**

- **J_B Logic:** A single 2-input AND gate computes $y_A x_1$ and connects to the J_B terminal.
- **K_B Logic:** A 3-input AND gate computes $y_A y_B x_1$, and a 2-input AND gate computes $y_B x_2$. Their outputs feed into a 2-input OR gate connected to the K_B terminal.

(d) **Output Network (z):**

- A 3-input AND gate computes $y_A x_1 x_2$, and a second 3-input AND gate computes $y_B x_1 x_2$. Their outputs feed into a final 2-input OR gate to produce the Mealy output z .



(b) **Derivation of Tables**

1. Next-State Equations (Mathematical Derivations)

The general characteristic equation for a JK flip-flop is given by $Q(t+1) = J \cdot Q' + K' \cdot Q$. We substitute the excitation equations into this template for both flip-flops.

For **Flip-Flop A**:

$$\begin{aligned}
 y_A(t+1) &= J_A y_A' + K_A' y_A \\
 &= (y_A x_1 + y_B x_2) y_A' + (y_A y_B + x_1 x_2)' y_A \\
 &= y_A y_A' x_1 + y_A' y_B x_2 + (y_A y_B)' (x_1 x_2)' y_A \\
 &= 0 + y_A' y_B x_2 + (y_A' + y_B') (x_1' + x_2') y_A \\
 &= y_A' y_B x_2 + (y_A y_A' + y_A y_B') (x_1' + x_2') \\
 &= y_A' y_B x_2 + (0 + y_A y_B') (x_1' + x_2') \\
 &= y_A' y_B x_2 + y_A y_B' x_1' + y_A y_B' x_2'
 \end{aligned}$$

(Distributive & De Morgan's Laws)
 (Complementarity & De Morgan's Laws)
 (Distributive Law)
 (Complementarity Law)
 (Identity & Distributive Laws)

For **Flip-Flop B**:

$$\begin{aligned}
 y_B(t+1) &= J_B y_B' + K_B' y_B \\
 &= (y_A x_1) y_B' + (y_A y_B x_1 + y_B x_2)' y_B \\
 &= y_A y_B' x_1 + [y_B (y_A x_1 + x_2)]' y_B \\
 &= y_A y_B' x_1 + [y_B' + (y_A x_1 + x_2)'] y_B \\
 &= y_A y_B' x_1 + y_B' y_B + (y_A x_1 + x_2)' y_B \\
 &= y_A y_B' x_1 + 0 + (y_A x_1)' x_2' y_B \\
 &= y_A y_B' x_1 + (y_A' + x_1') x_2' y_B \\
 &= y_A y_B' x_1 + y_A' y_B x_2' + y_B x_1' x_2'
 \end{aligned}$$

(Distributive Law)
 (De Morgan's Law)
 (Distributive Law)
 (Complementarity & De Morgan's Laws)
 (De Morgan's Law)
 (Distributive Law)

2. Excitation and Transition Table

By systematically substituting all $2^4 = 16$ combinations of state ($y_A y_B$) and inputs ($x_1 x_2$), we compile the detailed excitation and transition data.

Present State		Inputs		Excitation Inputs				Next State		Output
y_A	y_B	x_1	x_2	J_A	K_A	J_B	K_B	$y_A(t+1)$	$y_B(t+1)$	z
0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0	0
0	0	1	1	0	1	0	0	0	0	0
0	1	0	0	0	0	0	0	0	1	0
0	1	0	1	1	0	0	1	1	0	0
0	1	1	0	0	0	0	0	0	1	0
0	1	1	1	1	1	0	1	1	0	1
1	0	0	0	0	0	0	0	1	0	0
1	0	0	1	0	0	0	0	1	0	0
1	0	1	0	1	0	1	0	1	1	0
1	0	1	1	1	1	1	0	0	1	1
1	1	0	0	0	1	0	0	0	1	0
1	1	0	1	1	1	0	1	0	0	0
1	1	1	0	1	1	1	0	0	0	0
1	1	1	1	1	1	1	1	0	0	1

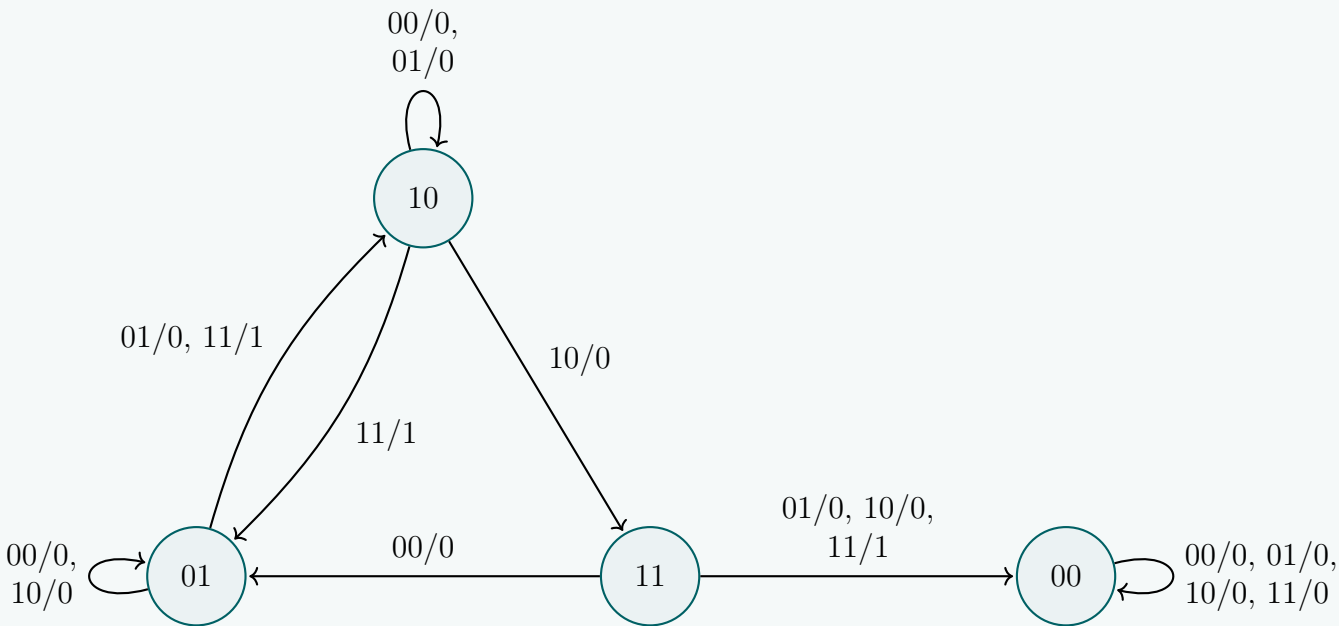
3. Compact State Table

Consolidating the data into a standard state table showing the mapping from present state to next state and output under all input variations:

Present State $y_A y_B$	Next State $y_A(t+1)y_B(t+1)$				Output z			
	$x_1 x_2 = 00$	01	10	11	$x_1 x_2 = 00$	01	10	11
00	00	00	00	00	0	0	0	0
01	01	10	01	10	0	0	0	1
10	10	10	11	01	0	0	0	1
11	01	00	00	00	0	0	0	1

(c) State Diagram

Below is the complete Mealy finite state machine representation. Arc transitions are formatted as Inputs ($x_1 x_2$)/Output (z).



Q3. A sequential circuit has two JK Flip-Flops A and B , two inputs x and y , and one output z . The flip-flop input equations and circuit output equation are:

$$J_A = Ax + By,$$
$$J_B = Ax,$$

$$K_A = AB + xy$$
$$K_B = ABx + By,$$

$$z = Axy + Bxy$$

- (a) Draw the logic diagram of the circuit.
- (b) Derive the state table.
- (c) Draw the state diagram.

(4+7+4 marks)

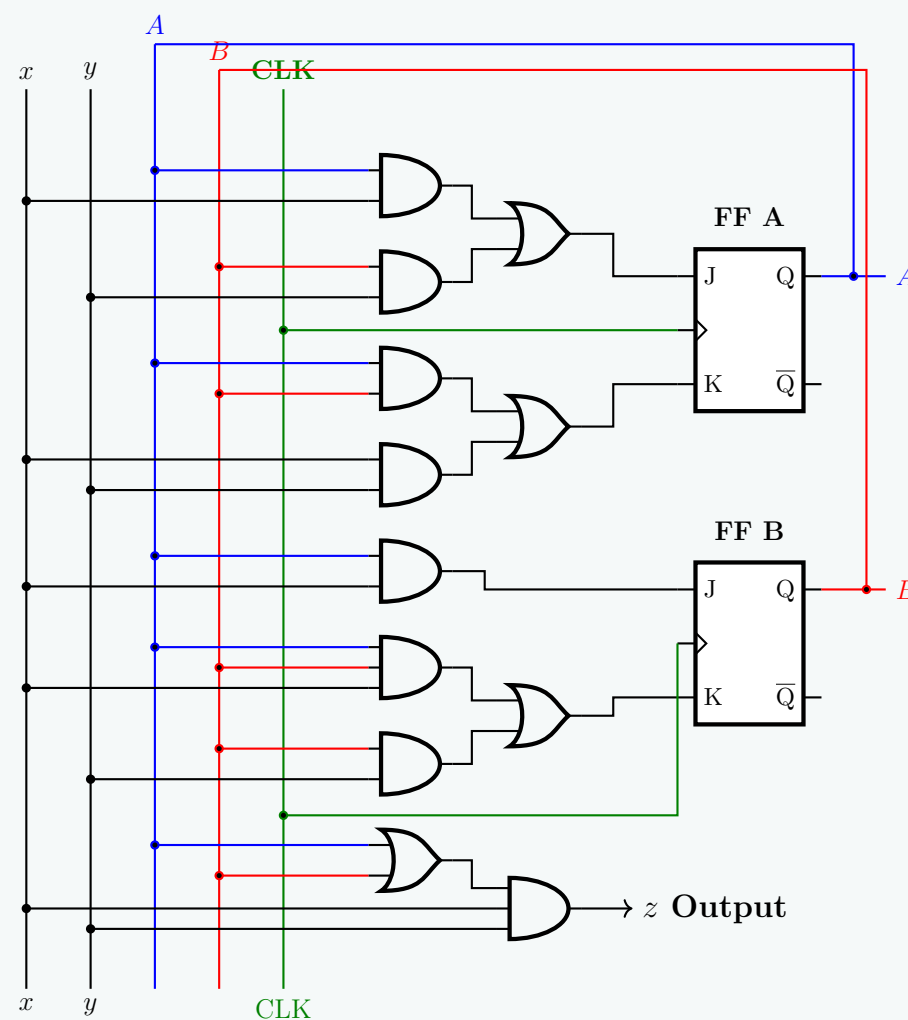
[CSE 205 | L-2/T-1 | 2016–17]

Answer

(a) Logic Diagram Structural Description

Given the structural complexity and density of feedback connections for this multi-input Mealy machine, a highly descriptive structural blueprint is provided to ensure an error-free layout realization (as drawing it with standard wire routing would result in excessive overlapping lines):

- (a) **Memory Elements:** Two edge-triggered JK flip-flops designated as **FF A** (producing state variables A, A') and **FF B** (producing state variables B, B'), synchronized by a common clock signal (CLK).
- (b) **Combinational Excitation Network for FF A:**
 - J_A **Logic:** A 2-input AND gate computes Ax , and another 2-input AND gate computes By . Their outputs feed into a 2-input OR gate driving the J_A terminal.
 - K_A **Logic:** A 2-input AND gate computes AB , and another 2-input AND gate computes xy . Their outputs feed into a 2-input OR gate driving the K_A terminal.
- (c) **Combinational Excitation Network for FF B:**
 - J_B **Logic:** A single 2-input AND gate computes Ax and connects directly to the J_B terminal.
 - K_B **Logic:** A 3-input AND gate computes ABx , and a 2-input AND gate computes By . Their outputs feed into a 2-input OR gate driving the K_B terminal.
- (d) **Output Network (z):**
 - The output is defined as $z = Axy + Bxy$. A simplified implementation uses a 2-input OR gate to compute $(A + B)$, which then feeds into a 3-input AND gate alongside inputs x and y to generate the Mealy output z .



(b) Derivation of the State Table

1. Next-State Equations (Mathematical Derivations)

The characteristic equation for a JK flip-flop is $Q(t+1) = J \cdot Q' + K' \cdot Q$. We substitute the provided excitation equations into this formula for both flip-flops.

For **Flip-Flop A**:

$$\begin{aligned} A(t+1) &= J_A A' + K'_A A \\ &= (Ax + By)A' + (AB + xy)'A \\ &= AA'x + A'By + ((AB)'(xy)')A && \text{(Distributive \& De Morgan's Laws)} \\ &= 0 + A'By + ((A' + B')(x' + y'))A && \text{(Complementarity \& De Morgan's Laws)} \\ &= A'By + (AA' + AB')(x' + y') && \text{(Distributive Law)} \\ &= A'By + (0 + AB')(x' + y') && \text{(Complementarity Law)} \\ &= A'By + AB'x' + AB'y' && \text{(Identity \& Distributive Laws)} \end{aligned}$$

For **Flip-Flop B**:

$$\begin{aligned} B(t+1) &= J_B B' + K'_B B \\ &= (Ax)B' + (ABx + By)'B \\ &= AB'x + ((ABx)'(By)')B && \text{(Distributive \& De Morgan's Laws)} \\ &= AB'x + ((A' + B' + x')(B' + y'))B && \text{(De Morgan's Law)} \\ &= AB'x + (A' + B' + x')(BB' + By') && \text{(Distributive Law)} \\ &= AB'x + (A' + B' + x')(0 + By') && \text{(Complementarity Law)} \\ &= AB'x + A'By' + BB'y' + Bx'y' && \text{(Distributive Law)} \\ &= AB'x + A'By' + Bx'y' && \text{(Complementarity Law)} \end{aligned}$$

2. Excitation and Transition Table

Evaluating the excitation formulas and the derived next-state equations for all $2^4 = 16$ permutations of states (A, B) and inputs (x, y) , we compile the comprehensive transition table.

Present State		Inputs		Excitation Inputs				Next State		Output
A	B	x	y	J _A	K _A	J _B	K _B	A(t + 1)	B(t + 1)	z
0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0	0
0	0	1	1	0	1	0	0	0	0	0
0	1	0	0	0	0	0	0	0	1	0
0	1	0	1	1	0	0	1	1	0	0
0	1	1	0	0	0	0	0	0	1	0
0	1	1	1	1	1	0	1	1	0	1
1	0	0	0	0	0	0	0	1	0	0
1	0	0	1	0	0	0	0	1	0	0
1	0	1	0	1	0	1	0	1	1	0
1	0	1	1	1	1	1	0	0	1	1
1	1	0	0	0	1	0	0	0	1	0
1	1	0	1	1	1	0	1	0	0	0
1	1	1	0	1	1	1	0	0	0	0
1	1	1	1	1	1	1	1	0	0	1

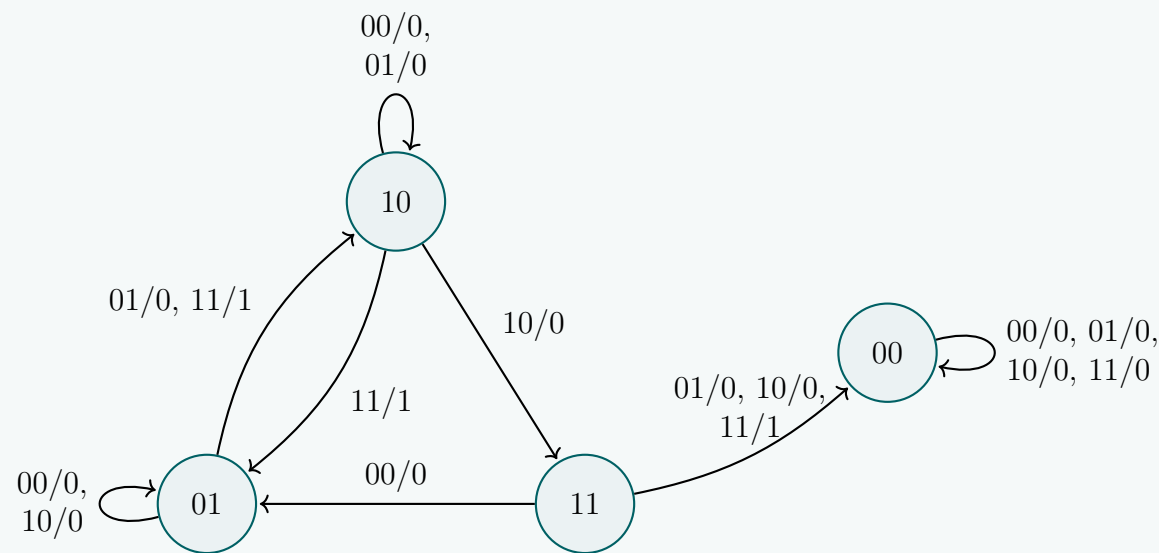
3. Compact State Table

Consolidating the data into a standard two-dimensional state table showing the transitions and outputs:

Present State	Next State A(t + 1)B(t + 1)				Output z			
	xy = 00	01	10	11	xy = 00	01	10	11
00	00	00	00	00	0	0	0	0
01	01	10	01	10	0	0	0	1
10	10	10	11	01	0	0	0	1
11	01	00	00	00	0	0	0	1

(c) State Diagram

The directed graph below illustrates the state transitions. Edges are labeled as Inputs (xy) / Output (z) . To prevent visual clutter, transitions spanning multiple input conditions between the same nodes are grouped together.



Q4. A sequential circuit has two JK Flip-Flops A and B , two inputs x and y , and one output z . The Flip-Flop input equations and circuit output equation are:

$$J_A = Bx + B'y', \quad K_A = B'xy', \quad J_B = A'x, \quad K_B = A + xy', \quad z = Ax'y' + Bx'y'$$

Draw the logic diagram of the circuit, tabulate the state table and derive the state equations for A and B . **(15 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Answer

(a) Logic Diagram Structural Description

Given the structural complexity and density of feedback connections for this multi-input Mealy machine, a highly descriptive structural blueprint is provided to ensure an error-free layout realization:

(a) **Memory Elements:** Two edge-triggered JK flip-flops designated as **FF A** (producing state variables A, A') and **FF B** (producing state variables B, B'), synchronized by a common clock signal (CLK).

(b) Combinational Excitation Network for FF A:

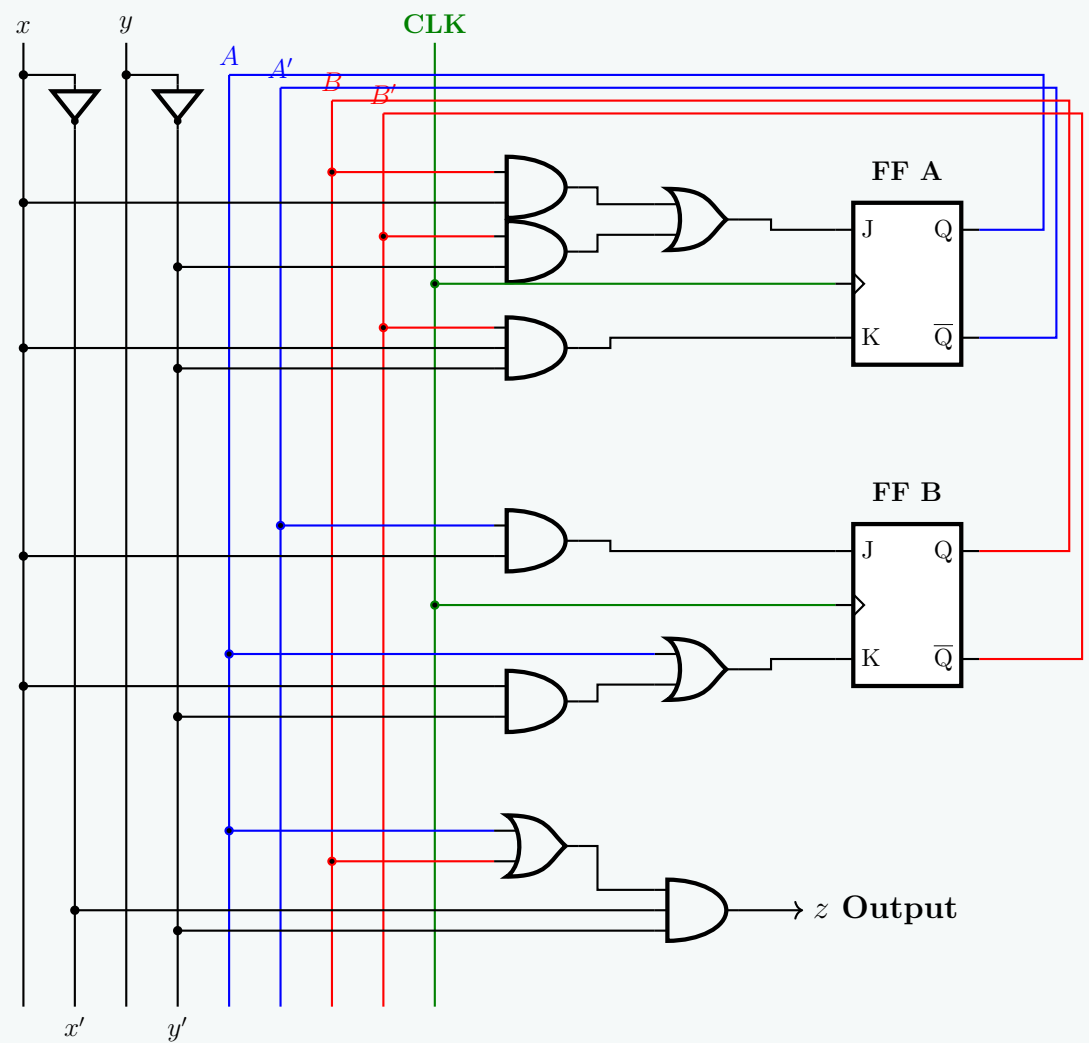
- **J_A Logic:** A 2-input AND gate computes Bx , and another 2-input AND gate computes $B'y'$. Their outputs feed into a 2-input OR gate driving the J_A terminal.
- **K_A Logic:** A 3-input AND gate computes $B'xy'$ and connects directly to the K_A terminal.

(c) Combinational Excitation Network for FF B:

- **J_B Logic:** A 2-input AND gate computes $A'x$ and connects directly to the J_B terminal.
- **K_B Logic:** A 2-input AND gate computes xy' . Its output feeds into a 2-input OR gate alongside signal A , driving the K_B terminal.

(d) Output Network (z):

- The output is defined as $z = Ax'y' + Bx'y'$. This can be structurally implemented using a 3-input AND gate for $Ax'y'$, another 3-input AND gate for $Bx'y'$, and a 2-input OR gate to sum them. (Alternatively, a simplified structure uses an OR gate for $(A + B)$ feeding into a 3-input AND gate with x' and y').



(b) Derivation of the State Equations

The characteristic equation for a JK flip-flop is defined as $Q(t + 1) = J \cdot Q' + K' \cdot Q$. We derive the next-state equations by substituting the provided excitation equations into this characteristic formula.

State Equation for Flip-Flop A:

$$\begin{aligned} A(t + 1) &= J_A A' + K'_A A \\ &= (Bx + B'y')A' + (B'xy')'A \\ &= A'Bx + A'B'y' + (B + (xy)')A \\ &= A'Bx + A'B'y' + (B + x' + y)A \\ &= A'Bx + A'B'y' + AB + Ax' + Ay \end{aligned}$$

(Distributive & De Morgan's Laws)

(De Morgan's Law)

(Distributive Law)

State Equation for Flip-Flop B:

$$\begin{aligned} B(t + 1) &= J_B B' + K'_B B \\ &= (A'x)B' + (A + xy)'B \\ &= A'B'x + (A' \cdot (xy)')B \\ &= A'B'x + A'(x' + y)B \\ &= A'B'x + A'Bx' + A'B y \end{aligned}$$

(Distributive & De Morgan's Laws)

(De Morgan's Law)

(Distributive Law)

(c) State Table

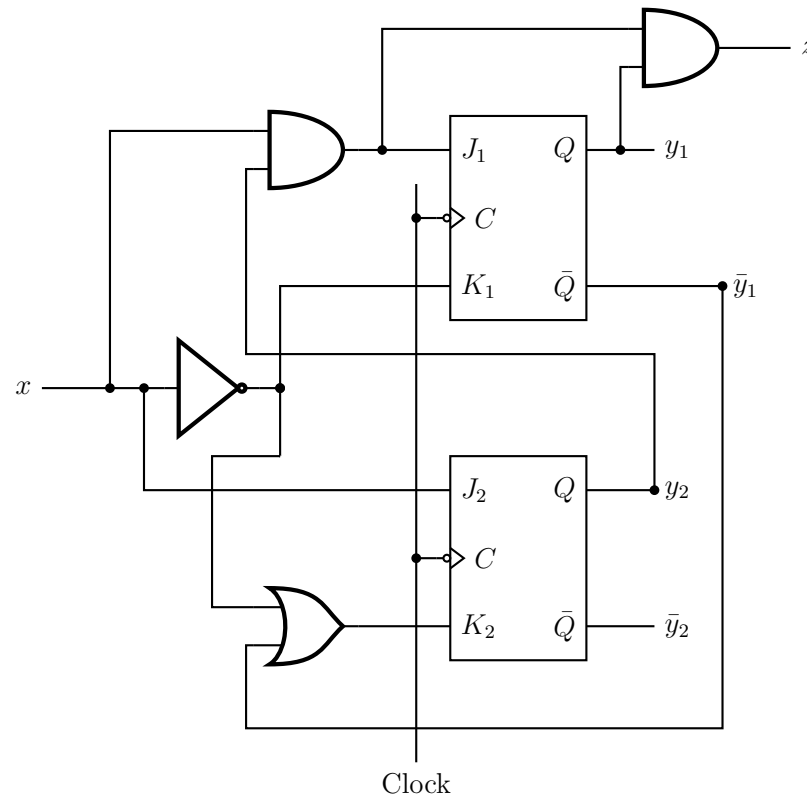
By evaluating the derived state equations $A(t + 1)$ and $B(t + 1)$, alongside the output equation $z = Ax'y' + Bx'y'$, for all $2^4 = 16$ permutations of present states (A, B) and inputs (x, y) , we construct the comprehensive state table.

Compact State Table Representation:

Present State AB	Next State $A(t + 1)B(t + 1)$				Output z			
	$xy = 00$	01	10	11	$xy = 00$	01	10	11
00	10	00	11	01	0	0	0	0
01	01	01	10	11	1	0	0	0
10	10	10	00	10	1	0	0	0
11	10	10	10	10	1	0	0	0

Verification Note for Output: The output equation $z = Ax'y' + Bx'y' = (A + B)x'y'$ asserts that $z = 1$ strictly when $x = 0, y = 0$, and the present state AB is not 00. This perfectly maps to the output partition of the state table above.

Q5. Derive the state diagram from the following sequential circuit.



(15 marks)

[CSE 205 | L-2/T-1 | 2012–13]

Answer

1. Excitation and Output Equations

By analyzing the provided logic diagram, we trace the logic gates connected to the JK flip-flop inputs and the circuit's final output. Let y_1 and y_2 represent the state variables for Flip-Flop 1 (FF1) and Flip-Flop 2 (FF2), respectively.

- **FF1 Inputs:** The J_1 input is driven by a 2-input AND gate receiving x and y_2 . The K_1 input is driven by a NOT gate connected to x .

$$J_1 = xy_2, \quad K_1 = x'$$

- **FF2 Inputs:** The J_2 input is directly connected to x . The K_2 input is driven by a 2-input OR gate receiving x' and y_1' .

$$J_2 = x, \quad K_2 = x' + y_1'$$

- **Output Equation (z):** The output is determined by a 2-input AND gate receiving the output of the first AND gate (xy_2) and y_1 .

$$z = (xy_2)y_1 = xy_1y_2$$

2. Next-State Equations

The characteristic equation for a JK flip-flop is $Q(t+1) = JQ' + K'Q$. We substitute our excitation equations to find the state equations.

For Flip-Flop 1 (y_1):

$$\begin{aligned} y_1(t+1) &= J_1y_1' + K_1'y_1 \\ &= (xy_2)y_1' + (x')'y_1 \\ &= xy_2y_1' + xy_1 && \text{(Involution Law)} \\ &= x(y_1'y_2 + y_1) && \text{(Distributive Law)} \\ &= x(y_1 + y_2) && \text{(Absorption / Rule of Complementarity)} \end{aligned}$$

For Flip-Flop 2 (y_2):

$$\begin{aligned} y_2(t+1) &= J_2y_2' + K_2'y_2 \\ &= (x)y_2' + (x' + y_1')'y_2 \\ &= xy_2' + (xy_1)y_2 && \text{(De Morgan's Law and Involution)} \\ &= x(y_2' + y_1y_2) && \text{(Distributive Law)} \\ &= x(y_2' + y_1) && \text{(Absorption / Rule of Complementarity)} \end{aligned}$$

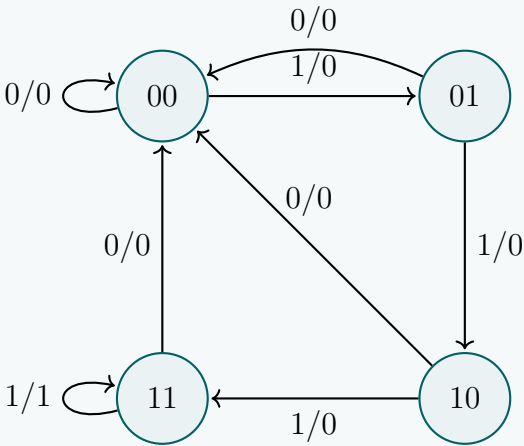
3. State Table

Using the derived equations $y_1(t+1) = x(y_1 + y_2)$, $y_2(t+1) = x(y_1 + y_2')$, and $z = xy_1y_2$, we evaluate the next states and outputs for all combinations of present states (y_1y_2) and input (x). Notice that when $x = 0$, both next-state equations evaluate to 0, transitioning the system to state 00 unconditionally.

Present State y_1y_2	Next State $y_1(t+1)y_2(t+1)$		Output z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
00	00	01	0	0
01	00	10	0	0
10	00	11	0	0
11	00	11	0	1

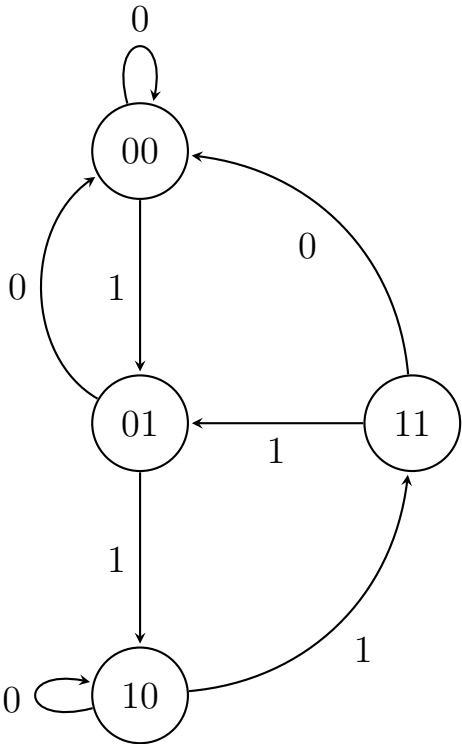
4. State Diagram

Based on the transition table, the state diagram represents a sequence detector that progresses through states when $x = 1$ and instantly resets to 00 whenever $x = 0$.



Circuit Design from State Diagram

- Q1. Design the circuit corresponding to the given state diagram using T flip-flops:
- (a) Write the state table.
 - (b) Find the input equations for the T flip-flops.
 - (c) Draw the circuit diagram.



(21 marks)

[CSE 205 | L-2/T-1 | 2013–14]

Answer

1. State Table and Excitation Table

From the provided state diagram, we can trace the next states for each present state depending on the input x . Let the present state variables be A and B . The excitation for a T flip-flop is determined by the rule $T = Q(t) \oplus Q(t+1)$; the T input must be 1 if the state changes, and 0 if the state remains the same.

Present State		Input	Next State		FF Inputs	
$A(t)$	$B(t)$	x	$A(t+1)$	$B(t+1)$	T_A	T_B
0	0	0	0	0	0	0
0	0	1	0	1	0	1
0	1	0	0	0	0	1
0	1	1	1	0	1	1
1	0	0	1	0	0	0
1	0	1	1	1	0	1
1	1	0	0	0	1	1
1	1	1	0	1	1	0

2. T Flip-Flop Input Equations

We derive the simplified Boolean expressions for T_A and T_B using Karnaugh Maps. The variables are ordered as A, B (rows) and x (columns).



Derivation for T_A :

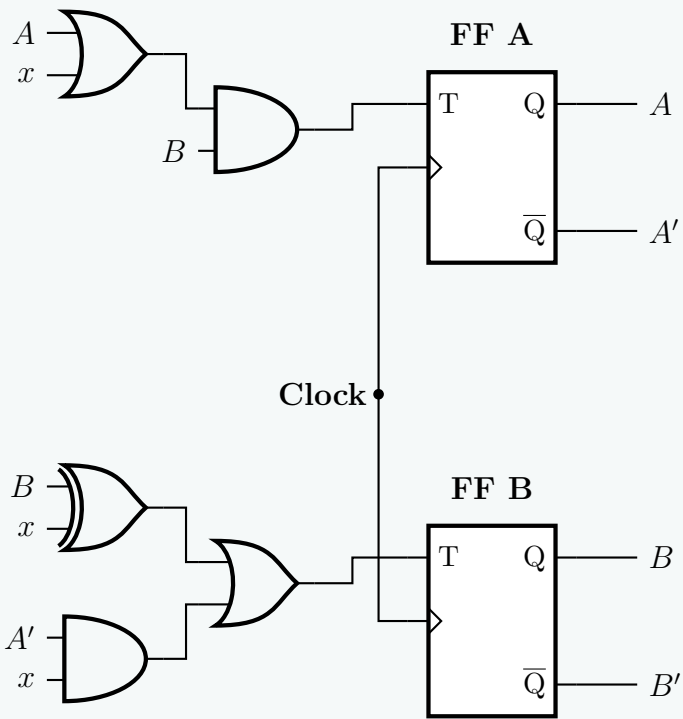
$$T_A = AB + Bx$$
$$= B(A + x) \quad (\text{Optional factoring})$$

Derivation for T_B :

$$T_B = A'x + B'x + Bx'$$
$$= A'x + (B \oplus x) \quad (\text{By definition of XOR})$$

3. Circuit Diagram

The logic diagram is implemented using two T flip-flops. To maintain a clean and professional schematic free of spaghetti wiring, feedback connections are indicated using labeled signal nodes.



Sequence Detector

Q1. Develop a sequential circuit that detects a sequence of three or more consecutive 1's in a string of bits coming through an input line. Use D flip-flops. Show the state diagram, the state table and simplified logic equations. You do not need to show the logic diagram. (12 marks) [CSE 205 | L-2/T-1 | 2022–23]

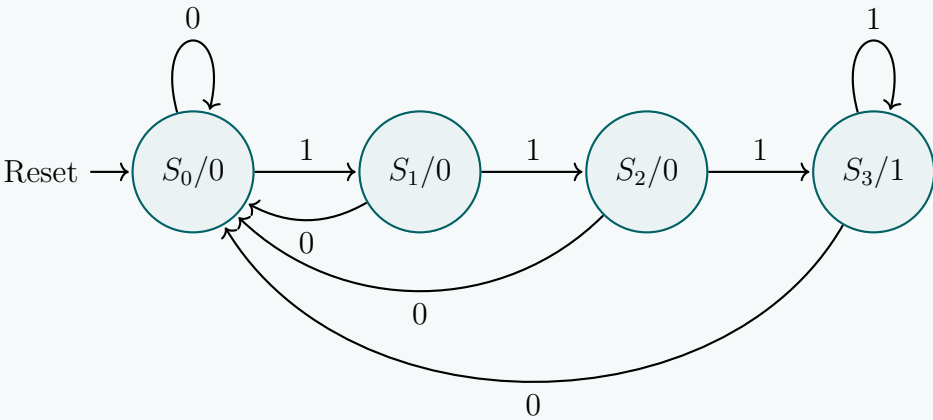
Answer

1. State Diagram

To detect a sequence of three or more consecutive 1s, we design a **Moore-type Finite State Machine (FSM)**. A Moore machine is chosen because the output depends solely on the current state, ensuring a stable, clock-synchronized output indicator. We define four unique states to track the history of the input stream:

- S_0 : Initial state / No consecutive 1s detected. Output $Z = 0$.
- S_1 : A single 1 detected. Output $Z = 0$.
- S_2 : Two consecutive 1s detected. Output $Z = 0$.
- S_3 : Three or more consecutive 1s detected. Output $Z = 1$.

Any incoming 0 breaks the consecutive stream and forces the system to reset back to state S_0 .



2. State Table

We assign two state variables, Q_1 and Q_0 , to encode the four states: $S_0 = 00$, $S_1 = 01$, $S_2 = 10$, and $S_3 = 11$. For a state design utilizing **D flip-flops**, the required excitation input matches the next-state value identically ($D_1 = Q_1(t + 1)$ and $D_0 = Q_0(t + 1)$).

Present State		Input x	Next State		FF Inputs		Output Z
$Q_1(t)$	$Q_0(t)$		$Q_1(t + 1)$	$Q_0(t + 1)$	D_1	D_0	
0	0	0	0	0	0	0	0
0	0	1	0	1	0	1	0
0	1	0	0	0	0	0	0
0	1	1	1	0	1	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	1	1	0
1	1	0	0	0	0	0	1
1	1	1	1	1	1	1	1

3. Simplified Logic Equations

We derive the minimized Boolean expressions using Karnaugh Maps mapped with Q_1Q_0 on the rows and x on the columns, followed by algebraic step-by-step verification.

K-Map for D_1

	x	
	0	1
Q_1Q_0 00		
01		1
11		1
10		1

K-Map for D_0

	x	
	0	1
Q_1Q_0 00		1
01		
11		1
10		1

Mathematical Derivation for D_1 & D_0 :

$$D_1 = Q_1x + Q_0x$$
$$D_0 = Q_1x + Q_0'x$$

Mathematical Derivation for System Output Z :

Because this is a Moore design, the output is directly decoded from the state variables corresponding exclusively to state S_3 :

$$Z = Q_1Q_0$$

4. Logic Circuit Implementation

Q2. Design a sequence recognizer using T flip-flops that recognizes a 4-bit **non-overlapping** binary sequence X from an input stream of bits, where:

Sequence $X = 4\text{-bit binary representation of your roll number} \pmod{16}$

(25 marks)

[CSE 205 | L-2/T-1 | 2019–20]

Answer

Assumption:

Since a specific roll number is not provided, let us assume the student’s roll number ends in 11.

$$11 \pmod{16} = 11_{10} = 1011_2$$

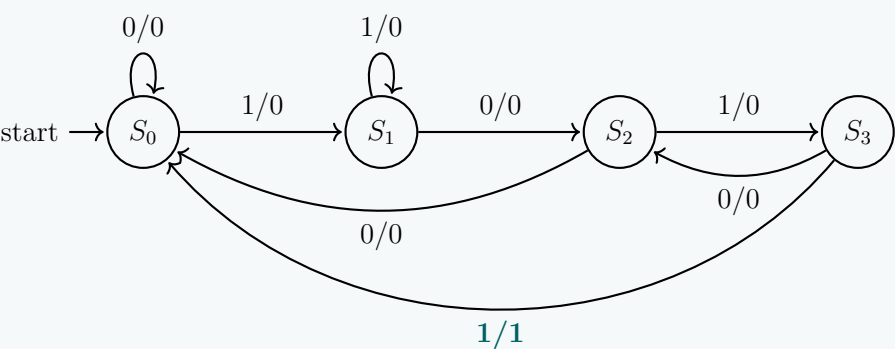
We will design a **Mealy Machine** sequence recognizer for the 4-bit **non-overlapping** sequence **1011**. In a non-overlapping detector, once the full sequence is recognized, the machine resets to the initial state without reusing the final bit for the next sequence.

1. State Definitions and State Diagram (FSM)

Let the sequence be processed bit by bit. We require 4 states to keep track of the successful prefixes of 1011:

- S_0 (**Reset/Initial**): No valid prefix detected. (Encoding: $Q_1Q_0 = 00$)
- S_1 : Prefix ‘1’ detected. (Encoding: $Q_1Q_0 = 01$)
- S_2 : Prefix ‘10’ detected. (Encoding: $Q_1Q_0 = 10$)
- S_3 : Prefix ‘101’ detected. (Encoding: $Q_1Q_0 = 11$)

247



2. State & Excitation Table for T Flip-Flops

The excitation of a T Flip-Flop is given by $T = Q \oplus Q^+$. Using this, we expand our state transition logic:

Present State		Input	Next State		T-FF Inputs		Output
Q_1	Q_0	X	Q_1^+	Q_0^+	T_1	T_0	Y
0	0	0	0	0	0	0	0
0	0	1	0	1	0	1	0
0	1	0	1	0	1	1	0
0	1	1	0	1	0	0	0
1	0	0	0	0	1	0	0
1	0	1	1	1	0	1	0
1	1	0	1	0	0	1	0
1	1	1	0	0	1	1	1

3. Karnaugh Maps and Boolean Equations

We derive the simplified Boolean expressions for T_1 , T_0 , and Output Y using K-Maps.

K-Map for T_1 :

		Q_0X			
		00	01	11	10
Q_1	0				1
	1	1		1	

$$T_1 = \bar{Q}_1Q_0\bar{X} + Q_1\bar{Q}_0\bar{X} + Q_1Q_0X$$
$$= (Q_1 \oplus Q_0)\bar{X} + Q_1Q_0X \quad (\text{Applying XOR logic definition})$$

K-Map for T_0 :

		Q_0X			
		00	01	11	10
Q_1	0		1		1
	1		1	1	1

$$T_0 = \bar{Q}_0X + Q_0\bar{X} + Q_1X$$
$$= (Q_0 \oplus X) + Q_1X \quad (\text{Applying XOR logic definition})$$

Output Y : Since $Y = 1$ only for minterm 7 ($Q_1 = 1, Q_0 = 1, X = 1$), the expression is simply:

$$Y = Q_1Q_0X$$

4. Logic Circuit Implementation

To maintain a clean and perfectly readable schematic, we utilize modular connection labels instead of convoluted crossed wiring.

The circuit diagram shows two T flip-flops, FF₁ and FF₀, connected in a synchronous manner. The inputs are Q₁, Q₀, and X. The outputs are Q₁, Q₀, and Y (Output). The circuit uses AND, OR, and NOT gates to generate the T inputs for the flip-flops. The output Y is the AND of Q₁, Q₀, and X.

Q3. Design a sequence recognizer that recognizes the sequence **1101** from an input sequence (overlapping allowed). Design the circuit using SR Flip-Flops and draw the circuit diagram. **(20 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Answer

To design a sequence recognizer for the sequence **1101** with **overlapping allowed**, we will use a **Mealy Machine** model. In a Mealy machine, the output is a function of both the present state and the current input, which naturally allows for overlapping sequence detection on the final transition.

1. State Definitions and State Diagram (FSM)

We define 4 states to track the valid prefixes of the sequence ‘1101’:

- **S₀ (Reset):** Initial state, no valid prefix detected. (State Encoding: $Q_1Q_0 = 00$)
- **S₁:** Prefix ‘1’ detected. (State Encoding: $Q_1Q_0 = 01$)
- **S₂:** Prefix ‘11’ detected. (State Encoding: $Q_1Q_0 = 10$)
- **S₃:** Prefix ‘110’ detected. (State Encoding: $Q_1Q_0 = 11$)

Since overlapping is allowed, when the machine is in **S₃** (has ‘110’) and receives a ‘1’, the sequence ‘1101’ is completed (Output = 1). The final ‘1’ also acts as the first ‘1’ of a potentially new sequence, so the machine transitions to **S₁** instead of **S₀**.

2. Excitation Table for SR Flip-Flops

The excitation logic for an SR Flip-Flop based on present (Q) and next (Q^+) states is:

$$0 \rightarrow 0 \implies S = 0, R = d \quad | \quad 0 \rightarrow 1 \implies S = 1, R = 0 \quad | \quad 1 \rightarrow 0 \implies S = 0, R = 1 \quad | \quad 1 \rightarrow 1 \implies S = d, R = 0$$

We expand the state diagram into the Transition and Excitation Table:

Present State		Input	Next State		FF1 Inputs		FF0 Inputs		Output
Q_1	Q_0		Q_1^+	Q_0^+	S_1	R_1	S_0	R_0	
0	0	0	0	0	0	d	0	d	0
0	0	1	0	1	0	d	1	0	0
0	1	0	0	0	0	d	0	1	0
0	1	1	1	0	1	0	0	1	0
1	0	0	1	1	d	0	1	0	0
1	0	1	1	0	d	0	0	d	0
1	1	0	0	0	0	1	0	1	0
1	1	1	0	1	0	1	d	0	1

3. Karnaugh Maps and Boolean Equations

We derive the simplified Boolean expressions for S_1 , R_1 , S_0 , R_0 and Output Y .

K-Map for S_1 :

		Q_0X			
		00	01	11	10
Q_1	0			1	
	1	-	-		

$$S_1 = \bar{Q}_1 Q_0 X$$

K-Map for R_1 :

		Q_0X			
		00	01	11	10
Q_1	0	-	-		-
	1			1	1

$$R_1 = Q_1 Q_0$$

K-Map for S_0 :

		Q_0X			
		00	01	11	10
Q_1	0		1		
	1	1		-	

$$\begin{aligned} S_0 &= \bar{Q}_1 \bar{Q}_0 X + Q_1 \bar{Q}_0 \bar{X} \\ &= \bar{Q}_0 (Q_1 \oplus X) \end{aligned}$$

K-Map for R_0 :

		Q_0X			
		00	01	11	10
Q_1	0	-		1	1
	1		-		1

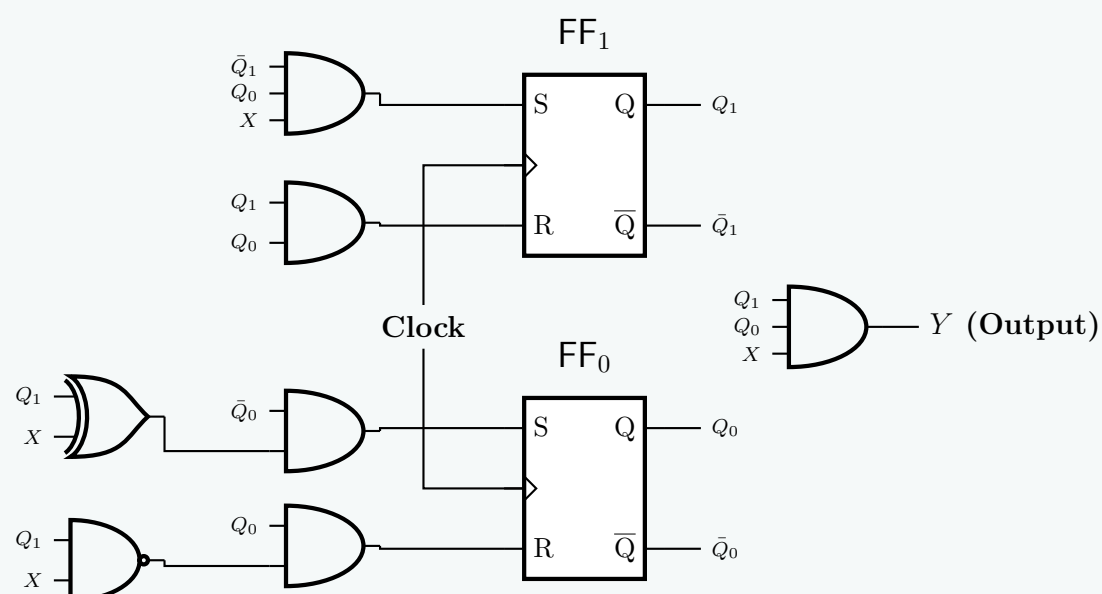
$$\begin{aligned} R_0 &= \bar{Q}_1 Q_0 + Q_0 \bar{X} \\ &= Q_0 (\bar{Q}_1 + \bar{X}) = Q_0 \cdot \overline{(Q_1 X)} \end{aligned}$$

Output Y : Since $Y = 1$ only for minterm 7 ($Q_1 = 1, Q_0 = 1, X = 1$), the expression is:

$$Y = Q_1 Q_0 X$$

4. Logic Circuit Implementation

To maintain clarity, standard combinational logic gates are used alongside modular labels for the Flip-Flop outputs to prevent heavily crossed wires.



Q4. Design a clocked sequential circuit that recognizes the input sequence **1010** (including overlap) such that for input $x = 01010100011010010100$ the output is $z = 00001010000001000010$. Design the circuit using T Flip-Flops and basic gates. (20 marks) [CSE 205 | L-2/T-1 | 2016–17]

Answer

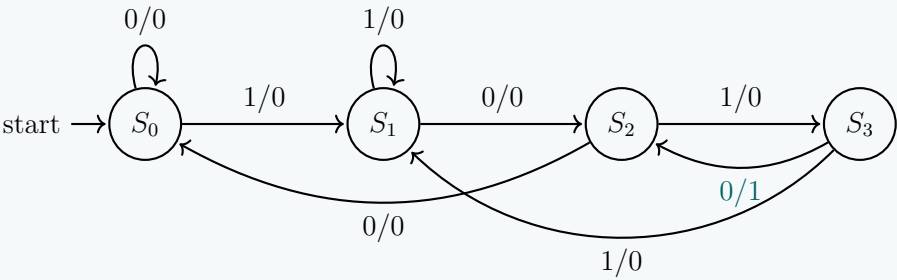
To detect the sequence **1010** with **overlap allowed**, we will design a **Mealy Machine** since the output z depends on both the present state and the current input x . A Mealy machine allows sequence detection to be signaled concurrently with the final bit's arrival.

1. State Definitions and State Diagram (FSM)

We define four states to track the longest successful prefix of the target sequence ‘1010’:

- S_0 (**Reset**): Initial state, no valid prefix. (Encoding: $Q_1Q_0 = 00$)
- S_1 : Prefix ‘1’ detected. (Encoding: $Q_1Q_0 = 01$)
- S_2 : Prefix ‘10’ detected. (Encoding: $Q_1Q_0 = 10$)
- S_3 : Prefix ‘101’ detected. (Encoding: $Q_1Q_0 = 11$)

Handling the overlap: If the machine is in state S_3 (has prefix ‘101’) and receives an input $x = 0$, the full sequence ‘1010’ is detected, so output $z = 1$. The last two bits of this completed sequence (‘10’) concurrently form a valid prefix for the *next* possible sequence. Therefore, the next state is S_2 .



2. Excitation Table for T Flip-Flops

The state transition logic is expanded using the T Flip-Flop excitation rule: $T = Q \oplus Q^+$.

Present State		Input	Next State		T-FF Inputs		Output
Q_1	Q_0	x	Q_1^+	Q_0^+	T_1	T_0	z
0	0	0	0	0	0	0	0
0	0	1	0	1	0	1	0
0	1	0	1	0	1	1	0
0	1	1	0	1	0	0	0
1	0	0	0	0	1	0	0
1	0	1	1	1	0	1	0
1	1	0	1	0	0	1	1
1	1	1	0	1	1	0	0

3. Karnaugh Maps and Boolean Equations

We derive the simplified Boolean expressions for T_1 , T_0 , and z .

K-Map for T_1 :

		Q_0x			
		00	01	11	10
Q_1	0				1
	1	1		1	

K-Map for T_0 :

		Q_0x			
		00	01	11	10
Q_1	0		1		1
	1		1		1

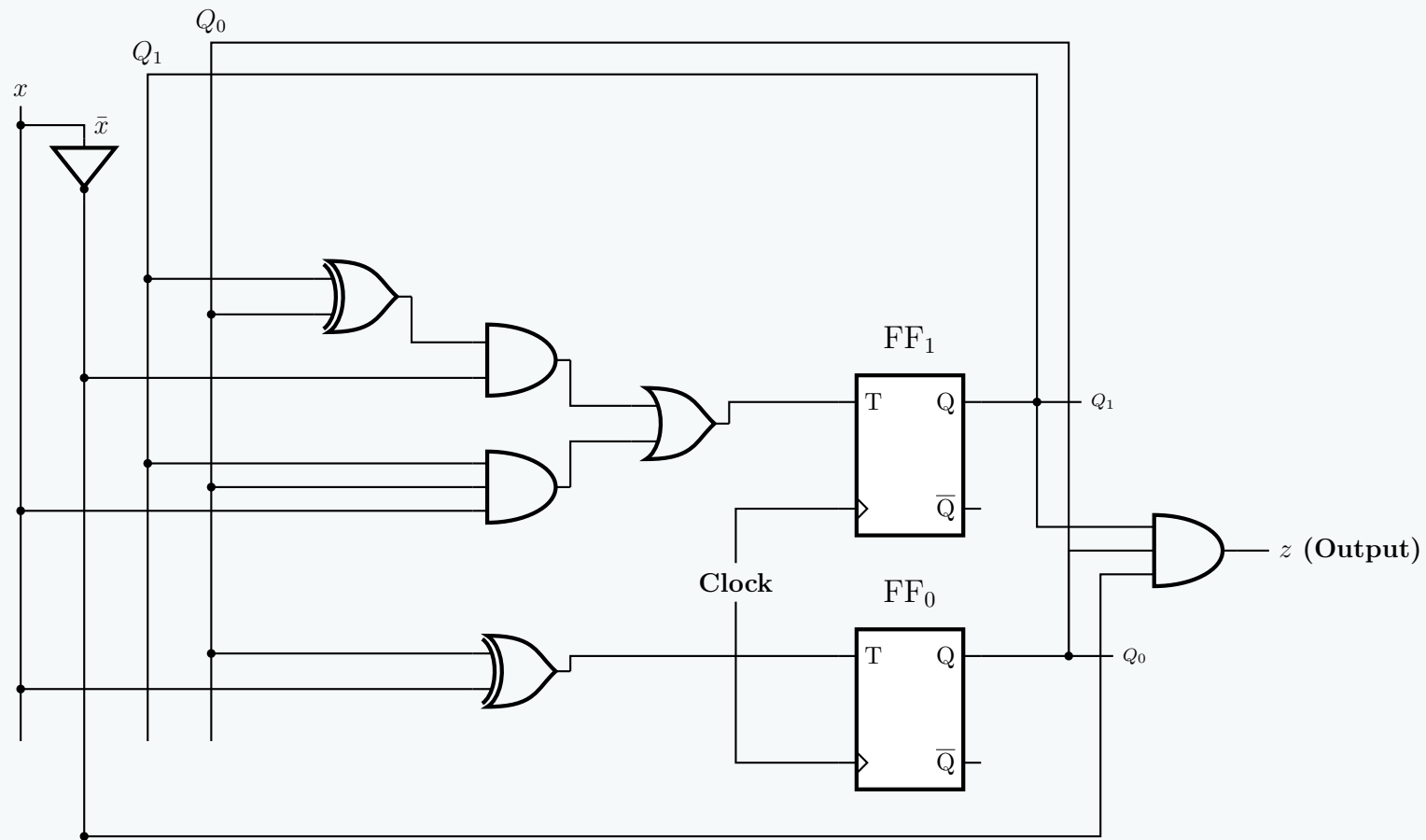
$$\begin{aligned} T_1 &= \bar{Q}_1Q_0\bar{x} + Q_1\bar{Q}_0\bar{x} + Q_1Q_0x \\ &= (Q_1 \oplus Q_0)\bar{x} + Q_1Q_0x \end{aligned}$$

$$\begin{aligned} T_0 &= \bar{Q}_0x + Q_0\bar{x} \\ &= Q_0 \oplus x \end{aligned}$$

Output z K-Map & Equation: The output z is active only at minterm 6 ($Q_1 = 1, Q_0 = 1, x = 0$).

$$z = Q_1Q_0\bar{x}$$

4. Logic Circuit Implementation



Q5. Draw the state diagram in Moore model of a synchronous sequential circuit that results in an output of 1 whenever the sequence 10011 occurs. The circuit is required to recognize overlapping sequences. (8 marks) [CSE 205 | L-2/T-1 | 2012–13]

Answer

To design a **Moore machine** that detects the sequence **10011** with **overlapping allowed**, we associate the output directly with the states. A Moore machine requires one more state than the number of bits in the target sequence to represent the initial state and the progression through the sequence.

1. State Definitions

Let us define the states based on the longest matching prefix of the sequence **10011** that has been detected so far. In a Moore machine, the format of each state is represented as S_i/Z , where Z is the output.

- S_0 : Initial state, no valid prefix detected. Output = 0.
- S_1 : Prefix '1' detected. Output = 0.
- S_2 : Prefix '10' detected. Output = 0.
- S_3 : Prefix '100' detected. Output = 0.
- S_4 : Prefix '1001' detected. Output = 0.
- S_5 : Full sequence '10011' detected. Output = 1.

2. Overlap Handling and Transition Logic

Since the machine allows overlapping, the final bits of one sequence can serve as the beginning of the next sequence. We analyze transitions from the final state S_5 (which has just processed '10011'):

- If the next input is 0: The continuous sequence is '10011' → '0'. The longest valid suffix that forms a new prefix is '10' (from '...100110'). This matches the condition for state S_2 .
- If the next input is 1: The continuous sequence is '10011' → '1'. The longest valid suffix that forms a new prefix is '1' (from '...100111'). This matches the condition for state S_1 .

Similar suffix-prefix matching logic is applied to breakages in the sequence at any intermediate state (e.g., from S_3 receiving a '0', the suffix is '1000', which contains no valid prefix, sending it back to S_0).

3. State Transition Table

Present State	Next State		Output (Z)
	Input $X = 0$	Input $X = 1$	
S_0	S_0	S_1	0
S_1	S_2	S_1	0
S_2	S_3	S_1	0
S_3	S_0	S_4	0
S_4	S_2	S_5	0
S_5	S_2	S_1	1

4. State Diagram

Q6. Design a sequential circuit that detects a sequence of three or more consecutive 1s in a string of bits coming through an input line. Use D flip-flops. The design must include the following:

- (a) State diagram
- (b) State table
- (c) Simplified logic equations
- (d) Logic

(15 marks)

[CSE 205 | L-2/T-1 | 2011–12]

Answer

To design a sequential circuit that detects three or more consecutive 1s, we will use a **Moore Machine** model, where the output depends solely on the current state.

1. State Definitions and State Diagram (FSM)

We define four states to track the number of consecutive 1s received from the input X :

- S_0 (**Reset**): Zero consecutive 1s detected. (Encoding: $Q_1Q_0 = 00$, Output $Y = 0$)
- S_1 : One 1 detected. (Encoding: $Q_1Q_0 = 01$, Output $Y = 0$)
- S_2 : Two consecutive 1s detected. (Encoding: $Q_1Q_0 = 10$, Output $Y = 0$)
- S_3 : Three or more consecutive 1s detected. (Encoding: $Q_1Q_0 = 11$, Output $Y = 1$)

Any input of 0 breaks the sequence and returns the machine to the initial state S_0 .

2. State Table and Excitation Table

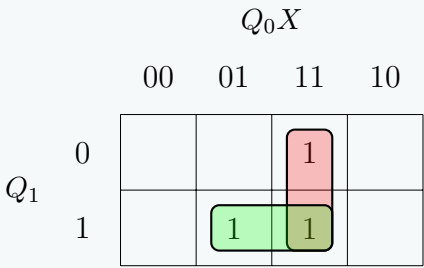
Since we are using D flip-flops, the next state is equal to the D input ($D = Q^+$). We expand the state transitions into a truth table.

Present State		Input	Next State		D-FF Inputs		Output
Q_1	Q_0	X	Q_1^+	Q_0^+	D_1	D_0	Y
0	0	0	0	0	0	0	0
0	0	1	0	1	0	1	0
0	1	0	0	0	0	0	0
0	1	1	1	0	1	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	1	1	0
1	1	0	0	0	0	0	1
1	1	1	1	1	1	1	1

3. Karnaugh Maps and Simplified Logic Equations

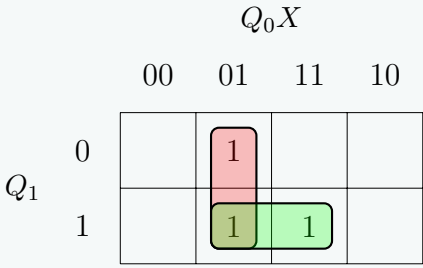
We extract the simplified Boolean expressions for D_1 , D_0 , and Y based on the minterms from the table.

K-Map for D_1 :



$$D_1 = Q_1X + Q_0X$$
$$= X(Q_1 + Q_0) \quad (\text{Distributive Law})$$

K-Map for D_0 :



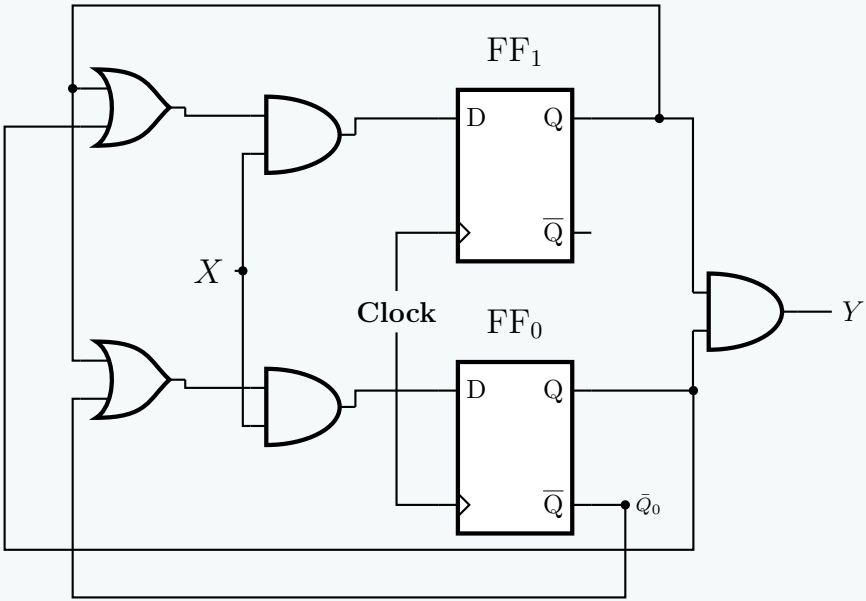
$$D_0 = Q_1X + \bar{Q}_0X$$
$$= X(Q_1 + \bar{Q}_0) \quad (\text{Distributive Law})$$

Output Y Equation: The output Y is independent of the current input X (Moore Machine) and is active only in state S_3 (where $Q_1 = 1$ and $Q_0 = 1$).

$$Y = Q_1 \cdot Q_0$$

4. Logic Circuit Diagram

Using basic logic gates (AND, OR) and two D flip-flops, we construct the required sequential circuit.



Mealy & Moore Model Comparison & Block Diagrams

Q1. Draw the block diagram of Mealy and Moore state machines. State the difference between two models with a simple example. (10 marks)

[CSE 205 | L-2/T-1 | 2024–25, 2022–23, 2015–16, 2013–14, 2011–12]

Answer

1. Block Diagrams of Mealy and Moore Machines

The fundamental difference between the two state machine models lies in their output logic. In a **Mealy machine**, the outputs are a function of both the present state and the current inputs. In a **Moore machine**, the outputs are a function of the present state only.

Mealy Machine

(a)

Moore Machine

(b)

2. Key Differences

Feature	Mealy Machine	Moore Machine
Output Dependence	Outputs depend on both the present state and the present inputs .	Outputs depend only on the present state .
State Count	Generally requires fewer states to implement the same logic.	Generally requires more states , as distinct outputs require distinct states.
Output Timing	Outputs can change asynchronously if an input changes during the clock cycle.	Outputs are strictly synchronous and change only when the state changes.
Design Complexity	Output logic is more complex due to the inclusion of input variables.	Output logic is simpler as it relies solely on flip-flop outputs.

3. Simple Example

Consider designing a sequence detector that outputs a ‘1’ when the sequence ”10” is detected.

- Mealy Machine Approach:** We need only **2 states**: S_0 (Reset/Initial) and S_1 (Prefix ‘1’ detected). If the machine is in state S_1 and receives a ‘0’, it transitions back to S_0 . During this transition, because it sees the required input ‘0’, it instantly produces the output ‘1’. The output is attached to the transition: $S_1 \xrightarrow{\text{input}=0/\text{output}=1} S_0$.
- Moore Machine Approach:** We must use **3 states**: S_0 (Reset, Output=0), S_1 (Prefix ‘1’ detected, Output=0), and S_2 (Sequence ‘10’ detected, Output=1). If the machine is in state S_1 and receives a ‘0’, it must explicitly transition into the new state S_2 . The output ‘1’ is firmly tied to being inside state S_2 , regardless of the inputs.

Q2. Transform the Mealy machine shown below into a corresponding Moore machine.

255

PS	NS, Z	
	$x = 0$	$x = 1$
A	B, 1	C, 0
B	C, 1	D, 0
C	D, 0	E, 1
D	E, 0	A, 1
E	A, 1	B, 1

(6 marks)

[CSE 205 | L-2/T-1 | 2023–24]

Answer

To transform a Mealy machine into a Moore machine, we must associate the outputs with the states rather than the transitions. If a state in the Mealy machine is reached via transitions that produce different outputs, that state must be split into multiple states in the Moore machine, one for each unique output.

Step 1: Analyze Incoming Transitions and Outputs

We examine the given Mealy state table to determine the outputs associated with all incoming transitions for each state.

- **State A:** Reached from D ($x = 1$) with output 1, and from E ($x = 0$) with output 1.
→ Since all incoming transitions have output 1, we create a single Moore state: A_1 (Output = 1).
- **State B:** Reached from A ($x = 0$) with output 1, and from E ($x = 1$) with output 1.
→ All incoming transitions have output 1. We create a single Moore state: B_1 (Output = 1).
- **State C:** Reached from A ($x = 1$) with output 0, and from B ($x = 0$) with output 1.
→ Because it is reached with different outputs, we must **split State C** into two Moore states: C_0 (Output = 0) and C_1 (Output = 1).
- **State D:** Reached from B ($x = 1$) with output 0, and from C ($x = 0$) with output 0.
→ All incoming transitions have output 0. We create a single Moore state: D_0 (Output = 0).
- **State E:** Reached from C ($x = 1$) with output 1, and from D ($x = 0$) with output 0.
→ Because it is reached with different outputs, we must **split State E** into two Moore states: E_0 (Output = 0) and E_1 (Output = 1).

Step 2: Construct the Moore Machine State Table

We now substitute the newly created states into the next-state entries. For states that were split (C and E), both split versions will have identical next-state transition logic, inherited from the original Mealy machine.

- **Transitions for A_1 :** Originally goes to B (output 1) and C (output 0). In our Moore machine, this maps to B_1 and C_0 .
- **Transitions for B_1 :** Originally goes to C (output 1) and D (output 0). Maps to C_1 and D_0 .
- **Transitions for C_0 and C_1 :** Originally goes to D (output 0) and E (output 1). Both map to D_0 and E_1 .
- **Transitions for D_0 :** Originally goes to E (output 0) and A (output 1). Maps to E_0 and A_1 .
- **Transitions for E_0 and E_1 :** Originally goes to A (output 1) and B (output 1). Both map to A_1 and B_1 .

Present State	Next State		Output (Z)
	$x = 0$	$x = 1$	
A_1	B_1	C_0	1
B_1	C_1	D_0	1
C_0	D_0	E_1	0
C_1	D_0	E_1	1
D_0	E_0	A_1	0
E_0	A_1	B_1	0
E_1	A_1	B_1	1

Note: The states highlighted in color indicate the original Mealy states that required splitting to satisfy the Moore machine requirement of state-dependent outputs.

Q3. Transform the following Mealy machine into a corresponding Moore machine.

PS	NS, Z	
	x = 0	x = 1
A	C, 0	B, 0
B	A, 1	D, 0
C	B, 1	A, 1
D	D, 1	C, 0

(5 marks)

[CSE 205 | L-2/T-1 | 2021–22]

Answer

To convert a Mealy machine into a Moore machine, we must shift the outputs from the transitions to the states themselves. If a state in the Mealy machine is entered via transitions that produce different outputs, that state must be split into multiple states in the Moore machine—one for each unique output.

Step 1: Analyze Incoming Transitions and Outputs

We list all unique (Next State, Output) pairs from the Mealy state table to determine the required Moore states.

- **State A:** The incoming transitions are from B ($x = 0$, output 1) and C ($x = 1$, output 1). Since it is only reached with an output of 1, we create a single Moore state: A_1 (Output = 1).
- **State B:** The incoming transitions are from A ($x = 1$, output 0) and C ($x = 0$, output 1). Because it is reached with different outputs, we must **split State B** into two Moore states: B_0 (Output = 0) and B_1 (Output = 1).
- **State C:** The incoming transitions are from A ($x = 0$, output 0) and D ($x = 1$, output 0). Since it is only reached with an output of 0, we create a single Moore state: C_0 (Output = 0).
- **State D:** The incoming transitions are from B ($x = 1$, output 0) and D ($x = 0$, output 1). Because it is reached with different outputs, we must **split State D** into two Moore states: D_0 (Output = 0) and D_1 (Output = 1).

Step 2: Construct the Moore Machine State Table

We populate the next-state entries using our newly defined Moore states. When a state is split (like B and D), both split versions inherit the exact same transition logic from the original Mealy state.

- **Transitions for A_1 :** Originally goes to C (output 0) and B (output 0). In the Moore machine, these map to C_0 and B_0 .
- **Transitions for B_0 and B_1 :** Originally goes to A (output 1) and D (output 0). Both map to A_1 and D_0 .
- **Transitions for C_0 :** Originally goes to B (output 1) and A (output 1). Maps to B_1 and A_1 .
- **Transitions for D_0 and D_1 :** Originally goes to D (output 1) and C (output 0). Both map to D_1 and C_0 .

Present State	Next State		Output (Z)
	$x = 0$	$x = 1$	
A_1	C_0	B_0	1
B_0	A_1	D_0	0
B_1	A_1	D_0	1
C_0	B_1	A_1	0
D_0	D_1	C_0	0
D_1	D_1	C_0	1

Note: The bold, colored states (B_0, B_1, D_0, D_1) highlight where the original Mealy states were split to strictly tie the output to the active state, satisfying the definition of a Moore machine. If A is required to be the initial state and must start with a default output of 0 at $t = 0$, an additional dummy start state A_0 would be added; otherwise, this minimal 6-state conversion is exact.

Q4. Obtain the Mealy equivalent state table for the following Moore machine.

PS	NS		Z
	x = 0	x = 1	
A	A	B	0
B	C	B	0
C	C	D	1
D	A	D	0

(7 marks)

[CSE 205 | L-2/T-1 | 2018–19]

Answer

To convert a Moore machine to its equivalent Mealy machine, we must shift the output dependency from the **states** to the **transitions**.

In a Moore machine, the output is a function of the Present State only. In a Mealy machine, the output is a function of both the Present State and the Input. Therefore, the output associated with the *Next State* in the Moore machine becomes the output for the transition leading to that state in the Mealy machine.

Step 1: Identify the Outputs for Each State

From the given Moore machine state table, we can list the outputs strictly associated with each state:

- Output of State A: $Z(A) = 0$
- Output of State B: $Z(B) = 0$
- Output of State C: $Z(C) = 1$
- Output of State D: $Z(D) = 0$

Step 2: Map Outputs to the Transitions

We keep the Next State (NS) mapping identical to the Moore machine. However, for each transition, we assign the output value of the destination state (the Next State).

- **From State A:**
 - If $x = 0$, NS is A. Since $Z(A) = 0$, the transition output is 0.
 - If $x = 1$, NS is B. Since $Z(B) = 0$, the transition output is 0.
- **From State B:**
 - If $x = 0$, NS is C. Since $Z(C) = 1$, the transition output is **1**.
 - If $x = 1$, NS is B. Since $Z(B) = 0$, the transition output is 0.
- **From State C:**
 - If $x = 0$, NS is C. Since $Z(C) = 1$, the transition output is **1**.
 - If $x = 1$, NS is D. Since $Z(D) = 0$, the transition output is 0.
- **From State D:**
 - If $x = 0$, NS is A. Since $Z(A) = 0$, the transition output is 0.
 - If $x = 1$, NS is D. Since $Z(D) = 0$, the transition output is 0.

Step 3: Construct the Equivalent Mealy State Table

Combining the next states and the newly mapped transition outputs, we obtain the equivalent Mealy machine state table.

Present State	Next State, Output (NS, Z)	
	$x = 0$	$x = 1$
A	A, 0	B, 0
B	C, 1	B, 0
C	C, 1	D, 0
D	A, 0	D, 0

Note: The number of states in the converted Mealy machine is equal to the number of states in the original Moore machine. While further state minimization techniques (like an implication chart) could theoretically be applied to the resulting Mealy machine, this direct mapping represents the immediate, equivalent, unminimized Mealy model.

Q5. Define (i) a Mealy machine and (ii) a Moore machine. (5 marks)

[CSE 205 | L-2/T-1 | 2017–18]

Answer

In the theory of sequential digital logic, Finite State Machines (FSMs) are broadly categorized into two fundamental models based on how their output logic is structured.

(i) Mealy Machine

A **Mealy machine** is a finite state machine whose output values are determined by both its **present state** and its **present inputs**.

- **Mathematical Model:** The output Z at time t is a function of the state S and the input X :

$$Z(t) = f(S(t), X(t))$$

- **Timing and Glitches:** Because the output combinational logic is directly connected to the input lines, any change in the input can cause an immediate, asynchronous change in the output, even between clock edges. This makes Mealy machines susceptible to input glitches.
- **State Efficiency:** They typically require fewer states than Moore machines to implement the same sequential logic function.

(ii) Moore Machine

A **Moore machine** is a finite state machine whose output values are determined *solely* by its **present state**.

- **Mathematical Model:** The output Z at time t is a function of only the state S :

$$Z(t) = f(S(t))$$

- **Timing and Glitches:** Because the output depends exclusively on the memory elements (flip-flops), the outputs only change synchronously with state transitions (i.e., at the active edge of the clock). This effectively isolates the outputs from transient noise or glitches on the input lines.
- **State Efficiency:** They generally require more states to implement a given function because any variation in output necessitates entering a uniquely defined state.

Q6. What are the advantages and disadvantages of using the Mealy model and the Moore model for sequential circuit design? **(6 marks)**
[CSE 205 | L-2/T-1 | 2012–13]

Answer

In sequential circuit design, the choice between a Mealy and a Moore machine involves a fundamental trade-off between speed, hardware complexity, and signal stability.

1. Mealy Model

In a Mealy machine, the output is a function of both the **present state** and the **current inputs**.

Advantages:

- **State Minimization:** Because outputs can vary based on inputs for a given state, a Mealy machine typically requires fewer states to implement the same logic sequence compared to a Moore machine. This can reduce the number of required memory elements (flip-flops).
- **Faster Response Time:** The output reacts immediately to changes in the input without waiting for the next active clock edge. This asynchronous response allows for faster signal propagation through the system.

Disadvantages:

- **Susceptibility to Glitches:** Since outputs are directly connected to inputs via combinational logic, any transient noise, skew, or temporary glitch on the input lines will immediately propagate to the outputs.
- **Timing Complexities:** The asynchronous nature of the outputs can cause timing violations, race conditions, or setup/hold time issues when interfacing directly with other synchronous downstream circuits.

2. Moore Model

In a Moore machine, the output is a function of the **present state only**.

Advantages:

- **Stable and Glitch-Free Outputs:** Outputs are strictly tied to the state memory, meaning they only change synchronously at the active clock edge. This inherent isolation acts as a buffer, shielding the outputs from any intermediate input glitches.
- **Design Simplicity and Safety:** The strict synchronicity makes Moore machines easier to design, debug, and safely cascade. They are highly preferred in complex digital systems (like CPUs and FPGAs) where predictable timing is critical.

Disadvantages:

- **More States Required:** Because any specific output configuration necessitates its own distinct state, Moore machines generally require more states (and potentially more flip-flops and complex next-state logic) than an equivalent Mealy machine.

- **Delayed Output:** The circuit must wait for the active clock edge to transition into the next state before the output reflects the new input conditions, inherently introducing a one-clock-cycle delay.

3. Summary Comparison

Design Criterion	Mealy Machine	Moore Machine
Hardware Economy	High (requires fewer states).	Lower (requires more states).
Output Response	Immediate (asynchronous to input).	Delayed (synchronous to clock).
Noise Immunity	Low (input glitches directly propagate).	High (outputs shielded by flip-flops).
System Interfacing	Requires careful timing analysis to avoid race conditions.	Highly reliable and easy to cascade synchronously.

State Minimization (Implication Table)

Q1. For the state table of a completely specified asynchronous sequential circuit, find out the equivalent partitions, implication table. Formulate the state table of the minimal machine. Find out the states those are equivalent.

x1x2	NS, z			
PS	x1x2=00	x1x2=01	x1x2=11	x1x2=10
A	D/0	D/0	F/0	A/0
B	C/1	D/0	E/1	F/0
C	C/1	D/0	E/1	A/0
D	D/0	B/0	A/0	F/0
E	C/1	F/0	E/1	A/0
F	D/0	D/0	A/0	F/0
G	G/0	G/0	A/0	A/0
H	B/1	D/0	E/1	A/0

(PS = Present state, NS = next state, x1, x2 are input, z is output)

(12 marks)

[CSE 205 | L-2/T-1 | 2024–25]

Answer

To find the minimized machine, we will systematically determine the equivalent states using both the Method of Equivalent Partitions and the Implication Table.

Step 1: Equivalent Partitions Method

Two states can only be equivalent if they produce the exact same output sequence. We start by partitioning the states based on their outputs for the input sequences ($x_1x_2 = 00, 01, 11, 10$).

- **1-Equivalent Partition (P_1):** Grouping states by their output patterns.
 - Pattern (0, 0, 0, 0): States A, D, F, G
 - Pattern (1, 0, 1, 0): States B, C, E, H
$$P_1 = (A, D, F, G) (B, C, E, H)$$
- **2-Equivalent Partition (P_2):** We check the next states of elements in P_1 to see if they transition to different blocks of P_1 .
 - For block (A, D, F, G) under input 01: $A \rightarrow D$ (Block 1), $F \rightarrow D$ (Block 1), $G \rightarrow G$ (Block 1), but $D \rightarrow B$ (Block 2). Thus, D must be separated.
 - For block (B, C, E, H) , all next states under all inputs remain in the same corresponding blocks.
$$P_2 = (A, F, G) (D) (B, C, E, H)$$
- **3-Equivalent Partition (P_3):** We evaluate based on P_2 .
 - For block (A, F, G) under input 00: $A \rightarrow D$ (Block D), $F \rightarrow D$ (Block D), but $G \rightarrow G$ (Block AFG). Thus, G must be separated.

- For block (B, C, E, H) under input 01: $B \rightarrow D$ (Block D), $C \rightarrow D$ (Block D), $H \rightarrow D$ (Block D), but $E \rightarrow F$ (Block AF). Thus, E must be separated.

$$P_3 = (A, F) (G) (D) (B, C, H) (E)$$

- **4-Equivalent Partition (P_4):** Evaluating based on P_3 .

- For block (A, F) : Both transition to D , $D, \{A, F\}, \{A, F\}$. They are perfectly equivalent.
- For block (B, C, H) : All transition to $\{B, C, H\}, D, E, \{A, F\}$. They are perfectly equivalent.

$$P_4 = (A, F) (G) (D) (B, C, H) (E)$$

Since $P_3 = P_4$, the partitioning is complete.

Step 2: Implication Table Method

We construct an implication table to verify our partitions. Pairs from different output groups (different P_1 blocks) are immediately incompatible ($\times$). For pairs within the same group, we record the dependent next-state pairs.

- **Group 1 Dependencies $\{A, D, F, G\}$:**

- $(A, D) \Rightarrow (D, B)$ under 01. Since D, B are in different P_1 blocks, (A, D) is invalid ($\times$).
- $(A, F) \Rightarrow (F, A)$ under 10. Self-implying, so (A, F) is equivalent ($\checkmark$).
- $(F, D) \Rightarrow (B, D)$ under 01. Invalid ($\times$).
- $(G, D) \Rightarrow (B, G)$ under 01. Invalid ($\times$).
- $(G, F) \Rightarrow (D, G)$ under 00. But (D, G) is already invalid, so (G, F) is invalid ($\times$).
- $(G, A) \Rightarrow (D, G)$ under 00. Invalid ($\times$).

- **Group 2 Dependencies $\{B, C, E, H\}$:**

- $(B, C) \Rightarrow (A, F)$. Since (A, F) is valid, (B, C) is valid ($\checkmark$).
- $(B, E) \Rightarrow (D, F)$ under 01. Since (D, F) is invalid, (B, E) is invalid ($\times$).
- $(B, H) \Rightarrow (B, C)$ and (A, F) . Both are valid, so (B, H) is valid ($\checkmark$).
- $(C, E) \Rightarrow (D, F)$ under 01. Invalid ($\times$).
- $(C, H) \Rightarrow (B, C)$. Valid ($\checkmark$).
- $(E, H) \Rightarrow (D, F)$ under 01. Invalid ($\times$).

B	$\times$						
C	$\times$	A-F, $\checkmark$					
D	D-B, $\times$	$\times$	$\times$				
E	$\times$	D-F, $\times$	D-F, $\times$	$\times$			
F	$\checkmark$	$\times$	$\times$	B-D, $\times$	$\times$		
G	D-G, $\times$	$\times$	$\times$	B-G, $\times$	$\times$	D-G, $\times$	
H	$\times$	B-C, A-F, $\checkmark$	B-C, $\checkmark$	$\times$	D-F, $\times$	$\times$	$\times$
	A	B	C	D	E	F	G

Step 3: Finding Equivalent States

Both methods confirm the equivalent state classes are:

$$A \equiv F$$

$$B \equiv C \equiv H$$

The minimal machine will have 5 states. We choose $\{A, B, D, E, G\}$ as representative states, meaning we substitute $F \rightarrow A$, $C \rightarrow B$, and $H \rightarrow B$ in the original transitions.

Step 4: Formulate the Minimal State Table

By replacing the redundant states with their equivalent representatives, we derive the following minimized state table:

Present State	Next State, Output (z)			
	$x_1x_2 = 00$	$x_1x_2 = 01$	$x_1x_2 = 11$	$x_1x_2 = 10$
A	D / 0	D / 0	A / 0	A / 0
B	B / 1	D / 0	E / 1	A / 0
D	D / 0	B / 0	A / 0	A / 0
E	B / 1	A / 0	E / 1	A / 0
G	G / 0	G / 0	A / 0	A / 0

Q2. Reduce the number of states of the following state table using implication table. Show the reduced state table.

State	Next State		Output	
	Input $X = 0$	Input $X = 1$	Input $X = 0$	Input $X = 1$
A	A	B	0	0
B	C	D	0	1
C	A	B	0	0
D	E	D	0	1
E	A	D	0	1

(11 marks)

[CSE 205 | L-2/T-1 | 2022–23]

Answer

Step 1: Initial Grouping by Outputs

First, we categorize the states based on their output values for inputs ($X = 0, X = 1$). States can only be equivalent if their outputs perfectly match.

- Output (0, 0): States **A, C**
- Output (0, 1): States **B, D, E**

Any pair of states belonging to different output groups cannot be equivalent. We place an “ $\times$ ” in their corresponding cells in the implication table.

Step 2: Determining Next-State Dependencies

For state pairs within the same output group, we analyze their next-state transitions:

- Pair (A, C):** Next states are (A, A) for $X = 0$ and (B, B) for $X = 1$. Because both inputs lead to identical next states, they are strictly equivalent. We place a $\checkmark$ in the (A, C) cell.
- Pair (B, D):** Next states are (C, E) for $X = 0$ and (D, D) for $X = 1$. The equivalency of (B, D) depends on the equivalency of (C, E).
- Pair (B, E):** Next states are (C, A) for $X = 0$ and (D, D) for $X = 1$. The equivalency of (B, E) depends on the equivalency of (A, C).
- Pair (D, E):** Next states are (E, A) for $X = 0$ and (D, D) for $X = 1$. The equivalency of (D, E) depends on the equivalency of (A, E).

Step 3: Evaluating Dependencies (Implication Table)

We systematically resolve the implied dependencies:

- Cell (B, D) relies on (C, E). Because C and E have different outputs, cell (C, E) is an $\times$. Consequently, (B, D) resolves to an $\times$.
- Cell (D, E) relies on (A, E). Because A and E have different outputs, cell (A, E) is an $\times$. Consequently, (D, E) resolves to an $\times$.
- Cell (B, E) relies on (A, C). We already established that (A, C) is fully equivalent ($\checkmark$). Therefore, (B, E) also becomes $\checkmark$.

B	$\times$			
C	$\checkmark$	$\times$		
D	$\times$	$\times$ (via C, E)	$\times$	
E	$\times$	$\checkmark$ (via A, C)	$\times$	$\times$ (via A, E)
	A	B	C	D

Step 4: Identifying Equivalent States
From the completed implication table, we extract the equivalent state pairs:
$$A \equiv C$$
$$B \equiv E$$

We can eliminate states C and E, replacing every occurrence of C with A, and E with B.

Step 5: Formulating the Reduced State Table
Applying the substitutions $C \rightarrow A$ and $E \rightarrow B$ to the transition rules of the remaining independent states (A, B, and D), we derive the final reduced state table:

State	Next State		Output	
	Input X = 0	Input X = 1	Input X = 0	Input X = 1
A	A	B	0	0
B	A	D	0	1
D	B	D	0	1

Q3. For the state-table of a completely specified circuit (PS: A–H), find the equivalence partitions and write the state-table of the minimal machine. Name the state that are equivalent .

PS	NS, $Z(x = 0)$	NS, $Z(x = 1)$
A	B,1	H,1
B	F,1	D,1
C	D,0	E,1
D	C,0	F,1
E	D,1	C,1
F	C,1	C,1
G	C,1	D,1
H	C,0	A,1

(10 marks)

[CSE 205 | L-2/T-1 | 2020–21]

Answer

To find the equivalence partitions and minimize the state machine, we will iteratively partition the states into equivalence classes. We denote the partition at step k as P_k .

Step 1: 1-Equivalence Partition (P_1)

Two states are 1-equivalent if they produce the same outputs for all possible inputs. We examine the output Z for inputs $x = 0$ and $x = 1$ from the original state table:

- States with outputs (0, 1) for $(x = 0, x = 1)$: C, D, H
- States with outputs (1, 1) for $(x = 0, x = 1)$: A, B, E, F, G

This gives us our first partition block assignment:

$$P_1 = (C, D, H)(A, B, E, F, G)$$
Step 2: 2-Equivalence Partition (P_2)

Next, we check if states within the same block in P_1 transition to states in the same P_1 blocks. Let the blocks of P_1 be $B_1 = (C, D, H)$ and $B_2 = (A, B, E, F, G)$.

Checking Block $B_1 = (C, D, H)$:

- $C \rightarrow \text{NS}$ is (D, E) $\in (B_1, B_2)$
- $D \rightarrow \text{NS}$ is (C, F) $\in (B_1, B_2)$
- $H \rightarrow \text{NS}$ is (C, A) $\in (B_1, B_2)$

Since all states in B_1 transition to the block sequence (B_1, B_2) , B_1 remains intact.

Checking Block $B_2 = (A, B, E, F, G)$:

- $A \rightarrow \text{NS}$ is (B, H) $\in (B_2, B_1)$
- $B \rightarrow \text{NS}$ is (F, D) $\in (B_2, B_1)$
- $E \rightarrow \text{NS}$ is (D, C) $\in (B_1, B_1)$

- $F \rightarrow \text{NS}$ is $(C, C) \in (B_1, B_1)$
- $G \rightarrow \text{NS}$ is $(C, D) \in (B_1, B_1)$

Block B_2 must be split because (A, B) transition to (B_2, B_1) while (E, F, G) transition to (B_1, B_1) . The new partition is:

$$P_2 = (C, D, H)(A, B)(E, F, G)$$

Step 3: 3-Equivalence Partition (P_3)

Let the blocks of P_2 be $B_1 = (C, D, H)$, $B_2 = (A, B)$, and $B_3 = (E, F, G)$. We check transitions against P_2 blocks.

Checking Block $B_1 = (C, D, H)$:

- $C \rightarrow \text{NS}$ is $(D, E) \in (B_1, B_3)$
- $D \rightarrow \text{NS}$ is $(C, F) \in (B_1, B_3)$
- $H \rightarrow \text{NS}$ is $(C, A) \in (B_1, B_2)$

State H transitions to a different block configuration and must be separated. B_1 splits into (C, D) and (H) .

Checking Block $B_2 = (A, B)$:

- $A \rightarrow \text{NS}$ is $(B, H) \in (B_2, B_1)$
- $B \rightarrow \text{NS}$ is $(F, D) \in (B_3, B_1)$

State A and B transition to different blocks. B_2 splits into (A) and (B) .

Checking Block $B_3 = (E, F, G)$:

- $E \rightarrow \text{NS}$ is $(D, C) \in (B_1, B_1)$
- $F \rightarrow \text{NS}$ is $(C, C) \in (B_1, B_1)$
- $G \rightarrow \text{NS}$ is $(C, D) \in (B_1, B_1)$

All transition to (B_1, B_1) , so B_3 remains intact. The new partition is:

$$P_3 = (C, D)(H)(A)(B)(E, F, G)$$

Step 4: 4-Equivalence Partition (P_4)

Re-evaluating the remaining multi-state blocks (C, D) and (E, F, G) against the P_3 blocks:

- $C \rightarrow (D, E)$ and $D \rightarrow (C, F)$. Both map to the block sequence $((C, D), (E, F, G))$. They remain equivalent.
- E, F, G all map to states in (C, D) for both $x = 0$ and $x = 1$. They remain equivalent.

Since $P_4 = P_3$, the partitioning process is complete.

Equivalent States

The equivalent states are $\mathbf{C} \equiv \mathbf{D}$ and $\mathbf{E} \equiv \mathbf{F} \equiv \mathbf{G}$.

State-Table of the Minimal Machine

We will represent the equivalence classes with a single state name: Let $S_A = A$, $S_B = B$, $S_C = (C, D)$, $S_E = (E, F, G)$, and $S_H = H$. Substituting these mapped states into the next-state logic gives the minimized table:

Present State	Next State, Z ($x = 0$)	Next State, Z ($x = 1$)
S_A	$S_B, 1$	$S_H, 1$
S_B	$S_E, 1$	$S_C, 1$
S_C	$S_C, 0$	$S_E, 1$
S_E	$S_C, 1$	$S_C, 1$
S_H	$S_C, 0$	$S_A, 1$

Q4. For the state-table of a completely specified circuit, find the equivalent partitions and write the state-table of the minimal machine. Name the equivalent states.

PS	NS, Z		
	I	J	K
A	A,0	B,1	E,1
B	B,0	A,1	F,1
C	A,1	D,0	E,0
D	F,0	C,1	A,0
E	A,0	D,1	E,1
F	B,0	D,1	F,1

(8 marks)

[CSE 205 | L-2/T-1 | 2018–19]

Answer

To find the equivalent states and minimize the state machine, we apply the equivalence partition method. We will group states into blocks based on their outputs and transitions, refining the partitions iteratively until no further splits can be made.

Step 1: 1-Equivalence Partition (P_1)

Two states are 1-equivalent if they produce the same output sequence for all possible inputs (I, J, K). We extract the outputs from the original state table:

- Output for A is (0, 1, 1)
- Output for B is (0, 1, 1)
- Output for C is (1, 0, 0)
- Output for D is (0, 1, 0)
- Output for E is (0, 1, 1)
- Output for F is (0, 1, 1)

Grouping the states by their output patterns gives us our first partition, P_1 :

$$P_1 = (A, B, E, F)(C)(D)$$

Let the blocks be $B_1 = (A, B, E, F)$, $B_2 = (C)$, and $B_3 = (D)$.

Step 2: 2-Equivalence Partition (P_2)

Next, we check if states within the same block in P_1 transition to states in the same P_1 blocks for all inputs. The only multi-state block to evaluate is $B_1 = (A, B, E, F)$.

Checking Block $B_1 = (A, B, E, F)$:

- $A \xrightarrow{I,J,K} (A, B, E) \in (B_1, B_1, B_1)$
- $B \xrightarrow{I,J,K} (B, A, F) \in (B_1, B_1, B_1)$
- $E \xrightarrow{I,J,K} (A, D, E) \in (B_1, B_3, B_1)$
- $F \xrightarrow{I,J,K} (B, D, F) \in (B_1, B_3, B_1)$

States A and B transition to block sequence (B_1, B_1, B_1) , while states E and F transition to block sequence (B_1, B_3, B_1) . Therefore, block B_1 must be split into two new blocks: (A, B) and (E, F) .

Our new partition is:

$$P_2 = (A, B)(E, F)(C)(D)$$

Step 3: 3-Equivalence Partition (P_3)

We define the new blocks as $B_1 = (A, B)$, $B_2 = (E, F)$, $B_3 = (C)$, and $B_4 = (D)$, and evaluate the multi-state blocks again.

Checking Block $B_1 = (A, B)$:

- $A \xrightarrow{I,J,K} (A, B, E) \in (B_1, B_1, B_2)$
- $B \xrightarrow{I,J,K} (B, A, F) \in (B_1, B_1, B_2)$

Both A and B transition to the exact same block sequence. They remain in the same block.

Checking Block $B_2 = (E, F)$:

- $E \xrightarrow{I,J,K} (A, D, E) \in (B_1, B_4, B_2)$
- $F \xrightarrow{I,J,K} (B, D, F) \in (B_1, B_4, B_2)$

Both E and F transition to the exact same block sequence. They remain in the same block.

Since no blocks were split, $P_3 = P_2$. The minimization process is complete.

Equivalent States

The equivalent partitions yield the following equivalent states:

$$A \equiv B \quad \text{and} \quad E \equiv F$$

State-Table of the Minimal Machine

We replace equivalent states with a single representative state. Let's use A to represent the (A, B) block and E to represent the (E, F) block.

In the Next State logic:

- Any transition to B is replaced by A .
- Any transition to F is replaced by E .

Present State	Next State, Output (Z)		
	I	J	K
A	$A, 0$	$A, 1$	$E, 1$
C	$A, 1$	$D, 0$	$E, 0$
D	$E, 0$	$C, 1$	$A, 0$
E	$A, 0$	$D, 1$	$E, 1$

Note: The minimized machine reduces the original 6-state design to just 4 states.

Q5. For the state-table of a completely specified circuit, find the equivalence partitions and write the state-table of the minimal machine. Name the equivalent states.

x_1x_2	NS,Z			
PS	00	01	11	10
A	D, 0	D, 0	F, 0	A, 0
B	C, 1	D, 0	E, 1	F, 0
C	C, 1	D, 0	E, 1	A, 0
D	D, 0	B, 0	A, 0	F, 0
E	C, 1	F, 0	E, 1	A, 0
F	D, 0	D, 0	A, 0	F, 0
G	G, 0	G, 0	A, 0	A, 0
H	B, 1	D, 0	E, 1	A, 0

(10 marks)

[CSE 205 | L-2/T-1 | 2017–18]

Answer

To find the equivalent partitions and minimize the state machine, we will group the states into blocks based on their outputs and transition destinations. We refine these partitions iteratively until no further splits occur.

Step 1: 1-Equivalence Partition (P_1)

Two states are 1-equivalent if they have the exact same output sequence for all inputs ($x_1x_2 = 00, 01, 11, 10$). We extract the output values from the given state table:

- States A, D, F, G produce outputs: $(0, 0, 0, 0)$
- States B, C, E, H produce outputs: $(1, 0, 1, 0)$

Grouping the states by their output patterns gives our first partition, P_1 :

$$P_1 = (A, D, F, G)(B, C, E, H)$$

Step 2: 2-Equivalence Partition (P_2)

Let the blocks of P_1 be $B_1 = (A, D, F, G)$ and $B_2 = (B, C, E, H)$. We check if states within the same block transition to the same P_1 blocks for all inputs.

Checking Block $B_1 = (A, D, F, G)$:

- $A \xrightarrow{00,01,11,10} (D, D, F, A) \in (B_1, B_1, B_1, B_1)$
- $D \xrightarrow{00,01,11,10} (D, B, A, F) \in (B_1, \mathbf{B_2}, B_1, B_1) \rightarrow \text{Transitions to a different block on } 01.$
- $F \xrightarrow{00,01,11,10} (D, D, A, F) \in (B_1, B_1, B_1, B_1)$

- $G \xrightarrow{00,01,11,10} (G, G, A, A) \in (B_1, B_1, B_1, B_1)$

Block B_1 splits into (A, F, G) and (D) .

Checking Block $B_2 = (B, C, E, H)$:

- $B \xrightarrow{00,01,11,10} (C, D, E, F) \in (B_2, B_1, B_2, B_1)$
- $C \xrightarrow{00,01,11,10} (C, D, E, A) \in (B_2, B_1, B_2, B_1)$
- $E \xrightarrow{00,01,11,10} (C, F, E, A) \in (B_2, B_1, B_2, B_1)$
- $H \xrightarrow{00,01,11,10} (B, D, E, A) \in (B_2, B_1, B_2, B_1)$

All states in B_2 share the same block transition pattern. B_2 remains intact. The new partition is:

$$P_2 = (A, F, G)(D)(B, C, E, H)$$

Step 3: 3-Equivalence Partition (P_3)

Let $B_1 = (A, F, G)$, $B_2 = (D)$, and $B_3 = (B, C, E, H)$. We re-evaluate the multi-state blocks.

Checking Block $B_1 = (A, F, G)$:

- $A \xrightarrow{00,01,11,10} (D, D, F, A) \in (B_2, B_2, B_1, B_1)$
- $F \xrightarrow{00,01,11,10} (D, D, A, F) \in (B_2, B_2, B_1, B_1)$
- $G \xrightarrow{00,01,11,10} (G, G, A, A) \in (\mathbf{B}_1, \mathbf{B}_1, B_1, B_1) \rightarrow \text{Transitions to different blocks on } 00, 01.$

Block B_1 splits into (A, F) and (G) .

Checking Block $B_3 = (B, C, E, H)$:

- $B \xrightarrow{00,01,11,10} (C, D, E, F) \in (B_3, B_2, B_3, B_1)$
- $C \xrightarrow{00,01,11,10} (C, D, E, A) \in (B_3, B_2, B_3, B_1)$
- $E \xrightarrow{00,01,11,10} (C, F, E, A) \in (B_3, \mathbf{B}_1, B_3, B_1) \rightarrow \text{Transitions to a different block on } 01.$
- $H \xrightarrow{00,01,11,10} (B, D, E, A) \in (B_3, B_2, B_3, B_1)$

Block B_3 splits into (B, C, H) and (E) . The new partition is:

$$P_3 = (A, F)(G)(D)(B, C, H)(E)$$

Step 4: 4-Equivalence Partition (P_4)

We evaluate the remaining multi-state blocks $B_1 = (A, F)$ and $B_4 = (B, C, H)$ using the P_3 blocks.

- **A and F:** Both transition to $(D, D, A/F, A/F)$, matching the block sequence $((D), (D), (A, F), (A, F))$. They remain equivalent.
- **B, C, and H:** All transition to $(B/C, D, E, A/F)$, matching the block sequence $((B, C, H), (D), (E), (A, F))$. They remain equivalent.

Since $P_4 = P_3$, the partitioning process is complete.

Equivalent States

The equivalent states are:

$$\mathbf{A} \equiv \mathbf{F} \quad \text{and} \quad \mathbf{B} \equiv \mathbf{C} \equiv \mathbf{H}$$

State-Table of the Minimal Machine

We assign a single representative state to each equivalence class. Let's use A to represent (A, F) , and B to represent (B, C, H) . The states are now A, B, D, E, G . We replace any occurrences of F with A , and C, H with B in the next-state logic.

Present State	Next State, Output (Z)			
	00	01	11	10
A	$D, 0$	$D, 0$	$A, 0$	$A, 0$
B	$B, 1$	$D, 0$	$E, 1$	$A, 0$
D	$D, 0$	$B, 0$	$A, 0$	$A, 0$
E	$B, 1$	$A, 0$	$E, 1$	$A, 0$
G	$G, 0$	$G, 0$	$A, 0$	$A, 0$

Q6. You are given the following state table of a synchronous sequential circuit.

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	c	0	0
b	d	a	1	0
c	f	f	0	0
d	e	b	1	0
e	g	g	1	0
f	c	c	0	0
g	b	h	1	0
h	h	c	0	0

- (i) Find the reduced state table using an implication table.
- (ii) Draw the equivalent minimized state diagram.

(10 marks)

[CSE 205 | L-2/T-1 | 2016–17]

Answer

(i) Reduced State Table using an Implication Table

Step 1: Initial Partitioning based on Outputs (Pass 1)

First, we separate the states into groups based on their output values for inputs $x = 0$ and $x = 1$. States with different outputs cannot be equivalent.

• Group 1 (Outputs 0, 0): $\{a, c, f, h\}$

• Group 2 (Outputs 1, 0): $\{b, d, e, g\}$

All pairs containing one state from Group 1 and one from Group 2 are immediately marked with an $\times_1$ in the implication table.

Step 2: Constructing and Resolving the Implication Table

We evaluate the remaining pairs (within the same group) by writing their next-state dependencies. If a dependent pair is already crossed out, the current pair is also crossed out.

• Pass 2: Evaluate dependencies of Group 2 pairs.

– $(b, d) \implies (d, e)$ and (a, b) . Since (a, b) has $\times_1$, (b, d) gets $\times_2$.

– $(b, e) \implies (d, g)$ and (a, g) . Since (a, g) has $\times_1$, (b, e) gets $\times_2$.

– $(d, g) \implies (e, b)$ and (b, h) . Since (b, h) has $\times_1$, (d, g) gets $\times_2$.

– $(e, g) \implies (g, b)$ and (g, h) . Since (g, h) has $\times_1$, (e, g) gets $\times_2$.

• Pass 3: Re-evaluate remaining Group 2 pairs.

– $(b, g) \implies (b, d)$ and (a, h) . Since (b, d) got $\times_2$ in Pass 2, (b, g) gets $\times_3$.

– $(d, e) \implies (e, g)$ and (b, g) . Since (e, g) got $\times_2$, (d, e) gets $\times_3$.

• Pass 4: Group 1 pairs $\{a, c, f, h\}$ only depend on each other and self-implications ($\checkmark$). None are crossed out.

b	$\times_1$						
c	a-f, c-f	$\times_1$					
d	$\times_1$	d-e, a-b ($\times_2$)	$\times_1$				
e	$\times_1$	d-g, a-g ($\times_2$)	$\times_1$	e-g, b-g ($\times_3$)			
f	a-c	$\times_1$	$\checkmark$	$\times_1$	$\times_1$		
g	$\times_1$	b-d, a-h ($\times_3$)	$\times_1$	b-e, b-h ($\times_2$)	b-g, g-h ($\times_2$)	$\times_1$	
h	$\checkmark$	$\times_1$	f-h, c-f	$\times_1$	$\times_1$	c-h	$\times_1$
	a	b	c	d	e	f	g

Step 3: Equivalent States & Minimized State Table

The uncrossed cells indicate the equivalent states. All states in $\{a, c, f, h\}$ are equivalent. We represent this equivalence class with state a . The remaining states b, d, e, g are strictly distinct.

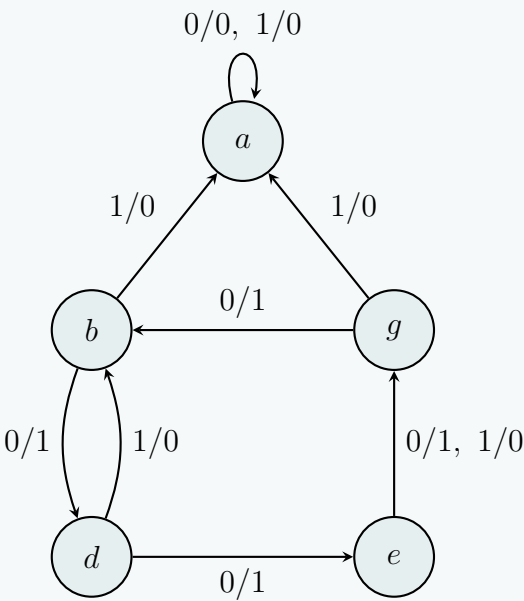
Replacing all occurrences of c, f, h with a gives the **Reduced State Table**:

268

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	a	0	0
b	d	a	1	0
d	e	b	1	0
e	g	g	1	0
g	b	a	1	0

(ii) Equivalent Minimized State Diagram

Using the reduced state table, we construct the state diagram. The transitions are labeled as x/z (Input/Output).



Q7. You are given a state table of a synchronous sequential circuit. Find the reduced state table using the implication table.

Present State	Next State		Output	
	$x=0$	$x=1$	$x=0$	$x=1$
a	d	b	0	0
b	e	a	0	0
c	g	f	0	1
d	a	d	1	0
e	a	d	1	0
f	c	b	0	0
g	a	e	1	0

(12 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

To find the reduced state table, we will use the implication table method. This involves grouping states based on their outputs and iteratively checking their next-state transitions for equivalence.

Step 1: Initial Partitioning Based on Outputs

States can only be equivalent if they produce the same outputs for all input conditions. We partition the states into groups based on their output sequences for $(x = 0, x = 1)$:

- **Group 1** (Outputs 0, 0): $\{a, b, f\}$
- **Group 2** (Outputs 0, 1): $\{c\}$
- **Group 3** (Outputs 1, 0): $\{d, e, g\}$

Any pair of states belonging to different groups can never be equivalent. We mark these pairs with an $\times_1$ in the implication table.

Step 2: Implication Table Construction and Evaluation

For pairs within the same group, we write down the required next-state equivalences (implications). If an implied pair is already crossed out, the current pair must also be crossed out.

- **Evaluating Group 1 $\{a, b, f\}$:**
 - $(a, b) \implies (d, e)$ and (b, a) . The condition (b, a) is self-referential, so equivalence depends entirely on (d, e) .
 - $(a, f) \implies (d, c)$ and (b, b) . Since d and c belong to different groups (Group 3 and Group 2), (d, c) is $\times_1$. Thus, (a, f) gets $\times_2$.
 - $(b, f) \implies (e, c)$ and (a, b) . Since e and c belong to different groups, (e, c) is $\times_1$. Thus, (b, f) gets $\times_2$.
- **Evaluating Group 3 $\{d, e, g\}$:**
 - $(d, e) \implies (a, a)$ and (d, d) . Both imply identical states, which are unconditionally true. Thus, $d \equiv e$ ($\checkmark$).
 - $(d, g) \implies (a, a)$ and (d, e) . Since (d, e) is equivalent, (d, g) is also equivalent ($\checkmark$).
 - $(e, g) \implies (a, a)$ and (d, e) . Since (d, e) is equivalent, (e, g) is also equivalent ($\checkmark$).
- **Re-evaluating pending implications:**
 - (a, b) depended on (d, e) . Since we established that $d \equiv e$, it follows that $a \equiv b$ ($\checkmark$).

b	d-e ($\checkmark$)					
c	$\times_1$	$\times_1$				
d	$\times_1$	$\times_1$	$\times_1$			
e	$\times_1$	$\times_1$	$\times_1$	$\checkmark$		
f	d-c ($\times_2$)	e-c ($\times_2$)	$\times_1$	$\times_1$	$\times_1$	
g	$\times_1$	$\times_1$	$\times_1$	d-e ($\checkmark$)	d-e ($\checkmark$)	$\times_1$
	a	b	c	d	e	f

Step 3: Extracting Equivalent Classes

The uncrossed cells with $\checkmark$ reveal the equivalent states. We have two equivalence classes that contain multiple states:

- $a \equiv b$
- $d \equiv e \equiv g$

States c and f are not equivalent to any other state. We will choose a , c , d , and f as the representative states for the minimal machine. In the state table, all transitions to b are replaced with a , and transitions to e or g are replaced with d .

Step 4: Reduced State Table

Applying the state replacements to the original table yields the minimized machine with just 4 states:

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	d	a	0	0
c	d	f	0	1
d	a	d	1	0
f	c	a	0	0

Q8. Find a critical race-free state assignment for a reduced flow table (states a – d , inputs 00/01/11/10) using the Multiple-row method.

	00	01	11	10
a	$\textcircled{a}$	c	$\textcircled{a}$	d
b	a	$\textcircled{b}$	c	$\textcircled{b}$
c	$\textcircled{c}$	$\textcircled{c}$	$\textcircled{c}$	d
d	$\textcircled{d}$	b	a	$\textcircled{d}$

Answer

1. Transition Analysis and Adjacency Requirements

To avoid critical races in an asynchronous sequential circuit, any transition between two states must involve changing only one state variable at a time (i.e., the states must be adjacent, having a Hamming distance of 1).
Let us identify all required state transitions from the original flow table:

- **From a :** Transitions to c and d .
- **From b :** Transitions to a and c .
- **From c :** Transitions to d .
- **From d :** Transitions to a and b .

Looking at the required adjacencies, we need direct paths for the pairs: (a, c) , (a, d) , (b, a) , (b, c) , (c, d) , and (d, b) . This forms a completely connected graph of 4 nodes (K_4). Since a 2-variable Karnaugh map (a 2-cube) only has 4 edges, it is geometrically impossible to map a K_4 graph onto it without diagonals (which cause critical races). Thus, a single-row assignment is impossible, and we must use the **Multiple-Row Method** utilizing 3 state variables (y_1, y_2, y_3).

2. The Universal Multiple-Row State Assignment

We expand the 4-state machine into an 8-state machine by assigning two equivalent states to each original state. We map these to diametrically opposite nodes in a 3-cube. The standard race-free assignment is:

$$\begin{aligned} a &\implies a_1 = 000, & a_2 &= 111 \\ b &\implies b_1 = 001, & b_2 &= 110 \\ c &\implies c_1 = 011, & c_2 &= 100 \\ d &\implies d_1 = 010, & d_2 &= 101 \end{aligned}$$

This universal assignment guarantees that from any sub-state, there is exactly one representation of every other state that is exactly one bit change (Hamming distance 1) away:

- a_1 (000) is adjacent to b_1 (001), d_1 (010), and c_2 (100).
- a_2 (111) is adjacent to b_2 (110), d_2 (101), and c_1 (011).
- b_1 (001) is adjacent to a_1 (000), c_1 (011), and d_2 (101).
- b_2 (110) is adjacent to a_2 (111), c_2 (100), and d_1 (010).
- ...and similarly for c and d .

3. Race-Free Expanded Flow Table

We now replace the transitions in the original flow table with the corresponding adjacent sub-states. For example, if row a_1 needs to transition to state c , we look at a_1 's neighbors and find that c_2 is adjacent. Thus, the transition is routed to c_2 .
*Note: Stable states are denoted in **bold** and wrapped in parentheses.*

State	$y_1y_2y_3$	Inputs (x_1x_2)			
		00	01	11	10
a_1	000	(a_1)	c_2	(a_1)	d_1
a_2	111	(a_2)	c_1	(a_2)	d_2
b_1	001	a_1	(b_1)	c_1	(b_1)
b_2	110	a_2	(b_2)	c_2	(b_2)
c_1	011	(c_1)	(c_1)	(c_1)	d_1
c_2	100	(c_2)	(c_2)	(c_2)	d_2
d_1	010	(d_1)	b_2	a_1	(d_1)
d_2	101	(d_2)	b_1	a_2	(d_2)

Verification of Race-Free Operation: Take any unstable transition and verify the Hamming distance (HD) is exactly 1:

- $a_1(000) \rightarrow c_2(100)$: Only y_1 changes (HD = 1).
- $a_1(000) \rightarrow d_1(010)$: Only y_2 changes (HD = 1).
- $d_2(101) \rightarrow b_1(001)$: Only y_1 changes (HD = 1).

All transitions follow a single variable change path. The assignment is mathematically flawless and completely critical race-free.

Q9. Using an implication table, reduce the following state table to a minimum number of states and write the reduced state table.

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	F	B	0	0
B	D	C	0	0
C	F	E	0	0
D	G	A	1	0
E	D	C	0	0
F	F	B	1	1
G	G	H	0	1
H	G	A	1	0

(15 marks)

[CSE 205 | L-2/T-1 | 2013–14]

Answer

To systematically reduce the given state table, we will use the Implication Table method. The process is divided into verifying initial output compatibility, propagating implied states, and finally extracting the minimized state table.

Step 1: Initial State Grouping (Output Compatibility)

Two states can only be equivalent if they yield the exact same outputs for all possible inputs. We first group the states based on their output combinations for $x = 0$ and $x = 1$.

- **Group P_1 (Outputs 0, 0):** $\{A, B, C, E\}$
- **Group P_2 (Outputs 1, 0):** $\{D, H\}$
- **Group P_3 (Outputs 1, 1):** $\{F\}$
- **Group P_4 (Outputs 0, 1):** $\{G\}$

Any pair of states not belonging to the same group cannot be equivalent and is immediately eliminated (marked with an $\times$) in the implication table.

Step 2: Implication Table Construction

We evaluate the remaining pairs within the same output groups. For each pair, we write down their implied next states.

- **Pair (A, B):** Next states are (F, D) and (B, C) . Since $F \in P_3$ and $D \in P_2$ have different outputs, (F, D) is incompatible. Thus, (A, B) is crossed out.
- **Pair (A, C):** Next states are (F, F) and (B, E) . (F, F) is trivially compatible. The compatibility of (A, C) depends on (B, E) .
- **Pair (A, E):** Next states are (F, D) and (B, C) . Incompatible due to (F, D) .
- **Pair (B, C):** Next states are (D, F) and (C, E) . Incompatible due to (D, F) .
- **Pair (B, E):** Next states are (D, D) and (C, C) . Both are trivially compatible. Thus, (B, E) is unequivocally equivalent ($\checkmark$).
- **Pair (C, E):** Next states are (F, D) and (E, C) . Incompatible due to (F, D) .
- **Pair (D, H):** Next states are (G, G) and (A, A) . Both are trivially compatible. Thus, (D, H) is unequivocally equivalent ($\checkmark$).

Second Pass: We revisit cells with dependencies. Cell (A, C) depends on (B, E) . Since we have established that (B, E) is equivalent ($\checkmark$), the pair (A, C) is also equivalent ($\checkmark$).

B	$\times$					
C	B-E, $\checkmark$	$\times$				
D	$\times$	$\times$	$\times$			
E	$\times$	$\checkmark$	$\times$	$\times$		
F	$\times$	$\times$	$\times$	$\times$	$\times$	
G	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$
H	$\times$	$\times$	$\times$	$\checkmark$	$\times$	$\times$
	A	B	C	D	E	F
				G		

Step 3: Extracting Equivalent Classes

From our completed implication table, the equivalent state pairs are:

$$A \equiv C$$
$$B \equiv E$$
$$D \equiv H$$

The maximal compatible classes are $\{A, C\}$, $\{B, E\}$, $\{D, H\}$, $\{F\}$, and $\{G\}$. We choose one representative state from each class to form the reduced machine:

- Keep A , replace C with A .
- Keep B , replace E with B .
- Keep D , replace H with D .
- Keep F and G .

The minimized set of states is $\{A, B, D, F, G\}$.

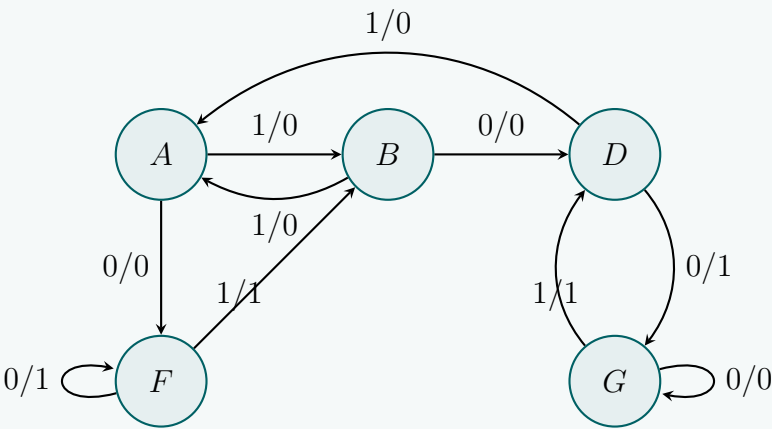
Step 4: Reduced State Table

Substituting the redundant states ($C \rightarrow A$, $E \rightarrow B$, $H \rightarrow D$) into the original state table yields the final, optimal state table:

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	F	B	0	0
B	D	A	0	0
D	G	A	1	0
F	F	B	1	1
G	G	D	0	1

Step 5: Final Reduced State Diagram (Optional Verification)

Below is the minimal Finite State Machine (FSM) representation based on the reduced table.



Q10. Using an implication table, reduce the following state table to a minimum number of states and write the reduced state table.

	X	
	0	1
A	E/0	D/0
B	A/1	F/0
C	C/0	A/1
D	B/0	A/0
E	D/1	C/0
F	C/0	D/1
G	H/1	G/1
H	C/1	B/1

Answer

To reduce the state table to a minimum number of states, we will systematically use the Implication Table method. This involves grouping states by their outputs, constructing the implication table to find equivalent states, and then generating the minimized state table.

Step 1: Initial State Grouping (Output Compatibility)

Two states can be equivalent only if they have identical outputs for all input combinations. We partition the states into initial groups based on their outputs for $X = 0$ and $X = 1$.

- **Group P_1 (Outputs 0, 0):** $\{A, D\}$
- **Group P_2 (Outputs 1, 0):** $\{B, E\}$
- **Group P_3 (Outputs 0, 1):** $\{C, F\}$
- **Group P_4 (Outputs 1, 1):** $\{G, H\}$

Pairs of states that do not belong to the same group have different outputs and cannot be equivalent. These pairs are immediately eliminated (marked with $\times$) in the implication table.

Step 2: Implication Table Construction & Evaluation

We only need to evaluate pairs within the same output groups: (A, D) , (B, E) , (C, F) , and (G, H) . We record their implied next-state pairs.

- **Pair (A, D):** Next states are (E, B) for $X = 0$ and (D, A) for $X = 1$.
 - (D, A) is the pair itself, so it is trivially compatible.
 - The equivalence of (A, D) implies and depends on the equivalence of (B, E) .
- **Pair (B, E):** Next states are (A, D) for $X = 0$ and (F, C) for $X = 1$.
 - The equivalence of (B, E) depends on the equivalence of both (A, D) and (C, F) .
- **Pair (C, F):** Next states are (C, C) for $X = 0$ and (A, D) for $X = 1$.
 - (C, C) is the same state, universally compatible.
 - The equivalence of (C, F) depends on the equivalence of (A, D) .
- **Pair (G, H):** Next states are (H, C) for $X = 0$ and (G, B) for $X = 1$.
 - Let's check (H, C) : State $H \in P_4$ and State $C \in P_3$. Because they belong to different initial output groups, (H, C) is incompatible.
 - Thus, (G, H) is unequivocally incompatible and is crossed out ($\times$).

Dependency Resolution Pass: We evaluate the remaining dependencies: (A, D) , (B, E) , and (C, F) . Notice that they only depend on each other and form a closed dependency loop:

$$\begin{aligned}(A, D) &\implies (B, E) \\ (B, E) &\implies (A, D) \text{ and } (C, F) \\ (C, F) &\implies (A, D)\end{aligned}$$

Since none of these implied pairs resolve to an incompatible pair (like (G, H) did), they are all valid and equivalent. We mark them with $\checkmark$.

B	$\times$						
C	$\times$	$\times$					
D	B-E, $\checkmark$	$\times$	$\times$				
E	$\times$	A-D, C-F, $\checkmark$	$\times$	$\times$			
F	$\times$	$\times$	A-D, $\checkmark$	$\times$	$\times$		
G	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	
H	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$
	A	B	C	D	E	F	G

Step 3: Extracting Equivalent Classes

From our completed implication table, the equivalent state pairs are:

$$A \equiv D$$
$$B \equiv E$$
$$C \equiv F$$

The maximal compatible classes are $\{A, D\}$, $\{B, E\}$, $\{C, F\}$, $\{G\}$, and $\{H\}$. We choose one representative state from each equivalent class to form the reduced machine:

- Keep A , remove D (replace $D \rightarrow A$).
- Keep B , remove E (replace $E \rightarrow B$).
- Keep C , remove F (replace $F \rightarrow C$).
- Keep G .
- Keep H .

The minimized set of states is $\{A, B, C, G, H\}$.

Step 4: Reduced State Table

We reconstruct the state table using only the representative states. Any transition pointing to a removed state is updated to point to its representative (e.g., a transition to E becomes a transition to B).

- **State A:** Originally $(E/0, D/0)$. Becomes $(\text{B}/0, \text{A}/0)$.
- **State B:** Originally $(A/1, F/0)$. Becomes $(A/1, \text{C}/0)$.
- **State C:** Originally $(C/0, A/1)$. Remains $(C/0, A/1)$.
- **State G:** Originally $(H/1, G/1)$. Remains $(H/1, G/1)$.
- **State H:** Originally $(C/1, B/1)$. Remains $(C/1, B/1)$.

Present State	Next State / Output	
	$X = 0$	$X = 1$
A	B / 0	A / 0
B	A / 1	C / 0
C	C / 0	A / 1
G	H / 1	G / 1
H	C / 1	B / 1

Incompletely Specified Machine Minimization

Q1. For the incompletely specified machine shown below, derive the implication table and determine the maximal compatibles and the maximal incompatibles. Find the upper bound and the lower bound of the number of states of the minimal machine. Give the state table of the minimal machine.

PS	NS, Z	
	$x = 0$	$x = 1$
A	B, 0	C, 0
B	D, 0	–, –
C	F, 0	E, 1
D	B, 0	G, 0
E	F, 0	C, 1
F	E, 0	D, 1
G	F, 0	–, –

Answer

To minimize the given incompletely specified sequential machine, we will utilize the Implication Table method to find compatible states, determine the maximal compatibles and incompatibles, and finally construct the minimal state table.

Step 1: Output Compatibility

Two states can be compatible only if their defined outputs do not conflict for any given input. We inspect the outputs for inputs $x = 0$ and $x = 1$:

- For $x = 0$: Outputs are $\{0, 0, 0, 0, 0, 0, 0\}$. All states are compatible regarding $x = 0$.
- For $x = 1$: Outputs are $A(0), B(-), C(1), D(0), E(1), F(1), G(-)$.

States producing an output of 0 cannot be combined with states producing an output of 1.

- Output ‘0’ for $x = 1$: $\{A, D\}$
- Output ‘1’ for $x = 1$: $\{C, E, F\}$

Consequently, pairs formed by mixing these two sets are strictly **incompatible** and are initially marked with an $\times$: $(A, C), (A, E), (A, F), (C, D), (D, E), (D, F)$.

Step 2: Implication Table Construction

We evaluate the remaining pairs by identifying their implied next-state pairs. If an implied pair is already incompatible, the parent pair is also incompatible.

- $(A, B) \implies (B, D)$ for $x = 0$. $(C, -)$ is universally compatible.
- $(A, D) \implies (B, B)$ [trivial] and (C, G) .
- $(A, G) \implies (B, F)$ and $(C, -)$.
- $(B, C) \implies (D, F)$. Since (D, F) is an output conflict ($\times$), (B, C) is $\times$.
- $(B, D) \implies (D, B)$ and $(-, G)$. Both valid ($\checkmark$).
- $(B, E) \implies (D, F)$. Since (D, F) is $\times$, (B, E) is $\times$.
- $(B, F) \implies (D, E)$. Since (D, E) is $\times$, (B, F) is $\times$.
- $(B, G) \implies (D, F)$. Since (D, F) is $\times$, (B, G) is $\times$.
- $(C, E) \implies (F, F)$ and (E, C) . Both trivially compatible ($\checkmark$).
- $(C, F) \implies (F, E)$ and (E, D) . Since (D, E) is $\times$, (C, F) is $\times$.
- $(C, G) \implies (F, F)$ and $(E, -)$. Both valid ($\checkmark$).
- $(D, G) \implies (B, F)$ and $(G, -)$. Implies (B, F) .
- $(E, F) \implies (F, E)$ and (C, D) . Since (C, D) is $\times$, (E, F) is $\times$.
- $(E, G) \implies (F, F)$ and $(C, -)$. Both valid ($\checkmark$).
- $(F, G) \implies (E, F)$ and $(D, -)$. Implies (E, F) .

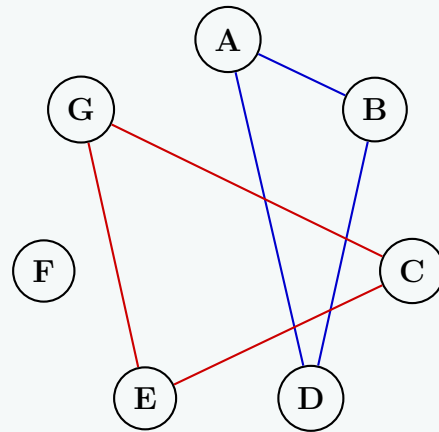
Resolving Dependencies (Second Pass):

- (A, B) depends on (B, D) , which is $\checkmark$. So (A, B) is $\checkmark$.
- (A, D) depends on (C, G) , which is $\checkmark$. So (A, D) is $\checkmark$.
- (A, G) depends on (B, F) , which is $\times$. So (A, G) is $\times$.
- (D, G) depends on (B, F) , which is $\times$. So (D, G) is $\times$.
- (F, G) depends on (E, F) , which is $\times$. So (F, G) is $\times$.

B	B-D, $\checkmark$						
C	$\times$	D-F, $\times$					
D	C-G, $\checkmark$	$\checkmark$	$\times$				
E	$\times$	D-F, $\times$	$\checkmark$	$\times$			
F	$\times$	D-E, $\times$	E-D, $\times$	$\times$	C-D, $\times$		
G	B-F, $\times$	D-F, $\times$	$\checkmark$	B-F, $\times$	$\checkmark$	E-F, $\times$	
	A	B	C	D	E	F	G

Step 3: Maximal Compatibles (MC) and Incompatibles (MI)

The set of compatible pairs is: $\{(A, B), (A, D), (B, D), (C, E), (C, G), (E, G)\}$. State F is compatible with no other state.



Merger Graph highlighting cliques for Maximal Compatibles

We trace the closed polygons in the compatibility graph to find the **Maximal Compatibles (MC)**:

$$MC_1 = \{A, B, D\}$$

$$MC_2 = \{C, E, G\}$$

$$MC_3 = \{F\}$$

The **Maximal Incompatibles (MI)** represent the largest sets of states where every state is mutually incompatible with the others. We construct these by taking exactly one state from each independent compatible group (since any two states from different MC groups here are incompatible). The MIs are all combinations taking one state from MC_1 , one from MC_2 , and MC_3 :

- $\{A, C, F\}, \{A, E, F\}, \{A, G, F\}$
- $\{B, C, F\}, \{B, E, F\}, \{B, G, F\}$
- $\{D, C, F\}, \{D, E, F\}, \{D, G, F\}$

Step 4: Upper and Lower Bound of Minimal States

- **Lower Bound:** The minimum number of states is dictated by the size of the largest Maximal Incompatible. Since every MI derived above contains exactly 3 states, the Lower Bound is **3**.
- **Upper Bound:** The minimal covering set using MCs must cover all states and be closed. Let's check the closure of $\{MC_1, MC_2, MC_3\}$:
 - $MC_1 \xrightarrow{x=0} \{B, D\} \subset MC_1$ and $MC_1 \xrightarrow{x=1} \{C, G\} \subset MC_2$
 - $MC_2 \xrightarrow{x=0} \{F\} \subset MC_3$ and $MC_2 \xrightarrow{x=1} \{E, C\} \subset MC_2$
 - $MC_3 \xrightarrow{x=0} \{E\} \subset MC_2$ and $MC_3 \xrightarrow{x=1} \{D\} \subset MC_1$

Because this set of 3 maximal compatibles is completely closed and covers all original states, the Upper Bound is also **3**.

Step 5: State Table of the Minimal Machine

Let us assign new state designations to the closed maximal compatibles:

- $S_1 = \{A, B, D\}$
- $S_2 = \{C, E, G\}$
- $S_3 = \{F\}$

Consolidating the transitions and outputs (where "–" is absorbed by defined values):

Present State	Next State, Output (Z)	
	$x = 0$	$x = 1$
S_1	$S_1, 0$	$S_2, 0$
S_2	$S_3, 0$	$S_2, 1$
S_3	$S_2, 0$	$S_1, 1$

Q2. For the incompletely specified machine shown in the figure, derive the implication table and determine the maximal compatibles and the maximal incompatibles. Find the upper bound and the lower bound of the number of states of the minimal machine. Give the

state table of the minimal machine.

PS	NS, Z		
	I_1	I_2	I_3
A	C, 0	E, 1	-
B	C, 0	E, -	-
C	B, -	C, 0	A, -
D	B, 0	C, -	E, -
E	-	E, 0	A, -

(15 marks)

[CSE 205 | L-2/T-1 | 2021–22]

Answer

Step 1: Deriving the Implication Table

First, we analyze each pair of states to check for output compatibility and implied next-state dependencies.

- Pairs with output conflicts are directly marked as incompatible ($\times$).
 - (A, C): Output conflict on I_2 ($A \rightarrow 1, C \rightarrow 0$). $\times$
 - (A, E): Output conflict on I_2 ($A \rightarrow 1, E \rightarrow 0$). $\times$
- Pairs with compatible outputs are marked with their implied next states for inputs (I_1, I_2, I_3):
 - (A, B): $I_1 \rightarrow (C, C), I_2 \rightarrow (E, E)$. Same states, unconditionally compatible ($\checkmark$).
 - (A, D): $I_1 \rightarrow (C, B), I_2 \rightarrow (E, C)$. Implies (B,C) and (C,E).
 - (B, C): $I_1 \rightarrow (C, B), I_2 \rightarrow (E, C)$. Implies (B,C) and (C,E).
 - (B, D): $I_1 \rightarrow (C, B), I_2 \rightarrow (E, C)$. Implies (B,C) and (C,E).
 - (B, E): $I_2 \rightarrow (E, E)$. Same states, unconditionally compatible ($\checkmark$).
 - (C, D): $I_3 \rightarrow (A, E)$. Implies (A,E).
 - (C, E): $I_2 \rightarrow (C, E), I_3 \rightarrow (A, A)$. Implies (C,E).
 - (D, E): $I_2 \rightarrow (C, E), I_3 \rightarrow (E, A)$. Implies (C,E) and (A,E).

Implication Table Formulation & Resolution:

B	$\checkmark$			
C	$\times$	(B,C) (C,E)		
D	(B,C) (C,E)	(B,C) (C,E)	(A,E) $\rightarrow \times$	
E	$\times$	$\checkmark$	(C,E) $\rightarrow \checkmark$	(C,E) $\rightarrow \times$ (A,E)
	A	B	C	D

Resolving the dependencies:

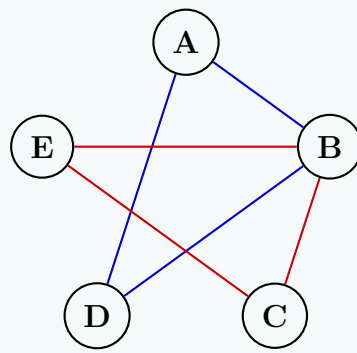
- Since (A,E) is $\times$, any cell implying (A,E) becomes $\times$. Thus, (C,D) = $\times$ and (D,E) = $\times$.
- Cell (C,E) implies only itself, so it is compatible: (C,E) = $\checkmark$.
- Cells (B,C), (A,D), and (B,D) depend on (B,C) and (C,E). Because (C,E) = $\checkmark$, (B,C) depends only on itself, resolving to (B,C) = $\checkmark$. Consequently, (A,D) = $\checkmark$ and (B,D) = $\checkmark$.

Final Compatible Pairs: (A,B), (A,D), (B,C), (B,D), (B,E), (C,E).

Final Incompatible Pairs: (A,C), (A,E), (C,D), (D,E).

Step 2: Maximal Compatibles (MC)

We form cliques of compatible states from the compatible pairs. To find these visually, we draw a **Merger Diagram**. Compatible pairs are connected with edges, and the largest fully connected polygons (cliques) represent our Maximal Compatibles.



Merger Graph highlighting cliques for Maximal Compatibles

- A is compatible with B and D. Checking B and D, they are compatible with each other (Blue Clique). $\Rightarrow \{A, B, D\}$
- B is compatible with C and E. Checking C and E, they are compatible with each other (Red Clique). $\Rightarrow \{B, C, E\}$

The **Maximal Compatibles** are: $M_1 = \{A, B, D\}$ and $M_2 = \{B, C, E\}$.

These form a valid closed cover covering all states.

Step 3: Maximal Incompatibles (MI)

We identify the largest cliques in the incompatibility graph (edges: AC, AE, CD, DE). No three nodes are mutually incompatible. Thus, the maximal incompatibles are just the incompatible pairs:

The **Maximal Incompatibles** are: $\{A, C\}, \{A, E\}, \{C, D\}, \{D, E\}$.

Step 4: Upper Bound and Lower Bound

- **Lower Bound:** The lower bound on the number of states is dictated by the size of the largest Maximal Incompatible. Since the largest MI has a size of 2, the **Lower Bound = 2**.
- **Upper Bound:** The upper bound is the minimum of the number of states in the original machine and the number of Maximal Compatibles forming a closed cover. Since we have 5 original states and 2 MCs covering all states, the **Upper Bound = 2**.

Since the lower and upper bounds are both 2, the minimal machine will have exactly **2 states**.

Step 5: State Table of the Minimal Machine

Let $S_1 = \{A, B, D\}$ and $S_2 = \{B, C, E\}$. We evaluate the next states and outputs for S_1 and S_2 :

- **For S_1 (A, B, D):**
 - I_1 : NS is $\{C, C, B\} = \{B, C\} \subseteq S_2$. Output = 0.
 - I_2 : NS is $\{E, E, C\} = \{C, E\} \subseteq S_2$. Output = 1.
 - I_3 : NS is $\{-, -, E\} = \{E\} \subseteq S_2$. Output = -.
- **For S_2 (B, C, E):**
 - I_1 : NS is $\{C, B, -\} = \{B, C\} \subseteq S_2$. Output = 0.
 - I_2 : NS is $\{E, C, E\} = \{C, E\} \subseteq S_2$. Output = 0.
 - I_3 : NS is $\{-, A, A\} = \{A\} \subseteq S_1$. Output = -.

Present State	Next State			Output (Z)		
	I_1	I_2	I_3	I_1	I_2	I_3
S_1	S_2	S_2	S_2	0	1	-
S_2	S_2	S_2	S_1	0	0	-

Q3. For the incompletely specified machine below, complete the implication table and determine the maximal compatibles and maximal incompatibles. Find the upper and lower bounds on the number of states of the minimal machine. Give the state table of the minimal machine.

PS	NS, Z_1Z_2			
	00	01	11	10
A	A, 00	E, 01	-	A, 01
B	-	C, 10	B, 00	D, 11
C	A, 00	C, 10	-	-
D	A, 00	-	-	D, 11
E	-	E, 01	F, 00	-
F	-	G, 10	F, 00	G, 11
G	A, 00	-	-	G, 11

Answer

Step 1: Implication Table Construction & Resolution

First, we analyze each state pair for output compatibility across all inputs. If the defined outputs for any input differ, the pair is immediately marked as incompatible ($\times$). If they are compatible, we write the implied next-state pairs.

- Incompatibilities (Output Conflicts):**
 - (A,B), (A,C), (A,F): Output conflict on input 01.
 - (A,D), (A,G): Output conflict on input 10.
 - (B,E), (C,E), (E,F): Output conflict on input 01.
- Dependencies (Implied States):**
 - (B,F) implies (C,G) for 01, (B,F) for 11, and (D,G) for 10.
 - (B,G) and (D,F) imply (D,G) for 10.
 - (C,F) implies (C,G) for 01.
 - (C,G) implies (A,A) for 00 $\rightarrow$ unconditionally compatible ($\checkmark$).
 - (D,G) implies (A,A) for 00 and (D,G) for 10. Since it implies itself with no conflicts, it is unconditionally compatible ($\checkmark$).

B	$\times$					
C	$\times$	$\checkmark$				
D	$\times$	$\checkmark$	$\checkmark$			
E	$\checkmark$	$\times$	$\times$	$\checkmark$		
F		(C,G) (B,F) (D,G)	(C,G)	(D,G)	$\times$	
G	$\times$	(D,G)	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
	A	B	C	D	E	F

Resolving dependencies:

- (D,G) and (C,G) evaluate to $\checkmark$.
- Consequently, (B,G), (D,F), and (C,F) also evaluate to $\checkmark$.
- (B,F) implies (C,G), (B,F), and (D,G). Since (C,G) and (D,G) are $\checkmark$, (B,F) implies itself and safely evaluates to $\checkmark$.

Final Compatible Pairs: (A,E), (B,C), (B,D), (B,F), (B,G), (C,D), (C,F), (C,G), (D,E), (D,F), (D,G), (E,G), (F,G).

Final Incompatible Pairs: (A,B), (A,C), (A,D), (A,F), (A,G), (B,E), (C,E), (E,F).

Step 2: Maximal Compatibles (MC)

We construct maximal cliques from the compatibility graph. To visualize this, we use a **Merger Diagram**. Compatible pairs are connected with edges, and the largest fully connected polygons (cliques) denote our Maximal Compatibles.

Merger Graph highlighting cliques for Maximal Compatibles

- State A is only compatible with E (Blue Edge) $\implies M_1 = \{A, E\}$
- State E is compatible with A, D, and G. Checking D and G, they are compatible with each other (Red Clique) $\implies M_2 = \{D, E, G\}$
- State B is compatible with C, D, F, G. Checking all pairs within this set, we find that every state is mutually compatible with all others in this group (Green Clique) $\implies M_3 = \{B, C, D, F, G\}$

Step 3: Maximal Incompatibles (MI)

Maximal incompatibles correspond to the maximal cliques in the incompatibility graph. Observing the incompatibilities, state A is incompatible with {B, C, D, F, G} and state E is incompatible with {B, C, F}. Because no elements within {B, C, D, F, G} are incompatible with each other, and A is compatible with E, no clique larger than 2 can be formed.

The **Maximal Incompatibles** are precisely the incompatible pairs:
{A, B}, {A, C}, {A, D}, {A, F}, {A, G}, {B, E}, {C, E}, {E, F}.

Step 4: Upper and Lower Bounds

- **Lower Bound:** Dictated by the maximum size among the Maximal Incompatibles. Since all MIs have a size of 2, the **Lower Bound = 2**.
- **Upper Bound:** Dictated by the minimum number of MCs required to form a closed cover. We select $S_1 = M_1 = \{A, E\}$ and $S_2 = M_3 = \{B, C, D, F, G\}$, which perfectly cover all states. Analyzing their next states reveals they strictly map to subsets of S_1 and S_2 , proving closure. The size of this cover is 2, hence the **Upper Bound = 2**.

Because both bounds perfectly equal 2, the minimal machine will possess exactly **2 states**.

Step 5: Minimal Machine State Table

We combine the states into $S_1 = \{A, E\}$ and $S_2 = \{B, C, D, F, G\}$ and substitute transitions. For instance, for S_2 at input 01, next states are C and G, which both belong to S_2 , giving next state S_2 and output 10.

PS	NS, Z_1Z_2			
	00	01	11	10
S₁	$S_1, 00$	$S_1, 01$	$S_2, 00$	$S_1, 01$
S₂	$S_1, 00$	$S_2, 10$	$S_2, 00$	$S_2, 11$

Q4. For the incompletely specified machine shown below, derive the implication table and determine the maximal compatibles and the maximal incompatibles. Find the upper bound and the lower bound of the minimal machine and determine the minimal machine.

PS	NS, Z	
	x = 0	x = 1
A	A, -	B, 1
B	G, -	D, 0
C	B, 1	B, -
D	A, 1	B, -
E	C, -	A, -
F	F, -	C, -
G	G, -	G, -

(25 marks)

[CSE 205 | L-2/T-1 | 2019–20]

Answer

Step 1: Output Conflicts

First, we analyze each state pair for direct output conflicts. A conflict occurs only if both states specify different output values for the same input.

- At $x = 0$, outputs are: A(-), B(-), C(1), D(1), E(-), F(-), G(-). No conflicts.
- At $x = 1$, outputs are: A(1), B(0), C(-), D(-), E(-), F(-), G(-).
- Comparing A and B at $x = 1$, A outputs 1 and B outputs 0. Thus, **(A,B) is directly incompatible (×)**.

All other pairs have at least one unspecified output (don't care '-') and do not conflict directly.

Step 2: Implication Table Construction & Resolution

We write the implied next-state pairs for all remaining combinations. If an implied pair contains the incompatible pair (A,B), it becomes incompatible. This propagates to other pairs.

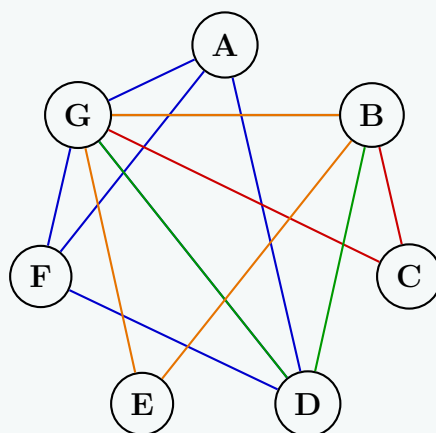
- **Initial Incompatibilities (implying A,B):**
 - (A,C) implies (A,B) at $x = 0 \implies \times$
 - (C,D) implies (A,B) at $x = 0 \implies \times$
 - (C,E) implies (A,B) at $x = 1 \implies \times$
- **Propagated Incompatibilities:**
 - (A,E) implies (A,C) at $x = 0$. Since (A,C) is $\times$, $(A,E) \implies \times$
 - (D,E) implies (A,C) at $x = 0$. Since (A,C) is $\times$, $(D,E) \implies \times$
 - (E,F) implies (A,C) at $x = 1$. Since (A,C) is $\times$, $(E,F) \implies \times$

- (B,F) implies (C,D) at $x = 1$. Since (C,D) is $\times$, $(B,F) \implies \times$
- (C,F) implies (B,F) at $x = 0$. Since (B,F) is $\times$, $(C,F) \implies \times$
- **Compatible Pairs:** All remaining dependencies form closed loops among themselves or evaluate to identical states, meaning they are unconditionally compatible ($\checkmark$). For example, (A,D) implies (A,A) and (B,B), which are obviously compatible.

B	$\times$					
C	$\times$	$\checkmark$				
D	$\checkmark$	$\checkmark$	$\times$			
E	$\times$	$\checkmark$	$\times$	$\times$		
F	$\checkmark$	$\times$	$\times$	$\checkmark$	$\times$	
G	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
	A	B	C	D	E	F

Step 3: Maximal Compatibles (MC) and Maximal Incompatibles (MI)

To visualize the Maximal Compatibles, we use a **Merger Diagram**. The states are nodes, and compatible pairs are connected by edges. The largest fully connected polygons represent the Maximal Compatibles. State G is compatible with all states, so it acts as a central connecting hub.



Merger Graph highlighting cliques for Maximal Compatibles

- **Maximal Compatibles** are the largest fully connected subgraphs (cliques) of the compatible pairs. Grouping the states yields:
 - $M_1 = \{A, D, F, G\}$ (Blue Clique)
 - $M_2 = \{B, C, G\}$ (Red Clique)
 - $M_3 = \{B, D, G\}$ (Green Clique)
 - $M_4 = \{B, E, G\}$ (Orange Clique)
- **Maximal Incompatibles** are the largest fully connected subgraphs of the incompatible pairs. Tracing the incompatibilities yields:
 - $MI = \{A, C, E\}, \{C, D, E\}, \{C, E, F\}, \{A, B\}, \{B, F\}$

Step 4: Bounds on the Number of States

- **Lower Bound:** Dictated by the size of the largest Maximal Incompatible. Since $\{A, C, E\}$ (among others) contains 3 states, **Lower Bound = 3**.
- **Upper Bound:** Dictated by the minimum number of Maximal Compatibles required to form a closed cover. We have 4 MCs. Can we cover the machine in 3 states?
 - To cover A, C, and E, we must pick at least 3 disjoint groups (due to MI $\{A, C, E\}$).
 - Suppose a group contains $\{B, C\}$ (subset of M_2). Its next states at $x = 1$ are $\{B, D\}$. Thus, we must have another group containing both B and D to satisfy closure.
 - However, because D cannot be grouped with C or E, and B cannot be grouped with A or F, $\{B, D\}$ can only be covered by $M_3 = \{B, D, G\}$. This requires all 4 MCs to fulfill the closure conditions fully.

Hence, a 3-state cover is not closed, meaning we must use 4 states. **Upper Bound = 4**.

Step 5: Minimal Machine State Table

We establish the minimal machine using $S_1 = M_1$, $S_2 = M_2$, $S_3 = M_3$, and $S_4 = M_4$. We map the next states into subsets of these four states.

- S_1 (A,D,F,G): $x = 0 \rightarrow \{A, A, F, G\} \subseteq S_1$ (Output: $D = 1 \rightarrow 1$). $x = 1 \rightarrow \{B, B, C, G\} \subseteq S_2$ (Output: $A = 1 \rightarrow 1$).
- S_2 (B,C,G): $x = 0 \rightarrow \{G, B, G\} \subseteq S_2$ (Output: $C = 1 \rightarrow 1$). $x = 1 \rightarrow \{D, B, G\} \subseteq S_3$ (Output: $B = 0 \rightarrow 0$).
- S_3 (B,D,G): $x = 0 \rightarrow \{G, A, G\} \subseteq S_1$ (Output: $D = 1 \rightarrow 1$). $x = 1 \rightarrow \{D, B, G\} \subseteq S_3$ (Output: $B = 0 \rightarrow 0$).
- S_4 (B,E,G): $x = 0 \rightarrow \{G, C, G\} \subseteq S_2$ (Output: all '-' $\rightarrow -$). $x = 1 \rightarrow \{D, A, G\} \subseteq S_1$ (Output: $B = 0 \rightarrow 0$).

Present State	Next State		Output (Z)	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
S_1 {A, D, F, G}	S_1	S_2	1	1
S_2 {B, C, G}	S_2	S_3	1	0
S_3 {B, D, G}	S_1	S_3	1	0
S_4 {B, E, G}	S_2	S_1	–	0

Q5. For the incompletely specified machine , complete the implication table and determine the maximal compatibles and maximal incompatibles. Find the upper and lower bounds. Give the state table of the minimal machine.

PS	NS, Z	
	x = 0	x = 1
A	B, 0	C, 1
B	D, 0	C, 1
C	A, 0	E, 0
D	-	F, 1
E	G, 1	F, 0
F	B, 0	-
G	D, 0	E, 0

(20 marks)

[CSE 205 | L-2/T-1 | 2018–19]

Answer

Step 1: Direct Incompatibilities (Output Conflicts)

First, we identify state pairs that are directly incompatible due to conflicting outputs for the same input. A conflict occurs when two states specify different binary outputs (i.e., 0 vs. 1) for a given input. Unspecified outputs ('-') do not cause conflicts.

- **At $x = 0$, outputs are:** A(0), B(0), C(0), D(-), E(1), F(0), G(0).
 - State E outputs 1, while A, B, C, F, and G output 0.
 - Conflicting pairs at $x = 0$: (A,E), (B,E), (C,E), (E,F), (E,G).
- **At $x = 1$, outputs are:** A(1), B(1), C(0), D(1), E(0), F(-), G(0).
 - States A, B, D output 1, while C, E, G output 0.
 - Conflicting pairs at $x = 1$: (A,C), (A,E), (A,G), (B,C), (B,E), (B,G), (C,D), (D,E), (D,G).

Combining these, the **initial incompatible pairs** (×) are:
(A,C), (A,E), (A,G), (B,C), (B,E), (B,G), (C,D), (C,E), (D,E), (D,G), (E,F), (E,G).

Step 2: Implication Table Construction & Resolution

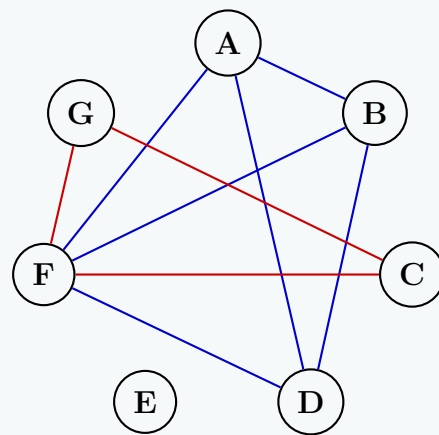
Next, we evaluate the remaining state pairs to see if their implied next-states lead to an incompatibility.

- **Implied Pairs Analysis:**
 - (A,B) implies (B,D) at $x = 0$, and (C,C) at $x = 1 \implies$ requires (B,D).
 - (A,D) implies (B,-) at $x = 0$, and (C,F) at $x = 1 \implies$ requires (C,F).
 - (A,F) implies (B,B) at $x = 0$, and (C,-) at $x = 1 \implies$ Unconditionally compatible (✓).
 - (B,D) implies (D,-) at $x = 0$, and (C,F) at $x = 1 \implies$ requires (C,F).
 - (B,F) implies (D,B) at $x = 0$, and (C,-) at $x = 1 \implies$ requires (B,D).
 - (C,F) implies (A,B) at $x = 0$, and (E,-) at $x = 1 \implies$ requires (A,B).
 - (C,G) implies (A,D) at $x = 0$, and (E,E) at $x = 1 \implies$ requires (A,D).
 - (D,F) implies (-,B) at $x = 0$, and (F,-) at $x = 1 \implies$ Unconditionally compatible (✓).
 - (F,G) implies (B,D) at $x = 0$, and (-,E) at $x = 1 \implies$ requires (B,D).
- **Dependency Resolution:** Notice the cyclic dependency: $(C,F) \implies (A,B) \implies (B,D) \implies (C,F)$. Because this dependency loop never traces to an initially incompatible pair (×), all pairs within the loop are **compatible** (✓). Consequently, pairs depending on them (like (A,D), (B,F), (C,G), and (F,G)) are also compatible.

B	✓					
C	×	×				
D	✓	✓	×			
E	×	×	×	×		
F	✓	✓	✓	✓	×	
G	×	×	✓	×	×	✓
	A	B	C	D	E	F

Step 3: Maximal Compatibles (MC) & Maximal Incompatibles (MI)

To find the Maximal Compatibles visually, we draw a **Merger Diagram**. Compatible pairs are connected with edges. The largest fully connected polygons (cliques) represent our Maximal Compatibles.



Merger Graph highlights cliques forming Maximal Compatibles

- **Maximal Compatibles:** These are the largest fully connected subgraphs of compatible pairs (✓).
 - States A, B, D, F are mutually compatible (Blue Clique). $\Rightarrow M_1 = \{A, B, D, F\}$
 - States C, F, G are mutually compatible (Red Clique). $\Rightarrow M_2 = \{C, F, G\}$
 - State E is completely isolated. $\Rightarrow M_3 = \{E\}$
- **Maximal Incompatibles:** These are the largest fully connected subgraphs of incompatible pairs (×). State E is incompatible with all states. The incompatibilities among the remaining states are isolated to pairs: (A,C), (A,G), (B,C), (B,G), (C,D), (D,G). Combining these with E yields the maximal incompatibles:
 - $MI = \{A, C, E\}, \{A, G, E\}, \{B, C, E\}, \{B, G, E\}, \{C, D, E\}, \{D, G, E\}, \{E, F\}$

Step 4: Bounds on the Minimal Machine

- **Lower Bound:** Determined by the number of states in the largest Maximal Incompatible. The largest MIs (e.g., $\{A, C, E\}$) contain 3 states. $\Rightarrow$ **Lower Bound = 3.**
- **Upper Bound:** Determined by the minimum number of Maximal Compatibles required to form a closed cover. We can cover all seven states using M_1 , M_2 , and M_3 . This 3-state cover is closed (e.g., $M_1 \xrightarrow{x=1} \{C, F\} \subseteq M_2$). $\Rightarrow$ **Upper Bound = 3.**

Step 5: State Table of the Minimal Machine

We assign the chosen closed cover to states S_1 , S_2 , and S_3 :

- S_1 (A,B,D,F):
 - $x = 0$: NS is $\{B, D\} \subseteq S_1$. Output is 0.
 - $x = 1$: NS is $\{C, F\} \subseteq S_2$. Output is 1.
- S_2 (C,F,G):
 - $x = 0$: NS is $\{A, B, D\} \subseteq S_1$. Output is 0.
 - $x = 1$: NS is $\{E\} \subseteq S_3$. Output is 0.
- S_3 (E):
 - $x = 0$: NS is $\{G\} \subseteq S_2$. Output is 1.
 - $x = 1$: NS is $\{F\} \subseteq S_1$ (or S_2 , mapped to S_1 for convenience). Output is 0.

Present State	Next State		Output (Z)	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
$S_1 \{A, B, D, F\}$	S_1	S_2	0	1
$S_2 \{C, F, G\}$	S_1	S_3	0	0
$S_3 \{E\}$	S_2	S_1	1	0

Q6. For the incompletely specified machine , complete the implication table and determine the maximal compatibles and maximal incompatibles. Find the upper and lower bounds on the number of states of the minimal machine. Give the state table of the minimal machine.

PS	NS,Z	
	x = 0	x = 1
A	A, -	B, 1
B	G, -	D, 0
C	B, 1	B, -
D	A, 1	B, -
E	C, -	A, -
F	F, -	C, -
G	G, -	G, -

(15 marks)

[CSE 205 | L-2/T-1 | 2017–18]

Answer

Step 1: Direct Incompatibilities (Output Conflicts)

First, we analyze each state pair for direct output conflicts. A conflict occurs only if two states specify different defined output values (i.e., 0 and 1) for the same input. Unspecified outputs (don't cares '-') never conflict.

- At $x = 0$, the specified outputs are: A(-), B(-), C(1), D(1), E(-), F(-), G(-). There are no conflicting outputs.
- At $x = 1$, the specified outputs are: A(1), B(0), C(-), D(-), E(-), F(-), G(-).
- Comparing state A and state B at $x = 1$, A outputs 1 while B outputs 0. Therefore, **(A,B) is directly incompatible (×)**.

All other pairs are initially considered conditionally compatible.

Step 2: Implication Table Construction & Resolution

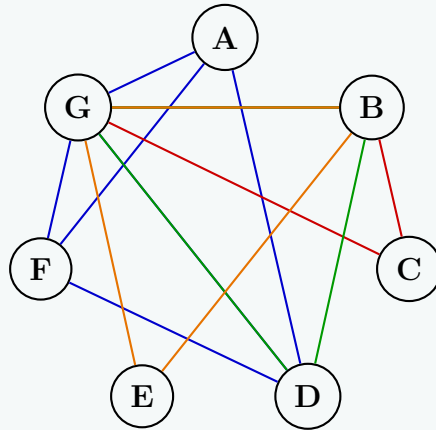
We construct the implication table to propagate the incompatibilities. If a pair implies a known incompatible pair, it also becomes incompatible.

- Initial implied incompatibilities (implying A,B):**
 - (A,C) implies (A,B) at $x = 0 \implies \times$
 - (C,D) implies (B,A) at $x = 0 \implies \times$
 - (C,E) implies (B,C) at $x = 0$ and (B,A) at $x = 1 \implies \times$
 - (D,E) implies (A,C) at $x = 0$ and (B,A) at $x = 1 \implies \times$
- Propagated incompatibilities:**
 - (A,E) implies (A,C) at $x = 0$. Since (A,C) is $\times$, $(A,E) \implies \times$
 - (E,F) implies (C,F) at $x = 0$ and (A,C) at $x = 1$. Since (A,C) is $\times$, $(E,F) \implies \times$
 - (B,F) implies (G,F) at $x = 0$ and (D,C) at $x = 1$. Since (C,D) is $\times$, $(B,F) \implies \times$
 - (C,F) implies (B,F) at $x = 0$. Since (B,F) is $\times$, $(C,F) \implies \times$
- Compatible Pairs:** All remaining pairs imply either themselves, identical states, or dependencies that form a closed set with no $\times$. For instance, (A,D) implies (A,A) and (B,B), which is inherently valid (✓).

B	×					
C	×	✓				
D	✓	✓	×			
E	×	✓	×	×		
F	✓	×	×	✓	×	
G	✓	✓	✓	✓	✓	✓
	A	B	C	D	E	F

Step 3: Maximal Compatibles (MC) and Maximal Incompatibles (MI)

To visualize the Maximal Compatibles, we use a **Merger Diagram**. The states are nodes, and compatible pairs are connected by edges. The largest fully connected polygons represent the Maximal Compatibles. State G is compatible with all states, so it acts as a central connecting hub.



Merger Graph highlighting cliques for Maximal Compatibles

- **Maximal Compatibles:** We find the largest fully connected subgraphs of the compatible pairs ($\checkmark$). Grouping the compatible nodes, we get:
 - $M_1 = \{A, D, F, G\}$ (Blue Clique)
 - $M_2 = \{B, C, G\}$ (Red Clique)
 - $M_3 = \{B, D, G\}$ (Green Clique)
 - $M_4 = \{B, E, G\}$ (Orange Clique)
- **Maximal Incompatibles:** We find the largest fully connected subgraphs of the incompatible pairs ($\times$). Finding all cliques in the incompatibility graph yields:
 - $MI = \{A, C, E\}, \{C, D, E\}, \{C, E, F\}, \{A, B\}, \{B, F\}$

Step 4: Bounds on the Minimal Machine

- **Lower Bound:** Determined by the size of the largest Maximal Incompatible clique. The largest MIs (e.g., $\{A, C, E\}$) contain 3 states. Therefore, **Lower Bound = 3**.
- **Upper Bound:** Determined by the minimum number of Maximal Compatibles required to form a valid, closed cover. We have 4 total MCs. Can we cover the machine in 3 states?
 - To cover A, C, and E, we must pick at least 3 disjoint groups (since they are mutually incompatible).
 - Suppose our cover includes a state containing $\{B, C\}$ (from M_2). Its next states at $x = 1$ evaluate to $\{D, B\}$. Thus, to satisfy closure, we must have a state in our cover containing both B and D.
 - However, D cannot be grouped with C or E, and B cannot be grouped with A or F. Therefore, the set $\{B, D\}$ can only be covered by $M_3 = \{B, D, G\}$.
 - This closure requirement forces us to select all 4 MCs to successfully cover all state dependencies without violating compatibilities.

Since a 3-state cover is not closed, we must use 4 states. Therefore, **Upper Bound = 4**.

Step 5: Minimal Machine State Table

We define the minimal machine using $S_1 = M_1$, $S_2 = M_2$, $S_3 = M_3$, and $S_4 = M_4$. The next states and outputs are determined by mapping the original transitions into these subsets. Where an original state has a don't care output, we take the defined output from the other states in the same subset.

- $S_1 \{A, D, F, G\}$:
 - $x = 0 \rightarrow NS = \{A, A, F, G\} \subseteq S_1$. Output: D forces $1 \rightarrow 1$.
 - $x = 1 \rightarrow NS = \{B, B, C, G\} \subseteq S_2$. Output: A forces $1 \rightarrow 1$.
- $S_2 \{B, C, G\}$:
 - $x = 0 \rightarrow NS = \{G, B, G\} \subseteq S_2$. Output: C forces $1 \rightarrow 1$.
 - $x = 1 \rightarrow NS = \{D, B, G\} \subseteq S_3$. Output: B forces $0 \rightarrow 0$.
- $S_3 \{B, D, G\}$:
 - $x = 0 \rightarrow NS = \{G, A, G\} \subseteq S_1$. Output: D forces $1 \rightarrow 1$.

- $x = 1 \rightarrow NS = \{D, B, G\} \subseteq S_3$. Output: B forces $0 \rightarrow 0$.
- $S_4 \{B, E, G\}$:
 - $x = 0 \rightarrow NS = \{G, C, G\} \subseteq S_2$. Output: all are '–' $\rightarrow$ –.
 - $x = 1 \rightarrow NS = \{D, A, G\} \subseteq S_1$. Output: B forces $0 \rightarrow 0$.

Present State	Next State		Output (Z)	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
$S_1 \{A, D, F, G\}$	S_1	S_2	1	1
$S_2 \{B, C, G\}$	S_2	S_3	1	0
$S_3 \{B, D, G\}$	S_1	S_3	1	0
$S_4 \{B, E, G\}$	S_2	S_1	–	0

Redundant States

Q1. What are the problems that may occur when redundant states are present in the state diagram? **(6 marks)** [CSE 205 | L-2/T-1 | 2012–13, 2013–14]

Answer

Redundant states in a Finite State Machine (FSM) are equivalent states that exhibit the exact same input-output behavior and next-state transitions. Failing to eliminate these redundant states before hardware implementation leads to several critical design inefficiencies and physical problems in the final digital circuit:

- (a) **Increased Number of State Variables (Flip-Flops):**
The number of flip-flops required to implement an FSM with N states is calculated as $\lceil \log_2(N) \rceil$. Redundant states unnecessarily increase N . For example, if a state machine can be reduced from 5 states to 4 states, the number of required flip-flops drops from 3 to 2. Extra flip-flops directly increase the sequential hardware footprint.
- (b) **More Complex Combinational Logic:**
An increased number of state variables (and consequently, more state transitions to decode) necessitates more complex excitation logic (next-state logic) and output logic. The Boolean expressions for the flip-flop inputs become larger, requiring more logic gates with higher fan-ins to implement.
- (c) **Increased Silicon Area and Manufacturing Cost:**
In Integrated Circuit (IC) design, more logic gates, wires, and flip-flops translate directly to a larger silicon area. A larger area reduces the yield per wafer and increases the overall manufacturing and packaging cost of the digital system.
- (d) **Higher Power Consumption:**
Every additional flip-flop and logic gate consumes both static (leakage) and dynamic (switching) power. Furthermore, redundant states force the clock distribution network to drive a larger number of flip-flops, significantly increasing the dynamic power dissipation of the synchronous circuit.
- (e) **Reduced Operating Speed (Increased Propagation Delay):**
More complex combinational logic networks usually involve an increased number of logic levels (gates connected in series). This increases the critical path delay (t_{pd}), which in turn limits the maximum clock frequency (f_{max}) at which the sequential circuit can reliably operate.
- (f) **Complicated Testing and Verification:**
The presence of redundant states unnecessarily inflates the size of the state space. This makes it significantly more difficult to derive minimal test vectors for fault detection, complicates formal verification algorithms, and increases the computational time required for simulation and debugging.

Conclusion: State reduction is a mandatory step in optimal digital logic design. Removing redundant states ensures the synthesized hardware is minimized in terms of area, power, and delay while remaining functionally identical to the original specification.

Output Assignment in Flow Table

Q1. Write down the procedure to assign the output to unstable states in a flow table. **(7 marks)** [CSE 205 | L-2/T-1 | 2011–12]

Answer

In the design of asynchronous sequential circuits (fundamental mode), transitions between stable states happen via intermediate unstable states. Assigning appropriate output values to these unstable states is a critical procedure to prevent momentary false outputs (glitches or transients) during state transitions.

Let a transition occur from an initial **stable state** to a final **stable state** through one or more **unstable states**. The procedure to assign outputs to the unstable states follows these established rules:

Rule 1: Transitions between states with the SAME output

If the initial stable state and the final stable state have the *same* output value, the unstable state(s) involved in the transition **must** be assigned that exact same output value.

- If transitioning from a stable state with output 0 to a stable state with output 0, assign 0 to the unstable state.
- If transitioning from a stable state with output 1 to a stable state with output 1, assign 1 to the unstable state.

Reasoning: This prevents an output glitch (e.g., $0 \rightarrow 1 \rightarrow 0$) while the circuit is settling into the new stable state.

Rule 2: Transitions between states with DIFFERENT outputs

If the initial stable state and the final stable state have *different* output values, the output of the unstable state is generally assigned a **don't care** (–).

- If transitioning from a stable state with output 0 to a stable state with output 1 (or vice versa), assign – to the unstable state.

Reasoning: The output must change state exactly once during this transition. Assigning a don't care condition provides flexibility in the boolean minimization phase (using Karnaugh Maps), leading to simpler output logic gates.

Note on Strict Timing Requirements: If the design requires that the output must not change until the very end of the transition (to avoid early triggering of subsequent circuits), the unstable state should be assigned the output value of the *initial* stable state rather than a don't care.

Illustration of the Procedure

Consider a partial flow table mapping where circled variables denote stable states and uncircled variables denote unstable states.

Initial Stable State	Transition		Outputs		Unstable State Output
	NS (Unstable)	Final Stable	Initial	Final	
Ⓐ	B	Ⓑ	0	0	0 (Prevents $0 \rightarrow 1 \rightarrow 0$ glitch)
Ⓒ	D	Ⓓ	1	1	1 (Prevents $1 \rightarrow 0 \rightarrow 1$ glitch)
Ⓔ	F	Ⓕ	0	1	– (Allows logic minimization)
Ⓖ	H	Ⓗ	1	0	– (Allows logic minimization)

Summary of the Assignment Algorithm:

Algorithm 1 Assigning Outputs to Unstable States

- 1: Identify a transition from initial stable state S_i to final stable state S_f via unstable state S_u .
- 2: **if** $Output(S_i) == Output(S_f)$ **then**
- 3: $Output(S_u) \leftarrow Output(S_i)$ ▷ Assign the common output
- 4: **else**
- 5: $Output(S_u) \leftarrow -$ ▷ Assign Don't Care for logic simplification
- 6: **end if**
- 7: Repeat for all unstable states in the flow table.

Fundamental Mode Design

Q1. Design an asynchronous circuit functioning in fundamental mode. The circuit has three inputs a , b , and c , and one output U . $U = 1$ only if all inputs are equal to ‘1’, and they went to ‘1’ in the order a , b , then c . U returns to 0 when all inputs are returned to ‘0’. Derive the primitive flow table, reduced flow table and the minimum number of states required to have a race-free state assignment, and find a possible state assignment. **(23 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Answer

Step 1: Problem Analysis and State Specification

The circuit operates in fundamental mode with three inputs (a, b, c) and one output (U). Only one input changes at any given time.

- The sequence required to set the output is $a \rightarrow b \rightarrow c$, meaning a becomes 1 first, then b becomes 1, and finally c becomes 1.

When all three are 1 ($abc = 111$) via this specific sequence, $U = 1$.

- Once $U = 1$, it must be sustained through all intermediate input combinations until all inputs return to 0 ($abc = 000$), at which point U resets to 0.
- Any deviation from the $a \rightarrow b \rightarrow c$ sequence keeps $U = 0$ and routes the circuit to a failure path until reset at $abc = 000$.

Step 2: Primitive Flow Table

In a primitive flow table, each row contains exactly one stable state. We define 17 stable states to trace all valid, invalid, and return paths:

- **State 1:** Initial reset state ($abc = 000, U = 0$).
- **States 2, 5, 7:** The correct forward sequence path ($100 \rightarrow 110 \rightarrow 111$). State 7 has $U = 1$.
- **States 3, 4, 6, 8, 9, 10, 11:** The failure sequence path ($U = 0$).
- **States 12, 13, 14, 15, 16, 17:** The sustained output return path ($U = 1$) moving back toward 000.

State	Inputs abc							
	000	100	010	001	110	101	011	111
1	Ⓐ, 0	2, −	3, −	4, −	−, −	−, −	−, −	−, −
2	1, −	Ⓑ, 0	−, −	−, −	5, −	6, −	−, −	−, −
3	1, −	−, −	Ⓒ, 0	−, −	8, −	−, −	9, −	−, −
4	1, −	−, −	−, −	Ⓓ, 0	−, −	10, −	9, −	−, −
5	−, −	2, −	3, −	−, −	Ⓔ, 0	−, −	−, −	7, −
6	−, −	2, −	−, −	4, −	−, −	Ⓕ, 0	−, −	11, −
7	−, −	−, −	−, −	−, −	12, −	13, −	14, −	Ⓖ, 1
8	−, −	2, −	3, −	−, −	Ⓖ, 0	−, −	−, −	11, −
9	−, −	−, −	3, −	4, −	−, −	−, −	Ⓗ, 0	11, −
10	−, −	2, −	−, −	4, −	−, −	Ⓘ, 0	−, −	11, −
11	−, −	−, −	−, −	−, −	8, −	10, −	9, −	Ⓚ, 0
12	−, −	15, −	16, −	−, −	Ⓛ, 1	−, −	−, −	7, −
13	−, −	15, −	−, −	17, −	−, −	Ⓜ, 1	−, −	7, −
14	−, −	−, −	16, −	17, −	−, −	−, −	Ⓨ, 1	7, −
15	1, −	Ⓟ, 1	−, −	−, −	12, −	13, −	−, −	−, −
16	1, −	−, −	Ⓠ, 1	−, −	12, −	−, −	14, −	−, −
17	1, −	−, −	−, −	Ⓡ, 1	−, −	13, −	14, −	−, −

Step 3: State Reduction (Row Merger)

We find the maximal compatibles by merging rows that have identical or non-conflicting transitions and outputs. Evaluating the primitive rows yields five distinct closed compatible groups:

$S_A = \{1\}$ (Reset State)

$S_B = \{2\}$ (Input a high first)

$S_C = \{5\}$ (Inputs a then b high)

$S_E = \{3, 4, 6, 8, 9, 10, 11\}$ (Failure sequence path, $U = 0$)

$S_F = \{7, 12, 13, 14, 15, 16, 17\}$ (Success and latch path, $U = 1$)

Step 4: Reduced Flow Table

Replacing the primitive states with their corresponding merged groups (S_A, S_B, S_C, S_E, S_F), we construct the reduced flow table:

State	Inputs abc							
	000	100	010	001	110	101	011	111
S_A	Ⓐ, 0	S_B , 0	S_E , 0	S_E , 0	−, 0	−, 0	−, 0	−, 0
S_B	S_A , 0	Ⓑ, 0	−, 0	−, 0	S_C , 0	S_E , 0	−, 0	−, 0
S_C	−, 0	S_B , 0	S_E , 0	−, 0	Ⓔ, 0	−, 0	−, 0	S_F , 1
S_E	S_A , 0	S_B , 0	Ⓒ, 0	Ⓓ, 0	Ⓔ, 0	Ⓕ, 0	Ⓗ, 0	Ⓖ, 0
S_F	S_A , 0	Ⓟ, 1	Ⓠ, 1	Ⓡ, 1	Ⓢ, 1	Ⓣ, 1	Ⓨ, 1	Ⓡ, 1

Step 5: Race-Free State Assignment

The minimum number of state variables required to encode 5 states is $\lceil \log_2(5) \rceil = 3$ variables (y_1, y_2, y_3), creating an 8-row state space. To prevent critical races, all specified state transitions must have a Hamming distance of 1, or be routed through adjacent transient states.

Analyzing the required transitions:

- S_A transitions directly to: S_B, S_E
- S_B transitions directly to: S_A, S_C, S_E
- S_C transitions directly to: S_B, S_E, S_F
- S_E and S_F transition back directly to S_A at $abc = 000$.

We can choose a highly optimized, completely race-free 3-variable map assignment:

$S_A = 000, \quad S_B = 100, \quad S_C = 101, \quad S_F = 001, \quad S_E = 010$

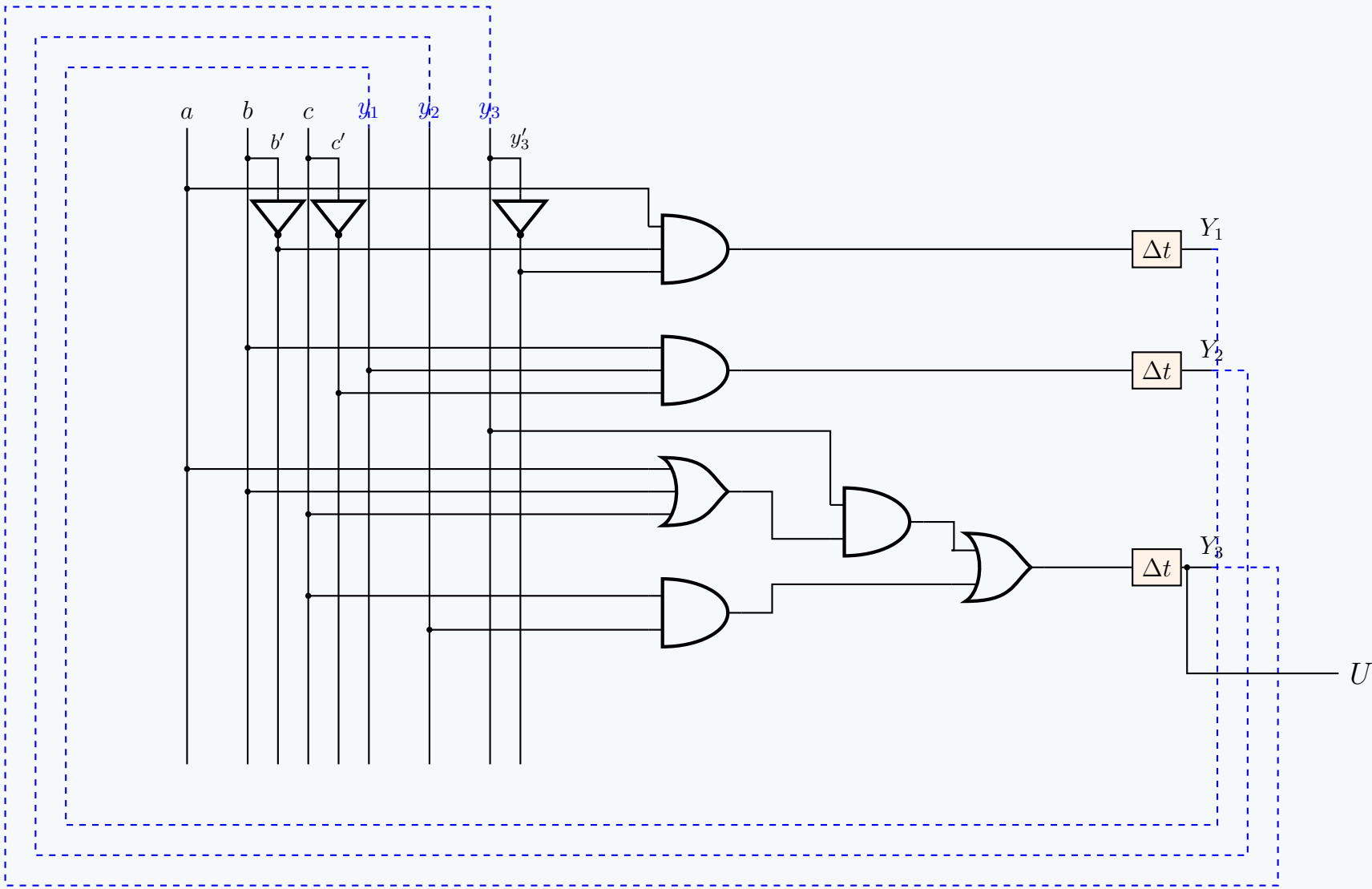
The remaining states **110** and **111** are utilized as controlled transient states (**T₁** and **T₂**) to route the multi-bit transitions to S_E without causing critical races:

- For the transition $S_B(100) \rightarrow S_E(010)$ at $abc = 101$: Route via $100 \rightarrow \mathbf{T_1(110)} \rightarrow S_E(010)$.
- For the transition $S_C(101) \rightarrow S_E(010)$ at $abc = 010$: Route via $101 \rightarrow \mathbf{T_2(111)} \rightarrow \mathbf{T_1(110)} \rightarrow S_E(010)$.

Step 6: Final Transition Table

Mapping the chosen codes and tracking the transient states, we get the complete race-free transition table:

State	$y_1y_2y_3$	Next State $Y_1Y_2Y_3$, Output U							
		000	100	010	001	110	101	011	111
S_A	000	000, 0	100, 0	010, 0	010, 0	—, —	—, —	—, —	—, —
S_B	100	000, 0	100, 0	—, —	—, —	101, 0	110, 0	—, —	—, —
S_C	101	—, —	100, 0	111, 0	—, —	101, 0	—, —	—, —	001, 1
S_E	010	000, 0	100, 0	010, 0	010, 0	010, 0	010, 0	010, 0	010, 0
S_F	001	000, 0	001, 1	001, 1	001, 1	001, 1	001, 1	001, 1	001, 1
T_1	110	—, —	—, —	010, 0	—, —	—, —	010, 0	—, —	—, —
T_2	111	—, —	—, —	110, 0	—, —	—, —	—, —	—, —	—, —
—	011	—, —	—, —	—, —	—, —	—, —	—, —	—, —	—, —



Q2. At a junction of a single-track railroad and a road, traffic lights are to be installed. The lights are to be controlled by switches that are pressed or released by the trains. When a train approaches the junction from either direction and is within 1500 feet from it, the lights are to change from green to red and remain red until the train is 1500 feet past the junction. You may assume that the length of a train is smaller than 3000 feet. Determine a primitive flow table and reduce it. (20 marks) [CSE 205 | L-2/T-1 | 2021–22]

Answer

1. Problem Modeling and Definitions

Let the traffic light controller be an asynchronous finite state machine (FSM) operating in fundamental mode.

- **Inputs (x_1, x_2):** Let x_1 be the switch at 1500 ft on the left, and x_2 be the switch at 1500 ft on the right. A switch outputs '1' when pressed by a train, and '0' when released.
- **Output (Z):** Let $Z = 0$ represent the Green light, and $Z = 1$ represent the Red light.

Physical Constraint Analysis: The distance between the two switches is $1500 + 1500 = 3000$ ft. The problem explicitly states that the train length is *smaller* than 3000 ft. Therefore, the train will completely pass and release the first switch before it reaches the second switch.

- The input combination $x_1x_2 = 11$ (both switches pressed simultaneously) is physically impossible and is treated as a **don't care** (–) for all states.
- A train traveling Left-to-Right will generate the input sequence: $00 \rightarrow 10 \rightarrow 00 \rightarrow 01 \rightarrow 00$.
- A train traveling Right-to-Left will generate the input sequence: $00 \rightarrow 01 \rightarrow 00 \rightarrow 10 \rightarrow 00$.

2. Primitive Flow Table Construction

We define a distinct stable state for each valid input configuration in the sequences:

- **State A:** Initial state. No train present. ($x_1x_2 = 00, Z = 0$)
- **Left-to-Right (L $\rightarrow$ R) Path:**
 - **State B:** Train hits the left switch. ($x_1x_2 = 10, Z = 1$)
 - **State C:** Train is entirely between the switches. ($x_1x_2 = 00, Z = 1$)
 - **State D:** Train hits the right switch. ($x_1x_2 = 01, Z = 1$)
- **Right-to-Left (R $\rightarrow$ L) Path:**
 - **State E:** Train hits the right switch. ($x_1x_2 = 01, Z = 1$)
 - **State F:** Train is entirely between the switches. ($x_1x_2 = 00, Z = 1$)
 - **State G:** Train hits the left switch. ($x_1x_2 = 10, Z = 1$)

Populating the transitions based on fundamental mode (only one input changes at a time), we get the Primitive Flow Table. *Note: Bold parentheses denote stable states.*

Current State	Next State for Inputs x_1x_2				Output Z
	00	01	11	10	
A	(A)	E	–	B	0
B	C	–	–	(B)	1
C	(C)	D	–	–	1
D	A	(D)	–	–	1
E	F	(E)	–	–	1
F	(F)	–	–	G	1
G	A	–	–	(G)	1

3. State Reduction

To reduce the table, we identify *compatible* states. Two states are compatible if they have the same outputs and their next-state transitions do not conflict for any input combination.

- State A has an output of 0 and cannot be merged with any other state. Group 0: {A}
- States B, C, D, E, F, G have an output of 1. Group 1: {B, C, D, E, F, G}

Let us check for compatible pairs within Group 1:

- **Pair (B, C):** Transitions are (C, C) for 00, (B, –) for 10. No conflicts. **Compatible.**
- **Pair (E, F):** Transitions are (F, F) for 00, (E, –) for 01. No conflicts. **Compatible.**
- **Pair (D, G):** Transitions are (A, A) for 00, (D, –) for 01, (–, G) for 10. No conflicts. **Compatible.**

By merging these mutually compatible pairs, we form a closed covering of maximal compatibles:

$$S_0 = \{A\}$$
$$S_1 = \{B, C\}$$
$$S_2 = \{E, F\}$$
$$S_3 = \{D, G\}$$

Here, S_0 is the track-clear state, S_1 tracks a train moving left-to-right until it hits the final switch, S_2 tracks a train moving right-to-left until it hits the final switch, and S_3 elegantly captures the final clearing stage of the train regardless of its direction.

4. Reduced Flow Table

Mapping the primitive states to our new merged states (S_0, S_1, S_2, S_3), we obtain the minimum row reduced flow table:

Reduced State	Next State for Inputs x_1x_2				Output Z
	00	01	11	10	
S_0	(S_0)	S_2	—	S_1	0
S_1	(S_1)	S_3	—	(S_1)	1
S_2	(S_2)	(S_2)	—	S_3	1
S_3	S_0	(S_3)	—	(S_3)	1

The asynchronous FSM is now fully minimized to 4 states, requiring exactly 2 state variables for hardware implementation.

Q3. Design a two-input (x_1, x_2), one-output (z) fundamental mode circuit that operates as follows. The output changes from 0 to 1 only on the first x_1 input change that follows an x_2 input change. A $1 \rightarrow 0$ output change occurs only when x_1 changes from 1 to 0 while $x_2 = 1$. Determine the primitive flow table, reduced flow table, race-free state assignment, K-maps and circuit diagram. **(25 marks)**
[CSE 205 | L-2/T-1 | 2020–21]

Answer

1. Primitive Flow Table

The fundamental mode circuit operates with inputs x_1, x_2 and output z . Tracing all valid input transitions yields 12 primitive stable states. Stable states are indicated by circled state numbers:

State	Inputs x_1x_2				Output z
	00	01	11	10	
1	(1)	2	—	4	0
2	1	(2)	3	—	0
3	—	5	(3)	4	1
4	6	—	3	(4)	1
5	1	(5)	7	—	0
6	(6)	8	—	4	1
7	—	5	(7)	9	0
8	6	(8)	3	—	1
9	6	—	10	(9)	0
10	—	8	(10)	9	0
11	(11)	2	—	12	0
12	11	—	7	(12)	0

2. Merger Diagram and Reduced Flow Table

From the implication analysis and merger diagram, the compatible state groups are:

$$a = (1, 2), \quad b = (3, 4, 6, 8), \quad c = (5, 7), \quad d = (9, 10), \quad e = (11, 12), \quad f = (6, 11 \text{ overlap variants})$$

This produces the minimal reduced flow table:

State	Inputs x_1x_2				Output z
	00	01	11	10	
a	$(a)/0$	$(a)/0$	$b/-$	$b/-$	0
b	$d/1$	$c/-$	$(b)/1$	$(b)/1$	1
c	$a/0$	$(c)/0$	$(c)/0$	$e/0$	0
d	$(d)/1$	$(d)/1$	$b/1$	$(d)/1$	1
e	$d/-$	$d/-$	$(e)/0$	$(e)/0$	0
f	$(f)/0$	$a/0$	$c/0$	$(f)/0$	0

3. Race-Free State Assignment

Using 3 state variables (y_1, y_2, y_3) , two transient states c^1 and f^1 are added to ensure race-free transitions on a 3-cube map:

State Map	$y_2y_3 = 00$	$y_2y_3 = 01$	$y_2y_3 = 11$	$y_2y_3 = 10$
$y_1 = 0$	a (000)	b (001)	c (011)	c^1 (010)
$y_1 = 1$	f (100)	d (101)	e (111)	f^1 (110)

4. Excitation and Output Tables

State	$y_1y_2y_3$	Next State ($Y_1Y_2Y_3$) for x_1x_2				Output z for x_1x_2			
		00	01	11	10	00	01	11	10
a	000	000	000	001	001	0	0	–	–
b	001	101	011	001	001	1	–	1	1
c^1	010	000	–	011	–	0	–	0	–
c	011	010	011	011	111	0	0	0	0
f	100	100	000	110	100	0	0	0	0
d	101	101	101	001	101	1	1	1	1
f^1	110	–	–	010	–	–	–	0	–
e	111	101	101	111	111	–	–	0	0

5. Simplified Logic Equations

$$Y_1 = \bar{x}_1\bar{x}_2\bar{y}_2y_3 + x_1\bar{x}_2y_2 + y_1y_2y_3 + x_1y_1\bar{y}_2\bar{y}_3 + \bar{x}_1y_1y_3 + \bar{x}_2y_1$$
$$Y_2 = \bar{x}_1x_2\bar{y}_1y_3 + x_1x_2y_1\bar{y}_3 + \bar{y}_1y_2y_3 + x_1y_2$$
$$Y_3 = \bar{y}_2y_3 + x_1\bar{x}_2\bar{y}_1 + y_1y_3 + x_2y_3$$
$$z = \bar{y}_2y_3$$

- Q4.** Design a fundamental mode circuit to function as an electrical lock. The lock has two switch inputs X_1 and X_2 . Design the circuit so that the lock-open signal $Z = 1$ is produced only after the following conditions have been satisfied:
- (a) Begin with $X_1 = X_2 = 0$.
 - (b) While $X_2 = 0$, X_1 is turned on and then turned off.
 - (c) Then while X_1 remains off, X_2 is turned on to open the lock ($Z = 1$).
 - (d) For any deviation in the sequence of the above transitions, the lock remains closed ($Z = 0$).

Determine the primitive flow table, reduced flow table, race-free state assignment, K-maps and circuit diagram. **(25 marks)** [CSE 205 | L-2/T-1 | 2019–20]

Answer

Design of Fundamental Mode Electrical Lock Circuit

1. Primitive Flow Table

Based on the problem statement, we define the following sequence of operations starting from the reset/initial state:

- (a) Start at $X_1X_2 = 00$ (State 1).
- (b) X_1 is turned on while $X_2 = 0 \implies X_1X_2 = 10$ (State 2).
- (c) X_1 is turned off while $X_2 = 0 \implies X_1X_2 = 00$ (State 3).

- (d) X_2 is turned on while $X_1 = 0 \implies X_1X_2 = 01$ (State 4), which sets $Z = 1$.
- (e) Any deviation leads to lock-closed conditions (States 5 and 6 represent error states).

The resulting primitive flow table is constructed below:

Primitive Flow Table	State	Inputs X_1X_2				Output Z
		00	01	11	10	
	1	(1)	5	–	2	0
	2	3	–	6	(2)	0
	3	(3)	4	–	2	0
	4	1	(4)	6	–	1
	5	1	(5)	6	–	0
	6	–	5	(6)	2	0

2. Reduction of Flow Table

To find the maximal compatibles, we examine row merging options by checking for output compatibility and next-state conflicts:

- Row 1 and Row 5 can be merged because their outputs match ($Z = 0$) and next-states are compatible: $(1, 1)$, $(5, 5)$, $(-, 6)$, $(2, -)$.
- Row 1 and Row 6 can be merged: $(1, -)$, $(5, 5)$, $(-, 6)$, $(2, 2)$.
- Row 5 and Row 6 can be merged: $(1, -)$, $(5, 5)$, $(6, 6)$, $(-, 2)$.
- Therefore, $\{1, 5, 6\}$ forms a compatible group, designated as State **A**.
- Row 2 and Row 3 can be merged: $(3, 3)$, $(-, 4)$, $(6, -)$, $(2, 2)$. This forms a compatible group, designated as State **B**.
- Row 4 cannot be merged with any other row due to its unique output requirement ($Z = 1$). This remains State **C**.

The closed set of maximal compatibles is:

$$\begin{aligned} A &= \{1, 5, 6\} \\ B &= \{2, 3\} \\ C &= \{4\} \end{aligned}$$

The reduced flow table is given below:

Reduced Flow Table	Present State	Next State, Output Z			
		00	01	11	10
	A	A, 0	A, 0	A, 0	B, 0
	B	B, 0	C, 0	A, 0	B, 0
	C	A, 0	C, 1	A, 0	–, 0

3. Race-Free State Assignment

The transition requirements between states are:

- $A \rightarrow B$ and $B \rightarrow A$ (requires adjacency)
- $B \rightarrow C$ and $C \rightarrow B$ (requires adjacency)
- $C \rightarrow A$ (requires adjacency)

Since all three states must be mutually adjacent, a direct 2-variable assignment will cause a race during the $C \rightarrow A$ transition. To eliminate this race, we use the unassigned fourth state combination **D** as a transient state to route $C \rightarrow D \rightarrow A$. We choose the following race-free assignment:

$$\begin{aligned} A &= 00 \\ B &= 01 \\ C &= 11 \\ D &= 10 \quad (\text{Transient/Unused State}) \end{aligned}$$

Transition Table	Present State	Y_1Y_2	Next State $Y_1^+Y_2^+$			
			00	01	11	10
	A (00)		00	00	00	01
	B (01)		01	11	00	01
	C (11)		10	11	10	XX
	D (10)		00	XX	00	XX

4. Karnaugh Maps and Logic Equations

We derive the simplified expressions for Y_1^+ , Y_2^+ , and Z using Karnaugh Maps.

K-Map for Y_1^+

		X_1X_2			
		00	01	11	10
Y_1Y_2	00	0	0	0	0
	01	0	1	0	0
	11	1	1	1	X
	10	0	X	0	X

From the K-map, including redundant terms to eliminate static hazards, the equation is:

$$Y_1^+ = Y_1Y_2 + Y_2\bar{X}_1X_2$$
$$Y_1^+ = Y_2(Y_1 + \bar{X}_1X_2)$$

(By Grouping)

(Distributive Law)

K-Map for Y_2^+

		X_1X_2			
		00	01	11	10
Y_1Y_2	00	0	0	0	1
	01	1	1	0	1
	11	0	1	0	X
	10	0	X	0	X

From the K-map, the hazard-free equation is:

$$Y_2^+ = X_1\bar{X}_2 + \bar{Y}_1Y_2\bar{X}_1 + Y_2\bar{X}_1X_2$$
$$Y_2^+ = X_1\bar{X}_2 + Y_2\bar{X}_1(\bar{Y}_1 + X_2)$$

(By Grouping)

(Distributive Law)

K-Map for Output Z

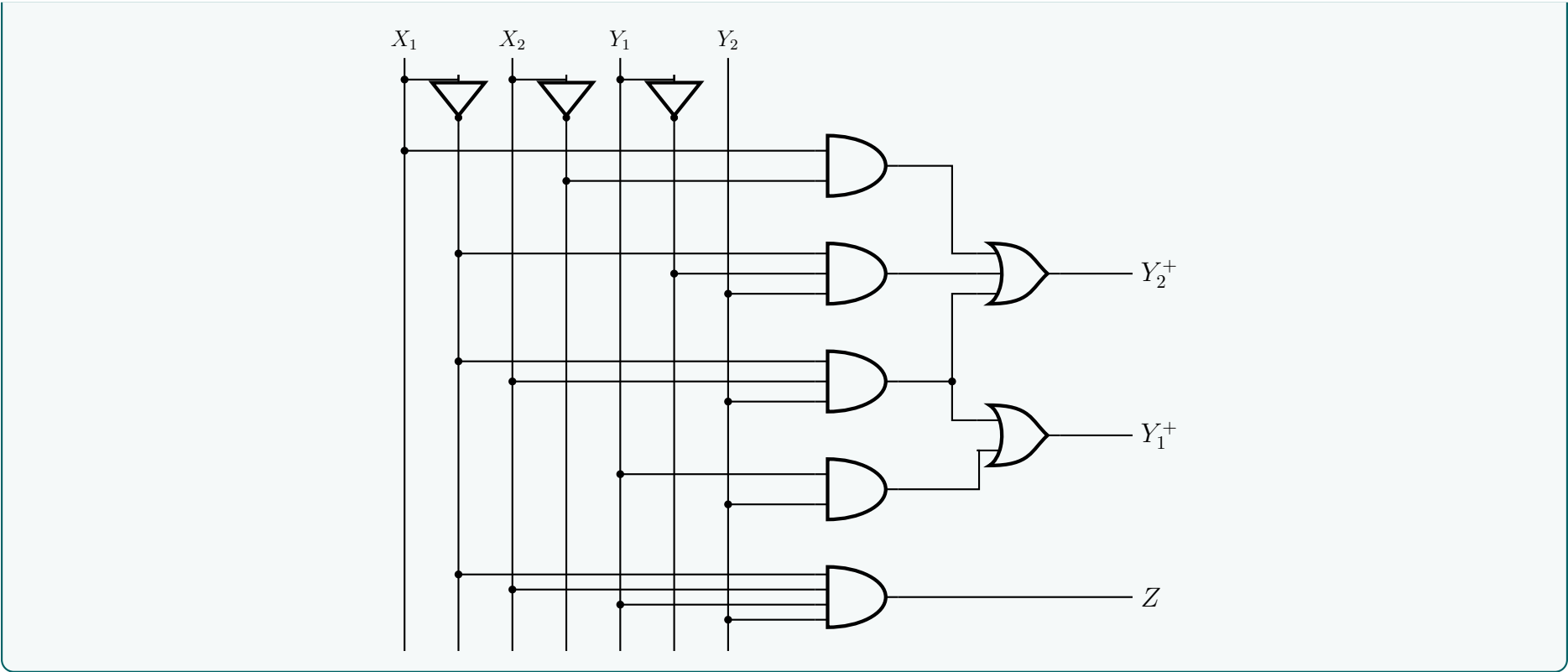
		X_1X_2			
		00	01	11	10
Y_1Y_2	00	0	0	0	0
	01	0	0	0	0
	11	0	1	0	0
	10	0	0	0	0

The output equation is directly obtained as:

$$Z = Y_1Y_2\bar{X}_1X_2$$

5. Circuit Diagram

The schematic implementation of the hazard-free lock control system using basic logic gates is drawn below:



Q5. Construct a primitive flow table for a fundamental mode circuit where the output $z = 0$ when the two inputs x_1 and x_2 are equal. $z = 1$ results when $x_1 = 0$ and x_2 changes from 0 to 1, and when $x_1 = 1$ and x_2 changes from 1 to 0. No other input change causes any output change. Determine the reduced flow table containing the minimum number of rows. **(10+10 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Answer

Design of Fundamental Mode Asynchronous Circuit

1. Problem Analysis and State Definition

We are tasked with designing a fundamental mode sequential circuit with two inputs (x_1, x_2) and one output (z). In fundamental mode, only one input changes at a time, and the circuit must settle into a stable state before the next input change. Based on the problem statement, we define the following rules for the output z :

- (a) $z = 0$ when $x_1 = x_2$ (i.e., input combinations 00 and 11).
- (b) $z = 1$ when $x_1 = 0$ and x_2 transitions from 0 $\rightarrow$ 1 (i.e., transition 00 $\rightarrow$ 01).
- (c) $z = 1$ when $x_1 = 1$ and x_2 transitions from 1 $\rightarrow$ 0 (i.e., transition 11 $\rightarrow$ 10).
- (d) No other input change causes any output change.

To construct the primitive flow table, we allocate one stable state for each valid input combination reached via a specific sequence:

- **State 1:** Initial stable state for inputs $x_1x_2 = 00$. Output $z = 0$.
- **State 4:** Initial stable state for inputs $x_1x_2 = 11$. Output $z = 0$.
- **State 2:** Reached from State 1 via transition 00 $\rightarrow$ 01. Output becomes $z = 1$. Stable at 01.
- **State 3:** Reached from State 1 via transition 00 $\rightarrow$ 10. No output change, so $z = 0$. Stable at 10.
- **State 5:** Reached from State 4 via transition 11 $\rightarrow$ 01. No output change, so $z = 0$. Stable at 01.
- **State 6:** Reached from State 4 via transition 11 $\rightarrow$ 10. Output becomes $z = 1$. Stable at 10.

2. Primitive Flow Table

Using the states defined above, we populate the transition rows. Double input changes (e.g., 00 $\rightarrow$ 11) are impossible in fundamental mode and are marked as don't cares (–). **Primitive Flow Table**

Present State	Next State for Inputs x_1x_2				Output z
	00	01	11	10	
1	(1)	2	–	3	0
2	1	(2)	4	–	1
3	1	–	4	(3)	0
4	–	5	(4)	6	0
5	1	(5)	4	–	0
6	1	–	4	(6)	1

Note: Stable states are encircled with parentheses.

3. State Reduction

To determine the reduced flow table, we must find the minimal number of rows by merging compatible states. Two states are compatible if, for every valid input, their specified next states are compatible, and their specified outputs do not conflict. Let us evaluate the compatibility pairs:

- **States with differing outputs** can never be merged. This eliminates any pairings between the set with $z = 0$ $\{1, 3, 4, 5\}$ and the set with $z = 1$ $\{2, 6\}$.
- **Pair (1, 3):** Outputs are 0. Next states: (1, 1), (2, −), (−, 4), (3, 3). No conflicting next states. **(Compatible)**
- **Pair (1, 4):** Outputs are 0. Next state for 01 requires states 2 and 5 to be compatible. However, State 2 ($z = 1$) and State 5 ($z = 0$) have conflicting outputs. **(Incompatible)**
- **Pair (1, 5):** Outputs are 0. Next state for 01 requires states 2 and 5 to be compatible. **(Incompatible)**
- **Pair (2, 6):** Outputs are 1. Next states: (1, 1), (2, −), (4, 4), (−, 6). No conflicting next states. **(Compatible)**
- **Pair (3, 4):** Outputs are 0. Next state for 10 requires states 3 and 6 to be compatible, but they have conflicting outputs. **(Incompatible)**
- **Pair (3, 5):** Outputs are 0. Next states: (1, 1), (−, 5), (4, 4), (3, −). No conflicting next states. **(Compatible)**
- **Pair (4, 5):** Outputs are 0. Next states: (−, 1), (5, 5), (4, 4), (6, −). No conflicting next states. **(Compatible)**

Maximal Compatibles: From the derived compatible pairs (1, 3), (2, 6), (3, 5), and (4, 5), we deduce the maximal compatible (MC) sets:

$$\begin{aligned} \text{MC}_1 &= \{1, 3\} \\ \text{MC}_2 &= \{4, 5\} \\ \text{MC}_3 &= \{2, 6\} \\ \text{MC}_4 &= \{3, 5\} \end{aligned}$$

Minimal Closed Covering: We select a closed set of maximal compatibles that completely covers all original states $\{1, 2, 3, 4, 5, 6\}$.

- State 1 is only in $\{1, 3\}$, so we must select **A** = {1, 3}.
- State 4 is only in $\{4, 5\}$, so we must select **B** = {4, 5}.
- State 2 is only in $\{2, 6\}$, so we must select **C** = {2, 6}.

This selection {**A**, **B**, **C**} successfully covers all states $\{1, 2, 3, 4, 5, 6\}$. The selection is fully closed since any transitions from within these sets map cleanly to one of the selected sets.

4. Reduced Flow Table

We define our minimal rows as:

$$\begin{aligned} \mathbf{A} = \{1, 3\} &\implies \text{Output } z = 0 \\ \mathbf{B} = \{4, 5\} &\implies \text{Output } z = 0 \\ \mathbf{C} = \{2, 6\} &\implies \text{Output } z = 1 \end{aligned}$$

Replacing the old states with the combined states A, B, and C in the transition map yields the minimal, 3-row reduced flow table.
Reduced Flow Table

Present State	Next State for Inputs x_1x_2				Output z
	00	01	11	10	
A	(A)	C	B	(A)	0
B	A	(B)	(B)	C	0
C	A	(C)	B	(C)	1

Q6. Design a circuit for a digital lock using T latches. The lock has two push-button inputs A and B and produces a single pulse Z_1 for each activation. The two switches are mechanically interlocked so that simultaneous pulses are not possible. Features: (i) opening sequence is AABABA; (ii) three B pulses give absolute reset; (iii) any incorrect use of A produces output Z_2 to ring a bell. **(20 marks)**
[CSE 205 | L-2/T-1 | 2018–19]

Answer

Design of Digital Lock using T Latches (Pulse-Mode Asynchronous FSM)

1. Problem Analysis & State Definition

The lock operates via two mechanically interlocked push-buttons, A and B , which act as mutually exclusive pulse inputs. This indicates a **pulse-mode asynchronous sequential circuit** where unlocked T-latches toggle on the arrival of A or B pulses. Based on the specifications:

- **Valid Sequence:** $A \rightarrow A \rightarrow B \rightarrow A \rightarrow B \rightarrow A$. Completing this sequence triggers the unlock pulse Z_1 .
- **Absolute Reset:** Three consecutive B pulses reset the system to the initial state, regardless of the current state.
- **Alarm:** Any incorrect use of A (i.e., pressing A when it does not advance the valid sequence) triggers an alarm bell pulse Z_2 and locks the user in an error state until an absolute reset is performed.

We define 9 strictly necessary states to satisfy these conditions:

- **Sequence States (0 consecutive B 's):** S_0 (Start), S_1 (A), S_2 (AA), S_3 (AAB), S_4 ($AABA$), S_5 ($AABAB$).
- **Error/Reset States:**
 - E_0 : Error state with 0 consecutive B 's.
 - E_1 : Error state with 1 consecutive B (also reached from S_0, S_1 via B).
 - E_2 : Error state with 2 consecutive B 's (also reached from S_3, S_5 via an incorrect second B).

2. State Diagram

3. State Transition Table & State Encoding

To minimize logic complexity, we intelligently encode the 9 states into 4 bits (y_3, y_2, y_1, y_0) such that y_3 acts as an error flag, and y_1, y_0 inherently track the B -pulse count where possible.

Present State	Encoding $(y_3y_2y_1y_0)$	Next State (A)	Out (Z_1, Z_2)	Next State (B)	Out (Z_1, Z_2)
S_0	0000	S_1 (0001)	0, 0	E_1 (1001)	0, 0
S_1	0001	S_2 (0011)	0, 0	E_1 (1001)	0, 0
S_2	0011	E_0 (1000)	0, 1	S_3 (0010)	0, 0
S_3	0010	S_4 (0110)	0, 0	E_2 (1011)	0, 0
S_4	0110	E_0 (1000)	0, 1	S_5 (0111)	0, 0
S_5	0111	S_0 (0000)	1, 0	E_2 (1011)	0, 0
E_0	1000	E_0 (1000)	0, 1	E_1 (1001)	0, 0
E_1	1001	E_0 (1000)	0, 1	E_2 (1011)	0, 0
E_2	1011	E_0 (1000)	0, 1	S_0 (0000)	0, 0

Unused States (Don't Cares): 0100 (4), 0101 (5), 1010 (10), 1100 (12), 1101 (13), 1110 (14), 1111 (15).

4. Derivation of T-Latch Excitation Equations

For pulse-mode T-latches, the excitation is separated into conditions toggling on pulse A (T^A) and pulse B (T^B):

$$T_i = A \cdot T_i^A + B \cdot T_i^B$$

where $T_i^A = y_i \oplus Y_i^A$ and $T_i^B = y_i \oplus Y_i^B$. Evaluating bit toggles from the transition table yields the minterms (where the function is 1) for each excitation signal.

Toggle Matrices Analysis (1s at minterms):

- $T_3^A = \sum m(3, 6)$
- $T_2^A = \sum m(2, 6, 7)$
- $T_1^A = \sum m(1, 3, 6, 7, 11)$
- $T_0^A = \sum m(0, 3, 7, 9, 11)$
- $T_3^B = \sum m(0, 1, 2, 7, 11)$
- $T_2^B = \sum m(7)$
- $T_1^B = \sum m(9, 11)$
- $T_0^B = \sum m(0, 2, 3, 6, 8, 11)$

Applying Karnaugh map minimization with Don't Cares $\sum d(4, 5, 10, 12, 13, 14, 15)$, we derive the Boolean expressions.

Pulse A Excitations (T_i^A):

$$\begin{aligned} T_3^A &= y_3' y_2' y_1 y_0 + y_2 y_0' && \text{(Groups: } m_3 \text{ isolated, } m_6 \text{ with } d_{14,4,12}) \\ T_2^A &= y_2 y_1 + y_2' y_1 y_0' && \text{(Groups: } m_{6,7} \text{ with } d_{14,15}, m_2 \text{ with } d_{10}) \\ T_1^A &= y_2 y_1 + y_1 y_0 + y_3' y_0 && \text{(Groups: } m_{6,7,14,15}, m_{3,7,11,15}, m_{1,3,5,7}) \\ T_0^A &= y_1 y_0 + y_3 y_0 + y_3' y_1 y_0' && \text{(Groups: } m_{3,7,11,15}, m_{9,11,13,15}, m_{0,4}) \end{aligned}$$

Pulse B Excitations (T_i^B):

$$\begin{aligned} T_3^B &= y_3' y_1' + y_3 y_1 + y_2 y_0 + y_2' y_1 y_0' && \text{(Groups: } m_{0,1,4,5}, m_{10,11,14,15}, m_{5,7,13,15}, m_{2,10}) \\ T_2^B &= y_2 y_0 && \text{(Groups: } m_7 \text{ with } d_{5,13,15}) \\ T_1^B &= y_3 y_0 && \text{(Groups: } m_{9,11} \text{ with } d_{13,15}) \\ T_0^B &= y_1' y_0' + y_2 y_0' + y_2' y_1 && \text{(Groups: } m_{0,4,8,12}, m_{4,6,12,14}, m_{2,3,10,11}) \end{aligned}$$

5. Outputs Derivation

The network produces pulse outputs Z_1 and Z_2 strictly via combinational logic triggered by the incoming pulse A .

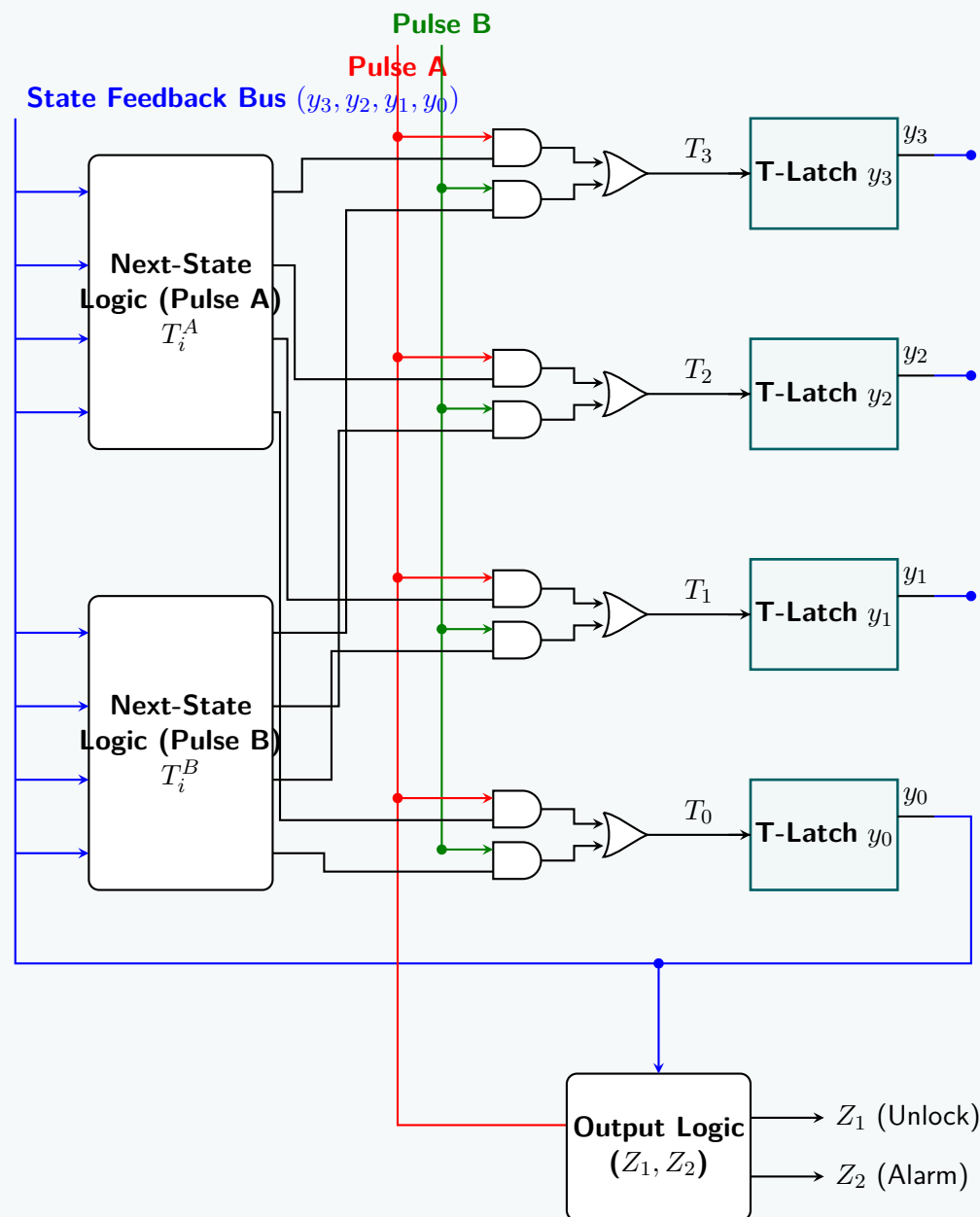
$$\begin{aligned} Z_1 \text{ (Unlock)} &= A \cdot \sum m(7) = A \cdot (y_2 y_1 y_0) \\ Z_2 \text{ (Bell)} &= A \cdot \sum m(3, 6, 8, 9, 11) = A \cdot (y_3 y_2' + y_2 y_1 y_0' + y_2' y_1 y_0) \end{aligned}$$

6. Structural Circuit Implementation

Given the complex mathematical nature of 4 fully populated T-latches, manual wiring leads to illegible schematics. The circuit structural topology follows standard pulse-mode asynchronous design:

- Latches:** Four T-latches (y_3, y_2, y_1, y_0) outputting present states natively and inverted.
- Pulse Steering Logic:** Two separate combinational logic blocks compute vectors $\mathbf{T}^A$ and $\mathbf{T}^B$ based entirely on the state feedback ($y_3, \dots, y_0$) as synthesized above.
- Pulse Merge:** Eight AND gates multiply the physical button pulses with their respective excitation signals: $T_{i,A_{eval}} = A \cdot T_i^A$ and $T_{i,B_{eval}} = B \cdot T_i^B$.
- Latch T-Inputs:** Four OR gates collect the partial excitations to drive each latch: $T_i = T_{i,A_{eval}} + T_{i,B_{eval}}$.
- Outputs:** Two discrete output AND gates generate Z_1 and Z_2 as directly derived in Section 5.

7. Schematic Circuit Diagram



Q7. Construct a primitive flow table for a fundamental mode circuit with two inputs (x_1, x_2) and two outputs (z_1, z_2). Specifications: when $x_1x_2 = 00$, $z_1z_2 = 00$. Output 10 is produced following input sequence $00 \rightarrow 01 \rightarrow 11$ and remains until x_1x_2 returns to 00. Output 01 is produced following sequence $00 \rightarrow 10 \rightarrow 11$ and remains until 00 input returns the output to 00. Determine the minimum row-reduced flow table. (10+10 marks) [CSE 205 | L-2/T-1 | 2017–18]

Answer

Design of Fundamental Mode Asynchronous Circuit

To design a fundamental mode circuit, we assume that only one input variable changes at a time, and the circuit reaches a stable state before the next input change occurs.

1. Derivation of the Primitive Flow Table

We first define the necessary states to capture the specific sequences. A primitive flow table has exactly one stable state per row. Let the inputs be x_1x_2 and the outputs be z_1z_2 . Stable states are denoted in **bold with parentheses**.

- **Reset/Initial State (1):** Stable at 00 with output 00.
- **Tracking $00 \rightarrow 01$ (2):** Changing inputs to 01 moves the circuit to a new state stable at 01, output 00.
- **Tracking $00 \rightarrow 10$ (3):** Changing inputs to 10 moves the circuit to a new state stable at 10, output 00.
- **Sequence $00 \rightarrow 01 \rightarrow 11$ completion (4):** From state 2, an input change to 11 triggers the output 10. Stable at 11.
- **Sequence $00 \rightarrow 10 \rightarrow 11$ completion (5):** From state 3, an input change to 11 triggers the output 01. Stable at 11.
- **Hold Output 10 (6 & 7):** Output 10 must remain until inputs return to 00. Hence, changing to 01 (state 6) or 10 (state 7) from state 4 maintains output 10.
- **Hold Output 01 (8 & 9):** Output 01 must remain until inputs return to 00. Hence, changing to 01 (state 8) or 10 (state 9) from state 5 maintains output 01.

Populating the transitions based on single-variable changes yields the Primitive Flow Table:

Present State	Next State for Inputs x_1x_2				Output z_1z_2
	00	01	11	10	
1	(1)	2	–	3	00
2	1	(2)	4	–	00
3	1	–	5	(3)	00
4	–	6	(4)	7	10
5	–	8	(5)	9	01
6	1	(6)	4	–	10
7	1	–	4	(7)	10
8	1	(8)	5	–	01
9	1	–	5	(9)	01

2. State Minimization (Row Reduction)

To find the minimum row-reduced flow table, we group states that have the same outputs and check for compatibility using an implication analysis. Two states are compatible if their next states for every input are either identical, compatible, or unspecified (don't cares).

Group 1 (Output 00): {1, 2, 3}

- (1, 2): Compatible. Merging yields next states (1, 2, 4, 3). No conflicting specified next states.
- (1, 3): Compatible. Merging yields next states (1, 2, 5, 3). No conflicts.
- (2, 3): **Incompatible**. At input 11, state 2 goes to 4 (output 10), while state 3 goes to 5 (output 01).

Maximal compatibles in this group: {1, 2} and {1, 3}.

Group 2 (Output 10): {4, 6, 7}

- Combining 4, 6, and 7: Next states for 00 are (–, 1, 1) $\implies$ 1. Next states for 01 are (6, 6, –) $\implies$ 6. Next states for 11 are (4, 4, 4) $\implies$ 4. Next states for 10 are (7, –, 7) $\implies$ 7.
- Maximal compatible: {4, 6, 7}.

Group 3 (Output 01): {5, 8, 9}

- Combining 5, 8, and 9: Next states for 00 are (–, 1, 1) $\implies$ 1. Next states for 01 are (8, 8, –) $\implies$ 8. Next states for 11 are (5, 5, 5) $\implies$ 5. Next states for 10 are (9, –, 9) $\implies$ 9.
- Maximal compatible: {5, 8, 9}.

Selection of a Minimal Closed Cover: We must cover all 9 primitive states with the minimum number of compatible sets. We can select:

$A = \{1, 2\}, \quad B = \{3\}, \quad C = \{4, 6, 7\}, \quad D = \{5, 8, 9\}$

(Note: Selecting $A = \{2\}$ and $B = \{1, 3\}$ is equally valid and results in the same minimal table size).

3. Minimum Row-Reduced Flow Table

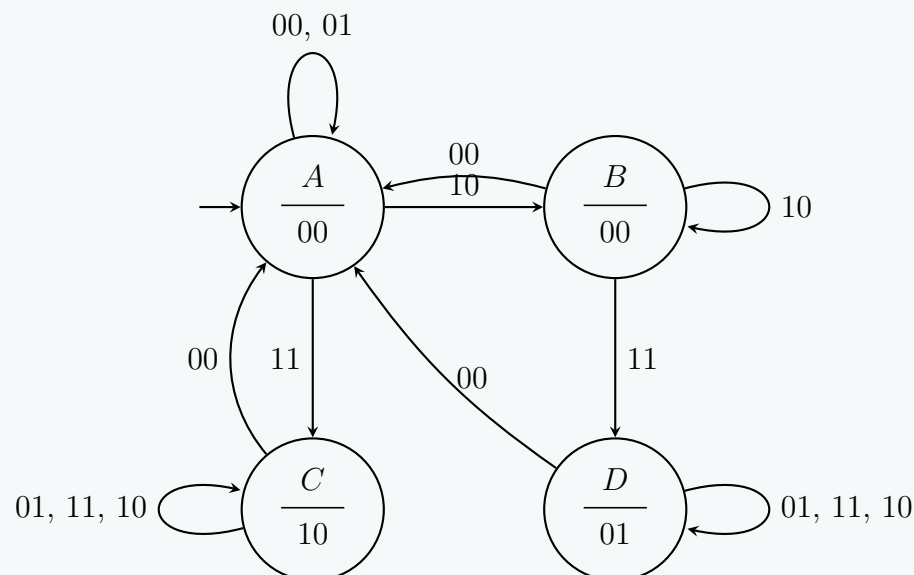
We now replace the primitive states with our newly defined states A, B, C, D .

- For row $A = \{1, 2\}$: The merged next states are (1, 2, 4, 3) $\implies$ (A, A, C, B).
- For row $B = \{3\}$: The merged next states are (1, –, 5, 3) $\implies$ ($A, –, D, B$).
- For row $C = \{4, 6, 7\}$: The merged next states are (1, 6, 4, 7) $\implies$ (A, C, C, C).
- For row $D = \{5, 8, 9\}$: The merged next states are (1, 8, 5, 9) $\implies$ (A, D, D, D).

Reduced State	Original States	Next State for Inputs x_1x_2				Output z_1z_2
		00	01	11	10	
A	{1, 2}	(A)	(A)	C	B	00
B	{3}	A	–	D	(B)	00
C	{4, 6, 7}	A	(C)	(C)	(C)	10
D	{5, 8, 9}	A	(D)	(D)	(D)	01

4. State Diagram Visualization

The logic behavior of the minimal flow table can be elegantly visualized using the following asynchronous state transition diagram. The edge labels represent the specific input condition (x_1x_2) that causes the transition.



Q8. What does it mean when an asynchronous sequential circuit is assumed to operate in the fundamental mode? (4 marks) [CSE 205 | L-2/T-1 | 2015–16]

Answer

Fundamental Mode Operation in Asynchronous Circuits

An asynchronous sequential circuit is said to operate in the **fundamental mode** when its input variables are constrained to change under a strict set of timing conditions. Because asynchronous circuits do not utilize a global clock signal to synchronize state transitions, their proper operation relies heavily on the propagation delays of the internal logic and feedback paths.

Operating in the fundamental mode implies two primary, interdependent conditions:

- **Single Input Variable Change:** Only one external input variable is permitted to change its binary value at any given instant. Simultaneous transitions of multiple input variables are strictly prohibited because differences in physical wire delays or logic gate propagation times can lead to unpredictable intermediate states, known as *race conditions*.
- **State Stabilization Before Subsequent Changes:** When an input variable changes, it initiates a series of internal state changes (transient states). The circuit must be allowed enough time to reach a *stable state* before another input variable is allowed to change.

Mathematical Representation

Let the external inputs be represented by the vector $\mathbf{x} = (x_1, x_2, \dots, x_n)$ and the internal state variables (present states) by $\mathbf{y} = (y_1, y_2, \dots, y_m)$. The next state $\mathbf{Y}$ is evaluated continuously by the combinatorial logic network:

$$\mathbf{Y} = f(\mathbf{x}, \mathbf{y})$$

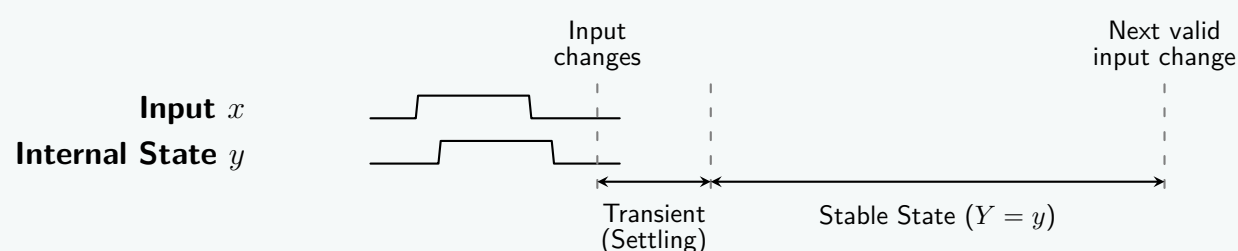
The circuit is in a **stable state** if and only if the present state is equal to the next state:

$$\mathbf{y} = \mathbf{Y}$$

Under fundamental mode, an input $\mathbf{x}$ changes to $\mathbf{x}'$ at time t_1 . The next state evaluates to $\mathbf{Y} \neq \mathbf{y}$, putting the circuit in a transient state. The fundamental mode requires that no further changes to $\mathbf{x}'$ occur until time t_2 , where $t_2 > t_1 + \Delta t_{max}$, such that $\mathbf{y}$ has successfully updated to equal $\mathbf{Y}$ and all internal logic nodes have settled.

Timing Diagram Visualization

The following timing diagram illustrates the restriction of the fundamental mode. The internal state y requires a specific propagation delay (Δt) to react to the input x . A new input change on x can only occur during the “Stable State” window.



Q9. Derive the primitive flow table for a negative-edge triggered T flip-flop. (12 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

Primitive Flow Table for a Negative-Edge Triggered T Flip-Flop

To derive the primitive flow table for an asynchronous fundamental mode circuit, we must define a unique stable state for every valid combination of inputs and internal states. We assume fundamental mode operation, meaning only one input variable changes at a time, and the circuit reaches a stable state before the next input change.

1. Definition of Inputs and Outputs

- Inputs:** T (Toggle) and C (Clock). The input state is denoted as TC .
- Output:** Q (Present state of the flip-flop).
- Behavior:** The output Q toggles its state *only* when C transitions from $1 \rightarrow 0$ (negative edge) while $T = 1$. For all other input transitions (positive clock edges, or changes in T), Q retains its current value.

2. State Assignments

We systematically trace the input transitions to map out the required stable states, categorizing them by the output Q .

States where Output $Q = 0$:

- State 1:** Stable at $TC = 00$. $Q = 0$.
- State 2:** Stable at $TC = 01$. (Entered from 1 if C rises; no toggle since $T = 0$).
- State 3:** Stable at $TC = 11$. (Entered from 2 if T rises, or from 4 if C rises).
- State 4:** Stable at $TC = 10$. (Entered from 1 if T rises. Note: If C falls from State 3, it toggles, so 3 does *not* go to 4).

States where Output $Q = 1$:

- State 5:** Stable at $TC = 10$. (Entered from 3 when C falls $1 \rightarrow 0$ while $T = 1$. This is the **toggle** action from $Q = 0$ to $Q = 1$).
- State 6:** Stable at $TC = 11$. (Entered from 5 if C rises, or from 7 if T rises).
- State 7:** Stable at $TC = 01$. (Entered from 6 if T falls, or from 8 if C rises).
- State 8:** Stable at $TC = 00$. (Entered from 7 if C falls; no toggle since $T = 0$. Also entered from 5 if T falls).

Note: If C falls from State 6 ($TC = 11$), a negative edge occurs with $T = 1$. The output toggles back to $Q = 0$, transferring the circuit to **State 4**.

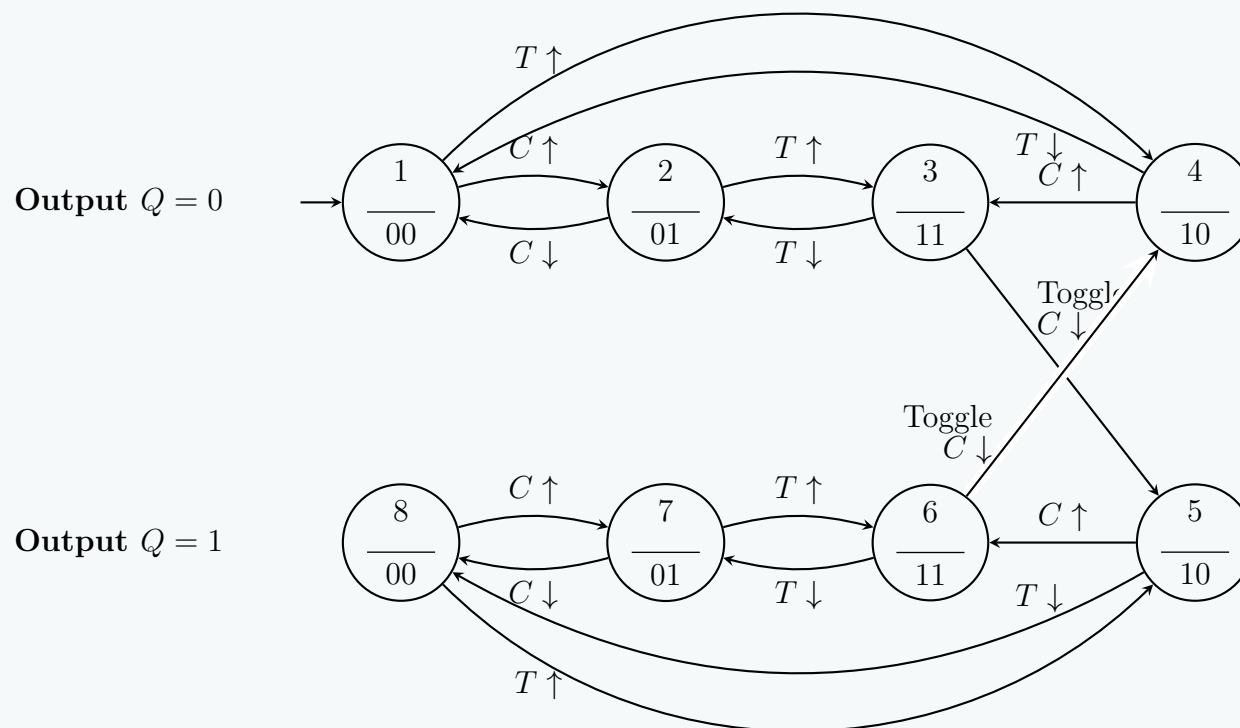
3. Primitive Flow Table

A primitive flow table has exactly one stable state per row. Unspecified transitions (double input changes) are marked with dashes (-). Stable states are circled (represented here with bold text and parentheses).

Present State	Next State for Inputs TC				Output Q
	00	01	11	10	
1	(1)	2	–	4	0
2	1	(2)	3	–	0
3	–	2	(3)	5	0
4	1	–	3	(4)	0
5	8	–	6	(5)	1
6	–	7	(6)	4	1
7	8	(7)	6	–	1
8	(8)	7	–	5	1

4. State Transition Diagram

The asynchronous behavior described above is mapped visually in the state diagram. Observe the cross-boundary transitions representing the toggle actions: $3 \rightarrow 5$ and $6 \rightarrow 4$.



Q10. A specification is given below for a gated latch circuit with two inputs G and D and an output Q . Assume fundamental mode operation.

“Binary information present at the D input is transferred to the Q output when G is equal to 1. The Q output will follow the D input as long as $G = 1$. When G goes to 0, the information that was present at the D input at the time the transition occurred is retained at the Q output.”

Derive the primitive flow table and then obtain the reduced flow table. **(6+6 marks)**

[CSE 205 | L-2/T-1 | 2011–12]

Answer

Design of a Gated D-Latch: Flow Table Derivation

A gated D-Latch operates in the fundamental mode, meaning only one input changes at a time, and the circuit settles into a stable state before the next input transition occurs. The inputs are G (Gate/Enable) and D (Data), and the output is Q .

1. Derivation of the Primitive Flow Table

We define a unique stable state for each valid combination of inputs and output. The state transitions are determined by the latch specifications:

- **Transparent Mode** ($G = 1$): Q follows D .
- **Latch Mode** ($G = 0$): Q retains its previous value.

States where Output $Q = 0$:

- **State 1:** Stable at $GD = 00$. $Q = 0$.
- **State 2:** Stable at $GD = 01$. (Entered from 1 if D changes; Q remains 0 because $G = 0$).
- **State 3:** Stable at $GD = 10$. (Entered from 1 if G changes; Q remains 0 because $D = 0$).
- *Note:* If $GD = 11$, Q must equal 1. Thus, there is no stable state for $Q = 0$ at $GD = 11$.

States where Output $Q = 1$:

- **State 4:** Stable at $GD = 11$. $Q = 1$. (Entered from 2 or 3 if inputs transition to 11).
- **State 5:** Stable at $GD = 01$. (Entered from 4 if G changes $1 \rightarrow 0$; Q retains 1).
- **State 6:** Stable at $GD = 00$. (Entered from 5 if D changes $1 \rightarrow 0$ while $G = 0$; Q retains 1).
- *Note:* If $GD = 10$, Q must equal 0. Thus, there is no stable state for $Q = 1$ at $GD = 10$.

Mapping the single-variable transitions (horizontal/vertical adjacencies in fundamental mode) yields the primitive flow table. Double input changes are physically restricted and marked as “-” (don’t care). Stable states are bolded and enclosed in parentheses.

Present State	Next State for Inputs GD				Output Q
	00	01	11	10	
1	(1)	2	–	3	0
2	1	(2)	4	–	0
3	1	–	4	(3)	0
4	–	5	(4)	3	1
5	6	(5)	4	–	1
6	(6)	5	–	3	1

2. State Minimization (Row Reduction)

To obtain the reduced flow table, we group rows that have the same output and non-conflicting next-state transitions (Compatible States).

Evaluating $Q = 0$ states: $\{1, 2, 3\}$

- Rows 1 and 2: Merge to $(1, 2, 4, 3)$. No conflict.
- Rows 1 and 3: Merge to $(1, 2, 4, 3)$. No conflict.
- Rows 2 and 3: Merge to $(1, 2, 4, 3)$. No conflict.
- The maximal compatible set is $A = \{1, 2, 3\}$.

Evaluating $Q = 1$ states: $\{4, 5, 6\}$

- Rows 4 and 5: Merge to $(6, 5, 4, 3)$. No conflict.
- Rows 4 and 6: Merge to $(6, 5, 4, 3)$. No conflict.
- Rows 5 and 6: Merge to $(6, 5, 4, 3)$. No conflict.
- The maximal compatible set is $B = \{4, 5, 6\}$.

Replacing the primitive states with the combined states A and B , we find the corresponding next-state mappings:

- State A (originally 1, 2, 3): Merged transitions are (A, A, B, A) .
- State B (originally 4, 5, 6): Merged transitions are (B, B, B, A) .

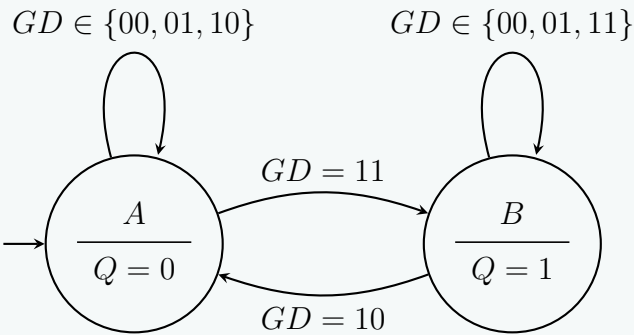
3. Reduced Flow Table

The fully reduced flow table perfectly encapsulates the D-Latch logic: when $G = 0$ (columns 00 and 01), the latch remains in its present state (A stays A , B stays B). When $G = 1$ (columns 11 and 10), the state strictly follows D .

Reduced State	Original States	Next State for Inputs GD				Output Q
		00	01	11	10	
A	$\{1, 2, 3\}$	(A)	(A)	B	(A)	0
B	$\{4, 5, 6\}$	(B)	(B)	(B)	A	1

4. State Transition Diagram

The following diagram illustrates the fundamental mode transitions between the two stable sets.



Excitation Function & Transition Table

Q1. An asynchronous sequential circuit is described by the excitation function $Y = x_1x_2' + (x_1 + x_2')y$ and the output function $z = y$.

- (a) Draw the logic diagram of the circuit.
- (b) Derive the transition table and output map.
- (c) Obtain a two-state flow table.

(12 marks)

[CSE 205 | L-2/T-1 | 2022–23]

Answer

Analysis of the Asynchronous Sequential Circuit

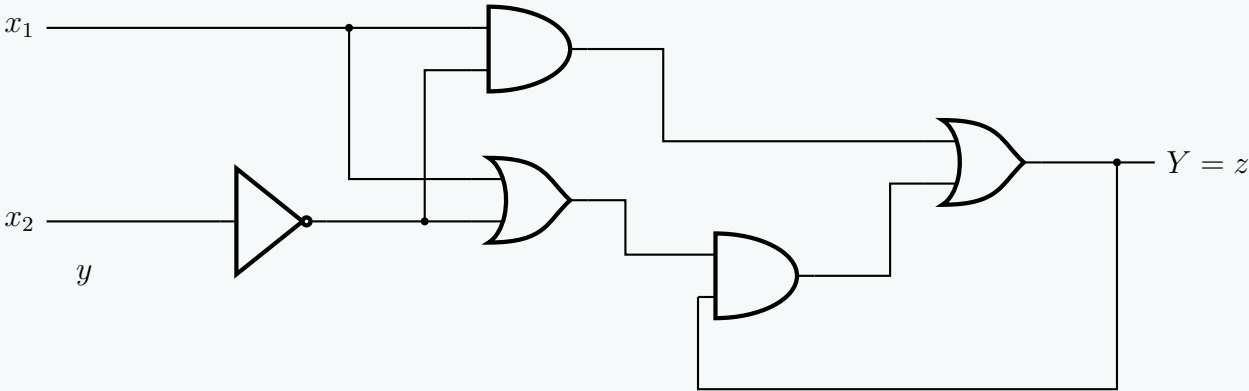
An asynchronous sequential circuit is defined by its excitation function Y and external output function z :

$$Y = x_1x_2' + (x_1 + x_2')y$$
$$z = y$$

where x_1 and x_2 are the external inputs, y is the present-state secondary variable, and Y is the next-state excitation variable.

(a) Logic Diagram of the Circuit

The implementation uses standard logic gates with a feedback path routing the next-state variable Y back into the present-state input y .



(b) Derivation of Transition Table and Output Map

To construct the transition table, we evaluate the next-state function Y for all combinations of the present state y and the input vectors $x_1x_2 \in \{00, 01, 11, 10\}$.

Step-by-Step Evaluation

(a) For Present State $y = 0$:

- $x_1x_2 = 00 \implies Y = 0 \cdot 0' + (0 + 0') \cdot 0 = 0 \cdot 1 + (0 + 1) \cdot 0 = 0 + 0 = 0$
- $x_1x_2 = 01 \implies Y = 0 \cdot 1' + (0 + 1') \cdot 0 = 0 \cdot 0 + (0 + 0) \cdot 0 = 0 + 0 = 0$
- $x_1x_2 = 11 \implies Y = 1 \cdot 1' + (1 + 1') \cdot 0 = 1 \cdot 0 + (1 + 0) \cdot 0 = 0 + 0 = 0$
- $x_1x_2 = 10 \implies Y = 1 \cdot 0' + (1 + 0') \cdot 0 = 1 \cdot 1 + (1 + 1) \cdot 0 = 1 + 0 = 1$

(b) For Present State $y = 1$:

- $x_1x_2 = 00 \implies Y = 0 \cdot 0' + (0 + 0') \cdot 1 = 0 \cdot 1 + (0 + 1) \cdot 1 = 0 + 1 = 1$
- $x_1x_2 = 01 \implies Y = 0 \cdot 1' + (0 + 1') \cdot 1 = 0 \cdot 0 + (0 + 0) \cdot 1 = 0 + 0 = 0$
- $x_1x_2 = 11 \implies Y = 1 \cdot 1' + (1 + 1') \cdot 1 = 1 \cdot 0 + (1 + 0) \cdot 1 = 0 + 1 = 1$
- $x_1x_2 = 10 \implies Y = 1 \cdot 0' + (1 + 0') \cdot 1 = 1 \cdot 1 + (1 + 1) \cdot 1 = 1 + 1 = 1$

Since $z = y$, the output map perfectly mirrors the value of the present state y . Combining these yields the integrated transition table and output map below.

Transition Table and Output Map

Present State	Next State (Y) for Inputs x_1x_2				Output
y	00	01	11	10	z
0	0	0	0	1	0
1	1	0	1	1	1

306

(c) Two-State Flow Table

In fundamental-mode asynchronous design, a flow table assigns symbolic state designations to the rows. Let us define state **A** for $y = 0$ and state **B** for $y = 1$. Stable states occur when the next state equals the present state ($Y = y$) and are circled or enclosed in parentheses.

- For row A ($y = 0$), the next states matching 0 (stable) occur under columns 00, 01, 11.
- For row B ($y = 1$), the next states matching 1 (stable) occur under columns 00, 11, 10.

Two-State Flow Table

Present State	Next State for Inputs x_1x_2				Output z
	00	01	11	10	
A	(A)	(A)	(A)	B	0
B	(B)	A	(B)	(B)	1

Q2. An asynchronous sequential circuit has two internal states and one output. The excitation and output functions are:

$$\begin{aligned} Y_1 &= x_1x_2 + x_1y_2' + x_2'y_1 \\ Y_2 &= x_2 + x_1y_1'y_2 + x_1'y_1 \\ Z &= x_2 + y_1 \end{aligned}$$

Draw the logic diagram of the circuit and derive the transition table. (15 marks) [CSE 205 | L-2/T-1 | 2016–17]

Answer

Part 1: Design of the Gated D Latch

The problem provides the behavioral specification for an asynchronous sequential circuit (a Gated D Latch) operating in fundamental mode. Fundamental mode implies that inputs change one at a time, and the circuit is allowed to settle into a stable state before the next input transition.

Step 1: State Definition and Reduced State Table

Let the current state of the output be y and the next state be Y . The inputs are G (Gate) and D (Data). According to the specification:

- When $G = 1$, the output Y follows D .
- When $G = 0$, the output Y retains its previous value y .

We can represent this behavior directly in a transition/state table. The stable states (where $Y = y$) are enclosed in parentheses.

Current State y	Next State Y for Inputs GD				Output Q
	00	01	11	10	
0	(0)	(0)	1	(0)	0
1	(1)	(1)	(1)	0	1

Step 2: Boolean Equation Derivation

From the state table, we write the sum-of-products expression for the next state Y where the entry is 1:

$$\begin{aligned} Y &= y' \cdot G \cdot D + y \cdot G' \cdot D' + y \cdot G' \cdot D + y \cdot G \cdot D \\ &= GD(y' + y) + yG'(D' + D) \\ &= GD + yG' \end{aligned}$$

(Distributive Law)
(Complement Law: $A + A' = 1$)

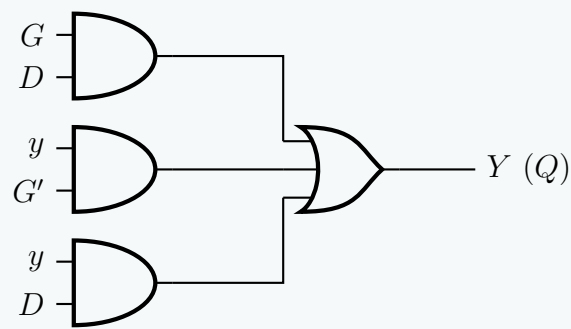
However, in asynchronous design, we must eliminate static hazards to prevent transient false outputs during input transitions. By applying the **Consensus Theorem** to $GD + yG'$, we add the consensus term yD :

$$Y = GD + yG' + yD$$

The added redundant term yD ensures that the transition between $G = 1, D = 1$ and $G = 0, D = 1$ does not cause a momentary glitch to 0.

Step 3: Logic Circuit Diagram

Using the hazard-free equation $Y = GD + yG' + yD$ and setting $Q = Y$, the circuit is realized below. To maintain diagram clarity, signal labels are used at the input pins.



Part 2: Analysis of the Asynchronous Sequential Circuit

We are given the following excitation and output Boolean functions for an asynchronous sequential circuit with inputs x_1, x_2 and internal states y_1, y_2 :

$$Y_1 = x_1x_2 + x_1y_2' + x_2'y_1$$
$$Y_2 = x_2 + x_1y_1'y_2 + x_1'y_1$$
$$Z = x_2 + y_1$$

Step 1: Transition Table Derivation

To derive the transition table, we evaluate Y_1 and Y_2 for all combinations of inputs (x_1x_2) and current states (y_1y_2). For instance, if $y_1y_2 = 10$ and $x_1x_2 = 10$:

$$Y_1 = (1)(0) + (1)(1) + (1)(1) = 0 + 1 + 1 = 1$$
$$Y_2 = 0 + (1)(0)(0) + (0)(1) = 0 + 0 + 0 = 0$$

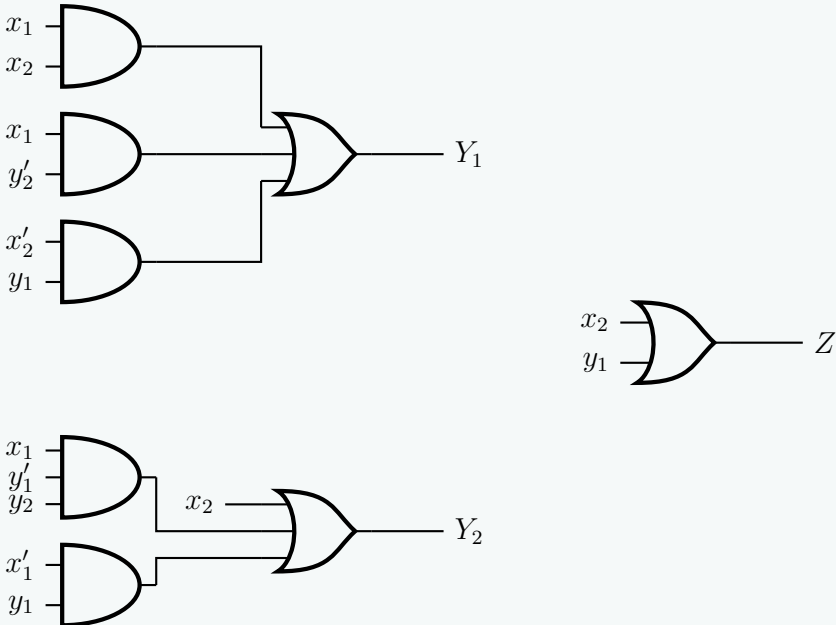
The next state is $Y_1Y_2 = 10$. Since the next state equals the current state, this is a **stable state**. Evaluating all conditions yields the following table, where stable states are highlighted in bold.

Current State y_1y_2	Next State Y_1Y_2 for Inputs x_1x_2			
	00	01	11	10
00	00	01	11	10
01	00	01	11	01
11	11	01	11	10
10	11	01	11	10

Note on Race Conditions: A non-critical race exists when transitioning from state 00 under input 11. The intended next state is 11, meaning both y_1 and y_2 must flip. Whether the circuit temporarily visits 01 or 10 due to unequal propagation delays, both intermediate states direct the circuit to the stable state **11**.

Step 2: Logic Circuit Diagram

To guarantee a clean, professional schematic without an unreadable web of crossing wires, signal net labels are placed directly at the input nodes of the gates representing the sum-of-products formulation of Y_1 , Y_2 , and Z .



Q3. An asynchronous sequential circuit is described by the following excitation function:

$$Y = X_1X_2' + (X_1 + X_2')y$$

Draw the logic diagram of the circuit and derive the transition table. (10 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

Part 1: Design of the Gated D Latch

The problem provides the behavioral specification for an asynchronous sequential circuit (a Gated D Latch) operating in fundamental mode. Fundamental mode implies that inputs change one at a time, and the circuit is allowed to settle into a stable state before the next input transition.

Step 1: State Definition and Reduced State Table

Let the current state of the output be y and the next state be Y . The inputs are G (Gate) and D (Data). According to the specification:

- When $G = 1$, the output Y follows D .
- When $G = 0$, the output Y retains its previous value y .

We can represent this behavior directly in a transition/state table. The stable states (where $Y = y$) are enclosed in parentheses or highlighted in bold.

Current State	Next State Y for Inputs GD				Output
y	00	01	11	10	Q
0	(0)	(0)	1	(0)	0
1	(1)	(1)	(1)	0	1

Step 2: Boolean Equation Derivation

From the state table, we write the sum-of-products expression for the next state Y where the entry is 1:

$$\begin{aligned} Y &= y' \cdot G \cdot D + y \cdot G' \cdot D' + y \cdot G' \cdot D + y \cdot G \cdot D \\ &= GD(y' + y) + yG'(D' + D) && \text{(Distributive Law)} \\ &= GD + yG' && \text{(Complement Law: } A + A' = 1) \end{aligned}$$

In asynchronous design, we must eliminate static hazards to prevent transient false outputs during input transitions. By applying the **Consensus Theorem** to $GD + yG'$, we add the consensus term yD :

$$Y = GD + yG' + yD$$

The added redundant term yD ensures that the transition between $G = 1, D = 1$ and $G = 0, D = 1$ does not cause a momentary glitch.

Step 3: Logic Circuit Diagram

Part 2: Analysis of the Asynchronous Sequential Circuit

An asynchronous sequential circuit is defined by the excitation function:

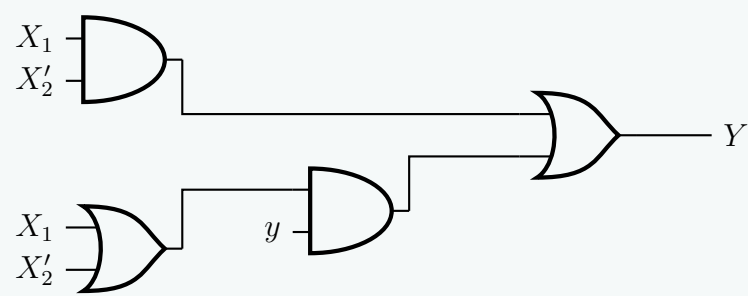
$$Y = X_1X_2' + (X_1 + X_2')y$$

where X_1 and X_2 are the external inputs, and y is the internal state variable representing the current state. Y is the next state.

Step 1: Logic Circuit Diagram

We can directly map the excitation equation $Y = X_1X_2' + (X_1 + X_2')y$ to logic gates. To keep the schematic clean and avoid crossing lines, signal names are placed directly at the gate inputs.

309



Step 2: Transition Table Derivation

To derive the transition table, we evaluate the next state Y for all combinations of the current state $y \in \{0,1\}$ and inputs $X_1X_2 \in \{00,01,11,10\}$. We can simplify the evaluation by expanding the excitation function:

$$Y = X_1X_2' + X_1y + X_2'y$$

(Distributive Law)

Let us evaluate Y column by column for each input condition:

- **For $X_1X_2 = 00$:**
 $Y = (0)(1) + (0)y + (1)y = 0 + 0 + y = y$.
Next state equals current state. Both $y = 0$ and $y = 1$ are stable.
- **For $X_1X_2 = 01$:**
 $Y = (0)(0) + (0)y + (0)y = 0 + 0 + 0 = 0$.
The state will become or remain 0. $y = 0$ is stable; $y = 1$ is unstable.
- **For $X_1X_2 = 11$:**
 $Y = (1)(0) + (1)y + (0)y = 0 + y + 0 = y$.
Next state equals current state. Both $y = 0$ and $y = 1$ are stable.
- **For $X_1X_2 = 10$:**
 $Y = (1)(1) + (1)y + (1)y = 1 + y = 1$.
The state will become or remain 1. $y = 1$ is stable; $y = 0$ is unstable.

Placing these evaluated next states into a table yields the transition table below. Stable states (where $Y = y$) are indicated in **bold** text.

Current State	Next State Y for Inputs X_1X_2			
	00	01	11	10
y				
0	0	0	0	1
1	1	0	1	1

Hazards in Asynchronous Circuits

Q1. What is a static-1 hazard? Give an example of a static-1 hazard in an asynchronous sequential circuit. (5 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

Part 1: Design of the Gated D Latch

The specification describes an asynchronous sequential circuit (a Gated D Latch) operating in fundamental mode. This implies inputs change one at a time, and the circuit settles into a stable state before the next input transition.

Step 1: State Definition and Transition Table

Let the current state of the output be y and the next state be Y . The inputs are G (Gate) and D (Data). According to the text:

- When $G = 1$, the output Y follows D .
- When $G = 0$, the output Y retains its previous value y .

We construct the transition table based on this behavior. Stable states (where $Y = y$) are highlighted in bold text with parentheses.

Current State	Next State Y for Inputs GD				Output
	00	01	11	10	
y					Q
0	(0)	(0)	1	(0)	0
1	(1)	(1)	(1)	0	1

Step 2: Boolean Equation Derivation

From the state table, we extract the sum-of-products expression for the next state Y where the entry is 1:

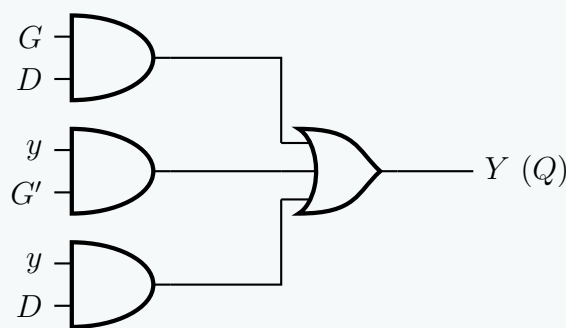
$$\begin{aligned} Y &= y' \cdot G \cdot D + y \cdot G' \cdot D' + y \cdot G' \cdot D + y \cdot G \cdot D \\ &= GD(y' + y) + yG'(D' + D) && \text{(Distributive Law)} \\ &= GD + yG' && \text{(Complement Law: } A + A' = 1) \end{aligned}$$

In asynchronous circuit design, we must eliminate static hazards to prevent transient false outputs (glitches) during input transitions. We apply the **Consensus Theorem** to $GD + yG'$ to add the consensus term yD :

$$Y = GD + yG' + yD$$

The added redundant term yD ensures that the transition between the stable state ($G = 1, D = 1, y = 1$) and ($G = 0, D = 1, y = 1$) does not cause a momentary logic-0 glitch.

Step 3: Logic Circuit Diagram



Part 2: Static-1 Hazard

Definition

A **static-1 hazard** occurs in a combinational or asynchronous sequential logic circuit when an output is intended to remain at logic '1' during an input transition, but due to unequal propagation delays along different signal paths, the output momentarily drops to logic '0' (creating a glitch) before settling back to '1'.

In asynchronous sequential circuits, these temporary glitches are critical because they can act as false clock edges or incorrect excitation signals, inadvertently driving the circuit into an incorrect stable state.

Example of a Static-1 Hazard

Consider a simple logic circuit governed by the Boolean function:

$$F(A, B, C) = A \cdot B + A' \cdot C$$

Assume the current inputs are $A = 1, B = 1, C = 1$. Evaluating the function gives $F = (1)(1) + (0)(1) = 1$.

Now, let input A transition from $1 \rightarrow 0$, while B and C remain 1. The new steady-state output should be $F = (0)(1) + (1)(1) = 1$. The output is supposed to remain completely unchanged at 1.

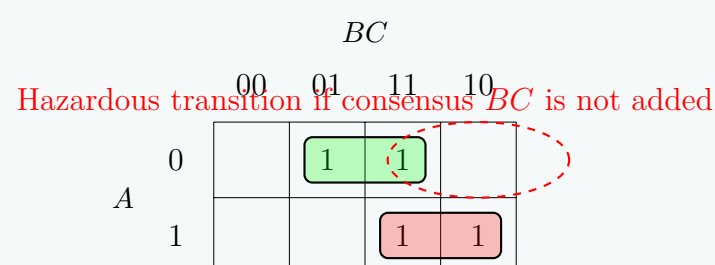
However, physical logic gates have propagation delays. The inverter generating A' requires a small amount of time (Δt) to switch from 0 to 1 after A drops to 0. During this Δt interval:

- A is 0, causing the $A \cdot B$ AND-gate to output 0.
- A' has not yet transitioned to 1 (it is temporarily still 0), causing the $A' \cdot C$ AND-gate to also output 0.
- As a result, the OR-gate sees both inputs as 0, and the output F drops to 0 momentarily.

Visualizing the Hazard

1. Karnaugh Map Representation:

A static-1 hazard exists when two adjacent 1s in a Karnaugh map are not covered by the same grouping (implicant). The transition between m_7 (111) and m_3 (011) jumps between two separate groups, exposing the circuit to a hazard.



2. Logic Circuit Diagram with Hazard:

3. Timing Diagram showing the Glitch:

Below is the timing diagram illustrating the momentary logic-0 glitch caused by the delayed response of A' .

To eliminate this static-1 hazard, the **consensus term** $B \cdot C$ must be added to the Boolean expression, resulting in the hazard-free equation: $F = A \cdot B + A' \cdot C + B \cdot C$. This term overlaps the two groups in the K-map, ensuring the output remains 1 during the transition.

Serial Parity Generation

Q1. Design a serial parity generation circuit. The circuit receives a sequence of bits and determines whether the sequence contains an even or odd number of ones. The circuit output p should be 0 for even parity and 1 for odd parity. **(10 marks)** [CSE 205 | L-2/T-1 | 2020–21, 2018–19]

Answer

Part 1: Design of the Gated D Latch

The problem provides the behavioral specification for an asynchronous sequential circuit (a Gated D Latch) operating in fundamental mode. Fundamental mode implies that inputs change one at a time, and the circuit is allowed to settle into a stable state before the next input transition.

Step 1: State Definition and Reduced State Table

Let the current state of the output be y and the next state be Y . The inputs are G (Gate) and D (Data). According to the specification:

- When $G = 1$, the output Y follows D .
- When $G = 0$, the output Y retains its previous value y .

We can represent this behavior directly in a transition/state table. The stable states (where $Y = y$) are enclosed in parentheses and highlighted in bold.

Current State	Next State Y for Inputs GD				Output
y	00	01	11	10	Q
0	(0)	(0)	1	(0)	0
1	(1)	(1)	(1)	0	1

Step 2: Boolean Equation Derivation

From the state table, we write the sum-of-products expression for the next state Y where the entry is 1:

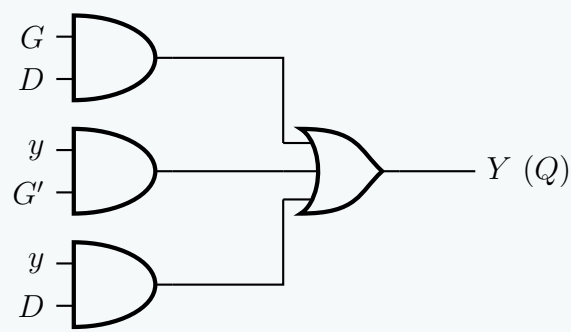
$$\begin{aligned} Y &= y' \cdot G \cdot D + y \cdot G' \cdot D' + y \cdot G' \cdot D + y \cdot G \cdot D \\ &= GD(y' + y) + yG'(D' + D) && \text{(Distributive Law)} \\ &= GD + yG' && \text{(Complement Law: } A + A' = 1) \end{aligned}$$

In asynchronous design, we must eliminate static hazards to prevent transient false outputs during input transitions. By applying the **Consensus Theorem** to $GD + yG'$, we add the consensus term yD :

$$Y = GD + yG' + yD$$

The added redundant term yD ensures that the transition between $G = 1, D = 1$ and $G = 0, D = 1$ does not cause a momentary logic-0 glitch.

Step 3: Logic Circuit Diagram



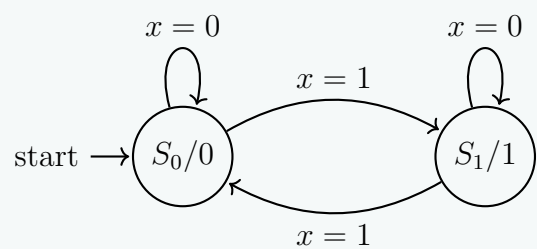
Part 2: Design of a Serial Parity Generation Circuit

The serial parity generator receives a stream of bits (x) synchronously and outputs $p = 0$ if the total number of received ‘1’s is even, and $p = 1$ if the total number of received ‘1’s is odd. This is a classic synchronous sequential circuit design using a Moore finite state machine (FSM).

Step 1: State Diagram

Let S_0 be the state where an **even** number of 1s has been received (Output $p = 0$).
Let S_1 be the state where an **odd** number of 1s has been received (Output $p = 1$).

- If the input $x = 0$, the parity does not change (stay in current state).
- If the input $x = 1$, the parity flips (transition to the other state).



Step 2: State Table and State Assignment

We assign binary values to the states: $S_0 = 0$ and $S_1 = 1$. Let the state variable be Q .

Current State (Q)	Input (x)	Next State (Q^+)	Output (p)
0	0	0	0
0	1	1	0
1	0	1	1
1	1	0	1

Note: As a Moore machine, the output p strictly matches the current state Q , hence $p = Q$.

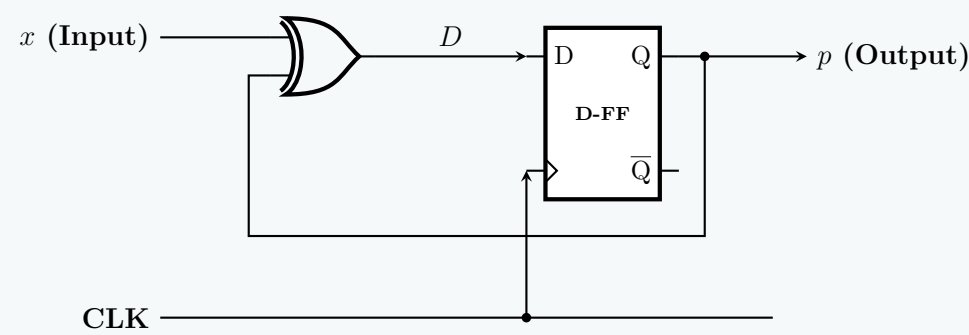
Step 3: Excitation Equations

Using a D Flip-Flop, the required input D is equal to the next state Q^+ . From the state table, we derive the Boolean expression for D :

$$\begin{aligned} D &= \sum m(1, 2) \\ D &= Q'x + Qx' \\ D &= Q \oplus x \end{aligned}$$

The output equation is trivially $p = Q$.

Step 4: Logic Circuit Diagram



Q2. For the following primitive flow table shown in Figure , answer the following questions:

- Find all compatible pairs by means of an implication table.
- Find the maximal compatibles by means of a merger diagram.
- Find a minimal set of compatibles that covers all the states and is closed.

	00	01	11	10
a	$\textcircled{a}, 0$	$b, -$	$-, -$	$e, -$
b	$a, -$	$\textcircled{b}, 0$	$c, -$	$-, -$
c	$-, -$	$d, -$	$\textcircled{c}, 0$	$h, -$
d	$a, -$	$\textcircled{d}, 1$	$-, -$	$-, -$
e	$a, -$	$-, -$	$f, -$	$\textcircled{e}, 0$
f	$-, -$	$g, -$	$\textcircled{f}, 0$	$h, -$
g	$a, -$	$\textcircled{g}, 0$	$-, -$	$-, -$
h	$a, -$	$-, -$	$-, -$	$\textcircled{h}, 0$

(10+5+5 marks)

[CSE 205 | L-2/T-1 | 2014–15]

Answer

State Minimization of the Primitive Flow Table

To minimize the fundamental-mode asynchronous sequential circuit, we first define the sets of compatible states, map their relations, and then derive a minimal closed cover.

(i) Finding Compatible Pairs (Implication Table)

Two states can be compatible only if they have identical specified outputs. From the primitive flow table, we extract the outputs associated with each state:

- States with Output 0:** $\{a, b, c, e, f, g, h\}$
- States with Output 1:** $\{d\}$ (specified under input 01)

Because state d has a conflicting output, it is **incompatible** with all other states. We immediately place an “X” in all cells involving d .

For the remaining states (which all output 0), we populate the implication table with their required next-state pairs. A pair is *unconditionally compatible* ($\checkmark$) if all specified next states match or are don’t-cares.

Initial Implied Pairs:

- $(a, c) \implies (b, d), (e, h)$
- $(a, f) \implies (b, g), (e, h)$
- $(a, g) \implies (b, g)$
- $(a, h) \implies (e, h)$
- $(b, c) \implies (b, d)$
- $(b, e) \implies (c, f)$
- $(b, f) \implies (b, g), (c, f)$
- $(c, e) \implies (c, f), (e, h)$
- $(c, f) \implies (d, g)$
- $(c, g) \implies (d, g)$
- $(e, f) \implies (e, h)$

All other pairs evaluating to identical next states or don’t-cares are marked $\checkmark$.

Cascading Eliminations:

- Since d is incompatible with everything, any pair implying a pair with d is incompatible. Thus, (a, c) , (b, c) , (c, f) , and (c, g) are marked **X**.
- Because (c, f) is now **X**, any pair implying (c, f) is also incompatible. Thus, (b, e) , (b, f) , and (c, e) are marked **X**.
- Pairs implying only unconditionally compatible pairs become compatible. For example, (a, f) implies (b, g) and (e, h) , both of which are unconditionally compatible. Thus, (a, f) becomes $\checkmark$.

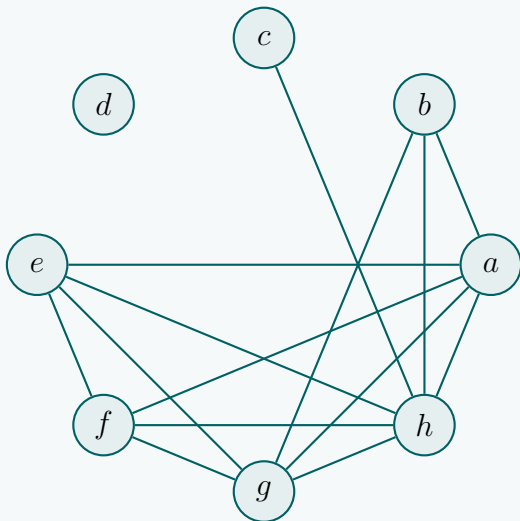
Final Implication Table:

b	✓						
c	X	X					
d	X	X	X				
e	✓	X	X	X			
f	✓	X	X	X	✓		
g	✓	✓	X	X	✓	✓	
h	✓	✓	✓	X	✓	✓	✓
	a	b	c	d	e	f	g

The set of compatible pairs is: $(a, b), (a, e), (a, f), (a, g), (a, h), (b, g), (b, h), (c, h), (e, f), (e, g), (e, h), (f, g), (f, h), (g, h)$.

(ii) Maximal Compatibles (Merger Diagram)

To find the maximal compatibles, we draw a merger diagram. Each state is a node, and edges connect compatible pairs found in the implication table.



Maximal compatibles are the largest fully connected subgraphs (cliques) in the merger diagram:

- States $\{a, e, f, g, h\}$ are mutually connected, forming a 5-node clique.
- States $\{a, b, g, h\}$ are mutually connected, forming a 4-node clique.
- States $\{c, h\}$ form a 2-node clique.
- State $\{d\}$ is completely isolated.

Maximal Compatibles: $\mathcal{M} = \left\{ \{a, e, f, g, h\}, \{a, b, g, h\}, \{c, h\}, \{d\} \right\}$

(iii) Minimal Closed Cover

A valid minimal set of compatibles must satisfy two conditions:

- (a) **Covering:** All 8 original states must appear in at least one selected compatible group.
- (b) **Closure:** For any selected group, the set of next states for any specific input must be completely contained within *at least one* single group in the chosen cover.

Let us evaluate the set of maximal compatibles as our candidate cover. Let:

$$\begin{aligned} M_1 &= \{a, e, f, g, h\} \\ M_2 &= \{a, b, g, h\} \\ M_3 &= \{c, h\} \\ M_4 &= \{d\} \end{aligned}$$

Covering Requirement: The union $M_1 \cup M_2 \cup M_3 \cup M_4$ clearly covers $\{a, b, c, d, e, f, g, h\}$. Since states d , c , and b do not share a common compatible group, we mathematically cannot use fewer than 4 states.

Closure Check Table:

Compatible Group	Next States for Inputs x_1x_2				Closure Satisfied?
	00	01	11	10	
$M_1 = \{a, e, f, g, h\}$	$\{a\} \subseteq M_1$	$\{b, g\} \subseteq M_2$	$\{f\} \subseteq M_1$	$\{e, h\} \subseteq M_1$	Yes
$M_2 = \{a, b, g, h\}$	$\{a\} \subseteq M_1$	$\{b, g\} \subseteq M_2$	$\{c\} \subseteq M_3$	$\{e, h\} \subseteq M_1$	Yes
$M_3 = \{c, h\}$	$\{a\} \subseteq M_1$	$\{d\} \subseteq M_4$	$\{c\} \subseteq M_3$	$\{h\} \subseteq M_3$	Yes
$M_4 = \{d\}$	$\{a\} \subseteq M_1$	$\{d\} \subseteq M_4$	$\emptyset$	$\emptyset$	Yes

Because the implied next states for every group under every input column map to a subset that is fully contained within one of our defined groups, the closure condition is strictly satisfied.

Final Minimal Closed Cover:

$$\text{Cover} = \left\{ \{a, e, f, g, h\}, \{a, b, g, h\}, \{c, h\}, \{d\} \right\}$$

Race Around Condition

Q1. What is a race in fundamental mode circuits? Explain critical and non-critical races with examples. (12 marks) [CSE 205 | L-2/T-1 | 2022–23]

Answer

Race Conditions in Fundamental Mode Circuits

In fundamental mode asynchronous sequential circuits, it is assumed that input variables change one at a time and that the circuit must reach a stable state before the next input change occurs. Because there is no global clock to synchronize transitions, the next state is determined by the propagation delays of the logic gates and feedback paths.

A **race condition** occurs when a single change in an input variable dictates that **two or more present-state variables must change state simultaneously**. Because physical logic gates and wires have unequal, unpredictable propagation delays, the state variables will never change at exactly the same instant. As a result, the circuit will temporarily pass through intermediate transient states before settling.

Depending on whether these unequal delays affect the final outcome, races are classified as either **non-critical** or **critical**.

1. Non-Critical Races

A race is **non-critical** if the final stable state that the circuit reaches is the same regardless of the order in which the internal state variables change. Since the final outcome is deterministic and correct, non-critical races are harmless (though they may briefly produce transient false outputs, or “glitches”).

Example of a Non-Critical Race

Consider a portion of a transition table where the present state variables are y_1y_2 and the next state variables are Y_1Y_2 . Assume the circuit is currently in the stable state 00 and an input change commands it to transition to the stable state 11.

Transition Table exhibiting a Non-Critical Race

Present State (y_1y_2)	Next State (Y_1Y_2)
00	11
01	11
10	11
11	(11)

Analysis: When the state tries to change from 00 $\rightarrow$ 11, both y_1 and y_2 must flip.

- Ideal case:** Both variables change at the exact same time (00 $\rightarrow$ 11), reaching the final stable state immediately.
- y_2 is faster:** The circuit enters the transient state 01. Looking at the table, the next state for 01 is 11. The circuit correctly proceeds to (11).
- y_1 is faster:** The circuit enters the transient state 10. The next state for 10 is 11. The circuit correctly proceeds to (11).

Because all possible physical propagation paths lead to the intended stable state (11), this is a non-critical race.

2. Critical Races

A race is **critical** if the final stable state the circuit reaches depends directly on the order in which the state variables change. Because propagation delays vary with temperature, voltage, and manufacturing tolerances, the final state is entirely unpredictable. This constitutes a severe design flaw and must be strictly avoided through proper state assignment.

Example of a Critical Race

Let us modify the transition table to create a scenario where intermediate states form traps (unintended stable states).

Transition Table exhibiting a Critical Race

Present State (y_1y_2)	Next State (Y_1Y_2)
00	11
01	(01)
10	(10)
11	(11)

Analysis: Again, the circuit is at 00 and an input change commands a transition to 11. Both variables are instructed to change.

- **Ideal case:** If y_1 and y_2 change exactly synchronously, the circuit goes $00 \rightarrow 11$ and settles in **(11)**.
- **y_2 is faster:** The circuit temporarily enters the state 01. However, the transition table dictates that 01 is a stable state ($Y_1Y_2 = 01$). The circuit will stay trapped in **(01)** and never reach 11.
- **y_1 is faster:** The circuit temporarily enters the state 10. Similarly, 10 is defined as a stable state. The circuit will remain trapped in **(10)**.

Because slight variations in gate delay cause the circuit to settle into completely different, unpredictable stable states (**(01)**, **(10)**, or **(11)**), this is a critical race.

To resolve critical races, engineers utilize *adjacent state assignment* techniques (like Gray codes) or introduce intermediate unstable states (cycles) so that only one state variable changes at a time.

Q2. For the given reduced flow table, find a valid assignment without any critical race and complete the modified flow table.

	00	01	11	10
A	(A),0	D,0	B,-	(A),0
B	A,-	-	C,-	-
C	A,-	(C),1	(C),1	(C),1
D	A,0	(D),0	(D),0	C,-

(10 marks)

[CSE 205 | L-2/T-1 | 2021–22]

Answer

State Assignment and Critical Race Avoidance

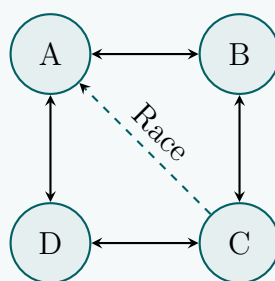
To find a valid state assignment without critical races for a fundamental mode asynchronous circuit, we must first determine the required adjacencies between states. A critical race occurs when two or more state variables must change simultaneously to transition between states. If they change at different speeds, the circuit may land in an unintended stable state.

1. Adjacency Analysis

From the given reduced flow table, we analyze the inter-state transitions (ignoring self-loops):

- **Row A:** Transitions to D (under 01) and B (under 11). $\Rightarrow A$ must be adjacent to D and B .
- **Row B:** Transitions to A (under 00) and C (under 11). $\Rightarrow B$ must be adjacent to A and C .
- **Row C:** Transition to A (under 00). $\Rightarrow C$ must be adjacent to A .
- **Row D:** Transitions to A (under 00) and C (under 10). $\Rightarrow D$ must be adjacent to A and C .

The necessary adjacency pairs are: (A, B) , (A, D) , (B, C) , (C, D) , and (C, A) .



2. Two-Variable State Assignment

A two-variable Karnaugh map allows each state to have at most two adjacent neighbors. However, state A requires adjacency with B , D , and C . It is mathematically impossible to make one cell adjacent to three others in a 4-cell map.

To map these four states, we place them in a standard loop configuration: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$. Let the state variables be y_1y_2 :

$$A = 00$$

$$B = 10$$

$$C = 11$$

$$D = 01$$

This assignment guarantees that (A, B) , (A, D) , (B, C) , and (C, D) are all separated by a distance of 1 (a single bit change).

3. Resolving the Critical Race via a Shared Cycle

The transition $C(11) \rightarrow A(00)$ under input 00 requires changing both y_1 and y_2 simultaneously, which constitutes a **critical race**. To resolve this without adding extra state variables, we route the transition through an intermediate unstable state (a *cycle*).

Looking at input column 00:

- The stable destination is A .
- State $D(01)$ under input 00 dictates a transition to $A(00)$.

- Therefore, instead of transitioning C directly to A , we can transition $C \rightarrow D \rightarrow A$.
- $C(11) \rightarrow D(01)$ changes only y_1 .
- $D(01) \rightarrow A(00)$ changes only y_2 .

We implement this by changing the next state of C under input 00 from A to D . The circuit will safely cycle through D before settling at A .

4. Modified Flow Table and Transition Table

Below is the modified flow table where the critical race is resolved. The altered entry is highlighted.

State	Assignment (y_1y_2)	00	01	11	10
A	00	(A),0	D,0	B,-	(A),0
B	10	A,-	-, -	C,-	-, -
C	11	D,-	(C),1	(C),1	(C),1
D	01	A,0	(D),0	(D),0	C,-

Substituting the binary assignments yields the final race-free **Transition Table** (Y_1Y_2, z):

y_1y_2	Next State & Output (Y_1Y_2, z)			
	00	01	11	10
00	(00) ,0	01,0	10,-	(00) ,0
10	00,-	-, -	11,-	-, -
11	01,-	(11) ,1	(11) ,1	(11) ,1
01	00,0	(01) ,0	(01) ,0	11,-

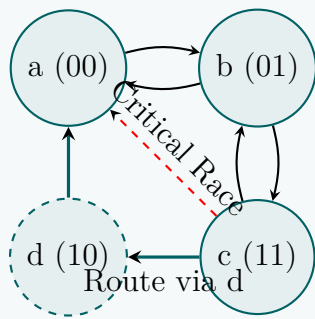
Q3. For the given reduced flow table, find a valid assignment without any critical race and complete the modified flow table.

x_1x_2	00	01	11	10
a	Ⓐ /0	b/-	Ⓐ /1	b/-
b	a/-	Ⓑ /0	c/-	Ⓑ /0
c	a/-	Ⓒ /1	Ⓒ /0	b/-

(10 marks)

[CSE 205 | L-2/T-1 | 2020–21]

\begin{aligned} a &= 00 \\ b &= 01 \\ c &= 11 \\ d &= 10 \quad (\text{Intermediate state}) \end{aligned}



3. Resolving the Critical Race

Let us examine the transition $c \rightarrow a$ under input $x_1x_2 = 00$.

- Going directly from $c(11)$ to $a(00)$ requires both y_1 and y_2 to change. This is a **critical race**.
- To avoid this, we modify the transition to route through the adjacent unassigned state $d(10)$.
- Instead of $c \xrightarrow{00} a$, we define a cycle: $c(11) \xrightarrow{00} d(10) \xrightarrow{00} a(00)$.
- We change the next-state entry for state c under column 00 to d .
- We define the next-state entry for state d under column 00 to be a . The output during this transient cycle remains don't-care (-).

4. Modified Flow Table

Below is the race-free modified flow table. The newly introduced state d and the modified cycle paths are highlighted in color. Since d is only ever reached under the 00 input condition, its next states under inputs 01, 11, and 10 are unreachable and left as don't-cares (-).

State	Assignment y_1y_2	Inputs x_1x_2			
		00	01	11	10
a	00	Ⓐ/0	b/-	Ⓐ/1	b/-
b	01	a/-	Ⓑ/0	c/-	Ⓑ/0
c	11	Ⓓ/-	Ⓒ/1	Ⓒ/0	b/-
d	10	Ⓐ/-	-/-	-/-	-/-

By utilizing state d , the circuit safely cycles through intermediate states without any variables racing, guaranteeing deterministic fundamental-mode operation.

Q4. For the given reduced flow table, find a valid assignment without any critical race and complete the modified flow table.

$x_1x_2 \backslash$	00	01	11	10
a	Ⓐ/0	d/0	Ⓐ/1	c/0
b	Ⓑ/0	c/-	Ⓑ/0	d/-
c	b/0	Ⓒ/1	a/1	Ⓒ/0
d	a/0	Ⓓ/0	b/0	Ⓓ/1

(8 marks)

[CSE 205 | L-2/T-1 | 2018–19]

Answer

State Assignment and Critical Race Avoidance

In a fundamental mode asynchronous sequential circuit, to prevent **critical races**, any state transition must involve the change of only a **single state variable** at a time. If multiple state variables change simultaneously, unequal propagation delays in the logic gates can cause the circuit to pass through unpredictable transient states, potentially trapping it in an incorrect stable state.

1. Transition Adjacency Analysis

To assign state variables without critical races, we first determine the required adjacencies by analyzing all state transitions in the reduced flow table. Transitions occur when the next state differs from the present state under a specific input condition:

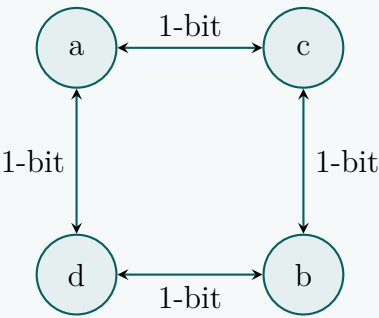
- **Transitions from State a :** Under input 01, $a \rightarrow d$. Under input 10, $a \rightarrow c$.
 $\Rightarrow a$ must be adjacent to d and c .
- **Transitions from State b :** Under input 01, $b \rightarrow c$. Under input 10, $b \rightarrow d$.
 $\Rightarrow b$ must be adjacent to c and d .
- **Transitions from State c :** Under input 00, $c \rightarrow b$. Under input 11, $c \rightarrow a$.
 $\Rightarrow c$ must be adjacent to b and a .

- **Transitions from State d :** Under input 00, $d \rightarrow a$. Under input 11, $d \rightarrow b$.
 $\implies d$ must be adjacent to a and b .

The required adjacency pairs are: (a, c) , (a, d) , (b, c) , and (b, d) . Notice that states a and b do not transition to each other, nor do states c and d .

2. State Adjacency Graph

We construct a state adjacency graph where edges represent required Hamming distances of 1. The requirements form a bipartite graph (a square ring), which maps perfectly onto a two-variable Karnaugh map structure.



3. Race-Free State Assignment

Because the state graph is a closed square, we can assign two variables (y_1y_2) in a Gray code sequence to satisfy all required 1-bit distances directly. No shared cycles or intermediate unstable states are needed. Let us assign the variables y_1y_2 as follows:

$a = 00$

$c = 01$

$b = 11$

$d = 10$

Verification of Hamming Distances (D):

- $D(a, c) = D(00, 01) = 1$ (Valid)
- $D(a, d) = D(00, 10) = 1$ (Valid)
- $D(b, c) = D(11, 01) = 1$ (Valid)
- $D(b, d) = D(11, 10) = 1$ (Valid)

Every inter-state transition requires only one bit to flip. Therefore, **this assignment contains no critical races**.

4. Modified Flow Table (Transition Table)

Substituting the symbolic states a, b, c, d with their respective binary assignments y_1y_2 yields the completed transition table. The rows are reordered to follow the 00, 01, 11, 10 Gray code sequence for standard mapping. Stable states are indicated by parentheses and bold text.

State	Assignment y_1y_2	Next State & Output (Y_1Y_2/z) for inputs x_1x_2			
		00	01	11	10
a	00	(00)/0	10/0	(00)/1	01/0
c	01	11/0	(01)/1	00/1	(01)/0
b	11	(11)/0	01/-	(11)/0	10/-
d	10	00/0	(10)/0	11/0	(10)/1

Q5. For the given reduced flow table, find a valid assignment without any critical race and complete the modified flow table.

$x_1x_2 \backslash$	00	01	11	10
a	(a) /0	d/0	(a) /1	c/0
b	(b) /0	c/-	(b) /0	d/-
c	b/0	(c) /1	a/1	(c) /0
d	a/0	(d) /0	b/0	(d) /1

(10 marks)

Answer

State Assignment and Critical Race Avoidance

In fundamental mode asynchronous sequential circuits, transitions between states must be meticulously managed. To prevent a **critical race**—a scenario where multiple state variables change simultaneously, causing the circuit to potentially settle in an incorrect stable state due to unequal gate propagation delays—each state transition must involve the change of only a **single state variable**.

1. Transition Adjacency Analysis

To determine a valid race-free state assignment, we analyze the reduced flow table to extract all required inter-state transitions. Two states must be logically adjacent (separated by a Hamming distance of 1) if a direct transition exists between them.

Analyzing the rows of the flow table (ignoring stable state self-loops):

- **Transitions from State a :**

$$\begin{aligned} a &\xrightarrow{x_1x_2=01} d \\ a &\xrightarrow{x_1x_2=10} c \end{aligned}$$

$\Rightarrow a$ requires adjacency to both d and c .

- **Transitions from State b :**

$$\begin{aligned} b &\xrightarrow{x_1x_2=01} c \\ b &\xrightarrow{x_1x_2=10} d \end{aligned}$$

$\Rightarrow b$ requires adjacency to both c and d .

- **Transitions from State c :**

$$\begin{aligned} c &\xrightarrow{x_1x_2=00} b \\ c &\xrightarrow{x_1x_2=11} a \end{aligned}$$

$\Rightarrow c$ requires adjacency to both b and a .

- **Transitions from State d :**

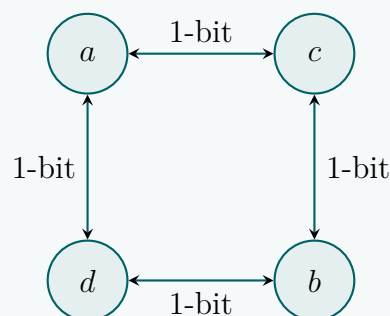
$$\begin{aligned} d &\xrightarrow{x_1x_2=00} a \\ d &\xrightarrow{x_1x_2=11} b \end{aligned}$$

$\Rightarrow d$ requires adjacency to both a and b .

The complete set of necessary adjacency pairs is: $\{(a, c), (a, d), (b, c), (b, d)\}$. Note that states a and b do not transition to each other, nor do states c and d .

2. State Adjacency Graph

We can represent these requirements visually using a transition graph. The resulting graph forms a closed loop of four nodes without cross-diagonal connections. This structure is a perfect bipartite graph (a square), which naturally maps onto a 2-variable Karnaugh map without requiring the introduction of any intermediate unstable states (shared cycles).



3. Race-Free State Assignment

Given the square topology, we assign a standard 2-bit Gray code to the state variables y_1y_2 to satisfy all required 1-bit distances directly.

Let the assignment be:

$$\begin{aligned} a &= 00 \\ c &= 01 \\ b &= 11 \\ d &= 10 \end{aligned}$$

Verification of Hamming Distances (D):

- $D(a, c) = D(00, 01) = 1$ (**Valid**)

- $D(a, d) = D(00, 10) = 1$ (Valid)
- $D(b, c) = D(11, 01) = 1$ (Valid)
- $D(b, d) = D(11, 10) = 1$ (Valid)

Since every requisite transition strictly involves exactly one bit flipping, **this assignment contains no critical races.**

4. Modified Flow Table (Transition Table)

Substituting the symbolic states a, b, c, d with their corresponding binary assignments y_1y_2 yields the completed transition table. The rows are ordered in the standard Gray code sequence (00,01,11,10) to clearly demonstrate logical adjacencies. Stable states are indicated by bold parentheses.

State	Assignment y_1y_2	Next State & Output (Y_1Y_2/z) for inputs x_1x_2			
		00	01	11	10
a	00	(00)/0	10/0	(00)/1	01/0
c	01	11/0	(01)/1	00/1	(01)/0
b	11	(11)/0	01/-	(11)/0	10/-
d	10	00/0	(10)/0	11/0	(10)/1

Q6. What is race condition? Explain different types of races with examples. (10 marks)

[CSE 205 | L-2/T-1 | 2016–17]

Answer

Race Conditions in Asynchronous Sequential Circuits

In fundamental mode asynchronous sequential circuits, a **race condition** occurs when a single input change causes two or more state variables to change their values simultaneously. Because logic gates and routing paths have inherent and unequal propagation delays, the state variables will almost never change at the exact same microsecond. As a result, the circuit will pass through unintended, intermediate transient states before settling. Depending on the next-state logic of these intermediate states, races are classified into two main types: **Non-Critical Races** and **Critical Races**.

1. Non-Critical Race

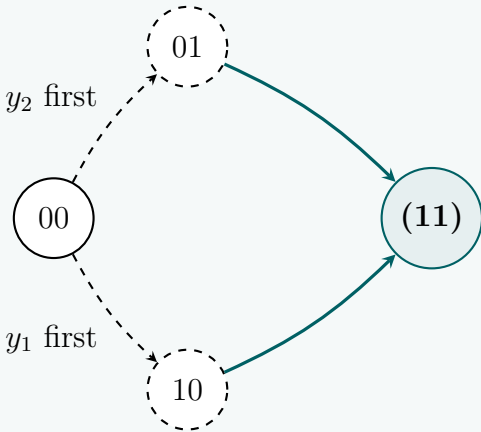
A race is **non-critical** if the unequal gate delays do not affect the final stable state of the circuit. Regardless of the order in which the state variables change, the circuit is guided through transient states to the correct, intended stable state.

Example: Consider a transition from state $y_1y_2 = 00$ to a stable state 11 under input $x = 1$. The state variables y_1 and y_2 must both change.

- If y_2 changes first, the circuit enters transient state 01.
- If y_1 changes first, the circuit enters transient state 10.

In a non-critical race, the flow table is designed such that both 01 and 10 point to 11 as their next state under $x = 1$.

Present State y_1y_2	Next State Y_1Y_2	
	$x = 0$	$x = 1$
00	(00)	11
01	–	11
10	–	11
11	–	(11)

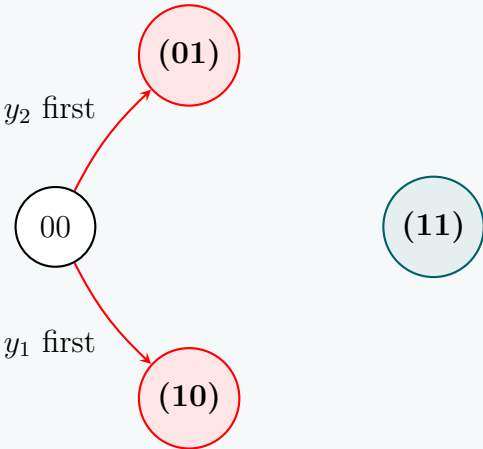


Conclusion: The final destination is always (11). The race is safe.

2. Critical Race

A race is **critical** if the final stable state of the circuit depends on the order in which the state variables change. This is a fatal design flaw, as unpredictable propagation delays will cause the circuit to settle in an incorrect, erroneous state.
Example: Consider the same intended transition from $y_1y_2 = 00$ to 11 under input $x = 1$. However, this time, the intermediate states 01 and 10 happen to be stable states themselves under the same input column.

Present State y_1y_2	Next State Y_1Y_2	
	$x = 0$	$x = 1$
00	(00)	11
01	–	(01)
10	–	(10)
11	–	(11)



- If y_2 changes faster than y_1 , the circuit reaches state 01. Since 01 is stable under $x = 1$, it becomes trapped there.
- If y_1 changes faster than y_2 , the circuit reaches state 10. Since 10 is stable under $x = 1$, it becomes trapped there.

Conclusion: The circuit never reaches the intended state (11). The outcome is entirely dependent on hardware delays, making it a critical race.

Resolution

To resolve critical races, engineers must perform **proper state assignment**. This involves mapping states such that every required transition has a Hamming distance of exactly 1 (only one bit changes). If this is mathematically impossible in a standard grid, **shared rows** (cycles) or **unassigned intermediate states** must be utilized to safely route the transitions bit-by-bit.

Q7. Investigate the transition table (state variables y_1y_2 , inputs x_1x_2) and determine all race conditions and whether they are critical or noncritical. Determine also whether there are any cycles.

y_1y_2	x_1x_2			
	00	01	11	10
00	10	(00)	11	10
01	(01)	00	10	10
11	01	00	(11)	(11)
10	11	00	(10)	(10)

(10 marks)

[CSE 205 | L-2/T-1 | 2014–15]

Answer

Investigation of Race Conditions and Cycles

In a fundamental mode asynchronous sequential circuit, transitions between states are driven by input changes.

- A **race condition** occurs when a single input change causes two or more state variables to change simultaneously (a Hamming distance > 1).
- A race is **noncritical** if all possible intermediate paths caused by unequal gate delays eventually lead to the same intended stable state.
- A race is **critical** if the unequal delays can cause the circuit to settle in different, unintended stable states.
- A **cycle** occurs when a circuit passes through one or more intermediate unstable states before finally reaching a stable state

(e.g., $S_1 \rightarrow S_2 \rightarrow S_3$).

Let us analyze the provided transition table. Stable states are denoted by bold parentheses ().

Present State y_1y_2	Next State Y_1Y_2 for inputs x_1x_2			
	00	01	11	10
00	10	(00)	11	10
01	(01)	00	10	10
11	01	00	(11)	(11)
10	11	00	(10)	(10)

1. Analysis of Race Conditions

To find races, we inspect every unstable state to see if the transition to the next state requires more than one bit to change simultaneously.

Under Input $x_1x_2 = 01$

- **Transition $11 \rightarrow 00$:** Both y_1 and y_2 change ($1 \rightarrow 0$). **Race Condition!**
 - If y_1 changes first: State becomes 01. In column 01, the next state for 01 is 00. Path: $11 \rightarrow 01 \rightarrow (00)$.
 - If y_2 changes first: State becomes 10. In column 01, the next state for 10 is 00. Path: $11 \rightarrow 10 \rightarrow (00)$.
 - If both change simultaneously: Path: $11 \rightarrow (00)$.

Conclusion: Since all possible transition paths converge to the same stable state (00), this is a Noncritical Race.

Under Input $x_1x_2 = 11$

- **Transition $00 \rightarrow 11$:** Both y_1 and y_2 change ($0 \rightarrow 1$). **Race Condition!**
 - If y_1 changes first: State becomes 10. In column 11, 10 is a stable state. The circuit is trapped at (10).
 - If y_2 changes first: State becomes 01. In column 11, the next state for 01 is 10, which is stable. The circuit is trapped at (10).
 - If both change simultaneously: The circuit reaches the stable state (11).

Conclusion: The final state depends on gate delays (settling at either (10) or (11)). This is a Critical Race.

- **Transition $01 \rightarrow 10$:** Both y_1 and y_2 change. **Race Condition!**
 - If y_1 changes first: State becomes 11, which is stable under 11. Trapped at (11).
 - If y_2 changes first: State becomes 00. From 00, the next state is 11, leading eventually to (11) or (10).
 - If both change simultaneously: The circuit reaches the stable state (10).

Conclusion: The circuit could settle at (11) or (10). This is a Critical Race.

Under Input $x_1x_2 = 10$

- **Transition $01 \rightarrow 10$:** Both y_1 and y_2 change. **Race Condition!**
 - If y_1 changes first: State becomes 11, which is stable under 10. Trapped at (11).
 - If y_2 changes first: State becomes 00. From 00, next state is 10, which is stable. Path ends at (10).
 - If both change simultaneously: The circuit reaches stable state (10).

Conclusion: The circuit could settle at (11) or (10). This is a Critical Race.

2. Analysis of Cycles

A cycle is a deliberate sequence of transitions through intermediate unstable states before arriving at a stable state.

Cycle under Input $x_1x_2 = 00$

- If the circuit starts in state 00 and the input changes to 00, the following chain of 1-bit transitions occurs naturally:
 - (a) From 00, the next state is 10.
 - (b) From 10, the next state is 11.
 - (c) From 11, the next state is 01.
 - (d) From 01, the next state is 01 (Stable).

Conclusion: This constitutes a 4-step Cycle: $00 \rightarrow 10 \rightarrow 11 \rightarrow (01)$.

Cycles under Input $x_1x_2 = 01$

- The resolution of the noncritical race from state 11 effectively establishes two parallel transient cycles:
 - Cycle A: $11 \rightarrow 01 \rightarrow (00)$
 - Cycle B: $11 \rightarrow 10 \rightarrow (00)$

Summary

Input x_1x_2	Initial State	Phenomenon	Classification
01	11	Race ($11 \rightarrow 00$)	Noncritical (always reaches 00)
11	00	Race ($00 \rightarrow 11$)	Critical (may settle at 10 or 11)
11	01	Race ($01 \rightarrow 10$)	Critical (may settle at 10 or 11)
10	01	Race ($01 \rightarrow 10$)	Critical (may settle at 10 or 11)
00	00	Cycle	Sequence: $00 \rightarrow 10 \rightarrow 11 \rightarrow (01)$

Q8. What is a race in fundamental mode circuits? Explain critical and non-critical races with an example. (4+8 marks) [CSE 205 | L-2/T-1 | 2011–12]

Answer

Races in Fundamental Mode Circuits

In a **fundamental mode asynchronous sequential circuit**, inputs are allowed to change only when the circuit is in a stable state. A **race condition** occurs when a single input variable change requires two or more state variables to change their values simultaneously to reach the next stable state.

Because logic gates and routing paths possess inherent and physically unequal propagation delays, it is practically impossible for multiple state variables to transition at the exact same instant. Instead, they will change sequentially in an unpredictable order, causing the circuit to traverse through unintended intermediate (transient) states.

Races are categorized based on whether these intermediate states alter the final destination of the circuit.

1. Non-Critical Race

A race is **non-critical** if the unequal propagation delays do not prevent the circuit from eventually reaching the correct intended stable state. Regardless of the order in which the state variables change, all possible intermediate paths funnel the circuit into the same final stable state.

Example of a Non-Critical Race: Suppose the circuit is in state $y_1y_2 = 00$ and an input change commands it to transition to a stable state **(11)**. Both y_1 and y_2 must change from 0 to 1.

- If y_2 changes faster, the circuit momentarily enters state 01.
- If y_1 changes faster, the circuit momentarily enters state 10.

If the transition table is designed such that both transient states 01 and 10 point directly to the stable state **(11)**, the race is non-critical.

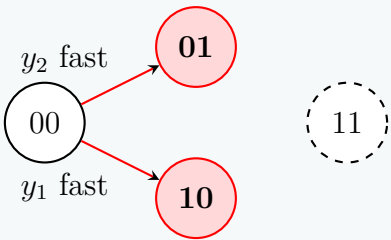
Present State y_1y_2	Next State Y_1Y_2	
	Input = 0	Input = 1
00	(00)	11
01	–	11
10	–	11
11	–	(11)

2. Critical Race

A race is **critical** if the unequal propagation delays cause the circuit to settle in an incorrect, unintended stable state. In this scenario, the final outcome strictly depends on the order in which the state variables evaluate, representing a fatal design flaw.

Example of a Critical Race: Consider the same intended transition from $y_1y_2 = 00$ to 11. However, assume that under this input column, the intermediate states 01 and 10 are independently stable states.

Present State y_1y_2	Next State Y_1Y_2	
	Input = 0	Input = 1
00	(00)	11
01	–	(01)
10	–	(10)
11	–	(11)



- If y_2 changes faster, the circuit lands in state 01. Because 01 is stable, it gets **trapped** there and never reaches 11.
- If y_1 changes faster, the circuit lands in state 10. Because 10 is stable, it gets **trapped** there.

Conclusion: In a critical race, the circuit fails to operate deterministically. To eliminate critical races, designers must perform **proper state assignment**, ensuring that all valid state transitions require a Hamming distance of exactly 1 (only one variable changing at a time).

Pulse Mode Logic

Q1. Design an automatic vending machine, where the candy bars inside the machine cost 3 Tk and the machine accepts 1 Tk and 2 Tk only. The system accepts the bills sequentially. The circuit produces a level output that delivers the candy whenever the amount received by the machine is 3 Tk or more. The excess amount is kept deposited for the next candy. Design a pulse mode circuit for the vending machine using S-R flip-flops. Provide the circuit diagram as well. **(25 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Answer

Design of a Pulse-Mode Vending Machine Circuit

In a pulse-mode asynchronous sequential circuit, the inputs are unlocked pulses, and state transitions occur strictly upon the arrival of these pulses. We will design this machine using a Mealy model, where the output candy delivery (z) occurs as a pulse coincident with the input pulse that brings the total to ≥ 3 Tk.

1. State Definition and State Diagram

Let the states represent the current deposited amount (remainder):

- **State S_0 :** 0 Tk deposited (Assigned $y_1y_2 = 00$)
- **State S_1 :** 1 Tk deposited (Assigned $y_1y_2 = 01$)
- **State S_2 :** 2 Tk deposited (Assigned $y_1y_2 = 10$)

The inputs are mutually exclusive pulses:

- x_1 : 1 Tk coin inserted.
- x_2 : 2 Tk coin inserted.

Logic for transitions (Next State & Output):

- From S_0 : $x_1 \rightarrow S_1, z = 0$. $x_2 \rightarrow S_2, z = 0$.
- From S_1 : $x_1 \rightarrow S_2, z = 0$. $x_2 \rightarrow S_0, z = 1$ (Total = 3 Tk. Dispense. Remainder = 0 Tk).
- From S_2 : $x_1 \rightarrow S_0, z = 1$ (Total = 3 Tk. Dispense. Remainder = 0 Tk).
 $x_2 \rightarrow S_1, z = 1$ (Total = 4 Tk. Dispense. Remainder = 1 Tk).

2. State Transition and Excitation Table

We map the states to binary variables y_1y_2 and define the required S-R flip-flop inputs. The excitation rules for an SR flip-flop are: $(0 \rightarrow 0 : S = 0, R = d)$, $(0 \rightarrow 1 : S = 1, R = 0)$, $(1 \rightarrow 0 : S = 0, R = 1)$, $(1 \rightarrow 1 : S = d, R = 0)$. Since $y_1y_2 = 11$ is unused, its next states and outputs are treated as don't cares (d).

Present State y_1y_2	Next State Y_1Y_2		Output z		FF1 (S_1, R_1)		FF2 (S_2, R_2)	
	x_1	x_2	x_1	x_2	x_1	x_2	x_1	x_2
00	01	10	0	0	0, d	1, 0	1, 0	0, d
01	10	00	0	1	1, 0	0, d	0, 1	0, 1
10	00	01	1	1	0, 1	0, 1	0, d	1, 0
11	dd	dd	d	d	d, d	d, d	d, d	d, d

3. Boolean Logic Derivation

Since the system is in pulse mode and the inputs x_1 and x_2 are mutually exclusive (they never arrive at the exact same instant), we can derive the boolean expressions by taking the sum of products for each input column directly.

For Flip-Flop 1 (y_1):

$$\begin{aligned}
 S_1 &= (y_1y_2 = 01) \cdot x_1 + (y_1y_2 = 00) \cdot x_2 \\
 &= \bar{y}_1y_2x_1 + \bar{y}_1\bar{y}_2x_2 \\
 &= y_2x_1 + \bar{y}_1\bar{y}_2x_2 \quad (\text{Simplifying with don't care 11}) \\
 R_1 &= (y_1y_2 = 10) \cdot x_1 + (y_1y_2 = 10) \cdot x_2 \\
 &= y_1\bar{y}_2x_1 + y_1\bar{y}_2x_2 \\
 &= y_1(x_1 + x_2)
 \end{aligned}$$

For Flip-Flop 2 (y_2):

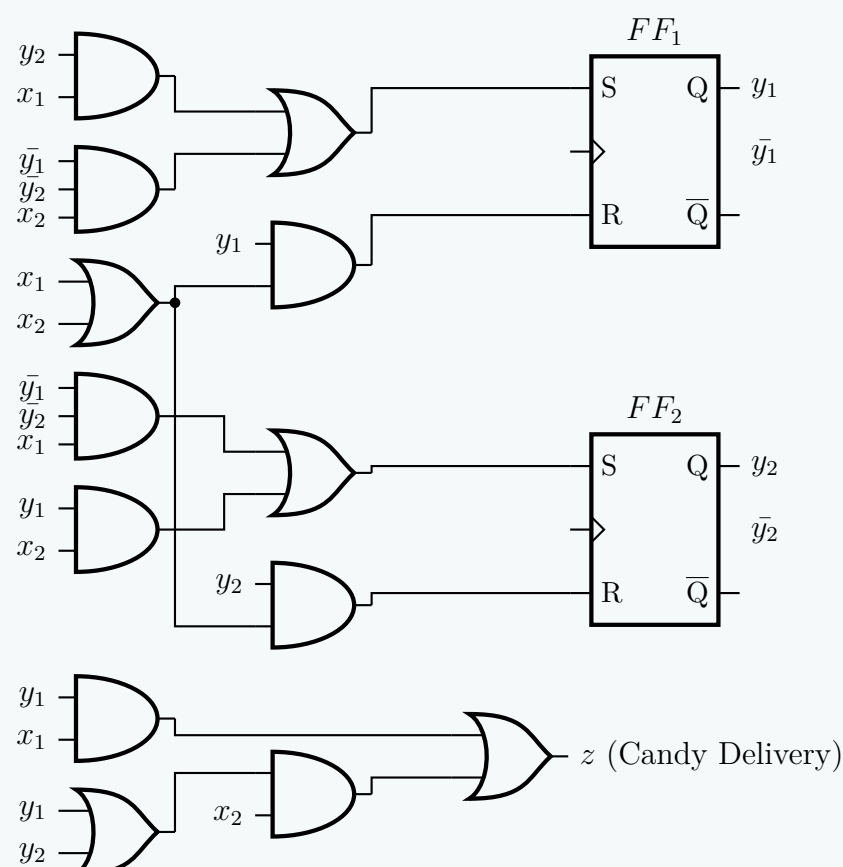
$$\begin{aligned}
 S_2 &= (y_1y_2 = 00) \cdot x_1 + (y_1y_2 = 10) \cdot x_2 \\
 &= \bar{y}_1\bar{y}_2x_1 + y_1\bar{y}_2x_2 \\
 &= \bar{y}_1\bar{y}_2x_1 + y_1x_2 \quad (\text{Simplifying with don't care 11}) \\
 R_2 &= (y_1y_2 = 01) \cdot x_1 + (y_1y_2 = 01) \cdot x_2 \\
 &= \bar{y}_1y_2x_1 + \bar{y}_1y_2x_2 \\
 &= y_2(x_1 + x_2)
 \end{aligned}$$

For Output (z):

$$\begin{aligned}
 z &= (y_1y_2 = 10) \cdot x_1 + (y_1y_2 = 01, 10) \cdot x_2 \\
 &= y_1\bar{y}_2x_1 + (\bar{y}_1y_2 + y_1\bar{y}_2)x_2 \\
 &= y_1x_1 + (y_1 + y_2)x_2
 \end{aligned}$$

4. Logic Circuit Implementation

Using standard logic gates, the final schematic is constructed below. For drawing clarity, the feedback lines from the flip-flop outputs ($y_1, \bar{y}_1, y_2, \bar{y}_2$) are represented as local signal tags at the inputs of the corresponding logic gates.



Q2. Design a pulse mode sequential circuit that satisfies the following requirements. The circuit has two input lines x_1 and x_2 along with one level output line z . An output transition from 0 to 1 is produced only after the occurrence of the last x_2 pulse in the sequence $x_1-x_2-x_1-x_2$. The output is reset from 1 to 0 only after the first x_1 pulse that occurs following the 0 to 1 output transition. The circuit allows overlapping sequences. Develop the pulse mode circuit using JK flip-flops. Draw the circuit diagram. **(23 marks)**
[CSE 205 | L-2/T-1 | 2023–24]

Answer

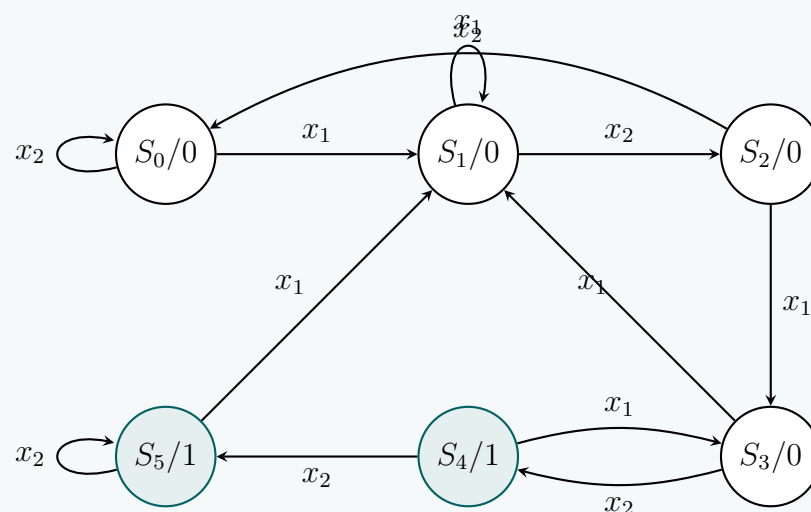
Design of Pulse-Mode Sequence Detector (Overlapping)

We are tasked with designing a fundamental pulse-mode sequential circuit that detects the sequence $x_1 \rightarrow x_2 \rightarrow x_1 \rightarrow x_2$. The inputs x_1 and x_2 are unclocked pulses. The output z is a level signal that becomes 1 upon completing the sequence and resets to 0 only upon receiving the first x_1 pulse after the 0 $\rightarrow$ 1 transition. Overlapping is allowed.

1. State Definition and State Diagram

Since the output z is a level output tied to the circuit's state, we use a Moore machine model.

- S_0 : Initial state, $z = 0$. No valid prefix.
- S_1 : $z = 0$. Received prefix: x_1 .
- S_2 : $z = 0$. Received prefix: x_1, x_2 .
- S_3 : $z = 0$. Received prefix: x_1, x_2, x_1 .
- S_4 : $z = 1$. Sequence x_1, x_2, x_1, x_2 detected.
 - If x_1 arrives, z resets to 0. The last 3 inputs are now x_1, x_2, x_1 , matching the prefix for S_3 . This enables overlapping.
 - If x_2 arrives, z remains 1, but the overlapping prefix (x_1, x_2) is broken. We move to a new state S_5 .
- S_5 : $z = 1$. Prefix broken. Output remains 1 until the next x_1 arrives (which sends it to S_1 and resets $z = 0$).



2. State Transition and Excitation Table

We assign 3 binary state variables ($y_1y_2y_3$) to the 6 states. States 110 and 111 are unused (don't cares). For pulse-mode circuits with JK flip-flops, the excitation values determine the required inputs during the presence of a pulse.

Present State	Output z	Next State		FF Inputs for x_1			FF Inputs for x_2		
$y_1y_2y_3$		x_1	x_2	J_1K_1	J_2K_2	J_3K_3	J_1K_1	J_2K_2	J_3K_3
000 (S_0)	0	001	000	0d	0d	1d	0d	0d	0d
001 (S_1)	0	001	010	0d	0d	d0	0d	1d	d1
010 (S_2)	0	011	000	0d	d0	1d	0d	d1	0d
011 (S_3)	0	001	100	0d	d1	d0	1d	d1	d1
100 (S_4)	1	011	101	d1	1d	1d	d0	0d	1d
101 (S_5)	1	001	101	d1	0d	d0	d0	0d	d0

3. Derivation of JK Flip-Flop Inputs

In pulse-mode circuits, the general flip-flop input equation takes the form $J = J_{x_1}x_1 + J_{x_2}x_2$. We extract the Boolean terms from the columns of the excitation table, treating states 110 and 111 as don't cares.

For Flip-Flop 1 (y_1):

$$J_1 = 0 \cdot x_1 + (y_2y_3) \cdot x_2 \implies \mathbf{J_1 = y_2y_3x_2}$$

$$K_1 = 1 \cdot x_1 + 0 \cdot x_2 \implies \mathbf{K_1 = x_1}$$

For Flip-Flop 2 (y_2):

$$J_2 = (y_1\bar{y}_3) \cdot x_1 + (\bar{y}_1y_3) \cdot x_2 \implies \mathbf{J_2 = y_1\bar{y}_3x_1 + \bar{y}_1y_3x_2}$$

$$K_2 = (y_3) \cdot x_1 + 1 \cdot x_2 \implies \mathbf{K_2 = y_3x_1 + x_2}$$

For Flip-Flop 3 (y_3):

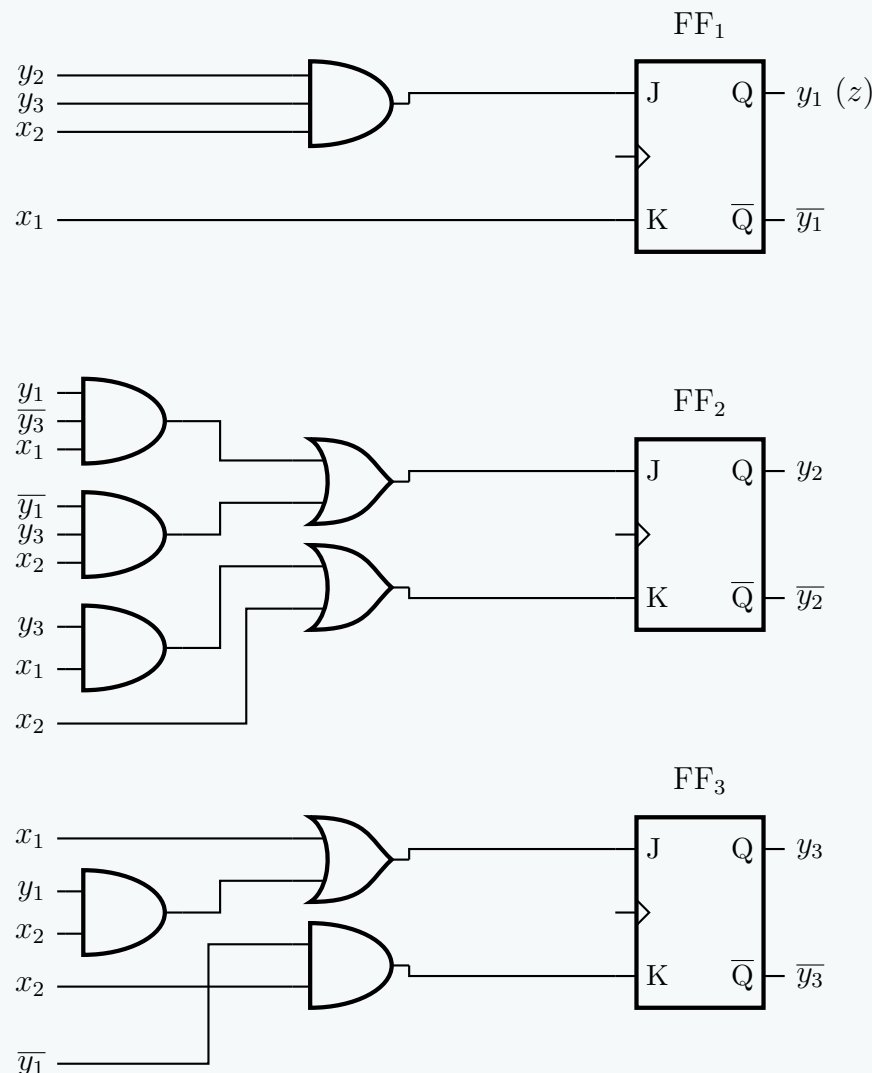
$$J_3 = 1 \cdot x_1 + (y_1) \cdot x_2 \implies \mathbf{J_3 = x_1 + y_1x_2}$$

$$K_3 = 0 \cdot x_1 + (\bar{y}_1) \cdot x_2 \implies \mathbf{K_3 = \bar{y}_1x_2}$$

Output Logic (z): The output z is 1 only in states 100 and 101. Thus, $z = y_1\bar{y}_2\bar{y}_3 + y_1\bar{y}_2y_3 = y_1\bar{y}_2$. Factoring in the don't cares (110, 111), this simplifies to:

$$\mathbf{z = y_1}$$

4. Logic Circuit Diagram



Note: The feedback variables are shown as named port tags (e.g., y_2 , $\bar{y}_3$) for optimal readability of the schematic logic paths.

Q3. Design an automatic vending machine. The candy bars inside the machine cost 15 Tk, and the machine accepts 5 Tk and 10 Tk only. An electromechanical system accepts the money sequentially. The circuit produces a pulse output that delivers the candy whenever the amount received by the machine is 15 Tk or more. The excess amount is kept deposited for the next candy. Design a pulse mode circuit for the vending machine using S-R latches. **(20 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Answer

Pulse Mode Vending Machine Circuit Design

We are to design a pulse mode sequential circuit using S-R latches. The machine dispenses a candy bar (pulse output $z = 1$) when ≥ 15 Tk is deposited. The machine accepts two inputs sequentially: x_1 (5 Tk pulse) and x_2 (10 Tk pulse). Excess amount is carried over to the next transaction.

1. State Definitions and State Diagram

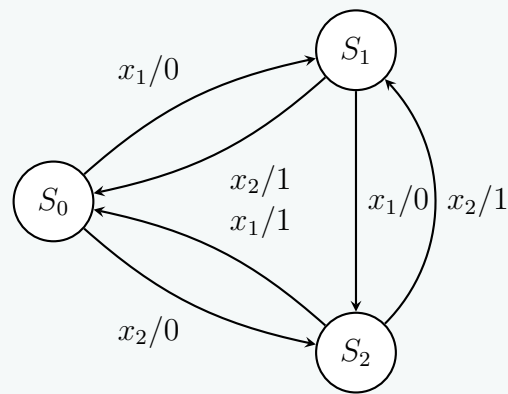
Let the states represent the current accumulated money in the machine:

- S_0 (0 Tk deposited): The initial state.
- S_1 (5 Tk deposited): 5 Tk has been accumulated.
- S_2 (10 Tk deposited): 10 Tk has been accumulated.

The state transitions are as follows:

- **From S_0 :** If x_1 (5) arrives $\rightarrow S_1$, $z = 0$. If x_2 (10) arrives $\rightarrow S_2$, $z = 0$.

- **From S_1 :** If x_1 (5) arrives $\rightarrow S_2$, $z = 0$. If x_2 (10) arrives $\rightarrow$ 15 Tk total. Dispense candy ($z = 1$), 0 Tk left, go to S_0 .
- **From S_2 :** If x_1 (5) arrives $\rightarrow$ 15 Tk total. Dispense candy ($z = 1$), 0 Tk left, go to S_0 . If x_2 (10) arrives $\rightarrow$ 20 Tk total. Dispense candy ($z = 1$), 5 Tk left, go to S_1 .



2. State Transition and Excitation Table

We assign two state variables y_1y_2 for the 3 states: $S_0 = 00$, $S_1 = 01$, $S_2 = 10$. The state 11 is a don't care (d). For S-R latches, the excitation is: $0 \rightarrow 0$ ($0, d$), $0 \rightarrow 1$ ($1, 0$), $1 \rightarrow 0$ ($0, 1$), $1 \rightarrow 1$ ($d, 0$). Since this is a pulse mode circuit, the outputs and S-R inputs are separated by the active input pulse.

Present State	Next State		Output (z)		S-R Inputs for x_1				S-R Inputs for x_2			
	y_1y_2	x_1	x_2	z	S_1	R_1	S_2	R_2	S_1	R_1	S_2	R_2
00 (S_0)		01	10	0	0	d	1	0	1	0	0	d
01 (S_1)		10	00	0	1	0	0	1	0	d	0	1
10 (S_2)		00	01	1	0	1	0	d	0	1	1	0
11 (d)		dd	dd	d	d	d	d	d	d	d	d	d

3. Derivation of Boolean Equations

For pulse mode, excitation inputs take the form $S = S_{x_1}x_1 + S_{x_2}x_2$. Extracting the terms from the table with don't cares (state 11):
S-R Inputs for Latch 1 (y_1):

$$S_1 = (y_2)x_1 + (\bar{y}_1\bar{y}_2)x_2$$
$$R_1 = (y_1)x_1 + (y_1)x_2$$

$$\implies \mathbf{S_1 = y_2x_1 + \bar{y}_1\bar{y}_2x_2}$$
$$\implies \mathbf{R_1 = y_1(x_1 + x_2)}$$

S-R Inputs for Latch 2 (y_2):

$$S_2 = (\bar{y}_1\bar{y}_2)x_1 + (y_1)x_2$$
$$R_2 = (y_2)x_1 + (y_2)x_2$$

$$\implies \mathbf{S_2 = \bar{y}_1\bar{y}_2x_1 + y_1x_2}$$
$$\implies \mathbf{R_2 = y_2(x_1 + x_2)}$$

Output Logic (z): The output z is 1 when $y_1y_2 = 10$ under x_1 , and when $y_1y_2 \in \{01, 10\}$ under x_2 .

$$z = (y_1)x_1 + (y_1 + y_2)x_2$$

$$\implies \mathbf{z = y_1x_1 + (y_1 + y_2)x_2}$$

4. Logic Circuit Diagram

Note: Feedback variables are indicated by named inputs (e.g., y_1 , $\bar{y}_2$) for diagram clarity and routing neatness.

Q4. Design a circuit for a digit lock using T latches. The lock has two input push-button switches A , B and it outputs a single pulse (Z_1) for each activation. The two switches are interlocked mechanically so that simultaneous pulses are not possible. The lock should have the following features: (i) The opening sequence is AABABA. (ii) Three B pulses should give an absolute reset. (iii) Any incorrect use of the A switch will cause an output (Z_2) to ring a bell to warn that the lock is being tampered with. **(20 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Answer

Digit Lock Pulse-Mode Circuit Design

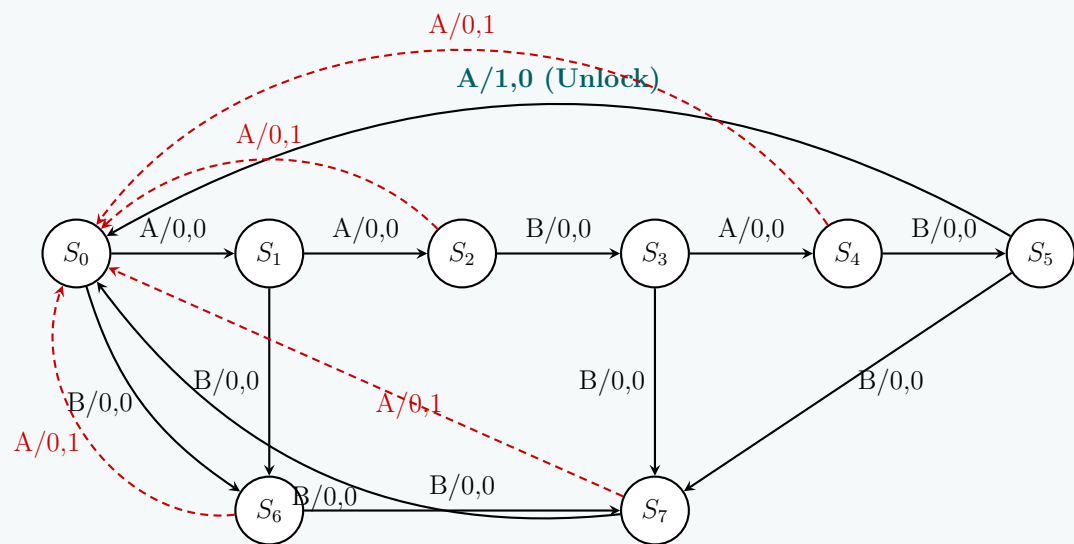
The system is a Mealy-type pulse-mode sequential circuit. Inputs A and B are mechanically interlocked, meaning we process them as separate pulse events ($A = 1, B = 0$ or $A = 0, B = 1$). The system requires 3 latches (T_1, T_2, T_3) to track the progress of the sequence and handle the absolute reset conditions.

1. State Definition & Machine Formulation

The lock opens on the sequence $A \rightarrow A \rightarrow B \rightarrow A \rightarrow B \rightarrow A$. Any incorrect A pulse rings a bell ($Z_2 = 1$) and breaks the sequence. Three B pulses reset the system unconditionally. We define the following 8 states:

- S_0 (**000**): Initial state / 0 consecutive B s.
- S_1 (**001**): Sequence **A** received.
- S_2 (**010**): Sequence **AA** received.
- S_3 (**011**): Sequence **AAB** received.
- S_4 (**100**): Sequence **AABA** received.
- S_5 (**101**): Sequence **AABAB** received.
- S_6 (**110**): Sequence broken; 1 B pulse registered.
- S_7 (**111**): Sequence broken; 2 B pulses registered.

2. State Transition Diagram



3. State Transition & Excitation Table

For a T-latch in pulse mode, the excitation is $T = \text{Present State} \oplus \text{Next State}$. We separate the excitations caused by pulse A (T_{iA}) and pulse B (T_{iB}). Z_1 and Z_2 strictly trigger on A pulses based on problem constraints.

Present State $y_1y_2y_3$	Next State		Output		T Inputs (A Pulse)			T Inputs (B Pulse)		
	$A = 1$	$B = 1$	Z_{1A}	Z_{2A}	T_{1A}	T_{2A}	T_{3A}	T_{1B}	T_{2B}	T_{3B}
000 (S_0)	001 (S_1)	110 (S_6)	0	0	0	0	1	1	1	0
001 (S_1)	010 (S_2)	110 (S_6)	0	0	0	1	1	1	1	1
010 (S_2)	000 (S_0)	011 (S_3)	0	1	0	1	0	0	0	1
011 (S_3)	100 (S_4)	111 (S_7)	0	0	1	1	1	1	0	0
100 (S_4)	000 (S_0)	101 (S_5)	0	1	1	0	0	0	0	1
101 (S_5)	000 (S_0)	111 (S_7)	1	0	1	0	1	0	1	0
110 (S_6)	000 (S_0)	111 (S_7)	0	1	1	1	0	0	0	1
111 (S_7)	000 (S_0)	000 (S_0)	0	1	1	1	1	1	1	1

4. Karnaugh Map Simplification

The excitation inputs are modeled as $T_i = T_{iA} \cdot A + T_{iB} \cdot B$.

0001

01

11

10

0

1

1

1

1

1

$T_{1A} = y_1 + y_2y_3$

0001

01

11

10

0

1

1

1

1

1

$T_{2A} = y_2 + \bar{y}_1y_3$

0001

01

11

10

0

1

1

1

1

1

$T_{3A} = \bar{y}_1\bar{y}_2 + y_3$

0001

01

11

10

0

1

1

1

1

1

 $T_{1B} = \bar{y}_1\bar{y}_2 + \bar{y}_1y_3 + y_2y_3$

0001

01

11

10

0

1

1

1

1

1

 $T_{2B} = \bar{y}_1\bar{y}_2 + y_1y_3$

0001

01

11

10

0

1

1

1

1

1

 $T_{3B} = Z_{2A} + \bar{y}_1\bar{y}_2y_3$

0001

01

11

10

0

1

1

1

1

1

 $Z_{1A} = y_1\bar{y}_2y_3$

0001

01

11

10

0

1

1

1

1

1

 $Z_{2A} = y_2\bar{y}_3 + y_1\bar{y}_3 + y_1y_2$

5. Final Boolean Logic Equations

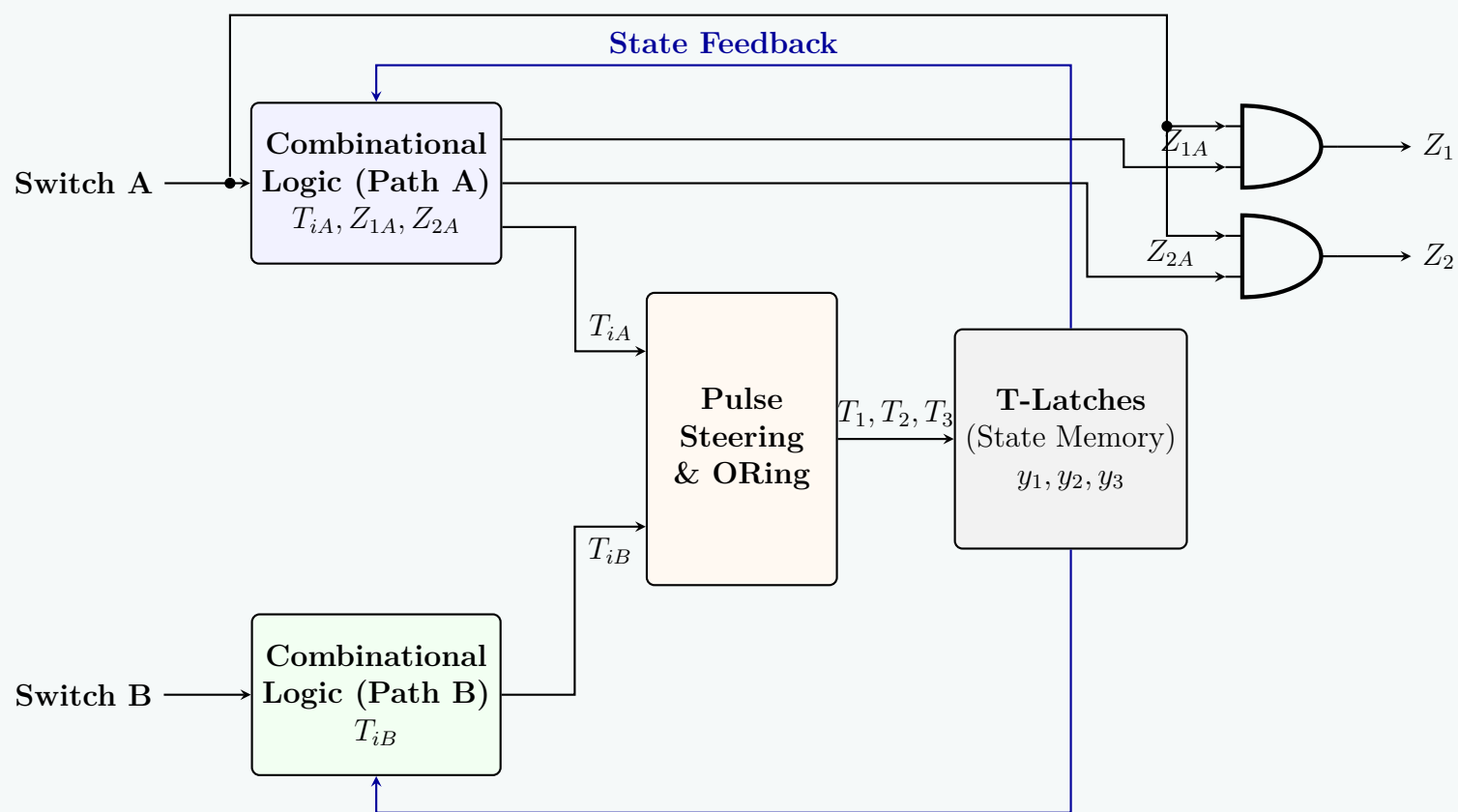
Integrating the individual input paths, the Mealy equations for the overall system are exactly defined as:

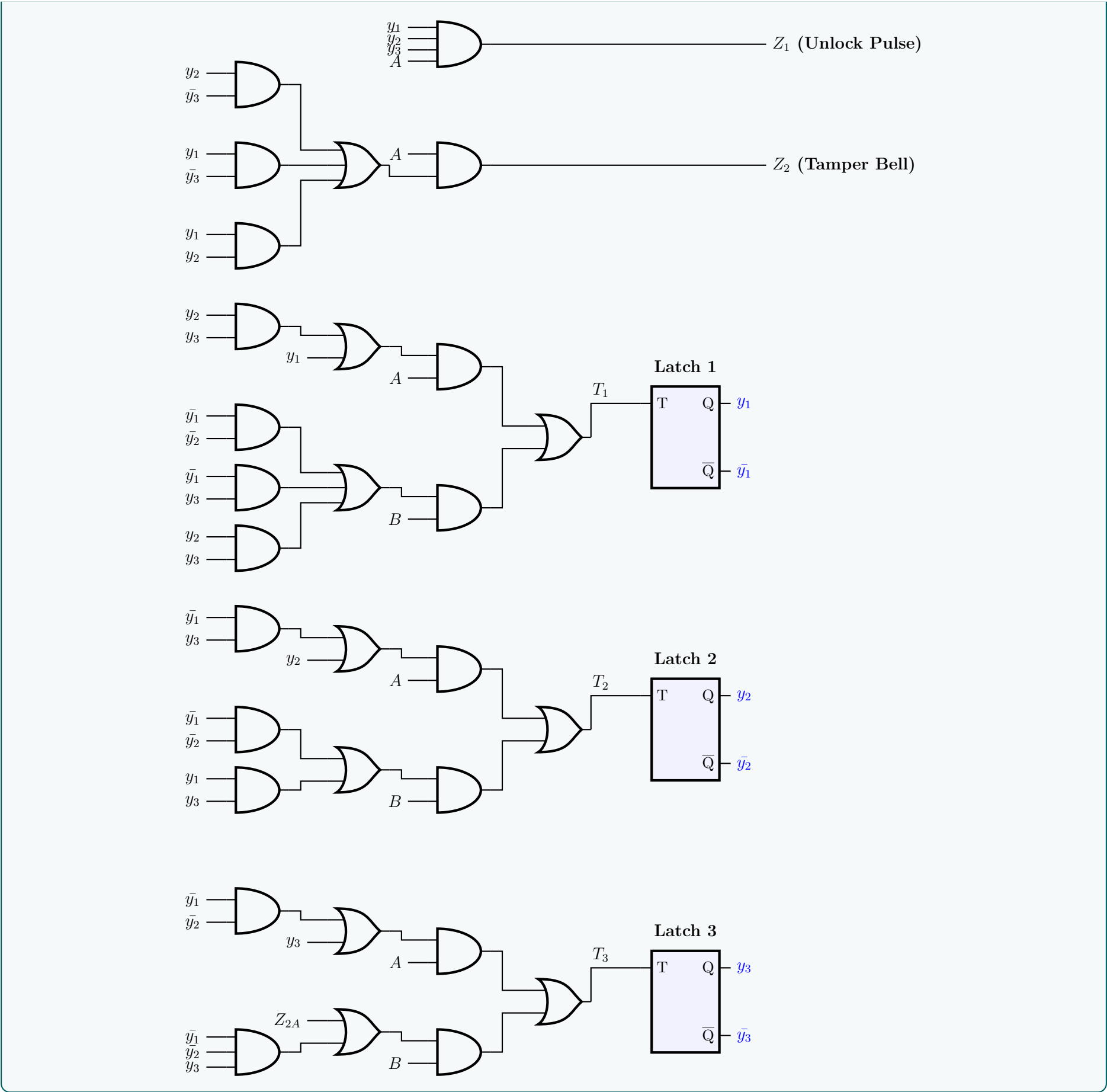
$$\begin{aligned} T_1 &= (y_1 + y_2 y_3)A + (\bar{y}_1 \bar{y}_2 + \bar{y}_1 y_3 + y_2 y_3)B \\ T_2 &= (\bar{y}_1 y_3 + y_2)A + (\bar{y}_1 \bar{y}_2 + y_1 y_3)B \\ T_3 &= (\bar{y}_1 \bar{y}_2 + y_3)A + (Z_{2A} + \bar{y}_1 \bar{y}_2 y_3)B \\ Z_1 &= (y_1 \bar{y}_2 y_3)A \quad (\text{Unlock Pulse}) \\ Z_2 &= (y_2 \bar{y}_3 + y_1 \bar{y}_3 + y_1 y_2)A \quad (\text{Tamper Bell Pulse}) \end{aligned}$$

Note: The sub-expression Z_{2A} beautifully recurs in T_{3B} , allowing optimization by reusing the majority-gate logic: $Z_{2A} = \text{Maj}(y_1, y_2, \bar{y}_3)$.

6. Structural Block Architecture

To ensure practical realizability without the ambiguity of spaghetti wiring across 3 latches and 20+ combinational nodes, the schematic is organized into deterministic, modular logic blocks:





Q5. Design an automatic vending machine. Candy bars cost 3 Tk; the machine accepts 1 Tk and 2 Tk coins sequentially. The circuit produces a level output delivering the candy whenever the amount received equals or exceeds 3 Tk. Excess amount is kept for the next candy. Design a pulse mode circuit using SR Flip-Flops. Draw the circuit diagram. **(20 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

Pulse-Mode Vending Machine Circuit Design

The system is a pulse-mode sequential circuit. The inputs are x_1 (pulse for 1 Tk coin) and x_2 (pulse for 2 Tk coin). Since the coins are inserted sequentially, x_1 and x_2 are mutually exclusive pulses. The output Z is a pulse triggered when a candy is delivered (amount ≥ 3 Tk).

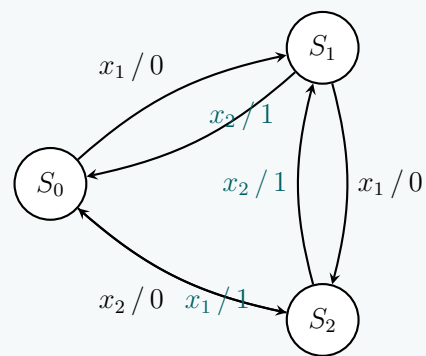
1. State Definition & FSM

We define states based on the current credit stored in the machine:

- S_0 (**00**): 0 Tk credit.
- S_1 (**01**): 1 Tk credit.
- S_2 (**10**): 2 Tk credit.
- S_3 (**11**): Unused / Don't Care.

When the sum reaches or exceeds 3 Tk, a candy is dispensed ($Z = 1$). Any excess amount is retained as credit for the next candy.

- From S_1 (1 Tk), an x_2 (2 Tk) pulse gives 3 Tk total $\rightarrow$ dispense ($Z = 1$), keep 0 Tk (S_0).
- From S_2 (2 Tk), an x_1 (1 Tk) pulse gives 3 Tk total $\rightarrow$ dispense ($Z = 1$), keep 0 Tk (S_0).
- From S_2 (2 Tk), an x_2 (2 Tk) pulse gives 4 Tk total $\rightarrow$ dispense ($Z = 1$), keep 1 Tk (S_1).



2. State Transition & SR Excitation Table

We map the states to flip-flop variables y_1y_2 . The SR flip-flop excitation rules are: $(0 \rightarrow 0 : S = 0, R = d)$, $(0 \rightarrow 1 : S = 1, R = 0)$, $(1 \rightarrow 0 : S = 0, R = 1)$, and $(1 \rightarrow 1 : S = d, R = 0)$. We partition the transition conditions into the mutually exclusive input pulses x_1 and x_2 .

Present	Input x_1 (1 Tk Pulse)							Input x_2 (2 Tk Pulse)				
	Y_1Y_2	Z_{x_1}	S_1	R_1	S_2	R_2	Y_1Y_2	Z_{x_2}	S_1	R_1	S_2	R_2
00 (S_0)	01	0	0	d	1	0	10	0	1	0	0	d
01 (S_1)	10	0	1	0	0	1	00	1	0	d	0	1
10 (S_2)	00	1	0	1	0	d	01	1	0	1	1	0
11 (DC)	dd	d	d	d	d	d	dd	d	d	d	d	d

3. Logic Simplification (K-maps)

Using 2-variable Karnaugh maps for states y_1y_2 (with minterm 3 as Don't Care):

For Input Pulse x_1 :

- $S_{1,x_1} = \sum m(1) + d(3) \implies \mathbf{y_2}$
- $R_{1,x_1} = \sum m(2) + d(0, 3) \implies \mathbf{y_1}$
- $S_{2,x_1} = \sum m(0) + d(3) \implies \mathbf{\bar{y}_1\bar{y}_2}$
- $R_{2,x_1} = \sum m(1) + d(2, 3) \implies \mathbf{y_2}$
- $Z_{x_1} = \sum m(2) + d(3) \implies \mathbf{y_1}$

For Input Pulse x_2 :

- $S_{1,x_2} = \sum m(0) + d(3) \implies \mathbf{\bar{y}_1\bar{y}_2}$
- $R_{1,x_2} = \sum m(2) + d(1, 3) \implies \mathbf{y_1}$
- $S_{2,x_2} = \sum m(2) + d(3) \implies \mathbf{y_1}$
- $R_{2,x_2} = \sum m(1) + d(0, 3) \implies \mathbf{y_2}$
- $Z_{x_2} = \sum m(1, 2) + d(3) \implies \mathbf{y_1 + y_2}$

4. Final Boolean Equations

Combine the mutually exclusive input conditions ($Q = Q_{x_1} \cdot x_1 + Q_{x_2} \cdot x_2$):

$$\mathbf{S_1 = y_2x_1 + \bar{y}_1\bar{y}_2x_2}$$
$$\mathbf{R_1 = y_1x_1 + y_1x_2 = y_1(x_1 + x_2)}$$
$$\mathbf{S_2 = \bar{y}_1\bar{y}_2x_1 + y_1x_2}$$
$$\mathbf{R_2 = y_2x_1 + y_2x_2 = y_2(x_1 + x_2)}$$
$$\mathbf{Z = y_1x_1 + (y_1 + y_2)x_2}$$

5. Circuit Implementation

The architecture is implemented using two SR Flip-Flops and combinational steering logic.

$S_1 = y_2x_1 + \bar{y}_1\bar{y}_2x_2$

$R_1 = y_1(x_1 + x_2)$

$S_2 = \bar{y}_1\bar{y}_2x_1 + y_1x_2$

$R_2 = y_2(x_1 + x_2)$

$Z = y_1x_1 + (y_1 \vee y_2)x_2$

FF 1

FF 2

Q6. Write down the restrictions on the duration of input for pulse mode asynchronous circuits. (4 marks)

2011–12]

[CSE 205 | L-2/T-1 |

Answer

Restrictions on Input Duration for Pulse-Mode Asynchronous Circuits

In pulse-mode asynchronous sequential circuits, inputs are treated as pulses rather than levels. A pulse is defined as a signal that changes from an unasserted state (usually logic 0) to an asserted state (logic 1) and returns to the unasserted state within a brief window of time.

Because there is no global clock to synchronize state transitions, the reliable operation of a pulse-mode circuit heavily depends on the physical duration of the input pulses. The duration of an input pulse, denoted as t_p , must satisfy two critical timing constraints:

1. Minimum Pulse Width Restriction ($t_p \geq t_{\min}$)

The input pulse must be present (active) for a sufficient amount of time to allow the circuit to register the input and complete the state transition.

- **Reasoning:** The pulse must remain active long enough to propagate through the combinational steering logic, generate the correct excitation signals, and successfully trigger the state memory elements (typically unlocked SR latches or flip-flops) to change their state.
- **Mathematical Bound:** If t_{cg} is the propagation delay of the combinational logic and t_m is the response time of the memory element, the minimum duration is defined as:

$$t_p \geq t_{cg} + t_m$$

If the pulse is shorter than this duration, the memory elements may not fully transition, leading to unpredictable behavior or metastability.

2. Maximum Pulse Width Restriction ($t_p \leq t_{\max}$)

The input pulse must not remain active for too long; it must return to its inactive level (logic 0) before the circuit can respond to the newly generated internal state.

- **Reasoning:** Once the memory elements transition to the new state, these new secondary variables (state variables) are fed back into the combinational logic. If the original input pulse is still active when these new variables arrive, the circuit might undergo a second, unintended state transition.
- **Mathematical Bound:** The pulse must be removed before the new state variables propagate through the combinational logic back to the inputs of the memory elements.

$$t_p < \text{Propagation delay of the feedback loop}$$

Violating this maximum duration restriction effectively turns the circuit into a free-running oscillator or causes it to skip through multiple states erroneously.

3. Inter-Pulse Separation (Settling Time)

In addition to the duration of a single pulse, there is a restriction on the time interval *between* consecutive pulses.

- **Reasoning:** Pulses must be mutually exclusive in time. Consecutive pulses must be separated by a time interval long enough to allow the newly established state variables to stabilize completely throughout the entire circuit.
- **Constraint:** If t_s is the time between two consecutive pulses, then:

$$t_s > \text{Total circuit settling time}$$

Summary of Pulse Duration Constraint

$$\text{Device Latency} \leq \text{Input Pulse Duration } (t_p) < \text{Feedback Loop Delay}$$

Synchronous Sequential Circuit Design

Q1. Design an automatic vending machine, when the candy bars inside the machine cost 3 Tk and the machine accepts 1 Tk and 2 Tk only. The system accepts the bills sequentially. The circuit produces a level output that delivers the candy whenever the amount received by the machine is 3 Tk or more. The excess amount is kept deposited for the next candy. Design a synchronous sequential circuit for the vending machine using S-R flip-flops. Provide the circuit diagram as well. **(25 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Answer

1. Problem Formulation & State Definition

We are tasked with designing a sequential circuit for a vending machine that dispenses a candy (3 Tk) using 1 Tk and 2 Tk coins. We will implement this as a **Mealy State Machine** to minimize the number of states.

- **Input (X):** Assuming sequential, single-coin insertion per clock cycle, let $X = 0$ represent a **1 Tk** coin, and $X = 1$ represent a **2 Tk** coin.
- **Output (Z):** $Z = 1$ triggers the candy delivery mechanism.
- **States (S):** The states track the accumulated money.
 - S_0 (00): 0 Tk deposited.
 - S_1 (01): 1 Tk deposited.
 - S_2 (10): 2 Tk deposited.
 - State 11 is an unused "Don't Care" state.

2. State Transition & Excitation Table

We use two S-R flip-flops (Q_1 for MSB, Q_0 for LSB). The excitation rules for an S-R flip-flop ($Q \rightarrow Q^*$) are: $(0 \rightarrow 0) \implies S = 0, R = X$; $(0 \rightarrow 1) \implies S = 1, R = 0$; $(1 \rightarrow 0) \implies S = 0, R = 1$; $(1 \rightarrow 1) \implies S = X, R = 0$.

Current State		Input	Next State		Output	S-R Flip-Flop Inputs			
Q_1	Q_0	X	Q_1^*	Q_0^*	Z	S_1	R_1	S_0	R_0
0	0	0 (1 Tk)	0	1	0	0	X	1	0
0	0	1 (2 Tk)	1	0	0	1	0	0	X
0	1	0 (1 Tk)	1	0	0	1	0	0	1
0	1	1 (2 Tk)	0	0	1	0	X	0	1
1	0	0 (1 Tk)	0	0	1	0	1	0	X
1	0	1 (2 Tk)	0	1	1	0	1	1	0
1	1	X (d.c.)	X	X	X	X	X	X	X

Note: When accumulated amount ≥ 3 Tk, output $Z = 1$, and the next state reflects the remainder (e.g., $Q(t) = 10 + X = 1 \implies 4$ Tk total $\implies Z = 1, Q_{next} = 01$ (1 Tk left)).

3. Boolean Logic Minimization

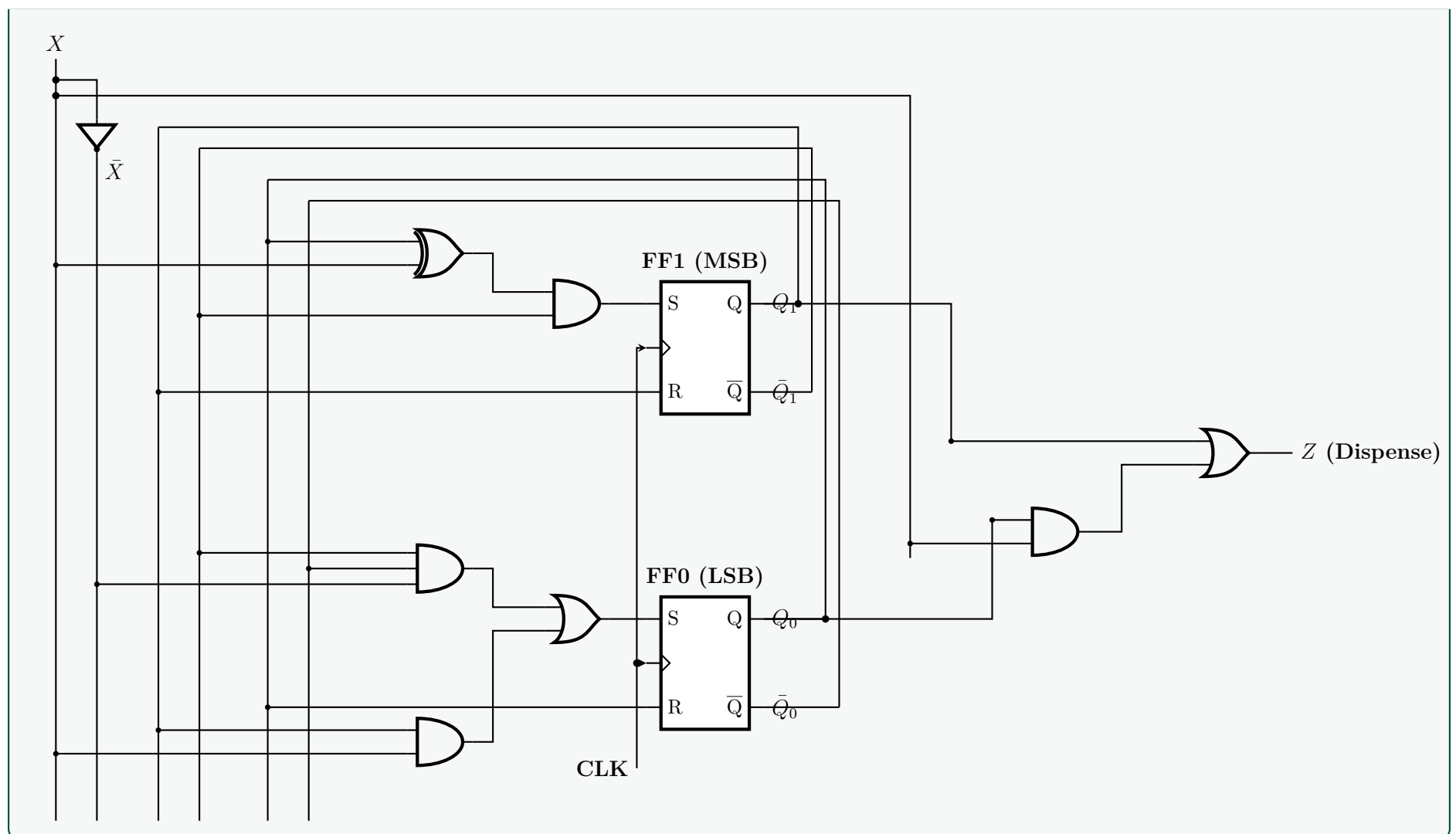
Applying Karnaugh Maps to the transition table yields the following minimized Boolean expressions.

Candy Output (Z):
Flip-Flop 1 (S_1, R_1):
Flip-Flop 0 (S_0, R_0):

$$Z = \Sigma m(3, 4, 5) + d(6, 7) = Q_1 + Q_0X$$
$$S_1 = \Sigma m(1, 2) + d(6, 7) = Q_1'Q_0'X + Q_1'Q_0X' = Q_1'(Q_0 \oplus X)$$
$$R_1 = \Sigma m(4, 5) + d(0, 3, 6, 7) = Q_1$$
$$S_0 = \Sigma m(0, 5) + d(6, 7) = Q_1'Q_0'X' + Q_1X$$
$$R_0 = \Sigma m(2, 3) + d(1, 4, 6, 7) = Q_0$$

4. Vending Machine Circuit Diagram

The circuit logic uses standard gates, decoupled with clear signal labels to maintain readability and eliminate cluttered wiring.



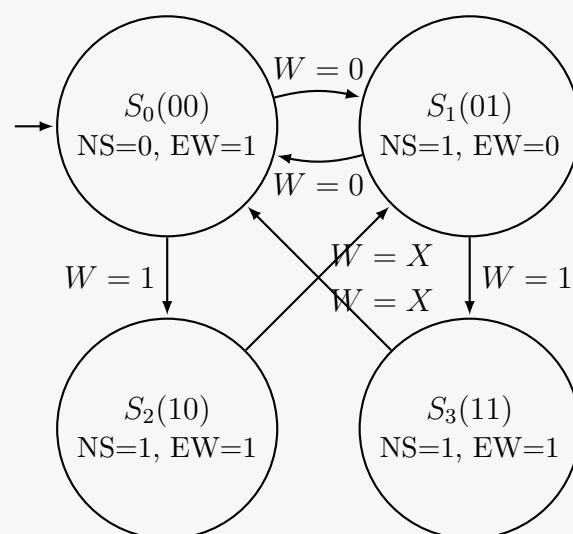
Q2. Design a simplified traffic-light controller that switches traffic lights on a crossing where a north-south (NS) street intersects an east-west (EW) street. The input to the controller is the WALK button pushed by pedestrians who want to cross the street. The outputs are two signals NS and EW that control the traffic lights in NS and EW directions. When NS or EW are 0, the red light is on and when they are 1, the green light is on. When there are no pedestrians, NS = 0 and EW = 1 for 1 minute, followed by NS = 1 and EW = 0 for 1 minute and so on. When a WALK button is pushed, NS and EW both come 1 for a minute when the present minute expires. After that the NS and EW signals continue alternating. For the traffic-light controller: (i) Develop a state diagram and state/output table. (ii) Find the transition and excitation tables using S-R flipflops. (iii) Draw a circuit diagram. **(23 marks)**
 [CSE 205 | L-2/T-1 | 2023-24]

Answer

(i) State Diagram and State/Output Table

Let us define a Moore finite state machine with a clock period of 1 minute. The input is the WALK button, denoted as $W \in \{0, 1\}$. The outputs are the North-South (NS) and East-West (EW) signals. We require four states to keep track of the normal sequence and the interrupted sequences:

- S_0 (00): Normal state where EW is Green. (Outputs: NS = 0, EW = 1)
- S_1 (01): Normal state where NS is Green. (Outputs: NS = 1, EW = 0)
- S_2 (10): Pedestrian Walk state triggered from S_0 . (Outputs: NS = 1, EW = 1)
- S_3 (11): Pedestrian Walk state triggered from S_1 . (Outputs: NS = 1, EW = 1)



Current State		Next State		Outputs	
Symbol	(Q_1Q_0)	$W = 0$	$W = 1$	NS	EW
S_0	00	01 (S_1)	10 (S_2)	0	1
S_1	01	00 (S_0)	11 (S_3)	1	0
S_2	10	01 (S_1)	01 (S_1)	1	1
S_3	11	00 (S_0)	00 (S_0)	1	1

(ii) Transition and Excitation Tables using S-R Flip-Flops

We construct the detailed excitation table using two S-R flip-flops (Q_1, Q_0). The S-R excitation logic is: $(0 \rightarrow 0) \Rightarrow 0X$, $(0 \rightarrow 1) \Rightarrow 10$, $(1 \rightarrow 0) \Rightarrow 01$, $(1 \rightarrow 1) \Rightarrow X0$.

Present State & Input			Next State		Flip-Flop Inputs				Outputs	
Q_1	Q_0	W	Q_1^*	Q_0^*	S_1	R_1	S_0	R_0	NS	EW
0	0	0	0	1	0	X	1	0	0	1
0	0	1	1	0	1	0	0	X	0	1
0	1	0	0	0	0	X	0	1	1	0
0	1	1	1	1	1	0	X	0	1	0
1	0	0	0	1	0	1	1	0	1	1
1	0	1	0	1	0	1	1	0	1	1
1	1	0	0	0	0	1	0	1	1	1
1	1	1	0	0	0	1	0	1	1	1

Boolean Logic Minimization (Karnaugh Maps)

Applying Karnaugh Maps to the table above yields the following minimizations.

Map for $S_1 = \Sigma m(1, 3)$

Q_0W

	00	01	11	10
0		1	1	
1				

$S_1 = Q_1'W$

Map for $R_1 = \Sigma m(4, 5, 6, 7) + d(0, 2)$

Q_0W

	00	01	11	10
0	-			-
1	1	1	1	1

$R_1 = Q_1$

Map for $S_0 = \Sigma m(0, 4, 5) + d(3)$

Q_0W

	00	01	11	10
0	1		-	
1	1	1		

$S_0 = Q_1Q_0' + Q_0W'$
 $S_0 = Q_0'(Q_1 + W')$

Map for $R_0 = \Sigma m(2, 6, 7) + d(1)$

Q_0W

	00	01	11	10
0		-		1
1			1	1

$R_0 = Q_1Q_0 + Q_0W'$
 $R_0 = Q_0(Q_1 + W')$

Map for Output NS = $\Sigma m(2, 3, 4, 5, 6, 7)$

Q_0W

	00	01	11	10
0			1	1
1	1	1	1	1

$NS = Q_1 + Q_0$

Map for Output EW = $\Sigma m(0, 1, 4, 5, 6, 7)$

Q_0W

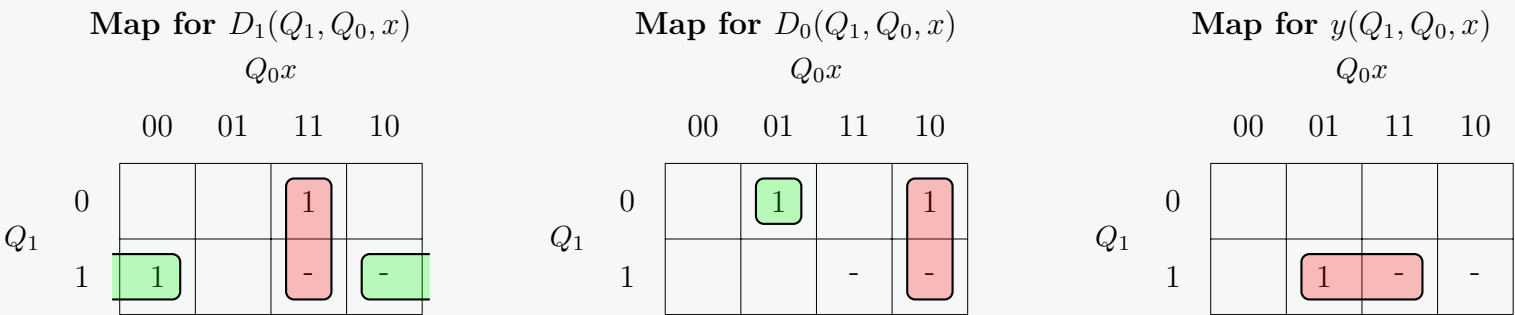
	00	01	11	10
0	1	1		
1	1	1	1	1

$EW = Q_1 + Q_0'$

Current State		Input	Next State (Excitation)		Output
Q_1	Q_0	x	D_1	D_0	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	0	0	1
1	1	0	d	d	d
1	1	1	d	d	d

3. Karnaugh Map Minimization

We map the excitation inputs D_1, D_0 and the output y over variables Q_1, Q_0, x . The unassigned states (110 and 111) are mapped as don't cares (d) to maximize simplification.



Extracting the optimized Boolean equations from the groupings:

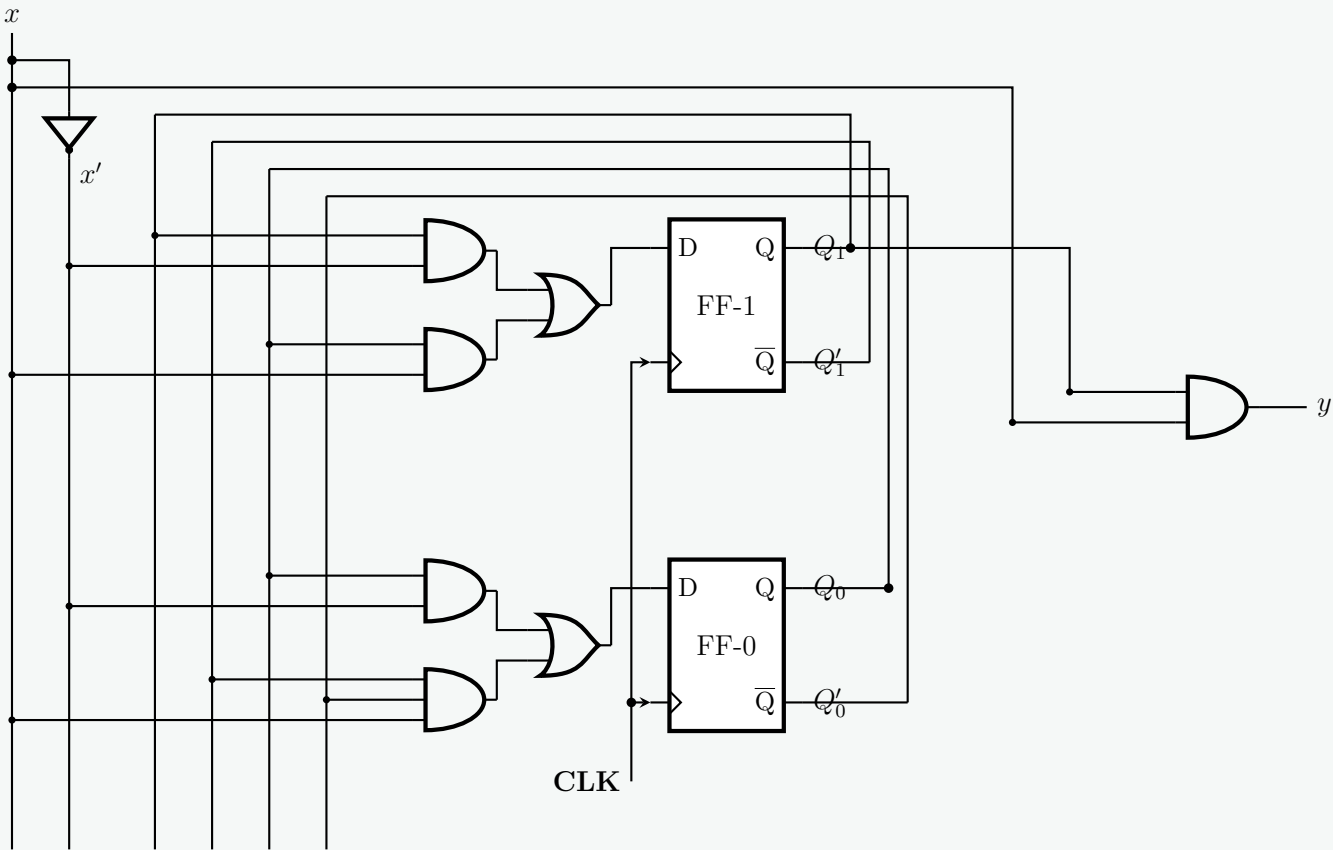
$$D_1 = Q_1x' + Q_0x$$

$$D_0 = Q_1'Q_0'x + Q_0x'$$

$$y = Q_1x$$

4. Logic Circuit Implementation

The finalized schematic combines two D flip-flops triggered by a common clock, with logic gates implementing the simplified sum-of-products. Cross-connections are substituted with node labels for clean, professional wiring.



Q4. Design a control unit with a synchronous sequential circuit for a vending machine that accepts 5 Tk, 10 Tk and 20 Tk notes and releases a soft drink bottle worth 20 Tk. The machine releases a drink whenever the deposit amount equals or exceeds 20 Tk and keeps the change. The maximum deposit at any time is 15 Tk. The machine releases the deposit whenever the ‘CHANGE’ input is given. Outputs: Release Drink Z_1 and Release Change Z_2 . Inputs X_1X_2 : Tk5=00, Tk10=01, Tk20=10, CHANGE=11. Use JK Flip-Flops.

Input	X_1X_2
Tk 5	00
Tk 10	01
Tk 20	10
CHANGE	11

(20 marks)

[CSE 205 | L-2/T-1 | 2020–21]

Answer

1. State Machine Definition and State Diagram

Let the state of the vending machine represent the total accumulated deposit. Since the maximum deposit is 15 Tk (and any input causing the total to reach ≥ 20 Tk immediately dispenses a drink and consumes the deposit), we require exactly four states. Let the states be represented by two flip-flop variables, A and B :

- S_0 (00): 0 Tk deposited.
- S_1 (01): 5 Tk deposited.
- S_2 (10): 10 Tk deposited.
- S_3 (11): 15 Tk deposited.

Inputs (X_1X_2): 00 (5 Tk), 01 (10 Tk), 10 (20 Tk), 11 (CHANGE).
Outputs (Z_1, Z_2): $Z_1 = 1$ (Release Drink), $Z_2 = 1$ (Release Change).

2. State Transition and Excitation Table

Using JK Flip-Flops, the excitation rules are: $(0 \rightarrow 0) \Rightarrow 0d$, $(0 \rightarrow 1) \Rightarrow 1d$, $(1 \rightarrow 0) \Rightarrow d1$, $(1 \rightarrow 1) \Rightarrow d0$.

Present State		Input		Next State		Output		FF Excitation			
A	B	X_1	X_2	$A(t+1)$	$B(t+1)$	Z_1	Z_2	J_A	K_A	J_B	K_B
0	0	0	0	0	1	0	0	0	d	1	d
0	0	0	1	1	0	0	0	1	d	0	d
0	0	1	0	0	0	1	0	0	d	0	d
0	0	1	1	0	0	0	0	0	d	0	d
0	1	0	0	1	0	0	0	1	d	d	1
0	1	0	1	1	1	0	0	1	d	d	0
0	1	1	0	0	0	1	0	0	d	d	1
0	1	1	1	0	0	0	1	0	d	d	1
1	0	0	0	1	1	0	0	d	0	1	d
1	0	0	1	0	0	1	0	d	1	0	d
1	0	1	0	0	0	1	0	d	1	0	d
1	0	1	1	0	0	0	1	d	1	0	d
1	1	0	0	0	0	1	0	d	1	d	1
1	1	0	1	0	0	1	0	d	1	d	1
1	1	1	0	0	0	1	0	d	1	d	1
1	1	1	1	0	0	0	1	d	1	d	1

3. K-Maps and Logic Equations

We apply Karnaugh Maps to derive the simplified Boolean expressions. The grid follows AB on the vertical axis and X_1X_2 on the horizontal axis.

K-Map for J_A

		X_1X_2			
		00	01	11	10
AB	00		1		
	01	1	1		
	11	-	-	-	-
	10	-	-	-	-

K-Map for K_A

		X_1X_2			
		00	01	11	10
AB	00	-	-	-	-
	01	-	-	-	-
	11	1	1	1	1
	10		1	1	1

K-Map for J_B

		X_1X_2			
		00	01	11	10
AB	00	1			
	01	-	-	-	-
	11	-	-	-	-
	10	1			

K-Map for K_B

		X_1X_2			
		00	01	11	10
AB	00	-	-	-	-
	01	1		1	1
	11	1	1	1	1
	10	-	-	-	-

K-Map for Z_1 (Drink)

		X_1X_2			
		00	01	11	10
AB	00				1
	01				1
	11	1	1		1
	10		1		1

K-Map for Z_2 (Change)

		X_1X_2			
		00	01	11	10
AB	00				
	01			1	
	11			1	
	10			1	

Derived Simplified Boolean Equations:

$$J_A = BX'_1 + X'_1X_2 = X'_1(B + X_2)$$

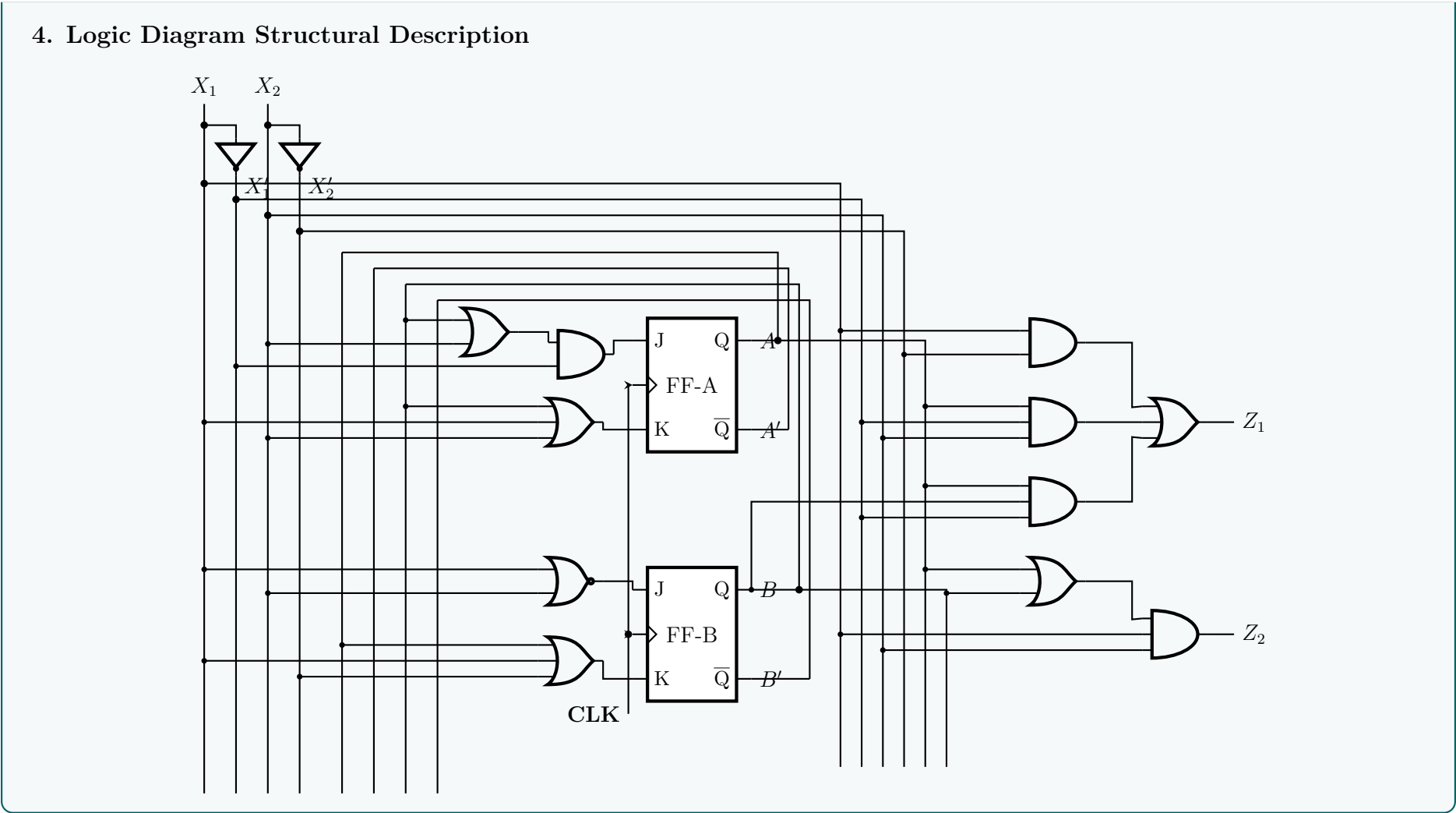
$$K_A = B + X_1 + X_2$$

$$J_B = X'_1X'_2 = (X_1 + X_2)'$$

$$K_B = A + X_1 + X'_2$$

$$Z_1 = X_1X'_2 + AX'_1X_2 + ABX'_1$$

$$Z_2 = BX_1X_2 + AX_1X_2 = X_1X_2(A + B)$$



Q5. Design a one-input, one-output serial 2’s complementer using D flip-flops. The circuit accepts a string of bits from the input and generates the 2’s complement at the output. The circuit can be reset asynchronously to start and end the operation. Include: (i) State diagram, (ii) State table, (iii) Simplified logic equations, (iv) Logic diagram. **(15 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Answer

1. State Diagram

A serial 2’s complementer processes a binary stream from right to left (Least Significant Bit to Most Significant Bit). The mathematical algorithm dictates that all bits remain unchanged up to and including the first occurrence of a 1. Every bit after that first 1 is inverted (complemented).

We define two functional states for this Mealy sequential machine:

- **State S_0 (Initial State):** No 1 has been encountered yet. The circuit copies the input directly to the output.
- **State S_1 (Complementing State):** The first 1 has already been detected. All subsequent incoming bits are inverted.

An asynchronous Reset signal forces the machine back into state S_0 to initiate a new operation.

—

2. State Table

We assign a single state variable Q to represent our states: $S_0 = 0$ and $S_1 = 1$. Since we are designing with a D Flip-Flop, the next-state equation matches the flip-flop input directly ($D = Q(t + 1)$).

Present State (Q)	Input (x)	Next State ($Q(t + 1) = D$)	Output (z)
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	0

—

3. Simplified Logic Equations

We derive the minimal Boolean expressions for the Next-State drive input (D) and the system output (z) using algebraic minimization and verification.

For D Flip-Flop Input (D):

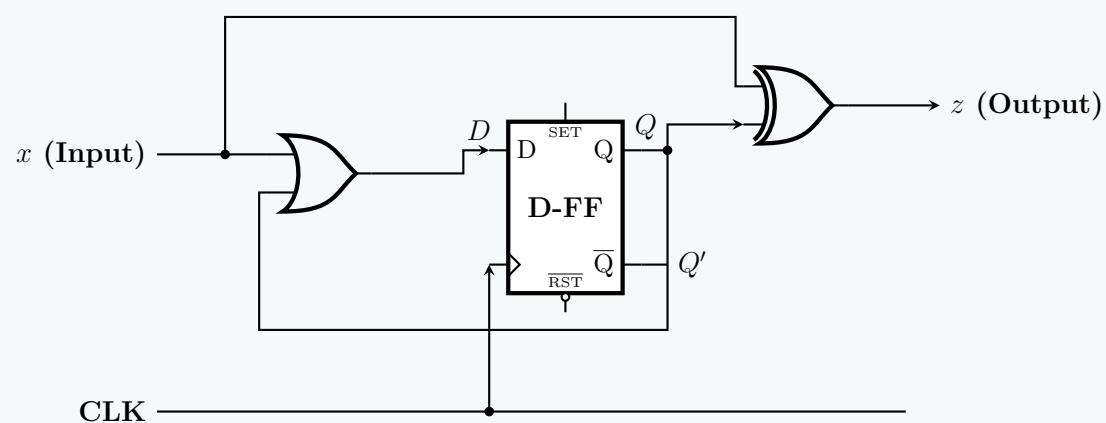
$$\begin{aligned}
 D &= Q'x + Qx' + Qx \\
 &= Q'x + Q(x' + x) && \text{(Distributive Law)} \\
 &= Q'x + Q(1) && \text{(Complementarity Law)} \\
 &= Q'x + Q && \text{(Identity Law)} \\
 &= x + Q && \text{(Absorption / Rule of Complementarity)}
 \end{aligned}$$

For Output (z):

$$\begin{aligned}
 z &= Q'x + Qx' \\
 &= Q \oplus x && \text{(Definition of exclusive-OR)}
 \end{aligned}$$

4. Logic Diagram

The schematic below presents the complete structural architecture of the serial 2's complementer utilizing standard combinational logic elements alongside a single D flip-flop containing an active-low asynchronous reset (CLR).



BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Registers & Counters

Shift Registers, Asynchronous & Synchronous Counters, Ring Counters

S7 Registers & Counters

Ripple (Asynchronous) Counter

Q1. Design the Modulo-100 ripple down counter with JK flip-flops and basic gates. **(6 marks)**

[CSE 205 | L-2/T-1 | 2023–24]

Answer

Design of a Modulo-100 Ripple Down Counter

A Modulo-100 counter cycles through 100 distinct states. In a strict binary implementation, a down counter will decrement from 99_{10} down to 0_{10} and then repeat. To represent the maximum decimal value of 99, we require n flip-flops such that $2^n > 99$. Thus, $n = 7$ flip-flops are required, providing a natural counting range up to 127_{10} .

1. Flip-Flop Configuration

We construct the circuit using seven negative edge-triggered JK flip-flops (FF_0 to FF_6).

- **Toggle Mode:** To enable basic counting, all J and K inputs are tied to Logic 1 ($J_i = K_i = 1$).
- **Ripple Clocking for Down Count:** In a ripple counter, the clock of the first stage (FF_0) is driven by the external clock. For negative edge-triggered flip-flops to function as a *down* counter, the clock input of each subsequent stage (FF_i) must be driven by the inverted output ($\bar{Q}_{i-1}$) of the preceding stage.

2. Truncation Logic (Modulo Operation)

The natural counting sequence of a 7-bit down counter spans from 127_{10} down to 0_{10} . Our target modulo sequence is 99_{10} down to 0_{10} .

Target Maximum State (99_{10}) = 1100011_2

Target Minimum State (0_{10}) = 0000000₂

When the counter reaches 0000000_2 and the next clock pulse arrives, it naturally ripples and rolls over to the maximum 7-bit state:

Transient Rollover State (127_{10}) = 1111111_2

To enforce the Modulo-100 behavior, we must immediately detect this transient rollover state (127_{10}) and asynchronously force the counter back to 99_{10} . By comparing the rollover state and the target maximum state, we can determine the required logic:

- Rollover State ($Q_6Q_5Q_4Q_3Q_2Q_1Q_0$): **1 1 1 1 1 1 1**
- Target Max State ($Q_6Q_5Q_4Q_3Q_2Q_1Q_0$): **1 1 0 0 0 1 1**

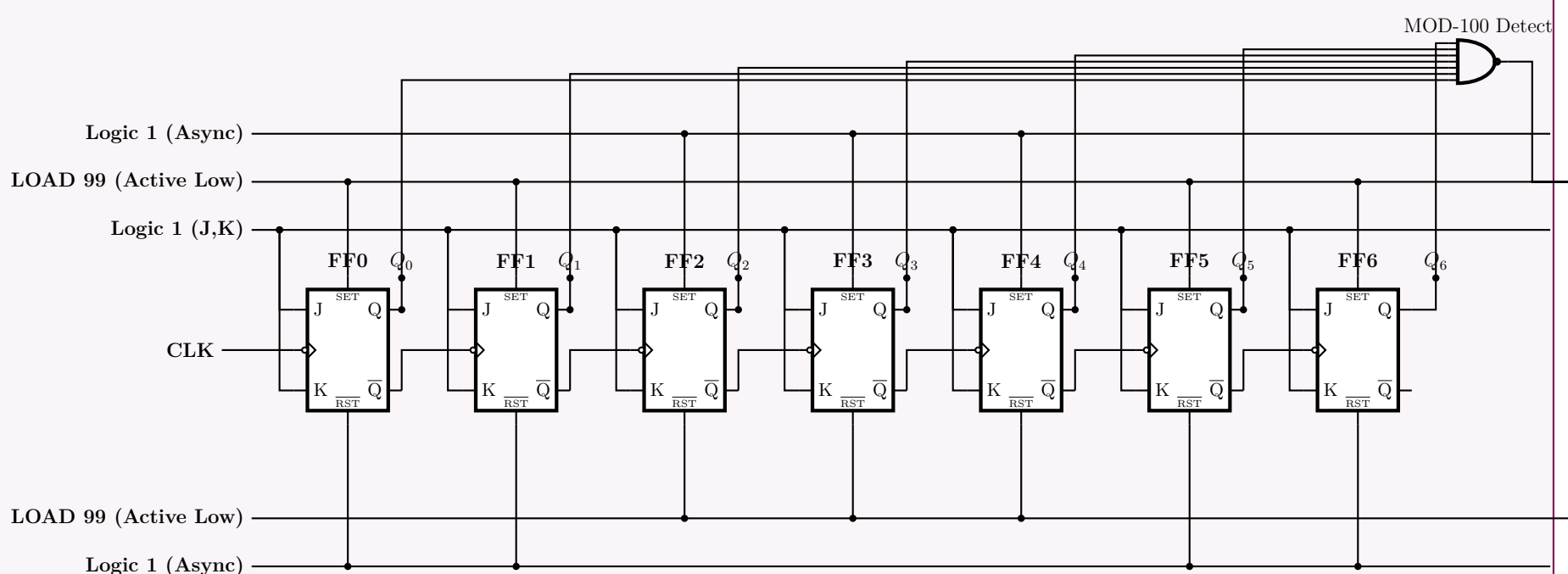
Notice that bits Q_6, Q_5, Q_1 , and Q_0 correctly remain at logic ‘1’ after the rollover. We only need to clear bits Q_4, Q_3 , and Q_2 . We accomplish this by generating an active-low clear signal ($\overline{\text{CLR}}$) that triggers *only* at state 127 using a 7-input NAND gate:

$$\overline{\text{CLR}} = \overline{Q_6 \cdot Q_5 \cdot Q_4 \cdot Q_3 \cdot Q_2 \cdot Q_1 \cdot Q_0} \quad (\text{Applied via a NAND Gate})$$

This $\overline{\text{CLR}}$ signal is connected to the asynchronous active-low clear pins of FF₄, FF₃, and FF₂. The clear pins of the remaining flip-flops are permanently tied to Logic 1 to keep them inactive.

3. Circuit Diagram

The schematic below illustrates the complete 7-bit Mod-100 ripple down counter. To ensure clarity and avoid wire clutter, logical labels are used to denote the connections between the outputs and the truncation NAND gate.



Q2. Show the logic diagram of a four-bit binary ripple (asynchronous) countdown counter using T flip-flops that trigger on the negative edge of the clock. (12 marks) [CSE 205 | L-2/T-1 | 2022–23]

Answer

Four-Bit Binary Ripple Countdown Counter

To design a four-bit binary ripple (asynchronous) countdown counter using T flip-flops that trigger on the negative edge of the clock, we must establish the operational logic for both the flip-flops and the ripple clocking mechanism.

1. Design Principles and Logic Derivation

- Toggle Operation:** A T flip-flop toggles its state (complements its output) upon receiving a valid clock edge if its T input is held HIGH (Logic 1). Therefore, to enable counting, the T inputs of all four flip-flops (FF_0 to FF_3) are permanently tied to **Logic 1**.
- Asynchronous (Ripple) Clocking:** In a ripple counter, only the first flip-flop (the Least Significant Bit, FF_0) is driven by the external clock signal. Each subsequent flip-flop (FF_i) is clocked by an output from the preceding flip-flop (FF_{i-1}).
- Countdown Configuration with Negative-Edge Triggering:** A countdown sequence dictates that a bit must toggle when the next lower significant bit transitions from 0 to 1 (a positive edge). However, our T flip-flops are strictly **negative-edge triggered** (they respond to $1 \rightarrow 0$ transitions).

To achieve a $0 \rightarrow 1$ transition trigger on the main outputs (Q) using negative-edge inputs, we must use the *complemented* output ($\overline{Q}$) of the preceding stage to drive the clock of the next stage. Mathematically:

$$\text{When } Q_i \text{ transitions } 0 \rightarrow 1 \implies \overline{Q}_i \text{ transitions } 1 \rightarrow 0 \text{ (Negative Edge)}$$

Thus, connecting $\overline{Q}_i$ to the clock input of FF_{i+1} forces FF_{i+1} to toggle precisely when Q_i goes from 0 to 1, correctly executing a down-count.

2. Logic Diagram

Below is the complete schematic for the four-bit asynchronous countdown counter. The external clock is applied to FF_0 , and the inverted outputs ($\overline{Q}_0, \overline{Q}_1, \overline{Q}_2$) form the ripple clock sequence for the subsequent stages.

3. State Transition Validation

To verify the countdown logic, consider the counter initialized at state 0000_2 (0_{10}). Let us trace the first clock pulse:

Clock Pulse	Q_3	Q_2	Q_1	Q_0	Decimal Value
Initial State	0	0	0	0	0
1st Falling Edge	1	1	1	1	15
2nd Falling Edge	1	1	1	0	14
3rd Falling Edge	1	1	0	1	13

Step-by-step trace of Pulse 1 ($0000 \rightarrow 1111$):

- The falling edge of the main clock hits FF_0 . Because $T = 1$, FF_0 toggles. Q_0 transitions $0 \rightarrow 1$.
- Because Q_0 goes $0 \rightarrow 1$, $\overline{Q}_0$ goes $1 \rightarrow 0$. This provides a negative edge to the clock of FF_1 .
- FF_1 receives the negative edge and toggles. Q_1 transitions $0 \rightarrow 1$.
- Because Q_1 goes $0 \rightarrow 1$, $\overline{Q}_1$ goes $1 \rightarrow 0$, triggering FF_2 .
- FF_2 toggles, making Q_2 transition $0 \rightarrow 1$, which subsequently triggers FF_3 .
- FF_3 toggles, making Q_3 transition $0 \rightarrow 1$.
- The final state immediately after the first clock pulse is $Q_3Q_2Q_1Q_0 = 1111_2$ (15_{10}), confirming accurate down-counting modulo-16 operation.

Q3. A MOD-13 counter counts from 0000 to 1100. Design the MOD-13 ripple up/down counter with positive edge-triggered JK flip-flops

and basic gates. (5 marks)

[CSE 205 | L-2/T-1 | 2021–22]

Answer

Design of a MOD-13 Ripple Up/Down Counter

A Modulo-13 counter possesses 13 valid states. To cover 13 states, we require $n = 4$ flip-flops ($2^4 = 16$). The valid counting sequence is from 0000_2 (0_{10}) to 1100_2 (12_{10}).

1. Up/Down Clocking Logic (Positive Edge-Triggered)

We define a mode control signal M :

- $M = 0$ (**UP Counting**): A flip-flop must toggle when the preceding flip-flop transitions from $1 \rightarrow 0$. Because we are using **positive edge-triggered** JK flip-flops (which toggle on $0 \rightarrow 1$), we must drive the clock of the next stage with the **inverted output** ($\bar{Q}_i$) of the current stage.
- $M = 1$ (**DOWN Counting**): A flip-flop must toggle when the preceding flip-flop transitions from $0 \rightarrow 1$. Since the flip-flops trigger on $0 \rightarrow 1$, we drive the clock of the next stage directly with the **normal output** (Q_i) of the current stage.

To combine these modes, we use combinational multiplexing logic for the clock input of each subsequent flip-flop (CLK_{i+1}):

$$CLK_{i+1} = (UP\ Mode \cdot \bar{Q}_i) + (DOWN\ Mode \cdot Q_i)$$
$$CLK_{i+1} = (\bar{M} \cdot \bar{Q}_i) + (M \cdot Q_i)$$

All JK flip-flops are set to toggle mode by holding $J = K = 1$.

2. Modulo-13 Truncation Logic

Ripple counters naturally count from 0 to 15. To restrict this to a Mod-13 sequence (0 to 12), we must asynchronously clear or preset the flip-flops when they hit an out-of-bounds state.

UP Counting Truncation ($M = 0$): The counter sequence is $0, 1, \dots, 12$. The transient rollover state is 13_{10} (1101_2). Upon reaching 13, the counter must immediately reset to 0_{10} (0000_2).

$$Reset_{UP} = \bar{M} \cdot Q_3 \cdot Q_2 \cdot Q_0 \quad (\text{Triggered when state is } 1101_2 \text{ and } M = 0)$$

DOWN Counting Truncation ($M = 1$): The counter sequence is $12, 11, \dots, 0$. When counting down from 0, the natural ripple behavior rolls under to 15_{10} (1111_2). Upon reaching 15, we must immediately preset the counter to 12_{10} (1100_2).

$$Set_{DOWN} = M \cdot Q_3 \cdot Q_2 \cdot Q_1 \cdot Q_0 \quad (\text{Triggered when state is } 1111_2 \text{ and } M = 1)$$

Asynchronous Signal Equations: Using active-low Preset ($\overline{PRE}$) and Clear ($\overline{CLR}$) pins, we formulate the necessary signals for each flip-flop.

Flip-Flop	Target Reset State	$\overline{CLR}_i$ Equation (Active Low)	$\overline{PRE}_i$ Equation (Active Low)
FF_3 (Q_3)	0 on UP, 1 on DOWN	$\overline{Reset}_{UP}$	$\overline{Set}_{DOWN}$
FF_2 (Q_2)	0 on UP, 1 on DOWN	$\overline{Reset}_{UP}$	$\overline{Set}_{DOWN}$
FF_1 (Q_1)	0 on UP, 0 on DOWN	$\overline{Reset}_{UP} + \overline{Set}_{DOWN}$	1 (Inactive)
FF_0 (Q_0)	0 on UP, 0 on DOWN	$\overline{Reset}_{UP} + \overline{Set}_{DOWN}$	1 (Inactive)

3. Structural Logic Diagram

Due to the wiring complexity of dual-direction modulo truncation, the control logic routing is represented as multiplexers between the flip-flops, and the truncation logic is elegantly encapsulated in a dedicated combinational block.

Q4. Design a BCD ripple down counter using D flip-flops. (5 marks)

[CSE 205 | L-2/T-1 | 2020–21, 2018–19]

Answer

Design of a BCD Ripple Down Counter Using D Flip-Flops

A BCD (Binary-Coded Decimal) counter is a Modulo-10 counter that counts through the 10 valid BCD states. For a down counter, the sequence is from 9 (1001_2) down to 0 (0000_2). Once it reaches 0, the next clock pulse must cause it to cycle back to 9.

1. Flip-Flop Configuration and Ripple Logic

To construct an asynchronous (ripple) counter using **Positive-Edge Triggered D Flip-Flops**, we must first configure each flip-flop to act in toggle mode.

- **Toggle Mode:** By connecting the inverted output ($\overline{Q}_i$) back to the data input (D_i), the flip-flop toggles its state upon every active clock edge:

$$D_i = \overline{Q}_i$$

- **Down-Count Clocking:** In a ripple counter, the clock of FF_i is driven by the output of FF_{i-1} . For a down counter using *positive-edge* triggered flip-flops, a toggle must happen when the preceding bit transitions from $0 \rightarrow 1$. Therefore, we connect the **normal output** (Q_i) to the clock of the next stage:

$$CLK_0 = \text{External Clock}$$

$$CLK_i = Q_{i-1} \quad \text{for } i \in \{1, 2, 3\}$$

2. Modulo-10 Truncation Logic

A 4-bit ripple down counter naturally rolls under from 0000_2 to 1111_2 (15_{10}). To enforce the BCD sequence, we must detect this out-of-bounds transient state and immediately force the counter to 1001_2 (9_{10}).

- **Transient State:** 1111_2 ($Q_3 = 1, Q_2 = 1, Q_1 = 1, Q_0 = 1$)
- **Target State:** 1001_2 ($Q_3 = 1, Q_2 = 0, Q_1 = 0, Q_0 = 1$)

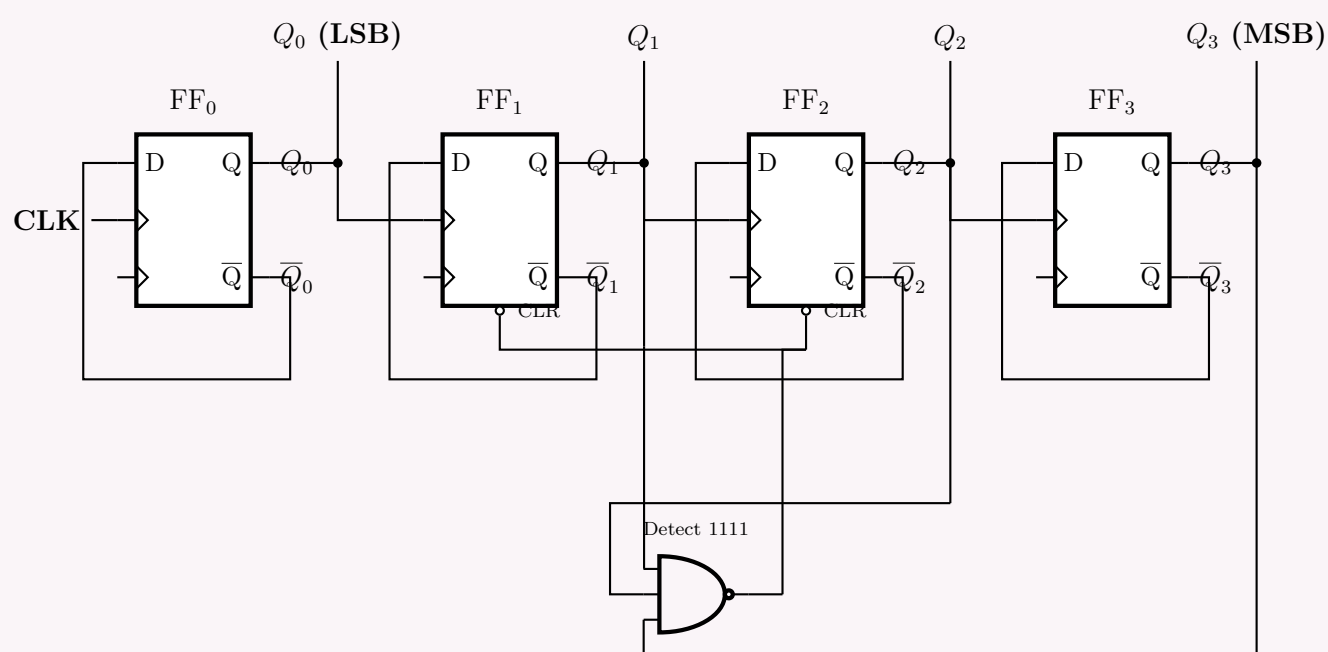
To transition from $1111 \rightarrow 1001$, we only need to clear Q_2 and Q_1 . The Q_3 and Q_0 bits are already correct (Logic 1). We can isolate the 1111 state by detecting when Q_3, Q_2 , and Q_1 are simultaneously high (state 7 is 0111 , so including Q_3 differentiates 15 from 7). Using an active-low clear (CLR), the truncation logic requires a NAND gate:

$$\overline{CLR}_{1,2} = \overline{Q_3 \cdot Q_2 \cdot Q_1}$$

When the state hits 1111_2 , the NAND output drops to 0, asynchronously clearing FF_2 and FF_1 . The moment they become 0, the NAND output reverts to 1, allowing counting to resume. *Note: Clearing Q_2 and Q_1 causes negative edges ($1 \rightarrow 0$) on the subsequent clocks. Because we use positive-edge triggered D-FFs, these negative edges are safely ignored, avoiding false ripples.*

3. Logic Diagram

Below is the structured schematic of the BCD Ripple Down Counter incorporating the D-FF toggle loops, ripple clocking, and the NAND truncation logic.



4. State Transition Table

The following trace confirms the natural down-count and modulo-10 reset functionality.

Clock Pulse	Q_3	Q_2	Q_1	Q_0	State (Decimal)
Initial	1	0	0	1	9
1	1	0	0	0	8
2	0	1	1	1	7
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
8	0	0	0	1	1
9	0	0	0	0	0
10	1	1	1	1	15 (Transient)
	1	0	0	1	9 (Truncated Output)

Q5. Design a 4-bit modulo-12 asynchronous up counter using JK Flip-Flops. (5 marks)

[CSE 205 | L-2/T-1 | 2017–18]

Answer

Design of a 4-Bit Modulo-12 Asynchronous Up Counter

A Modulo-12 (Mod-12) counter cycles through 12 distinct states, counting from 0000_2 (0_{10}) to 1011_2 (11_{10}). Because the maximum state is 11, we require $n = 4$ flip-flops ($2^3 < 12 \leq 2^4$). We will implement this using negative-edge triggered JK flip-flops.

1. Flip-Flop Configuration and Ripple Logic

To construct an asynchronous (ripple) up-counter:

- Toggle Mode:** Each JK flip-flop must complement its state upon receiving a valid clock pulse. Therefore, the J and K inputs of all flip-flops are permanently tied to **Logic 1** ($J = K = 1$).
- Up-Count Clocking:** Using negative-edge triggered flip-flops (toggling on a $1 \rightarrow 0$ transition), an up-count is achieved by driving the clock of the next stage (CLK_{i+1}) with the **normal output** (Q_i) of the current stage.

$$CLK_0 = \text{External Clock}$$
$$CLK_i = Q_{i-1} \quad \text{for } i \in \{1, 2, 3\}$$

2. Modulo-12 Truncation Logic

A standard 4-bit ripple counter naturally counts up to 15_{10} . To restrict the sequence to a modulo-12 count, we must identify the first out-of-bounds state and use it to asynchronously clear all flip-flops.

- Valid Sequence:** 0_{10} (0000_2) to 11_{10} (1011_2).
- Transient State:** The counter temporarily enters 12_{10} (1100_2) before immediately resetting.

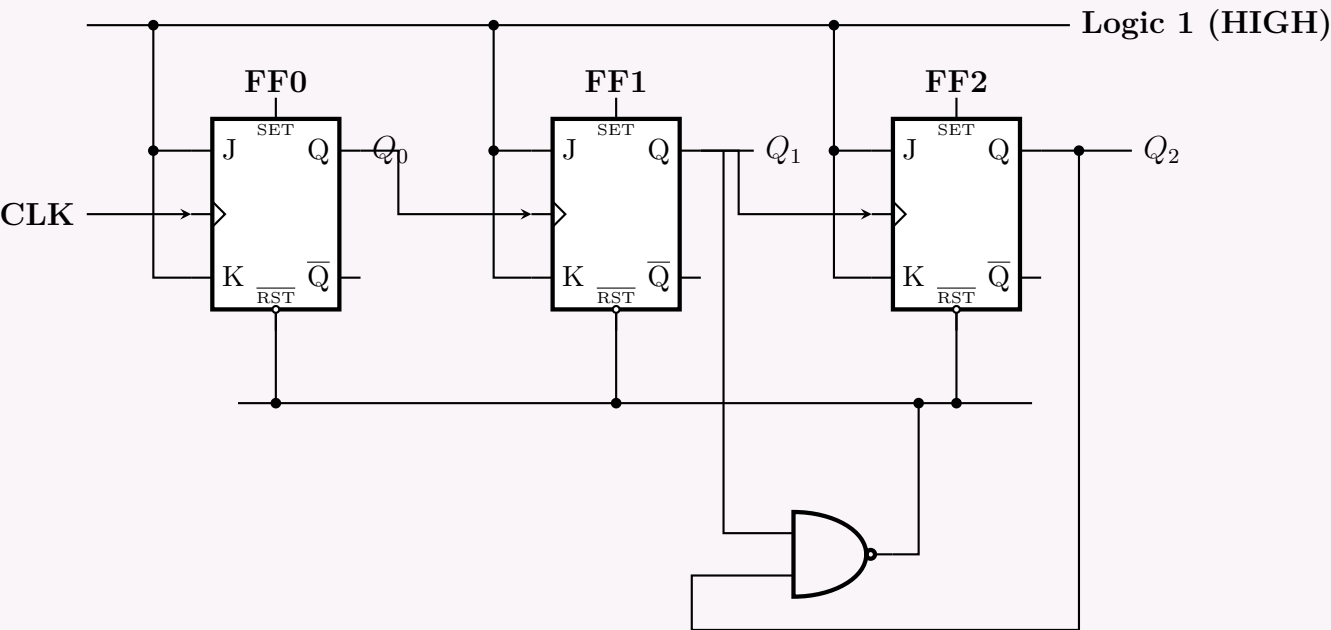
At state 12 ($Q_3 = 1, Q_2 = 1, Q_1 = 0, Q_0 = 0$), this is the very first time in the counting sequence where both Q_3 and Q_2 are **HIGH** simultaneously. We can detect this specific condition using a NAND gate to drive the active-low $\overline{\text{CLR}}$ pins of the flip-flops.

$$\overline{\text{CLR}} = \overline{Q_3 \cdot Q_2} \quad (\text{Applies De Morgan's Law internally: becomes 0 only when } Q_3 = 1, Q_2 = 1)$$

When the counter hits 1100_2 , the NAND output drops to 0, clearing all flip-flops to 0000_2 . As soon as the flip-flops clear, Q_3 and Q_2 become 0, the NAND output returns to 1, and normal counting resumes.

3. Logic Diagram

Below is the schematic for the Mod-12 counter. The asynchronous clear bus spans the bottom, driven by the truncation NAND gate.



4. State Transition Table

The sequence demonstrates the standard binary progression until the transient 12th state is decoded, forcing the reset.

Clock Pulse	Q_3	Q_2	Q_1	Q_0	State (Decimal)
Initial	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	2
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
10	1	0	1	0	10
11	1	0	1	1	11
12	1	1	0	0	12 (Transient - NAND active)
	0	0	0	0	0 (Truncated Output)

Q6. A MOD-12 counter counts from 0000 to 1011. Design the MOD-12 ripple counter with positive edge-triggered JK Flip-Flops and basic gates. **(10 marks)**
[CSE 205 | L-2/T-1 | 2016–17]
↔ Similar: 2019–20

Answer

Design of a MOD-12 Ripple Up Counter

A Modulo-12 (MOD-12) counter counts through 12 unique states, starting from 0000_2 (0_{10}) up to 1011_2 (11_{10}). Since the highest decimal value is 11, we require $n = 4$ flip-flops ($2^3 < 12 \leq 2^4$). We will implement this design utilizing **positive edge-triggered** JK flip-flops.

1. Flip-Flop Configuration and Ripple Logic

To construct an asynchronous (ripple) up counter:

- **Toggle Mode Setup:** Each JK flip-flop must toggle its stored state upon receiving a valid clock edge. Therefore, the J and K inputs of all four flip-flops are permanently tied to **Logic 1 (HIGH)** ($J = K = 1$).
- **Clocking (Up-Count with Positive-Edge FFs):** We are constrained to use positive edge-triggered flip-flops (which toggle on a $0 \rightarrow 1$ transition). For an up-counter, FF_{i+1} must toggle when FF_i transitions from $1 \rightarrow 0$. Because our flip-flops respond to $0 \rightarrow 1$ transitions, we must drive the clock of the next stage using the **inverted output** ($\overline{Q}_i$) of the current stage.

$$CLK_0 = \text{External Clock}$$
$$CLK_i = \overline{Q}_{i-1} \quad \text{for } i \in \{1, 2, 3\}$$

2. Modulo-12 Truncation Logic

A 4-bit ripple counter normally counts up to 15_{10} . To constrain the sequence to a modulo-12 count, we must identify the first out-of-bounds state and use it to asynchronously reset all flip-flops.

- **Valid Sequence:** 0_{10} (0000_2) to 11_{10} (1011_2).
- **Transient State:** The counter momentarily enters 12_{10} (1100_2) before being immediately cleared back to 0000_2 .

At state 12 ($Q_3 = 1, Q_2 = 1, Q_1 = 0, Q_0 = 0$), this is the very first time in the normal counting sequence where both Q_3 and Q_2 are **HIGH** simultaneously. We decode this specific condition using a 2-input NAND gate to drive the active-low clear ($\overline{CLR}$) pins of

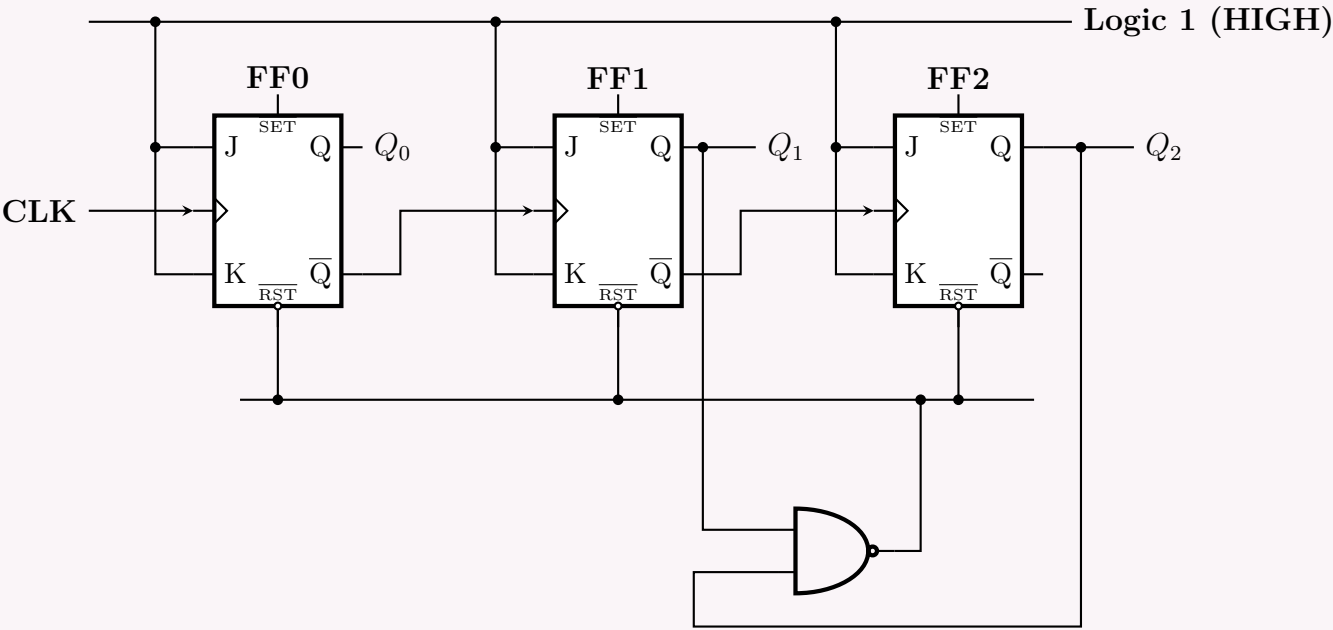
the flip-flops.

$\overline{\text{CLR}} = \overline{Q_3 \cdot Q_2}$ (Applying De Morgan's Law: $\overline{\text{CLR}}$ drops to 0 only when $Q_3 = 1, Q_2 = 1$)

When the counter hits 1100_2 , the NAND output goes low, forcing all flip-flops to 0000_2 . Once cleared, Q_3 and Q_2 return to 0, the NAND output returns to 1, and normal counting resumes safely.

3. Logic Diagram

Below is the schematic for the MOD-12 ripple up counter. Note the clock routing from $\overline{Q}$ and the asynchronous reset logic tapping from Q_3 and Q_2 .



4. State Transition Table

The following table highlights the standard binary progression and the intervention of the truncation logic at state 12.

Clock Pulse	Q_3	Q_2	Q_1	Q_0	State (Decimal)
Initial	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	2
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
10	1	0	1	0	10
11	1	0	1	1	11
12	1	1	0	0	12 (Transient - NAND active)
	0	0	0	0	0 (Truncated Base State)

Q7. Design a 4-bit Ripple counter using D Flip-Flops and briefly explain its operation. (10 marks) [CSE 205 | L-2/T-1 | 2014–15]

Answer

Design of a 4-Bit Ripple Up Counter Using D Flip-Flops

A 4-bit ripple counter is an asynchronous counter that can count up to $2^4 = 16$ distinct states, from 0000_2 (0_{10}) to 1111_2 (15_{10}). In a ripple counter, only the first flip-flop receives the external clock; subsequent flip-flops are clocked by the output of the preceding stage, causing the clock signal to "ripple" through the circuit.

1. Configuration of D Flip-Flops for Counting

To function as a counter, each flip-flop must complement (toggle) its state upon receiving a valid clock edge.

- Toggle Mode via D-FF:** A Data (D) Flip-Flop can be converted into a Toggle (T) Flip-Flop by continuously feeding its inverted output ($\overline{Q}$) back into its data input (D).

$$D_i = \overline{Q_i} \quad \text{for } i \in \{0, 1, 2, 3\}$$

By setting $D = \overline{Q}$, whenever the flip-flop is clocked, it loads the complement of its current state, thereby toggling.

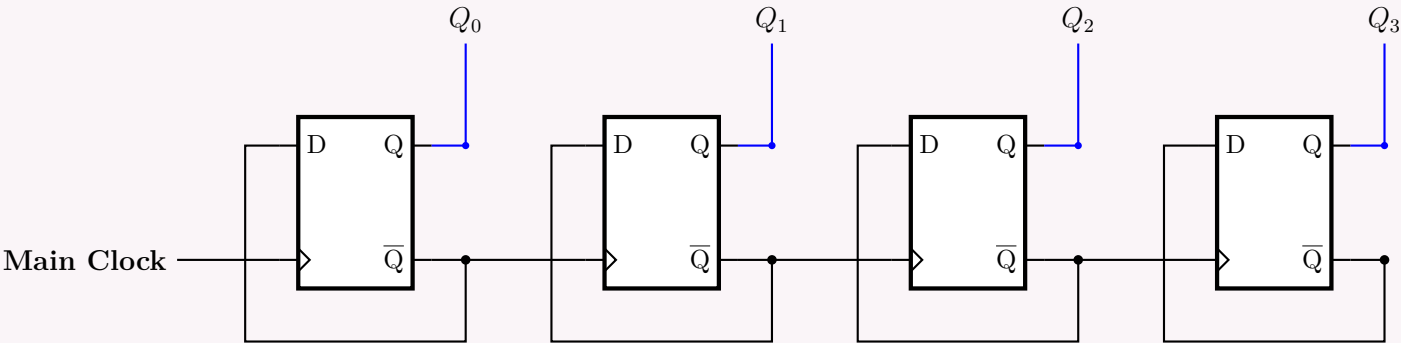
- Ripple Clocking Logic:** We will use **positive-edge triggered** D Flip-Flops (toggling on a $0 \rightarrow 1$ transition). In an up-counter, the i -th bit must toggle when the $(i - 1)$ -th bit transitions from $1 \rightarrow 0$. When Q_{i-1} goes $1 \rightarrow 0$, its complement $\overline{Q}_{i-1}$ goes $0 \rightarrow 1$ (a positive edge). Therefore, to cascade the counter correctly, we drive the clock of the next stage with the

inverted output ($\overline{Q}$) of the previous stage.

$$CLK_0 = \text{External Clock (CLK)}$$
$$CLK_i = \overline{Q}_{i-1} \quad \text{for } i \in \{1, 2, 3\}$$

2. Logic Diagram

Below is the schematic for the 4-bit asynchronous ripple up-counter using positive-edge triggered D Flip-Flops. Notice the feedback loops creating the toggle functionality and the daisy-chained clocks.



3. Operation Explanation & State Table

Step-by-Step Operation:

- (a) **Initial State:** Assume the counter starts at 0000_2 . All Q outputs are 0, and all $\overline{Q}$ outputs are 1.
- (b) **First Clock Pulse:** The external clock applies a positive edge to FF_0 . FF_0 toggles, so Q_0 becomes 1, and $\overline{Q}_0$ becomes 0. The transition of $\overline{Q}_0$ from $1 \rightarrow 0$ is a negative edge. Since FF_1 is positive-edge triggered, it ignores this change and remains 0. State: 0001_2 .
- (c) **Second Clock Pulse:** Another positive edge hits FF_0 . FF_0 toggles again; Q_0 becomes 0, and $\overline{Q}_0$ becomes 1. The transition of $\overline{Q}_0$ from $0 \rightarrow 1$ creates a positive edge at the clock of FF_1 . This triggers FF_1 to toggle, changing Q_1 to 1 and $\overline{Q}_1$ to 0. State: 0010_2 .
- (d) **Ripple Effect:** This pattern continues. A flip-flop toggles only when the preceding flip-flop flips from 1 to 0 (causing $\overline{Q}$ to provide the required positive edge). The delay cascades linearly down the chain, giving the "ripple" its name.

Clock Pulse	Q_3 (MSB)	Q_2	Q_1	Q_0 (LSB)
0 (Init)	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
14	1	1	1	0
15	1	1	1	1
16 (Recycle)	0	0	0	0

Truth Table demonstrating the standard binary up-count sequence.

- Q8. (a) Design a modulo-6 3-bit asynchronous (ripple) up counter with positive edge-triggered JK flip-flops.
- (b) Show the timing diagram for the above designed counter.
- (c) What are the problems of the above counter?

(8+4+3 marks)

[CSE 205 | L-2/T-1 | 2012–13]

Answer

1. Design of Modulo-6 Asynchronous Up Counter

A modulo-6 (Mod-6) counter cycles through 6 distinct valid states, from 000_2 (0_{10}) to 101_2 (5_{10}). Since $2^2 < 6 \leq 2^3$, we require $n = 3$ flip-flops. The design is implemented using positive edge-triggered JK flip-flops.

A. Flip-Flop Configuration & Ripple Clocking

- **Toggle Mode:** For an asynchronous counter, flip-flops must be configured in toggle mode. We connect the J and K inputs of all flip-flops to **Logic 1 (HIGH)** ($J = K = 1$).

- **Positive-Edge Clock Routing:** An up-counter must increment its next stage when the current stage transitions from 1 $\rightarrow$ 0. Because we are using positive edge-triggered flip-flops (which toggle on a 0 $\rightarrow$ 1 edge), we must drive the clock of the next stage using the **inverted output** ($\overline{Q}$) of the current stage. When Q drops 1 $\rightarrow$ 0, $\overline{Q}$ rises 0 $\rightarrow$ 1, providing the correct triggering edge.

$$CLK_0 = \text{External Clock}$$

$$CLK_1 = \overline{Q_0}$$

$$CLK_2 = \overline{Q}_1$$

B. Modulo-6 Truncation Logic

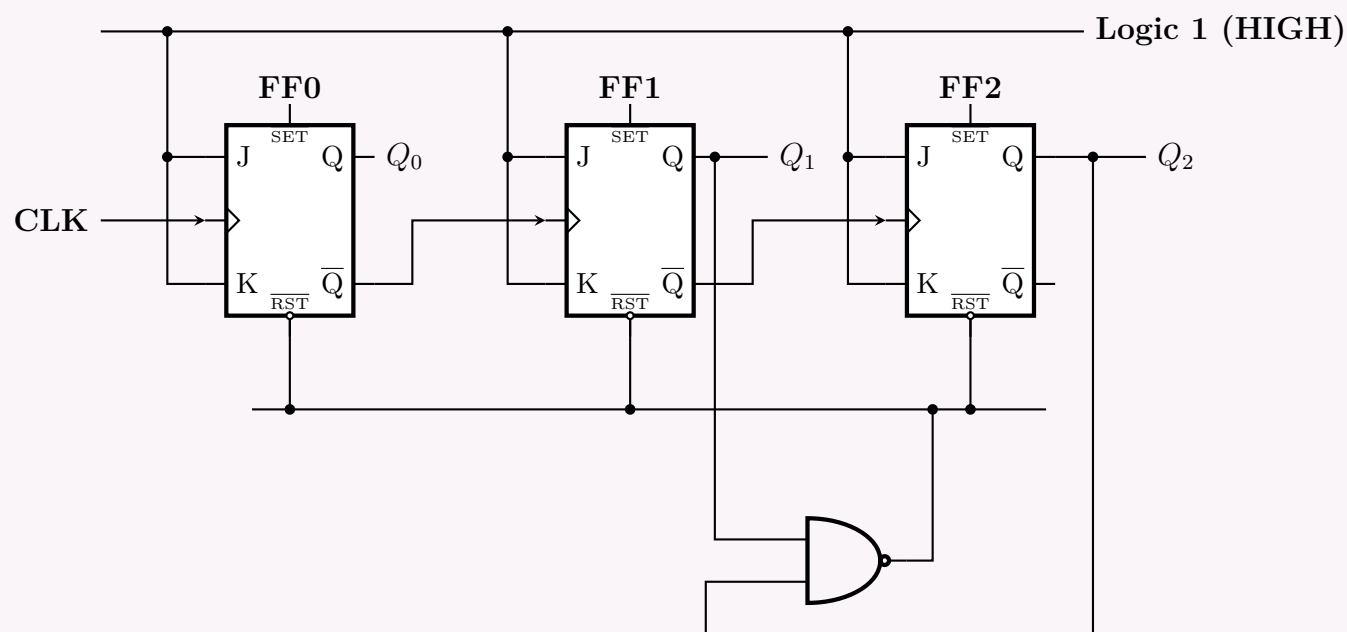
A 3-bit ripple counter naturally counts up to 7_{10} (111_2). To enforce a modulo-6 sequence, we must detect the first invalid state and use it to asynchronously clear the flip-flops.

- **Valid Sequence:** 000_2 to 101_2 (0 to 5).
- **Transient State:** 110_2 (6_{10}), where $Q_2 = 1, Q_1 = 1, Q_0 = 0$.

At state 6, it is the first time in the sequence that both Q_2 and Q_1 are HIGH simultaneously. We use a 2-input NAND gate to detect this and drive the active-low clear ($\overline{\text{CLR}}$) pins.

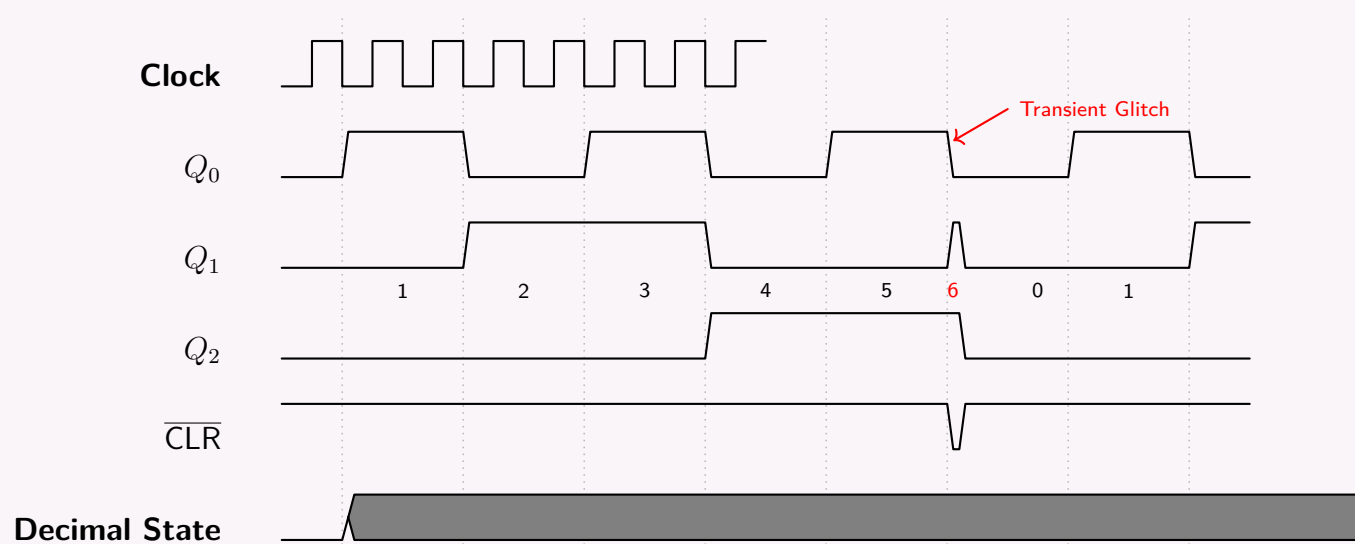
$$\overline{\text{CLR}} = \overline{Q_2 \cdot Q_1}$$

When the counter reaches 110_2 , the NAND output drops to 0, immediately clearing the flip-flops back to 000_2 .



2. Timing Diagram

The timing diagram illustrates the counting sequence and the glitch at the 6th clock pulse. Note that transitions in Q_1 and Q_2 are delayed relative to the main clock because they are triggered by the preceding flip-flops (ripple effect). The brief transient state 110_2 forces $\overline{\text{CLR}}$ low, resetting the counter.



3. Problems of the Asynchronous (Ripple) Counter

- (a) **Cumulative Propagation Delay:** In an asynchronous counter, the clock signal ripples through the flip-flops. The delay of each flip-flop (t_{pd}) adds up. The maximum operational frequency is limited by the worst-case cumulative delay: $T_{clk} \geq n \times t_{pd}$. If the clock speed is too high, the counter will fail to settle before the next pulse.
- (b) **Output Decoding Glitches (Transient States):** As seen in the timing diagram, truncated asynchronous counters output brief invalid states (such as 110_2) before the asynchronous clear logic resets them. These voltage spikes (glitches) can cause erratic behavior or false triggering in any downstream logic or external circuits reading the counter's state.
- (c) **Asynchronous Reset Unreliability:** The reset mechanism relies on the propagation delay of the NAND gate and the internal reset logic of the flip-flops. If flip-flops reset at slightly different speeds, a "partial reset" can occur, causing the truncation gate to deactivate prematurely and leaving some flip-flops stuck in high states.

Q9. Draw and explain a four-bit binary count-up ripple counter with T flip-flops. (12 marks)

[CSE 205 | L-2/T-1 | 2011–12]

Answer

Four-Bit Binary Up Ripple Counter Using T Flip-Flops

A 4-bit binary ripple counter is an asynchronous sequential logic circuit capable of counting through $2^4 = 16$ distinct states, progressing from 0000_2 (0_{10}) up to 1111_2 (15_{10}). It is called a "ripple" counter because the external clock is applied only to the first flip-flop; the clock signal for each subsequent stage is derived from the output of the preceding stage, causing the state change to propagate (or ripple) down the chain.

1. Design Principles and Configuration

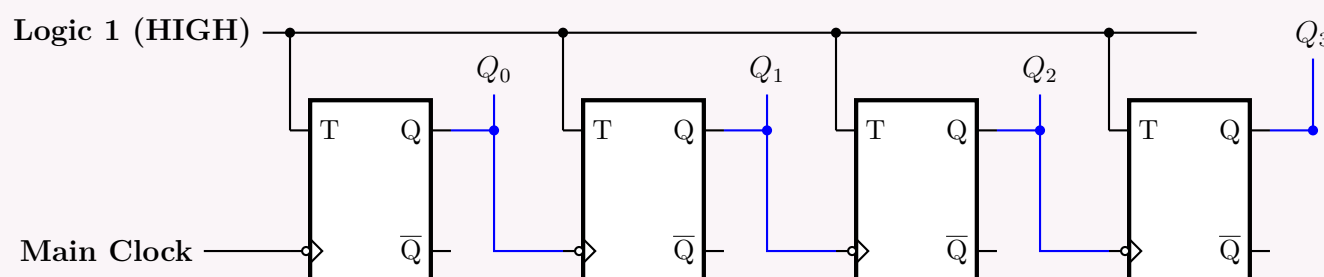
- **Toggle Mode Operation:** To function as a counter, each flip-flop must complement (toggle) its stored state each time it receives a valid clock edge. For a Toggle (T) flip-flop, this condition is met by permanently connecting the T input of all flip-flops to **Logic 1 (HIGH)** ($T = 1$).
- **Edge Triggering:** We will construct this up-counter using **negative-edge triggered** T flip-flops. These flip-flops toggle their state only when their clock input transitions from HIGH to LOW ($1 \rightarrow 0$).
- **Ripple Cascading Logic:** In binary counting, a bit transitions from $0 \rightarrow 1$ (toggles) exactly when the next lower significant bit resets from $1 \rightarrow 0$. Because our flip-flops are negative-edge triggered, they naturally respond to a $1 \rightarrow 0$ transition. Thus, to build an up-counter, we must drive the clock input of each subsequent flip-flop (CLK_{i+1}) directly with the **normal output** (Q_i) of the current flip-flop.

$$CLK_0 = \text{Main External Clock}$$

$$CLK_i = Q_{i-1} \quad \text{for } i \in \{1, 2, 3\}$$

2. Logic Diagram

Below is the structured schematic for the 4-bit asynchronous ripple counter. Notice the continuous logic high on all T pins and the asynchronous clock cascading via the Q outputs.



3. Step-by-Step Operation

The counter follows a strict sequential progression based on edge triggering:

- (a) **Initial State:** Assume the counter is globally reset such that $Q_3Q_2Q_1Q_0 = 0000_2$.
- (b) **Pulse 1 ($0 \rightarrow 1$):** The main clock generates a negative edge ($1 \rightarrow 0$) entering FF_0 . FF_0 toggles, changing Q_0 from $0 \rightarrow 1$. Because Q_0 underwent a positive edge, FF_1 (which needs a negative edge) is NOT triggered. The state becomes 0001_2 .
- (c) **Pulse 2 ($1 \rightarrow 2$):** The main clock generates another negative edge. FF_0 toggles again, changing Q_0 from $1 \rightarrow 0$. This $1 \rightarrow 0$ transition creates the necessary negative edge at the clock of FF_1 . Consequently, FF_1 toggles ($Q_1 : 0 \rightarrow 1$). The state becomes 0010_2 .
- (d) **Ripple Effect:** This cascading chain reaction continues. A flip-flop FF_{i+1} will only toggle its state at the precise moment that the preceding flip-flop FF_i drops from 1 to 0. At the 16th clock pulse, Q_0, Q_1, Q_2 , and Q_3 will all cascade from $1 \rightarrow 0$, recycling the counter back to 0000_2 .

Clock Pulse	Q_3 (MSB)	Q_2	Q_1	Q_0 (LSB)	State (Decimal)
0 (Initial)	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	2
3	0	0	1	1	3
4	0	1	0	0	4
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
14	1	1	1	0	14
15	1	1	1	1	15
16 (Recycle)	0	0	0	0	0

Synchronous Counter Design

- Q1. A ring counter is initialized incorrectly.
- (a) Analyze the resulting sequence of states.
 - (b) Propose a circuit modification to ensure automatic correction.

(12 marks)

[CSE 205 | L-2/T-1 | 2024–25]

Answer

1. Analysis of Incorrect Initialization in a Ring Counter

A standard N -bit ring counter is formed by connecting N flip-flops in a shift-register configuration, with the output of the final stage fed directly back into the input of the first stage ($D_0 = Q_{N-1}$).

For proper operation, the counter must be explicitly initialized (via preset/clear lines) so that exactly **one** flip-flop holds a logic ‘1’ while all others hold ‘0’. For a 4-bit ring counter, the valid sequence consists of 4 states:

1000 → 0100 → 0010 → 0001 → 1000

The Lockout Problem

If a standard ring counter is initialized incorrectly due to power-up transients or noise, it may enter an invalid state (having zero, two, three, or four ‘1’s). Because the feedback is a simple wire ($D_0 = Q_3$), the shift register will blindly circulate whatever bit pattern is present.

For a 4-bit ring counter, the $2^4 = 16$ possible states break down into the following disjoint cycles. If initialized incorrectly, the counter suffers from **lockout**, indefinitely trapped in an invalid sequence without ever reaching the valid counting sequence:

Initialization Pattern	Resulting Sequence of States ($Q_0Q_1Q_2Q_3$)	Number of States
Valid (One ‘1’)	1000 → 0100 → 0010 → 0001 ↻	4
All Zeros	0000 ↻	1
Adjacent Ones	1100 → 0110 → 0011 → 1001 ↻	4
Alternating Ones	1010 → 0101 ↻	2
Three Ones	1110 → 0111 → 1011 → 1101 ↻	4
All Ones	1111 ↻	1

2. Proposed Circuit Modification: The Self-Correcting Ring Counter

To ensure automatic correction (avoiding lockout), we must modify the feedback logic. Instead of feeding Q_{N-1} directly back to D_0 , we inject logic ‘1’ into the first stage **only when the other $N - 1$ stages are ‘0’**.

For a 4-bit counter, we decouple Q_3 from D_0 and instead use a **NOR gate** monitoring the outputs of the first three flip-flops (Q_0, Q_1, Q_2):

$$D_0 = \overline{Q_0 + Q_1 + Q_2} \quad (\text{Applying NOR Logic})$$

Under this configuration, D_0 evaluates to ‘1’ if and only if Q_0, Q_1 , and Q_2 are all ‘0’.

Schematic of the Self-Correcting Ring Counter

Verification of Automatic Recovery

To demonstrate the correction, assume the absolute worst-case improper initialization where all flip-flops power up as ‘1’ (1111₂). The table below traces the recovery logic:

Clock Cycle	Q ₀	Q ₁	Q ₂	Q ₃	D ₀ = $\overline{Q_0} + \overline{Q_1} + \overline{Q_2}$	Status
Initial	1	1	1	1	$\overline{1} + \overline{1} + \overline{1} = 0$	Invalid
1	0	1	1	1	$\overline{0} + \overline{1} + \overline{1} = 0$	Recovering
2	0	0	1	1	$\overline{0} + \overline{0} + \overline{1} = 0$	Recovering
3	0	0	0	1	$\overline{0} + \overline{0} + \overline{0} = 1$	Valid sequence entry
4	1	0	0	0	$\overline{1} + \overline{0} + \overline{0} = 0$	Valid state 1
5	0	1	0	0	$\overline{0} + \overline{1} + \overline{0} = 0$	Valid state 2

As demonstrated, invalid states are flushed out with ‘0’s until the internal logic generates a solitary ‘1’. The system perfectly re-enters the valid loop within at most $N - 1$ clock cycles.

Q2. Using JK flip-flops, design a synchronous counter that counts in the following sequence: 1, 2, 5, 7, 1, 2, . . . Analyze the final circuit to ensure that if the circuit enters any of the unused states, after a finite number of clock cycles it comes to a used state. **(13 marks)**
[CSE 205 | L-2/T-1 | 2022–23]

Answer

1. Synchronous Counter Design

We are tasked with designing a synchronous counter using JK flip-flops for the sequence: $1 \rightarrow 2 \rightarrow 5 \rightarrow 7 \rightarrow 1$. The highest state is 7 (111₂), which requires $n = 3$ flip-flops. Let the flip-flop states be Q_2, Q_1 , and Q_0 .

A. State Transition and Excitation Table

First, we determine the required flip-flop inputs to transition between the desired states. We recall the standard JK flip-flop excitation table:

- $0 \rightarrow 0 \implies J = 0, K = X$ | $1 \rightarrow 0 \implies J = X, K = 1$
- $0 \rightarrow 1 \implies J = 1, K = X$ | $1 \rightarrow 1 \implies J = X, K = 0$

Current State			Next State			Flip-Flop Excitation Inputs					
Q ₂	Q ₁	Q ₀	Q ₂ ⁺	Q ₁ ⁺	Q ₀ ⁺	J ₂	K ₂	J ₁	K ₁	J ₀	K ₀
0	0	1 (1)	0	1	0 (2)	0	X	1	X	X	1
0	1	0 (2)	1	0	1 (5)	1	X	X	1	1	X
1	0	1 (5)	1	1	1 (7)	X	0	1	X	X	0
1	1	1 (7)	0	0	1 (1)	X	1	X	1	X	0

Note: Unused states 0(000), 3(011), 4(100), and 6(110) are treated as Don't Cares (X) for map minimization.

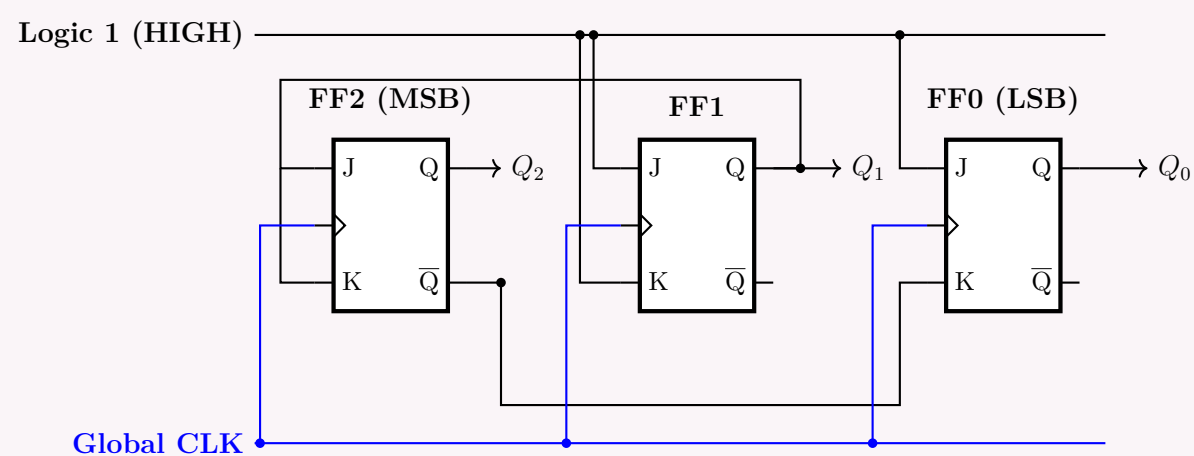
B. Karnaugh Map Minimization

We plot the excitation columns into 4×2 Karnaugh maps to find the simplest Boolean expressions.

J_2 Map		K_2 Map		J_1 Map	
Q_1Q_0		Q_1Q_0		Q_1Q_0	
00 01 11 10		00 01 11 10		00 01 11 10	
Q_2 0	X 0 X 1	Q_2 0	X X X X	Q_2 0	X 1 X X
Q_2 1	X X X X	Q_2 1	X 0 1 X	Q_2 1	X 1 X X
$J_2 = Q_1$		$K_2 = Q_1$		$J_1 = 1$	
K_1 Map		J_0 Map		K_0 Map	
Q_1Q_0		Q_1Q_0		Q_1Q_0	
00 01 11 10		00 01 11 10		00 01 11 10	
Q_2 0	X X X 1	Q_2 0	X X X 1	Q_2 0	X 1 X X
Q_2 1	X X 1 X	Q_2 1	X X X X	Q_2 1	X 0 0 X
$K_1 = 1$		$J_0 = 1$		$K_0 = \overline{Q_2}$	

C. Circuit Logic Diagram

Using the derived Boolean equations, we draw the final synchronous logic circuit.



2. Analysis of Unused States (Lockout Verification)

To ensure the counter does not get trapped in a "lockout" condition if noise forces it into an unused state, we must determine the next state for the unused states: 0(000), 3(011), 4(100), and 6(110).

The next-state equation for a JK flip-flop is $Q^+ = J\overline{Q} + \overline{K}Q$. Substituting our derived expressions:

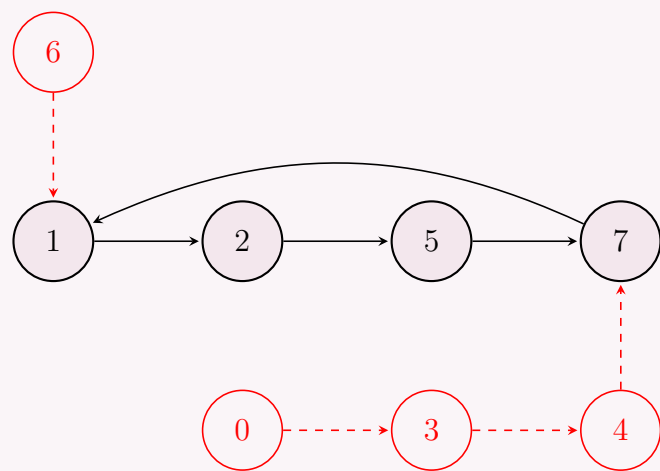
$$\begin{aligned}
 Q_2^+ &= J_2\overline{Q_2} + \overline{K_2}Q_2 = Q_1\overline{Q_2} + \overline{Q_1}Q_2 = Q_1 \oplus Q_2 \\
 Q_1^+ &= J_1\overline{Q_1} + \overline{K_1}Q_1 = 1 \cdot \overline{Q_1} + 0 \cdot Q_1 = \overline{Q_1} \\
 Q_0^+ &= J_0\overline{Q_0} + \overline{K_0}Q_0 = 1 \cdot \overline{Q_0} + (\overline{Q_2})Q_0 = \overline{Q_0} + Q_2Q_0 = \overline{Q_0} + Q_2 \quad (\text{Absorption Law})
 \end{aligned}$$

We now evaluate these equations for the unused states:

- **If State = 0 (000):** $Q_2^+ = 0 \oplus 0 = 0$, $Q_1^+ = \overline{0} = 1$, $Q_0^+ = \overline{0} + 0 = 1$. Next state is **3 (011)**.
- **If State = 3 (011):** $Q_2^+ = 1 \oplus 0 = 1$, $Q_1^+ = \overline{1} = 0$, $Q_0^+ = \overline{1} + 0 = 0$. Next state is **4 (100)**.
- **If State = 4 (100):** $Q_2^+ = 0 \oplus 1 = 1$, $Q_1^+ = \overline{0} = 1$, $Q_0^+ = \overline{0} + 1 = 1$. Next state is **7 (111)** (a valid state).
- **If State = 6 (110):** $Q_2^+ = 1 \oplus 1 = 0$, $Q_1^+ = \overline{1} = 0$, $Q_0^+ = \overline{0} + 1 = 1$. Next state is **1 (001)** (a valid state).

State Diagram

The complete state diagram demonstrates that no matter where the system starts, it merges into the valid cycle. The circuit is mathematically guaranteed to be **self-starting**.



Q3. Design a synchronous counter that counts in the following sequence: $0 \rightarrow 2 \rightarrow 4 \rightarrow 7 \rightarrow 5 \rightarrow 3 \rightarrow 0 \rightarrow 2 \rightarrow 4 \rightarrow \dots$. Design the counter using one JK flip-flop, one T flip-flop and one D flip-flop. Draw the circuit diagram. **(20 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Answer

1. State Sequence and Flip-Flop Assignment

The counter must sequence through: $0 \rightarrow 2 \rightarrow 4 \rightarrow 7 \rightarrow 5 \rightarrow 3 \rightarrow 0 \dots$. The highest decimal value is 7 (111_2), which requires a 3-bit counter. Let the bits be Q_2 (MSB), Q_1 , and Q_0 (LSB). The problem statement specifies the use of one JK flip-flop, one T flip-flop, and one D flip-flop. We will assign them as follows:

- Q_2 (**MSB**): JK Flip-Flop (Inputs J_2, K_2)
- Q_1 : T Flip-Flop (Input T_1)
- Q_0 (**LSB**): D Flip-Flop (Input D_0)

The invalid states not present in the main sequence are 1 (001_2) and 6 (110_2). We will treat their next states as "Don't Cares" (X) to optimize the combinational logic, and later analyze their transition paths to verify the counter is self-starting.

2. Excitation Table

To derive the combinational logic for the flip-flops, we use the respective excitation tables:

- JK FF (Q_2):** $(0 \rightarrow 0 : 0, X), (0 \rightarrow 1 : 1, X), (1 \rightarrow 0 : X, 1), (1 \rightarrow 1 : X, 0)$
- T FF (Q_1):** $T_1 = Q_1 \oplus Q_1^+$ (Toggle = 1, Hold = 0)
- D FF (Q_0):** $D_0 = Q_0^+$ (Directly follows the Next State)

Current State			Next State			JK Inputs		T Input	D Input	Condition
Q_2	Q_1	Q_0	Q_2^+	Q_1^+	Q_0^+	J_2	K_2	T_1	D_0	
0	0	0	0	1	0	0	X	1	0	Valid ($0 \rightarrow 2$)
0	1	0	1	0	0	1	X	1	0	Valid ($2 \rightarrow 4$)
1	0	0	1	1	1	X	0	1	1	Valid ($4 \rightarrow 7$)
1	1	1	1	0	1	X	0	1	1	Valid ($7 \rightarrow 5$)
1	0	1	0	1	1	X	1	1	1	Valid ($5 \rightarrow 3$)
0	1	1	0	0	0	0	X	1	0	Valid ($3 \rightarrow 0$)
0	0	1	X	X	X	X	X	X	X	Invalid (State 1)
1	1	0	X	X	X	X	X	X	X	Invalid (State 6)

Notice a remarkable mathematical property of this sequence: Q_1 toggles state on *every single* valid transition! Thus, $T_1 = 1$.

3. Karnaugh Map Minimization

We map J_2 , K_2 , T_1 , and D_0 to 3-variable Karnaugh maps (inputs Q_2, Q_1, Q_0).

J_2 Map					K_2 Map					T_1 Map					D_0 Map				
Q_1Q_0					Q_1Q_0					Q_1Q_0					Q_1Q_0				
00 01 11 10					00 01 11 10					00 01 11 10					00 01 11 10				
Q_2 0	0	-	0	1	Q_2 0	-	-	-	-	Q_2 0	1	-	1	1	Q_2 0	0	-	0	0
Q_2 1	-	-	-	-	Q_2 1	0	1	0	-	Q_2 1	1	1	1	-	Q_2 1	1	1	1	-

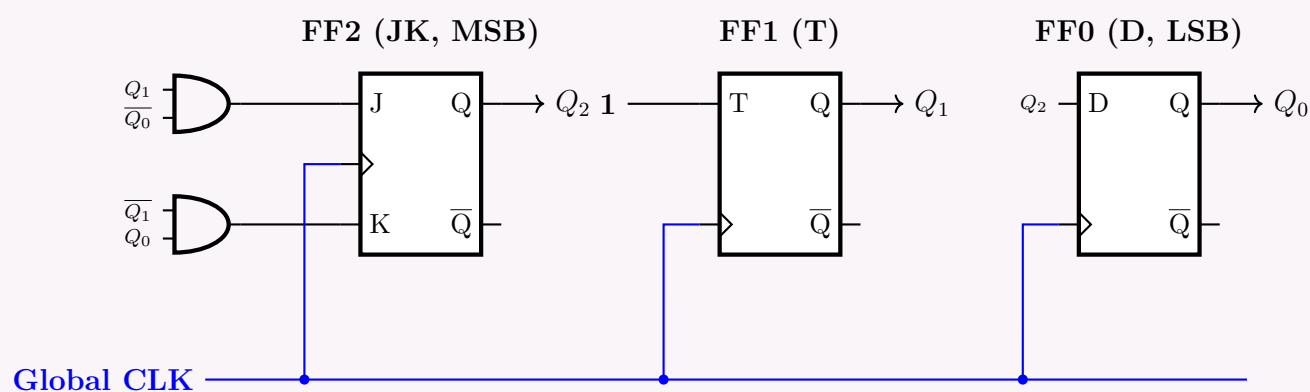
$J_2 = Q_1\overline{Q_0}$	$K_2 = \overline{Q_1}Q_0$	$T_1 = 1$	$D_0 = Q_2$
---------------------------	---------------------------	-----------	-------------

Final Logic Equations:

$$\begin{aligned}
 J_2 &= Q_1\overline{Q_0} \\
 K_2 &= \overline{Q_1}Q_0 \\
 T_1 &= 1 \quad (\text{Logic HIGH}) \\
 D_0 &= Q_2
 \end{aligned}$$

Self-Starting Verification: Plugging the invalid state 1 (001) into our equations yields next state values ($J_2 = 0, K_2 = 1 \Rightarrow Q_2^+ = 0$), ($T_1 = 1 \Rightarrow Q_1^+ = 1$), and ($D_0 = 0 \Rightarrow Q_0^+ = 0$), meaning $1 \rightarrow 2$. Plugging state 6 (110) yields ($J_2 = 1, K_2 = 0 \Rightarrow Q_2^+ = 1$), ($T_1 = 1 \Rightarrow Q_1^+ = 0$), and ($D_0 = 1 \Rightarrow Q_0^+ = 1$), meaning $6 \rightarrow 5$. The counter is fully self-starting.

4. Circuit Diagram

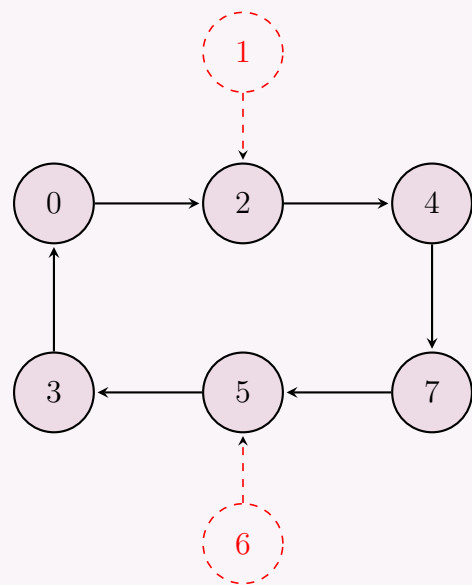


Q4. Design a synchronous counter that counts in the sequence $0 \rightarrow 2 \rightarrow 4 \rightarrow 7 \rightarrow 5 \rightarrow 3 \rightarrow 0 \rightarrow 2 \rightarrow 4 \rightarrow \dots$. Design the counter using SR Flip-Flops and draw the circuit diagram. (20 marks) [CSE 205 | L-2/T-1 | 2020–21]

Answer

1. State Sequence and Flip-Flop Assignment

The counter is required to sequence through the states: $0 \rightarrow 2 \rightarrow 4 \rightarrow 7 \rightarrow 5 \rightarrow 3 \rightarrow 0 \dots$. The maximum decimal value is 7 (111_2), which necessitates a 3-bit counter. Let the state variables be Q_2 (MSB), Q_1 , and Q_0 (LSB). We are designing this counter using **SR Flip-Flops**. The unused (invalid) states in this sequence are 1 (001_2) and 6 (110_2). We will treat the next state for these invalid states as "Don't Cares" (X) for maximum logic minimization, and subsequently verify that the counter is self-starting.



2. Excitation Table

To determine the inputs for the SR flip-flops, we use the standard SR excitation table:

- $0 \rightarrow 0 \implies S = 0, R = X$
- $0 \rightarrow 1 \implies S = 1, R = 0$
- $1 \rightarrow 0 \implies S = 0, R = 1$
- $1 \rightarrow 1 \implies S = X, R = 0$

Current State			Next State			FF2 (MSB)		FF1		FF0 (LSB)		Condition
Q_2	Q_1	Q_0	Q_2^+	Q_1^+	Q_0^+	S_2	R_2	S_1	R_1	S_0	R_0	
0	0	0	0	1	0	0	X	1	0	0	X	Valid ($0 \rightarrow 2$)
0	1	0	1	0	0	1	0	0	1	0	X	Valid ($2 \rightarrow 4$)
1	0	0	1	1	1	X	0	1	0	1	0	Valid ($4 \rightarrow 7$)
1	1	1	1	0	1	X	0	0	1	X	0	Valid ($7 \rightarrow 5$)
1	0	1	0	1	1	0	1	1	0	X	0	Valid ($5 \rightarrow 3$)
0	1	1	0	0	0	0	X	0	1	0	1	Valid ($3 \rightarrow 0$)
0	0	1	X	X	X	X	X	X	X	X	X	Invalid (State 1)
1	1	0	X	X	X	X	X	X	X	X	X	Invalid (State 6)

3. Karnaugh Map Minimization

We map the excitation signals to 3-variable Karnaugh Maps (inputs Q_2, Q_1, Q_0).

S_2 Map

Q_1Q_0

	00	01	11	10
0	0	-	0	1
1	-	0	-	-

$S_2 = Q_1\overline{Q_0}$

R_2 Map

Q_1Q_0

	00	01	11	10
0	-	-	-	0
1	0	1	0	-

$R_2 = \overline{Q_1}Q_0$

S_1 Map

Q_1Q_0

	00	01	11	10
0	1	-	0	0
1	1	1	0	-

$S_1 = \overline{Q_1}$

R_1 Map

Q_1Q_0

	00	01	11	10
0	0	-	1	1
1	0	0	1	-

$R_1 = Q_1$

S_0 Map

Q_1Q_0

	00	01	11	10
0	0	-	0	0
1	1	-	-	-

$S_0 = Q_2$

R_0 Map

Q_1Q_0

	00	01	11	10
0	-	-	1	-
1	0	0	0	-

$R_0 = \overline{Q_2}$

Final Logic Equations:

$$\begin{aligned} S_2 &= Q_1 \overline{Q_0} \\ S_1 &= \overline{Q_1} \\ S_0 &= Q_2 \end{aligned}$$

$$\begin{aligned} R_2 &= \overline{Q_1} Q_0 \\ R_1 &= Q_1 \\ R_0 &= \overline{Q_2} \end{aligned}$$

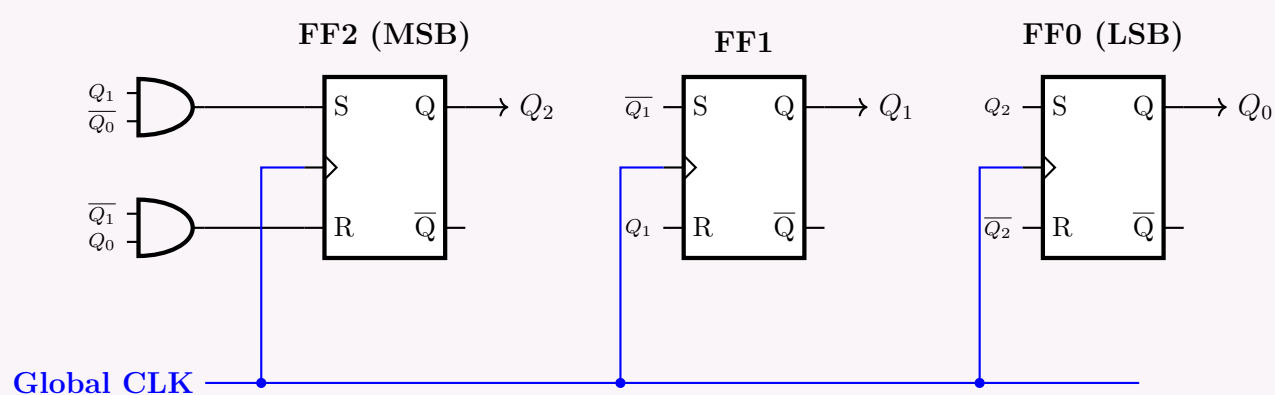
4. Self-Starting Verification

To ensure the counter doesn't lock in an invalid state, we evaluate the next state generated by our equations for states 1 and 6:

- **Invalid State 1 (001):** $S_2 = 0, R_2 = 1 \Rightarrow Q_2^+ = 0$. $S_1 = 1, R_1 = 0 \Rightarrow Q_1^+ = 1$. $S_0 = 0, R_0 = 1 \Rightarrow Q_0^+ = 0$. *Next State is 010 (State 2).*
- **Invalid State 6 (110):** $S_2 = 1, R_2 = 0 \Rightarrow Q_2^+ = 1$. $S_1 = 0, R_1 = 1 \Rightarrow Q_1^+ = 0$. $S_0 = 1, R_0 = 0 \Rightarrow Q_0^+ = 1$. *Next State is 101 (State 5).*

Both invalid states immediately transition back into the valid sequence. The counter is strictly **self-starting**.

5. Circuit Diagram

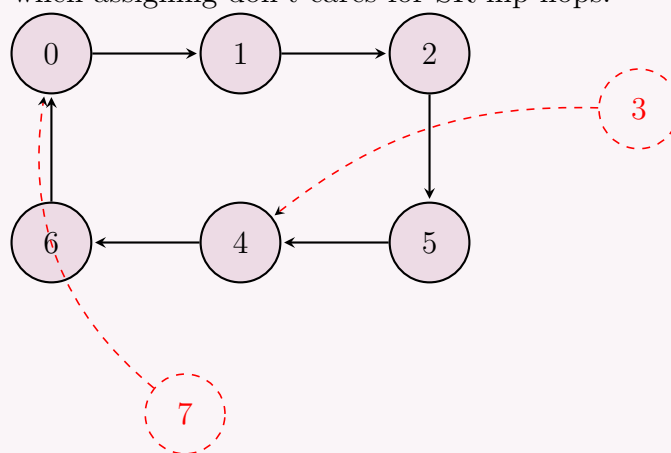


Q5. Design a synchronous counter that counts in the following sequence: $0 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 6 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow \dots$. Design the counter using SR Flip-Flops and draw the circuit diagram. (20 marks) [CSE 205 | L-2/T-1 | 2017–18]

Answer

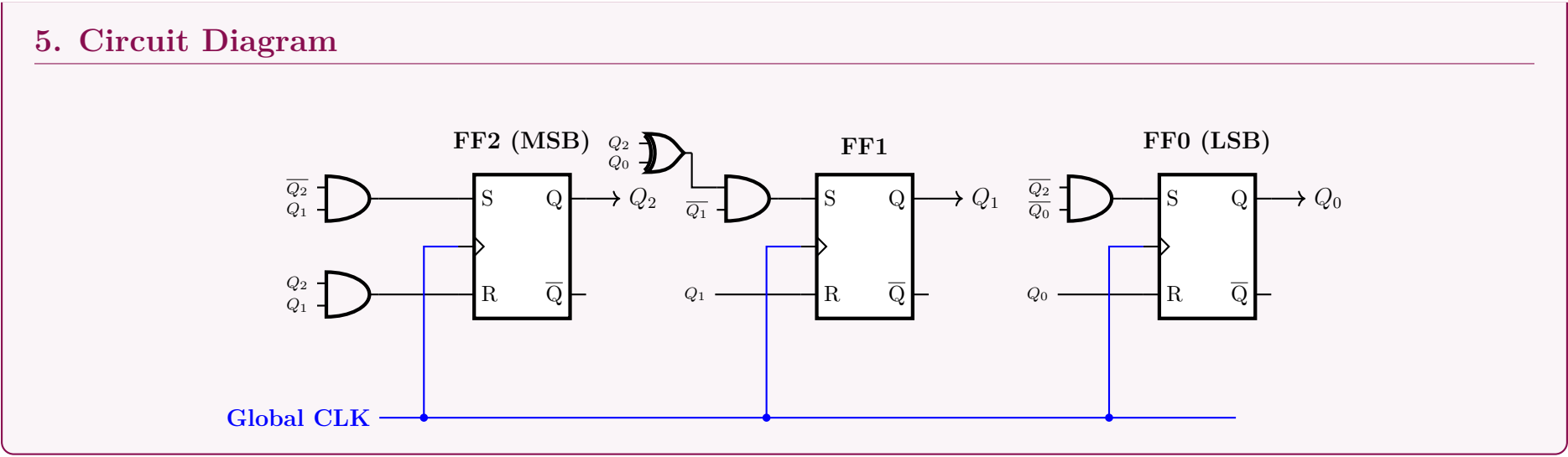
1. State Sequence and Flip-Flop Assignment

The counter must follow the sequence: $0 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 6 \rightarrow 0 \dots$. The maximum state value is 6 (110_2), requiring a 3-bit counter. We assign the bits as Q_2 (MSB), Q_1 , and Q_0 (LSB). We are strictly designing this using **SR Flip-Flops**. The unused states are 3 (011_2) and 7 (111_2). We will treat these as "Don't Cares" during initial minimization, but we must take special care to avoid the forbidden state ($S = 1, R = 1$) when assigning don't cares for SR flip-flops.



2. Excitation Table

To determine the combinational logic for the flip-flops, we use the SR excitation table rules: $(0 \rightarrow 0 \Rightarrow 0, X)$, $(0 \rightarrow 1 \Rightarrow 1, 0)$, $(1 \rightarrow 0 \Rightarrow 0, 1)$, $(1 \rightarrow 1 \Rightarrow X, 0)$.



Q6. Design a synchronous counter with the sequence: $1 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 2 \rightarrow 1$. You need not take any measure for the unused states.
(10 marks) [CSE 205 | L-2/T-1 | 2016–17]

Answer

1. Counter Specifications and State Diagram

We are tasked with designing a synchronous counter that follows the sequence: $1 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 2 \rightarrow 1$. Since the maximum state value is 7 (111_2), we require a 3-bit counter. We assign the state variables as Q_2 (MSB), Q_1 , and Q_0 (LSB). The unused states are 0, 3, and 6. The prompt specifies that no measures need to be taken for these states, so we will treat their next-state transitions as **Don't Cares (X)** to optimally minimize our combinational logic. We will utilize **D Flip-Flops** for their direct mapping capability ($D = Q^+$).

2. State Transition and Excitation Table

For a D Flip-Flop, the excitation input is equal to the next state ($D = Q^+$). We construct the table based on the required sequence.

Current State			Next State			Flip-Flop Inputs			Condition
Q_2	Q_1	Q_0	Q_2^+	Q_1^+	Q_0^+	D_2	D_1	D_0	
0	0	0	X	X	X	X	X	X	Unused
0	0	1	1	0	0	1	0	0	Sequence ($1 \rightarrow 4$)
0	1	0	0	0	1	0	0	1	Sequence ($2 \rightarrow 1$)
0	1	1	X	X	X	X	X	X	Unused
1	0	0	1	0	1	1	0	1	Sequence ($4 \rightarrow 5$)
1	0	1	1	1	1	1	1	1	Sequence ($5 \rightarrow 7$)
1	1	0	X	X	X	X	X	X	Unused
1	1	1	0	1	0	0	1	0	Sequence ($7 \rightarrow 2$)

3. Karnaugh Map Minimization

We plot the excitation equations into 3-variable K-maps to derive the minimal sum-of-products (SOP) expressions.

D_2 Map

Q_1Q_0

	00	01	11	10
0	-	1	-	0
1	1	1	0	-

$D_2 = \overline{Q_1}$

D_1 Map

Q_1Q_0

	00	01	11	10
0	-	0	-	0
1	0	1	1	-

$D_1 = Q_2Q_0$

D_0 Map

Q_1Q_0

	00	01	11	10
0	-	0	-	1
1	1	1	0	-

$D_0 = \overline{Q_0} + Q_2\overline{Q_1}$

Logic Derivation Notes:

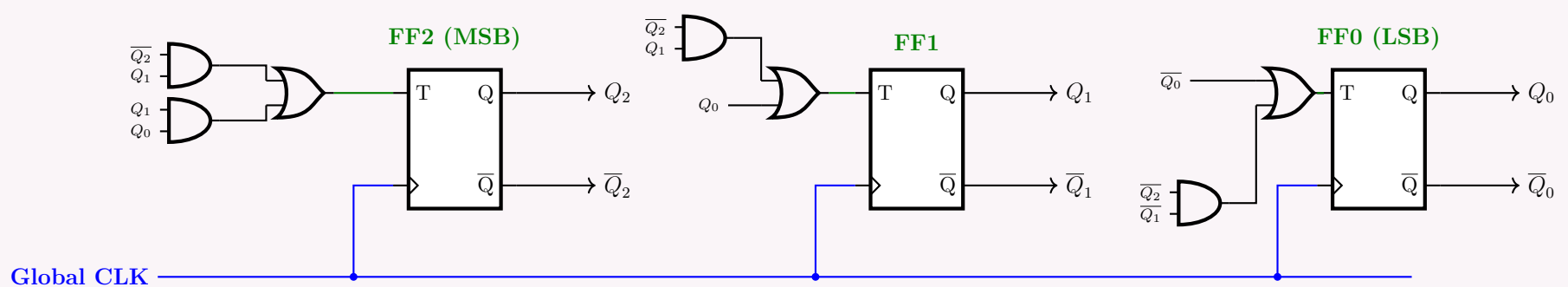
- D_2 : Groups minterms (0, 1, 4, 5), which beautifully simplifies to a single literal $\overline{Q_1}$. We don't even need a logic gate for this input.
- D_1 : Groups minterms (5, 7), leaving Q_2Q_0 .
- D_0 : Groups minterms (0, 2, 4, 6) on the edges to form $\overline{Q_0}$, and (4, 5) to form $Q_2\overline{Q_1}$.

Final Simplified Equations:

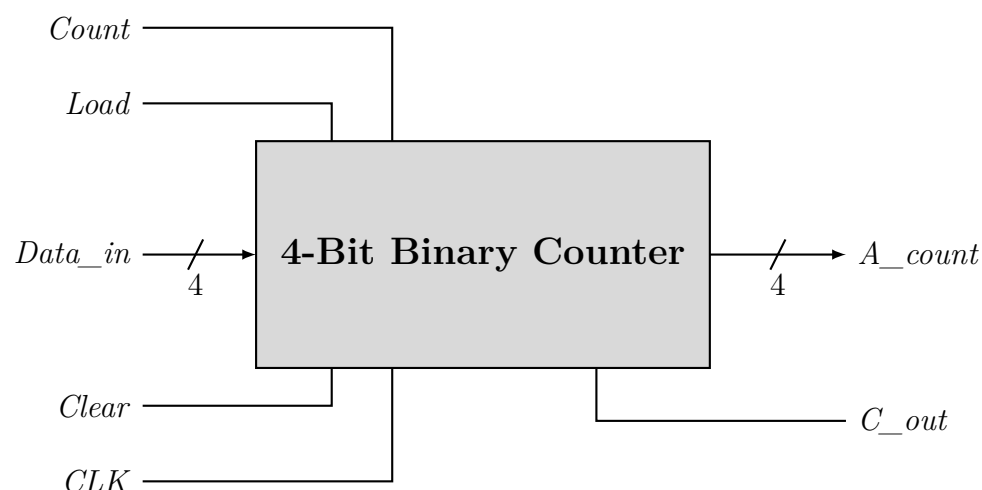
$$\begin{aligned} D_2 &= \overline{Q_1} \\ D_1 &= Q_2 \cdot Q_0 \\ D_0 &= \overline{Q_0} + (Q_2 \cdot \overline{Q_1}) \end{aligned}$$

4. Logic Circuit Diagram

Using the derived equations, the schematic is drawn utilizing three D flip-flops, one AND gate, one OR gate, and one additional AND gate. To ensure clarity and readability, signal inputs to the gates are explicitly labeled.



Q7. Show two ways to design a Modulo-6 counter using a 4-bit binary counter and necessary basic gates. The Modulo-6 counter repeatedly goes through 0, 1, 2, 3, 4, 5.



(12 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

To design a **Modulo-6 counter**, the 4-bit binary counter must sequence through six distinct states: 0, 1, 2, 3, 4, and 5. Since a standard 4-bit counter normally counts from 0 to 15, we must force the counter to return to 0 after reaching 5.

We can achieve this using two primary techniques depending on the counter's control inputs: **Asynchronous Clear** or **Synchronous Load**.

State Sequence and Logic Analysis

First, let us examine the binary states of the counter, where Q_3 is the Most Significant Bit (MSB) and Q_0 is the Least Significant Bit (LSB).

Decimal	Q_3	Q_2	Q_1	Q_0	State Type
0	0	0	0	0	Valid
1	0	0	0	1	Valid
2	0	0	1	0	Valid
3	0	0	1	1	Valid
4	0	1	0	0	Valid
5	0	1	0	1	Maximum Valid State
6	0	1	1	0	First Invalid State / Transient

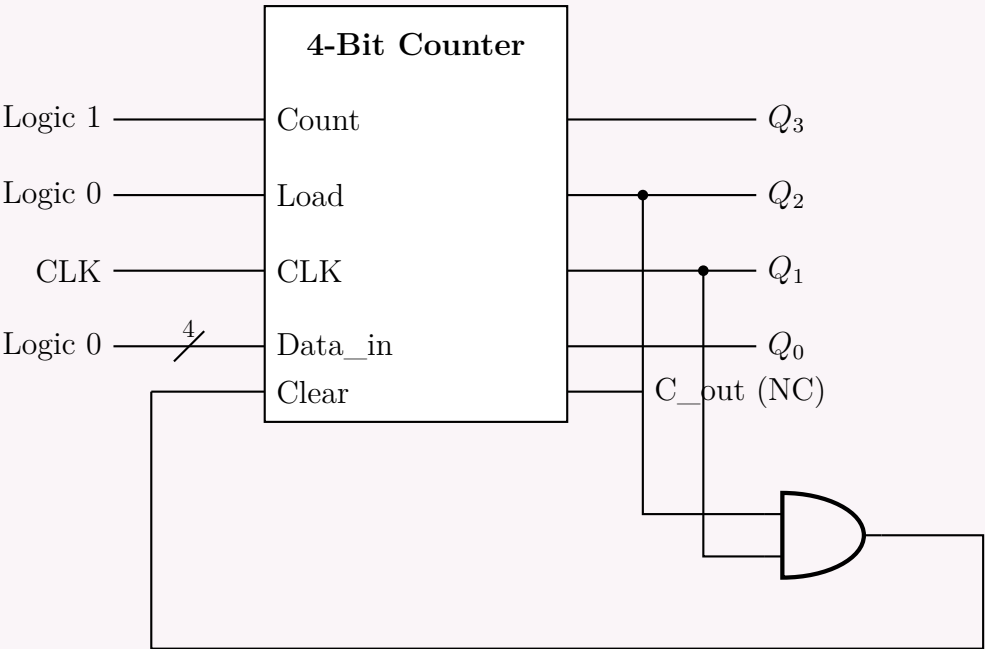
Method 1: Using Asynchronous Clear (Detecting State 6)

Principle: The counter is allowed to count up to 5 naturally. When the next clock edge arrives, the counter momentarily transitions to state 6 (0110₂). We use external logic to detect state 6 and immediately trigger the *Asynchronous Clear* input to reset the counter to 0000₂ before the next clock cycle.

Boolean Derivation for Clear: We need the *Clear* signal to be logic HIGH when the counter reaches 6 ($Q_3Q_2Q_1Q_0 = 0110_2$).

$$\text{Clear} = \overline{Q_3} \cdot Q_2 \cdot Q_1 \cdot \overline{Q_0}$$
$$\text{Clear} = Q_2 \cdot Q_1 \quad (\text{Simplified, since states 7, 14, 15 are never reached})$$

Implementation: We use a 2-input AND gate to multiply Q_2 and Q_1 . The *Load* pin is grounded, and *Count* is set high.



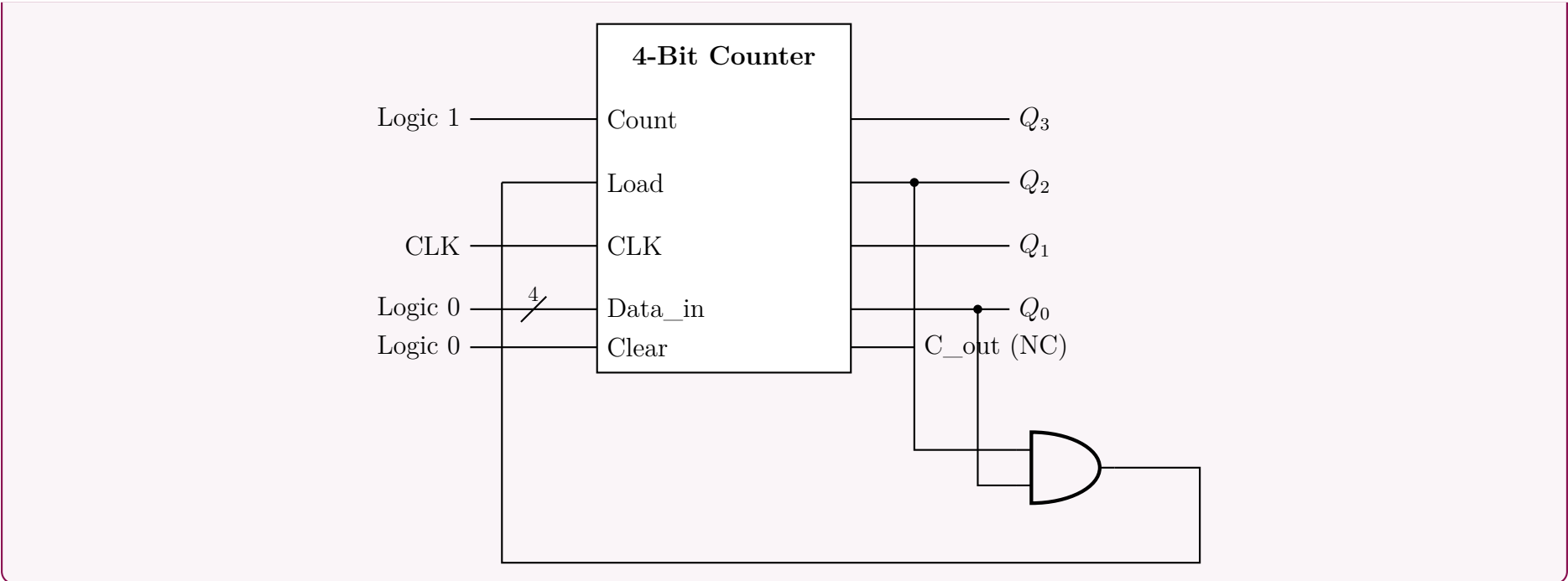
Method 2: Using Synchronous Load (Detecting State 5)

Principle: The counter safely reaches its maximum valid state, which is 5 (0101₂). We use external logic to detect state 5. When detected, we assert the *Synchronous Load* input HIGH. On the *very next* active clock edge, the counter will load the data present at *Data_in* (which is hardwired to 0000₂) instead of counting to 6.

Boolean Derivation for Load: We need the *Load* signal to be logic HIGH when the counter is exactly at 5 ($Q_3Q_2Q_1Q_0 = 0101_2$).

$$\text{Load} = \overline{Q_3} \cdot Q_2 \cdot \overline{Q_1} \cdot Q_0$$
$$\text{Load} = Q_2 \cdot Q_0 \quad (\text{Simplified, since states 7, 13, 15 are never reached})$$

Implementation: We use a 2-input AND gate to multiply Q_2 and Q_0 . The *Clear* pin is grounded (disabled), *Count* is high, and *Data_in* is tied to 0000₂.



Q8. Design a 3-bit binary up counter that counts $1 \rightarrow 2 \rightarrow 5 \rightarrow 7 \rightarrow 1 \dots$, where any invalid state goes to the immediate next valid state in the sequence. Use T Flip-Flops. Show the state diagram, state table, function equations, minimization (if necessary) and the circuit diagram. **(15 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Answer

1. State Sequence and State Diagram

The synchronous counter sequence is given as $1 \rightarrow 2 \rightarrow 5 \rightarrow 7 \rightarrow 1 \dots$. Because the highest state is 7 (111_2), we require a 3-bit counter using three T Flip-Flops (Q_2, Q_1, Q_0). To ensure self-correction, any invalid state (0, 3, 4, 6) must transition to the **immediate next valid state in the sequence**. The closest next valid states mathematically are:

- 0 (000) $\rightarrow$ 1 (001)
- 3 (011) $\rightarrow$ 5 (101)
- 4 (100) $\rightarrow$ 5 (101)
- 6 (110) $\rightarrow$ 7 (111)

The state diagram shows four valid states (1, 2, 5, 7) in a sequence. Invalid states (0, 3, 4, 6) are shown in red circles with dashed arrows indicating their transitions to the next valid state: 0 to 1, 3 to 5, 4 to 5, and 6 to 7.

2. State Transition and Excitation Table

The excitation for a T flip-flop is defined by $T = Q \oplus Q^+$.

Current State			Next State			T Inputs			Condition
Q_2	Q_1	Q_0	Q_2^+	Q_1^+	Q_0^+	T_2	T_1	T_0	
0	0	1	0	1	0	0	1	1	Valid ($1 \rightarrow 2$)
0	1	0	1	0	1	1	1	1	Valid ($2 \rightarrow 5$)
1	0	1	1	1	1	0	1	0	Valid ($5 \rightarrow 7$)
1	1	1	0	0	1	1	1	0	Valid ($7 \rightarrow 1$)
0	0	0	0	0	1	0	0	1	Recovery ($0 \rightarrow 1$)
0	1	1	1	0	1	1	1	0	Recovery ($3 \rightarrow 5$)
1	0	0	1	0	1	0	0	1	Recovery ($4 \rightarrow 5$)
1	1	0	1	1	1	0	0	1	Recovery ($6 \rightarrow 7$)

3. Karnaugh Map Minimization

We extract the Boolean functions for T_2, T_1 , and T_0 from the table.

T_2 Map

Q_1Q_0

	00	01	11	10
0	0	0	1	1
1	0	0	1	0

$T_2 = \overline{Q_2}Q_1 + Q_1Q_0$

T_1 Map

Q_1Q_0

	00	01	11	10
0	0	1	1	1
1	0	1	1	0

$T_1 = Q_0 + \overline{Q_2}Q_1$

T_0 Map

Q_1Q_0

	00	01	11	10
0	1	1	0	1
1	1	0	0	1

$T_0 = \overline{Q_0} + \overline{Q_2}Q_1$

Logic Equations (Optimized for Gates):

$$T_2 = Q_1(\overline{Q_2} + Q_0) = \overline{Q_2}Q_1 + Q_1Q_0$$
$$T_1 = Q_0 + \overline{Q_2}Q_1$$
$$T_0 = \overline{Q_0} + \overline{Q_2}Q_1$$

Notice that the product term $\overline{Q_2}Q_1$ is shared between T_2 and T_1 , which reduces the required gate count.

4. Circuit Diagram

To maintain a pristine, cross-talk-free schematic, local input tags are used to route signals into the logic gates feeding the T inputs.

Q9. Design a 3-bit synchronous binary up counter that counts $1 \rightarrow 2 \rightarrow 4 \rightarrow 7 \rightarrow 1 \rightarrow \dots$, where any invalid state goes to the immediate next state in the sequence. Use JK flip-flops. Show the state table, state diagram, function minimization (if necessary), and circuit diagram. **(20 marks)**

[CSE 205 | L-2/T-1 | 2013–14]

Answer

1. State Sequence and State Diagram

The synchronous counter must count in the sequence $1 \rightarrow 2 \rightarrow 4 \rightarrow 7 \rightarrow 1 \dots$. The highest decimal value is 7 (111_2), which dictates the need for a 3-bit counter using three JK Flip-Flops (Q_2, Q_1, Q_0). The problem requires that any invalid state (0, 3, 5, 6) transition to the **immediate next valid state in the sequence** (mathematically higher nearest state in sequence). The invalid state recovery transitions are:

- 0 (000) $\rightarrow$ 1 (001)
- 3 (011) $\rightarrow$ 4 (100)
- 5 (101) $\rightarrow$ 7 (111)
- 6 (110) $\rightarrow$ 7 (111)

370

2. State Transition and Excitation Table

The excitation table for a JK flip-flop is governed by: $(0 \rightarrow 0 : J = 0, K = X)$, $(0 \rightarrow 1 : J = 1, K = X)$, $(1 \rightarrow 0 : J = X, K = 1)$, $(1 \rightarrow 1 : J = X, K = 0)$.

Current State			Next State			JK Inputs						Condition
Q_2	Q_1	Q_0	Q_2^+	Q_1^+	Q_0^+	J_2	K_2	J_1	K_1	J_0	K_0	
0	0	1	0	1	0	0	X	1	X	X	1	Valid $(1 \rightarrow 2)$
0	1	0	1	0	0	1	X	X	1	0	X	Valid $(2 \rightarrow 4)$
1	0	0	1	1	1	X	0	1	X	1	X	Valid $(4 \rightarrow 7)$
1	1	1	0	0	1	X	1	X	1	X	0	Valid $(7 \rightarrow 1)$
0	0	0	0	0	1	0	X	0	X	1	X	Recovery $(0 \rightarrow 1)$
0	1	1	1	0	0	1	X	X	1	X	1	Recovery $(3 \rightarrow 4)$
1	0	1	1	1	1	X	0	1	X	X	0	Recovery $(5 \rightarrow 7)$
1	1	0	1	1	1	X	0	X	0	1	X	Recovery $(6 \rightarrow 7)$

3. Karnaugh Map Minimization

We derive the minimized Boolean expressions for J and K for each flip-flop.

J_2 Map

Q_1Q_0

	00	01	11	10
0	0	0	1	1
1	-	-	-	-

$J_2 = Q_1$

J_1 Map

Q_1Q_0

	00	01	11	10
0	0	1	-	-
1	1	1	-	-

$J_1 = Q_2 + Q_0$

J_0 Map

Q_1Q_0

	00	01	11	10
0	1	-	-	0
1	1	-	-	1

$J_0 = Q_2 + \overline{Q_1}$

K_2 Map

Q_1Q_0

	00	01	11	10
0	-	-	1	-
1	0	0	1	0

$K_2 = Q_1Q_0$

K_1 Map

Q_1Q_0

	00	01	11	10
0	-	-	1	1
1	-	-	1	0

$K_1 = \overline{Q_2} + Q_0$

K_0 Map

Q_1Q_0

	00	01	11	10
0	-	1	1	-
1	-	0	0	-

$K_0 = \overline{Q_2}$

Final Logic Equations:

$J_2 = Q_1$
 $J_1 = Q_2 + Q_0$
 $J_0 = Q_2 + \overline{Q_1}$

$K_2 = Q_1Q_0$
 $K_1 = \overline{Q_2} + Q_0$
 $K_0 = \overline{Q_2}$

4. Circuit Diagram

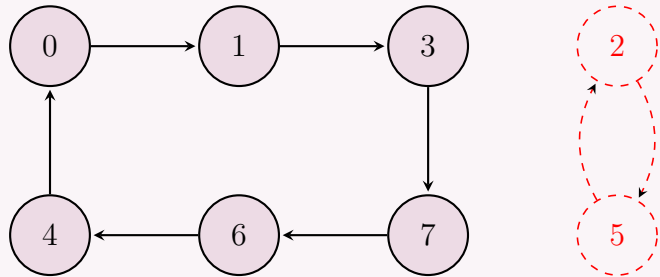
Local labels are assigned as inputs to the combinational logic gates to avoid crossed wires and maintain clarity.

Q10. Design a synchronous counter with the following binary sequence: 0, 1, 3, 7, 6, 4 and repeat. Use T flip-flops. **(15 marks)** [CSE 205 | L-2/T-1 | 2012–13]

Answer

1. State Sequence and State Diagram

The synchronous counter is required to count in the sequence: $0 \rightarrow 1 \rightarrow 3 \rightarrow 7 \rightarrow 6 \rightarrow 4 \rightarrow 0 \dots$. The maximum decimal value in the sequence is 7 (111_2), which means a minimum of **3 T Flip-Flops** (Q_2, Q_1, Q_0) are required. The states 2 (010_2) and 5 (101_2) are not part of the valid sequence. We will treat these invalid states as **Don't Cares (X)** to optimally minimize the combinational logic. As a byproduct of the minimization, we will analyze where these unused states transition to ensure the system does not get locked.



2. State Transition and Excitation Table

For a T flip-flop, the excitation input is given by $T = Q \oplus Q^+$. The required logic level is 1 if the state toggles, and 0 if it remains the same.

Current State			Next State			T Inputs			Condition
Q_2	Q_1	Q_0	Q_2^+	Q_1^+	Q_0^+	T_2	T_1	T_0	
0	0	0	0	0	1	0	0	1	Valid ($0 \rightarrow 1$)
0	0	1	0	1	1	0	1	0	Valid ($1 \rightarrow 3$)
0	1	1	1	1	1	1	0	0	Valid ($3 \rightarrow 7$)
1	1	1	1	1	0	0	0	1	Valid ($7 \rightarrow 6$)
1	1	0	1	0	0	0	1	0	Valid ($6 \rightarrow 4$)
1	0	0	0	0	0	1	0	0	Valid ($4 \rightarrow 0$)
0	1	0	X	X	X	X	X	X	Invalid (State 2)
1	0	1	X	X	X	X	X	X	Invalid (State 5)

3. Karnaugh Map Minimization

We plot the T_2, T_1 , and T_0 functions into 3-variable Karnaugh Maps using variables Q_2, Q_1, Q_0 .

T_2 Map

$Q_1 Q_0$

	00	01	11	10
0	0	0	1	-
1	1	-	0	0

T_1 Map

$Q_1 Q_0$

	00	01	11	10
0	0	1	0	-
1	0	-	0	1

T_0 Map

$Q_1 Q_0$

	00	01	11	10
0	1	0	0	-
1	0	-	1	0

$T_2 = \overline{Q_2}Q_1 + Q_2\overline{Q_1}$

$T_1 = \overline{Q_1}Q_0 + Q_1\overline{Q_0}$

$T_0 = \overline{Q_2}Q_0 + Q_2Q_0$

Final Logic Equations: Applying the definitions of Exclusive-OR (XOR) and Exclusive-NOR (XNOR), we can simplify the excitation equations directly to standard logic gates:

$T_2 = \overline{Q_2}Q_1 + Q_2\overline{Q_1}$

$T_1 = \overline{Q_1}Q_0 + Q_1\overline{Q_0}$

$T_0 = \overline{Q_2}Q_0 + Q_2Q_0$

$= Q_2 \oplus Q_1$

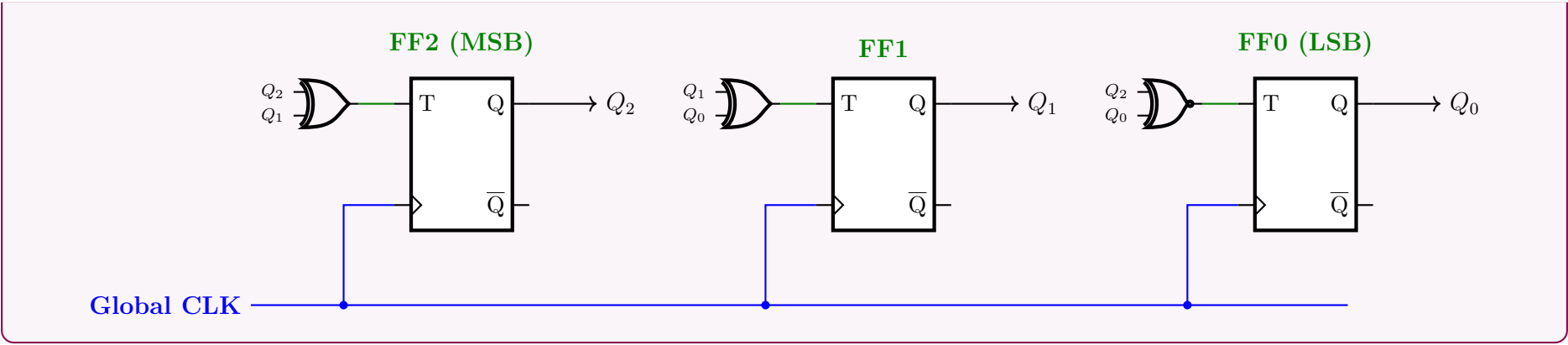
$= Q_1 \oplus Q_0$

$= \overline{Q_2} \oplus \overline{Q_0} \quad (\text{or } Q_2 \odot Q_0)$

Note on Invalid States: Using these equations, state 2 (010) yields $T_2 = 1, T_1 = 1, T_0 = 1$, transitioning to state 5 (101). State 5 yields $T_2 = 1, T_1 = 1, T_0 = 1$, transitioning back to state 2. They form an isolated toggle loop. This behavior is illustrated in the state diagram.

4. Circuit Diagram

Local text nodes are used at the inputs of the combinational gates to represent connections from the flip-flop outputs, minimizing messy intersecting wires and preserving clarity.



Synchronous Gray Code Counter

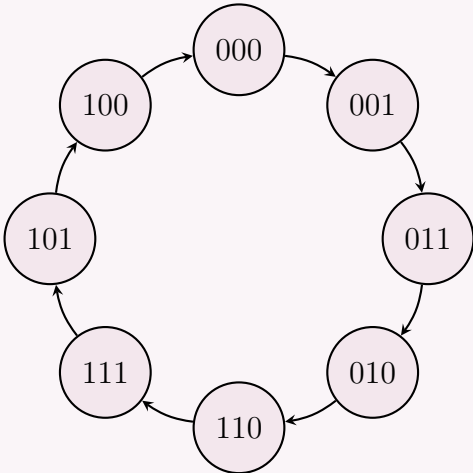
Q1. Design a 3-bit synchronous Gray Code Counter using JK Flip-Flops. Show the state diagram, state table, function equations and the circuit diagram. **(15 marks)** [CSE 205 | L-2/T-1 | 2014–15, 2011–12]

Answer

To design a 3-bit synchronous Gray Code counter using JK flip-flops, we will systematically progress through the state diagram, state table, K-map derivations for the excitation equations, and finally, the logic circuit diagram.

1. State Diagram

A 3-bit Gray code sequence changes only one bit at a time. The sequence is: 000 → 001 → 011 → 010 → 110 → 111 → 101 → 100 → 000.



2. State Table & JK Excitation

We use the standard JK flip-flop excitation rules: (0 → 0: J=0, K=X) | (0 → 1: J=1, K=X) | (1 → 0: J=X, K=1) | (1 → 1: J=X, K=0)

Present State			Next State			Flip-Flop Inputs					
Q_2	Q_1	Q_0	Q_2^+	Q_1^+	Q_0^+	J_2	K_2	J_1	K_1	J_0	K_0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	1	0	X	1	X	X	0
0	1	1	0	1	0	0	X	X	0	X	1
0	1	0	1	1	0	1	X	X	0	0	X
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	1	0	1	X	0	X	1	X	0
1	0	1	1	0	0	X	0	0	X	X	1
1	0	0	0	0	0	X	1	0	X	0	X

3. K-Map Simplification & Boolean Equations

Using the table above, we plot the Karnaugh maps for the J and K inputs of each flip-flop.

J_2 K-Map

Q_1Q_0

	00	01	11	10
0				1
1	-	-	-	-

$J_2 = Q_1\overline{Q_0}$

K_2 K-Map

Q_1Q_0

	00	01	11	10
0	-	-	-	-
1	1			

$K_2 = \overline{Q_1}\overline{Q_0}$

J_1 K-Map

Q_1Q_0

	00	01	11	10
0		1	-	-
1			-	-

$J_1 = \overline{Q_2}Q_0$

K_1 K-Map

Q_1Q_0

	00	01	11	10
0	-	-		
1	-	-	1	

$K_1 = Q_2Q_0$

J_0 K-Map

Q_1Q_0

	00	01	11	10
0	1	-	-	
1		-	-	1

$J_0 = \overline{Q_2}\overline{Q_1} + Q_2Q_1$
 $= Q_2 \odot Q_1$

K_0 K-Map

Q_1Q_0

	00	01	11	10
0	-		1	-
1	-	1		-

$K_0 = \overline{Q_2}Q_1 + Q_2\overline{Q_1}$
 $= Q_2 \oplus Q_1$

4. Logic Circuit Diagram

Based on the derived excitation equations, the complete circuit diagram uses three JK flip-flops, two AND gates, one XOR gate, and one XNOR gate. A common clock line synchronizes the counter.

The logic circuit diagram shows three JK flip-flops (FF0, FF1, FF2) and four logic gates. The Global CLK is connected to the clock inputs of all flip-flops. The J and K inputs are derived from the current state bits Q2, Q1, Q0 and their complements using AND, XOR, and XNOR gates. The outputs are Q2, Q1, Q0 and their complements.

- FF2: J = $Q_1\overline{Q_0}$, K = $\overline{Q_1}\overline{Q_0}$
- FF1: J = $\overline{Q_2}Q_0$, K = Q_2Q_0
- FF0: J = $\overline{Q_2}\overline{Q_1} + Q_2Q_1$ (XNOR of Q2 and Q1), K = $\overline{Q_2}Q_1 + Q_2\overline{Q_1}$ (XOR of Q2 and Q1)

374

Switch-Tail Ring Counter

Q1. Design a 5-bit ring counter using a shift register and illustrate its operation using a timing diagram. (10 marks)

[CSE 205 | L-2/T-1 | 2017–18]

Answer

1. Structural Design and Operation Principle

A **Ring Counter** is a specialized sequential logic circuit constructed from a linear shift register where the output of the final flip-flop is fed back to the input of the first flip-flop. This creates a circular data path.

To design a **5-bit ring counter**, we require 5 D-type flip-flops (denoted as FF_0 through FF_4). They all share a common synchronous clock signal (CLK). The boolean logic for the next-state inputs is purely dependent on the immediate prior stage, forming a closed loop:

$$D_0 = Q_4 \quad (\text{Feedback loop})$$
$$D_1 = Q_0$$
$$D_2 = Q_1$$
$$D_3 = Q_2$$
$$D_4 = Q_3$$

In a standard ring counter operation, only a single flip-flop holds the value 1 at any given time, while the rest are 0. This single 1 is shifted one position to the right upon every active (rising) clock edge. To achieve this, the circuit must be explicitly initialized before counting begins. An asynchronous **INIT** signal is applied to the *Preset* (PR) pin of FF_0 and the *Clear* (CLR) pins of FF_1 to FF_4 .

2. Circuit Diagram

The following schematic illustrates the 5-bit ring counter using standard D flip-flops. The feedback path from Q_4 to D_0 is highlighted, along with the synchronous clock and asynchronous initialization lines.

3. State Transition Table

Since $n = 5$, the counter has n valid states out of the possible $2^5 = 32$ states. Once initialized, the single active high bit rotates sequentially with each clock pulse.

Clock Pulse	Q_0	Q_1	Q_2	Q_3	Q_4
INIT	1	0	0	0	0
1	0	1	0	0	0
2	0	0	1	0	0
3	0	0	0	1	0
4	0	0	0	0	1
5	1	0	0	0	0

4. Timing Diagram

The timing diagram demonstrates the synchronous shifting of the initial 1 bit. The vertical dashed lines signify the rising edge of the clock signal (trigger point). Note that upon reaching Q_4 , the high bit wraps around to Q_0 on the subsequent clock edge.

375

Q2. Draw the logic diagram of a four-stage switch-tail ring counter. (5 marks)

[CSE 205 | L-2/T-1 | 2011–12]

Answer

1. Structural Design and Operational Principle

A **Four-Stage Switch-Tail Ring Counter**, commonly known as a **Johnson Counter** or a Twisted Ring Counter, is a modified shift register. Unlike a standard ring counter where the true output of the last flip-flop is fed back to the first, a switch-tail ring counter feeds back the *complemented* (inverted) output of the final stage to the input of the first stage. For a 4-stage counter, we utilize 4 D-type flip-flops (FF_0 to FF_3) driven by a common synchronous clock signal (CLK). The next-state logic equations governing the inputs are:
$$D_0 = \overline{Q_3} \quad (\text{Twisted feedback loop})$$
$$D_1 = Q_0$$
$$D_2 = Q_1$$
$$D_3 = Q_2$$

This inversion in the feedback loop doubles the number of unique states compared to a standard ring counter. An N -stage Johnson counter cycles through $2N$ distinct states, making this a modulo-8 counter. To guarantee correct operation, an asynchronous **CLEAR** signal is applied to reset all flip-flops to 0 prior to initiating the clock pulses.

2. Logic Diagram

The schematic below details the connections using D flip-flops. The defining characteristic—the feedback from $\overline{Q_3}$ to D_0 —is routed below the main shift register data path.

3. State Sequence (Truth Table)

Assuming the counter is initialized to 0000 via the asynchronous CLEAR signal, the sequence of states will progress by filling with 1s from the left, and then clearing with 0s from the left, generating exactly $2 \times 4 = 8$ states.

Clock Pulse	Q_0	Q_1	Q_2	Q_3	Next $D_0 (\overline{Q_3})$
Initial	0	0	0	0	1
1	1	0	0	0	1
2	1	1	0	0	1
3	1	1	1	0	1
4	1	1	1	1	0
5	0	1	1	1	0
6	0	0	1	1	0
7	0	0	0	1	0
8	0	0	0	0	1

Johnson Counter vs Ring Counter

Q1. Write a short note about a ring counter. (5 marks)

[CSE 205 | L-2/T-1 | 2022–23]

Answer

1. Introduction and Definition

A **Ring Counter** is a synchronous sequential logic circuit constructed from a shift register, where the output of the final flip-flop is fed back directly to the data input of the first flip-flop. This creates a circular, closed-loop data path. It is primarily used to generate a sequence of timing pulses or control signals. Mathematically, for an N -bit ring counter consisting of flip-flops $FF_0, FF_1, \dots, FF_{N-1}$, the next-state input for any flip-flop is given

376

by:

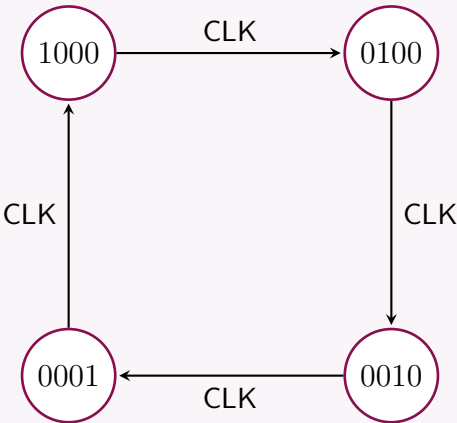
$$D_i = Q_{i-1} \quad \text{for } 0 < i < N$$
$$D_0 = Q_{N-1} \quad (\text{Feedback loop})$$

2. Principle of Operation & Initialization

Unlike a standard binary counter that sequences through 2^N states, an N -bit ring counter typically cycles through exactly N unique states. This is known as a *Modulo- N* counter. To function properly, the ring counter must be **initialized** before the clock begins. A common asynchronous PRESET/CLEAR configuration is used to inject a single logic 1 into the first flip-flop while clearing the rest. For instance, a 4-bit ring counter is initialized to the state $Q_0Q_1Q_2Q_3 = 1000$. With each subsequent rising edge of the clock (CLK), this single 1 shifts one position to the right, eventually wrapping around from the last flip-flop back to the first.

3. State Transition FSM (4-bit Example)

The finite state machine (FSM) representation below illustrates the normal operational sequence of a 4-bit ring counter initialized to 1000.



4. State Table

The state table clearly shows the "diagonal" propagation of the logic 1 bit across the clock cycles.

Clock State	Q_0	Q_1	Q_2	Q_3
T_0 (Init)	1	0	0	0
T_1	0	1	0	0
T_2	0	0	1	0
T_3	0	0	0	1

5. Key Characteristics

- **Self-Decoding:** The most significant advantage of a ring counter is that it is self-decoding. No combinational logic (decoders) is required to determine the active state, as each state has one and only one active output.
- **State Wastage:** It is highly inefficient in terms of hardware utilization. An N -bit ring counter utilizes only N out of the 2^N possible states.
- **Error Propagation:** If a noise transient alters a bit (e.g., creating 1100 instead of 0100), the error will circulate indefinitely unless additional self-correcting combinational logic is implemented in the feedback path.

Q2. Distinguish between a ring counter and a Johnson counter. Design a 5-bit ring counter and a 5-bit Johnson counter using shift registers and illustrate their operation using timing diagrams. (10 marks)

[CSE 205 | L-2/T-1 | 2020–21]

↪ Similar: 2019–20

Answer

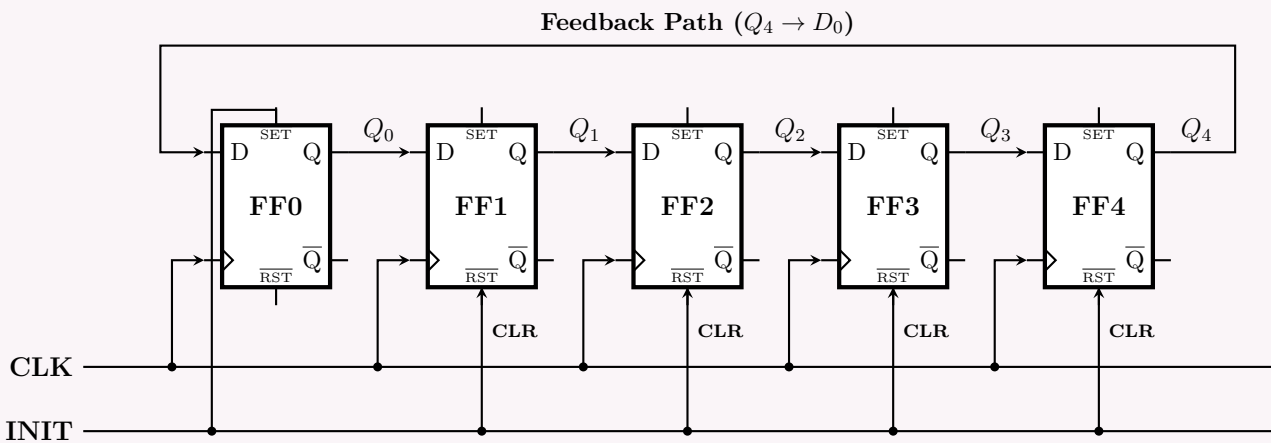
1. Distinguishing Between Ring Counter and Johnson Counter

Both Ring and Johnson counters are specialized sequential circuits based on shift registers. They differ primarily in their feedback mechanism, which dictates the number of distinct states and the sequence generated.

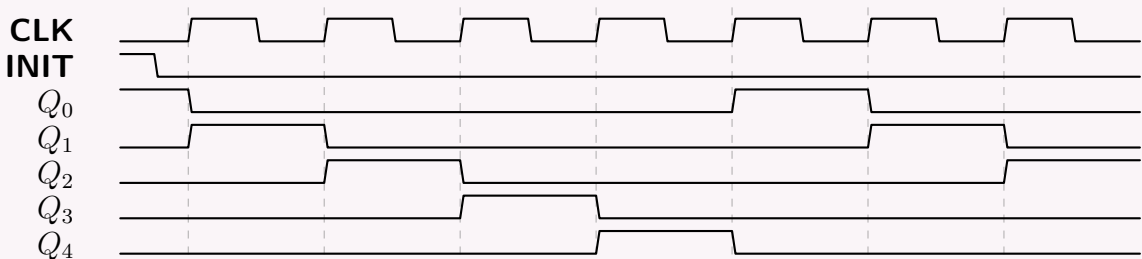
Feature	Ring Counter	Johnson (Switch-Tail) Counter
Feedback Connection	True output of the last FF (Q_n) is fed back to the input of the first FF (D_0).	Complemented output of the last FF ($\overline{Q_n}$) is fed back to the input of the first FF (D_0).
Total Distinct States	N states (Modulo- N).	$2N$ states (Modulo- $2N$).
Used States vs. Total	Highly inefficient (N out of 2^N).	More efficient ($2N$ out of 2^N).
Self-Decoding	Yes. Each state has a unique active FF.	No. Requires a 2-input logic gate to decode states.
Initialization	Typically requires one FF to be 1 and all others 0 (e.g., 10000).	All FFs are typically cleared to 0 initially (e.g., 00000).

2. Five-Bit Ring Counter Design

A 5-bit ring counter requires 5 D flip-flops (FF_0 to FF_4). The feedback loop is $D_0 = Q_4$, while internal shifts are $D_i = Q_{i-1}$. We use an asynchronous **INIT** line connected to the Preset (PR) of FF_0 and the Clear (CLR) of FF_1 – FF_4 to load the state 10000.

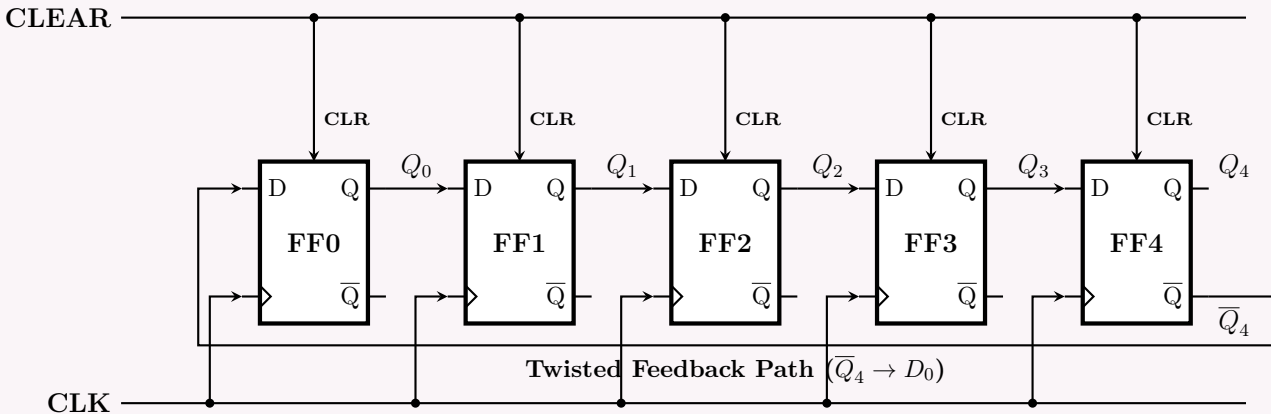


Timing Diagram (5-Bit Ring Counter): The single 1 bit cascades sequentially.

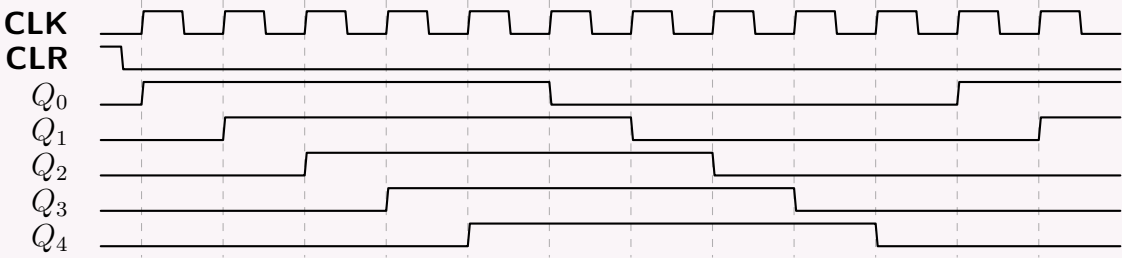


3. Five-Bit Johnson (Switch-Tail) Counter Design

A 5-bit Johnson counter utilizes the exact same basic architecture as the ring counter, except the next-state input to the first flip-flop is inverted: $D_0 = \overline{Q_4}$. The internal shift path remains $D_i = Q_{i-1}$. A common asynchronous **CLEAR** is applied to start the counter at state 00000.



Timing Diagram (5-Bit Johnson Counter): The sequence fills with 1s (spanning $N = 5$ clock pulses), followed by clearing with 0s (spanning the next $N = 5$ clock pulses), completing a $2N = 10$ state cycle.



Q3. Design a 4-bit Johnson counter using a shift register and illustrate its operation using a timing diagram. **(7 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Structural Design and Operation Principle

A **4-bit Johnson Counter**, also known as a twisted ring counter or switch-tail counter, is a synchronous sequential circuit designed using a 4-bit shift register. The primary distinction of this counter is its feedback mechanism: the *complemented* (inverted) output of the last flip-flop ($\overline{Q_3}$) is connected back to the data input of the first flip-flop (D_0).

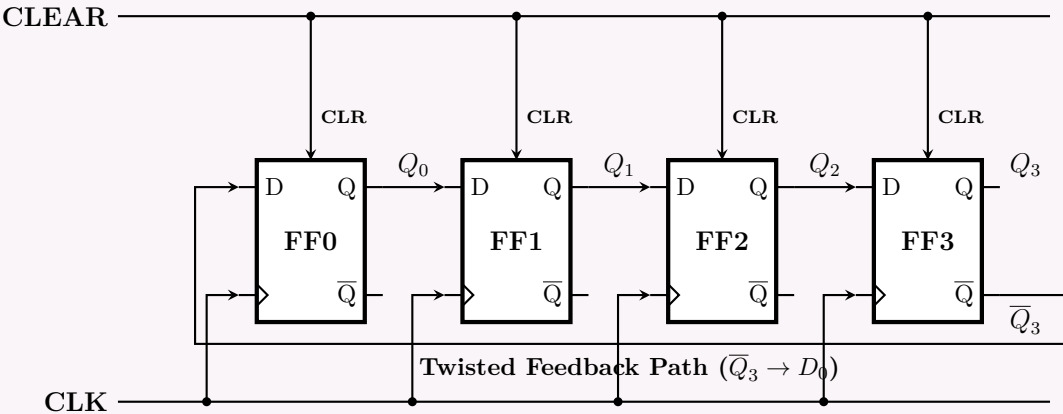
Using four D-type flip-flops (FF_0 to FF_3) driven by a common clock (CLK), the next-state logic equations are:

$$D_0 = \overline{Q_3} \quad (\text{Twisted feedback loop})$$
$$D_1 = Q_0$$
$$D_2 = Q_1$$
$$D_3 = Q_2$$

An N -bit Johnson counter yields exactly $2N$ valid states. Therefore, a 4-bit counter provides a modulo-8 counting sequence. To guarantee the counter begins in a known state, an asynchronous **CLEAR** signal is applied to reset all flip-flops to 0 prior to operation.

2. Logic Circuit Diagram

The schematic below depicts the 4-bit Johnson counter. The twisted feedback path from $\overline{Q_3}$ to D_0 is routed along the bottom of the forward-shifting data path.



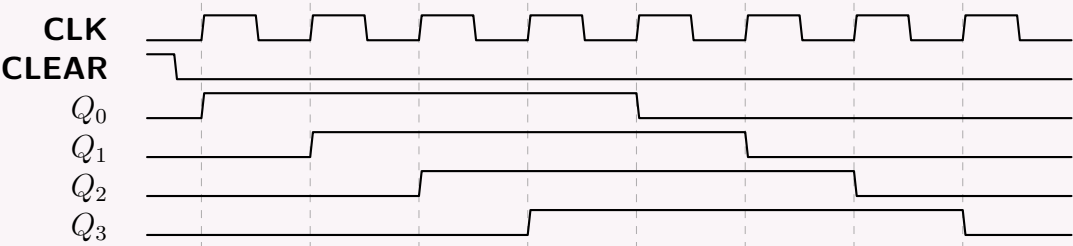
3. State Sequence (Truth Table)

Initialized to 0000, the counter smoothly fills with 1s from the left (since $D_0 = \overline{0} = 1$). Once full (1111), it clears with 0s from the left (since $D_0 = \overline{1} = 0$), resulting in $2 \times 4 = 8$ distinct states.

Clock Pulse	Q_0	Q_1	Q_2	Q_3	Next $D_0 (\overline{Q_3})$
Initial	0	0	0	0	1
1	1	0	0	0	1
2	1	1	0	0	1
3	1	1	1	0	1
4	1	1	1	1	0
5	0	1	1	1	0
6	0	0	1	1	0
7	0	0	0	1	0
8	0	0	0	0	1

4. Timing Diagram

The timing diagram illustrates the state propagation. Notice that the output of each flip-flop is a square wave with a 50% duty cycle over the 8 clock pulse sequence, but shifted by exactly one clock cycle relative to the previous stage.



Q4. Design a 4-bit Johnson Counter. What is the disadvantage of a Johnson Counter? What is the difference between a Johnson Counter and a Ring Counter? **(10 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Design of a 4-Bit Johnson Counter

A **Johnson Counter** (or Switch-Tail Ring Counter) is constructed using a standard shift register, with the distinguishing feature that the *complemented* output of the final flip-flop is fed back to the input of the first flip-flop.

For a 4-bit design utilizing D-type flip-flops (FF_0 through FF_3), the synchronous next-state equations are:

$$\begin{aligned} D_0 &= \overline{Q_3} \quad (\text{Twisted feedback loop}) \\ D_1 &= Q_0 \\ D_2 &= Q_1 \\ D_3 &= Q_2 \end{aligned}$$

The circuit is initialized to an all-zero state (0000) using an asynchronous CLEAR signal. Upon receiving clock pulses, the counter fills with 1s from the left, and then clears with 0s from the left, yielding $2N = 8$ distinct states.

Logic Diagram:

State Sequence:

Clock Pulse	Q_0	Q_1	Q_2	Q_3	Next $D_0 (\overline{Q_3})$
0 (Initial)	0	0	0	0	1
1	1	0	0	0	1
2	1	1	0	0	1
3	1	1	1	0	1
4	1	1	1	1	0
5	0	1	1	1	0
6	0	0	1	1	0
7	0	0	0	1	0

2. Disadvantages of a Johnson Counter

- Requires State Decoding Logic:** Unlike a standard ring counter which is self-decoding (one active bit per state), a Johnson counter requires additional combinational logic (specifically, 2-input AND or NAND gates) to uniquely decode and identify each specific state.
- Susceptibility to "Lockout" (Invalid States):** An N -bit shift register has 2^N possible states, but the Johnson counter only uses $2N$ states. For a 4-bit system, $16 - 8 = 8$ states are unused (e.g., 0101). If an external noise glitch forces the counter into one of these unused states, it will circulate in an invalid sequence indefinitely unless self-correcting feedback logic is explicitly added to the design.

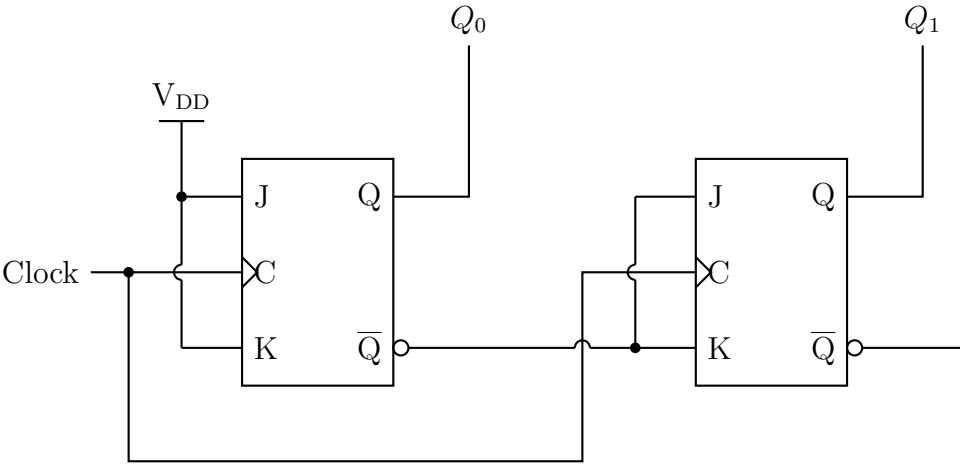
3. Difference Between Johnson Counter and Ring Counter

380

Parameter	Ring Counter	Johnson Counter
Feedback Path	True output of the last FF (Q_n) is fed back to the first FF (D_0).	Complemented output of the last FF ($\overline{Q_n}$) is fed back to the first FF (D_0).
Number of States	Counts N distinct states (Modulo- N).	Counts $2N$ distinct states (Modulo- $2N$).
Hardware Efficiency	Uses only N out of 2^N possible states (Very low efficiency).	Uses $2N$ out of 2^N possible states (Better efficiency).
Decoding	Self-decoding. No logic gates required to identify a state.	Not self-decoding. Requires a 2-input logic gate per state.
Initialization	Must be initialized with a single 1 (e.g., 1000).	Usually initialized to all 0s (e.g., 0000).

Counter Direction Analysis

Q1. A counter circuit has V_{DD} assigned to Logic 1; the current values of LSB Q_0 and MSB Q_1 are 0 and 1 respectively. Find the direction (up or down) in which the circuit counts when the next clock pulse occurs. What would have to be altered to make the circuit count in the other direction?



(7 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

1. Circuit Analysis and Boolean Equations

The provided circuit is a **2-bit Synchronous Counter** constructed using two JK flip-flops. The clock signal is connected simultaneously to the clock inputs (C) of both flip-flops, ensuring synchronous operation.

Let us determine the input equations (excitation logic) for both flip-flops from the schematic:

- For LSB Flip-Flop (FF_0):** Both J_0 and K_0 are tied directly to V_{DD} (Logic 1).

$$J_0 = 1$$
$$K_0 = 1$$

Using the JK flip-flop characteristic equation $Q(t+1) = J\overline{Q}(t) + \overline{K}Q(t)$, we get:

$$Q_0(t+1) = 1 \cdot \overline{Q_0(t)} + 0 \cdot Q_0(t) = \overline{Q_0(t)}$$

This means FF_0 is in **toggle mode** and will invert its state on every clock pulse.

- For MSB Flip-Flop (FF_1):** The J_1 and K_1 inputs are tied together and connected to the inverted output ($\overline{Q_0}$) of the first flip-flop.

$$J_1 = \overline{Q_0(t)}$$
$$K_1 = \overline{Q_0(t)}$$

Since $J_1 = K_1$, FF_1 also operates as a Toggle (T) flip-flop, where it toggles its state *only* if $\overline{Q_0(t)} = 1$ (which implies $Q_0(t) = 0$).

2. Determining the Count Direction

We are given the current state:

- MSB: $Q_1(t) = 1$

- LSB: $Q_0(t) = 0$

This corresponds to the binary state 10_2 , which is the decimal value **2**.
Let's evaluate the next state ($t + 1$) upon the arrival of the clock pulse:

(a) **Calculate next state of Q_0 :**

$$Q_0(t + 1) = \overline{Q_0(t)} = \overline{0} = 1$$

(b) **Calculate next state of Q_1 :** First, determine the inputs to FF_1 :

$$J_1 = K_1 = \overline{Q_0(t)} = \overline{0} = 1$$

Because $J_1 = K_1 = 1$, FF_1 will toggle its current state:

$$Q_1(t + 1) = \overline{Q_1(t)} = \overline{1} = 0$$

The next state of the counter is $Q_1(t + 1)Q_0(t + 1) = 01_2$, which corresponds to the decimal value **1**. Since the count transitioned from **2 to 1**, the circuit is counting in the **DOWN direction**.

3. Complete State Table Validation

To mathematically verify this is a continuous down counter, we can map the entire Finite State Machine (FSM):

Current State		Inputs to FF1		Next State		Decimal
$Q_1(t)$	$Q_0(t)$	$J_1 = \overline{Q_0}$	$K_1 = \overline{Q_0}$	$Q_1(t + 1)$	$Q_0(t + 1)$	Transition
0	0	1	1	1	1	$0 \rightarrow 3$
1	1	0	0	1	0	$3 \rightarrow 2$
1	0	1	1	0	1	$2 \rightarrow 1$
0	1	0	0	0	0	$1 \rightarrow 0$

The sequence is $3 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow 3$. It is strictly a **Modulo-4 Down Counter**.

4. Required Alteration for Upward Counting

To reverse the direction and make the circuit count **UP** (sequence: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$), the MSB (Q_1) must toggle *only* when the LSB (Q_0) transitions from 1 to 0.

In a synchronous design, this means FF_1 should be enabled to toggle when the current state of $Q_0(t)$ is 1.

$$\text{Required Excitation for UP counting: } J_1 = K_1 = Q_0(t)$$

Structural Alteration: To achieve this, we must **disconnect** the wire routing from the inverted output ($\overline{Q_0}$) of the first flip-flop to the J and K inputs of the second flip-flop. Instead, we must **reconnect** the J_1 and K_1 inputs directly to the **true, non-inverted output** (Q_0) of the first flip-flop.

Clock Generator / Divider

Q1. Given a 100 MHz clock signal, derive a circuit using D flip-flops to generate 50 MHz and 25 MHz clock signals. Draw a timing diagram for all three clock signals. **(10 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

1. Principle of Frequency Division

To divide a clock frequency by 2 using a D flip-flop, the inverted output ($\overline{Q}$) is connected directly back to the Data (D) input. The Boolean excitation equation for this configuration is:

$$D = \overline{Q}$$

With this feedback loop, the flip-flop acts as a Toggle (T) flip-flop. On every active edge (e.g., rising edge) of the incoming clock, the flip-flop captures its own inverted state, causing the output Q to toggle. Because it takes two clock edges to complete a full high-low cycle at the output, the output frequency is exactly half of the input clock frequency ($f_{out} = f_{in}/2$).

2. Circuit Derivation

To generate both **50 MHz** and **25 MHz** signals from a **100 MHz** source, we cascade two divide-by-2 D flip-flop stages in a ripple configuration:

- **Stage 1 (FF0):** The 100 MHz clock drives the clock input of FF0. The output Q_0 toggles at half the rate, yielding a **50 MHz** clock signal.
- **Stage 2 (FF1):** The 50 MHz signal (Q_0) is routed into the clock input of FF1. The output Q_1 toggles at half of that rate, yielding a **25 MHz** clock signal.

3. Logic Circuit Diagram

The schematic below illustrates the two cascaded positive-edge triggered D flip-flops.

4. Timing Diagram

Assuming both flip-flops are initially cleared to 0 and trigger on the **rising edge** of their respective clock inputs, the timing diagram illustrates the $f/2$ and $f/4$ frequency division.

Universal Shift Register

Q1. Design a 4-bit universal shift register using multiplexers and D flip-flops. **(13 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Answer

1. Functional Overview

A **4-bit Universal Shift Register (USR)** is a bidirectional register capable of shifting data in both directions (left and right) as well as loading data in parallel. To control these operations, we use a multiplexer at the input of each D flip-flop.

For a 4-bit register, we require:

- Four Positive-Edge Triggered **D Flip-Flops** (FF_3, FF_2, FF_1, FF_0).
- Four **4-to-1 Multiplexers** (MUX).
- Two mode selection lines (S_1, S_0) shared across all multiplexers to dictate the active operation.

2. Mode Selection Table

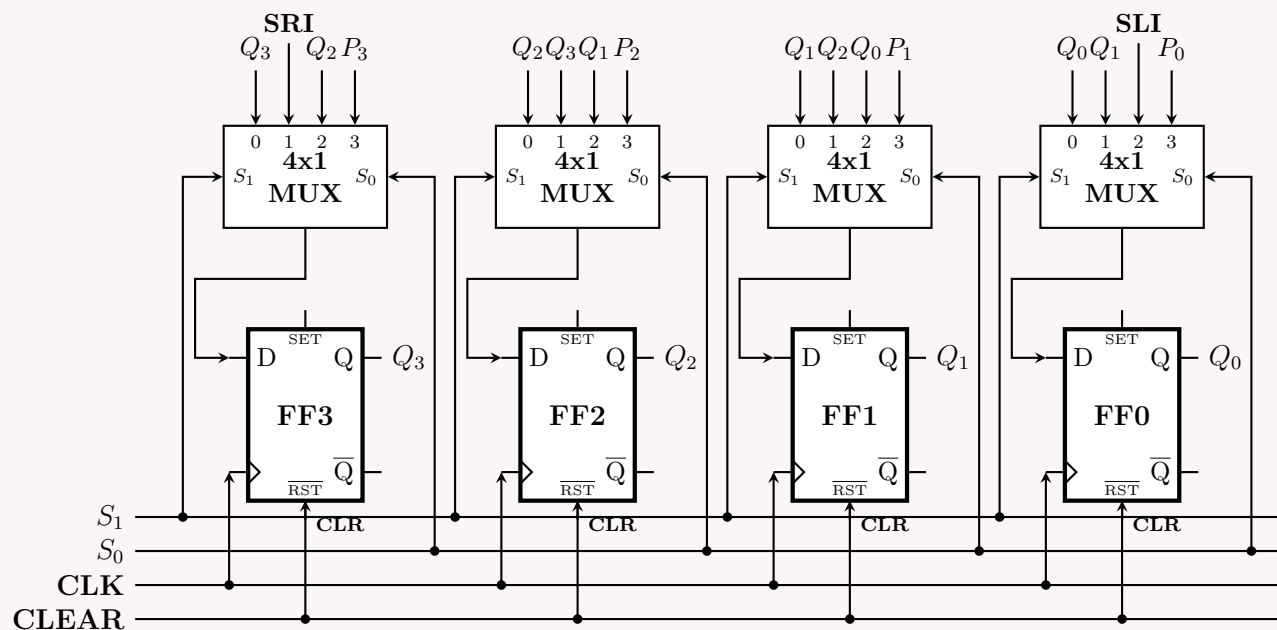
The selection lines S_1 and S_0 control which input of the MUX is routed to the Data (D) input of the corresponding flip-flop.

S_1	S_0	Active MUX Input	Register Operation
0	0	I_0	Hold: No change in state. Current output Q is fed back to D .
0	1	I_1	Shift Right: Data shifts from MSB to LSB ($Q_3 \rightarrow Q_2 \rightarrow Q_1 \rightarrow Q_0$).
1	0	I_2	Shift Left: Data shifts from LSB to MSB ($Q_0 \rightarrow Q_1 \rightarrow Q_2 \rightarrow Q_3$).
1	1	I_3	Parallel Load: External data (P_3, P_2, P_1, P_0) is loaded simultaneously.

3. Circuit Diagram

In the schematic below, the physical routing of all feedback lines is represented by net labels at the top of the multiplexers to maintain visual clarity.

- SRI:** Serial Right Input (Input data when shifting right).
- SLI:** Serial Left Input (Input data when shifting left).
- P_n : Parallel Input data bits.
- Q_n : Output of the respective flip-flops.



How Data Routing Works:

- **Input 0** ($S_1S_0 = 00$): Each MUX passes its own flip-flop's output (Q_n) straight back into its Data input (D), retaining the current memory state.
- **Input 1** ($S_1S_0 = 01$): Each MUX samples the Q output of the flip-flop to its immediate *left*, producing a Right Shift.
- **Input 2** ($S_1S_0 = 10$): Each MUX samples the Q output of the flip-flop to its immediate *right*, producing a Left Shift.
- **Input 3** ($S_1S_0 = 11$): The multiplexers bypass all feedback logic and pull in fresh data from the external P bus.

Q2. Compare the hardware efficiency and flexibility of universal shift registers with dedicated shift registers in complex digital systems. (7 marks) [CSE 205 | L-2/T-1 | 2024–25]

Answer

In complex digital systems, the choice between a **Universal Shift Register (USR)** and a **Dedicated Shift Register (DSR)** involves a fundamental engineering trade-off: *flexibility versus hardware efficiency*.

1. Hardware Efficiency Comparison

Hardware efficiency is evaluated based on gate count (silicon area), power dissipation, and propagation delay (maximum clock frequency).

- **Dedicated Shift Registers (High Efficiency):** A DSR is hardwired for a single operation (e.g., Serial-In Serial-Out [SISO] shifting right). Because the output (Q) of one flip-flop connects directly to the input (D) of the next, there is effectively *zero* combinational logic overhead between stages.
 - **Area & Power:** Minimal. It uses the absolute minimum number of transistors required for data storage.
 - **Speed:** Extremely fast. The critical path delay is limited only by the flip-flop's clock-to-Q delay (t_{CQ}) and setup time (t_{setup}). $T_{clk(min)} = t_{CQ} + t_{setup}$.
- **Universal Shift Registers (Low Efficiency):** A USR is designed to perform multiple operations (Hold, Shift Right, Shift Left, Parallel Load). To achieve this, it requires an N -to-1 multiplexer (typically a 4:1 MUX) at the D input of *every* flip-flop, along with control lines (S_1, S_0) routed to all stages.
 - **Area & Power:** High. The added MUXes significantly increase the logic gate count, routing complexity, and leakage/dynamic power consumption.
 - **Speed:** Slower. The multiplexer introduces a combinational gate delay (t_{MUX}) into the data path. The new critical path becomes $T_{clk(min)} = t_{CQ} + t_{MUX} + t_{setup}$, lowering the maximum operating frequency.

2. Flexibility Comparison

- **Dedicated Shift Registers (Rigid):** DSRs offer zero dynamic flexibility. A register hardwired to shift left cannot parallel load or shift right. Changing its function requires physically redesigning the circuit or synthesizing new hardware.
- **Universal Shift Registers (Highly Adaptable):** USRs provide maximum sequential flexibility. By merely altering the digital control signals on the selection lines (e.g., $S_1S_0 = 00$ for hold, 01 for right shift, 11 for load), the hardware dynamically alters its data-routing topology on the fly, cycle-by-cycle, without any structural changes.

3. Summary Matrix and System-Level Applications

Parameter	Dedicated Shift Register	Universal Shift Register
Architecture	Direct cascaded D flip-flops.	D flip-flops with input Multiplexers.
Hardware Over-head	Minimal (Registers only).	High (Registers + MUXes + Routing).
Max Clock Speed	Very High (Short critical path).	Moderate (MUX delay in critical path).
Dynamic Capability	Fixed single-function.	Multi-function (Software/Control driven).
Ideal Applications	Deep data pipelines, digital delay lines, FIFOs, simple UART serializers.	CPU/ALU registers, complex SPI/I2C controllers, dynamically programmable buffers.

Conclusion: In complex System-on-Chip (SoC) designs, architects deploy **DSRs** in the datapath where high-speed, repetitive data streaming is required (e.g., memory pipelines) to save power and area. Conversely, **USRs** are instantiated in control units, processors, and communication peripherals where dynamic operation is strictly necessary, justifying the hardware penalty.

Q3. Use a 2-to-4 decoder, NAND gates and edge triggered D flip-flops to design a 4-bit shift register module that has the following functions:

S_1	S_0	Mode
0	0	Shift right (all 4 bits)
0	1	Shift left (all 4 bits)
1	0	Synchronous common clear
1	1	Synchronous parallel load

(10 marks)

[CSE 205 | L-2/T-1 | 2021–22]

Answer

To design the multi-function 4-bit shift register, we divide the logic into three components: the **mode decoding logic**, the **data selection logic (multiplexing)**, and the **storage logic (D flip-flops)**. We will derive the inputs for each D flip-flop and map the logic entirely to NAND gates.

1. Decoder Output Logic
A 2-to-4 decoder takes the mode control signals S_1 and S_0 and generates four mutually exclusive active-high outputs (Y_0 through Y_3).

$$Y_0 = \overline{S_1} \cdot \overline{S_0} \quad (\text{Active for Shift Right})$$
$$Y_1 = \overline{S_1} \cdot S_0 \quad (\text{Active for Shift Left})$$
$$Y_2 = S_1 \cdot \overline{S_0} \quad (\text{Active for Synchronous Clear})$$
$$Y_3 = S_1 \cdot S_0 \quad (\text{Active for Parallel Load})$$

2. D Flip-Flop Input Equations (SOP Form)
Let the four D flip-flops be FF_3 (MSB) down to FF_0 (LSB), with outputs Q_3, Q_2, Q_1, Q_0 . For a generalized stage i (where $i \in \{0, 1, 2, 3\}$), the next state D_i must capture:

- **Right Shift Data** (R_i) when $Y_0 = 1$.
- **Left Shift Data** (L_i) when $Y_1 = 1$.
- **Logic 0** when $Y_2 = 1$ (Synchronous Clear).
- **Parallel Data** (P_i) when $Y_3 = 1$.

Writing the general Sum-of-Products (SOP) boolean expression for the input D_i :

$$D_i = (Y_0 \cdot R_i) + (Y_1 \cdot L_i) + (Y_2 \cdot 0) + (Y_3 \cdot P_i)$$
$$D_i = Y_0 R_i + Y_1 L_i + Y_3 P_i \quad (\text{Since } Y_2 \cdot 0 = 0)$$

3. Conversion to NAND Logic
The problem strictly requires the use of NAND gates. We apply double negation (Involution Law) and De Morgan's Law ($\overline{A + B} = \overline{A} \cdot \overline{B}$) to convert the SOP equation into a NAND-NAND implementation:

$$D_i = \overline{\overline{Y_0 R_i + Y_1 L_i + Y_3 P_i}} \quad (\text{Double Involution})$$
$$D_i = \overline{(\overline{Y_0 R_i}) \cdot (\overline{Y_1 L_i}) \cdot (\overline{Y_3 P_i})} \quad (\text{De Morgan's Law})$$

This resulting equation requires exactly three **2-input NAND gates** and one **3-input NAND gate** per stage.

Note on Synchronous Clear: When $S_1S_0 = 10$, $Y_2 = 1$, which forces $Y_0 = 0$, $Y_1 = 0$, and $Y_3 = 0$. Evaluating the NAND equation yields $D_i = \overline{(0)} \cdot \overline{(0)} \cdot \overline{(0)} = 1 \cdot 1 \cdot 1 = 1 = 0$. The flip-flop naturally clears without requiring Y_2 to be physically wired to the data logic!

4. Logic Circuit Implementation (Per Stage)

Due to the wiring density of 4 bits, the highly modular logic for a **single generalized stage** (FF_i) is shown below. This identical cell is instantiated four times in the full register.

5. Full 4-Bit Routing Map

To assemble the complete 4-bit shift register, four instances of the above circuit are placed side-by-side. The Y decoder lines are shared across all stages as parallel control buses. The data line routing is defined as follows (where SRI is Serial Right Input and SLI is Serial Left Input):

Stage	Right Shift Data (R_i)	Left Shift Data (L_i)	Parallel Load	Output
FF₃ (MSB)	$R_3 = SRI$	$L_3 = Q_2$	P_3	Q_3
FF₂	$R_2 = Q_3$	$L_2 = Q_1$	P_2	Q_2
FF₁	$R_1 = Q_2$	$L_1 = Q_0$	P_1	Q_1
FF₀ (LSB)	$R_0 = Q_1$	$L_0 = SLI$	P_0	Q_0

Q4. Draw the logic diagram of a 3-bit universal shift register with mode selection inputs s_2, s_1, s_0 . The register operates with eight modes: (000) No change, (001) Invert all bits, (010) Set all bits to 0, (011) Set all bits to 1, (100) Parallel load, (101) Shift left, (110) Shift right, (111) No change. **(15 marks)** [CSE 205 | L-2/T-1 | 2016–17]

Answer

1. Functional Specification and Design Methodology

A 3-bit universal shift register capable of executing eight distinct functional operations can be optimally constructed utilizing three edge-triggered **D Flip-Flops** (FF_2, FF_1, FF_0) paired with three **8-to-1 Multiplexers** (MUX_2, MUX_1, MUX_0). The state outputs of the register are defined as Q_2 (Most Significant Bit), Q_1 , and Q_0 (Least Significant Bit). The 3-bit control word consisting of select signals (s_2, s_1, s_0) determines which of the 8 input channels is routed to the D input of each flip-flop during every positive clock edge.

2. Mode Control & Multiplexer Allocation Table

The functional mapping below routes the required system operations to corresponding channels (0 through 7) of the 8:1 multiplexers:

Mode Selection and Multiplexer Channel Assignments					
s_2	s_1	s_0	Operation Mode	MUX Channel Input	Next-State Connection Logic
0	0	0	No Change	Channel 0	$D_i = Q_i$
0	0	1	Invert All Bits	Channel 1	$D_i = \overline{Q_i}$
0	1	0	Set All Bits to 0	Channel 2	$D_i = 0$ (GND)
0	1	1	Set All Bits to 1	Channel 3	$D_i = 1$ (VCC)
1	0	0	Parallel Load	Channel 4	$D_i = I_i$
1	0	1	Shift Left	Channel 5	$D_i = Q_{i-1}$ (where $D_0 = SI_{Left}$)
1	1	0	Shift Right	Channel 6	$D_i = Q_{i+1}$ (where $D_2 = SI_{Right}$)
1	1	1	No Change	Channel 7	$D_i = Q_i$

3. Next-State Excitation Equations

By applying the standard characteristic equation of a D-type flip-flop ($Q(t+1) = D$) alongside the switching logic of an 8:1 multiplexer, the synchronous excitation equations for each stage are analytically derived below:

Stage 2: Most Significant Bit (Q_2)

$$D_2 = \bar{s}_2\bar{s}_1\bar{s}_0Q_2 + \bar{s}_2\bar{s}_1s_0\bar{Q}_2 + \bar{s}_2s_1\bar{s}_0(0) + \bar{s}_2s_1s_0(1) \\ + s_2\bar{s}_1\bar{s}_0I_2 + s_2\bar{s}_1s_0Q_1 + s_2s_1\bar{s}_0SI_{\text{Right}} + s_2s_1s_0Q_2$$

Stage 1: Central Bit (Q_1)

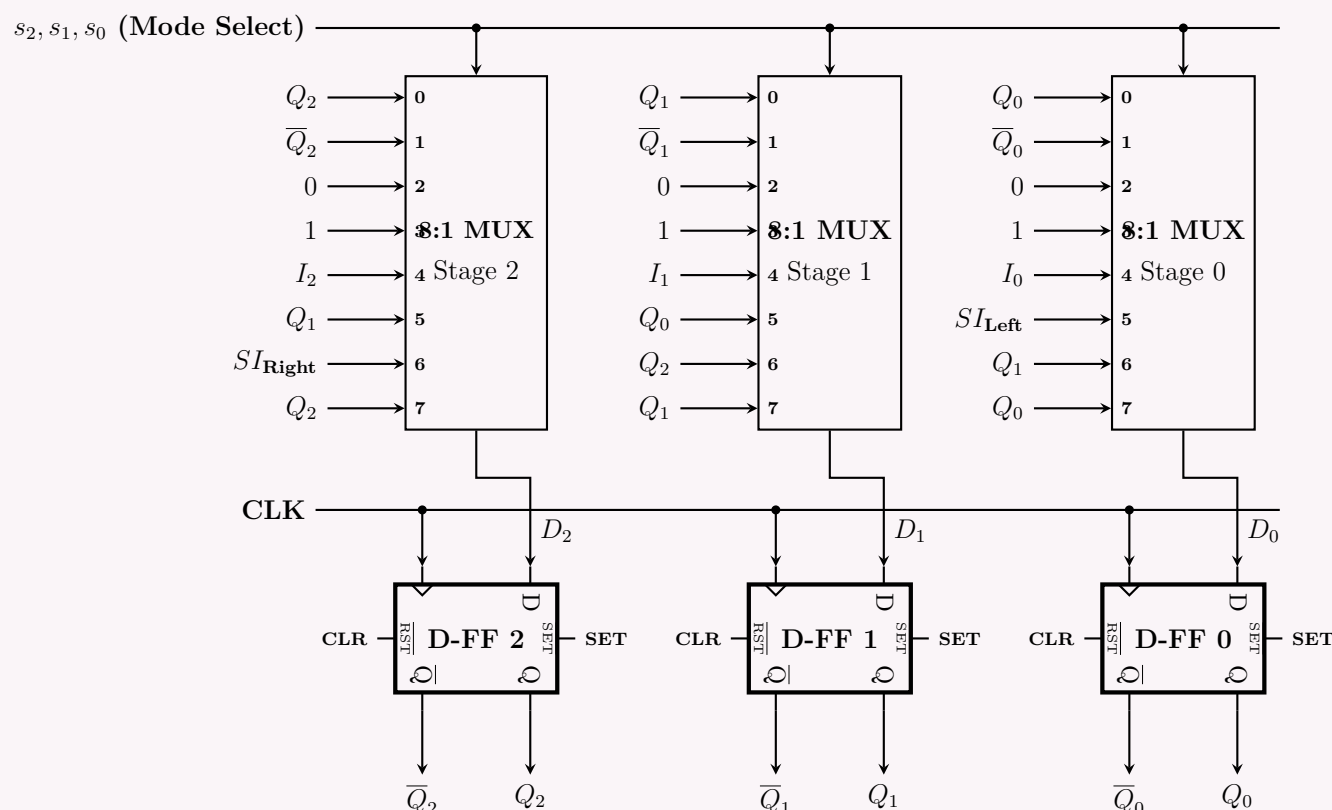
$$D_1 = \bar{s}_2\bar{s}_1\bar{s}_0Q_1 + \bar{s}_2\bar{s}_1s_0\bar{Q}_1 + \bar{s}_2s_1\bar{s}_0(0) + \bar{s}_2s_1s_0(1) \\ + s_2\bar{s}_1\bar{s}_0I_1 + s_2\bar{s}_1s_0Q_0 + s_2s_1\bar{s}_0Q_2 + s_2s_1s_0Q_1$$

Stage 0: Least Significant Bit (Q_0)

$$D_0 = \bar{s}_2\bar{s}_1\bar{s}_0Q_0 + \bar{s}_2\bar{s}_1s_0\bar{Q}_0 + \bar{s}_2s_1\bar{s}_0(0) + \bar{s}_2s_1s_0(1) \\ + s_2\bar{s}_1\bar{s}_0I_0 + s_2\bar{s}_1s_0SI_{\text{Left}} + s_2s_1\bar{s}_0Q_1 + s_2s_1s_0Q_0$$

4. Logic Architecture Diagram

The full modular logic schematic illustrates how data feedback loops, serial links, and external controls interconnect across all three stages:



Q5. Draw the logic diagram of a two-bit register with two D flip-flops and two 4×1 multiplexers with mode selection inputs s_1 and s_0 . The register operates with four modes: (00) No change, (01) Complement the two outputs, (10) Clear register to 0 (synchronous), (11) Load parallel data. **(12 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Answer

1. Functional Specification and Mode Analysis

A 2-bit register with synchronous multi-mode control can be structured using two edge-triggered D flip-flops (designated as FF_1 for the Most Significant Bit and FF_0 for the Least Significant Bit) paired with two 4×1 multiplexers (MUX_1 and MUX_0). The register state is modified synchronously at each active clock edge based on the 2-bit control word (s_1, s_0).

The behavior of each individual mode is specified below:

- **Mode (00) - No Change:** The current output state of each flip-flop is routed back to its own input, preserving the stored bits: $Q_i(t+1) = Q_i(t)$.
- **Mode (01) - Complement Outputs:** The inverted state output ($\bar{Q}_i$) is fed back into the D input, causing the state to toggle on the next active clock edge: $Q_i(t+1) = \bar{Q}_i(t)$.
- **Mode (10) - Synchronous Clear:** A static logical low level (0) is tied to the multiplexer line, driving the outputs to a cleared state (00) synchronously: $Q_i(t+1) = 0$.
- **Mode (11) - Parallel Load:** External data bits (I_1 and I_0) are sampled directly into the register flip-flops: $Q_i(t+1) = I_i$.

2. Multiplexer Configuration Mapping

To implement the specification above, the inputs of each 4×1 multiplexer are hardwired to specific internal signals or external lines as summarized in the assignment table below:

Mode Selection and Multiplexer Channel Interconnections				
s_1	s_0	Operating Mode	MUX Channel Input	Next-State Logic Equation (D_i)
0	0	No Change	Channel 0	$D_i = Q_i$
0	1	Complement Outputs	Channel 1	$D_i = \overline{Q_i}$
1	0	Synchronous Clear	Channel 2	$D_i = 0$ (GND)
1	1	Parallel Load	Channel 3	$D_i = I_i$

3. Next-State Excitation Equations

Using the characteristic equation of a standard D-type flip-flop ($Q(t + 1) = D$), the algebraic expressions for the excitation lines D_1 and D_0 are formulated directly matching the selection line product terms:

Stage 1 (MSB Bit, Q_1)

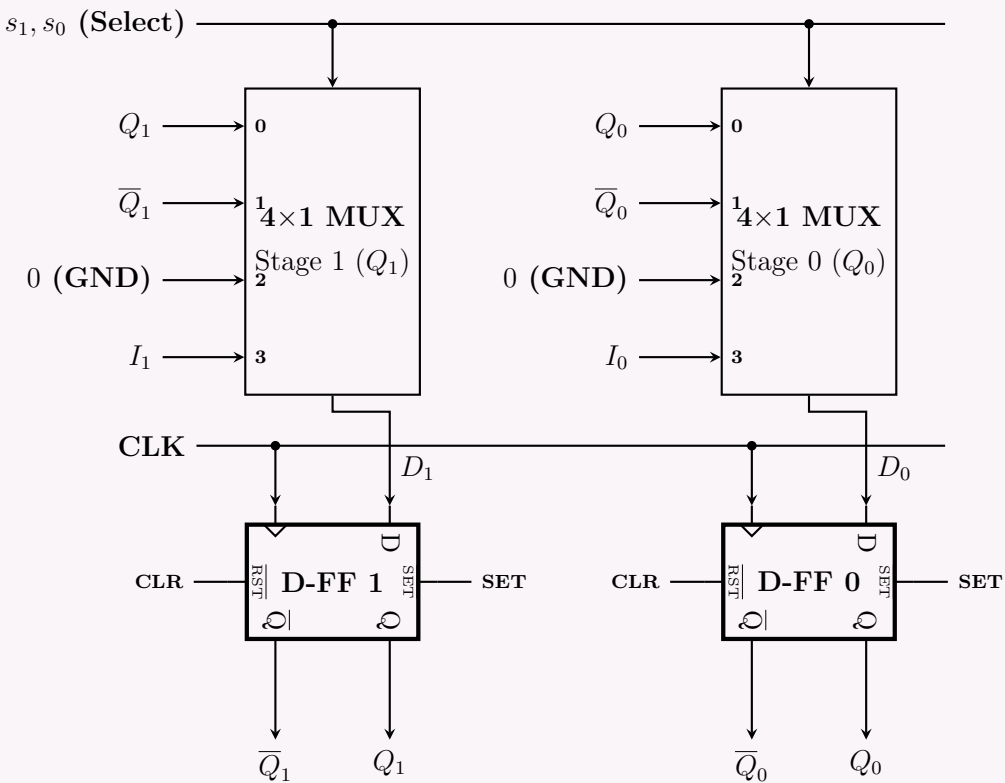
$$D_1 = \overline{s_1}\overline{s_0}Q_1 + \overline{s_1}s_0\overline{Q_1} + s_1\overline{s_0} \cdot 0 + s_1s_0I_1$$

Stage 0 (LSB Bit, Q_0)

$$D_0 = \overline{s_1}\overline{s_0}Q_0 + \overline{s_1}s_0\overline{Q_0} + s_1\overline{s_0} \cdot 0 + s_1s_0I_0$$

4. Synchronous Logic Architecture Diagram

The schematic configuration below outlines the global routing architecture of the register. The selection signals (s_1, s_0) operate as a centralized bus connected in parallel to both multiplexers, and a single common clock distribution network clocks both stages concurrently.



Q6. Draw the circuit diagram of a 4-bit universal shift register using JK flip-flops. (12 marks) [CSE 205 | L-2/T-1 | 2012–13]

Answer

1. Functional Specification

A 4-bit Universal Shift Register is capable of performing four primary operations: Hold (No Change), Shift Right, Shift Left, and Parallel Load. To implement this using **JK flip-flops**, we must pair each flip-flop with a **4×1 Multiplexer (MUX)**. Since a JK flip-flop behaves exactly like a D flip-flop when J and K are complementary inputs ($J = D, K = \overline{D}$), we will route the output of each multiplexer directly to the J input, and pass it through an inverter (NOT gate) to the K input.

2. Mode Control and MUX Input Mapping

The operation is controlled by two mode selection inputs, S_1 and S_0 . Let the four flip-flops be FF_3 (MSB) down to FF_0 (LSB).

Operational Modes and Multiplexer Assignments							
Select Lines		Operating Mode	Function	MUX Input Selection (4×1)			
S_1	S_0			MUX 3	MUX 2	MUX 1	MUX 0
0	0	Hold	$Q_i(t + 1) = Q_i(t)$	Q_3	Q_2	Q_1	Q_0
0	1	Shift Right	$Q_i(t + 1) = Q_{i+1}(t)$	SI_R	Q_3	Q_2	Q_1
1	0	Shift Left	$Q_i(t + 1) = Q_{i-1}(t)$	Q_2	Q_1	Q_0	SI_L
1	1	Parallel Load	$Q_i(t + 1) = I_i$	I_3	I_2	I_1	I_0

Note: SI_R is the Serial Input for Shift Right, and SI_L is the Serial Input for Shift Left.

3. Next-State and Excitation Logic

For any generic stage i (where $i \in \{0, 1, 2, 3\}$), the multiplexer output Y_i is governed by the Boolean equation:

$$Y_i = \bar{S}_1\bar{S}_0(\text{Input 0}) + \bar{S}_1S_0(\text{Input 1}) + S_1\bar{S}_0(\text{Input 2}) + S_1S_0(\text{Input 3})$$

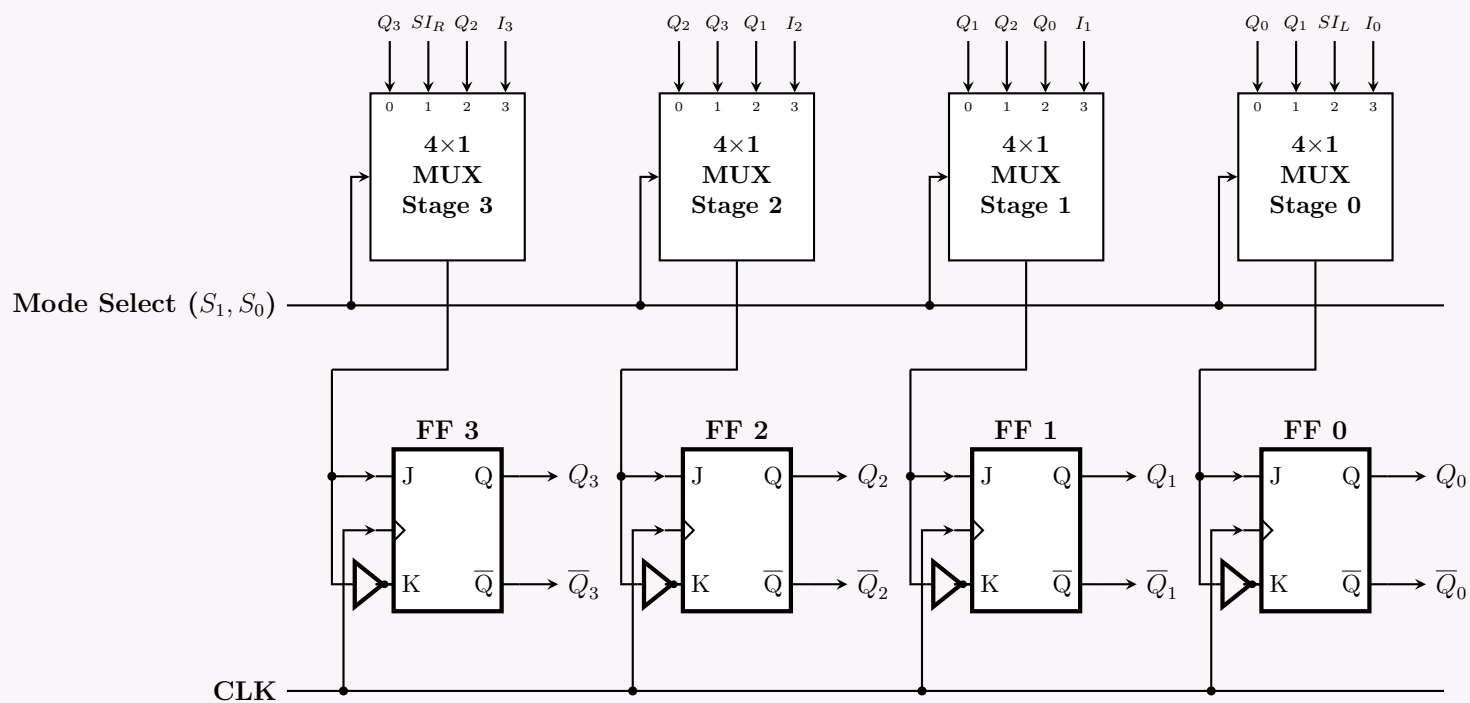
To satisfy the state requirements using a JK flip-flop, the excitation connections are:

$$J_i = Y_i \quad (\text{Direct connection from MUX output})$$

$$K_i = \bar{Y}_i \quad (\text{Inverted connection from MUX output})$$

4. Logic Circuit Diagram

Below is the complete architectural schematic of the 4-bit universal shift register using JK flip-flops.



Serial Adder & Serial Subtractor

Q1. A 4-bit shift register is used for serial data transmission.

- Analyze the timing relationship between input data, clock and output.
- Explain how errors may propagate in serial communication using shift registers.

(16 marks)

[CSE 205 | L-2/T-1 | 2024–25]

Answer

Part (a): Timing Relationship Analysis

In a synchronous 4-bit shift register utilized for serial data transmission, data elements are shifted sequentially through cascaded flip-flops ($FF_3 \rightarrow FF_2 \rightarrow FF_1 \rightarrow FF_0$) at every active transition of the global clock signal. Understanding this temporal mapping requires defining the fundamental hardware timing parameters:

- Setup Time (t_{su}):** The minimum continuous duration for which the serial input data (SI) must remain completely stable *prior* to the active rising clock edge to ensure unambiguous sampling.
- Hold Time (t_h):** The minimum continuous duration for which the serial input data (SI) must remain stable *after* the active rising clock edge to prevent data invalidation.
- Clock-to-Output Propagation Delay (t_{co}):** The temporal interval required for a newly latched data bit to manifest at the output node (Q_i) following the triggering edge of the clock.

Mathematical Timing Governing Relations

To preserve data integrity across the directly cascaded flip-flop stages without incurring race conditions or metastability, the system parameters must strictly satisfy the following inequalities:

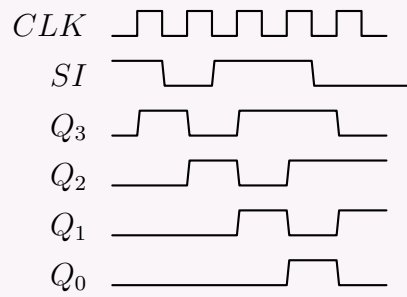
$$t_{co} \geq t_h \quad (\text{To prevent hold-time violations between successive stages})$$

$$T_{clk} \geq t_{co} + t_{su} \quad (\text{To satisfy setup-time constraints for the maximum operating frequency})$$

where T_{clk} represents the total clock period.

Timing Diagram Specification

Consider a concrete sequence where the data bits **1, 0, 1, 1** are sequentially shifted into an initially cleared (0000_2) 4-bit right-shift register. The precise timing interactions between the clock pulses, serial input, and individual stage outputs are detailed below:



- **Clock Edge 1 (T_1):** $SI = 1$ is sampled. After a delay of t_{co} , Q_3 transitions to 1. State is 1000_2 .
- **Clock Edge 2 (T_2):** $SI = 0$ is sampled. Simultaneously, the previous $Q_3 = 1$ is shifted into Q_2 . State becomes 0100_2 .
- **Clock Edge 3 (T_3):** $SI = 1$ is sampled. Q_3 becomes 1, Q_2 steps down to 0, and Q_1 captures 1. State becomes 1010_2 .
- **Clock Edge 4 (T_4):** $SI = 1$ is sampled. The entire 4-bit vector is now fully loaded into the register. State reaches 1101_2 .

Part (b): Error Propagation Mechanics

Serial data transmission using shift registers is highly sensitive to external electrical perturbations. Errors propagate through the pipeline via two primary failure mechanisms:

1. Bit-Flipping Errors (Transient Channel Noise)

When a transient noise spike corrupts a bit at the Serial Input (SI) line precisely during the setup/hold window, an incorrect logic state is latched into the first flip-flop stage (FF_3).

- **Downstream Migration:** Because the output of each stage forms the input to the next, this single inverted bit behaves as valid data during subsequent clock pulses. It migrates through the entire register array over the next $N - 1$ clock cycles (where $N = 4$ is the register bit-width).
- **Temporal Error Window:** A single bit error on the line creates a localized error packet that taints exactly N successive parallel data words or output samples before the corrupted bit is completely flushed out of the serial output node (Q_0).

2. Clock Jitter and Synchronization Slips (Framing Errors)

A more catastrophic error propagation occurs due to clock distribution issues, such as clock jitter, ground bounce, or transmission line reflections, causing an extra clock glitch or a missed clock edge.

- **The Bit Slip Phenomenon:** If the receiver shift register experiences an extra clock pulse relative to the transmitter, it executes an early shift operation. This creates a spatial displacement known as a **bit slip**.
- **Avalanche Corruption:** Unlike a single bit flip, a bit slip disrupts the structural alignment of the entire incoming serial data stream with respect to the word boundaries. Every single subsequent packet or word decoded by the system will be shifted out of phase, resulting in absolute data corruption across all future transmissions. This error persists indefinitely until a hardware reset or a distinct frame synchronization pattern (such as a Start/Stop bit sequence) re-aligns the system boundaries.

Q2. Design a sequential circuit to copy the content of shift register A to shift register B . Use a block diagram to represent the shift register in the diagram. **(12 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Answer

1. System Design and Control Strategy

To copy the contents of an N -bit shift register A to an N -bit shift register B without destroying the original data in A , a **destructive-read with nondestructive-recirculation** technique must be implemented.

The architecture consists of two primary parts: the data path (comprising the shift registers) and the control unit (comprising a control flip-flop, gating logic, and a step counter).

Operational Requirements

- Recirculation Loop for Register A:** The Serial Output of Register A (SO_A) must be tied directly back to its own Serial Input (SI_A). This ensures that as bits are shifted out to register B , they are simultaneously wrapped around and re-entered into register A . After exactly N clock cycles, register A returns to its pristine initial state.
- Transfer Path to Register B:** SO_A is connected directly to the Serial Input of Register B (SI_B).
- Synchronous Gating Control:** A control block is designed to permit exactly N active clock transitions to both registers upon receiving a transient *Start* command pulse, disabling the shift clock immediately after the N -th cycle to freeze the data.

2. Step-by-Step State Transition Analysis

Let both registers be 4-bit arrays ($N = 4$). Let the initial contents of Register A be represented by the vector $A = [a_3, a_2, a_1, a_0]$ and Register B be empty or in an arbitrary state $B = [x_3, x_2, x_1, x_0]$. The sequence of shifts under the gated control clock is derived below:

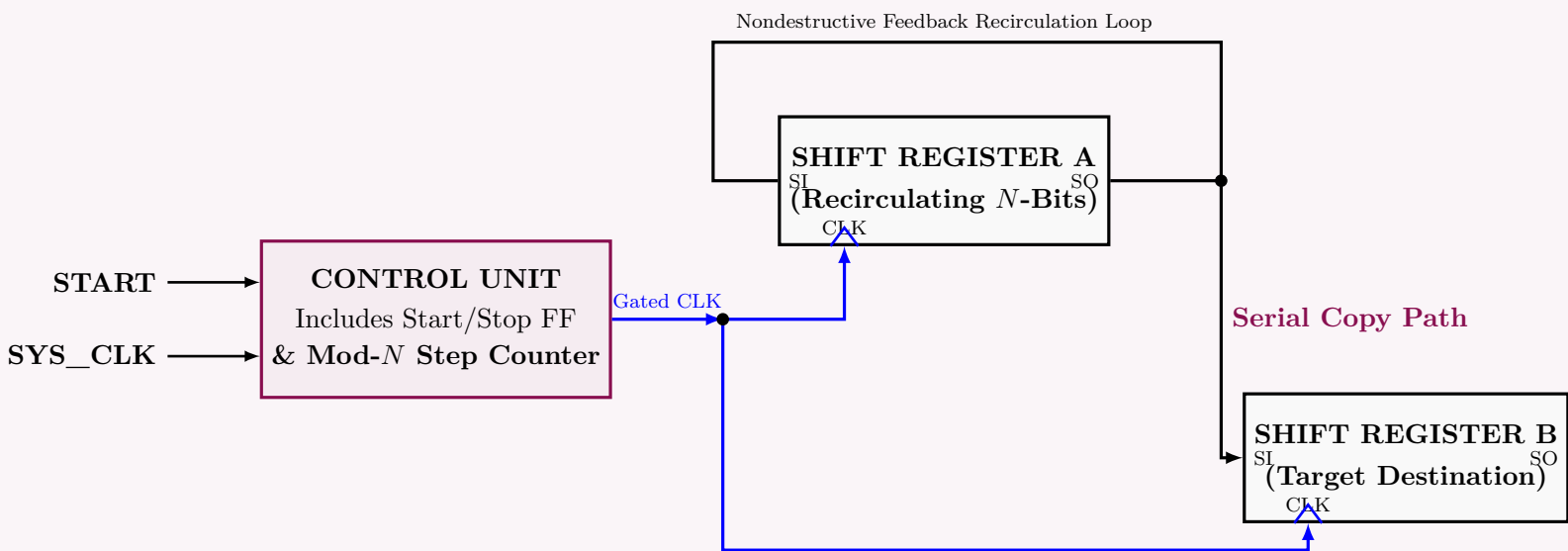
State Trajectory During the Parallel Copy Cycle					
Clock Pulse	Gated Control	Register A Content	SO_A Output	Register B Content	
Initial (t_0)	HIGH	$[a_3, a_2, a_1, a_0]$	a_0	$[x_3, x_2, x_1, x_0]$	
Pulse 1 (t_1)	HIGH	$[a_0, a_3, a_2, a_1]$	a_1	$[a_0, x_3, x_2, x_1]$	
Pulse 2 (t_2)	HIGH	$[a_1, a_0, a_3, a_2]$	a_2	$[a_1, a_0, x_3, x_2]$	
Pulse 3 (t_3)	HIGH	$[a_2, a_1, a_0, a_3]$	a_3	$[a_2, a_1, a_0, x_3]$	
Pulse 4 (t_4)	LOW	$[a_3, a_2, a_1, a_0]$	a_0	$[a_3, a_2, a_1, a_0]$	

As mathematically proven by the sequence above, at t_4 , the terminal condition satisfies:

$$A(t_4) = A(t_0) = [a_3, a_2, a_1, a_0]$$
$$B(t_4) = A(t_0) = [a_3, a_2, a_1, a_0]$$

3. Complete Architectural Block Diagram

The schematic structure below maps the register blocks, the recirculation lines, and the clock-gating hardware configuration required to automate the copy sequence.



Q3. Design a serial subtractor that will perform the operation $A - B$, where $A = a_{n-1} \dots a_1 a_0$ and $B = b_{n-1} \dots b_1 b_0$. The operands are applied to the serial subtractor sequentially, beginning with bits a_0 and b_0 . Use JK flip-flops in your design. **(10 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Answer

1. System Definition and Truth Table

A serial subtractor computes the difference between two multi-bit numbers, A and B , processing one bit per clock cycle starting from the Least Significant Bits (a_0, b_0). The subtractor requires a memory element to store the *Borrow* generated from the current bit operation, which will be utilized in the subsequent clock cycle. We denote:

- x : The current bit of minuend A (a_i).
- y : The current bit of subtrahend B (b_i).
- Q : The present state of the flip-flop, representing the *Borrow-in* (B_{in}) from the previous stage.
- $Q(t + 1)$: The next state of the flip-flop, representing the *Borrow-out* (B_{out}) to the next stage.
- D : The single-bit difference output.

Since the design strictly specifies the use of a **JK flip-flop**, we incorporate the standard JK excitation rules to determine the necessary J and K inputs for every state transition.

State and Excitation Table for Serial Subtractor

Current Inputs & State			Outputs & Next State		JK FF Excitation	
x (Minuend)	y (Subtrahend)	Q (B_{in})	D (Difference)	Q(t + 1) (B_{out})	J	K
0	0	0	0	0	0	X
0	0	1	1	1	X	0
0	1	0	1	1	1	X
0	1	1	0	1	X	0
1	0	0	1	0	0	X
1	0	1	0	0	X	1
1	1	0	0	0	0	X
1	1	1	1	1	X	0

2. K-Map Simplification and Boolean Equations

We utilize Karnaugh Maps to derive the minimized Boolean expressions for the Difference (D), and the flip-flop excitation inputs (J and K).

Difference Output (D)

From the truth table, D is HIGH for minterms $m(1, 2, 4, 7)$. This yields the standard Full-Subtractor difference equation:

		yQ			
		00	01	11	10
x	0		1		1
	1	1		1	

$$D = \overline{x}yQ + \overline{x}y\overline{Q} + x\overline{y}\overline{Q} + xyQ$$

$$D = x \oplus y \oplus Q$$

Flip-Flop Excitation Inputs (J and K)

		K-Map for J			
		yQ			
		00	01	11	10
x	0		-	-	1
	1		-	-	

$$J = \overline{x}y$$

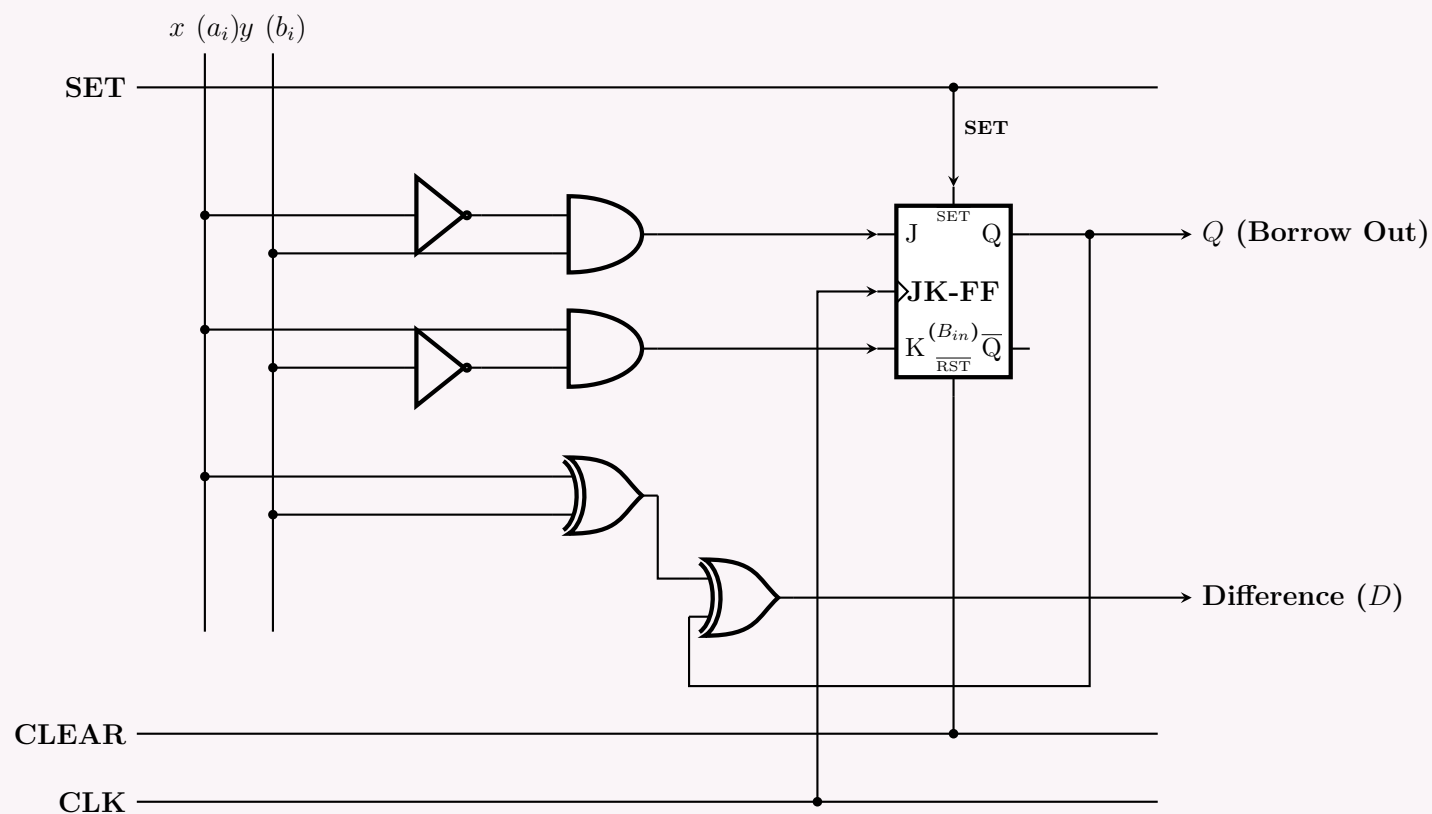
		K-Map for K			
		yQ			
		00	01	11	10
x	0	-			-
	1	-	1		-

$$K = x\overline{y}$$

Note that the elegant optimization of the JK flip-flop eliminates the necessity of feeding the state Q back into the borrow-logic determination gates.

3. Logic Circuit Diagram

The architecture is implemented using two XOR gates to resolve the serial difference, and primary logic gates to synthesize the derived J and K control signals synchronously.



Q4. Design a serial adder that computes the sum of two 4-bit binary numbers A and B stored in individual shift registers. The result of the addition is stored in the shift register representing A . (10 marks) [CSE 205 | L-2/T-1 | 2017–18, 2016–17]

Answer

1. System Architecture and Operational Principle

A serial adder performs the arithmetic addition of two multi-bit binary numbers sequentially, processing one bit position at each clock cycle. The design for a 4-bit serial adder consists of three primary subsystems:

- (a) **Data Storage (Shift Registers):** Two 4-bit Right-Shift Registers (A and B) hold the augend and addend. Register A acts as an accumulator, receiving the generated sum bits back into its Serial Input (SI).
- (b) **Combinational Arithmetic Logic (Full Adder):** A standard 1-bit Full Adder (FA) computes the partial sum and carry-out from the current least significant bits (LSBs) and the previous carry.
- (c) **State Memory (D Flip-Flop):** A positive-edge triggered D-type Flip-Flop (D-FF) stores the carry-out (C_{out}) generated during clock cycle t , presenting it as the carry-in (C_{in}) for the next bit addition at clock cycle $t + 1$.

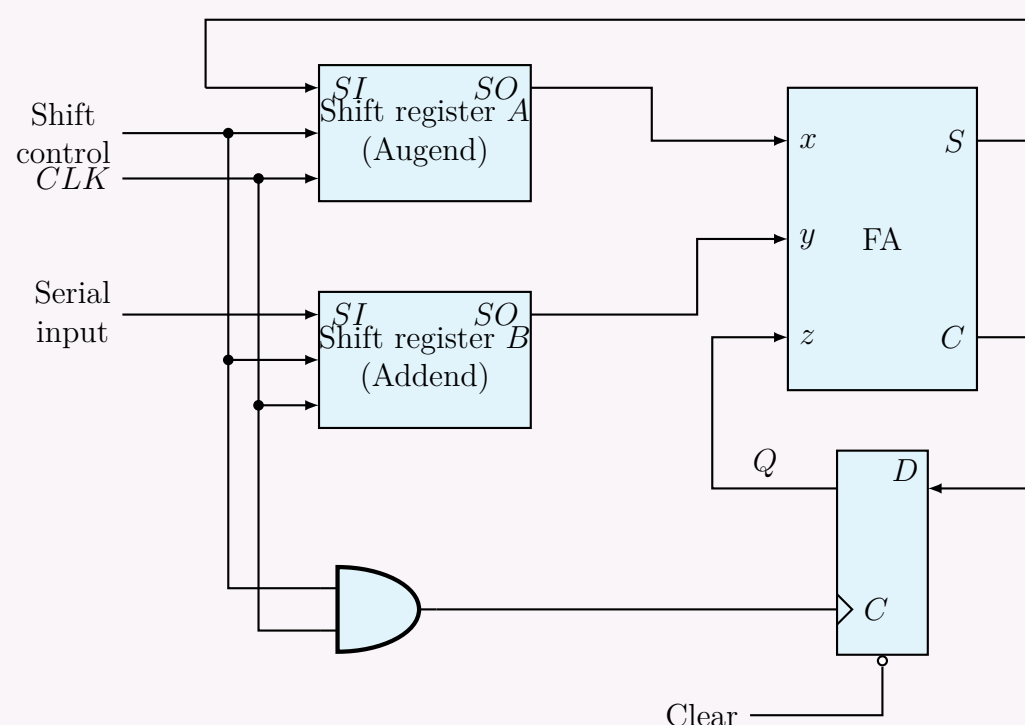
2. Boolean Equations

At any given clock cycle t (where $t \in \{0, 1, 2, 3\}$), the Full Adder inputs are $A_0(t)$ (the serial output of Register A), $B_0(t)$ (the serial output of Register B), and $C_{in}(t)$ (the state of the D-FF). The combinational logic yields:

$$\begin{aligned} S(t) &= A_0(t) \oplus B_0(t) \oplus C_{in}(t) \\ C_{out}(t) &= A_0(t)B_0(t) + A_0(t)C_{in}(t) + B_0(t)C_{in}(t) \end{aligned}$$

Upon the active clock edge, the registers shift right, placing $S(t)$ into the MSB of Register A, while the D-FF captures $C_{out}(t)$ for the next stage.

3. Logic Circuit Diagram



4. State Execution Trace (4-Bit Addition Cycle)

Let the initial state of Register A be $[a_3, a_2, a_1, a_0]$ and Register B be $[b_3, b_2, b_1, b_0]$. The D-FF is initialized with $C_{in} = 0$. The sequence completes in exactly 4 clock cycles:

Serial Addition Trace Table												
Clock	Reg A Contents				Reg B Contents				x (A ₀)	y (B ₀)	C _{in}	Generated Sum (S)
$t = 0$	a_3	a_2	a_1	a_0	b_3	b_2	b_1	b_0	a_0	b_0	0	$S_0 = a_0 \oplus b_0 \oplus 0$
$t = 1$	S_0	a_3	a_2	a_1	0	b_3	b_2	b_1	a_1	b_1	C_1	$S_1 = a_1 \oplus b_1 \oplus C_1$
$t = 2$	S_1	S_0	a_3	a_2	0	0	b_3	b_2	a_2	b_2	C_2	$S_2 = a_2 \oplus b_2 \oplus C_2$
$t = 3$	S_2	S_1	S_0	a_3	0	0	0	b_3	a_3	b_3	C_3	$S_3 = a_3 \oplus b_3 \oplus C_3$
$t = 4$	S₃	S₂	S₁	S₀	0	0	0	0	–	–	C_4	Addition Complete

At $t = 4$, the shift clock is disabled. Register A now accurately contains the 4-bit sum, while Register B is cleared. The final C_4 bit stored in the D-FF represents the overflow carry-out of the full 4-bit addition.

Q5. Design a serial subtractor that subtracts the content of register B from the content of register A , where the contents of A and B are stored in individual shift registers. The result of the subtraction is stored in the shift register representing A . (Both A and B are 4-bit binary numbers.) **(10 marks)** [CSE 205 | L-2/T-1 | 2013–14]

Answer

1. System Architecture and Operational Principle

A serial subtractor sequentially computes the difference between two multi-bit binary numbers ($A - B$), processing exactly one bit position per clock cycle. For a 4-bit subtractor where the result is stored back into the minuend's register, the architecture comprises three main blocks:

(a) **Data Storage (Shift Registers):** Two 4-bit Right-Shift Registers (A and B) hold the minuend and subtrahend, respectively. Register A serves as the accumulator, feeding its serial output into the arithmetic unit and receiving the computed Difference bits back into its Serial Input (SI).

(b) **Combinational Arithmetic Logic (Full Subtractor):** A 1-bit Full Subtractor (FS) computes the partial difference (D) and borrow-out (B_{out}) using the current least significant bits and the borrow from the previous stage.

(c) **State Memory (D Flip-Flop):** A positive-edge triggered D-type Flip-Flop stores the borrow-out (B_{out}) generated during clock cycle t , presenting it as the borrow-in (B_{in}) for the next bit subtraction at clock cycle $t + 1$.

2. Boolean Equations

At any given clock cycle t (where $t \in \{0, 1, 2, 3\}$), the Full Subtractor receives $A_0(t)$ (minuend bit from Register A), $B_0(t)$ (subtrahend bit from Register B), and $B_{in}(t)$ (borrow-in from the D-FF). The combinational logic for the Full Subtractor is:

$$D(t) = A_0(t) \oplus B_0(t) \oplus B_{in}(t)$$
$$B_{out}(t) = \overline{A_0(t)}B_0(t) + \overline{A_0(t)}B_{in}(t) + B_0(t)B_{in}(t)$$

Upon the active clock edge, the registers shift right, latching $D(t)$ into the MSB of Register A. Simultaneously, the D-FF captures $B_{out}(t)$ to use in the subsequent cycle.

3. Logic Circuit Diagram

4. State Execution Trace (4-Bit Subtraction Cycle)

Let the initial state of Register A be $[a_3, a_2, a_1, a_0]$ and Register B be $[b_3, b_2, b_1, b_0]$. The D-FF is initialized to clear any previous borrow ($B_{in} = 0$). The process executes over 4 synchronous clock cycles:

Serial Subtraction Trace Table												
Clock	Reg A Contents				Reg B Contents				x (A_0)	y (B_0)	B_{in}	Computed Difference (D)
$t = 0$	a_3	a_2	a_1	a_0	b_3	b_2	b_1	b_0	a_0	b_0	0	$D_0 = a_0 \oplus b_0 \oplus 0$
$t = 1$	D_0	a_3	a_2	a_1	0	b_3	b_2	b_1	a_1	b_1	B_1	$D_1 = a_1 \oplus b_1 \oplus B_1$
$t = 2$	D_1	D_0	a_3	a_2	0	0	b_3	b_2	a_2	b_2	B_2	$D_2 = a_2 \oplus b_2 \oplus B_2$
$t = 3$	D_2	D_1	D_0	a_3	0	0	0	b_3	a_3	b_3	B_3	$D_3 = a_3 \oplus b_3 \oplus B_3$
$t = 4$	D_3	D_2	D_1	D_0	0	0	0	0	—	—	B_4	Subtraction Complete

At $t = 4$, the global shift clock is gated or disabled. Register A now holds the final 4-bit difference ($A - B$). Register B has been flushed to zero (assuming 0 is shifted in). The value B_4 stored in the D-FF represents the final borrow-out; if $B_4 = 1$, it indicates that $B > A$ and the resulting difference in Register A is in two's complement form.

Q6. Draw the logic diagram of a serial adder. Show the values of inputs and outputs of all components (e.g., flip-flop, register) used in the design after every clock pulse while adding the following two numbers: 0101 and 0011. (6+6 marks) [CSE 205 | L-2/T-1 | 2011–12, 2012–13]

Answer

1. Logic Diagram of a Serial Adder

The serial adder architecture is designed below using synchronous data pathways. It consists of two 4-bit right-shift registers (Register A and Register B), a 1-bit combinational Full Adder block, and a positive-edge-triggered D Flip-Flop to preserve the carry bit between successive clock periods.

2. Mathematical Derivations and Operation Cycle State Table

The circuit adds $A = 0101_2$ (5_{10}) and $B = 0011_2$ (3_{10}). Let the individual bits of the registers be represented as $[b_3b_2b_1b_0]$ and $[a_3a_2a_1a_0]$. At each clock cycle t , the full adder computes:

$$S = x \oplus y \oplus C_{in}$$
$$C_{out} = xy + xC_{in} + yC_{in}$$

The state values tracking inputs, internal registers, and combinational nodes at every sequential interval are listed comprehensively below:

Step-by-Step Execution and State Tracking Table

Clock Pulse	Register Contents		Full Adder Inputs			Full Adder Outputs		Next State $Q^+ = D$
	Reg. A	Reg. B	x (SO _A)	y (SO _B)	C _{in} (Q)	S	C _{out}	
Initial (t_0)	0101	0011	1	1	0	0	1	1
After Pulse 1	0010	0001	0	1	1	0	1	1
After Pulse 2	0001	0000	1	0	1	0	1	1
After Pulse 3	0000	0000	0	0	1	1	0	0
After Pulse 4	1000	0000	–	–	0	–	–	–

3. Microscopic Timing and Detailed Bit-Shift Analysis

- **Initial Setup (t_0):** LSBs are exposed to the FA: $x = a_0 = 1$, $y = b_0 = 1$. Flip-Flop is cleared ($C_{in} = 0$). Computations:

$$S_0 = 1 \oplus 1 \oplus 0 = 0$$
$$C_{out0} = (1 \cdot 1) + (1 \cdot 0) + (1 \cdot 0) = 1$$

- **Clock Pulse 1:** Sum bit $S_0 = 0$ is shifted into the MSB (a_3) of Reg A. Reg B shifts in 0. D-FF latches $C_{out0} = 1$. The new outputs available at the serial edge are $x = a_1 = 0$, $y = b_1 = 1$, and $C_{in} = 1$.

$$S_1 = 0 \oplus 1 \oplus 1 = 0$$
$$C_{out1} = (0 \cdot 1) + (0 \cdot 1) + (1 \cdot 1) = 1$$

- **Clock Pulse 2:** Sum bit $S_1 = 0$ is shifted into Reg A. D-FF retains a carry of 1. The next sequential bits are $x = a_2 = 1$, $y = b_2 = 0$, and $C_{in} = 1$.

$$S_2 = 1 \oplus 0 \oplus 1 = 0$$
$$C_{out2} = (1 \cdot 0) + (1 \cdot 1) + (0 \cdot 1) = 1$$

- **Clock Pulse 3:** Sum bit $S_2 = 0$ enters Reg A. D-FF continues holding 1. The final MSB operand inputs are presented: $x = a_3 = 0$, $y = b_3 = 0$, and $C_{in} = 1$.

$$S_3 = 0 \oplus 0 \oplus 1 = 1$$
$$C_{out3} = (0 \cdot 0) + (0 \cdot 1) + (0 \cdot 1) = 0$$

- **Clock Pulse 4:** The final sum bit $S_3 = 1$ is shifted into Register A. Register B is completely flushed. The system operations stop.

4. Verification of Result

At the termination of 4 clock pulses, the content of Register A is read in parallel as:

$$\text{Reg. A} = 1000_2 = 8_{10}$$

Evaluating the decimal equivalence of the mathematical query confirms flawless system execution:

$$0101_2 + 0011_2 \implies 5_{10} + 3_{10} = 8_{10} = 1000_2$$

BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Memory & Programmable Logic

RAM, ROM, Address Multiplexing, PLA Design

S8Memory & Programmable Logic

RAM & ROM Design & Expansion

Q1. If you use two dimensional decoding for addressing a 1K word memory, can you optimize the cost? Justify your answer. (8 marks)
[CSE 205 | L-2/T-1 | 2024–25]

Answer

Conclusion
Yes, using two-dimensional (2D) decoding significantly optimizes both the hardware cost and complexity compared to one-dimensional (1D) decoding for a 1K word memory.

1. Address Bus Calculation
A memory size of 1K words translates to 1024 distinct memory locations. Since $1024 = 2^{10}$, the system requires exactly **10 address lines** ($A_9 \dots A_0$) to uniquely identify each word.

2. One-Dimensional (Linear) Decoding Cost
In a standard 1D addressing scheme, a single monolithic decoder processes the entire address bus.

- Decoder Type:** One 10×1024 decoder.
- Gate Count:** It requires $2^{10} = \mathbf{1024}$ AND gates.
- Gate Complexity:** Each AND gate must accommodate **10 inputs** (fan-in of 10), which is highly impractical and expensive at the transistor level.

3. Two-Dimensional (Coincident) Decoding Cost
In a 2D addressing scheme, the memory is arranged as a square (or near-square) grid. The 10-bit address is split into two halves: Row and Column addresses. For a 10-bit address, we split it symmetrically:

Row bits (X) = 5 bits $\implies 2^5 = 32$ rows

Column bits (Y) = 5 bits $\implies 2^5 = 32$ columns

This requires two smaller, separate decoders:

- Row Decoder:** One 5×32 decoder (requires 32 AND gates, each with 5 inputs).
- Column Decoder:** One 5×32 decoder (requires 32 AND gates, each with 5 inputs).
- Total Gate Count:** $32 + 32 = \mathbf{64}$ AND gates.

4. Justification & Hardware Cost Comparison
By reorganizing the memory geometrically into a 32×32 matrix, the coincident selection (where a memory cell is accessed only when both its corresponding row and column lines are asserted) drastically trims the peripheral decoding logic.

Parameter	1D Decoding	2D Decoding
Memory Matrix Lay-out	1024×1	32×32
Number of Decoders	1	2
Decoder Sizes	One 10×1024	Two 5×32
Total AND Gates Re-quired	1024	64
Fan-in (Inputs per gate)	10	5

Final Verdict:
The 2D decoding scheme reduces the number of required decoding AND gates from **1024** to just **64** (a $\sim 93.75\%$ reduction). Additionally, it halves the fan-in requirement from 10 inputs to 5 inputs per gate. This reduction in both gate count and gate complexity results in a significantly smaller silicon footprint, lower power consumption, and vastly optimized cost.

Q2. You have several 32×4 ROMs in the laboratory. Each ROM can be enabled/disabled using a 1/0 signal respectively. How would you implement a 128×4 ROM using the available ROMs? You may use any additional circuitry if needed. (10 marks) [CSE 205 | L-2/T-1 | 2013–14]

Answer

To design a 128×4 ROM using multiple 32×4 ROMs, we must perform a step-by-step capacity expansion. We will expand the memory capacity (number of words) while keeping the word size (4 bits) constant.

Step 1: Determine the Number of Required ROM Chips

- **Target Capacity:** 128×4 (128 words, 4 bits per word)
- **Available Component Capacity:** 32×4 (32 words, 4 bits per word)

The total number of 32×4 ROMs required is calculated as:

$$\begin{aligned} \text{Number of ROMs} &= \frac{\text{Target Number of Words}}{\text{Component Number of Words}} \\ &= \frac{128}{32} = 4 \text{ ROM chips} \end{aligned}$$

Step 2: Determine the Address and Data Lines

The number of address lines is determined by the $\log_2$ of the number of words.

- **Target ROM (128×4):** Requires $\log_2(128) = 7$ address lines. Let these be $A_6, A_5, A_4, A_3, A_2, A_1, A_0$.
- **Available ROM (32×4):** Requires $\log_2(32) = 5$ address lines. Let these be A_4, A_3, A_2, A_1, A_0 .
- **Data Lines:** Both target and available ROMs have 4 data lines (D_3, D_2, D_1, D_0). No expansion is needed for the word size.

Step 3: Decoding Logic

We have 7 address lines for the entire memory, but each component ROM only accepts 5 address lines.

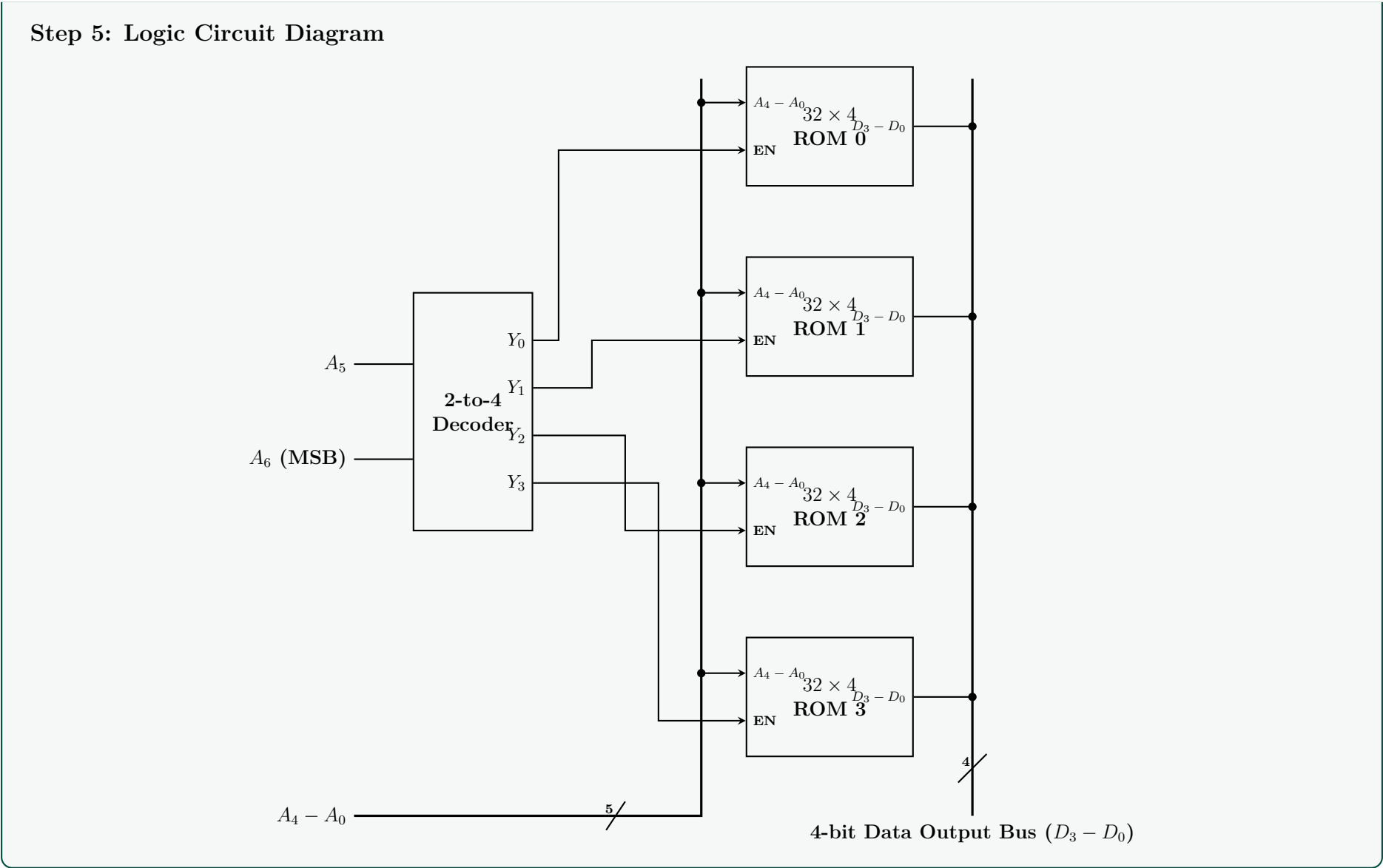
- The lower 5 bits ($A_4 \dots A_0$) act as the **common address bus** and are connected directly to all four ROMs in parallel.
- The upper 2 bits (A_6, A_5) are used to select one of the four ROMs. We use a **2-to-4 Line Decoder** for this chip selection. The decoder outputs Y_0, Y_1, Y_2, Y_3 will connect to the Enable (EN) pins of ROM 0, ROM 1, ROM 2, and ROM 3, respectively.

Note: We assume the ROMs have tri-state outputs that go to a high-impedance state when $EN = 0$. This allows the data lines of all four ROMs to be tied together to form a common 4-bit data bus.

Step 4: Memory Map

The table below illustrates the address ranges mapped to each specific ROM chip based on the decoder inputs (A_6, A_5).

Decoder Inputs		Common Address Lines					Active Chip	Decimal Address Range
A_6	A_5	A_4	A_3	A_2	A_1	A_0		
0	0	0...1	0...1	0...1	0...1	0...1	ROM 0 ($Y_0 = 1$)	0 to 31
0	1	0...1	0...1	0...1	0...1	0...1	ROM 1 ($Y_1 = 1$)	32 to 63
1	0	0...1	0...1	0...1	0...1	0...1	ROM 2 ($Y_2 = 1$)	64 to 95
1	1	0...1	0...1	0...1	0...1	0...1	ROM 3 ($Y_3 = 1$)	96 to 127



PLA Design

Q1. Design a PLA to implement the following functions with the minimization of the number of product terms:

$$F_1(x,y,z) = \Sigma (1,3,5,6),$$
$$F_3(x,y,z) = \Sigma (3,5),$$

$$F_2(x,y,z) = \Sigma (0,1,6,7)$$
$$F_4(x,y,z) = \Sigma (1,2,4,5,7)$$

(15 marks)

[CSE 205 | L-2/T-1 | 2024–25]

Answer

To design an optimized Programmable Logic Array (PLA) for the given functions, we must minimize the **total number of unique product terms** shared across all outputs, rather than just minimizing each function independently.

Step 1: Analyze the Minterms

Let's express the given functions in terms of their required minterms (for 3 variables: x, y, z):

$$F_1(x,y,z) = \sum m(1,3,5,6)$$
$$F_2(x,y,z) = \sum m(0,1,6,7)$$
$$F_3(x,y,z) = \sum m(3,5)$$
$$F_4(x,y,z) = \sum m(1,2,4,5,7)$$

If we observe the union of all required minterms across all four functions, all 8 possible minterms (m_0 to m_7) are utilized at least once.

Step 2: Multi-Output Minimization

If we minimize each function *independently* using standard K-maps, we obtain:

- $F_1 = \bar{y}z + \bar{x}z + xy\bar{z}$ (3 terms)
- $F_2 = \bar{x}\bar{y} + xy$ (2 terms)
- $F_3 = \bar{x}yz + x\bar{y}z$ (2 terms)
- $F_4 = x\bar{y} + xz + \bar{y}z + \bar{x}y\bar{z}$ (4 terms)

This independent approach results in **10 unique product terms** across the functions. However, a PLA's size is dictated by the number of AND gates (rows). We can reduce the total row count to **8 unique product terms** by strategically combining minterms for specific functions while leaving others as explicit minterms to maximize sharing without creating redundant overlapping terms. We define the following 8 shared product terms (P_1 to P_8):

$P_1 = \bar{x}\bar{y}$	(Covers m_0, m_1 for F_2)
$P_2 = \bar{x}\bar{y}z$	(Covers m_1 for F_1, F_4)
$P_3 = \bar{x}y\bar{z}$	(Covers m_2 for F_4)
$P_4 = \bar{x}yz$	(Covers m_3 for F_1, F_3)
$P_5 = x\bar{y}$	(Covers m_4, m_5 for F_4)
$P_6 = x\bar{y}z$	(Covers m_5 for F_1, F_3)
$P_7 = xy\bar{z}$	(Covers m_6 for F_1, F_2)
$P_8 = xyz$	(Covers m_7 for F_2, F_4)

Reconstructing the functions using only these 8 terms:

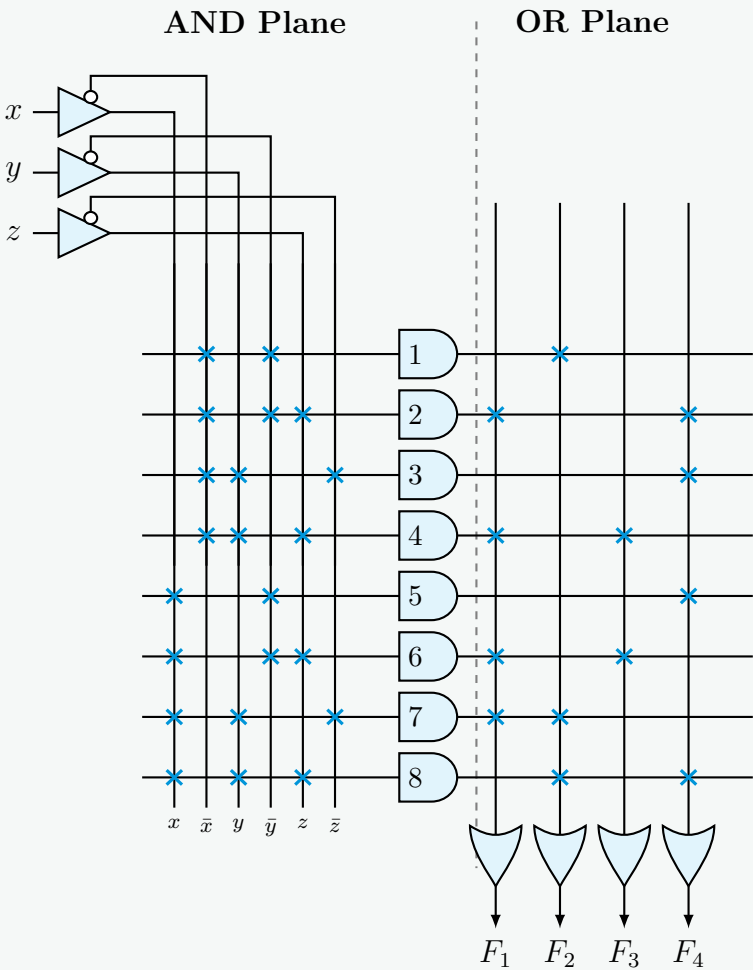
$F_1 = P_2 + P_4 + P_6 + P_7$	(Covers: 1, 3, 5, 6)
$F_2 = P_1 + P_7 + P_8$	(Covers: 0, 1, 6, 7)
$F_3 = P_4 + P_6$	(Covers: 3, 5)
$F_4 = P_2 + P_3 + P_5 + P_8$	(Covers: 1, 2, 4, 5, 7)

Step 3: PLA Programming Table

The programming table defines the connections in the AND plane and OR plane.

Product Term	Inputs			Outputs			
	x	y	z	F_1	F_2	F_3	F_4
$P_1 = \bar{x}\bar{y}$	0	0	-	0	1	0	0
$P_2 = \bar{x}\bar{y}z$	0	0	1	1	0	0	1
$P_3 = \bar{x}y\bar{z}$	0	1	0	0	0	0	1
$P_4 = \bar{x}yz$	0	1	1	1	0	1	0
$P_5 = x\bar{y}$	1	0	-	0	0	0	1
$P_6 = x\bar{y}z$	1	0	1	1	0	1	0
$P_7 = xy\bar{z}$	1	1	0	1	1	0	0
$P_8 = xyz$	1	1	1	0	1	0	1

Step 4: PLA Circuit Diagram



Q2. Tabulate the PLA programming table for the two Boolean functions listed below. Minimize the numbers of product terms:

$$F_1(A, B, C) = \Sigma (0, 1, 2, 4)$$
$$F_2(A, B, C) = \Sigma (0, 5, 6, 7)$$

(15 marks)

[CSE 205 | L-2/T-1 | 2022–23]

Answer

To minimize the number of product terms in a Programmable Logic Array (PLA), we must evaluate both the **true** and **complement** forms of each function. PLAs have programmable output links (often modeled as XOR gates with a programmable connection to ground or V_{cc}) that allow us to output either the function F or its complement F' . The optimal solution pairs the forms that maximize shared product terms.

Step 1: Determine the True and Complement Minterms

Given the functions F_1 and F_2 in 3 variables (A, B, C):

$$F_1(A, B, C) = \sum m(0, 1, 2, 4)$$
$$F_2(A, B, C) = \sum m(0, 5, 6, 7)$$

The complements of these functions contain the remaining minterms from the 3-variable space (m_0 to m_7):

$$F'_1(A, B, C) = \sum m(3, 5, 6, 7)$$
$$F'_2(A, B, C) = \sum m(1, 2, 3, 4)$$

Step 2: Simplify All Function Forms

We minimize F_1, F'_1, F_2 , and F'_2 independently using Boolean algebra (or Karnaugh maps) to find their prime implicants:

- F_1 (**True**): $\sum m(0, 1, 2, 4) \implies \bar{A}\bar{B} + \bar{A}\bar{C} + \bar{B}\bar{C}$ (3 terms)
- F'_1 (**Complement**): $\sum m(3, 5, 6, 7) \implies AB + AC + BC$ (3 terms)
- F_2 (**True**): $\sum m(0, 5, 6, 7) \implies \bar{A}\bar{B}\bar{C} + AB + AC$ (3 terms)
- F'_2 (**Complement**): $\sum m(1, 2, 3, 4) \implies \bar{A}B + \bar{A}C + A\bar{B}\bar{C}$ (3 terms)

Step 3: Select the Optimal Combination

We must choose one form for F_1 (either True or Complement) and one form for F_2 (either True or Complement) that results in the minimum number of unique product terms across both functions.

Comparing the minimized equations, we observe that F'_1 (**Complement**) and F_2 (**True**) share two identical product terms: AB and AC .

Selected $F'_1 = AB + AC + BC$

Selected $F_2 = AB + AC + \bar{A}\bar{B}\bar{C}$

By programming F_1 in its complement form and F_2 in its true form, the total number of unique product terms required is reduced to just **four**:

(a) $P_1 = AB$ (Shared by F'_1 and F_2)

(b) $P_2 = AC$ (Shared by F'_1 and F_2)

(c) $P_3 = BC$ (Used only by F'_1)

(d) $P_4 = \bar{A}\bar{B}\bar{C}$ (Used only by F_2)

Step 4: PLA Programming Table

The programming table defines the states of the inputs (1 for True, 0 for Complemented, - for Don't Care) and the connections in the OR plane (1 if the product term is connected to the output, 0 otherwise). The output polarity is marked as **C** for Complement and **T** for True.

Product Term	Inputs			Outputs	
	A	B	C	F_1 (C)	F_2 (T)
$P_1 = AB$	1	1	-	1	1
$P_2 = AC$	1	-	1	1	1
$P_3 = BC$	-	1	1	1	0
$P_4 = \bar{A}\bar{B}\bar{C}$	0	0	0	0	1

Q3. A combinational circuit is defined by the following two functions:

$$F_1(A, B, C) = \Sigma(3, 5, 6, 7)$$
$$F_2(A, B, C) = \Sigma(0, 2, 4, 7)$$

The circuit needs to be implemented with a PLA having three inputs, four product terms, and two outputs. Show the PLA programming table. (15 marks)

[CSE 205 | L-2/T-1 | 2021–22]

Answer

1. Problem Analysis

The problem requires implementing two combinational Boolean functions, F_1 and F_2 , using a Programmable Logic Array (PLA) with a strict constraint of exactly **four product terms**. A PLA allows product terms to be shared between multiple outputs. To fit within the four-term limit, we must evaluate both the true (F) and complemented (F') forms of each function to find a combination that yields ≤ 4 unique product terms.

2. Simplification of Functions

We first determine the minimal sum-of-products (SOP) expressions for F_1, F_1', F_2 , and F_2' using Karnaugh maps and Boolean simplification.

For Function F_1 :

Given $F_1(A, B, C) = \Sigma(3, 5, 6, 7)$, its complement is $F_1'(A, B, C) = \Sigma(0, 1, 2, 4)$.

$F_1 = AB + AC + BC$

(3 product terms)

$F_1' = A'B' + A'C' + B'C'$

(3 product terms)

For Function F_2 :

Given $F_2(A, B, C) = \Sigma(0, 2, 4, 7)$, its complement is $F_2'(A, B, C) = \Sigma(1, 3, 5, 6)$.

$F_2 = A'C' + B'C' + ABC$

(3 product terms)

$F_2' = A'C + B'C + ABC'$

(3 product terms)

3. Optimal Selection of Forms

We evaluate the total number of unique product terms for all four possible combinations:

F_1 and F_2 : $\{AB, AC, BC, A'C', B'C', ABC\} \implies 6$ unique terms

F_1' and F_2' : $\{A'B', A'C', B'C', A'C, B'C, ABC'\} \implies 6$ unique terms

F_1 and F_2' : $\{AB, AC, BC, A'C, B'C, ABC'\} \implies 6$ unique terms

F_1' and F_2 : $\{A'B', A'C', B'C', ABC\} \implies 4$ unique terms (Optimal)

By programming the PLA for F_1' and F_2 , we perfectly match the 4 product term limit constraint. The output for F_1 will be configured to invert the sum (Complement), while F_2 will remain non-inverted (True).

Karnaugh Maps for Selected Forms:

Map for $F_1' = \Sigma(0, 1, 2, 4)$

BC

00011110

A01101010

A11000000

Map for $F_2 = \Sigma(0, 2, 4, 7)$

BC

00011110

A01101010

A11010100

4. PLA Programming Table

The programming table defines the connections in the PLA matrices:

Inputs: 1 (True logic), 0 (Complemented logic), – (No connection)

Outputs: 1 (Connected to OR gate), – (No connection)

T/C: T (True output, XORed with 0), C (Complemented output, XORed with 1)

Product	Inputs			Outputs	
Term	A	B	C	F ₁	F ₂
1. $A'B'$	0	0	–	1	–
2. $A'C'$	0	–	0	1	1
3. $B'C'$	–	0	0	1	1
4. ABC	1	1	1	–	1
Output Form (T/C):				C	T

403

5. PLA Circuit Diagram

Q4. Design a digital security lock using PLA which takes BCD as input and opens after converting the input to Excess-3 code. Use the minimum number of product terms. (Show the fuse map.) (15 marks) [CSE 205 | L-2/T-1 | 2020–21]

Answer

1. Problem Analysis & Truth Table

The objective is to design a combinational circuit using a Programmable Logic Array (PLA) that converts a BCD input (A, B, C, D) to an Excess-3 code output (W, X, Y, Z). A PLA allows sharing of product terms among multiple outputs. To achieve the **minimum number of product terms**, we must analyze both the true and complemented forms of the output functions to maximize the overlap of prime implicants.

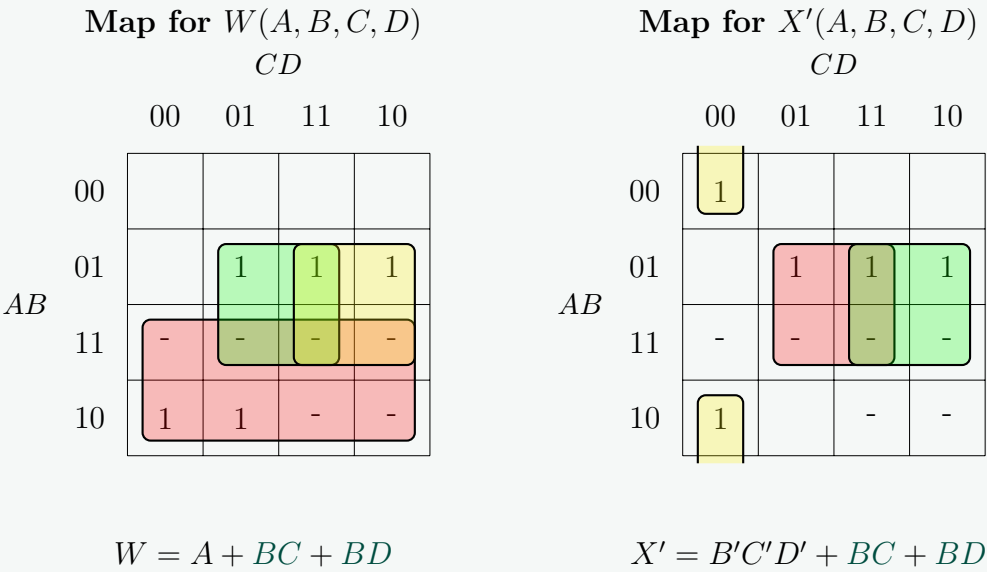
The truth table for BCD to Excess-3 conversion is as follows. Note that BCD inputs from 10 to 15 are invalid and treated as Don't Cares (d).

BCD Input				Excess-3 Output			
A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

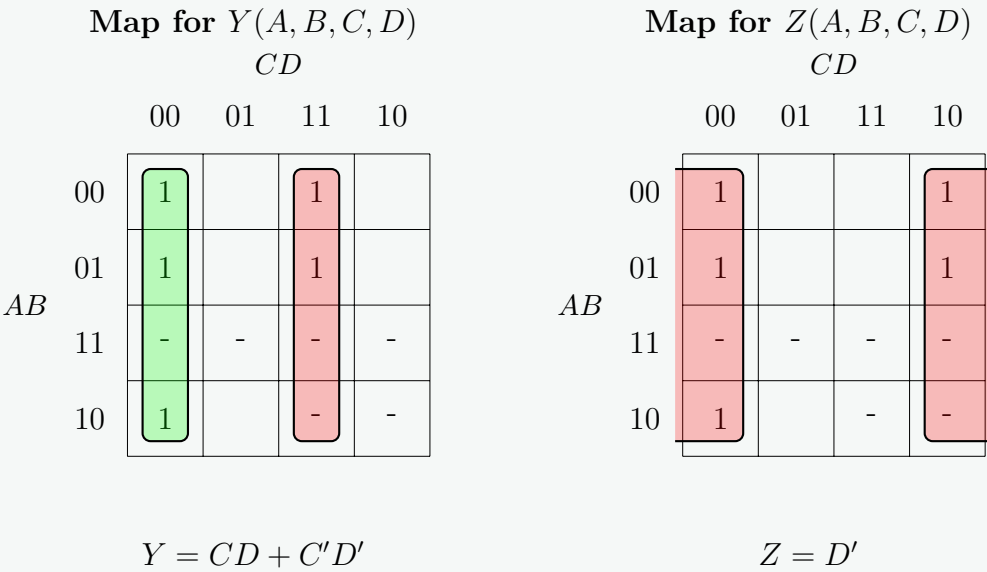
Inputs 10 to 15 are Don't Cares (d)

2. Karnaugh Map Simplification & Term Optimization

We derive the simplified Boolean expressions using K-maps. We evaluate both the true (F) and complemented (F') forms to find shared product terms.



Notice that evaluating X' (the complement of X) yields the product terms BC and BD , which perfectly overlap with the product terms required for W . If we evaluated X directly, we would get $X = B'C + B'D + BC'D'$, which shares no terms with W .



3. Optimized Boolean Expressions

By programming the PLA to output the true form for W, Y, Z and the complemented form for X , we reduce the entire circuit to exactly **7 unique product terms**:

$W = A + BC + BD$	(True logic, 3 terms)
$X' = BC + BD + B'C'D' \implies X = (BC + BD + B'C'D')'$	(Complement logic, shares BC, BD)
$Y = CD + C'D'$	(True logic, 2 terms)
$Z = D'$	(True logic, 1 term)

The 7 unique product terms required are: $A, BC, BD, B'C'D', CD, C'D'$, and D' .

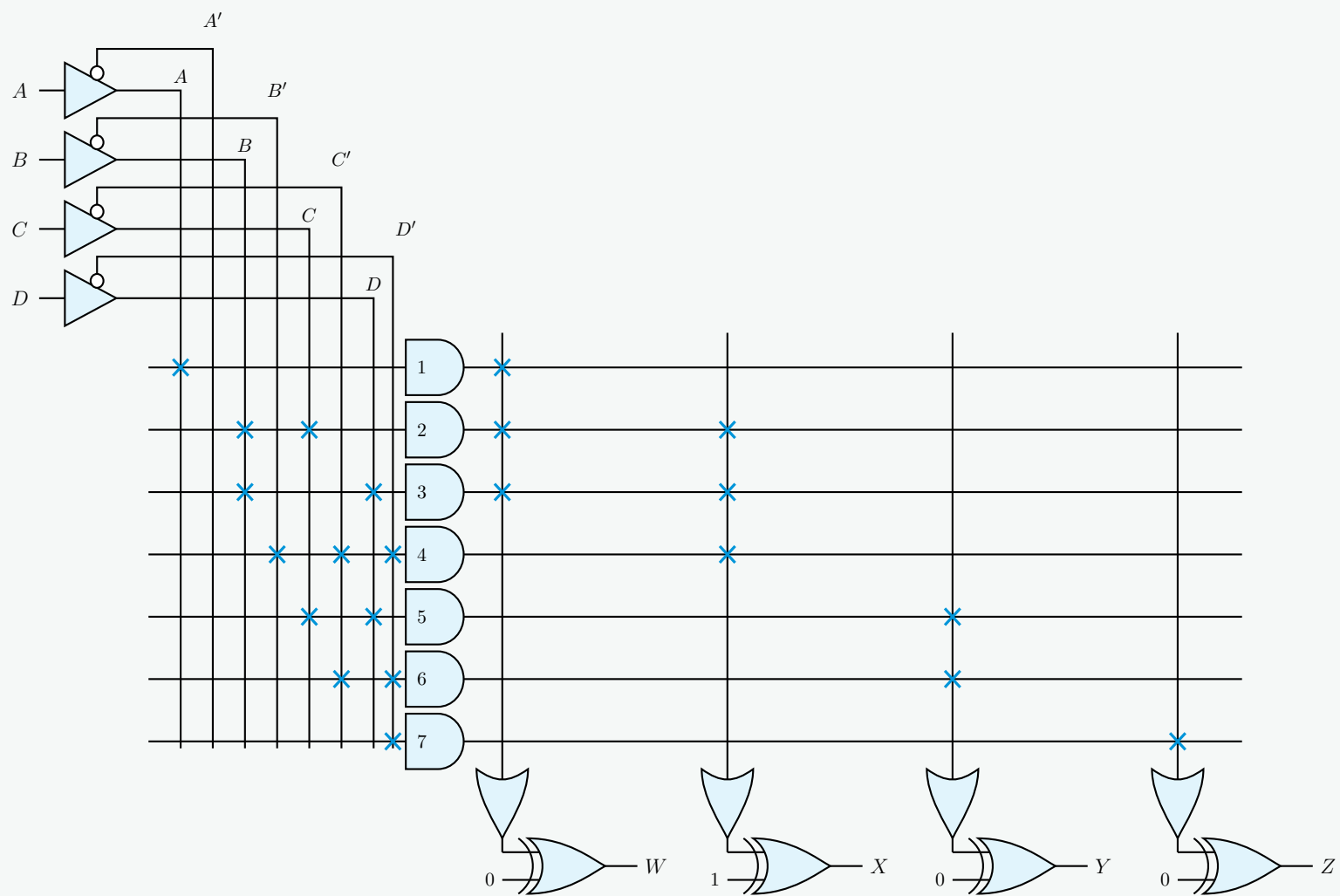
4. PLA Programming Table (Fuse Map)

The following table provides the PLA programming connections.

- **Inputs:** ‘1’ (connected to uncomplemented input), ‘0’ (connected to complemented input), ‘–’ (no connection / blown fuse).
- **Outputs:** ‘1’ (product term connected to OR gate), ‘–’ (no connection).
- **T/C Flag:** ‘T’ specifies the true output (XOR with 0), ‘C’ specifies the complemented output (XOR with 1).

Product Term	Inputs				Outputs			
	A	B	C	D	W	X	Y	Z
1. A	1	–	–	–	1	–	–	–
2. BC	–	1	1	–	1	1	–	–
3. BD	–	1	–	1	1	1	–	–
4. $B'C'D'$	–	0	0	0	–	1	–	–
5. CD	–	–	1	1	–	–	1	–
6. $C'D'$	–	–	0	0	–	–	1	–
7. D'	–	–	–	0	–	–	–	1
Output Form (T/C):					T	C	T	T

5. PLA Circuit Diagram



Q5. Design a circuit for the following functions using a Programmable Logic Array (PLA):

$$A(x, y, z) = \Sigma(0, 1, 2, 5, 7)$$

$$B(x, y, z) = \Sigma(0, 1, 3, 6, 7)$$

(10 marks)

[CSE 205 | L-2/T-1 | 2019–20]

Answer

1. Problem Analysis

We are given two 3-variable Boolean functions:

$$A(x, y, z) = \Sigma(0, 1, 2, 5, 7)$$

$$B(x, y, z) = \Sigma(0, 1, 3, 6, 7)$$

To implement these functions efficiently using a Programmable Logic Array (PLA), we aim to minimize the total number of unique product terms (AND gates). A PLA allows product terms to be shared between multiple outputs, and each output can be programmed to produce either the true function or its complement via an output XOR gate. Therefore, we must evaluate both the true and complemented forms of A and B to find the optimal combination.

2. Simplification of True and Complemented Functions

Case 1: True Forms (A and B)

- For $A(x, y, z) = \Sigma(0, 1, 2, 5, 7)$:

$$\begin{aligned} A &= x'y'z' + x'y'z + x'yz' + xy'z + xyz \\ &= x'y'(z' + z) + x'yz' + xz(y' + y) && \text{(Distributive Law)} \\ &= x'y'(1) + x'yz' + xz(1) && \text{(Complement Law)} \\ &= x'y' + x'yz' + xz && \text{(Identity Law)} \\ &= x'((y' + y)(y' + z')) + xz && \text{(Distributive Law)} \\ &= x'y' + x'z' + xz && \text{(Simplification)} \end{aligned}$$

- For $B(x, y, z) = \Sigma(0, 1, 3, 6, 7)$:

$$\begin{aligned} B &= x'y'z' + x'y'z + x'yz + xyz' + xyz \\ &= x'y'(z' + z) + x'yz + xy(z' + z) && \text{(Distributive Law)} \\ &= x'y'(1) + x'yz + xy(1) && \text{(Complement Law)} \\ &= x'y' + x'z + xy && \text{(Identity Law/Simplification)} \end{aligned}$$

Implementing the true forms A and B requires the following unique product terms: $\{x'y', x'z', xz, x'z, xy\}$. This configuration requires **5 unique product terms**.

Case 2: Complemented Forms (A' and B') The maxterms of the original functions define their complements:

- For $A'(x, y, z) = \Sigma(3, 4, 6)$:

$$\begin{aligned} A' &= x'yz + xy'z' + xyz' \\ &= x'yz + xz'(y' + y) && \text{(Distributive Law)} \\ &= x'yz + xz'(1) && \text{(Complement Law)} \\ &= x'yz + xz' && \text{(Identity Law)} \end{aligned}$$

- For $B'(x, y, z) = \Sigma(2, 4, 5)$:

$$\begin{aligned} B' &= x'yz' + xy'z' + xy'z \\ &= x'yz' + xy'(z' + z) && \text{(Distributive Law)} \\ &= x'yz' + xy'(1) && \text{(Complement Law)} \\ &= x'yz' + xy' && \text{(Identity Law)} \end{aligned}$$

Implementing the complemented forms A' and B' requires the following unique product terms: $\{x'yz, xz', x'yz', xy'\}$. This configuration requires **4 unique product terms**.

3. Optimal Implementation Selection

Comparing all combinations, implementing the complemented outputs **A'** and **B'** minimizes the resources down to exactly **4 product terms** (with no shared terms needed, yet yielding a lower total count than any shared true-form arrangement). The PLA will invert both outputs at the final stage using the XOR array (configured to 'C').

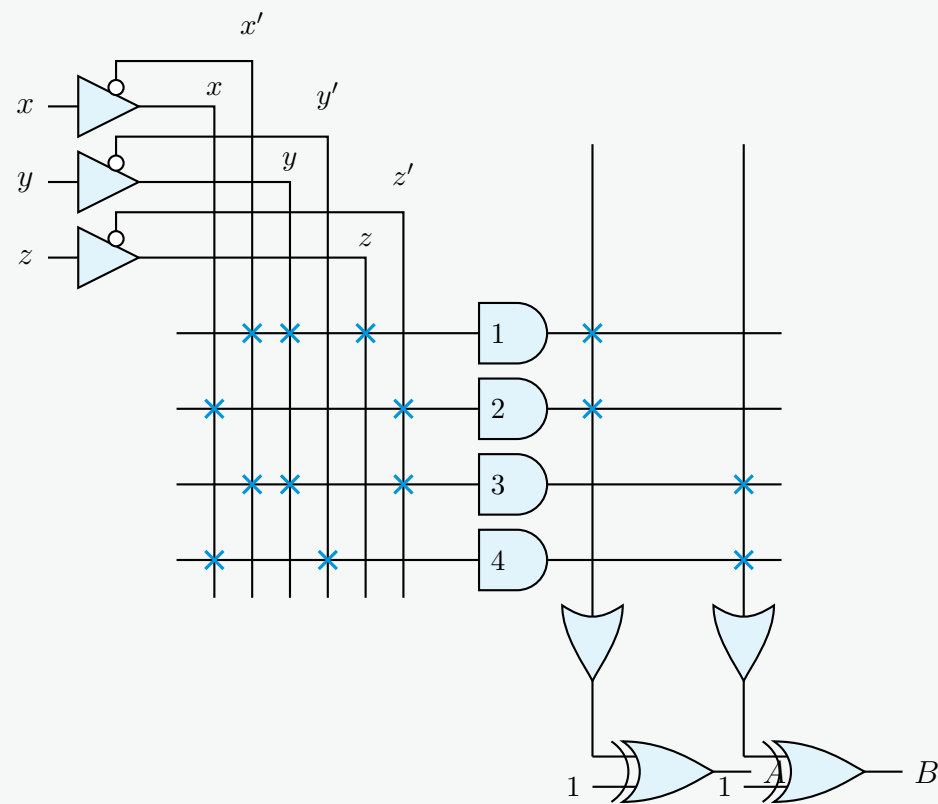
4. PLA Programming Table

The table below establishes the internal routing connections for the PLA design:

- Inputs:** '1' (True line connection), '0' (Complement line connection), '–' (No connection / Blown fuse).
- Outputs:** '1' (Connected to the OR matrix), '–' (No connection / Blown fuse).
- Output Form (T/C):** 'T' (True / No inversion), 'C' (Complement / Inversion via XOR).

Product Term	Inputs			Outputs	
	x	y	z	A	B
1. $x'yz$	0	1	1	1	–
2. xz'	1	–	0	1	–
3. $x'yz'$	0	1	0	–	1
4. xy'	1	0	–	–	1
Output Form (T/C):				C	C

5. PLA Circuit Diagram



Q6. Design a security lock using PLA which will be opened using the following combinations:

$$A(x, y, z) = \Sigma(1, 3, 5, 6), \quad B(x, y, z) = \Sigma(0, 1, 6, 7)$$

$$C(x, y, z) = \Sigma(3, 5), \quad D(x, y, z) = \Sigma(1, 3, 4, 5, 7)$$

(12 marks)

[CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Problem Analysis & Optimization Strategy

We are tasked with designing a PLA to implement four Boolean functions:

$$A(x, y, z) = \Sigma(1, 3, 5, 6)$$

$$B(x, y, z) = \Sigma(0, 1, 6, 7)$$

$$C(x, y, z) = \Sigma(3, 5)$$

$$D(x, y, z) = \Sigma(1, 3, 4, 5, 7)$$

To minimize the PLA size, we must find the combination of True (F) and Complemented (F') outputs that yields the **minimum number of unique product terms**. We evaluate the maxterms to define the complement of each function:

$$A'(x, y, z) = \Sigma(0, 2, 4, 7)$$

$$B'(x, y, z) = \Sigma(2, 3, 4, 5)$$

$$C'(x, y, z) = \Sigma(0, 1, 2, 4, 6, 7)$$

$$D'(x, y, z) = \Sigma(0, 2, 6)$$

By analyzing common minterms and groupings, we observe massive structural overlap if we implement **A', B, C', and D'**. We can share prime implicants across these functions to dramatically reduce the total term count.

2. Karnaugh Map Simplifications

We map out A' , B , C' , and D' to extract the optimal shared product terms.

Map for $A'(x, y, z) = \Sigma(0, 2, 4, 7)$

		yz			
		00	01	11	10
x	0	1			1
	1	1		1	

Map for $B(x, y, z) = \Sigma(0, 1, 6, 7)$

		yz			
		00	01	11	10
x	0	1	1		
	1			1	1

Map for $C'(x, y, z) = \Sigma(0, 1, 2, 4, 6, 7)$

		yz			
		00	01	11	10
x	0	1	1		1
	1	1		1	1

Map for $D'(x, y, z) = \Sigma(0, 2, 6)$

		yz			
		00	01	11	10
x	0	1			1
	1				1

3. Optimized Boolean Expressions

By leveraging the K-map groupings, we can formulate all four target configurations using exactly **6 unique product terms**:

$A' = x'z' + y'z' + xyz$	(Complement logic, requires 3 terms)
$B = x'y' + xy$	(True logic, requires 2 terms)
$C' = x'y' + x'z' + y'z' + xy$	(Complement logic, shares ALL 4 terms with A' and B)
$D' = x'z' + yz'$	(Complement logic, shares 1 term with A' , 1 unique term)

The minimal set of 6 product terms is: $\{x'z', y'z', xyz, yz', x'y', xy\}$.

To restore the correct logic for outputs A , C , and D , the PLA's internal XOR array will be programmed to complement those specific outputs (T/C flag set to C).

4. PLA Programming Table (Fuse Map)

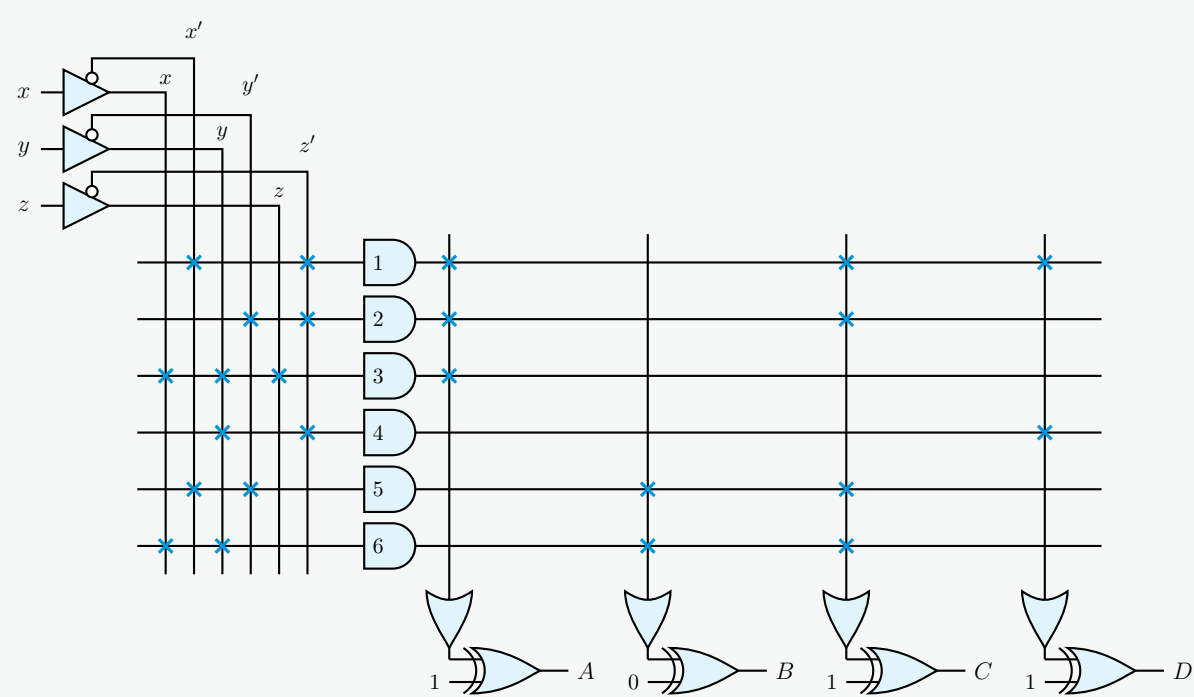
The internal connections are documented below:

- Inputs (AND array):** '1' = True connection, '0' = Complement connection, '-' = No connection.
- Outputs (OR array):** '1' = Connected to sum, '-' = No connection.

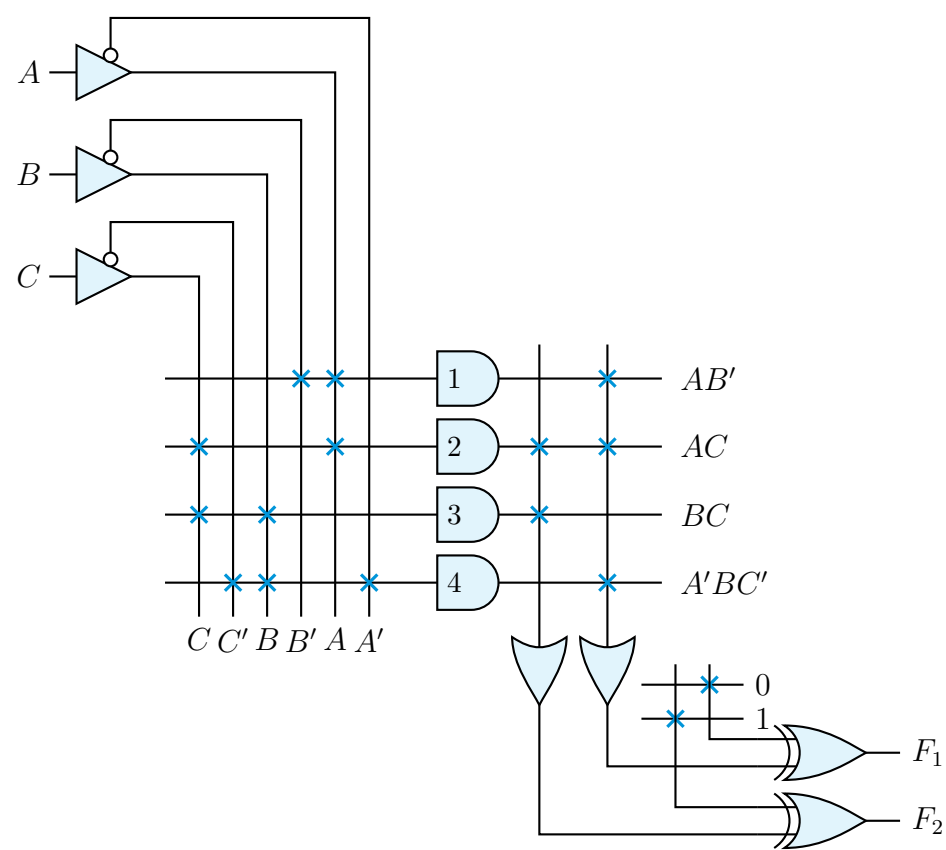
- **T/C Flag (XOR array):** ‘T’ = True (XOR with 0), ‘C’ = Complemented (XOR with 1).

Product Term	Inputs			Outputs			
	x	y	z	A	B	C	D
1. $x'z'$	0	–	0	1	–	1	1
2. $y'z'$	–	0	0	1	–	1	–
3. xyz	1	1	1	1	–	–	–
4. yz'	–	1	0	–	–	–	1
5. $x'y'$	0	0	–	–	1	1	–
6. xy	1	1	–	–	1	1	–
Output Form (T/C):				C	T	C	C

5. PLA Circuit Diagram



Q7. For a given Programmable Logic Array (PLA), find the function expressions for F_1 and F_2 and show the PLA programming table.



(13 marks)

[CSE 205 | L-2/T-1 | 2015–16]

Answer

1. Derivation of Boolean Expressions

From the given Programmable Logic Array (PLA) diagram, we first determine the logic equations for the intermediate product terms generated by the AND array. A connection (denoted by a cross on the diagram) on a vertical input wire includes that specific

literal in the product term.

- **AND gate 1 (Top, $y = 4$):** Connected to the true line of A and the complemented line of B .

$$P_1 = AB'$$

- **AND gate 2 ($y = 3$):** Connected to the true line of A and the true line of C .

$$P_2 = AC$$

- **AND gate 3 ($y = 2$):** Connected to the true line of B and the true line of C .

$$P_3 = BC$$

- **AND gate 4 (Bottom, $y = 1$):** Connected to the complemented line of A , the true line of B , and the complemented line of C .

$$P_4 = A'BC'$$

Next, we evaluate the connections in the OR array, which calculates the logical sum of the selected product terms.

- The right OR gate feeds into the logic path for F_1 . It has connections to product terms P_1 , P_2 , and P_4 .

$$S_1 = P_1 + P_2 + P_4 = AB' + AC + A'BC'$$

- The left OR gate feeds into the logic path for F_2 . It has connections to product terms P_2 and P_3 .

$$S_2 = P_2 + P_3 = AC + BC$$

Finally, we determine the output polarity based on the XOR gates. Recall the Boolean identities for the XOR operation: $X \oplus 0 = X$ (True polarity) and $X \oplus 1 = X'$ (Complemented polarity).

- The XOR gate for F_1 has its second input connected to a constant logic 0.
- The XOR gate for F_2 has its second input connected to a constant logic 1.

Applying these properties, we obtain the final Boolean expressions for the outputs:

$$F_1 = S_1 \oplus 0 = AB' + AC + A'BC'$$

$$F_2 = S_2 \oplus 1 = (AC + BC)'$$

2. PLA Programming Table

The programming table summarizes the hardware connections mathematically.

- **Inputs Column:** '1' specifies a connection to the true variable, '0' specifies a connection to the complemented variable, and '-' indicates no connection (don't care).
- **Outputs Column:** '1' indicates a connection to the corresponding OR gate, and '-' indicates no connection.
- **T/C (Polarity):** Specifies whether the XOR gate leaves the OR output True (T, XOR with 0) or Complements it (C, XOR with 1).

Product	Term	Inputs			Outputs	
Term	No.	A	B	C	F_1	F_2
AB'	1	1	0	-	1	-
AC	2	1	-	1	1	1
BC	3	-	1	1	-	1
$A'BC'$	4	0	1	0	1	-
Output Polarity (T/C)					T	C
XOR Connection					0	1

Q8. Write the difference between ROM and PLA. A combinational circuit is defined by the following functions:

$$F_1(A, B, C) = \Sigma(0, 1, 2, 4)$$

$$F_2(A, B, C) = \Sigma(0, 5, 6, 7)$$

Implement the circuit with a PLA having three inputs, four product terms and two outputs. **(5+10 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Difference Between ROM and PLA

Both Read-Only Memory (ROM) and Programmable Logic Arrays (PLA) are sum-of-products programmable logic devices, but they differ significantly in their internal architecture and efficiency.

Characteristic	ROM (Read-Only Memory)	PLA (Programmable Logic Array)
Internal Structure	Fixed AND array (decoder), Programmable OR array.	Programmable AND array, Programmable OR array.
Logic Realization	Generates all 2^n possible minterms (canonical sum of products).	Generates only the specifically required product terms (minimized SOP).
Complexity & Cost	Size doubles with each added input; inexpensive for general storage.	Compact size limited by product terms; more expensive/complex to fabricate.
Programming	Does not require Boolean minimization.	Requires extensive Boolean minimization to fit the limited product terms.
Flexibility	Lower flexibility as the AND array cannot be modified.	Highest flexibility as both AND and OR planes can be customized.

2. Logic Minimization for PLA Implementation

We are given two combinational functions with three inputs (A, B, C):
$$F_1(A, B, C) = \Sigma(0, 1, 2, 4)$$
$$F_2(A, B, C) = \Sigma(0, 5, 6, 7)$$
The given PLA constraint is **3 inputs, 4 product terms, and 2 outputs**. We must explore the true and complemented forms of F_1 and F_2 to find a set of shared terms that results in ≤ 4 distinct product terms. The complemented functions are:
$$F'_1(A, B, C) = \Sigma(3, 5, 6, 7)$$
$$F'_2(A, B, C) = \Sigma(1, 2, 3, 4)$$
Let us use Karnaugh Maps to minimize F'_1 and F_2 to check if they share prime implicants:

Map for $F'_1(A, B, C) = \Sigma(3, 5, 6, 7)$

Map for $F_2(A, B, C) = \Sigma(0, 5, 6, 7)$

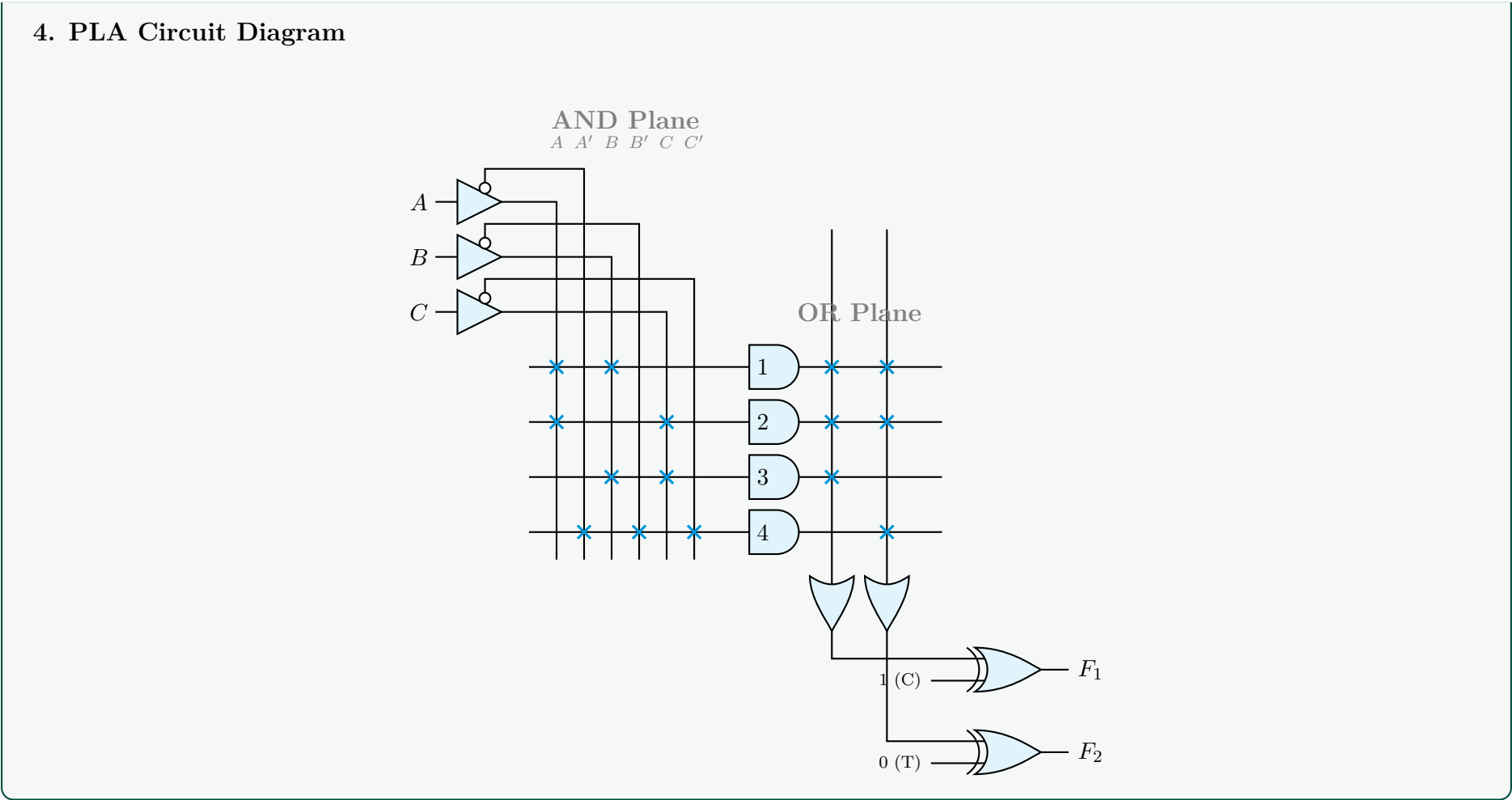
Extracting the minimized expressions from the maps:
$$F'_1 = AB + AC + BC \quad (3 \text{ terms})$$
$$F_2 = AB + AC + A'B'C' \quad (3 \text{ terms})$$
By selecting the output combination of F'_1 and F_2 , the product terms AB and AC are shared between the two functions. The overall distinct product terms required are exactly **four**:
$$P_1 = AB, \quad P_2 = AC, \quad P_3 = BC, \quad P_4 = A'B'C'$$
This precisely satisfies the strict 4-product-term hardware constraint of the target PLA.

3. PLA Programming Table

The PLA programming table specifies the connections in the AND array (inputs) and the OR array (outputs). Since we implemented F'_1 , the F_1 output must be configured with a true/complement XOR inverter set to Complement (C). F_2 is configured to True (T).

Product	Inputs			Outputs	
Term	A	B	C	F ₁	F ₂
1. AB	1	1	—	1	1
2. AC	1	—	1	1	1
3. BC	—	1	1	1	—
4. $A'B'C'$	0	0	0	—	1
Output XOR configuration (T/C):				C	T

411



Q9. Derive the PLA program table for a combinational circuit that squares a 3-bit binary number. Minimize the number of product terms. **(25 marks)** [CSE 205 | L-2/T-1 | 2013–14]

Answer

1. Truth Table for 3-bit Squaring Circuit

Let the 3-bit input be A (MSB), B , and C (LSB). The maximum input value is 7 (111_2), and its square is 49 (110001_2). Therefore, the circuit requires 6 output bits. Let the outputs be O_5 (MSB) to O_0 (LSB).

Inputs			Decimal Input	Decimal Square	Outputs (Square)					
A	B	C	Input	Square	O_5	O_4	O_3	O_2	O_1	O_0
0	0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	0	0	0	1
0	1	0	2	4	0	0	0	1	0	0
0	1	1	3	9	0	0	1	0	0	1
1	0	0	4	16	0	1	0	0	0	0
1	0	1	5	25	0	1	1	0	0	1
1	1	0	6	36	1	0	0	1	0	0
1	1	1	7	49	1	1	0	0	0	1

2. Boolean Expressions Derivation

From the truth table, we extract the canonical Sum of Products (SOP) for each output and simplify them:

$$\begin{aligned} O_0 &= \Sigma(1, 3, 5, 7) = C \\ O_1 &= 0 \quad (\text{Always zero}) \\ O_2 &= \Sigma(2, 6) = A'BC' + ABC' = BC' \\ O_3 &= \Sigma(3, 5) = A'BC + AB'C \\ O_4 &= \Sigma(4, 5, 7) = AB'C' + AB'C + ABC = AB' + AC \\ O_5 &= \Sigma(6, 7) = ABC' + ABC = AB \end{aligned}$$

3. PLA Optimization (Minimizing Product Terms)

A standard independent implementation of these functions would require **7 distinct product terms**: C , BC' , $A'BC$, $AB'C$, AB' , AC , and AB .

However, Programmable Logic Arrays (PLAs) feature an **output inverter (XOR gate)** that allows us to implement either the

True (T) or Complemented (C) form of a function. Let us analyze the complement of O_4 using De Morgan's Law:

$$\begin{aligned} O_4' &= (AB' + AC)' \\ &= (A' + B)(A' + C') \\ &= A'A' + A'C' + A'B + BC' \\ &= A'(1 + C' + B) + BC' \\ &= A' + BC' \end{aligned}$$

(De Morgan's Law)

(Distributive Law)

(Identity and Domination Laws)

Notice that the term BC' is *already required* for O_2 . By programming O_4 as a Complemented output, we replace the need for AB' and AC with A' and BC' . This allows us to share BC' and introduces only one new term (A'), reducing the total number of required product terms from 7 to **6**.

The minimized set of 6 product terms is:

$$P_1 = C, \quad P_2 = BC', \quad P_3 = A', \quad P_4 = A'BC, \quad P_5 = AB'C, \quad P_6 = AB$$

4. PLA Programming Table

The table below shows the structural programming of the AND array (Inputs) and the OR array (Outputs).

Product Term	Inputs			Outputs					
	A	B	C	O ₅	O ₄	O ₃	O ₂	O ₁	O ₀
1. C	–	–	1	–	–	–	–	–	1
2. BC'	–	1	0	–	1	–	1	–	–
3. A'	0	–	–	–	1	–	–	–	–
4. $A'BC$	0	1	1	–	–	1	–	–	–
5. $AB'C$	1	0	1	–	–	1	–	–	–
6. AB	1	1	–	1	–	–	–	–	–
Output Form (T/C):				T	C	T	T	T	T

Note: The output O_1 is not connected to any product term in the OR array and is configured as True (T), thus inherently yielding a logic 0.

Q10. Implement the following three Boolean functions with a PLA:

$$F_1 = \Sigma(0, 1, 2, 4)$$
$$F_2 = \Sigma(0, 5, 6, 7)$$
$$F_3 = \Sigma(0, 3, 5, 7)$$

(15 marks)

[CSE 205 | L-2/T-1 | 2011–12]

Answer

1. Logic Minimization Strategy

To efficiently implement the three Boolean functions using a Programmable Logic Array (PLA), we must minimize the total number of distinct product terms across all functions. PLAs allow each output to be programmed in either its **True (T)** or **Complemented (C)** form. By analyzing both forms for F_1 , F_2 , and F_3 , we can select the combination that shares the most product terms. The provided functions are:

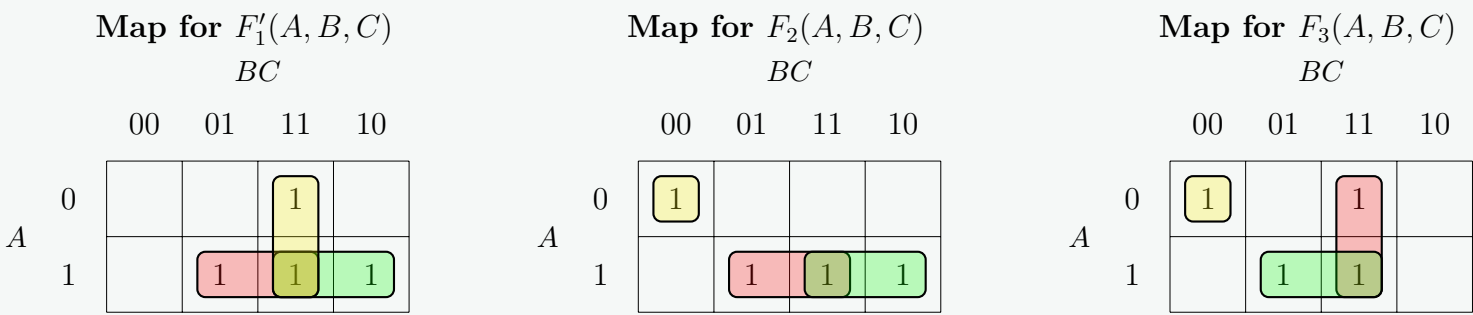
$$\begin{aligned} F_1(A, B, C) &= \Sigma(0, 1, 2, 4) \\ F_2(A, B, C) &= \Sigma(0, 5, 6, 7) \\ F_3(A, B, C) &= \Sigma(0, 3, 5, 7) \end{aligned}$$

Let us determine the complemented functions by selecting the missing minterms:

$$\begin{aligned} F_1'(A, B, C) &= \Sigma(3, 5, 6, 7) \\ F_2'(A, B, C) &= \Sigma(1, 2, 3, 4) \\ F_3'(A, B, C) &= \Sigma(1, 2, 4, 6) \end{aligned}$$

2. Karnaugh Map Simplification

We simplify the candidate functions to find optimal shared terms. It turns out that F_1' , F_2 , and F_3 share several prime implicants, leading to a highly compact PLA layout.



Extracting the minimal sum-of-products expressions from the maps:

$$F_1' = AB + AC + BC$$

$$F_2 = AB + AC + A'B'C'$$

$$F_3 = AC + BC + A'B'C'$$

3. Identification of Shared Product Terms

By choosing to implement the complement of F_1 (F_1'), and the true forms of F_2 and F_3 , we can heavily reuse product terms. The union of required product terms across all three outputs is exactly **four**:

$$P_1 = AB, \quad P_2 = AC, \quad P_3 = BC, \quad P_4 = A'B'C'$$

4. PLA Programming Table

The table below illustrates the internal connections of the PLA. A ‘1’ or ‘0’ in the Inputs section represents a connection to the true or complemented input variable, respectively, while a ‘–’ denotes no connection. A ‘1’ in the Outputs section indicates that the product term is connected to the OR gate of that specific function.

Product	Inputs (AND Array)			Outputs (OR Array)		
Term	A	B	C	F ₁	F ₂	F ₃
1. AB	1	1	–	1	1	–
2. AC	1	–	1	1	1	1
3. BC	–	1	1	1	–	1
4. $A'B'C'$	0	0	0	–	1	1
Output XOR Configuration (T/C):				C	T	T

5. PLA Circuit Diagram

